Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

Java Programming

Complete notes for the Learn Being Forward Java syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

22	3	85	450
modules	levels	minimum days	Java programs

How to use these notes

These notes follow the **Learn Being Forward Java Programming syllabus** exactly: the same 22 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 9	21 days	Write procedural programs with variables, loops, arrays, strings and methods
2. Intermediate	10 to 15	29 days	Design object-oriented programs, handle errors, use collections and files
3. Advanced	16 to 22	35 days	Concurrency, functional style, modern Java, JVM internals, databases and real-world tooling
Total	1 to 22	85 days	About 15 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every Java example in these notes was compiled with **JDK 21**, and the self-contained programs were run to produce the output shown. A few examples need something extra and say so in their box: features newer than Java 21 (labelled with the version they need), a running MySQL database (Module 20), or libraries that Maven downloads, such as JUnit, Mockito, SLF4J and Spring (Module 22). For those, the output shown is what the program prints in that setup.

HOW TO READ THE BOXES

Dark boxes with a gold top line are **Java code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to Java	Beginner	1
02	Setup and your first program	Beginner	1
03	Variables and data types	Beginner	2
04	Operators and expressions	Beginner	2
05	Input and output basics	Beginner	1
06	Control flow	Beginner	4
07	Arrays	Beginner	4
08	Strings	Beginner	3
09	Methods and recursion	Beginner	3
10	Object-Oriented Programming (OOP)	Intermediate	10
11	Packages, access modifiers and wrapper classes	Intermediate	3
12	Exception handling	Intermediate	3
13	Generics	Intermediate	2

Module	Topic	Level	Min days
14	Collections Framework	Intermediate	8
15	File handling and I/O	Intermediate	3
16	Multithreading and concurrency	Advanced	7
17	Functional programming in Java (Java 8+)	Advanced	5
18	Modern Java features (Java 9 to 25)	Advanced	3
19	JVM internals and memory management	Advanced	3
20	JDBC (Java Database Connectivity)	Advanced	5
21	Reflection, annotations, design principles and patterns	Advanced	5
22	Build tools, testing and the Java ecosystem	Advanced	7

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 9

Module 01 · Introduction to Java 8

1.1 What is Java? 8
1.2 History of Java 9
1.3 Why do we need Java? 10
1.4 Features of Java (the "buzzwords") 12
1.5 Where is Java used? 13
1.6 Java editions 14
1.7 JDK, JRE and JVM 15
1.8 How a Java program runs 16
1.9 Java compared with other languages 18

Module 02 · Setup and your first program 20

2.1 Installing the JDK 20
2.2 Development tools 21
2.3 Structure of a Java program 22
2.4 Hello World, line by line 23
2.5 Compiling and running 24
2.6 Basic syntax rules 25
2.7 Comments 27
2.8 Errors you will meet early 27

Module 03 · Variables and data types 30

3.1 What is a variable? 30
3.2 Creating variables 31
3.3 Rules and conventions for variable names 32
3.4 Types of variables (by where they are declared) 32
3.5 Scope and lifetime of variables 34
3.6 Data types overview 36
3.7 Primitive data types (8 types) 37
3.8 Reference data types 40
3.9 Literals 41
3.10 Type casting (type conversion) 42
3.11 Constants 45
3.12 Local variable type inference 45

Module 04 · Operators and expressions 47

4.1 Basics 47
4.2 Arithmetic operators 48
4.3 Unary operators 50
4.4 Assignment operators 52
4.5 Relational (comparison) operators 53
4.6 Logical operators 54
4.7 Bitwise operators 55
4.8 Shift operators 57
4.9 Ternary operator 58
4.10 instanceof operator 59
4.11 Operator precedence and associativity 60
4.12 The Math class 61

Module 05 · Input and output basics 64

5.1 Output 64
5.2 Input with Scanner 67
5.3 Other ways to read input 70
5.4 Parsing strings to numbers 71

Module 06 · Control flow 73

6.1 Decision-making statements 73
6.2 switch 76
6.3 Loops 79
6.4 Nested loops 83
6.5 Jump statements 88
6.6 Classic loop problems 91

Module 07 · Arrays 96

7.1 Introduction to arrays 96
7.2 One-dimensional arrays 97
7.3 Multi-dimensional arrays 99
7.4 Arrays and methods 101
7.5 Copying arrays 102
7.6 The Arrays utility class 103
7.7 Array algorithms 104
7.8 Command-line arguments array (String[] args) 109

Module 08 · Strings 111

8.1 Introduction to String 111
8.2 Immutability 112
8.3 Comparing strings 113
8.4 String methods 114
8.5 StringBuilder and StringBuffer 118
8.6 Characters and the Character class 120
8.7 Advanced string features 121
8.8 Classic string problems 124

Module 09 · Methods and recursion 128

9.1 Introduction to methods 128
9.2 Defining and calling methods 130
9.3 Parameters and arguments 131
9.4 Return values 133
9.5 Types of methods 136
9.6 Method overloading 137
9.7 Recursion 139

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 10 TO 15

Module 10 · Object-Oriented Programming (OOP) 152

10.1 Introduction to OOP 152
10.2 Classes and objects 153
10.3 Constructors 156
10.4 Keywords inside classes 160
10.5 Initialization blocks and order 164
10.6 Encapsulation 165
10.7 Inheritance 168
10.8 Polymorphism 172
10.9 Abstraction 178
10.10 Object class methods 182
10.11 Relationships between classes 185
10.12 Inner (nested) classes 187
10.13 Enums 188
10.14 Records (Java 16+) 190
10.15 Sealed classes (Java 17+) 191

Module 11 · Packages, access modifiers and wrapper classes 194

11.1 Packages 194
11.2 Access modifiers 197
11.3 Non-access modifiers summary 199
11.4 Wrapper classes 200
11.5 Useful java.lang and java.util classes 204
11.6 Date and Time API (java.time) 207

Module 12 · Exception handling 212
12.1 Introduction 212
12.2 Exception hierarchy 213
12.3 Handling exceptions 215
12.4 Throwing exceptions 221
12.5 Custom exceptions 223
12.6 Exception methods 224
12.7 Best practices 225
12.8 Assertions 227

Module 13 · Generics 230
13.1 Why generics 230
13.2 Generic classes 231
13.3 Generic methods 233
13.4 Generic interfaces 234
13.5 Bounded type parameters 235
13.6 Wildcards 236
13.7 Type erasure 238
13.8 Generics and inheritance 239

Module 14 · Collections Framework 242
14.1 Introduction 242
14.2 Iterating over collections 243
14.3 List interface (ordered, allows duplicates) 246
14.4 Set interface (no duplicates) 250
14.5 Queue and Deque 253
14.6 Map interface (key-value pairs) 256
14.7 Sorting and comparing objects 262
14.8 Collections utility class 264
14.9 Choosing the right collection 265
14.10 Data structures with collections 266

Module 15 · File handling and I/O 271
15.1 I/O fundamentals 271
15.2 The File class 272
15.3 Byte streams 273
15.4 Character streams 277
15.5 Serialization 280
15.6 NIO and NIO.2 (java.nio.file) 283
15.7 Working with common formats 287

LEVEL 3 · ADVANCED LEVEL · MODULES 16 TO 22

Module 16 · Multithreading and concurrency 293
16.1 Basics 293
16.2 Creating threads 294
16.3 Thread lifecycle 297
16.4 Thread methods 298
16.5 Synchronization 300
16.6 Inter-thread communication 304
16.7 Concurrency problems 305
16.8 java.util.concurrent 307
16.9 Asynchronous programming 315
16.10 Virtual threads (Java 21+) 317

Module 17 · Functional programming in Java (Java 8+) 321
17.1 Functional programming concepts 321
17.2 Lambda expressions 322
17.3 Functional interfaces 324
17.4 Method references 328
17.5 Stream API 329

17.6 Optional 338

Module 18 · Modern Java features (Java 9 to 25) 342

18.1 Java 9 342
18.2 Java 10 and 11 346
18.3 Java 12 to 17 348
18.4 Java 18 to 21 351
18.5 Java 22 to 25 354
18.6 Staying current 357

Module 19 · JVM internals and memory management 361

19.1 JVM architecture 361
19.2 Class loading 362
19.3 Runtime memory areas 364
19.4 Execution engine 366
19.5 Garbage collection 368
19.6 Memory problems 372
19.7 JVM tuning and monitoring 373
19.8 Bytecode 375

Module 20 · JDBC (Java Database Connectivity) 379

20.1 Introduction 379
20.2 Setup 380
20.3 Steps to connect 381
20.4 Statement types 382
20.5 Executing SQL 384
20.6 ResultSet 386
20.7 Transactions 388
20.8 Batch processing 391
20.9 Connection pooling 391
20.10 Project structure 392
20.11 Beyond JDBC (overview) 394

Module 21 · Reflection, annotations, design principles and patterns 398

21.1 Reflection API 398
21.2 Annotations 403
21.3 Object-oriented design principles 406
21.4 Design patterns 410
21.5 Clean code practices 425

Module 22 · Build tools, testing and the Java ecosystem 429

22.1 Build tools 429
22.2 Unit testing 433
22.3 Logging 438
22.4 Debugging 440
22.5 Version control 442
22.6 Networking basics 443
22.7 Next steps: the Java ecosystem 445
22.8 Capstone projects 448
22.9 Interview preparation 449

LEVEL 1

Beginner level

Outcome: write procedural programs with variables, loops, arrays, strings and methods.

9 MODULES · 21 DAYS MINIMUM

Module 01 · Introduction to Java	**1 day**
Module 02 · Setup and your first program	**1 day**
Module 03 · Variables and data types	**2 days**
Module 04 · Operators and expressions	**2 days**
Module 05 · Input and output basics	**1 day**
Module 06 · Control flow	**4 days**
Module 07 · Arrays	**4 days**
Module 08 · Strings	**3 days**
Module 09 · Methods and recursion	**3 days**

MODULE 01 · BEGINNER

Introduction to Java

Minimum duration **1 day** | Level **Beginner**

This module answers what Java is, why it exists and where it is used, before any code is written.

IN THIS MODULE

1.1 What is Java?
1.2 History of Java
1.3 Why do we need Java?
1.4 Features of Java (the "buzzwords")
1.5 Where is Java used?
1.6 Java editions
1.7 JDK, JRE and JVM
1.8 How a Java program runs
1.9 Java compared with other languages

1.1 What is Java?

Before we write a single line of code, we need a clear picture of what Java is. Java is two things at the same time: a **programming language** (the words and grammar we write) and a **platform** (the software that runs what we write). Understanding both halves explains almost everything about how Java behaves.

1.1.1 Definition: a high-level, class-based, object-oriented, general-purpose programming language

Every word in this definition tells us something important. Let us take them one at a time.

Word	Meaning	What it means for you
High-level	Written in English-like words, far away from the 0s and 1s the CPU understands	You write `total = price * quantity;`, not machine instructions
Class-based	All code lives inside **classes**	Even "Hello World" needs `class HelloWorld { … }`
Object-oriented	Programs are built from **objects** that hold data and behaviour together	A `Student` object has a name, marks and methods such as `getGrade()`
General-purpose	Not limited to one kind of job	Banking servers, Android apps, games and big-data tools are all written in Java

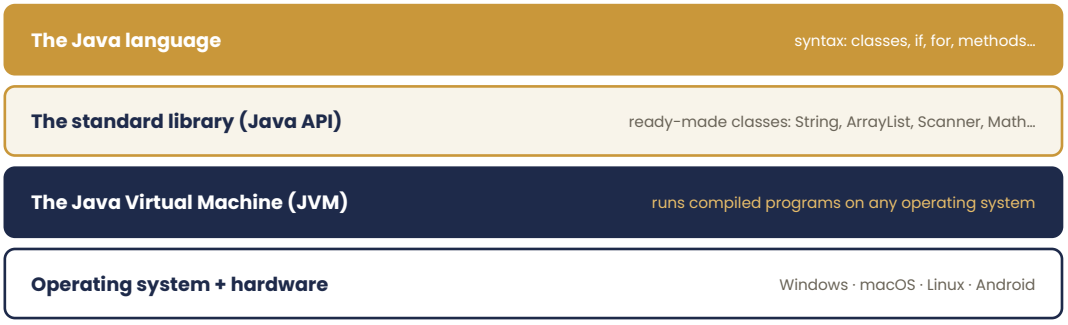


FROM LOW-LEVEL TO HIGH-LEVEL LANGUAGES

In one line: Java is an easy-to-read, object-oriented language whose programs are organised into classes and can be used to build almost any kind of software.

1.1.2 Java as a language vs Java as a platform (language + JVM + standard libraries)

When people say "Java", they may mean the **language** (the syntax and rules you learn in this course) or the **platform** (everything needed to actually run Java programs). The platform has three parts:



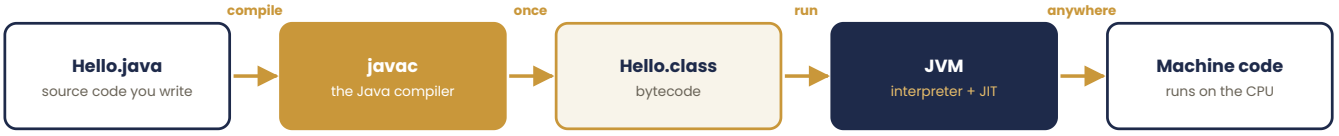
THE JAVA PLATFORM: WHAT SITS UNDER YOUR CODE

- **Language**: the rules for writing code, for example every statement ends with a semicolon.
- **Standard library**: thousands of tested classes you can use immediately, so you never write your own sorting or date handling from scratch.
- **JVM**: a program that runs your compiled code. Because a JVM exists for every major operating system, the same Java program runs everywhere.

> **NOTE**
> Other languages such as Kotlin, Scala and Groovy also compile to JVM bytecode, so they run on the same Java platform and can use the same standard library.

1.1.3 Compiled and interpreted: source code (.java) → bytecode (.class) → executed by the JVM

Some languages are **compiled** (C translates the whole program into machine code before it runs) and some are **interpreted** (Python reads and runs code line by line). Java does **both**, in two separate steps:



JAVA COMPILES ONCE TO BYTECODE, THEN THE JVM RUNS IT

1. **Compile:** the `javac` compiler checks your code for errors and translates it into **bytecode**, a compact set of instructions stored in a `.class` file. Bytecode is not tied to any CPU.
2. **Run:** the JVM reads the bytecode. It **interprets** it at first and, for code that runs often, the **JIT (Just-In-Time) compiler** converts it into fast native machine code.

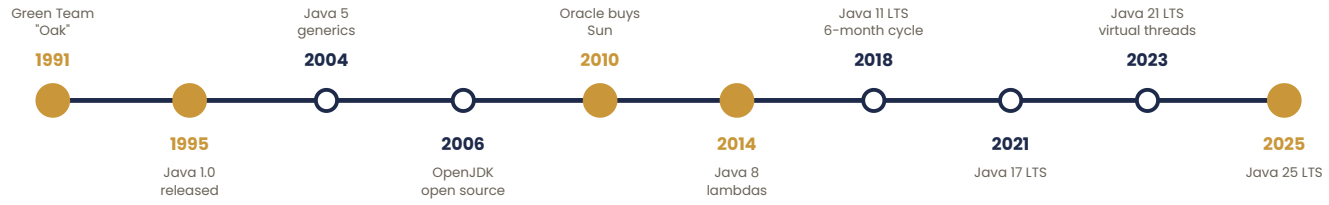
Why compile first?
Errors like a missing semicolon or a wrong type are caught before the program ever runs.

Why bytecode?
One .class file runs on Windows, macOS and Linux without recompiling.

Why a JIT?
Hot code becomes native machine code, so Java runs nearly as fast as C++ for long-running programs.

1.2 History of Java

Java was not designed for websites or phones. It started as a project for consumer electronics and found its real home when the internet arrived.



KEY MOMENTS IN JAVA'S HISTORY

1.2.1 James Gosling and the Green Team at Sun Microsystems (1991), the "Oak" project

In 1991 a small group of engineers at **Sun Microsystems**, led by **James Gosling** and known as the **Green Team**, started a project to build software for smart consumer devices such as TV set-top boxes. Devices used many different chips, so they needed a language whose programs could run on **any hardware**. C++ was too complex and too tied to the machine, so they created a new language. Gosling named it **Oak**, after an oak tree outside his office.

1.2.2 Renamed Java and released publicly in 1995

"Oak" was already a registered trademark, so the team picked a new name. **Java** comes from Java coffee (from the Indonesian island of Java), which is why the logo is a coffee cup. Java 1.0 was released publicly in **1995** with the slogan **"Write Once, Run Anywhere"**. Web browsers could run small Java programs called **applets**, and Java became famous almost overnight.

1.2.3 Oracle acquires Sun (2010)

In 2006 Sun released most of Java as free and open source software (**OpenJDK**). In **2010** Oracle Corporation bought Sun Microsystems, and Oracle has led Java's development ever since, together with the open-source OpenJDK community (Red Hat, Amazon, Microsoft, IBM and many others contribute).

1.2.4 Release timeline: Java 1.0 → 5 (generics) → 8 (lambdas, streams) → 11, 17, 21, 25 (LTS releases)

Version	Year	Headline features
Java 1.0	1996	The first public release: applets, AWT
Java 1.2	1998	Collections Framework, Swing
Java 5	2004	**Generics**, enhanced for-each loop, autoboxing, enums, annotations
Java 7	2011	try-with-resources, String in switch, diamond `<>`
Java 8 (LTS)	2014	**Lambdas, Stream API**, Optional, new date-time API
Java 9	2017	Module system, JShell
Java 11 (LTS)	2018	`var` in lambdas, HTTP Client, single-file launch
Java 17 (LTS)	2021	Records, sealed classes, text blocks, switch expressions
Java 21 (LTS)	2023	Virtual threads, pattern matching for switch, record patterns
Java 25 (LTS)	2025	Simplified main method, flexible constructor bodies, scoped values

1.2.5 Six-month release cycle and Long-Term Support (LTS) versions

Since Java 10 (2018), a new Java version is released **every six months**, in March and September. This brings new features quickly. But companies do not want to upgrade every six months, so every few releases is marked **LTS (Long-Term Support)**: it receives security and bug fixes for many years.



LTS VERSIONS (GOLD) ARRIVE EVERY TWO YEARS; THE OTHERS ARE SUPPORTED FOR ONLY SIX MONTHS

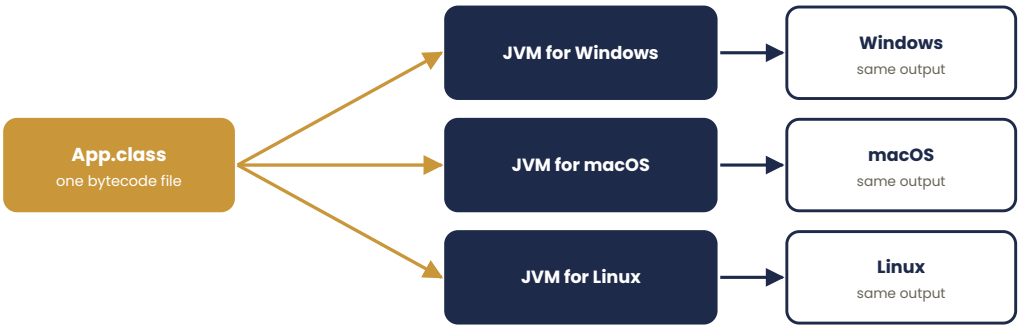
> **TIP**
> For learning and for real projects, always use the **latest LTS** version. This course uses Java 21 syntax and points out the newer Java 25 features where the syllabus mentions them.

1.3 Why do we need Java?

There are hundreds of programming languages. These eight reasons explain why Java has stayed one of the most used languages in the world for nearly 30 years.

1.3.1 Platform independence: "Write Once, Run Anywhere" (WORA)

A C program compiled on Windows produces a Windows `.exe` file that will not run on a Mac. A Java program is compiled to **bytecode**, and bytecode runs on any computer that has a JVM. You compile once and the same `.class` file runs on every operating system.



WRITE ONCE, RUN ANYWHERE: ONE BYTECODE FILE, MANY JVMS

WORA: the Java **program** is platform-independent; the **JVM** is platform-specific. Each operating system has its own JVM, and all JVMs understand the same bytecode.

1.3.2 Object-oriented design for large, maintainable programs

Real software has millions of lines of code written by hundreds of people. Java organises code into **classes and objects** that model real things (Student, Account, Order). Each class hides its details and exposes a clean set of methods, so teams can work on different parts without breaking each other's code. Module 10 covers this in depth.

1.3.3 Automatic memory management (garbage collection)

In C and C++ the programmer must free every piece of memory by hand. Forgetting causes **memory leaks**; freeing twice causes crashes. In Java you create objects with `new` and never free them: the **Garbage Collector (GC)** automatically finds objects that are no longer used and reclaims their memory.

1.3.4 Strong typing and compile-time error checking

Every variable in Java has a fixed **type** (int, String, double…). The compiler checks that types are used correctly **before** the program runs, so a whole class of bugs never reaches your users.

THE COMPILER STOPS THIS MISTAKE

java

```
int age = 21;
age = "twenty one";   // compile-time error: incompatible types: String cannot be converted to int
```

1.3.5 Security model (bytecode verification, no direct pointers)

- **No pointers:** Java code cannot read or write arbitrary memory addresses, which blocks a huge class of attacks and crashes.
- **Bytecode verifier:** before running a class, the JVM checks that its bytecode is valid and safe (no illegal jumps, no stack overflows, no type violations).
- **Array bounds checking:** reading past the end of an array throws an exception instead of silently reading other data.

1.3.6 Huge standard library and ecosystem

The JDK ships with thousands of classes for collections, files, networking, dates, databases, security and concurrency. Beyond that, the Java ecosystem has hundreds of thousands of open-source libraries (Spring, Hibernate, Jackson, JUnit, Apache Commons) available through build tools like Maven and Gradle.

1.3.7 Built-in multithreading support

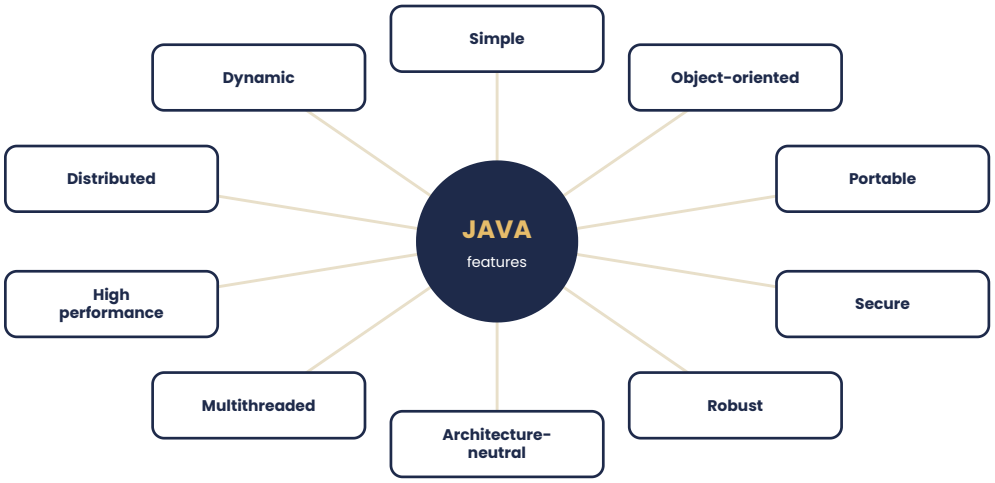
A **thread** is a separate path of execution. Java had threads built into the language from day one, so one program can download a file, update the screen and save data at the same time. Since Java 21, **virtual threads** make it possible to run millions of lightweight threads (Module 16).

1.3.8 Backward compatibility and long-term stability

Code written for Java 1.2 in 1998 usually still compiles and runs on Java 25 today. Companies trust Java for systems that must run for decades, such as banking and government software, because upgrading Java rarely breaks old code.

1.4 Features of Java (the "buzzwords")

When Sun released Java, it described the language with a list of features that are now known as the **Java buzzwords**. Interviewers love asking about them.



THE TEN JAVA BUZZWORDS

1.4.1 Simple

Java syntax is based on C and C++, so it looks familiar, but the confusing parts were removed: no pointers, no operator overloading, no multiple inheritance of classes, no manual memory management and no header files.

1.4.2 Object-oriented

Everything in Java is part of a class, and programs are built from objects. Java supports the four pillars of OOP: **encapsulation, inheritance, polymorphism and abstraction**.

1.4.3 Platform-independent and portable

Bytecode runs on any JVM (platform-independent). Java also fixes the size of every data type: an `int` is always 32 bits on every machine, unlike C where it can vary. So a program gives the **same results** everywhere (portable).

1.4.4 Secure

No pointers, bytecode verification, a security manager model in older versions, and strong cryptography libraries make Java a common choice for banking and payment systems.

1.4.5 Robust

Robust means strong and reliable. Java catches errors early with strong type checking, handles runtime problems with **exception handling**, and avoids memory corruption with garbage collection and array bounds checks.

1.4.6 Architecture-neutral

The compiler produces bytecode that does not depend on the processor architecture (Intel x86, ARM, Apple Silicon). Primitive sizes and behaviour are defined by the Java specification, not by the hardware.

1.4.7 Multithreaded

Java can run many tasks concurrently inside one program, with built-in keywords such as `synchronized` and a rich `java.util.concurrent` library.

1.4.8 High performance (JIT compilation)

Pure interpretation would be slow. The JVM's **JIT compiler** watches which methods run often ("hot spots") and compiles them into optimised native machine code while the program runs. Long-running Java applications therefore reach speeds close to C++.

1.4.9 Distributed

Java was designed for networks. It has built-in support for sockets, HTTP, URLs and remote method invocation (RMI), so programs on different computers can work together easily.

1.4.10 Dynamic

Classes are loaded into the JVM **only when they are needed**, at runtime. Programs can inspect classes and call methods discovered at runtime (reflection), which is what frameworks like Spring and JUnit rely on.

Buzzword	Remember it as
Simple	C-like syntax without the hard parts
Object-oriented	everything is a class / object
Portable	same results on every machine
Secure	no pointers + bytecode verifier
Robust	strong typing + exceptions + GC
Architecture-neutral	bytecode does not care about the CPU
Multithreaded	many tasks at once
High performance	JIT compiler
Distributed	built for networks
Dynamic	classes loaded at runtime

1.5 Where is Java used?

Java runs on billions of devices. Here are the main areas where you will meet it in real jobs.

1.5.1 Enterprise and backend applications (banking, insurance, e-commerce)

Most large banks, insurance companies, airlines and online stores run their core systems on Java: account processing, payments, order management. These systems must be reliable, secure and maintainable for years, which is exactly where Java shines.

1.5.2 Web applications and microservices (Spring Boot)

The **Spring Boot** framework is the most popular way to build REST APIs and microservices. Companies such as Netflix, Amazon and LinkedIn use Java services behind their websites.

1.5.3 Android app development

Android apps were originally written in Java, and the Android SDK is a Java API. Kotlin is now preferred for new apps, but it runs on the same platform and millions of apps still contain Java code.

1.5.4 Big data tools (Hadoop, Kafka, Spark, Elasticsearch)

Many big-data tools are written in Java or run on the JVM: Hadoop (distributed storage and processing), Kafka (event streaming), Spark (fast data processing, written in Scala on the JVM) and Elasticsearch (search engine).

1.5.5 Desktop applications (Swing, JavaFX, IntelliJ IDEA itself)

Swing and JavaFX are libraries for windowed desktop programs. Popular tools such as **IntelliJ IDEA**, Eclipse, NetBeans and Android Studio are themselves Java applications.

1.5.6 Cloud-native and distributed systems

Java services run in containers (Docker) and orchestration systems (Kubernetes) on AWS, Azure and Google Cloud. Frameworks like Quarkus and Micronaut make Java start fast and use little memory in the cloud.

1.5.7 Embedded systems, IoT and smart cards

Java Card runs on SIM cards and bank chip cards. Java ME and embedded JVMs run on set-top boxes, printers, car systems and IoT devices: the original goal of the Green Team.

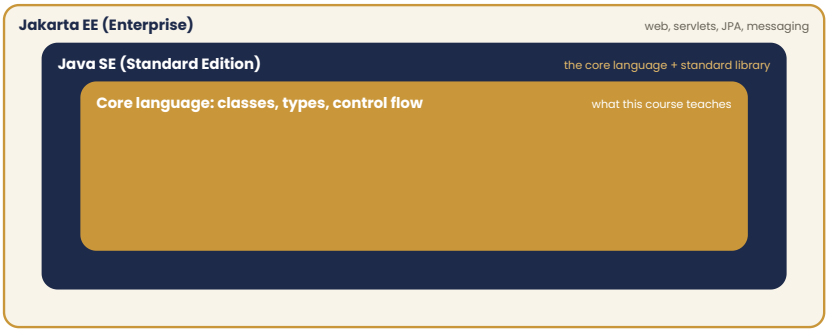
1.5.8 Scientific computing and games (Minecraft Java Edition)

Java is used for simulations, bioinformatics and research tools. The original **Minecraft** (Java Edition) is written in Java and is one of the best-selling games ever.

Banking & finance core banking, trading, payments	**Web & APIs** Spring Boot microservices	**Mobile** Android apps
Big data Hadoop, Kafka, Elasticsearch	**Desktop & tools** IntelliJ, Eclipse, JavaFX apps	**Cloud** containers on AWS / Azure / GCP
Embedded & IoT SIM cards, set-top boxes	**Games & science** Minecraft, simulations	**Enterprise** insurance, airlines, retail

1.6 Java editions

Java comes in different editions for different kinds of devices and applications. They share the same core language.



JAKARTA EE BUILDS ON TOP OF JAVA SE

1.6.1 Java SE (Standard Edition): the core language and libraries

Java SE is the foundation: the language, the JVM and the core libraries (`java.lang`, `java.util`, `java.io`, `java.time`…). When you install a JDK you get Java SE. **This entire course is Java SE.**

1.6.2 Jakarta EE (formerly Java EE): enterprise web and server APIs

Jakarta EE adds APIs for large server applications: Servlets and JSP for web pages, JPA for databases, JMS for messaging, and more. It was called Java EE (J2EE) until Oracle handed it to the Eclipse Foundation in 2017, when it was renamed Jakarta EE.

1.6.3 Java ME (Micro Edition): small and embedded devices

Java ME is a smaller version of Java for devices with very little memory: old feature phones, sensors and embedded controllers.

1.6.4 JavaFX: rich client applications

JavaFX is a modern library for desktop user interfaces, with CSS styling, charts, animation and media. It is now developed separately as OpenJFX.

Edition	For	Example
Java SE	desktop, console and core server code	everything in this course
Jakarta EE	large web and enterprise servers	online banking backend
Java ME	tiny devices	smart meters, old phones
JavaFX	rich desktop UIs	a billing desktop app

1.7 JDK, JRE and JVM

These three abbreviations confuse every beginner. Think of them as boxes inside boxes.



JDK ⊃ JRE ⊃ JVM

1.7.1 JVM (Java Virtual Machine): runs bytecode

The JVM is an abstract computer implemented in software. It loads `.class` files, verifies them, and executes their bytecode by interpreting it and JIT-compiling hot code. It also manages memory (heap, stack) and garbage collection. There is a different JVM build for each operating system.

1.7.2 JRE (Java Runtime Environment): JVM + core libraries

To run a Java program you need the JVM **plus** the standard library classes your program uses (String, ArrayList…). Together these make the **JRE**. Since Java 11 a separate JRE is no longer distributed; the JDK contains everything, and tools such as `jlink` can build a small custom runtime.

1.7.3 JDK (Java Development Kit): JRE + compiler (javac) + tools

Developers need to **create** programs, not just run them. The JDK adds development tools on top of the JRE:

Tool	Purpose
`javac`	compiles .java source into .class bytecode
`java`	starts the JVM and runs a program
`jshell`	interactive Java console (REPL)
`javadoc`	generates HTML documentation from comments
`jar`	packages classes into a .jar archive

Tool	Purpose
`jdb`, `jconsole`, `jcmd`	debugging and monitoring tools

1.7.4 How they relate (JDK ⊃ JRE ⊃ JVM)

	JVM	JRE	JDK
Contains	execution engine, memory manager, GC	JVM + standard libraries	JRE + compiler + tools
Used by	(inside the JRE)	users running programs	developers
Can compile code?	no	no	**yes**
Can run code?	yes	yes	yes
Platform-specific?	yes	yes	yes

Remember: JDK = JRE + development tools; JRE = JVM + libraries; JVM = runs bytecode. To **learn and write** Java you install the **JDK**.

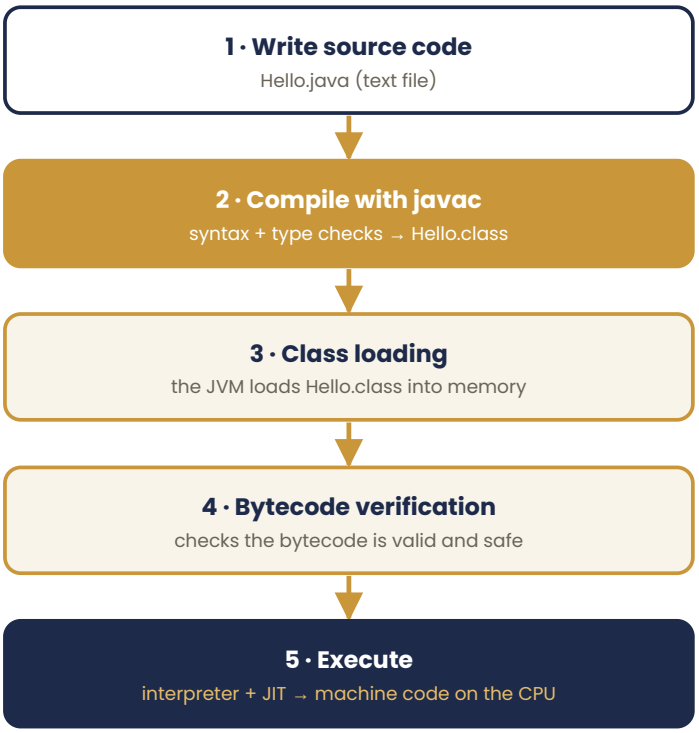
1.7.5 JDK distributions: Oracle JDK, OpenJDK, Eclipse Temurin, Amazon Corretto

Java is open source (OpenJDK), so several companies build and distribute their own JDKs from the same source code. They all behave the same for learning.

Distribution	By	Notes
OpenJDK builds	Oracle / community	reference builds from jdk.java.net
Oracle JDK	Oracle	free for development; check licence for production
Eclipse Temurin	Adoptium (Eclipse Foundation)	popular free choice, adoptium.net
Amazon Corretto	Amazon	free, long-term support, used on AWS
Microsoft Build of OpenJDK, Azul Zulu, Red Hat	others	all compatible

1.8 How a Java program runs

Here is the complete journey of a Java program, from your keyboard to the CPU.



THE FIVE STAGES OF RUNNING A JAVA PROGRAM

1.8.1 Writing source code

You write Java code in a plain text file whose name ends in `.java`, using any editor or IDE. The file contains one or more classes.

HELLO.JAVA — Hello.java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Java is running!");
    }
}
```

OUTPUT

```
Java is running!
```

1.8.2 Compilation with javac

`javac Hello.java` checks the code and produces `Hello.class`. If there are errors, no .class file is created and javac prints the file name, line number and reason.

Terminal

```
$ javac Hello.java        # compile: creates Hello.class
$ java Hello              # run: starts the JVM with class Hello
Java is running!
```

1.8.3 Class loading

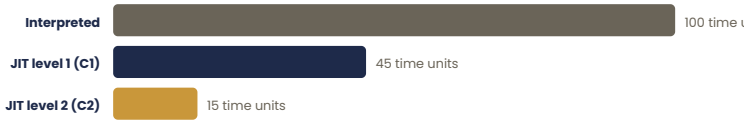
When you type `java Hello`, the JVM starts and its **class loader** finds `Hello.class` and loads it into memory. Other classes (String, System…) are loaded the first time they are needed. Module 19 explains the class loaders in detail.

1.8.4 Bytecode verification

Before any bytecode runs, the **bytecode verifier** checks it: correct format, valid jumps, correct types on the operand stack, no access to private members of other classes. A tampered or corrupted .class file is rejected.

1.8.5 Interpretation and JIT compilation to machine code

The JVM starts by **interpreting** bytecode instruction by instruction, which starts quickly. It counts how often each method runs; frequently used ("hot") methods are compiled by the **JIT compiler** into optimised native machine code, which then runs directly on the CPU.



THE SAME HOT METHOD GETS FASTER AFTER JIT COMPILATION (ILLUSTRATIVE)

1.9 Java compared with other languages

1.9.1 Java vs C and C++ (no pointers, garbage collection, no multiple class inheritance)

Topic	C / C++	Java
Compilation	to native machine code for one platform	to bytecode; runs on any JVM
Pointers	yes, direct memory access	**no** (only safe references)
Memory	manual: malloc/free, new/delete	**automatic** garbage collection
Multiple inheritance	C++ allows multiple base classes	not for classes; use **interfaces**
Operator overloading	C++ yes	no
Array bounds	not checked	checked (exception)
Typical use	OS kernels, drivers, games engines	enterprise, web, Android, big data

1.9.2 Java vs Python (static vs dynamic typing, speed, verbosity)

Topic	Python	Java
Typing	dynamic: types checked at runtime	**static**: types checked at compile time
Speed	slower (interpreted)	faster (JIT-compiled)
Verbosity	short, concise	more code, more explicit
Blocks	indentation	curly braces `{ }`
Strengths	data science, AI, scripting	large systems, backend, Android

SAME PROGRAM, TWO LANGUAGES java

```java
// Java: type declared, braces, semicolons
int total = 0;
for (int i = 1; i <= 5; i++) {
    total += i;
}
System.out.println(total);   // 15

# Python: no types, indentation
# total = 0
# for i in range(1, 6):
#     total += i
# print(total)
```

1.9.3 Java vs JavaScript (different languages despite the name)

JavaScript was named after Java in 1995 for marketing reasons, but the two languages are **completely different**, as different as "car" and "carpet".

Topic	JavaScript	Java
Runs in	web browsers and Node.js	the JVM
Typing	dynamic	static
Compiled?	interpreted / JIT in the browser	compiled to bytecode first
Main use	web pages, front-end, Node backends	backend, Android, enterprise

PRACTICE CHECKPOINT

Explain **WORA** and the **JDK / JRE / JVM** difference in your own words. A good answer mentions: bytecode, a JVM per operating system, JDK = JRE + tools, JRE = JVM + libraries.

MODULE 02 · BEGINNER

Setup and your first program

Minimum duration **1 day** | Level **Beginner**

By the end of this module the learner has a working JDK and IDE and understands every word of Hello World.

IN THIS MODULE

2.1 Installing the JDK
2.2 Development tools
2.3 Structure of a Java program
2.4 Hello World, line by line
2.5 Compiling and running
2.6 Basic syntax rules
2.7 Comments
2.8 Errors you will meet early

2.1 Installing the JDK

To write and run Java programs you need the **JDK** (Java Development Kit) from Module 1. This section installs it and makes the `java` and `javac` commands available everywhere on your computer.

2.1.1 Choosing a version (current LTS)

Always pick the newest **LTS (Long-Term Support)** release. At the time of writing that is **Java 25** (September 2025); Java 21 is the previous LTS and is still widely used. Any JDK 21 or newer is perfect for this course. Download a free build from `adoptium.net` (Eclipse Temurin) or `oracle.com/java`.

If you…	Choose
are learning (this course)	the latest LTS: JDK 25 (or JDK 21)
join a company project	whatever version the project uses (often 17 or 21)
want to try brand-new features	the latest non-LTS release

2.1.2 Installation on Windows, macOS and Linux

OS	Steps
Windows	Download the **.msi installer** (x64) → run it → Next → tick **"Set JAVA_HOME variable"** and **"Add to PATH"** (Temurin offers both) → Install.
macOS	Download the **.pkg** for your chip (**aarch64** for Apple M1/M2/M3/M4, **x64** for Intel) → double-click → Install. Or with Homebrew: `brew install --cask temurin`.
Linux	Ubuntu/Debian: `sudo apt install openjdk-21-jdk`. Fedora: `sudo dnf install java-21-openjdk-devel`. Or download the .tar.gz and extract it.



FIVE STEPS TO A WORKING JDK

2.1.3 Setting JAVA_HOME and the PATH variable

PATH is the list of folders your terminal searches when you type a command. Adding the JDK's `bin` folder to PATH lets you type `javac` from any folder. **JAVA_HOME** points to the JDK folder itself; tools like Maven, Gradle and IDEs read it to find Java.

OS	JAVA_HOME example	How to set it
Windows	`C:\Program Files\Eclipse Adoptium\jdk-21`	Search "Edit the system environment variables" → Environment Variables → New: `JAVA_HOME` ; then edit **Path** → New → `%JAVA_HOME%\bin`
macOS	`$(/usr/libexec/java_home)`	Add to `~/.zshrc` : `export JAVA_HOME=$(/usr/libexec/java_home)` and `export PATH="$JAVA_HOME/bin:$PATH"`
Linux	`/usr/lib/jvm/java-21-openjdk-amd64`	Add the same two `export` lines to `~/.bashrc` , then `source ~/.bashrc`

WATCH OUT
After changing PATH or JAVA_HOME, **close and reopen** the terminal. Old terminal windows still have the old values.

2.1.4 Verifying: java -version and javac -version

Terminal

```
$ java -version
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7)
OpenJDK 64-Bit Server VM (build 21.0.10+7, mixed mode, sharing)

$ javac -version
javac 21.0.10
```

Problem	Meaning and fix
`'javac' is not recognized` / `command not found`	the JDK `bin` folder is not on PATH, or you opened the terminal before setting it
`java` works but `javac` does not	only a JRE is installed, or PATH points to a JRE: install the full **JDK**
wrong version shows	an older Java appears earlier in PATH: move the new one to the top

2.2 Development tools

You can write Java with anything from Notepad to a professional IDE. Each tool has its place.

2.2.1 Text editor + command line

Write code in any plain-text editor (Notepad++, VS Code, Sublime, nano) and compile it yourself with `javac` and `java`. This is the best way to **understand** what happens, so try it first, even if you later move to an IDE.

2.2.2 IDEs: IntelliJ IDEA, Eclipse, VS Code, NetBeans

An **IDE (Integrated Development Environment)** combines editor, compiler, runner and debugger in one window, with auto-completion, error highlighting and refactoring.

IDE	Cost	Why people use it
IntelliJ IDEA	Community edition free	smartest code completion; industry favourite
Eclipse	free	long-time standard in companies and colleges
VS Code	free	lightweight; add the "Extension Pack for Java"
NetBeans	free	simple setup, good for beginners

2.2.3 Online compilers for quick practice

Websites such as **OnlineGDB**, **JDoodle**, **Replit** and **Programiz** run Java in the browser with nothing to install. They are great for trying a small idea on a phone or a shared computer, but use a real JDK for projects.

2.2.4 JShell: the interactive REPL (Java 9+)

JShell lets you type Java statements and see results immediately, without writing a class or a main method. REPL stands for **Read-Evaluate-Print Loop**.

jshell

```
$ jshell
|  Welcome to JShell -- Version 21.0.10
jshell> int a = 7
a ==> 7
jshell> a * 6
$2 ==> 42
jshell> "Java".toUpperCase()
$3 ==> "JAVA"
jshell> /exit
```

2.3 Structure of a Java program

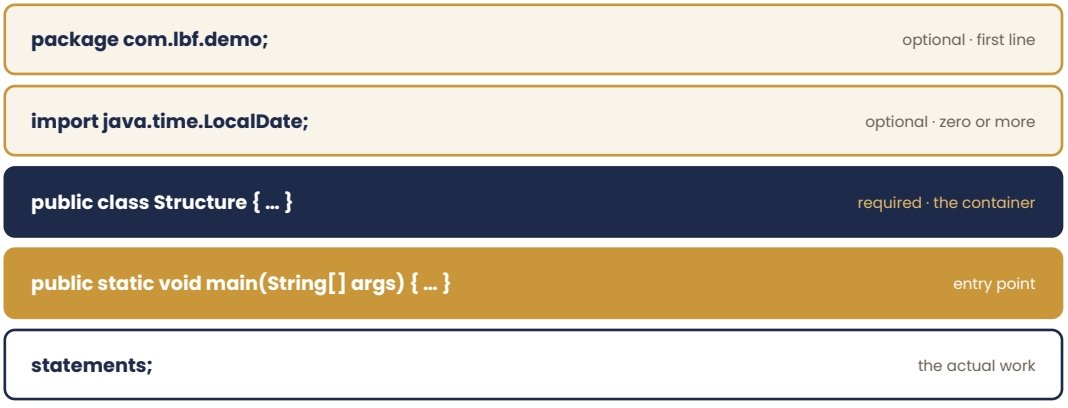
Every Java source file follows the same order. Some parts are optional, but when present they must appear in this order.

STRUCTURE.JAVA — Structure.java

```
package com.lbf.demo;            // 1. package declaration (optional)

import java.time.LocalDate;      // 2. import statements (optional)

public class Structure {         // 3. class declaration
    // 4. the main method: where the program starts
    public static void main(String[] args) {
        // 5. statements inside a block { }
        LocalDate today = LocalDate.of(2026, 10, 5);
        System.out.println("Today is " + today);
    }
}
```



THE ORDER OF PARTS IN A JAVA SOURCE FILE

2.3.1 Package declaration

A **package** groups related classes, like a folder. `package com.lbf.demo;` must be the **first** statement in the file, and the file must sit in the matching folder `com/lbf/demo/`. Small programs can skip it and live in the **default package**. Module 11 covers packages fully.

2.3.2 Import statements

`import` lets you use a class from another package by its short name. Without the import you would write the full name `java.time.LocalDate` every time. Classes in `java.lang` (String, System, Math) are imported automatically.

2.3.3 Class declaration

All Java code lives inside a class. `public class Structure` declares a class named `Structure`; its body is everything between the braces. A file may contain several classes but at most **one public** class.

2.3.4 The main method

`public static void main(String[] args)` is the **entry point**: when you run the class, the JVM looks for exactly this method and starts executing its first statement.

2.3.5 Statements and blocks { }

A **statement** is one instruction ending in a semicolon. A **block** is a group of statements between braces `{ }`. Classes, methods, if-statements and loops all use blocks. Every opening brace needs a matching closing brace.

2.4 Hello World, line by line

This is the first program everyone writes. It is only five lines, but every word has a meaning.

HELLOWORLD.JAVA

HelloWorld.java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

OUTPUT

```
Hello, World!
```



visible everywhere | no object needed | returns nothing | fixed name | command-line inputs

THE PARTS OF THE MAIN METHOD

2.4.1 public class HelloWorld

`class` declares a new class; `HelloWorld` is its name; `public` means any other code can use it. By convention class names use **PascalCase**: each word starts with a capital letter.

2.4.2 public static void main(String[] args)

This line declares the main method. Its signature must be exactly right (only the parameter name `args` may change), otherwise the JVM reports *"Main method not found in class HelloWorld"*.

2.4.2.1 Why public

The JVM is outside your class, so it must be allowed to call `main`. `public` makes the method accessible from anywhere. A `private main` would be invisible to the JVM.

2.4.2.2 Why static

When the program starts, no objects of `HelloWorld` exist yet. A `static` method belongs to the **class itself**, so the JVM can call `HelloWorld.main(...)` without creating an object first.

2.4.2.3 Why void

`void` means the method returns no value. `main` does not return anything to the JVM; to report an exit status a program calls `System.exit(code)` instead.

2.4.2.4 What String[] args holds (command-line arguments)

`args` is an array of Strings containing the words typed after the class name when the program is started. If none are given, `args.length` is 0.

GREET.JAVA

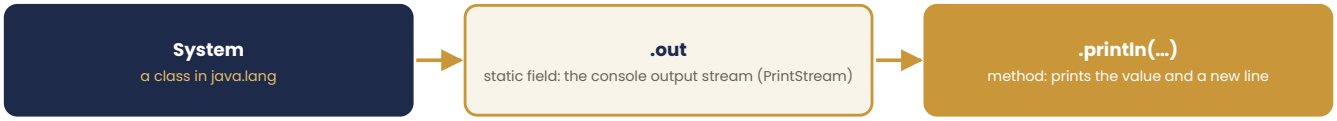
Greet.java

```java
public class Greet {
    public static void main(String[] args) {
        System.out.println("Number of arguments: " + args.length);
        for (int i = 0; i < args.length; i++) {
            System.out.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

Terminal

```
$ javac Greet.java
$ java Greet Lohit Java 2026
Number of arguments: 3
args[0] = Lohit
args[1] = Java
args[2] = 2026
```

2.4.3 System.out.println(): System class, out stream, println method



READING SYSTEM.OUT.PRINTLN FROM LEFT TO RIGHT

`System` is a built-in class. `out` is a `static` field inside it that refers to a `PrintStream` connected to the console. `println` is a method of PrintStream that prints its argument followed by a **new line**; `print` prints without the new line.

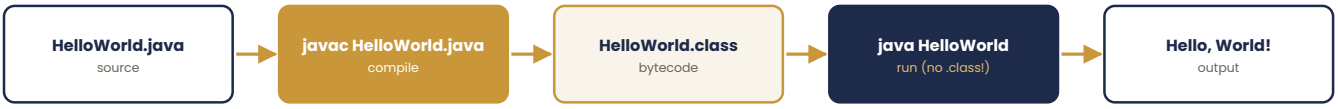
2.4.4 File name must match the public class name

A public class named `HelloWorld` must be saved in a file named exactly `HelloWorld.java`, with the same capital letters. Otherwise the compiler refuses:

Terminal

```
$ javac Hello.java
Hello.java:1: error: class HelloWorld is public, should be declared in a file named HelloWorld.java
public class HelloWorld {
       ^
1 error
```

2.5 Compiling and running



FROM SOURCE FILE TO OUTPUT

2.5.1 javac HelloWorld.java → HelloWorld.class

Open a terminal in the folder containing the file and run `javac HelloWorld.java`. If there are no errors, nothing is printed and a new file `HelloWorld.class` appears next to the source. Each class in the file produces its own .class file.

2.5.2 java HelloWorld

`java HelloWorld` starts the JVM and runs the class. Give the **class name**, not the file name: no `.class` extension.

Terminal

```
$ javac HelloWorld.java
$ ls
HelloWorld.class  HelloWorld.java
$ java HelloWorld
Hello, World!
```

2.5.3 Single-file source launch: java HelloWorld.java (Java 11+)

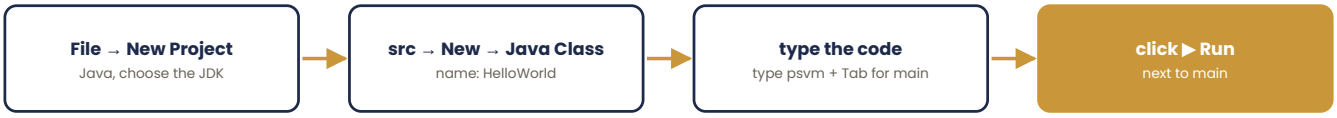
Since Java 11 you can run a single source file directly. The JVM compiles it in memory and runs it, without creating a .class file. This is perfect for small practice programs.

Terminal

```
$ java HelloWorld.java
Hello, World!
```

2.5.4 Running from the IDE

In IntelliJ IDEA or Eclipse, create a project, add a class with a main method, then click the green ▶ **Run** arrow next to `main` (or press Shift+F10 in IntelliJ, Ctrl+F11 in Eclipse). The IDE compiles and runs the class and shows the output in its Run window.



RUNNING HELLO WORLD IN INTELLIJ IDEA

2.6 Basic syntax rules

2.6.1 Case sensitivity

Java treats uppercase and lowercase letters as different. `main`, `Main` and `MAIN` are three different names; `System` works but `system` does not.

CASE-SENSITIVE NAMES

CaseDemo.java

```
public class CaseDemo {
    public static void main(String[] args) {
        int total = 10;
        int Total = 20;          // a DIFFERENT variable
        System.out.println(total + " " + Total);
    }
}
```

OUTPUT

```
10 20
```

2.6.2 Semicolons and statement termination

Every statement ends with a semicolon `;`. Declarations of classes and methods and the closing brace of a block do **not** need one. A missing semicolon is the most common beginner error: `error: ';' expected`.

2.6.3 Whitespace and indentation

Spaces, tabs and new lines are ignored by the compiler (except inside strings), so the code could be written on one line. Humans need structure, though: indent the body of every block by **4 spaces** so the nesting is visible at a glance.

WHITESPACE DOES NOT CHANGE THE MEANING

java

```java
// legal but unreadable
public class A{public static void main(String[] a){System.out.println("hi");}}

// same code, properly indented
public class A {
    public static void main(String[] a) {
        System.out.println("hi");
    }
}
```

2.6.4 Identifiers and naming rules

An **identifier** is a name you choose for a class, variable or method. The rules:

Rule	Valid	Invalid
letters, digits, `_` and `$` only	`total`, `_count`, `$price`	`my-name`, `marks%`
cannot start with a digit	`student1`	`1student`
cannot be a keyword	`klass`	`class`, `int`
no spaces	`firstName`	`first name`
case-sensitive, any length	`numberOfStudentsInClass`	—

2.6.5 Keywords (reserved words) list

Keywords have a special meaning and cannot be used as identifiers. Java has 51 reserved keywords plus some **contextual** keywords (like `var`, `record`, `yield`) that are special only in certain places.

`abstract`	`assert`	`boolean`	`break`	`byte`	`case`
`catch`	`char`	`class`	`const*`	`continue`	`default`
`do`	`double`	`else`	`enum`	`extends`	`final`
`finally`	`float`	`for`	`goto*`	`if`	`implements`
`import`	`instanceof`	`int`	`interface`	`long`	`native`
`new`	`package`	`private`	`protected`	`public`	`return`
`short`	`static`	`strictfp`	`super`	`switch`	`synchronized`
`this`	`throw`	`throws`	`transient`	`try`	`void`
`volatile`	`while`	`_` (underscore)			

NOTE

`const` and `goto` (marked *) are reserved but unused. `true`, `false` and `null` are **literals**, not keywords, but they cannot be used as names either. Contextual keywords: `var`, `yield`, `record`, `sealed`, `permits`, `non-sealed`, `module`, `requires`, `exports`, `opens`, `to`, `with`, `transitive`, `uses`, `provides`, `when`.

2.6.6 Naming conventions: PascalCase classes, camelCase methods and variables, UPPER_CASE constants

Kind of name	Convention	Examples
Classes, interfaces, enums, records	**PascalCase**	`StudentRecord`, `ArrayList`
Methods	**camelCase**, usually a verb	`calculateTotal()`, `getName()`
Variables	**camelCase**, a noun	`studentName`, `totalMarks`
Constants	**UPPER_SNAKE_CASE**	`MAX_SIZE`, `PI`

Kind of name	Convention	Examples
Packages	all lowercase, reversed domain	`com.lbf.bank`

TIP
Conventions are not enforced by the compiler, but every Java developer follows them. Code that ignores them looks wrong in interviews and code reviews.

2.7 Comments

Comments are notes for humans. The compiler ignores them completely. Good comments explain **why** the code does something; the code itself already shows **what** it does.

2.7.1 Single-line //

Everything from `//` to the end of the line is a comment.

2.7.2 Multi-line /* */

Everything between `/*` and `*/` is a comment, across any number of lines. Multi-line comments cannot be nested.

2.7.3 Documentation comments /** */ and Javadoc

A comment starting with `/**` placed just before a class or method is a **documentation comment**. The `javadoc` tool turns these into HTML documentation, the same format as the official Java API pages. Special tags such as `@param`, `@return` and `@author` describe the parts.

ALL THREE COMMENT STYLES — Interest.java

```
/**
 * Calculates simple interest.
 * @author Learn Being Forward
 */
public class Interest {

    /**
     * Returns simple interest for one period.
     * @param principal the amount borrowed
     * @param rate      yearly rate in percent
     * @param years     number of years
     * @return the interest amount
     */
    static double simple(double principal, double rate, int years) {
        return principal * rate * years / 100;   // formula: P * R * T / 100
    }

    public static void main(String[] args) {
        /* multi-line comment:
           the values below are only an example */
        System.out.println(simple(10000, 8.5, 2));
    }
}
```

OUTPUT
```
1700.0
```

Terminal
```
$ javadoc -d docs Interest.java      # creates docs/index.html
```

2.8 Errors you will meet early

Errors are normal: every programmer sees them every day. What matters is recognising the **kind** of error and reading the message.

Kind	When it appears	Who finds it	Example
Compile-time	during `javac`	the compiler	missing semicolon, wrong type
Runtime	while the program runs	the JVM (exception)	divide by zero, bad array index
Logical	program runs but the answer is wrong	you (testing)	using + instead of *

2.8.1 Compile-time (syntax) errors

The compiler refuses to create a .class file. Typical causes: missing `;` or `}`, misspelled names, wrong types, using an undeclared variable.

TWO COMPILE ERRORS java

```java
public class Oops {
    public static void main(String[] args) {
        int marks = 90
        System.out.println(Marks);
    }
}
```

Terminal

```
$ javac Oops.java
Oops.java:3: error: ';' expected
        int marks = 90
                    ^
1 error
```

NOTE
The compiler often stops after the first syntax error. Fix it, compile again, and the next error (`cannot find symbol: Marks`) appears.

2.8.2 Runtime errors

The code compiles, but something impossible happens while running. The JVM throws an **exception**, prints a **stack trace** and stops the program (unless the exception is handled, Module 12).

RUNTIME ERROR: ARITHMETICEXCEPTION Crash.java

```java
public class Crash {
    public static void main(String[] args) {
        int students = 0;
        int total = 450;
        System.out.println("Start");
        int average = total / students;     // division by zero!
        System.out.println("Average: " + average);
    }
}
```

OUTPUT

```
Start
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Crash.main(Crash.java:6)
```

2.8.3 Logical errors

The hardest kind: no error message at all, just a wrong answer. Only careful testing reveals them.

LOGICAL ERROR: OPERATOR PRECEDENCE

Average.java

```
public class Average {
    public static void main(String[] args) {
        int a = 80, b = 90, c = 70;
        double wrong = a + b + c / 3;        // only c is divided!
        double right = (a + b + c) / 3.0;
        System.out.println("Wrong: " + wrong);
        System.out.println("Right: " + right);
    }
}
```

OUTPUT

```
Wrong: 193.0
Right: 80.0
```

2.8.4 Reading error messages and line numbers

Every error message has the same parts. Learn to read them from left to right:



ANATOMY OF A COMPILER MESSAGE

For a runtime stack trace, read the **first line** (the exception type and message) and then the first `at …` line that mentions **your** class: it shows the method, file and line where it happened, e.g. `at Crash.main(Crash.java:6)`.

PRACTICE CHECKPOINT

Write, compile and run a program that prints your **name** and **course**, from both the **terminal** (javac + java) and an **IDE**.

SAMPLE SOLUTION

AboutMe.java

```
public class AboutMe {
    public static void main(String[] args) {
        System.out.println("Name   : Aarav Sharma");
        System.out.println("Course : Java Programming (Learn Being Forward)");
    }
}
```

OUTPUT

```
Name   : Aarav Sharma
Course : Java Programming (Learn Being Forward)
```

MODULE 03 · BEGINNER

Variables and data types

Minimum duration **2 days** | Level **Beginner**

A variable is a named memory location that holds a value of a fixed type. This module covers how variables are created, the kinds of variables, and every data type.

IN THIS MODULE

3.1 What is a variable?
3.2 Creating variables
3.3 Rules and conventions for variable names
3.4 Types of variables
3.5 Scope and lifetime of variables
3.6 Data types overview
3.7 Primitive data types (8 types)
3.8 Reference data types
3.9 Literals
3.10 Type casting
3.11 Constants
3.12 Local variable type inference

3.1 What is a variable?

3.1.1 Variable as a named memory location

A program needs to remember things: a student's marks, a running total, a name. It stores them in the computer's memory (RAM). A **variable** is a **named box in memory**: instead of remembering the address `0x7ffd4a2c`, you just use the name `marks`.



EACH VARIABLE IS A LABELLED BOX: TYPE + NAME ON THE LABEL, VALUE INSIDE

3.1.2 Name, type, value and address

Part	Meaning	For int age = 21;
Name	what you call it in code	`age`
Type	what kind of data it can hold and how much memory it uses	`int` (32-bit whole number)
Value	the data currently stored	`21`
Address	where in memory it lives (managed by the JVM, hidden from you)	chosen by the JVM

3.1.3 Why Java needs a declared type (static typing)

Java is **statically typed**: the type of every variable is fixed when you write the code and checked by the compiler. This brings three benefits:

- **Memory:** the JVM knows exactly how many bytes to reserve (4 for an int, 8 for a double).
- **Safety:** putting text into a number variable is caught at compile time, not discovered by a customer.
- **Speed and tooling:** the compiler and IDE know what each variable is, so they can optimise and auto-complete.

THE TYPE NEVER CHANGES

java

```java
int count = 10;
count = 11;          // OK: still an int
count = "eleven";    // compile error: incompatible types: String cannot be converted to int
```

3.2 Creating variables

3.2.1 Declaration: int age;

Declaring a variable tells the compiler its type and name and reserves the memory. Syntax: `type name;`

3.2.2 Initialization: age = 21;

Initialising means storing the first value in it using the assignment operator `=`. Read `=` as "becomes", not "equals".

3.2.3 Declaration + initialization together: int age = 21;

Usually both happen in one statement. This is the most common and safest form, because the variable is never left without a value.

3.2.4 Multiple variables in one line

Several variables of the **same type** can be declared together, separated by commas. Each may or may not be initialised.

3.2.5 Re-assignment

A (non-final) variable can be given a new value at any time; the old value is overwritten and lost. The new value can even be calculated from the old one.

DECLARING, INITIALISING AND RE-ASSIGNING

CreatingVariables.java

```java
public class CreatingVariables {
    public static void main(String[] args) {
        int age;                // 3.2.1 declaration
        age = 21;               // 3.2.2 initialization
        double height = 5.9;    // 3.2.3 both together
        int x = 1, y = 2, z;    // 3.2.4 several in one line (z not yet set)
        z = x + y;
        System.out.println("age=" + age + ", height=" + height + ", z=" + z);

        age = 22;               // 3.2.5 re-assignment
        age = age + 1;          // new value from the old one
        System.out.println("age now = " + age);
    }
}
```

OUTPUT

```
age=21, height=5.9, z=3
age now = 23
```

3.2.6 Default values vs uninitialized local variables

Fields (variables declared in a class) automatically get a **default value** such as 0 or null. **Local variables** (inside a method) get **no default**: using one before giving it a value is a compile-time error. This protects you from accidentally using garbage values.

LOCAL VARIABLES MUST BE INITIALISED BEFORE USE

java

```java
public static void main(String[] args) {
    int total;
    System.out.println(total);   // error: variable total might not have been initialized
}
```

Field type	Default value
byte, short, int, long	0
float, double	0.0
char	'\u0000' (the null character)
boolean	false
any reference (String, arrays, objects)	null

3.3 Rules and conventions for variable names

3.3.1 Allowed characters (letters, digits, _, $)

A variable name may contain letters (including Unicode letters such as `é`), digits, the underscore `_` and the dollar sign `$`. The `$` is legal but reserved by convention for generated code, so avoid it.

3.3.2 Cannot start with a digit or be a keyword

Name	Valid?	Reason
`marks1`	✔	digits allowed after the first character
`1marks`	✘	starts with a digit
`_temp`	✔	may start with underscore
`total-marks`	✘	`-` is the minus operator
`class`	✘	keyword
`Class`	✔ (but confusing)	case differs from the keyword
`_`	✘	a single underscore is a keyword since Java 9

3.3.3 camelCase convention and meaningful names

Variables use **camelCase**: start lowercase, and capitalise each following word: `studentName`, `totalMarks`, `isPassed`. Choose names that say what the value **means**. Boolean names often start with `is`, `has` or `can`.

Poor name	Better name
`a`	`age`
`tm`	`totalMarks`
`flag`	`isLoggedIn`
`data2`	`monthlySalary`

3.4 Types of variables (by where they are declared)

Java has four kinds of variables, depending on **where** they are declared. The place decides where they are stored, how long they live and whether they get default values.

ALL FOUR KINDS OF VARIABLES IN ONE CLASS

Student.java

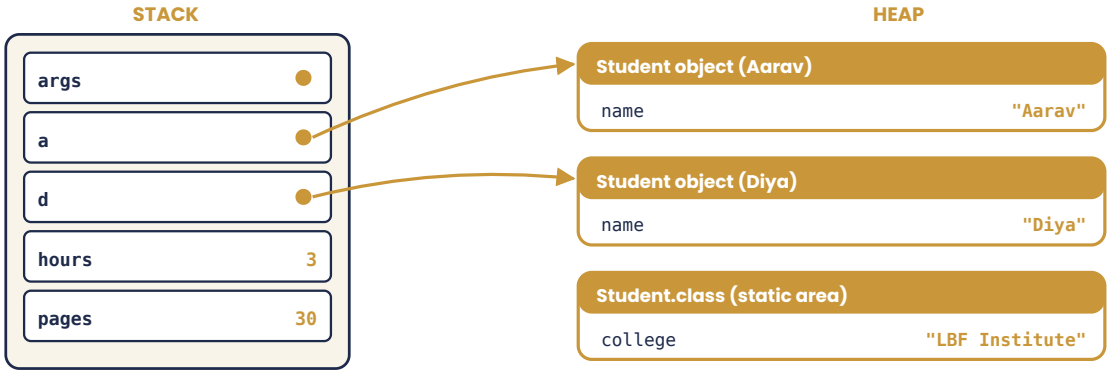
```
public class Student {
    String name;                // instance variable: one per object
    static String college = "LBF Institute";  // static variable: one per class

    void study(int hours) {     // hours: a parameter
        int pages = hours * 10;  // pages: a local variable
        System.out.println(name + " of " + college + " read " + pages + " pages");
    }

    public static void main(String[] args) {
        Student a = new Student();  a.name = "Aarav";
        Student d = new Student();  d.name = "Diya";
        a.study(2);
        d.study(3);
    }
}
```

OUTPUT

```
Aarav of LBF Institute read 20 pages
Diya of LBF Institute read 30 pages
```



LOCALS AND PARAMETERS LIVE ON THE STACK; INSTANCE FIELDS IN EACH OBJECT ON THE HEAP; THE STATIC FIELD ONCE PER CLASS

3.4.1 Local variables

3.4.1.1 Declared inside methods, constructors or blocks

A local variable exists only inside the method, constructor or block `{ }` where it is declared. `pages` above is local to `study`.

3.4.1.2 Stored on the stack

Each method call gets a **stack frame** holding its local variables and parameters. When the method returns, the frame is removed and its local variables disappear.

3.4.1.3 No default value; must be initialized before use

As shown in 3.2.6, the compiler insists that every local variable is assigned before it is read.

3.4.2 Instance variables (non-static fields)

3.4.2.1 Declared inside a class, outside methods

`name` is declared in the class body, not inside any method, so it is an **instance variable** (also called a field or attribute).

3.4.2.2 One copy per object, stored in the heap

Every object created with `new` gets its **own** copy of each instance variable, stored inside the object on the **heap**. Changing `a.name` does not affect `d.name`.

3.4.2.3 Get default values

If you do not assign an instance variable, it holds its default value (0, false or null).

3.4.3 Static variables (class variables)

3.4.3.1 Declared with static

Adding the keyword `static` to a field makes it a **class variable**.

3.4.3.2 One shared copy for the whole class

There is exactly **one** copy, shared by all objects. If one object changes it, every object sees the new value. Use static variables for data that belongs to the class as a whole, such as a counter of objects created or a college name shared by all students.

3.4.3.3 Accessed through the class name

Access it as `Student.college`. Accessing it through an object (`a.college`) compiles but is misleading, and IDEs warn about it.

A STATIC COUNTER SHARED BY ALL OBJECTS

Counter.java

```
public class Counter {
    static int created = 0;      // shared by all objects
    int id;                      // one per object

    Counter() {
        created++;               // every constructor call increases the shared count
        id = created;
    }

    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();
        System.out.println("ids: " + c1.id + ", " + c2.id + ", " + c3.id);
        System.out.println("objects created: " + Counter.created);
    }
}
```

OUTPUT

```
ids: 1, 2, 3
objects created: 3
```

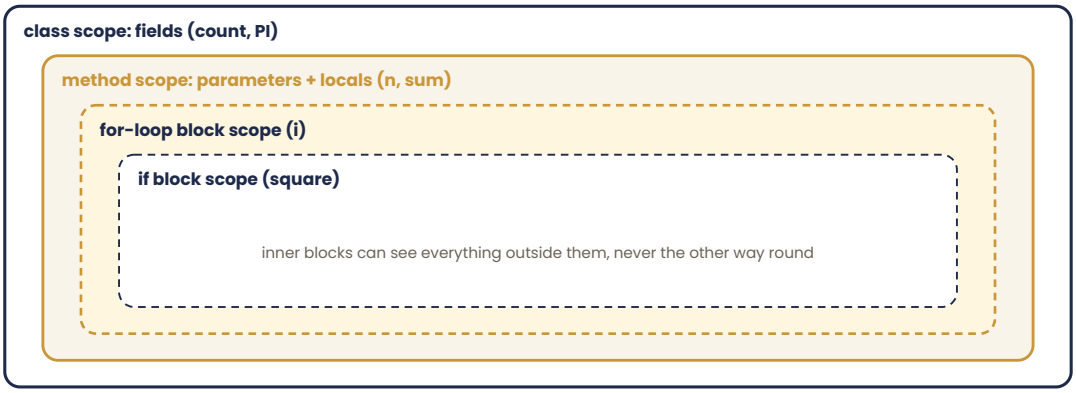
3.4.4 Parameters (method and constructor arguments)

Parameters are variables listed in a method's or constructor's parentheses, such as `hours` in `study(int hours)`. They behave like local variables that are initialised with the values passed by the caller (the **arguments**).

	Local	Instance	Static	Parameter
Declared	inside a method/block	in class, no static	in class, with static	in method ( )
Stored	stack	heap (in the object)	class area (metaspace/heap)	stack
Copies	one per call	one per object	one per class	one per call
Default value	none	yes	yes	set by caller
Lifetime	until block ends	as long as the object	as long as the class is loaded	until method returns

3.5 Scope and lifetime of variables

Scope is the region of code where a name can be used. **Lifetime** is how long the variable exists in memory. They are related but not identical.



NESTED SCOPES: A NAME IS VISIBLE INSIDE ITS OWN BOX AND ALL BOXES WITHIN IT

BLOCK, METHOD AND CLASS SCOPE ScopeDemo.java

```
public class ScopeDemo {
    static int count = 0;                    // class scope

    static int sumOfSquares(int n) {         // n: method scope
        int sum = 0;                         // method scope
        for (int i = 1; i <= n; i++) {       // i: block scope (the loop)
            if (i % 2 == 1) {
                int square = i * i;          // square: inner block scope
                sum += square;
                count++;
            }
            // square is NOT visible here
        }
        // i is NOT visible here
        return sum;
    }

    public static void main(String[] args) {
        System.out.println("sum of odd squares up to 7 = " + sumOfSquares(7));
        System.out.println("odd numbers counted = " + count);
    }
}
```

OUTPUT

```
sum of odd squares up to 7 = 84
odd numbers counted = 4
```

3.5.1 Block scope

A variable declared inside a block (`{ }` of an if, a loop, or a bare block) is visible from its declaration to the closing brace of that block. A `for` loop variable like `i` belongs to the loop.

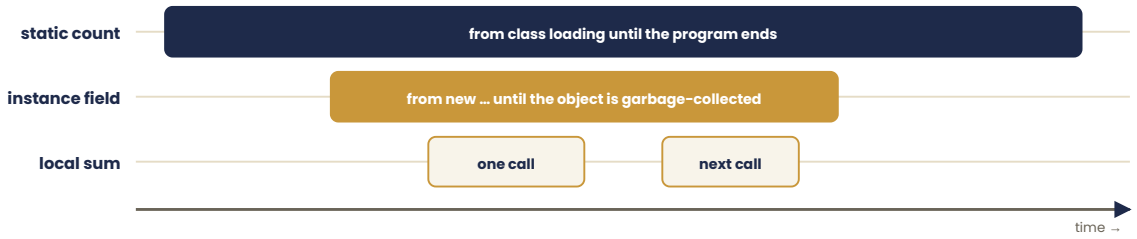
3.5.2 Method scope

Parameters and variables declared at the top level of a method body are visible anywhere later in that method, including in nested blocks, but not in other methods.

3.5.3 Class scope

Fields are visible in every method of the class (static methods can see only static fields directly). Access modifiers (Module 11) control visibility from **other** classes.

3.5.4 Lifetime of local, instance and static variables



HOW LONG EACH KIND OF VARIABLE LIVES

3.5.5 Variable shadowing

If a local variable or parameter has the **same name** as a field, the local one **shadows** (hides) the field inside that method. Use `this.name` to reach the field. This is exactly why constructors often write `this.name = name;`.

A PARAMETER SHADOWING A FIELD Shadow.java

```
public class Shadow {
    int value = 100;                    // field

    void show(int value) {              // parameter shadows the field
        System.out.println("parameter value = " + value);
        System.out.println("field value     = " + this.value);
    }

    public static void main(String[] args) {
        new Shadow().show(5);
    }
}
```

OUTPUT

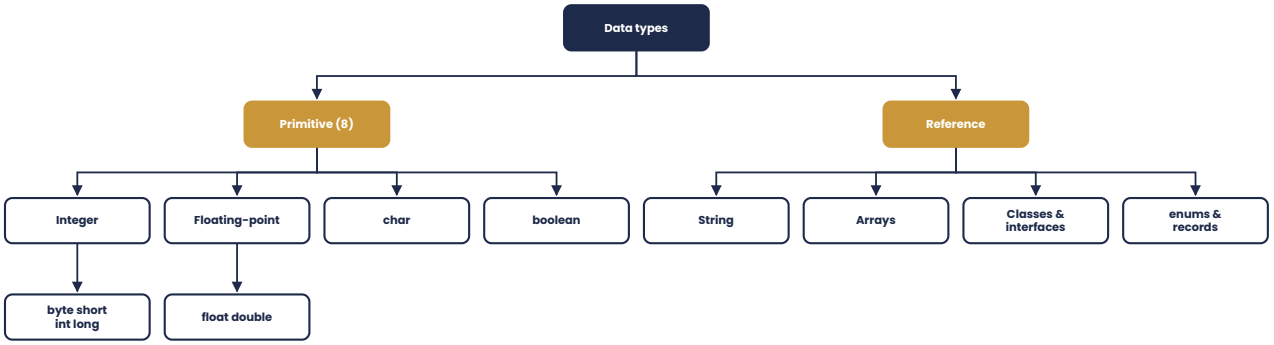
```
parameter value = 5
field value     = 100
```

WATCH OUT

Java does **not** allow a local variable to shadow another local variable or parameter of the same method: `int x = 1; { int x = 2; }` is a compile error ("variable x is already defined").

3.6 Data types overview

3.6.1 Primitive types vs reference (non-primitive) types



THE TWO FAMILIES OF JAVA DATA TYPES

	Primitive	Reference
What it holds	the **actual value**	the **address** of an object
How many	exactly 8, built into the language	unlimited: any class, array, interface

	Primitive	Reference
Size	fixed (1 to 8 bytes)	a reference is small; the object can be any size
Default (as a field)	0 / 0.0 / false / \u0000	`null`
Can be null?	no	yes
Has methods?	no	yes (`name.length()`)
Naming	lowercase: `int`	PascalCase: `String`

3.6.2 How each is stored in memory

A primitive local variable stores its value **directly** in the stack frame. A reference variable stores only an **arrow** (a reference) to an object that lives on the heap. Copying a primitive copies the value; copying a reference copies the arrow, so both variables point to the **same** object.

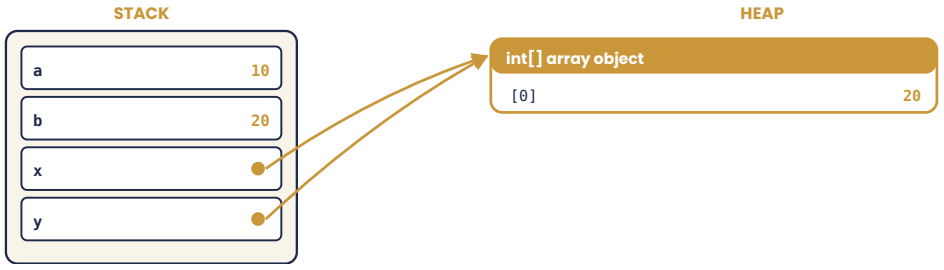
COPYING A PRIMITIVE VS COPYING A REFERENCE — PrimitiveVsReference.java

```
public class PrimitiveVsReference {
    public static void main(String[] args) {
        int a = 10;
        int b = a;          // copies the VALUE
        b = 20;
        System.out.println("a = " + a + ", b = " + b);

        int[] x = {10};
        int[] y = x;        // copies the REFERENCE (same array!)
        y[0] = 20;
        System.out.println("x[0] = " + x[0] + ", y[0] = " + y[0]);
    }
}
```

OUTPUT

```
a = 10, b = 20
x[0] = 20, y[0] = 20
```

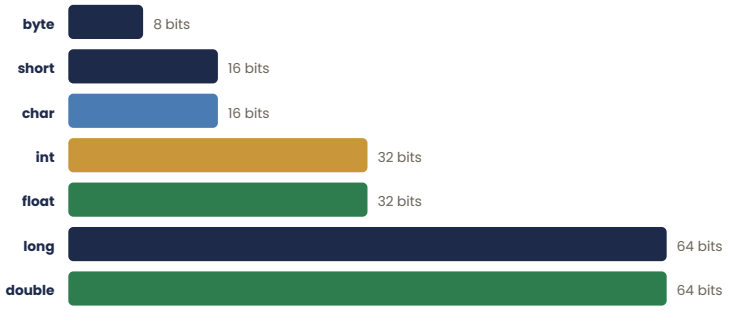


A AND B ARE INDEPENDENT; X AND Y SHARE ONE ARRAY

3.7 Primitive data types (8 types)

Type	Size	Range / precision	Default	Example
`byte`	1 byte (8 bits)	-128 to 127	0	`byte b = 100;`
`short`	2 bytes	-32,768 to 32,767	0	`short s = 30000;`
`int`	4 bytes	-2,147,483,648 to 2,147,483,647	0	`int i = 2_000_000;`
`long`	8 bytes	about ±9.2 × 10^{18}	0L	`long l = 9_000_000_000L;`
`float`	4 bytes	~7 significant digits	0.0f	`float f = 3.14f;`
`double`	8 bytes	~15–16 significant digits	0.0	`double d = 3.14159;`
`char`	2 bytes	0 to 65,535 (Unicode)	'\u0000'	`char c = 'A';`

Type	Size	Range / precision	Default	Example
`boolean`	JVM-dependent (1 bit of info)	true / false	false	`boolean ok = true;`



SIZE OF EACH PRIMITIVE TYPE IN BITS

3.7.1 Integer types

Four types store whole numbers (no decimal point). They differ only in size and range. All are **signed** (they can be negative) and use two's complement (Module 4).

3.7.1.1 byte: 1 byte, -128 to 127

The smallest integer type. Used for raw binary data such as file contents, images and network packets, or to save memory in very large arrays.

3.7.1.2 short: 2 bytes, -32,768 to 32,767

Rarely used in everyday code; sometimes in large arrays where memory matters or when reading binary formats that store 16-bit values.

3.7.1.3 int: 4 bytes, about ±2.1 billion (the default integer type)

The type you will use for almost every whole number: counters, ages, marks, loop indexes. A whole-number literal like `42` is an `int` by default.

3.7.1.4 long: 8 bytes, L suffix

For values beyond ±2.1 billion: phone numbers stored as numbers, population counts, milliseconds since 1970, bank balances in paise. Add an **L** to a literal that is too large for int.

THE FOUR INTEGER TYPES AND THEIR LIMITS

IntegerTypes.java

```
public class IntegerTypes {
    public static void main(String[] args) {
        byte  b = 127;
        short s = 32_767;
        int   i = 2_147_483_647;
        long  l = 9_223_372_036_854_775_807L;   // L is required for big literals
        System.out.println("byte : " + b + "   min " + Byte.MIN_VALUE);
        System.out.println("short: " + s + "   min " + Short.MIN_VALUE);
        System.out.println("int  : " + i + "   min " + Integer.MIN_VALUE);
        System.out.println("long : " + l + "   min " + Long.MIN_VALUE);
        long worldPopulation = 8_100_000_000L;
        System.out.println("world population: " + worldPopulation);
    }
}
```

OUTPUT

```
byte : 127   min -128
short: 32767   min -32768
int  : 2147483647   min -2147483648
long : 9223372036854775807   min -9223372036854775808
world population: 8100000000
```

3.7.2 Floating-point types

`float` and `double` store numbers with a fractional part (3.75, -0.001, 6.02e23). They follow the IEEE 754 standard and store an **approximation** of the number in binary.

3.7.2.1 float: 4 bytes, f suffix, about 7 digits precision

A decimal literal is a double by default, so a float literal needs an **f** suffix: `float f = 2.5f;`. Use float only where memory matters (graphics, large arrays); otherwise prefer double.

3.7.2.2 double: 8 bytes, about 15 digits precision (the default decimal type)

The standard choice for decimal numbers: percentages, averages, measurements, scientific values.

3.7.2.3 Precision problems (0.1 + 0.2) and when to use BigDecimal

Many decimal fractions, such as 0.1, cannot be stored exactly in binary (just like 1/3 cannot be written exactly in decimal). Small errors appear in calculations. For **money** and anything that must be exact, use `java.math.BigDecimal`.

FLOAT VS DOUBLE AND THE 0.1 + 0.2 PROBLEM — FloatPrecision.java

```java
import java.math.BigDecimal;

public class FloatPrecision {
    public static void main(String[] args) {
        float  f = 1.1234567890f;
        double d = 1.1234567890123456789;
        System.out.println("float : " + f);
        System.out.println("double: " + d);

        System.out.println("0.1 + 0.2 = " + (0.1 + 0.2));
        System.out.println("0.1 + 0.2 == 0.3 ? " + (0.1 + 0.2 == 0.3));

        BigDecimal a = new BigDecimal("0.1");
        BigDecimal b = new BigDecimal("0.2");
        System.out.println("BigDecimal: " + a.add(b));
    }
}
```

OUTPUT

```
float : 1.1234568
double: 1.1234567890123457
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 ? false
BigDecimal: 0.3
```

WATCH OUT

Never compare floating-point results with `==`. Check whether they are close enough instead: `Math.abs(x - y) < 1e-9`. And never use float/double for money: use `BigDecimal` or store whole paise in a `long`.

3.7.3 char: 2 bytes, Unicode, single quotes, ASCII/Unicode values

A `char` holds **one** character in **single quotes**: `'A'`, `'7'`, `'₹'`. Internally it is a 16-bit number, its **Unicode** code. The first 128 Unicode values are the same as **ASCII**: 'A' = 65, 'a' = 97, '0' = 48.

CHAR VALUES AND THEIR UNICODE NUMBERS CharDemo.java

```
public class CharDemo {
    public static void main(String[] args) {
        char letter = 'A';
        char digit = '7';
        char rupee = '\u20B9';          // Unicode escape for the rupee sign
        System.out.println(letter + " " + digit + " " + rupee);
        System.out.println("code of 'A' = " + (int) letter);
        System.out.println("code of 'a' = " + (int) 'a');
        char next = (char) (letter + 1);
        System.out.println("letter after A = " + next);
    }
}
```

OUTPUT

```
A 7 ₹
code of 'A' = 65
code of 'a' = 97
letter after A = B
```



CHARACTERS AND THEIR ASCII / UNICODE CODES

3.7.4 boolean: true / false

A `boolean` holds only `true` or `false`. It is the result of comparisons (`marks >= 40`) and drives every if-statement and loop. Unlike C, Java does **not** treat 0 and 1 as booleans.

BOOLEAN VALUES COME FROM COMPARISONS BooleanDemo.java

```
public class BooleanDemo {
    public static void main(String[] args) {
        int marks = 72;
        boolean passed = marks >= 40;
        boolean distinction = marks >= 75;
        System.out.println("passed = " + passed);
        System.out.println("distinction = " + distinction);
    }
}
```

OUTPUT

```
passed = true
distinction = false
```

3.8 Reference data types

Every type that is not one of the eight primitives is a **reference type**. A reference variable stores a reference (an arrow) to an object on the heap.

3.8.1 String

`String` is a class that represents text. It is so common that Java gives it special support: string literals in double quotes and the `+` operator for joining. Module 8 covers strings in depth.

3.8.2 Arrays

An array holds a fixed number of values of one type: `int[] marks = {80, 75, 92};`. Even an array of primitives is an object on the heap (Module 7).

3.8.3 Classes, interfaces and enums

Every class you write is a new type: `Student s = new Student();`. The same goes for interfaces, enums and records (Modules 10, 13).

3.8.4 null and what it means

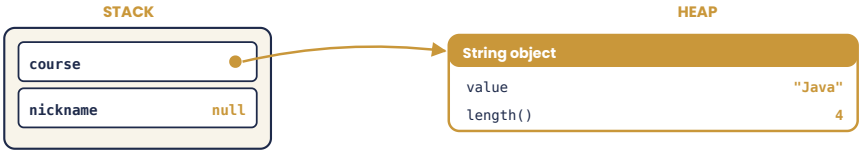
`null` means "this reference points to **no object**". It is the default value of reference fields. Calling a method on a null reference throws a **NullPointerException**, one of the most common runtime errors in Java.

USING A NULL REFERENCE — NullDemo.java

```
public class NullDemo {
    public static void main(String[] args) {
        String course = "Java";
        String nickname = null;          // refers to nothing
        System.out.println(course.length());
        System.out.println("nickname is null? " + (nickname == null));
        System.out.println(nickname.length());   // NullPointerException
    }
}
```

OUTPUT

```
4
nickname is null? true
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()" because "nickname" is null
        at NullDemo.main(NullDemo.java:7)
```



COURSE POINTS TO AN OBJECT; NICKNAME POINTS NOWHERE

3.9 Literals

A **literal** is a fixed value written directly in the code, such as `42`, `3.14`, `'A'` or `"Hello"`.

3.9.1 Integer literals: decimal, binary 0b, octal 0, hexadecimal 0x

THE SAME NUMBER 100 IN FOUR BASES — IntegerLiterals.java

```
public class IntegerLiterals {
    public static void main(String[] args) {
        int decimal = 100;          // base 10
        int binary  = 0b1100100;    // base 2  (prefix 0b)
        int octal   = 0144;         // base 8  (prefix 0)
        int hex     = 0x64;         // base 16 (prefix 0x)
        System.out.println(decimal + " " + binary + " " + octal + " " + hex);
        System.out.println("0xFF = " + 0xFF + ", 0b1111 = " + 0b1111);
    }
}
```

OUTPUT

```
100 100 100 100
0xFF = 255, 0b1111 = 15
```

> **WATCH OUT**
> A leading zero makes a number **octal**: `int x = 010;` is 8, not 10. And `09` does not compile.

3.9.2 Underscores in numbers: 1_000_000

Since Java 7 you may put underscores between digits to make long numbers readable: `1_000_000`, `98_480_11111L`, `0b1010_0101`. They are ignored by the compiler. They cannot appear at the start or end, or next to the decimal point.

3.9.3 Floating-point and scientific notation 1.5e3

Decimal literals are double by default; add `f` for float (or `d` for double, optional). Scientific notation writes `1.5e3` for 1.5 × 10^3 = 1500.0 and `2.5e-4` for 0.00025.

SCIENTIFIC NOTATION

FloatLiterals.java

```
public class FloatLiterals {
    public static void main(String[] args) {
        double a = 1.5e3;
        double b = 2.5e-4;
        float  c = 9.81f;
        double avogadro = 6.022e23;
        System.out.println(a + "  " + b + "  " + c + "  " + avogadro);
    }
}
```

OUTPUT

```
1500.0  2.5E-4  9.81  6.022E23
```

3.9.4 Character literals and escape sequences (\n, \t, \\, \', \", \u0041)

Some characters cannot be typed directly inside quotes. An **escape sequence** starts with a backslash.

Escape	Meaning
`\n`	new line
`\t`	tab
`\\`	a backslash
`\'`	a single quote (in a char)
`\"`	a double quote (in a String)
`\r`	carriage return
`\uXXXX`	the Unicode character with hex code XXXX, e.g. `\u0041` = A

ESCAPE SEQUENCES IN ACTION

Escapes.java

```
public class Escapes {
    public static void main(String[] args) {
        System.out.println("Name:\tAarav\nCourse:\tJava");
        System.out.println("He said \"Hello\"");
        System.out.println("Path: C:\\Users\\lbf");
        char quote = '\'';
        char a = '\u0041';
        System.out.println(quote + " " + a);
    }
}
```

OUTPUT

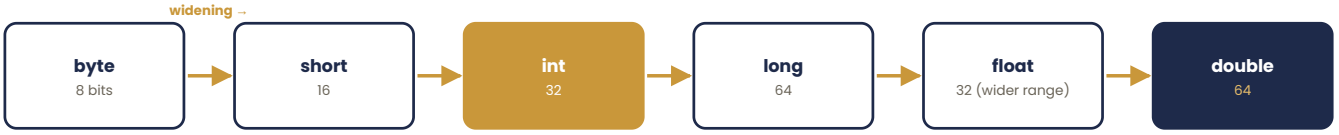
```
Name:   Aarav
Course: Java
He said "Hello"
Path: C:\Users\lbf
' A
```

3.9.5 Boolean, string and null literals

The boolean literals are `true` and `false`. A string literal is text in double quotes: `"Learn Being Forward"` (empty string: `""`). The null literal `null` can be assigned to any reference type.

3.10 Type casting (type conversion)

Sometimes a value of one type must be used as another type. **Casting** converts it. Converting to a **bigger** type is automatic and safe; converting to a **smaller** type must be written explicitly because data may be lost.



WIDENING ORDER: EACH TYPE CONVERTS AUTOMATICALLY TO ANY TYPE ON ITS RIGHT (CHAR ALSO WIDENS TO INT)

3.10.1 Implicit casting (widening): byte → short → int → long → float → double

When a smaller type is assigned to a larger one, Java converts it **automatically**, because no information is lost (long to float may lose precision but keeps the magnitude).

3.10.2 Explicit casting (narrowing): (int) 9.7

To go to a smaller type, write the target type in parentheses. Converting a double to an int **truncates** (cuts off) the fraction: it does not round.

3.10.3 Data loss and overflow when narrowing

If the value does not fit in the smaller type, the extra high bits are thrown away and the number **wraps around**. 130 does not fit in a byte (max 127), so it becomes -126.

WIDENING, NARROWING AND OVERFLOW

Casting.java

```
public class Casting {
    public static void main(String[] args) {
        // widening (automatic)
        int i = 100;
        long l = i;
        double d = l;
        System.out.println("int -> long -> double: " + d);

        // narrowing (explicit)
        double price = 9.7;
        int whole = (int) price;           // fraction is cut off
        System.out.println("(int) 9.7 = " + whole);
        System.out.println("rounded   = " + Math.round(price));

        // overflow when the value does not fit
        int big = 130;
        byte small = (byte) big;
        System.out.println("(byte) 130 = " + small);
        int huge = (int) 3_000_000_000L;
        System.out.println("(int) 3,000,000,000 = " + huge);
    }
}
```

OUTPUT

```
int -> long -> double: 100.0
(int) 9.7 = 9
rounded   = 10
(byte) 130 = -126
(int) 3,000,000,000 = -1294967296
```



COUNTING PAST 127 IN A BYTE WRAPS AROUND TO -128

3.10.4 Type promotion in expressions (byte + byte = int)

In arithmetic, Java first **promotes** byte, short and char operands to **int**. If any operand is long, float or double, the whole expression uses the largest of these types. That is why adding two bytes gives an int.

PROMOTION RULES IN EXPRESSIONS

Promotion.java

```
public class Promotion {
    public static void main(String[] args) {
        byte a = 10, b = 20;
        // byte c = a + b;          // error: possible lossy conversion from int to byte
        int c = a + b;              // byte + byte = int
        byte d = (byte) (a + b);    // or cast back explicitly
        System.out.println(c + " " + d);

        int x = 7;
        double y = 2;
        System.out.println("int / int    = " + (x / 2));     // 3
        System.out.println("int / double = " + (x / y));     // 3.5
        char ch = 'A';
        System.out.println("char + int   = " + (ch + 2));    // 67
    }
}
```

OUTPUT

```
30 30
int / int    = 3
int / double = 3.5
char + int   = 67
```

If the expression contains…	it is evaluated as
only byte, short, char, int	`int`
at least one long (and no float/double)	`long`
at least one float (and no double)	`float`
at least one double	`double`

3.10.5 char ↔ int conversion

A char is a number underneath, so it widens to int automatically; going from int to char needs a cast. This is the basis of many character tricks: shifting letters, converting a digit character to its value.

CONVERTING BETWEEN CHAR AND INT

CharInt.java

```
public class CharInt {
    public static void main(String[] args) {
        char c = 'D';
        int code = c;                   // char -> int (automatic)
        char back = (char) (code + 3);  // int -> char (cast)
        System.out.println(c + " -> " + code + " -> " + back);

        char digit = '8';
        int value = digit - '0';        // '8' (56) - '0' (48) = 8
        System.out.println("digit value = " + value);

        char upper = (char) ('g' - 32);  // lower to upper case
        System.out.println("upper of g = " + upper);
    }
}
```

OUTPUT

```
D -> 68 -> G
digit value = 8
upper of g = G
```

3.11 Constants

3.11.1 final variables

Adding `final` makes a variable a **constant**: it can be assigned exactly once. Any later assignment is a compile-time error. Use final for values that should never change, and to make your intent clear.

3.11.2 static final constants and naming

A constant that belongs to the whole class is declared `static final` and named in **UPPER_SNAKE_CASE**. The standard library is full of them: `Math.PI`, `Integer.MAX_VALUE`.

FINAL AND STATIC FINAL CONSTANTS — Constants.java

```
public class Constants {
    static final double GST_RATE = 0.18;          // class constant
    static final int MAX_STUDENTS_PER_BATCH = 40;

    public static void main(String[] args) {
        final int year = 2026;                    // local constant
        // year = 2027;                           // error: cannot assign a value to final variable year
        double fee = 15000;
        double total = fee + fee * GST_RATE;
        System.out.println("Year " + year + ", batch size " + MAX_STUDENTS_PER_BATCH);
        System.out.println("Fee with GST = " + total);
        System.out.println("PI = " + Math.PI);
    }
}
```

OUTPUT

```
Year 2026, batch size 40
Fee with GST = 17700.0
PI = 3.141592653589793
```

TIP

Replace "magic numbers" in your code with named constants. `fee * GST_RATE` explains itself; `fee * 0.18` does not.

3.12 Local variable type inference

3.12.1 var keyword (Java 10+)

With `var`, the compiler **infers** the type of a local variable from the value on the right. The variable is still statically typed: its type is fixed at compile time, you just do not have to write it.

VAR INFERS THE TYPE FROM THE INITIALIZER — VarDemo.java

```
import java.util.ArrayList;

public class VarDemo {
    public static void main(String[] args) {
        var count = 10;                    // int
        var price = 99.5;                  // double
        var name = "Diya";                 // String
        var list = new ArrayList<String>(); // ArrayList<String>
        list.add(name);
        // count = "ten";                  // error: count is an int
        System.out.println(count + " " + price + " " + name + " " + list);
    }
}
```

OUTPUT

```
10 99.5 Diya [Diya]
```

3.12.2 Where var is allowed and where it is not

Allowed ✔	Not allowed ✘
local variables with an initializer: `var x = 5;`	without an initializer: `var x;`
for-loop variables: `for (var s : names)`	initialised to null: `var x = null;`
try-with-resources variables	fields (instance or static variables)
lambda parameters (Java 11)	method parameters and return types
	array initializer: `var a = {1, 2};`

TIP

Use `var` when the type is obvious from the right-hand side (`var list = new ArrayList<String>()`). Write the type when it helps the reader (`int total = calculate();`).

PRACTICE CHECKPOINT

A program that declares **one variable of each primitive type**, prints them, and shows **widening** and **narrowing** casts.

SAMPLE SOLUTION AllPrimitives.java

```
public class AllPrimitives {
    public static void main(String[] args) {
        byte b = 120;
        short s = 32000;
        int i = 1_500_000;
        long l = 8_100_000_000L;
        float f = 3.14f;
        double d = 2.718281828;
        char c = 'J';
        boolean ok = true;
        System.out.println("byte=" + b + " short=" + s + " int=" + i + " long=" + l);
        System.out.println("float=" + f + " double=" + d + " char=" + c + " boolean=" + ok);

        double widened = i;              // int -> double (automatic)
        long fromInt = s;                // short -> long (automatic)
        int narrowed = (int) d;          // double -> int (explicit)
        byte overflow = (byte) 200;      // does not fit: wraps around
        System.out.println("widened=" + widened + " fromInt=" + fromInt);
        System.out.println("narrowed=" + narrowed + " overflow=" + overflow);
    }
}
```

OUTPUT

```
byte=120 short=32000 int=1500000 long=8100000000
float=3.14 double=2.718281828 char=J boolean=true
widened=1500000.0 fromInt=32000
narrowed=2 overflow=-56
```

MODULE 04 · BEGINNER

Operators and expressions

Minimum duration **2 days** | Level **Beginner**

Operators act on values to produce new values; this module covers every operator group and the order Java evaluates them in.

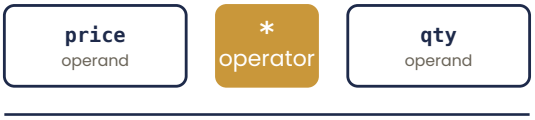
IN THIS MODULE

4.1 Basics
4.2 Arithmetic operators
4.3 Unary operators
4.4 Assignment operators
4.5 Relational (comparison) operators
4.6 Logical operators
4.7 Bitwise operators
4.8 Shift operators
4.9 Ternary operator
4.10 instanceof operator
4.11 Operator precedence and associativity
4.12 The Math class

4.1 Basics

4.1.1 Operator, operand and expression

An **operator** is a symbol that performs an action, such as `+` or `>`. The values it acts on are its **operands**. A combination of operands and operators that produces a value is an **expression**.



expression: price * qty → a value

OPERANDS, OPERATOR AND EXPRESSION

4.1.2 Unary, binary and ternary operators

Kind	Number of operands	Examples
Unary	1	`-x`, `++i`, `!done`
Binary	2	`a + b`, `x > y`, `p && q`
Ternary	3	`marks >= 40 ? "Pass" : "Fail"` (the only one)

Arithmetic
+ - * / %

Unary
+ - ++ -- !

Assignment
= += -= *= /= %=

Relational
== != > < >= <=

Logical
&& || !

Bitwise
& | ^ ~

Shift
<< >> >>>

Ternary
? :

Type check
instanceof

4.2 Arithmetic operators

4.2.1 + - * / %

Operator	Meaning	Example	Result
`+`	addition	`17 + 5`	22
`-`	subtraction	`17 - 5`	12
`*`	multiplication	`17 * 5`	85
`/`	division	`17 / 5`	3 (integer division)
`%`	remainder (modulus)	`17 % 5`	2

THE FIVE ARITHMETIC OPERATORS Arithmetic.java

```
public class Arithmetic {
    public static void main(String[] args) {
        int a = 17, b = 5;
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b));
        System.out.println("a % b = " + (a % b));
    }
}
```

OUTPUT

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
```

4.2.2 Integer division vs floating-point division

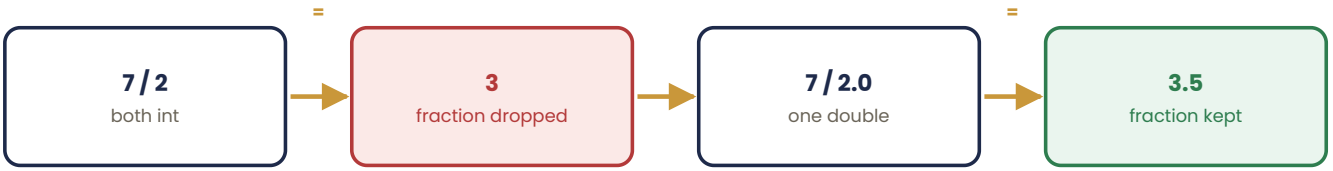
If **both** operands are integers, `/` performs **integer division**: the fractional part is thrown away (truncated toward zero). If at least one operand is a float or double, the result keeps the fraction.

WHEN THE FRACTION IS KEPT AND WHEN IT IS LOST Division.java

```
public class Division {
    public static void main(String[] args) {
        System.out.println(7 / 2);          // 3   (int / int)
        System.out.println(7 / 2.0);        // 3.5 (int / double)
        System.out.println(7.0 / 2);        // 3.5
        System.out.println((double) 7 / 2); // 3.5 (cast first)
        System.out.println((double) (7 / 2)); // 3.0 (too late: division already done)
        System.out.println(-7 / 2);         // -3  (truncates toward zero)
        int total = 250, count = 3;
        System.out.println("average = " + (double) total / count);
    }
}
```

OUTPUT

```
3
3.5
3.5
3.5
3.0
-3
average = 83.33333333333333
```



INTEGER DIVISION DROPS THE FRACTION

4.2.3 Modulus with negative numbers

`%` gives the remainder after integer division. In Java the result always has the **same sign as the left operand** (the dividend). This matters when checking odd numbers: `-7 % 2` is **−1**, not 1, so test odd numbers with `n % 2 != 0`.

THE SIGN OF % FOLLOWS THE LEFT OPERAND — Modulus.java

```
public class Modulus {
    public static void main(String[] args) {
        System.out.println(" 7 %  3 = " + ( 7 %  3));
        System.out.println("-7 %  3 = " + (-7 %  3));
        System.out.println(" 7 % -3 = " + ( 7 % -3));
        System.out.println("-7 % -3 = " + (-7 % -3));
        System.out.println("5.5 % 2 = " + (5.5 % 2));    // works on decimals too
        System.out.println("Math.floorMod(-7, 3) = " + Math.floorMod(-7, 3));
    }
}
```

OUTPUT

```
 7 %  3 = 1
-7 %  3 = -1
 7 % -3 = 1
-7 % -3 = -1
5.5 % 2 = 1.5
Math.floorMod(-7, 3) = 2
```

Even / odd
`n % 2 == 0` → even

Last digit
`n % 10` → 1234 % 10 = 4

Wrap around
`(i + 1) % size` → circular index

Divisible?
`year % 4 == 0`

4.2.4 + for string concatenation

If either operand of `+` is a String, Java joins (**concatenates**) them into a new String. Expressions are evaluated from left to right, so the position of the String matters.

STRING CONCATENATION AND EVALUATION ORDER — Concat.java

```
public class Concat {
    public static void main(String[] args) {
        String name = "Diya";
        int marks = 92;
        System.out.println("Name: " + name + ", marks: " + marks);
        System.out.println(10 + 20 + " apples");   // 30 apples   (numbers added first)
        System.out.println("apples " + 10 + 20);   // apples 1020 (string first)
        System.out.println("apples " + (10 + 20)); // apples 30   (parentheses)
        System.out.println('A' + 'B');            // 131 (char + char = int!)
        System.out.println("" + 'A' + 'B');       // AB
    }
}
```

OUTPUT

```
Name: Diya, marks: 92
30 apples
apples 1020
apples 30
131
AB
```

4.2.5 Division by zero (int vs double behaviour)

Integer division by zero is impossible, so it throws an **ArithmeticException**. Floating-point arithmetic follows IEEE 754 and instead produces special values: **Infinity**, **-Infinity** or **NaN** (Not a Number).

DIVIDING BY ZERO: INT VS DOUBLE DivideByZero.java

```
public class DivideByZero {
    public static void main(String[] args) {
        System.out.println(10.0 / 0);     // Infinity
        System.out.println(-10.0 / 0);    // -Infinity
        System.out.println(0.0 / 0);      // NaN
        double nan = 0.0 / 0;
        System.out.println("NaN == NaN ? " + (nan == nan));     // false!
        System.out.println("isNaN ? " + Double.isNaN(nan));
        System.out.println(10 / 0);       // ArithmeticException
    }
}
```

OUTPUT

```
Infinity
-Infinity
NaN
NaN == NaN ? false
isNaN ? true
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at DivideByZero.main(DivideByZero.java:9)
```

4.3 Unary operators

4.3.1 Unary plus and minus

Unary minus `-x` negates a value; unary plus `+x` leaves it unchanged (but promotes byte/short/char to int).

4.3.2 Increment ++ and decrement --

`++` adds 1 to a variable and `--` subtracts 1. They can be written **before** (prefix) or **after** (postfix) the variable. On its own line there is no difference; inside a bigger expression the difference matters.

4.3.2.1 Pre-increment ++x

Increment first, then use the new value. `y = ++x;` is the same as `x = x + 1; y = x;`.

4.3.2.2 Post-increment x++

Use the current value first, then increment. `y = x++;` is the same as `y = x; x = x + 1;`.

PREFIX VS POSTFIX

IncDec.java

```
public class IncDec {
    public static void main(String[] args) {
        int x = 5;
        int a = ++x;      // x becomes 6, a = 6
        System.out.println("after a = ++x : x=" + x + " a=" + a);

        int y = 5;
        int b = y++;      // b = 5, then y becomes 6
        System.out.println("after b = y++ : y=" + y + " b=" + b);

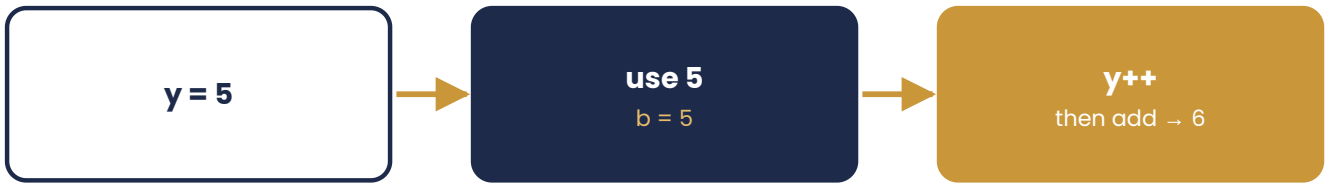
        int z = 10;
        System.out.println(z--);   // prints 10, z becomes 9
        System.out.println(--z);   // z becomes 8, prints 8
    }
}
```

OUTPUT

```
after a = ++x : x=6 a=6
after b = y++ : y=6 b=5
10
8
```



PRE-INCREMENT: CHANGE, THEN USE



POST-INCREMENT: USE, THEN CHANGE

4.3.2.3 Tricky expressions (x = x++ + ++x)

Interview favourite. Evaluate left to right, keeping track of x after every step:

WORKING THROUGH TRICKY INCREMENTS

Tricky.java

```
public class Tricky {
    public static void main(String[] args) {
        int x = 5;
        x = x++ + ++x;
        // step 1: x++  -> value 5, x becomes 6
        // step 2: ++x  -> x becomes 7, value 7
        // step 3: 5 + 7 = 12 is assigned to x
        System.out.println(x);

        int i = 1;
        i = i++;          // the old value (1) is assigned back!
        System.out.println(i);

        int a = 2;
        int b = a++ * 3 + --a;   // 2*3 + (a: 3 -> 2) = 6 + 2
        System.out.println(a + " " + b);
    }
}
```

OUTPUT

```
12
1
2 8
```

WATCH OUT

Never write code like `x = x++ + ++x` in real programs. It is legal but confusing. Increment on a separate line.

4.3.3 Logical complement !

`!` flips a boolean: `!true` is false and `!false` is true. Read it as "not": `if (!isLoggedIn)`.

4.4 Assignment operators

4.4.1 Simple assignment =

`=` evaluates the right side and stores the result in the variable on the left. It is **right-associative** and is itself an expression, so `a = b = c = 0;` sets all three to 0.

4.4.2 Compound assignment += -= *= /= %= and implicit casting

Operator	Example	Same as
`+=`	`x += 5`	`x = x + 5`
`-=`	`x -= 5`	`x = x - 5`
`*=`	`x *= 5`	`x = x * 5`
`/=`	`x /= 5`	`x = x / 5`
`%=`	`x %= 5`	`x = x % 5`
`&= |= ^= <<= >>=`	`x <<= 1`	`x = x << 1`

Compound operators contain a hidden **cast** back to the variable's type. `b += 5` on a byte compiles, but `b = b + 5` does not, because `b + 5` is an int.

COMPOUND ASSIGNMENT AND ITS HIDDEN CAST

CompoundAssign.java

```
public class CompoundAssign {
    public static void main(String[] args) {
        int total = 100;
        total += 50;  total -= 30;  total *= 2;  total /= 4;  total %= 7;
        System.out.println("total = " + total);

        byte b = 10;
        // b = b + 5;        // error: possible lossy conversion from int to byte
        b += 5;              // OK: means b = (byte) (b + 5)
        System.out.println("b = " + b);

        byte c = 120;
        c += 10;             // hidden cast: overflow wraps around!
        System.out.println("c = " + c);

        int x, y, z;
        x = y = z = 7;       // chained assignment (right to left)
        System.out.println(x + " " + y + " " + z);
    }
}
```

OUTPUT

```
total = 4
b = 15
c = -126
7 7 7
```

4.5 Relational (comparison) operators

4.5.1 == != > < >= <=

Relational operators compare two values and always produce a **boolean**.

Operator	Meaning	Example (a = 10, b = 20)	Result
`==`	equal to	`a == b`	false
`!=`	not equal to	`a != b`	true
`>`	greater than	`a > b`	false
`<`	less than	`a < b`	true
`>=`	greater or equal	`a >= 10`	true
`<=`	less or equal	`b <= 15`	false

4.5.2 == on primitives vs references

For primitives, `==` compares **values**. For objects, `==` compares **references**: it is true only if both variables point to the **same object**. To compare the contents of objects (such as Strings), use `equals()`.

== COMPARES REFERENCES FOR OBJECTS

EqualsDemo.java

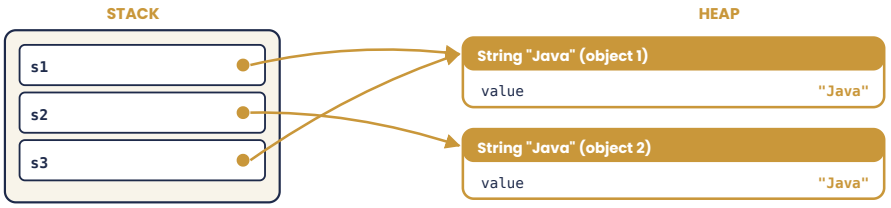
```
public class EqualsDemo {
    public static void main(String[] args) {
        int a = 100, b = 100;
        System.out.println("a == b : " + (a == b));            // true: same value

        String s1 = new String("Java");
        String s2 = new String("Java");
        System.out.println("s1 == s2      : " + (s1 == s2));      // false: two objects
        System.out.println("s1.equals(s2) : " + s1.equals(s2));   // true: same text

        String s3 = s1;
        System.out.println("s1 == s3      : " + (s1 == s3));      // true: same object
    }
}
```

OUTPUT

```
a == b : true
s1 == s2      : false
s1.equals(s2) : true
s1 == s3      : true
```



S1 == S3 IS TRUE (SAME ARROW TARGET); S1 == S2 IS FALSE

4.6 Logical operators

4.6.1 && AND, || OR, ! NOT

Logical operators combine boolean values. They let you express conditions such as "marks at least 40 **and** attendance at least 75".

COMBINING CONDITIONS

Logical.java

```
public class Logical {
    public static void main(String[] args) {
        int marks = 68, attendance = 80;
        boolean eligible = marks >= 40 && attendance >= 75;
        boolean scholarship = marks >= 90 || attendance == 100;
        boolean needsHelp = !eligible;
        System.out.println("eligible    = " + eligible);
        System.out.println("scholarship = " + scholarship);
        System.out.println("needsHelp   = " + needsHelp);
    }
}
```

OUTPUT

```
eligible    = true
scholarship = false
needsHelp   = false
```

4.6.2 Short-circuit evaluation

`&&` and `||` stop as soon as the answer is known. If the left side of `&&` is false, the right side is **never evaluated**; if the left side of `||` is true, the right side is skipped. This prevents errors such as dividing by zero or calling a method on null.

THE RIGHT SIDE IS SKIPPED WHEN THE ANSWER IS ALREADY KNOWN

ShortCircuit.java

```java
public class ShortCircuit {
    static boolean check(String label, boolean value) {
        System.out.println("  evaluating " + label);
        return value;
    }
    public static void main(String[] args) {
        System.out.println("false && ... :");
        boolean r1 = check("left", false) && check("right", true);
        System.out.println("true || ... :");
        boolean r2 = check("left", true) || check("right", false);

        int count = 0;
        if (count != 0 && 100 / count > 5) {        // safe: 100/0 never runs
            System.out.println("big average");
        }
        String name = null;
        if (name != null && name.length() > 3) {    // safe: no NullPointerException
            System.out.println("long name");
        }
        System.out.println("done: " + r1 + " " + r2);
    }
}
```

OUTPUT

```
false && ... :
  evaluating left
true || ... :
  evaluating left
done: false true
```

NOTE

The single `&` and `|` also work on booleans but always evaluate **both** sides (no short-circuit). Use `&&` and `||` for conditions.

4.6.3 Truth tables

A	B	A && B	A || B	!A	A ^ B (XOR)
false	false	false	false	true	false
false	true	false	true	true	true
true	false	false	true	false	true
true	true	**true**	true	false	false

AND is true only when **both** are true. **OR** is false only when **both** are false. **XOR** is true when they are **different**.

4.7 Bitwise operators

Bitwise operators work on the individual **bits** (0s and 1s) of integer values. They are used in low-level code, flags and permissions, graphics, hashing (HashMap uses them) and fast arithmetic tricks.

4.7.1 & AND, | OR, ^ XOR, ~ complement



12 AND 10 IN BINARY, COMBINED BIT BY BIT

Operator	Rule for each bit	12 op 10
`&` AND	1 only if both bits are 1	8
`|` OR	1 if either bit is 1	14
`^` XOR	1 if the bits differ	6
`~` NOT	flips every bit	~12 = -13

BITWISE OPERATORS Bitwise.java

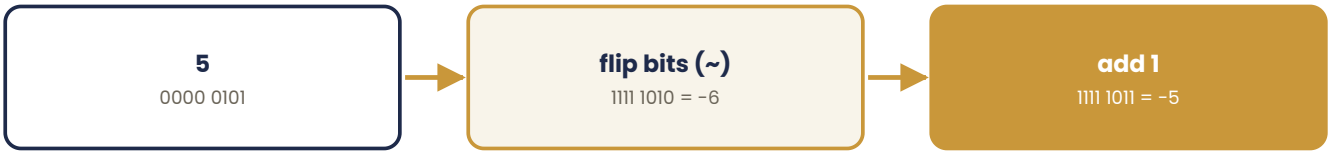
```
public class Bitwise {
    public static void main(String[] args) {
        int a = 12, b = 10;
        System.out.println("a & b = " + (a & b));
        System.out.println("a | b = " + (a | b));
        System.out.println("a ^ b = " + (a ^ b));
        System.out.println("~a    = " + (~a));
        System.out.println("12 in binary = " + Integer.toBinaryString(a));
        System.out.println("10 in binary = " + Integer.toBinaryString(b));
    }
}
```

OUTPUT

```
a & b = 8
a | b = 14
a ^ b = 6
~a    = -13
12 in binary = 1100
10 in binary = 1010
```

4.7.2 Binary representation and two's complement

An int is stored as 32 bits. The leftmost bit is the **sign bit**: 0 for positive, 1 for negative. Negative numbers use **two's complement**: to get -x, flip all the bits of x and add 1. That is why `~x == -x - 1`.



TWO'S COMPLEMENT OF 5 (SHOWN WITH 8 BITS)

NEGATIVE NUMBERS IN BINARY

TwosComplement.java

```java
public class TwosComplement {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(" 5 = " + String.format("%32s", Integer.toBinaryString(x)).replace(' ', '0'));
        System.out.println("-5 = " + Integer.toBinaryString(-x));
        System.out.println("~5 = " + (~x) + "  (= -5 - 1)");
        System.out.println("Integer.MAX_VALUE + 1 = " + (Integer.MAX_VALUE + 1));
    }
}
```

OUTPUT

```
 5 = 00000000000000000000000000000101
-5 = 11111111111111111111111111111011
~5 = -6  (= -5 - 1)
Integer.MAX_VALUE + 1 = -2147483648
```

4.7.3 Common tricks (even/odd check, swapping, power of two)

CLASSIC BIT TRICKS

BitTricks.java

```java
public class BitTricks {
    public static void main(String[] args) {
        // 1. even / odd: the last bit is 1 for odd numbers
        for (int n : new int[] {6, 7, -3}) {
            System.out.println(n + " is " + ((n & 1) == 0 ? "even" : "odd"));
        }
        // 2. swap two numbers with XOR (no third variable)
        int a = 15, b = 27;
        a = a ^ b;  b = a ^ b;  a = a ^ b;
        System.out.println("after swap: a=" + a + " b=" + b);
        // 3. power of two: exactly one bit set, so n & (n - 1) == 0
        for (int n : new int[] {16, 18, 64}) {
            boolean pow2 = n > 0 && (n & (n - 1)) == 0;
            System.out.println(n + " power of two? " + pow2);
        }
        // 4. count set bits
        System.out.println("bits set in 29: " + Integer.bitCount(29));
    }
}
```

OUTPUT

```
6 is even
7 is odd
-3 is odd
after swap: a=27 b=15
16 power of two? true
18 power of two? false
64 power of two? true
bits set in 29: 4
```

4.8 Shift operators

Shift operators move all bits left or right by a number of positions.

4.8.1 Left shift <<

`x << n` moves the bits left by n places and fills with 0s on the right. Each shift left **multiplies by 2** (as long as no bits fall off): `5 << 3` = 5 × 8 = 40.

4.8.2 Signed right shift >>

`x >> n` moves bits right and copies the **sign bit** into the empty places, so negative numbers stay negative. Each shift right **divides by 2** (rounding down): `40 >> 2` = 10, `-16 >> 2` = -4.

4.8.3 Unsigned right shift >>>

`x >>> n` always fills with **0s**, ignoring the sign. For negative numbers this produces a large positive number. HashMap uses `h ^ (h >>> 16)` to mix bits.



SHIFTING BITS LEFT AND RIGHT (GOLD = NEW ZERO BITS)

THE THREE SHIFT OPERATORS

Shifts.java

```
public class Shifts {
    public static void main(String[] args) {
        int x = 20;
        System.out.println("20 << 1  = " + (x << 1));
        System.out.println("20 << 3  = " + (x << 3));
        System.out.println("20 >> 2  = " + (x >> 2));
        int n = -16;
        System.out.println("-16 >> 2  = " + (n >> 2));
        System.out.println("-16 >>> 28 = " + (n >>> 28));
        System.out.println("-1 >>> 1  = " + (-1 >>> 1) + " (Integer.MAX_VALUE)");
    }
}
```

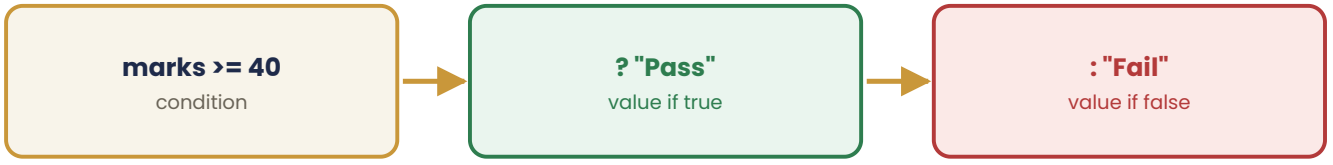
OUTPUT

```
20 << 1  = 40
20 << 3  = 160
20 >> 2  = 5
-16 >> 2  = -4
-16 >>> 28 = 15
-1 >>> 1  = 2147483647 (Integer.MAX_VALUE)
```

4.9 Ternary operator

4.9.1 condition ? a : b

The ternary (conditional) operator is a compact if-else that **produces a value**: if the condition is true the result is `a`, otherwise `b`.



READING THE TERNARY OPERATOR

CHOOSING A VALUE WITH ?: Ternary.java

```
public class Ternary {
    public static void main(String[] args) {
        int marks = 72;
        String result = marks >= 40 ? "Pass" : "Fail";
        System.out.println("Result: " + result);

        int a = 14, b = 9;
        int max = (a > b) ? a : b;
        System.out.println("max = " + max);

        int items = 1;
        System.out.println("You have " + items + (items == 1 ? " item" : " items"));
    }
}
```

OUTPUT

```
Result: Pass
max = 14
You have 1 item
```

4.9.2 Nested ternary and readability

A ternary can contain another ternary, which is handy for short multi-way choices but quickly becomes hard to read. Format it on several lines, or switch to if-else or a switch expression.

A READABLE NESTED TERNARY NestedTernary.java

```
public class NestedTernary {
    public static void main(String[] args) {
        int marks = 81;
        String grade = marks >= 90 ? "A+"
                     : marks >= 75 ? "A"
                     : marks >= 60 ? "B"
                     : marks >= 40 ? "C"
                     : "F";
        System.out.println("Grade: " + grade);
    }
}
```

OUTPUT

```
Grade: A
```

4.10 instanceof operator

4.10.1 Type checking

`obj instanceof Type` returns true if the object referred to by `obj` is of that type (or a subclass of it). It returns false for `null`. It is used before a cast to avoid a `ClassCastException`.

4.10.2 Pattern matching for instanceof (Java 16+)

Modern Java lets you test and cast in one step: `if (obj instanceof String s)` declares a variable `s` of type String that is available when the test succeeds.

INSTANCEOF WITH AND WITHOUT PATTERN MATCHING

InstanceofDemo.java

```
public class InstanceofDemo {
    public static void main(String[] args) {
        Object[] things = {"Learn Being Forward", 42, 3.75, null};
        for (Object o : things) {
            // old style: test, then cast
            if (o instanceof String) {
                String s = (String) o;
                System.out.println("String of length " + s.length());
            }
            // Java 16+: pattern matching declares the variable for you
            if (o instanceof Integer n) {
                System.out.println("Integer, doubled = " + (n * 2));
            } else if (o instanceof Double d && d > 3) {
                System.out.println("Double greater than 3: " + d);
            } else if (o == null) {
                System.out.println("null instanceof Object? " + (o instanceof Object));
            }
        }
    }
}
```

OUTPUT

```
String of length 19
Integer, doubled = 84
Double greater than 3: 3.75
null instanceof Object? false
```

4.11 Operator precedence and associativity

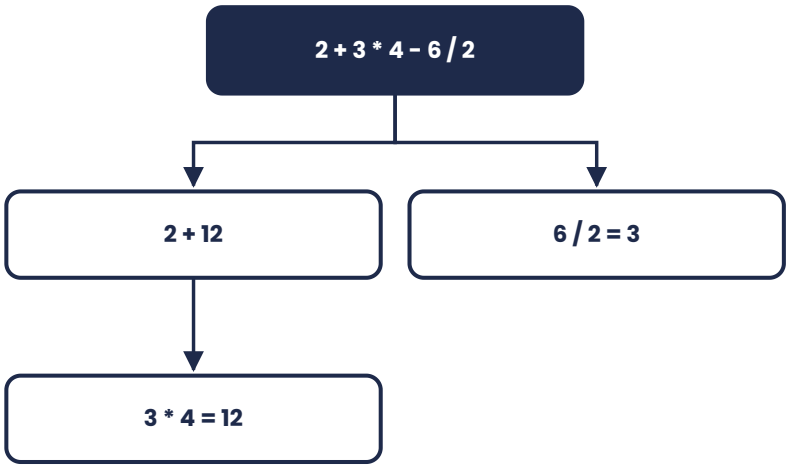
When an expression has several operators, **precedence** decides which is applied first (like BODMAS in maths). When two operators have the same precedence, **associativity** decides the direction.

4.11.1 Precedence table

Level (high → low)	Operators	Associativity
1 · postfix	`x++ x--`	left to right
2 · unary	`++x --x +x -x ~ ! (cast)`	right to left
3 · multiplicative	`* / %`	left to right
4 · additive	`+ -`	left to right
5 · shift	`<< >> >>>`	left to right
6 · relational	`< > <= >= instanceof`	left to right
7 · equality	`== !=`	left to right
8 · bitwise AND	`&`	left to right
9 · bitwise XOR	`^`	left to right
10 · bitwise OR	`|`	left to right
11 · logical AND	`&&`	left to right
12 · logical OR	`||`	left to right
13 · ternary	`? :`	right to left
14 · assignment	`= += -= *= /= %= …`	right to left

4.11.2 Left-to-right vs right-to-left associativity

Most binary operators are **left-associative**: `100 / 10 / 2` means `(100 / 10) / 2` = 5. Assignment, unary operators and the ternary are **right-associative**: `a = b = 5` means `a = (b = 5)`.



* AND / ARE DONE BEFORE + AND -, THEN LEFT TO RIGHT: 14 - 3 = 11

PRECEDENCE AND ASSOCIATIVITY IN ACTION

Precedence.java

```
public class Precedence {
    public static void main(String[] args) {
        System.out.println(2 + 3 * 4 - 6 / 2);      // 11
        System.out.println((2 + 3) * (4 - 6) / 2);  // -5
        System.out.println(100 / 10 / 2);           // 5  (left to right)
        System.out.println(2 * 3 % 4);              // 2  (same level: left to right)
        int a, b;
        a = b = 4;                                  // right to left
        System.out.println(a + b * 2 > 10 && a != b);   // 12 > 10 && false -> false
        System.out.println(-2 * -3 + ~0);           // unary first: 6 + (-1) = 5
    }
}
```

OUTPUT

```
11
-5
5
2
false
5
```

4.11.3 Using parentheses for clarity

Parentheses override precedence and, just as importantly, make your intention obvious. Even when they are not required, add them whenever a reader might hesitate: `(a > b) && (c < d)`, `(total * rate) / 100`.

4.12 The Math class

4.12.1 Math.abs, pow, sqrt, max, min, round, ceil, floor, random

`java.lang.Math` provides common mathematical functions as **static** methods, so you call them on the class: `Math.sqrt(16)`. No import is needed.

Method	Returns	Example → result
`Math.abs(x)`	absolute value	`Math.abs(-7.5)` → 7.5
`Math.pow(a, b)`	a to the power b (double)	`Math.pow(2, 10)` → 1024.0
`Math.sqrt(x)`	square root	`Math.sqrt(144)` → 12.0

Method	Returns	Example → result
`Math.cbrt(x)`	cube root	`Math.cbrt(27)` → 3.0
`Math.max(a, b)` / `min`	larger / smaller	`Math.max(4, 9)` → 9
`Math.round(x)`	nearest whole number (long)	`Math.round(2.5)` → 3
`Math.ceil(x)`	round up (double)	`Math.ceil(4.1)` → 5.0
`Math.floor(x)`	round down (double)	`Math.floor(4.9)` → 4.0
`Math.random()`	random double in [0.0, 1.0)	0.7311…

THE MOST USEFUL MATH METHODS

MathDemo.java

```
public class MathDemo {
    public static void main(String[] args) {
        System.out.println("abs(-7.5)   = " + Math.abs(-7.5));
        System.out.println("pow(2, 10)  = " + Math.pow(2, 10));
        System.out.println("sqrt(144)   = " + Math.sqrt(144));
        System.out.println("max(4, 9)   = " + Math.max(4, 9));
        System.out.println("min(4, 9)   = " + Math.min(4, 9));
        System.out.println("round(2.5)  = " + Math.round(2.5) + ", round(-2.5) = " + Math.round(-2.5));
        System.out.println("ceil(4.1)   = " + Math.ceil(4.1) + ", floor(4.9) = " + Math.floor(4.9));
        System.out.println("ceil(-4.1)  = " + Math.ceil(-4.1) + ", floor(-4.1) = " + Math.floor(-4.1));
        // random whole number between 1 and 6 (a dice roll)
        int dice = (int) (Math.random() * 6) + 1;
        System.out.println("dice is between 1 and 6: " + (dice >= 1 && dice <= 6));
        // area of a circle
        double r = 7;
        System.out.printf("area = %.2f%n", Math.PI * Math.pow(r, 2));
    }
}
```

OUTPUT

```
abs(-7.5)   = 7.5
pow(2, 10)  = 1024.0
sqrt(144)   = 12.0
max(4, 9)   = 9
min(4, 9)   = 4
round(2.5)  = 3, round(-2.5) = -2
ceil(4.1)   = 5.0, floor(4.9) = 4.0
ceil(-4.1)  = -4.0, floor(-4.1) = -5.0
dice is between 1 and 6: true
area = 153.94
```

TIP

To get a random int from `min` to `max` inclusive: `(int) (Math.random() * (max - min + 1)) + min`.

PRACTICE CHECKPOINT

Write a **simple calculator**, an **even/odd checker using bitwise AND**, and a **swap of two numbers without a third variable**.

SAMPLE SOLUTION

OperatorsCheckpoint.java

```java
public class OperatorsCheckpoint {
    public static void main(String[] args) {
        // 1. simple calculator
        double a = 24, b = 5;
        char op = '/';
        double result = op == '+' ? a + b
                      : op == '-' ? a - b
                      : op == '*' ? a * b
                      : op == '/' ? a / b
                      : Double.NaN;
        System.out.println(a + " " + op + " " + b + " = " + result);

        // 2. even / odd with bitwise AND
        int n = 37;
        System.out.println(n + " is " + ((n & 1) == 0 ? "even" : "odd"));

        // 3. swap without a third variable (arithmetic version)
        int x = 8, y = 13;
        x = x + y;   // 21
        y = x - y;   // 8
        x = x - y;   // 13
        System.out.println("x = " + x + ", y = " + y);
    }
}
```

OUTPUT

```
24.0 / 5.0 = 4.8
37 is odd
x = 13, y = 8
```

MODULE 05 · BEGINNER

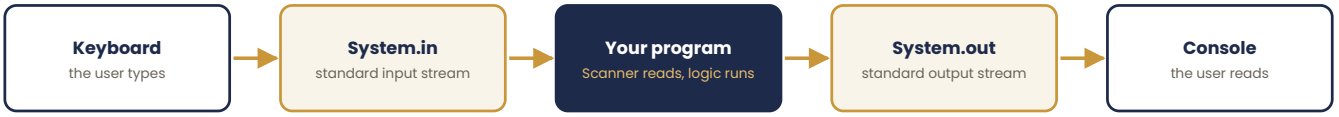
Input and output basics

Minimum duration **1 day** | Level **Beginner**

Programs become interactive here: printing formatted output and reading input from the user.

IN THIS MODULE

5.1 Output
5.2 Input with Scanner
5.3 Other ways to read input
5.4 Parsing strings to numbers



STANDARD INPUT AND OUTPUT: DATA FLOWS IN THROUGH SYSTEM.IN AND OUT THROUGH SYSTEM.OUT

5.1 Output

5.1.1 System.out.print() vs println()

`print()` writes its argument and leaves the cursor **on the same line**. `println()` writes its argument and then moves to a **new line**. `println()` with no argument just prints an empty line.

PRINT KEEPS THE LINE, PRINTLN ENDS IT

PrintVsPrintln.java

```
public class PrintVsPrintln {
    public static void main(String[] args) {
        System.out.print("Learn ");
        System.out.print("Being ");
        System.out.println("Forward");     // ends the line
        System.out.println("Java");
        System.out.println();              // blank line
        System.out.print("A\nB\n");        // \n also starts a new line
        for (int i = 1; i <= 5; i++) {
            System.out.print(i + " ");     // all on one line
        }
        System.out.println();
    }
}
```

OUTPUT

```
Learn Being Forward
Java

A
B
1 2 3 4 5
```

5.1.2 System.out.printf() and format specifiers (%d, %f, %.2f, %s, %c, %b, %n)

`printf` ("print formatted") takes a **format string** with placeholders, followed by the values to put into them. Each placeholder starts with `%` and says how to format one value.

Specifier	For	Example	Output
`%d`	integers (int, long)	`printf("%d", 42)`	42
`%f`	decimals (6 places by default)	`printf("%f", 3.14159)`	3.141590

Specifier	For	Example	Output
`%.2f`	decimals with 2 places (rounded)	`printf("%.2f", 3.14159)`	3.14
`%s`	String (or any object)	`printf("%s", "Java")`	Java
`%c`	char	`printf("%c", 'A')`	A
`%b`	boolean	`printf("%b", true)`	true
`%e`	scientific notation	`printf("%e", 12345.6)`	1.234560e+04
`%x`	hexadecimal	`printf("%x", 255)`	ff
`%n`	new line (platform-correct)		
`%%`	a literal % sign	`printf("90%%")`	90%

FORMATTING DIFFERENT KINDS OF VALUES

PrintfDemo.java

```
public class PrintfDemo {
    public static void main(String[] args) {
        String name = "Priya";
        int marks = 487;
        double percentage = 487 / 5.0;
        char grade = 'A';
        boolean passed = true;
        System.out.printf("Name: %s%n", name);
        System.out.printf("Marks: %d out of %d%n", marks, 500);
        System.out.printf("Percentage: %.2f%%%n", percentage);
        System.out.printf("Grade: %c, Passed: %b%n", grade, passed);
        System.out.printf("Default %%f: %f, scientific: %e%n", percentage, 123456.789);
    }
}
```

OUTPUT

```
Name: Priya
Marks: 487 out of 500
Percentage: 97.40%
Grade: A, Passed: true
Default %f: 97.400000, scientific: 1.234568e+05
```

5.1.3 Width, alignment and padding in printf

A number between `%` and the letter sets the **minimum width**. By default values are right-aligned; a `-` flag left-aligns them; a `0` flag pads numbers with zeros; a `,` flag adds thousands separators. This makes neat tables.

Format	Value	Result (· = space)
`%5d`	42	···42
`%-5d|`	42	42···|
`%05d`	42	00042
`%8.2f`	3.14159	····3.14
`%,d`	1500000	1,500,000
`%-10s|`	"Java"	Java·····|

A NEATLY ALIGNED TABLE WITH PRINTF

PrintfTable.java

```
public class PrintfTable {
    public static void main(String[] args) {
        String[] names = {"Aarav", "Diya", "Karthik"};
        int[] marks = {88, 92, 7};
        double[] fees = {15000, 8500.5, 120000};
        System.out.printf("%-10s %6s %12s%n", "NAME", "MARKS", "FEE (Rs)");
        System.out.println("-".repeat(30));
        for (int i = 0; i < names.length; i++) {
            System.out.printf("%-10s %6d %,12.2f%n", names[i], marks[i], fees[i]);
        }
        System.out.printf("Roll no: %05d%n", 42);
    }
}
```

OUTPUT

```
NAME        MARKS     FEE (Rs)
------------------------------
Aarav          88    15,000.00
Diya           92     8,500.50
Karthik         7   120,000.00
Roll no: 00042
```

5.1.4 String.format() and formatted()

`String.format(format, values…)` works exactly like printf but **returns** the formatted String instead of printing it, so you can store it or build bigger messages. Since Java 15 you can also call `"format".formatted(values…)` directly on the format string.

FORMATTING INTO A STRING

FormatDemo.java

```
public class FormatDemo {
    public static void main(String[] args) {
        double amount = 1234.5;
        String s1 = String.format("Total: Rs.%,.2f", amount);
        String s2 = "Student %s scored %d marks".formatted("Rohan", 81);
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s1.length() + " characters");
    }
}
```

OUTPUT

```
Total: Rs.1,234.50
Student Rohan scored 81 marks
18 characters
```

5.1.5 System.err for error output

`System.err` is a separate output stream meant for **error messages**. It prints to the console too (IDEs often show it in red), but it can be redirected separately from normal output: `java App > out.txt 2> errors.txt`.

NORMAL OUTPUT VS ERROR OUTPUT

ErrDemo.java

```
public class ErrDemo {
    public static void main(String[] args) {
        System.out.println("Processing file...");
        System.err.println("Warning: file students.csv not found, using defaults");
        System.out.println("Done.");
    }
}
```

OUTPUT

```
Processing file...
Warning: file students.csv not found, using defaults
Done.
```

5.2 Input with Scanner

The easiest way to read keyboard input is the `Scanner` class. It splits the input into **tokens** (pieces separated by spaces or new lines) and converts them to the type you ask for.

5.2.1 Importing java.util.Scanner

Scanner lives in the `java.util` package, so the file must start with `import java.util.Scanner;` (or `import java.util.*;`).

5.2.2 Creating a Scanner on System.in

`Scanner sc = new Scanner(System.in);` creates a Scanner connected to the keyboard (standard input). Create **one** Scanner and reuse it for all input in the program.

5.2.3 Reading methods: nextInt(), nextDouble(), nextBoolean(), next(), nextLine()

Method	Reads	Example input → value
`nextInt()`	the next token as an int	`21` → 21
`nextLong()`	a long	`9848011111`
`nextDouble()`	a double	`72.5` → 72.5
`nextBoolean()`	true / false	`true`
`next()`	one word (up to a space)	`Aarav Sharma` → "Aarav"
`nextLine()`	the rest of the current line	`Aarav Sharma` → "Aarav Sharma"

READING DIFFERENT TYPES WITH SCANNER — ReadInput.java

```
import java.util.Scanner;

public class ReadInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your full name: ");
        String name = sc.nextLine();
        System.out.print("Enter your age: ");
        int age = sc.nextInt();
        System.out.print("Enter your height in metres: ");
        double height = sc.nextDouble();
        System.out.print("Are you a student (true/false)? ");
        boolean student = sc.nextBoolean();
        System.out.print("Enter your city: ");
        String city = sc.next();
        System.out.println();
        System.out.println(name + ", " + age + " years, " + height + " m, student=" + student + ", " + city);
        sc.close();
    }
}
```

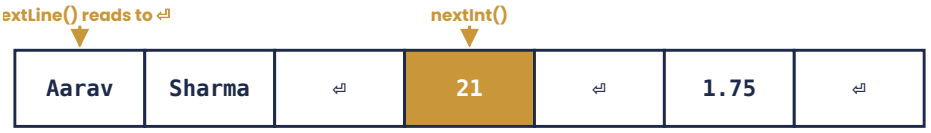
CONSOLE (user input in gold)

```
Enter your full name: Aarav Sharma
Enter your age: 21
Enter your height in metres: 1.75
Are you a student (true/false)? true
Enter your city: Hyderabad

Aarav Sharma, 21 years, 1.75 m, student=true, Hyderabad
```

NOTE

In console boxes like the one above, the text in **gold** is what the user typed; everything else was printed by the program.



THE INPUT IS A STREAM OF TOKENS AND LINE BREAKS (⏎); EACH METHOD CONSUMES PART OF IT

5.2.4 Reading a char: next().charAt(0)

Scanner has no `nextChar()`. Read a word with `next()` and take its first character with `charAt(0)`.

READING A SINGLE CHARACTER — ReadChar.java

```
import java.util.Scanner;

public class ReadChar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your grade: ");
        char grade = sc.next().charAt(0);
        System.out.println("Grade entered: " + grade + " (code " + (int) grade + ")");
    }
}
```

CONSOLE (user input in gold)

```
Enter your grade: B
Grade entered: B (code 66)
```

5.2.5 The nextInt() + nextLine() newline problem and its fix

`nextInt()` reads the number but **leaves the Enter key (\n) in the input**. A following `nextLine()` immediately reads that leftover empty line and returns `""`, so the user never gets to type the name.

THE BUG: NAME COMES OUT EMPTY — NewlineProblem.java

```
import java.util.Scanner;

public class NewlineProblem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Roll number: ");
        int roll = sc.nextInt();
        System.out.print("Name: ");
        String name = sc.nextLine();        // reads the leftover newline!
        System.out.println("roll=" + roll + ", name='" + name + "'");
    }
}
```

CONSOLE (user input in gold)

```
Roll number: 101
Name: roll=101, name=''
```



AFTER NEXTINT(), THE CURSOR IS JUST BEFORE ⏎, SO NEXTLINE() RETURNS ""

Fix: call an extra `sc.nextLine()` after `nextInt()` to consume the rest of the line, or read every line with `nextLine()` and convert it with `Integer.parseInt()`.

TWO WAYS TO FIX IT

NewlineFixed.java

```
import java.util.Scanner;

public class NewlineFixed {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Roll number: ");
        int roll = sc.nextInt();
        sc.nextLine();                      // fix 1: throw away the rest of the line
        System.out.print("Name: ");
        String name = sc.nextLine();
        System.out.print("Marks: ");
        int marks = Integer.parseInt(sc.nextLine().trim());   // fix 2: read a line, then parse
        System.out.println("roll=" + roll + ", name='" + name + "', marks=" + marks);
    }
}
```

CONSOLE (user input in gold)

```
Roll number: 101
Name: Diya Patel
Marks: 93
roll=101, name='Diya Patel', marks=93
```

5.2.6 hasNextInt() for input validation

If the user types `abc` when `nextInt()` expects a number, the program crashes with an `InputMismatchException`. `hasNextInt()` looks ahead **without consuming** and returns true only if the next token is a valid int, so you can ask again.

KEEP ASKING UNTIL THE INPUT IS VALID

ValidateInput.java

```
import java.util.Scanner;

public class ValidateInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int age;
        while (true) {
            System.out.print("Enter age: ");
            if (sc.hasNextInt()) {
                age = sc.nextInt();
                if (age > 0 && age < 120) break;
                System.out.println("  Age must be between 1 and 119.");
            } else {
                System.out.println("  '" + sc.next() + "' is not a number. Try again.");
            }
        }
        System.out.println("Valid age: " + age);
    }
}
```

CONSOLE (user input in gold)

```
Enter age: twenty
  'twenty' is not a number. Try again.
Enter age: -5
  Age must be between 1 and 119.
Enter age: 21
Valid age: 21
```

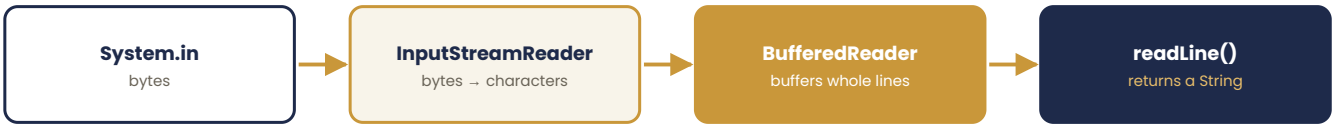
5.2.7 Closing the Scanner

When you are done, call `sc.close()` to release the underlying stream. Be careful: closing a Scanner on `System.in` also **closes System.in**, so no other part of the program can read the keyboard afterwards. In small programs, close it once at the very end (or use try-with-resources, Module 12).

5.3 Other ways to read input

5.3.1 BufferedReader with InputStreamReader (faster input)

`BufferedReader` reads whole lines into a buffer and is much faster than Scanner for large inputs (competitive programming, big files). It only reads text lines, so you convert numbers yourself.



WRAPPING SYSTEM.IN TO READ LINES EFFICIENTLY

BUFFEREDREADER + SPLIT

FastInput.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class FastInput {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.print("How many numbers? ");
        int n = Integer.parseInt(br.readLine().trim());
        System.out.print("Enter them on one line: ");
        String[] parts = br.readLine().trim().split("\\s+");
        long sum = 0;
        for (int i = 0; i < n; i++) sum += Long.parseLong(parts[i]);
        System.out.println("Sum = " + sum);
    }
}
```

CONSOLE (user input in gold)

```
How many numbers? 5
Enter them on one line: 10 20 30 40 50
Sum = 150
```

	Scanner	BufferedReader
Speed	slower (parses with regex)	**much faster**
Reads	tokens of any type	whole lines (Strings)
Conversion	built in (nextInt…)	manual (Integer.parseInt)
Exceptions	unchecked	`IOException` must be handled
Best for	beginners, small inputs	large inputs, files

5.3.2 Command-line arguments via args[]

Values typed after the class name arrive in `main`'s `args` array as Strings (Module 2). They are good for settings like file names that do not change while the program runs.

ADDING NUMBERS PASSED ON THE COMMAND LINE SumArgs.java

```java
public class SumArgs {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Usage: java SumArgs <numbers...>");
            return;
        }
        int sum = 0;
        for (String a : args) sum += Integer.parseInt(a);
        System.out.println("Sum of " + args.length + " numbers = " + sum);
    }
}
```

OUTPUT

```
Usage: java SumArgs <numbers...>
```

Terminal

```
$ java SumArgs 10 20 30
Sum of 3 numbers = 60
```

5.3.3 Console class

`System.console()` returns a `Console` object for interactive terminals. Its special feature is `readPassword()`, which reads text **without showing it** on the screen. It returns `null` when there is no real console (inside many IDEs, or when input is redirected), so always check for null.

READING A HIDDEN PASSWORD ConsoleDemo.java

```java
import java.io.Console;

public class ConsoleDemo {
    public static void main(String[] args) {
        Console console = System.console();
        if (console == null) {
            System.out.println("No console available (running inside an IDE or with redirected input).");
            return;
        }
        String user = console.readLine("Username: ");
        char[] password = console.readPassword("Password: ");   // typing is hidden
        console.printf("Welcome %s, your password has %d characters%n", user, password.length);
    }
}
```

OUTPUT

```
No console available (running inside an IDE or with redirected input).
```

5.4 Parsing strings to numbers

5.4.1 Integer.parseInt(), Double.parseDouble()

Input from files, command-line arguments, BufferedReader and web forms always arrives as **text**. The wrapper classes convert text to numbers.

Method	Converts	Example
`Integer.parseInt(s)`	String → int	`Integer.parseInt("42")` → 42
`Long.parseLong(s)`	String → long	`Long.parseLong("9848011111")`
`Double.parseDouble(s)`	String → double	`Double.parseDouble("72.5")` → 72.5
`Boolean.parseBoolean(s)`	String → boolean	`"TRUE"` → true, anything else → false
`Integer.parseInt(s, 2)`	binary text → int	`"1010"` → 10

Method	Converts	Example
`String.valueOf(x)`	number → String	`String.valueOf(42)` → "42"

5.4.2 NumberFormatException on bad input

If the text is not a valid number (`"12a"`, `"4.5"` for parseInt, an empty string), the method throws a **NumberFormatException**. Catch it with try-catch (Module 12) to show a friendly message.

PARSING SAFELY

ParseDemo.java

```
public class ParseDemo {
    public static void main(String[] args) {
        String[] inputs = {"42", " 17 ", "4.5", "abc", ""};
        for (String s : inputs) {
            try {
                int value = Integer.parseInt(s.trim());
                System.out.println("'" + s + "' -> " + value);
            } catch (NumberFormatException e) {
                System.out.println("'" + s + "' is not a whole number: " + e.getMessage());
            }
        }
        double d = Double.parseDouble("4.5");
        System.out.println("parsed double: " + (d * 2));
    }
}
```

OUTPUT

```
'42' -> 42
' 17 ' -> 17
'4.5' is not a whole number: For input string: "4.5"
'abc' is not a whole number: For input string: "abc"
'' is not a whole number: For input string: ""
parsed double: 9.0
```

PRACTICE CHECKPOINT

A program that reads a student's **name** and **three marks**, then prints the **total** and **average** to two decimal places.

SAMPLE SOLUTION

StudentReport.java

```
import java.util.Scanner;

public class StudentReport {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Student name: ");
        String name = sc.nextLine();
        System.out.print("Enter three marks: ");
        int m1 = sc.nextInt(), m2 = sc.nextInt(), m3 = sc.nextInt();
        int total = m1 + m2 + m3;
        double average = total / 3.0;
        System.out.println();
        System.out.printf("%-10s: %s%n", "Name", name);
        System.out.printf("%-10s: %d%n", "Total", total);
        System.out.printf("%-10s: %.2f%n", "Average", average);
        sc.close();
    }
}
```

CONSOLE (user input in gold)

```
Student name: Sneha Iyer
Enter three marks: 78 91 84

Name      : Sneha Iyer
Total     : 253
Average   : 84.33
```

MODULE 06 · BEGINNER

Control flow

Minimum duration **4 days** | Level **Beginner**

Control flow decides which statements run and how many times: decisions, loops and jumps.

IN THIS MODULE

6.1 Decision-making statements
6.2 switch
6.3 Loops
6.4 Nested loops
6.5 Jump statements
6.6 Classic loop problems

By default Java runs statements one after another, top to bottom. **Control-flow statements** change that order: they **choose** between paths (if, switch), **repeat** work (for, while, do-while) or **jump** (break, continue, return).

Sequence
statements run one after another

Selection
choose a path: `if`, `switch`

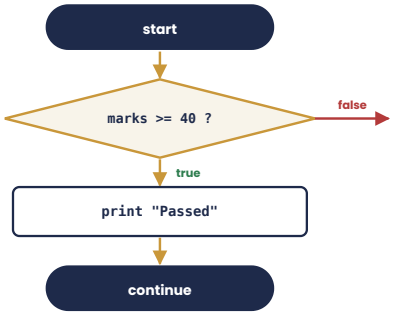
Iteration
repeat: `for`, `while`, `do-while`

Jump
leave early: `break`, `continue`, `return`

6.1 Decision-making statements

6.1.1 if

`if (condition) { … }` runs the block only when the condition is **true**. The condition must be a boolean expression.



IF: THE BLOCK RUNS ONLY ON THE TRUE PATH

A SIMPLE IF — IfDemo.java

```
public class IfDemo {
    public static void main(String[] args) {
        int marks = 72;
        if (marks >= 40) {
            System.out.println("Passed");
        }
        if (marks >= 90) {
            System.out.println("Outstanding!");   // skipped: condition is false
        }
        System.out.println("Result processed");
    }
}
```

OUTPUT

```
Passed
Result processed
```

6.1.2 if-else

`else` provides the block to run when the condition is **false**. Exactly one of the two blocks runs.

TWO PATHS: IF AND ELSE IfElse.java

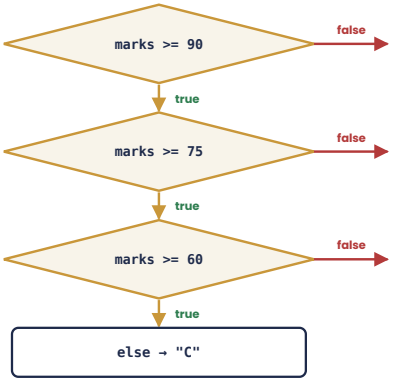
```
public class IfElse {
    public static void main(String[] args) {
        int age = 16;
        if (age >= 18) {
            System.out.println("Eligible to vote");
        } else {
            System.out.println("Not eligible yet, wait " + (18 - age) + " years");
        }
        int n = 7;
        if (n % 2 == 0) System.out.println(n + " is even");
        else            System.out.println(n + " is odd");
    }
}
```

OUTPUT

```
Not eligible yet, wait 2 years
7 is odd
```

6.1.3 else-if ladder

To choose between **many** options, chain conditions with `else if`. Java checks them from top to bottom and runs the **first** block whose condition is true; the rest are skipped. The final `else` catches everything else.



AN ELSE-IF LADDER TESTS CONDITIONS IN ORDER (FALSE MOVES DOWN TO THE NEXT TEST)

GRADING WITH AN ELSE-IF LADDER GradeLadder.java

```
public class GradeLadder {
    public static void main(String[] args) {
        int[] all = {95, 81, 64, 47, 30};
        for (int marks : all) {
            String grade;
            if (marks >= 90)      grade = "A+";
            else if (marks >= 75) grade = "A";
            else if (marks >= 60) grade = "B";
            else if (marks >= 40) grade = "C";
            else                  grade = "Fail";
            System.out.println(marks + " -> " + grade);
        }
    }
}
```

OUTPUT

```
95 -> A+
81 -> A
64 -> B
47 -> C
30 -> Fail
```

NOTE

Order matters. If `marks >= 40` were tested first, every passing student would get "C".

6.1.4 Nested if

An if-statement can sit inside another. Use nesting when the second question only makes sense after the first one is answered.

DECISIONS INSIDE DECISIONS

NestedIf.java

```
public class NestedIf {
    public static void main(String[] args) {
        int age = 22;
        boolean hasLicence = false;
        if (age >= 18) {
            if (hasLicence) {
                System.out.println("You may drive.");
            } else {
                System.out.println("Adult, but apply for a licence first.");
            }
        } else {
            System.out.println("Too young to drive.");
        }
        // the largest of three numbers
        int a = 12, b = 45, c = 30;
        if (a > b) {
            if (a > c) System.out.println("max = " + a); else System.out.println("max = " + c);
        } else {
            if (b > c) System.out.println("max = " + b); else System.out.println("max = " + c);
        }
    }
}
```

OUTPUT

```
Adult, but apply for a licence first.
max = 45
```

6.1.5 Common mistakes (= vs ==, missing braces, dangling else)

Mistake	What happens	Fix
`if (x = 5)`	compile error for int (an assignment is not a boolean). With booleans, `if (done = true)` compiles and is **always true**!	use `==` to compare
missing braces	only the **first** statement belongs to the if; the second always runs	always use `{ }`
semicolon after if: `if (x > 0);`	the empty statement is the body; the block below always runs	remove the `;`
dangling else	an `else` pairs with the **nearest** unmatched if, regardless of indentation	use braces to show which if you mean

TWO CLASSIC TRAPS

IfMistakes.java

```java
public class IfMistakes {
    public static void main(String[] args) {
        int balance = 500;
        if (balance > 1000)
            System.out.println("Premium customer");
            System.out.println("Free gift sent!");     // NOT part of the if: always runs

        int x = 5, y = -1;
        if (x > 0)
            if (y > 0)
                System.out.println("both positive");
        else                                          // belongs to if (y > 0)!
            System.out.println("x is not positive?");  // printed although x IS positive
    }
}
```

OUTPUT

```
Free gift sent!
x is not positive?
```

6.2 switch

When one value must be compared against many fixed options, a `switch` is clearer than a long else-if ladder.

6.2.1 Classic switch with case, break and default

The value in parentheses is compared with each `case` label. Execution jumps to the matching case and runs until a `break`. `default` runs when no case matches.

A CLASSIC SWITCH

ClassicSwitch.java

```java
public class ClassicSwitch {
    public static void main(String[] args) {
        int day = 3;
        String name;
        switch (day) {
            case 1:
                name = "Monday";
                break;
            case 2:
                name = "Tuesday";
                break;
            case 3:
                name = "Wednesday";
                break;
            case 6:
            case 7:
                name = "Weekend";       // two labels share one block
                break;
            default:
                name = "Another weekday";
        }
        System.out.println("Day " + day + " is " + name);
    }
}
```

OUTPUT

```
Day 3 is Wednesday
```

6.2.2 Fall-through behaviour

Without `break`, execution **falls through** into the next case and keeps going. This is occasionally useful (several labels sharing code) but usually a bug.

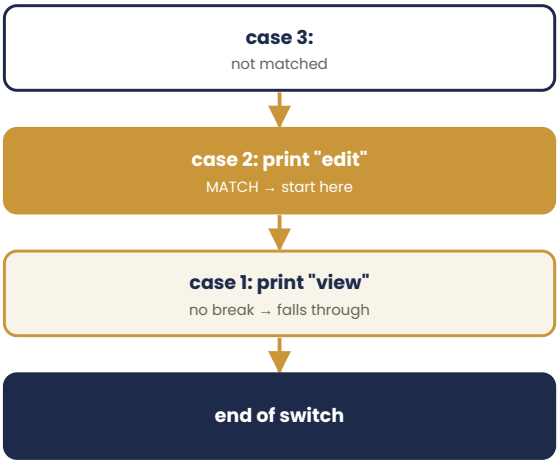
INTENTIONAL AND ACCIDENTAL FALL-THROUGH

FallThrough.java

```
public class FallThrough {
    public static void main(String[] args) {
        int level = 2;
        System.out.println("Access granted for level " + level + ":");
        switch (level) {
            case 3: System.out.println("  - delete records");
            case 2: System.out.println("  - edit records");
            case 1: System.out.println("  - view records");
        }
        System.out.println("Bug version, month 2:");
        int month = 2;
        switch (month) {
            case 1: System.out.println("  January");
            case 2: System.out.println("  February");     // matches here...
            case 3: System.out.println("  March");        // ...and falls through!
            default: System.out.println("  unknown");
        }
    }
}
```

OUTPUT

```
Access granted for level 2:
  - edit records
  - view records
Bug version, month 2:
  February
  March
  unknown
```



WITHOUT BREAK, EXECUTION CONTINUES INTO THE FOLLOWING CASES

6.2.3 Allowed types: int, char, String, enum, wrapper types

A classic switch works on `byte, short, char, int`, their wrappers (`Byte, Short, Character, Integer`), `String` (Java 7+) and `enum` types. It does **not** work on `long, float, double` or `boolean`. Case labels must be compile-time constants and unique. (Java 21 pattern matching extends switch to any type, see 6.2.6.)

SWITCH ON STRING, CHAR AND ENUM SwitchTypes.java

```
public class SwitchTypes {
    enum Size { SMALL, MEDIUM, LARGE }
    public static void main(String[] args) {
        String cmd = "stop";
        switch (cmd) {
            case "start": System.out.println("Starting"); break;
            case "stop":  System.out.println("Stopping"); break;
            default:      System.out.println("Unknown command");
        }
        char op = '*';
        switch (op) {
            case '+': System.out.println(6 + 3); break;
            case '*': System.out.println(6 * 3); break;
        }
        Size s = Size.MEDIUM;
        switch (s) {
            case SMALL:  System.out.println("Rs.99");  break;
            case MEDIUM: System.out.println("Rs.149"); break;
            case LARGE:  System.out.println("Rs.199"); break;
        }
    }
}
```

OUTPUT

```
Stopping
18
Rs.149
```

6.2.4 Enhanced switch with arrow -> (Java 14+)

The arrow form `case X ->` runs **only** the code on the right of the arrow: there is **no fall-through** and no `break` needed. Several labels can share one case, separated by commas.

ARROW LABELS: NO BREAK, NO FALL-THROUGH ArrowSwitch.java

```
public class ArrowSwitch {
    public static void main(String[] args) {
        int day = 6;
        switch (day) {
            case 1, 2, 3, 4, 5 -> System.out.println("Weekday: go to class");
            case 6, 7          -> System.out.println("Weekend: revise and practise");
            default            -> System.out.println("Invalid day");
        }
        String fruit = "mango";
        switch (fruit) {
            case "apple" -> System.out.println("Red");
            case "mango" -> {                    // a block for several statements
                System.out.println("Yellow");
                System.out.println("King of fruits");
            }
            default -> System.out.println("Unknown colour");
        }
    }
}
```

OUTPUT

```
Weekend: revise and practise
Yellow
King of fruits
```

6.2.5 Switch expressions and yield

A switch can also **produce a value**, like the ternary operator. Every possible input must be covered (usually with `default`). In a block, `yield` gives the value of that case.

SWITCH AS AN EXPRESSION

SwitchExpression.java

```
public class SwitchExpression {
    public static void main(String[] args) {
        int month = 2, year = 2028;
        int days = switch (month) {
            case 1, 3, 5, 7, 8, 10, 12 -> 31;
            case 4, 6, 9, 11 -> 30;
            case 2 -> {
                boolean leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
                yield leap ? 29 : 28;           // yield returns the value from a block
            }
            default -> throw new IllegalArgumentException("bad month " + month);
        };
        System.out.println("Days in " + month + "/" + year + ": " + days);

        char grade = 'B';
        String msg = switch (grade) {
            case 'A' -> "Excellent";
            case 'B', 'C' -> "Good";
            default -> "Keep trying";
        };
        System.out.println(msg);
    }
}
```

OUTPUT

```
Days in 2/2028: 29
Good
```

6.2.6 Pattern matching in switch (Java 21+)

Since Java 21 a switch can test the **type** of a value and bind it to a variable, optionally with a `when` guard. It can also handle `null` explicitly.

SWITCHING ON THE TYPE OF AN OBJECT

PatternSwitch.java

```
public class PatternSwitch {
    static String describe(Object o) {
        return switch (o) {
            case null -> "nothing (null)";
            case Integer i when i > 100 -> "a big integer: " + i;
            case Integer i -> "an integer: " + i;
            case String s -> "text of length " + s.length();
            case int[] arr -> "an int array with " + arr.length + " elements";
            default -> "something else: " + o.getClass().getSimpleName();
        };
    }
    public static void main(String[] args) {
        Object[] values = {42, 500, "Java", new int[] {1, 2, 3}, 3.14, null};
        for (Object v : values) System.out.println(describe(v));
    }
}
```

OUTPUT

```
an integer: 42
a big integer: 500
text of length 4
an int array with 3 elements
something else: Double
nothing (null)
```

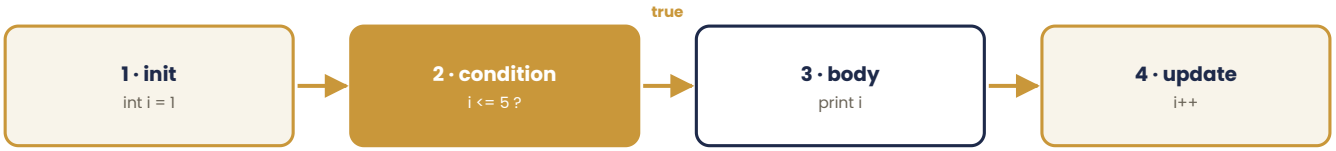
6.3 Loops

A **loop** repeats a block of code. Every loop has the same ingredients: a starting state, a **condition** that decides whether to continue, the **body**, and an **update** that moves towards stopping.

6.3.1 for loop

Use `for` when you know how many times to repeat, or when you are counting.

6.3.1.1 Initialization, condition, update



ORDER OF A FOR LOOP: 1 ONCE, THEN 2 → 3 → 4 → 2 … UNTIL THE CONDITION IS FALSE

COUNTING WITH FOR

ForLoop.java

```
public class ForLoop {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.print(i + " ");
        }
        System.out.println();
        // counting down by 2
        for (int i = 10; i > 0; i -= 2) {
            System.out.print(i + " ");
        }
        System.out.println();
        // multiplication table of 7
        for (int i = 1; i <= 10; i++) {
            System.out.println("7 x " + i + " = " + (7 * i));
        }
    }
}
```

OUTPUT

```
1 2 3 4 5
10 8 6 4 2
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Iteration	i before	i <= 5?	prints	i after i++
1	1	true	1	2
2	2	true	2	3
3	3	true	3	4
4	4	true	4	5
5	5	true	5	6
–	6	**false**	loop ends	

6.3.1.2 Multiple variables in a for loop

The initialisation and update parts can contain several comma-separated expressions (all variables must have the same type). A classic use is walking from both ends towards the middle.

TWO LOOP VARIABLES MOVING TOWARDS EACH OTHER TwoVariables.java

```
public class TwoVariables {
    public static void main(String[] args) {
        for (int i = 0, j = 10; i < j; i++, j--) {
            System.out.println("i=" + i + "  j=" + j);
        }
    }
}
```

OUTPUT

```
i=0  j=10
i=1  j=9
i=2  j=8
i=3  j=7
i=4  j=6
```

6.3.1.3 Infinite for loop

All three parts of a for loop are optional. `for (;;)` has no condition and therefore runs **forever**, until a `break` or `return` leaves it. Servers and game loops are built like this deliberately; elsewhere, an infinite loop is usually a bug (for example, forgetting the update).

LEAVING AN INFINITE LOOP WITH BREAK InfiniteFor.java

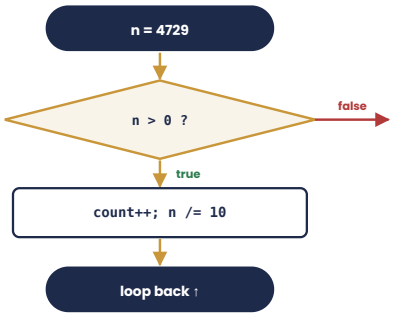
```
public class InfiniteFor {
    public static void main(String[] args) {
        int attempts = 0;
        for (;;) {                      // no init, no condition, no update
            attempts++;
            int dice = (attempts * 7) % 6 + 1;   // pretend dice roll
            System.out.println("roll " + attempts + ": " + dice);
            if (dice == 6) break;       // the only way out
        }
        System.out.println("Got a six after " + attempts + " rolls");
    }
}
```

OUTPUT

```
roll 1: 2
roll 2: 3
roll 3: 4
roll 4: 5
roll 5: 6
Got a six after 5 rolls
```

6.3.2 while loop

`while (condition) { … }` checks the condition **before** each repetition. Use it when you do not know in advance how many times the loop will run, for example "until the number becomes 0".



WHILE: TEST FIRST, THEN RUN THE BODY

WHILE: REPEAT UNTIL A CONDITION CHANGES

WhileLoop.java

```
public class WhileLoop {
    public static void main(String[] args) {
        int n = 4729, count = 0;
        while (n > 0) {
            n = n / 10;          // remove the last digit
            count++;
        }
        System.out.println("digits: " + count);

        // doubling money until it passes a target
        double amount = 1000;
        int years = 0;
        while (amount < 2000) {
            amount *= 1.08;      // 8% per year
            years++;
        }
        System.out.printf("Doubles after %d years (Rs.%.2f)%n", years, amount);
    }
}
```

OUTPUT

```
digits: 4
Doubles after 10 years (Rs.2158.92)
```

6.3.3 do-while loop (runs at least once)

`do { … } while (condition);` checks the condition **after** the body, so the body always runs **at least once**. It is ideal for menus and "ask until valid" input. Note the semicolon after the condition.

A MENU LOOP WITH DO-WHILE

DoWhileMenu.java

```
import java.util.Scanner;

public class DoWhileMenu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            System.out.println("1. Say hello   2. Show date   0. Exit");
            System.out.print("Choice: ");
            choice = sc.nextInt();
            if (choice == 1) System.out.println("Hello!");
            else if (choice == 2) System.out.println("Today is day 278 of the year");
        } while (choice != 0);
        System.out.println("Bye");

        int x = 100;
        do {
            System.out.println("runs once even though x < 10 is false");
        } while (x < 10);
    }
}
```

CONSOLE (user input in gold)

```
1. Say hello   2. Show date   0. Exit
Choice: 1
Hello!
1. Say hello   2. Show date   0. Exit
Choice: 2
Today is day 278 of the year
1. Say hello   2. Show date   0. Exit
Choice: 0
Bye
runs once even though x < 10 is false
```

	while	do-while
Condition checked	before the body	after the body

	while	do-while
Minimum runs	0	**1**
Semicolon at end	no	yes: `while (c);`
Typical use	loop while a condition holds	menus, input validation

6.3.4 Enhanced for-each loop (arrays and collections)

`for (Type item : collection)` visits every element of an array or collection in order, without an index. It is shorter and avoids index mistakes, but you cannot see the index, cannot modify the array elements through the loop variable, and always go from first to last.

VISITING EVERY ELEMENT

ForEach.java

```
import java.util.List;

public class ForEach {
    public static void main(String[] args) {
        int[] marks = {72, 88, 95, 61};
        int total = 0;
        for (int m : marks) {
            total += m;
        }
        System.out.println("total = " + total);

        List<String> cities = List.of("Hyderabad", "Chennai", "Pune");
        for (String c : cities) {
            System.out.println(c.toUpperCase());
        }

        for (int m : marks) m = 0;           // changes only the copy!
        System.out.println("marks[0] still " + marks[0]);
    }
}
```

OUTPUT

```
total = 316
HYDERABAD
CHENNAI
PUNE
marks[0] still 72
```

6.3.5 Choosing the right loop

Situation	Best loop
repeat a known number of times / counting	`for`
need the index of each element	`for`
visit every element, no index needed	**for-each**
repeat until something happens, maybe zero times	`while`
must run at least once (menu, input validation)	`do-while`
run until an explicit break	`while (true)` or `for (;;)`

6.4 Nested loops

6.4.1 Loop inside a loop

A loop can contain another loop. For every single repetition of the **outer** loop, the **inner** loop runs completely. Think of a clock: the minute hand goes all the way round for every step of the hour hand. Nested loops are how we process rows and columns.

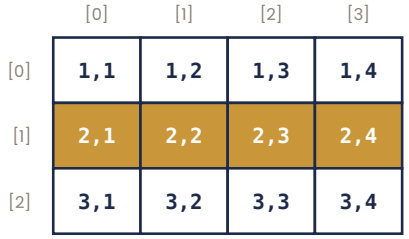
THE INNER LOOP FINISHES FOR EVERY OUTER STEP

Nested.java

```
public class Nested {
    public static void main(String[] args) {
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 4; col++) {
                System.out.print("(" + row + "," + col + ") ");
            }
            System.out.println();     // new line after each row
        }
    }
}
```

OUTPUT

```
(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)
```



OUTER LOOP = ROWS, INNER LOOP = COLUMNS (ROW 2 HIGHLIGHTED)

6.4.2 Time cost of nested loops

If the outer loop runs **n** times and the inner loop runs **n** times for each, the body runs **n × n = n²** times. Doubling the input makes the work four times bigger. Three nested loops give n³. This is written **O(n²)** and is why nested loops over large data become slow.

n	single loop (n)	nested loop (n²)	triple nested (n³)
10	10	100	1,000
100	100	10,000	1,000,000
1,000	1,000	1,000,000	1,000,000,000
10,000	10,000	100,000,000	too slow

COUNTING THE WORK OF A NESTED LOOP

NestedCost.java

```
public class NestedCost {
    public static void main(String[] args) {
        for (int n : new int[] {10, 100, 1000}) {
            long steps = 0;
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    steps++;
            System.out.println("n = " + n + " -> inner body ran " + steps + " times");
        }
    }
}
```

OUTPUT

```
n = 10 -> inner body ran 100 times
n = 100 -> inner body ran 10000 times
n = 1000 -> inner body ran 1000000 times
```

6.4.3 Pattern printing

Pattern programs are the best practice for nested loops. The method is always the same: the **outer loop counts rows**; for each row, work out **how many** spaces and symbols to print as a formula of the row number.

Pattern	Row i prints
Square (n×n)	n stars
Right triangle	i stars
Inverted triangle	n - i + 1 stars
Pyramid	n - i spaces, then 2i - 1 stars
Diamond	pyramid, then an inverted pyramid

6.4.3.1 Star patterns (square, triangle, pyramid, diamond)

SQUARE AND TRIANGLES

StarPatterns.java

```
public class StarPatterns {
    public static void main(String[] args) {
        int n = 4;
        System.out.println("Square:");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) System.out.print("* ");
            System.out.println();
        }
        System.out.println("Right triangle:");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) System.out.print("* ");
            System.out.println();
        }
        System.out.println("Inverted triangle:");
        for (int i = n; i >= 1; i--) {
            for (int j = 1; j <= i; j++) System.out.print("* ");
            System.out.println();
        }
    }
}
```

OUTPUT

```
Square:
* * * *
* * * *
* * * *
* * * *
Right triangle:
*
* *
* * *
* * * *
Inverted triangle:
* * * *
* * *
* *
*
```

PYRAMID AND DIAMOND

PyramidDiamond.java

```java
public class PyramidDiamond {
    public static void main(String[] args) {
        int n = 5;
        System.out.println("Pyramid:");
        for (int i = 1; i <= n; i++) {
            for (int s = 1; s <= n - i; s++) System.out.print(" ");      // spaces
            for (int k = 1; k <= 2 * i - 1; k++) System.out.print("*"); // stars
            System.out.println();
        }
        System.out.println("Diamond:");
        for (int i = 1; i <= n; i++) {                                  // top half
            System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
        }
        for (int i = n - 1; i >= 1; i--) {                              // bottom half
            System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
        }
    }
}
```

OUTPUT

```
Pyramid:
    *
   ***
  *****
 *******
*********
Diamond:
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Row i (n = 5)	spaces = n - i	stars = 2i - 1
1	4	1
2	3	3
3	2	5
4	1	7
5	0	9

6.4.3.2 Number patterns (Floyd's triangle, Pascal's triangle)

Floyd's triangle prints consecutive numbers, one more per row. **Pascal's triangle** prints binomial coefficients: each number is the sum of the two numbers above it. A neat formula computes each value from the previous one in the same row: `next = value × (i − k) / (k + 1)`.

FLOYD'S AND PASCAL'S TRIANGLES

NumberPatterns.java

```java
public class NumberPatterns {
    public static void main(String[] args) {
        System.out.println("Floyd's triangle:");
        int num = 1;
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) System.out.printf("%3d", num++);
            System.out.println();
        }
        System.out.println("Pascal's triangle:");
        int rows = 6;
        for (int i = 0; i < rows; i++) {
            System.out.print("  ".repeat(rows - i - 1));
            int value = 1;
            for (int k = 0; k <= i; k++) {
                System.out.printf("%4d", value);
                value = value * (i - k) / (k + 1);
            }
            System.out.println();
        }
        System.out.println("Number triangle:");
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) System.out.print(j);
            System.out.println();
        }
    }
}
```

OUTPUT

```
Floyd's triangle:
  1
  2  3
  4  5  6
  7  8  9 10
 11 12 13 14 15
Pascal's triangle:
             1
           1   1
         1   2   1
       1   3   3   1
     1   4   6   4   1
   1   5  10  10   5   1
Number triangle:
1
12
123
1234
```

6.4.3.3 Character patterns

Because chars are numbers, a loop can count letters: `(char) ('A' + j)` gives A, B, C…

PATTERNS WITH LETTERS

CharPatterns.java

```
public class CharPatterns {
    public static void main(String[] args) {
        System.out.println("Letter triangle:");
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j <= i; j++) System.out.print((char) ('A' + j) + " ");
            System.out.println();
        }
        System.out.println("Same letter per row:");
        for (char ch = 'A'; ch <= 'D'; ch++) {
            for (int j = 0; j <= ch - 'A'; j++) System.out.print(ch + " ");
            System.out.println();
        }
        System.out.println("Alphabet pyramid:");
        int n = 4;
        for (int i = 0; i < n; i++) {
            StringBuilder row = new StringBuilder(" ".repeat(n - i - 1));
            for (int j = 0; j <= i; j++) row.append((char) ('A' + j));
            for (int j = i - 1; j >= 0; j--) row.append((char) ('A' + j));
            System.out.println(row);
        }
    }
}
```

OUTPUT

```
Letter triangle:
A
A B
A B C
A B C D
A B C D E
Same letter per row:
A
B B
C C C
D D D D
Alphabet pyramid:
   A
  ABA
 ABCBA
ABCDCBA
```

6.5 Jump statements

6.5.1 break

`break` immediately **exits the nearest enclosing loop** (or switch). Execution continues with the first statement after the loop. Use it when you have found what you were looking for.

STOPPING A SEARCH EARLY BreakDemo.java

```
public class BreakDemo {
    public static void main(String[] args) {
        int[] rolls = {101, 104, 109, 113, 120};
        int target = 109;
        for (int i = 0; i < rolls.length; i++) {
            System.out.println("checking " + rolls[i]);
            if (rolls[i] == target) {
                System.out.println("found at index " + i);
                break;                  // stop searching
            }
        }
        System.out.println("search finished");
    }
}
```

OUTPUT

```
checking 101
checking 104
checking 109
found at index 2
search finished
```

6.5.2 continue

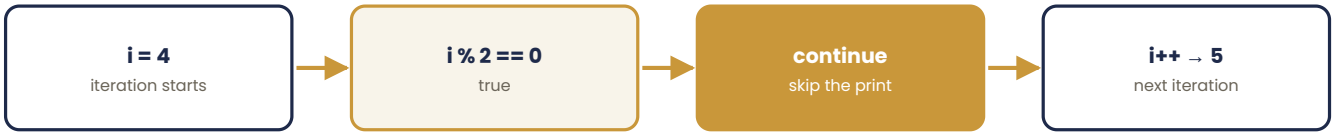
`continue` skips **the rest of the current iteration** and jumps to the next one (in a for loop, the update runs first). Use it to skip items you do not want to process.

SKIPPING UNWANTED ITEMS ContinueDemo.java

```
public class ContinueDemo {
    public static void main(String[] args) {
        System.out.print("odd numbers 1-10: ");
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) continue;   // skip even numbers
            System.out.print(i + " ");
        }
        System.out.println();
        int[] marks = {72, -1, 88, -1, 95};      // -1 = absent
        int sum = 0, count = 0;
        for (int m : marks) {
            if (m < 0) continue;        // skip absentees
            sum += m; count++;
        }
        System.out.println("average of present students = " + (double) sum / count);
    }
}
```

OUTPUT

```
odd numbers 1-10: 1 3 5 7 9
average of present students = 85.0
```



CONTINUE JUMPS STRAIGHT TO THE NEXT ITERATION

6.5.3 Labeled break and continue

A plain break or continue affects only the **innermost** loop. To control an **outer** loop, put a **label** (a name followed by a colon) before it and write `break label;` or `continue label;`.

BREAKING OUT OF NESTED LOOPS LabeledJumps.java

```
public class LabeledJumps {
    public static void main(String[] args) {
        int[][] seats = {{0, 0, 1}, {0, 1, 0}, {1, 1, 1}};   // 1 = free
        search:
        for (int r = 0; r < seats.length; r++) {
            for (int c = 0; c < seats[r].length; c++) {
                if (seats[r][c] == 1) {
                    System.out.println("first free seat: row " + r + ", col " + c);
                    break search;              // leaves BOTH loops
                }
            }
        }
        outer:
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (j == 2) continue outer;    // next i, skipping the rest of j
                System.out.println("i=" + i + " j=" + j);
            }
        }
    }
}
```

OUTPUT

```
first free seat: row 0, col 2
i=1 j=1
i=2 j=1
i=3 j=1
```

6.5.4 return

`return` ends the **whole method** immediately, optionally sending a value back to the caller. Inside `main`, a return ends the program (unless other threads are running). It is often used for "early exit" when the answer is already known.

RETURN EXITS THE METHOD ReturnDemo.java

```
public class ReturnDemo {
    static boolean isPrime(int n) {
        if (n < 2) return false;               // early return
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) return false;      // leaves the loop AND the method
        }
        return true;
    }
    public static void main(String[] args) {
        for (int n : new int[] {1, 2, 15, 17}) {
            System.out.println(n + " prime? " + isPrime(n));
        }
    }
}
```

OUTPUT

```
1 prime? false
2 prime? true
15 prime? false
17 prime? true
```

Statement	Jumps to	Affects
`break`	the first statement after the loop/switch	innermost loop or switch
`break label`	after the labelled loop	the labelled (outer) loop
`continue`	the next iteration (update/condition)	innermost loop
`continue label`	next iteration of the labelled loop	the labelled loop
`return`	back to the caller	the whole method

6.6 Classic loop problems

These problems appear in every exam and coding interview. Each one combines a loop with the arithmetic tricks `% 10` (last digit) and `/ 10` (remove last digit).



N % 10 = 9 (LAST DIGIT); N / 10 = 472 (THE REST)

6.6.1 Sum of digits, reverse a number, palindrome number

DIGIT BY DIGIT

DigitProblems.java

```
public class DigitProblems {
    public static void main(String[] args) {
        int n = 4729;
        int sum = 0, temp = n;
        while (temp > 0) {
            sum += temp % 10;          // take the last digit
            temp /= 10;                // drop the last digit
        }
        System.out.println("sum of digits of " + n + " = " + sum);

        int rev = 0;
        temp = n;
        while (temp > 0) {
            rev = rev * 10 + temp % 10;   // append digit to rev
            temp /= 10;
        }
        System.out.println("reverse of " + n + " = " + rev);

        for (int x : new int[] {12321, 4554, 1234}) {
            int r = 0, t = x;
            while (t > 0) { r = r * 10 + t % 10; t /= 10; }
            System.out.println(x + (r == x ? " is" : " is not") + " a palindrome");
        }
    }
}
```

OUTPUT

```
sum of digits of 4729 = 22
reverse of 4729 = 9274
12321 is a palindrome
4554 is a palindrome
1234 is not a palindrome
```

temp	temp % 10	rev = rev*10 + digit	temp / 10
4729	9	9	472
472	2	92	47
47	7	927	4
4	4	9274	0 → stop

6.6.2 Factorial and Fibonacci

Factorial n! = 1 × 2 × … × n (5! = 120). Use `long`: 13! already overflows an int. The **Fibonacci** sequence starts 0, 1 and each next number is the sum of the previous two.

FACTORIAL AND FIBONACCI WITH LOOPS

FactFib.java

```
public class FactFib {
    public static void main(String[] args) {
        int n = 10;
        long fact = 1;
        for (int i = 2; i <= n; i++) fact *= i;
        System.out.println(n + "! = " + fact);
        System.out.println("20! = " + factorial(20));

        System.out.print("First 12 Fibonacci numbers: ");
        long a = 0, b = 1;
        for (int i = 1; i <= 12; i++) {
            System.out.print(a + " ");
            long next = a + b;
            a = b;
            b = next;
        }
        System.out.println();
    }
    static long factorial(int n) {
        long f = 1;
        for (int i = 2; i <= n; i++) f *= i;
        return f;
    }
}
```

OUTPUT

```
10! = 3628800
20! = 2432902008176640000
First 12 Fibonacci numbers: 0 1 1 2 3 5 8 13 21 34 55 89
```

6.6.3 Prime check and primes in a range

A **prime** has exactly two divisors: 1 and itself. To test n, try divisors from 2 up to **√n** only: if n had a divisor larger than √n, it would also have one smaller than √n.

CHECKING DIVISORS UP TO THE SQUARE ROOT

Primes.java

```
public class Primes {
    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int d = 2; d * d <= n; d++) {
            if (n % d == 0) return false;
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println("97 prime? " + isPrime(97) + ", 91 prime? " + isPrime(91));
        System.out.print("primes between 1 and 60: ");
        int count = 0;
        for (int i = 1; i <= 60; i++) {
            if (isPrime(i)) { System.out.print(i + " "); count++; }
        }
        System.out.println("\n(" + count + " primes)");
    }
}
```

OUTPUT

```
97 prime? true, 91 prime? false
primes between 1 and 60: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
(17 primes)
```

6.6.4 Armstrong, perfect and strong numbers

Number type	Definition	Example
Armstrong	sum of each digit raised to the number of digits equals the number	153 = $1^3 + 5^3 + 3^3$
Perfect	sum of its proper divisors equals the number	28 = 1 + 2 + 4 + 7 + 14

Number type	Definition	Example
Strong	sum of the factorials of its digits equals the number	145 = 1! + 4! + 5!

THREE SPECIAL NUMBER CHECKS

SpecialNumbers.java

```
public class SpecialNumbers {
    static boolean armstrong(int n) {
        int digits = String.valueOf(n).length(), sum = 0, t = n;
        while (t > 0) { sum += (int) Math.pow(t % 10, digits); t /= 10; }
        return sum == n;
    }
    static boolean perfect(int n) {
        int sum = 0;
        for (int d = 1; d <= n / 2; d++) if (n % d == 0) sum += d;
        return n > 1 && sum == n;
    }
    static boolean strong(int n) {
        int[] fact = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
        int sum = 0, t = n;
        while (t > 0) { sum += fact[t % 10]; t /= 10; }
        return sum == n;
    }
    public static void main(String[] args) {
        System.out.print("Armstrong up to 1000: ");
        for (int i = 1; i <= 1000; i++) if (armstrong(i)) System.out.print(i + " ");
        System.out.print("\nPerfect up to 10000: ");
        for (int i = 1; i <= 10000; i++) if (perfect(i)) System.out.print(i + " ");
        System.out.print("\nStrong up to 100000: ");
        for (int i = 1; i <= 100000; i++) if (strong(i)) System.out.print(i + " ");
        System.out.println();
    }
}
```

OUTPUT

```
Armstrong up to 1000: 1 2 3 4 5 6 7 8 9 153 370 371 407
Perfect up to 10000: 6 28 496 8128
Strong up to 100000: 1 2 145 40585
```

6.6.5 GCD and LCM

The **GCD** (greatest common divisor, also HCF) is the largest number dividing both a and b. **Euclid's algorithm** repeatedly replaces (a, b) by (b, a % b) until b becomes 0; then a is the GCD. The **LCM** follows from the formula `lcm = a × b / gcd`.

a	b	a % b
48	18	12
18	12	6
12	6	0
6	0	stop: GCD = 6

EUCLID'S ALGORITHM

GcdLcm.java

```java
public class GcdLcm {
    public static void main(String[] args) {
        int a = 48, b = 18;
        int x = a, y = b;
        while (y != 0) {
            int r = x % y;
            x = y;
            y = r;
        }
        int gcd = x;
        int lcm = a / gcd * b;          // divide first to avoid overflow
        System.out.println("GCD(" + a + ", " + b + ") = " + gcd);
        System.out.println("LCM(" + a + ", " + b + ") = " + lcm);
    }
}
```

OUTPUT

```
GCD(48, 18) = 6
LCM(48, 18) = 144
```

PRACTICE CHECKPOINT

A **menu-driven program** using `switch` inside a `do-while` loop, plus **five pattern programs**. The sample below combines both: the menu prints a chosen pattern.

SAMPLE SOLUTION

PatternMenu.java

```
import java.util.Scanner;

public class PatternMenu {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            System.out.println("1 Square 2 Triangle 3 Pyramid 4 Floyd 5 Letters 0 Exit");
            System.out.print("Choice: ");
            choice = sc.nextInt();
            int n = 4;
            switch (choice) {
                case 1 -> { for (int i = 0; i < n; i++) System.out.println("* ".repeat(n)); }
                case 2 -> { for (int i = 1; i <= n; i++) System.out.println("* ".repeat(i)); }
                case 3 -> { for (int i = 1; i <= n; i++) System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
}
                case 4 -> {
                    int k = 1;
                    for (int i = 1; i <= n; i++) {
                        for (int j = 1; j <= i; j++) System.out.print(k++ + " ");
                        System.out.println();
                    }
                }
                case 5 -> {
                    for (int i = 0; i < n; i++) {
                        for (int j = 0; j <= i; j++) System.out.print((char) ('A' + j));
                        System.out.println();
                    }
                }
                case 0 -> System.out.println("Goodbye!");
                default -> System.out.println("Invalid choice");
            }
        } while (choice != 0);
    }
}
```

CONSOLE (user input in gold)

```
1 Square 2 Triangle 3 Pyramid 4 Floyd 5 Letters 0 Exit
Choice: 3
   *
  ***
 *****
*******
1 Square 2 Triangle 3 Pyramid 4 Floyd 5 Letters 0 Exit
Choice: 4
1
2 3
4 5 6
7 8 9 10
1 Square 2 Triangle 3 Pyramid 4 Floyd 5 Letters 0 Exit
Choice: 9
Invalid choice
1 Square 2 Triangle 3 Pyramid 4 Floyd 5 Letters 0 Exit
Choice: 0
Goodbye!
```

MODULE 07 · BEGINNER

Arrays

Minimum duration **4 days** | Level **Beginner**

An array stores a fixed number of values of one type in contiguous, indexed positions.

IN THIS MODULE

7.1 Introduction to arrays
7.2 One-dimensional arrays
7.3 Multi-dimensional arrays
7.4 Arrays and methods
7.5 Copying arrays
7.6 The Arrays utility class
7.7 Array algorithms
7.8 Command-line arguments array

7.1 Introduction to arrays

7.1.1 Why arrays (many values, one name)

Imagine storing the marks of 60 students. Sixty separate variables (`m1, m2, … m60`) would be impossible to loop over. An **array** stores all of them under **one name**, and a loop can process them with a few lines of code.

FIVE VARIABLES VS ONE ARRAY

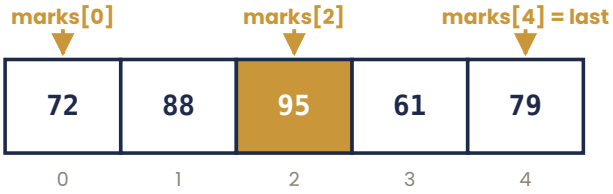
java

```
// without an array: a variable per student, no way to loop
int m1 = 72, m2 = 88, m3 = 95, m4 = 61, m5 = 79;
int total = m1 + m2 + m3 + m4 + m5;

// with an array: one name, any number of values, loops work
int[] marks = {72, 88, 95, 61, 79};
int sum = 0;
for (int m : marks) sum += m;
```

7.1.2 Indexing from 0

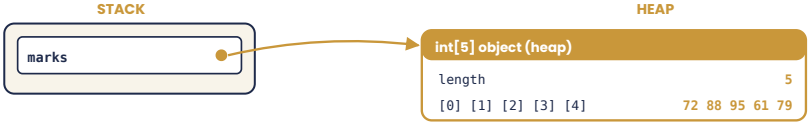
Each element has a numbered position called its **index**. Indexes start at **0**, so an array of length 5 has indexes 0 to 4. The last index is always `length - 1`. The element at index i is written `marks[i]`.



FIVE ELEMENTS, INDEXES 0 TO 4

7.1.3 Arrays are objects stored in the heap

In Java an array is an **object**. The variable `marks` is a reference stored on the stack; the actual elements sit side by side (contiguously) in one block on the **heap**. Because elements are contiguous and the same size, the JVM finds any element instantly: address = start + index × element size. That is why `a[i]` takes the same time for any i (O(1)).



THE REFERENCE LIVES ON THE STACK; THE ARRAY OBJECT LIVES ON THE HEAP

7.2 One-dimensional arrays

7.2.1 Declaration: int[] a; vs int a[];

Both forms declare a variable that can refer to an int array. `int[] a;` is the preferred Java style, because the type ("array of int") is kept together. `int a[];` comes from C and is allowed. Declaring does **not** create the array: the variable is still empty (null if a field).

WATCH OUT

`int[] a, b;` declares **two** arrays, but `int a[], b;` declares an array `a` and a plain int `b`. Another reason to prefer `int[] a`.

7.2.2 Creation with new int[5]

`new int[5]` creates an array object with 5 elements on the heap. The size can be any int expression, even a variable read at runtime, but once created the size **never changes**.

7.2.3 Initialization with values {1, 2, 3}

If you already know the values, list them in braces. Java counts them and creates an array of exactly that length.

THREE WAYS TO CREATE AN ARRAY CreateArrays.java

```
public class CreateArrays {
    public static void main(String[] args) {
        int[] a;                        // 7.2.1 declaration only
        a = new int[5];                 // 7.2.2 creation: 5 zeros
        a[0] = 10;  a[4] = 50;          // assign by index
        int n = 3;
        double[] prices = new double[n];        // size from a variable
        prices[1] = 99.5;
        String[] days = {"Mon", "Tue", "Wed"};  // 7.2.3 initializer list
        int[] primes = new int[] {2, 3, 5, 7};  // explicit form (needed outside a declaration)
        System.out.println(a[0] + " " + a[1] + " " + a[4]);
        System.out.println(prices[0] + " " + prices[1]);
        System.out.println(days[2] + " " + primes.length);
    }
}
```

OUTPUT

```
10 0 50
0.0 99.5
Wed 4
```

7.2.4 Default values by type

A new array is never full of garbage: every element starts with the default value of its type.

DEFAULT ELEMENT VALUES ArrayDefaults.java

```java
import java.util.Arrays;

public class ArrayDefaults {
    public static void main(String[] args) {
        System.out.println("int     : " + Arrays.toString(new int[3]));
        System.out.println("double  : " + Arrays.toString(new double[3]));
        System.out.println("boolean : " + Arrays.toString(new boolean[3]));
        System.out.println("char    : " + (int) (new char[3])[0] +" (the \\u0000 character)");
        System.out.println("String  : " + Arrays.toString(new String[3]));
    }
}
```

OUTPUT

```
int     : [0, 0, 0]
double  : [0.0, 0.0, 0.0]
boolean : [false, false, false]
char    : 0 (the \u0000 character)
String  : [null, null, null]
```

7.2.5 The length property

Every array has a **field** (not a method) called `length`: `marks.length`, with no parentheses. Always loop with `i < arr.length` instead of a hard-coded number, so the loop keeps working if the size changes.

> **WATCH OUT**
> Arrays use `length` (field), Strings use `length()` (method), collections use `size()` (method). Mixing them up is a very common compile error.

7.2.6 Traversal with for and for-each

VISITING EVERY ELEMENT Traversal.java

```java
public class Traversal {
    public static void main(String[] args) {
        int[] marks = {72, 88, 95, 61, 79};
        // classic for: index available, can modify elements
        for (int i = 0; i < marks.length; i++) {
            System.out.println("marks[" + i + "] = " + marks[i]);
        }
        // add 5 bonus marks to everybody (needs the index)
        for (int i = 0; i < marks.length; i++) marks[i] += 5;
        // for-each: read every element, no index
        int sum = 0;
        for (int m : marks) sum += m;
        System.out.println("sum after bonus = " + sum);
        // backwards
        for (int i = marks.length - 1; i >= 0; i--) System.out.print(marks[i] + " ");
        System.out.println();
    }
}
```

OUTPUT

```
marks[0] = 72
marks[1] = 88
marks[2] = 95
marks[3] = 61
marks[4] = 79
sum after bonus = 420
84 66 100 93 77
```

7.2.7 ArrayIndexOutOfBoundsException

Using an index below 0 or at/above `length` throws `ArrayIndexOutOfBoundsException` at runtime. The most common cause is `<=` instead of `<` in the loop condition ("off-by-one" error).

THE CLASSIC OFF-BY-ONE ERROR

OutOfBounds.java

```
public class OutOfBounds {
    public static void main(String[] args) {
        int[] a = {10, 20, 30};
        for (int i = 0; i <= a.length; i++) {     // BUG: <= should be <
            System.out.println(a[i]);
        }
    }
}
```

OUTPUT

```
10
20
30
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
        at OutOfBounds.main(OutOfBounds.java:5)
```

7.3 Multi-dimensional arrays

7.3.1 2D arrays (matrices): rows and columns

A 2D array is a table of rows and columns: `int[][] m = new int[3][4];` has 3 rows and 4 columns. Access an element with two indexes, **row first**: `m[row][col]`. `m.length` is the number of rows; `m[r].length` is the number of columns in row r.

A 3×3 TABLE OF MARKS

Matrix2D.java

```
public class Matrix2D {
    public static void main(String[] args) {
        int[][] marks = {
            {80, 75, 90},      // student 0: maths, physics, chemistry
            {60, 85, 70},      // student 1
            {95, 65, 88}       // student 2
        };
        System.out.println("rows = " + marks.length + ", cols = " + marks[0].length);
        System.out.println("student 1, physics = " + marks[1][1]);
        for (int r = 0; r < marks.length; r++) {
            int total = 0;
            for (int c = 0; c < marks[r].length; c++) {
                System.out.printf("%4d", marks[r][c]);
                total += marks[r][c];
            }
            System.out.println("   | total " + total);
        }
    }
}
```

OUTPUT

```
rows = 3, cols = 3
student 1, physics = 85
  80  75  90   | total 245
  60  85  70   | total 215
  95  65  88   | total 248
```

	[0]	[1]	[2]
[0]	80	75	90
[1]	60	85	70
[2]	95	65	88

MARKS[1][1] = ROW 1, COLUMN 1 = 85

7.3.2 Jagged (ragged) arrays

Because a 2D array is really an array of arrays, the rows may have **different lengths**. Create only the outer array first, then each row separately. Useful for triangles, seating with uneven rows, or students with different numbers of subjects.

ROWS OF DIFFERENT LENGTHS

Jagged.java

```
public class Jagged {
    public static void main(String[] args) {
        int[][] tri = new int[4][];          // 4 rows, columns decided later
        for (int r = 0; r < tri.length; r++) {
            tri[r] = new int[r + 1];          // row r has r+1 elements
            for (int c = 0; c <= r; c++) tri[r][c] = (r + 1) * 10 + c;
        }
        for (int[] row : tri) {
            for (int v : row) System.out.print(v + " ");
            System.out.println("  (length " + row.length + ")");
        }
    }
}
```

OUTPUT

```
10   (length 1)
20 21   (length 2)
30 31 32   (length 3)
40 41 42 43   (length 4)
```

7.3.3 3D arrays

A 3D array adds a third index, like several 2D tables stacked: for example `sales[branch][month][product]`. It is an array of arrays of arrays.

THREE INDEXES

ThreeD.java

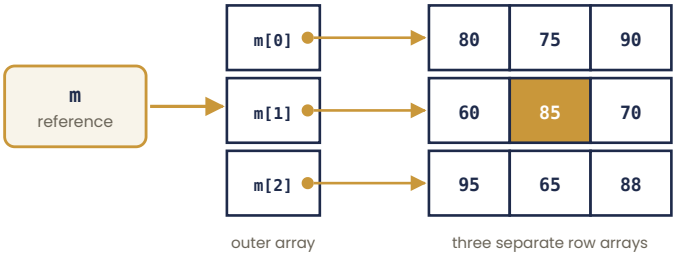
```
public class ThreeD {
    public static void main(String[] args) {
        // 2 buildings x 3 floors x 4 rooms, value = number of students
        int[][][] campus = new int[2][3][4];
        campus[1][2][3] = 42;
        int total = 0;
        for (int b = 0; b < campus.length; b++)
            for (int f = 0; f < campus[b].length; f++)
                for (int r = 0; r < campus[b][f].length; r++) {
                    campus[b][f][r] += 10;
                    total += campus[b][f][r];
                }
        System.out.println("rooms: " + 2 * 3 * 4 + ", students: " + total);
        System.out.println("building 1, floor 2, room 3: " + campus[1][2][3]);
    }
}
```

OUTPUT

```
rooms: 24, students: 282
building 1, floor 2, room 3: 52
```

7.3.4 Memory layout (array of arrays)

Java has no true rectangular block for 2D arrays. `int[3][4]` creates **one outer array** of 3 references, each pointing to a **separate** int[4] row object. That is why rows can have different lengths and why a single row can be passed around (`int[] row = m[1];`).



A 2D ARRAY IS AN ARRAY OF REFERENCES TO ROW ARRAYS

7.4 Arrays and methods

7.4.1 Passing an array to a method

When you pass an array, the method receives a copy of the **reference**, so it works on the **same** array object. Changes made to elements inside the method are visible to the caller.

7.4.2 Returning an array

A method can create an array and return it. This is the usual way to return several values of the same type.

7.4.3 Anonymous arrays

`new int[] {1, 2, 3}` creates an array without storing it in a variable. Use it to pass values directly to a method.

PASSING, MODIFYING AND RETURNING ARRAYS

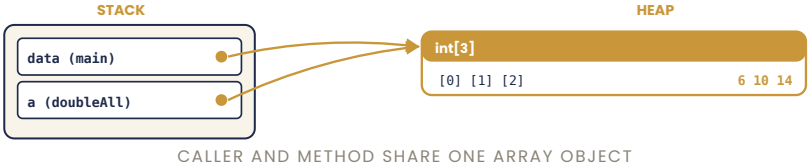
ArrayMethods.java

```
import java.util.Arrays;

public class ArrayMethods {
    static int sum(int[] a) {                  // 7.4.1 receives a reference
        int s = 0;
        for (int x : a) s += x;
        return s;
    }
    static void doubleAll(int[] a) {            // modifies the caller's array!
        for (int i = 0; i < a.length; i++) a[i] *= 2;
    }
    static int[] firstN(int n) {                // 7.4.2 returns a new array
        int[] r = new int[n];
        for (int i = 0; i < n; i++) r[i] = i + 1;
        return r;
    }
    public static void main(String[] args) {
        int[] data = {3, 5, 7};
        System.out.println("sum = " + sum(data));
        doubleAll(data);
        System.out.println("after doubleAll: " + Arrays.toString(data));
        int[] nums = firstN(6);
        System.out.println("firstN(6): " + Arrays.toString(nums));
        System.out.println("anonymous: " + sum(new int[] {10, 20, 30}));   // 7.4.3
    }
}
```

OUTPUT

```
sum = 15
after doubleAll: [6, 10, 14]
firstN(6): [1, 2, 3, 4, 5, 6]
anonymous: 60
```

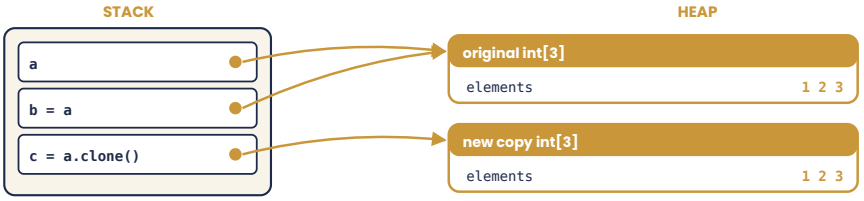


CALLER AND METHOD SHARE ONE ARRAY OBJECT

7.5 Copying arrays

7.5.1 Reference copy vs value copy

`int[] b = a;` does **not** copy the array. It copies the reference, so `a` and `b` are two names for the same object. To get an independent copy you must create a new array and copy the elements.



B SHARES THE ORIGINAL; C IS A REAL COPY

7.5.2 Loop copy

7.5.3 System.arraycopy()

`System.arraycopy(src, srcPos, dest, destPos, length)` copies a range of elements into an **existing** array very fast. It is used inside ArrayList when it grows.

7.5.4 Arrays.copyOf() and copyOfRange()

`Arrays.copyOf(a, newLength)` returns a new array (padded with defaults or truncated). `Arrays.copyOfRange(a, from, to)` copies indexes from (inclusive) to to (exclusive).

7.5.5 clone(); shallow vs deep copy

`a.clone()` returns a new array with the same elements. For arrays of primitives this is a full copy. For arrays of **objects** or 2D arrays it is a **shallow** copy: the new outer array still points to the **same** inner objects/rows. A **deep** copy also copies every inner object.

EVERY WAY TO COPY AN ARRAY

CopyArrays.java

```
import java.util.Arrays;

public class CopyArrays {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        int[] ref = a;                          // 7.5.1 reference copy
        int[] loop = new int[a.length];         // 7.5.2 loop copy
        for (int i = 0; i < a.length; i++) loop[i] = a[i];
        int[] sys = new int[7];
        System.arraycopy(a, 1, sys, 2, 3);      // 7.5.3 copy a[1..3] to sys[2..4]
        int[] bigger = Arrays.copyOf(a, 7);     // 7.5.4
        int[] middle = Arrays.copyOfRange(a, 1, 4);
        int[] cl = a.clone();                   // 7.5.5
        a[0] = 99;                              // change the original
        System.out.println("ref    : " + Arrays.toString(ref) + "  <- sees the change");
        System.out.println("loop   : " + Arrays.toString(loop));
        System.out.println("sys    : " + Arrays.toString(sys));
        System.out.println("copyOf : " + Arrays.toString(bigger));
        System.out.println("range  : " + Arrays.toString(middle));
        System.out.println("clone  : " + Arrays.toString(cl));

        int[][] grid = {{1, 2}, {3, 4}};
        int[][] shallow = grid.clone();
        int[][] deep = new int[grid.length][];
        for (int i = 0; i < grid.length; i++) deep[i] = grid[i].clone();
        grid[0][0] = 100;
        System.out.println("shallow: " + Arrays.deepToString(shallow) + "  <- shares rows");
        System.out.println("deep   : " + Arrays.deepToString(deep));
    }
}
```

OUTPUT

```
ref    : [99, 2, 3, 4, 5]  <- sees the change
loop   : [1, 2, 3, 4, 5]
sys    : [0, 0, 2, 3, 4, 0, 0]
copyOf : [1, 2, 3, 4, 5, 0, 0]
range  : [2, 3, 4]
clone  : [1, 2, 3, 4, 5]
shallow: [[100, 2], [3, 4]]  <- shares rows
deep   : [[1, 2], [3, 4]]
```

Method	Creates new array?	Copies	Best for
`b = a`	no	reference only	never, when you want a copy
loop	yes (you create it)	any logic you want	transforming while copying
`System.arraycopy`	no (into existing)	a range	fast bulk copies
`Arrays.copyOf/Range`	yes	all / a range, resizes	everyday copying
`clone()`	yes	all elements (shallow)	quick full copy

7.6 The Arrays utility class

`java.util.Arrays` is full of static helper methods for arrays. Import it with `import java.util.Arrays;`.

7.6.1 Arrays.toString() and deepToString()

Printing an array directly shows a cryptic type and hash (`[I@1b6d3586`). `Arrays.toString(a)` gives a readable `[1, 2, 3]`; `Arrays.deepToString(m)` does the same for 2D and nested arrays.

7.6.2 Arrays.sort()

Sorts the array in ascending order, in place. For primitives it uses a fast Dual-Pivot Quicksort (O(n log n)); for objects a stable merge sort (TimSort). You can also sort a range: `Arrays.sort(a, from, to)`.

7.6.3 Arrays.binarySearch()

Finds a value in a **sorted** array in O(log n). Returns the index if found; otherwise a negative number `-(insertionPoint) - 1`. On an unsorted array the result is meaningless.

7.6.4 Arrays.fill(), equals(), asList()

`fill(a, v)` sets every element to v. `equals(a, b)` compares contents element by element (`a == b` only compares references). `asList(…)` wraps an object array in a fixed-size List view.

THE MOST USEFUL ARRAYS METHODS

ArraysClass.java

```
import java.util.Arrays;
import java.util.List;

public class ArraysClass {
    public static void main(String[] args) {
        int[] a = {42, 7, 19, 3, 25};
        System.out.println("println(a)    : " + a);                    // not readable
        System.out.println("toString     : " + Arrays.toString(a));
        Arrays.sort(a);
        System.out.println("sorted       : " + Arrays.toString(a));
        System.out.println("search 19    : index " + Arrays.binarySearch(a, 19));
        System.out.println("search 20    : " + Arrays.binarySearch(a, 20) + " (not found)");
        int[] b = new int[5];
        Arrays.fill(b, 7);
        System.out.println("fill 7       : " + Arrays.toString(b));
        int[] c = {3, 7, 19, 25, 42};
        System.out.println("a == c       : " + (a == c));
        System.out.println("equals(a, c) : " + Arrays.equals(a, c));
        int[][] m = {{1, 2}, {3, 4}};
        System.out.println("deepToString : " + Arrays.deepToString(m));
        List<String> names = Arrays.asList("Diya", "Aarav", "Rohan");
        System.out.println("asList       : " + names + ", size " + names.size());
        String[] s = {"pear", "apple", "mango"};
        Arrays.sort(s);
        System.out.println("sorted words : " + Arrays.toString(s));
    }
}
```

OUTPUT

```
println(a)    : [I@2a139a55
toString     : [42, 7, 19, 3, 25]
sorted       : [3, 7, 19, 25, 42]
search 19    : index 2
search 20    : -4 (not found)
fill 7       : [7, 7, 7, 7, 7]
a == c       : false
equals(a, c) : true
deepToString : [[1, 2], [3, 4]]
asList       : [Diya, Aarav, Rohan], size 3
sorted words : [apple, mango, pear]
```

7.7 Array algorithms

7.7.1 Searching

7.7.1.1 Linear search

Check every element from the start until the target is found. Works on **any** array, sorted or not. Time: O(n): on average n/2 comparisons.

LINEAR SEARCH LinearSearch.java

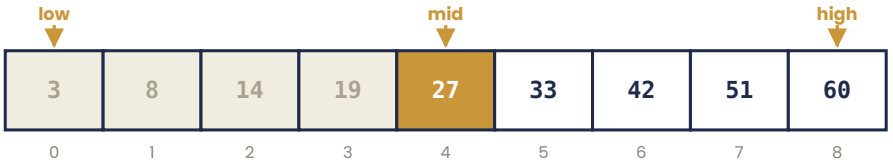
```
public class LinearSearch {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {14, 3, 27, 8, 19, 42};
        System.out.println("27 found at " + linearSearch(a, 27));
        System.out.println("5 found at " + linearSearch(a, 5));
    }
}
```

OUTPUT

```
27 found at 2
5 found at -1
```

7.7.1.2 Binary search

On a **sorted** array, look at the middle element. If it is the target, done. If the target is smaller, discard the right half; if larger, discard the left half. Each step halves the search space, so even 1,000,000 elements need at most 20 comparisons: O(log n).



SEARCHING 42: MID = 27 IS TOO SMALL, SO THE LEFT HALF (GREY) IS DISCARDED

BINARY SEARCH WITH A TRACE BinarySearch.java

```
public class BinarySearch {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;      // avoids int overflow
            System.out.println("  low=" + low + " high=" + high + " mid=" + mid + " a[mid]=" + a[mid]);
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;       // search right half
            else high = mid - 1;                   // search left half
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {3, 8, 14, 19, 27, 33, 42, 51, 60};
        System.out.println("search 42:");
        System.out.println("found at " + binarySearch(a, 42));
        System.out.println("search 20:");
        System.out.println("found at " + binarySearch(a, 20));
    }
}
```

OUTPUT

```
search 42:
  low=0 high=8 mid=4 a[mid]=27
  low=5 high=8 mid=6 a[mid]=42
found at 6
search 20:
  low=0 high=8 mid=4 a[mid]=27
  low=0 high=3 mid=1 a[mid]=8
  low=2 high=3 mid=2 a[mid]=14
  low=3 high=3 mid=3 a[mid]=19
found at -1
```

	Linear search	Binary search
Array must be sorted?	no	**yes**
Time	O(n)	O(log n)
1,000,000 elements	up to 1,000,000 checks	about 20 checks
Code	very simple	needs care with low/high/mid

7.7.2 Sorting

Sorting puts elements in order. In real code use `Arrays.sort()`, but writing the classic algorithms yourself teaches how loops, comparisons and swaps work. All three below are O(n²).

7.7.2.1 Bubble sort

Repeatedly walk through the array comparing **neighbours** and swapping them if they are in the wrong order. After each pass the largest remaining value has "bubbled" to the end. If a pass makes no swaps, the array is sorted and we can stop early.



BUBBLE SORT: THE SORTED PART (GOLD) GROWS FROM THE RIGHT

BUBBLE SORT WITH EARLY EXIT

BubbleSort.java

```
import java.util.Arrays;

public class BubbleSort {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int pass = 0; pass < a.length - 1; pass++) {
            boolean swapped = false;
            for (int i = 0; i < a.length - 1 - pass; i++) {
                if (a[i] > a[i + 1]) {            // wrong order: swap
                    int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t;
                    swapped = true;
                }
            }
            System.out.println("after pass " + (pass + 1) + ": " + Arrays.toString(a));
            if (!swapped) break;                  // already sorted
        }
    }
}
```

OUTPUT

```
after pass 1: [1, 4, 2, 5, 8]
after pass 2: [1, 2, 4, 5, 8]
after pass 3: [1, 2, 4, 5, 8]
```

7.7.2.2 Selection sort

Find the **smallest** element in the unsorted part and swap it into the next position. After pass i, the first i+1 elements are sorted. It makes at most n-1 swaps, which is useful when swapping is expensive.

SELECTION SORT

SelectionSort.java

```
import java.util.Arrays;

public class SelectionSort {
    public static void main(String[] args) {
        int[] a = {29, 10, 14, 37, 13};
        for (int i = 0; i < a.length - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[minIdx]) minIdx = j;     // remember the smallest
            }
            int t = a[i]; a[i] = a[minIdx]; a[minIdx] = t;
            System.out.println("pass " + (i + 1) + ": " + Arrays.toString(a));
        }
    }
}
```

OUTPUT

```
pass 1: [10, 29, 14, 37, 13]
pass 2: [10, 13, 14, 37, 29]
pass 3: [10, 13, 14, 37, 29]
pass 4: [10, 13, 14, 29, 37]
```

7.7.2.3 Insertion sort

Like sorting playing cards in your hand: take the next element and **shift** larger elements to the right until you find its place, then insert it. Very fast on arrays that are already almost sorted (O(n) best case).

INSERTION SORT

InsertionSort.java

```
import java.util.Arrays;

public class InsertionSort {
    public static void main(String[] args) {
        int[] a = {12, 11, 13, 5, 6};
        for (int i = 1; i < a.length; i++) {
            int key = a[i];
            int j = i - 1;
            while (j >= 0 && a[j] > key) {
                a[j + 1] = a[j];          // shift right
                j--;
            }
            a[j + 1] = key;               // insert in the gap
            System.out.println("insert " + key + ": " + Arrays.toString(a));
        }
    }
}
```

OUTPUT

```
insert 11: [11, 12, 13, 5, 6]
insert 13: [11, 12, 13, 5, 6]
insert 5: [5, 11, 12, 13, 6]
insert 6: [5, 6, 11, 12, 13]
```

Algorithm	Idea	Best	Average/Worst	Swaps
Bubble	swap neighbours	O(n) with early exit	O(n²)	many
Selection	select the minimum	O(n²)	O(n²)	at most n-1
Insertion	insert into sorted part	O(n)	O(n²)	shifts
`Arrays.sort`	quicksort / TimSort	O(n log n)	O(n log n)	—

7.7.3 Common problems: max/min, second largest, reverse, rotate, remove duplicates, frequency count, missing number

SEVEN CLASSIC ARRAY PROBLEMS

ArrayProblems.java

```
import java.util.Arrays;

public class ArrayProblems {
    public static void main(String[] args) {
        int[] a = {12, 35, 1, 10, 35, 34, 1};
        // max and min in one pass
        int max = a[0], min = a[0];
        for (int x : a) { if (x > max) max = x; if (x < min) min = x; }
        System.out.println("max=" + max + " min=" + min);

        // second largest (distinct)
        int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
        for (int x : a) {
            if (x > first) { second = first; first = x; }
            else if (x > second && x != first) second = x;
        }
        System.out.println("second largest=" + second);

        // reverse in place (two pointers)
        int[] r = {1, 2, 3, 4, 5};
        for (int i = 0, j = r.length - 1; i < j; i++, j--) { int t = r[i]; r[i] = r[j]; r[j] = t; }
        System.out.println("reversed=" + Arrays.toString(r));

        // rotate left by k
        int[] rot = {1, 2, 3, 4, 5, 6};
        int k = 2, n = rot.length;
        int[] res = new int[n];
        for (int i = 0; i < n; i++) res[i] = rot[(i + k) % n];
        System.out.println("rotated left by 2=" + Arrays.toString(res));

        // remove duplicates from a sorted array
        int[] s = {1, 1, 2, 2, 2, 3, 5, 5};
        int len = 1;
        for (int i = 1; i < s.length; i++) if (s[i] != s[len - 1]) s[len++] = s[i];
        System.out.println("unique=" + Arrays.toString(Arrays.copyOf(s, len)));

        // frequency count of values 0..9
        int[] data = {3, 7, 3, 1, 7, 7, 9};
        int[] freq = new int[10];
        for (int x : data) freq[x]++;
        for (int v = 0; v < 10; v++) if (freq[v] > 0) System.out.print(v + "x" + freq[v] + " ");
        System.out.println();

        // missing number in 1..n
        int[] nums = {1, 2, 4, 5, 6};
        int nn = nums.length + 1, expected = nn * (nn + 1) / 2, actual = 0;
        for (int x : nums) actual += x;
        System.out.println("missing number=" + (expected - actual));
    }
}
```

OUTPUT

```
max=35 min=1
second largest=34
reversed=[5, 4, 3, 2, 1]
rotated left by 2=[3, 4, 5, 6, 1, 2]
unique=[1, 2, 3, 5]
1x1 3x2 7x3 9x1
missing number=3
```

7.7.4 Matrix problems: addition, multiplication, transpose, spiral print

Matrix **addition** adds matching cells. **Multiplication** A(m×n) × B(n×p) gives C(m×p) where C[i][j] is the sum of A[i][k] × B[k][j] over k: a triple loop. The **transpose** swaps rows and columns. **Spiral** printing walks the outer ring, then the next ring inward.

MATRIX ADDITION, MULTIPLICATION, TRANSPOSE AND SPIRAL

MatrixOps.java

```
import java.util.Arrays;

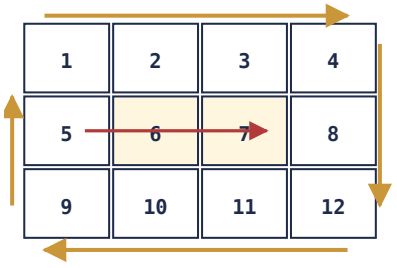
public class MatrixOps {
    public static void main(String[] args) {
        int[][] A = {{1, 2}, {3, 4}};
        int[][] B = {{5, 6}, {7, 8}};
        int[][] sum = new int[2][2], prod = new int[2][2];
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++) {
                sum[i][j] = A[i][j] + B[i][j];
                for (int k = 0; k < 2; k++) prod[i][j] += A[i][k] * B[k][j];
            }
        System.out.println("A + B = " + Arrays.deepToString(sum));
        System.out.println("A x B = " + Arrays.deepToString(prod));

        int[][] M = {{1, 2, 3}, {4, 5, 6}};
        int[][] T = new int[3][2];
        for (int i = 0; i < 2; i++) for (int j = 0; j < 3; j++) T[j][i] = M[i][j];
        System.out.println("transpose = " + Arrays.deepToString(T));

        int[][] S = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
        int top = 0, bottom = S.length - 1, left = 0, right = S[0].length - 1;
        StringBuilder out = new StringBuilder("spiral = ");
        while (top <= bottom && left <= right) {
            for (int j = left; j <= right; j++) out.append(S[top][j]).append(' ');
            top++;
            for (int i = top; i <= bottom; i++) out.append(S[i][right]).append(' ');
            right--;
            if (top <= bottom) { for (int j = right; j >= left; j--) out.append(S[bottom][j]).append(' '); bottom--; }
            if (left <= right) { for (int i = bottom; i >= top; i--) out.append(S[i][left]).append(' '); left++; }
        }
        System.out.println(out);
    }
}
```

OUTPUT

```
A + B = [[6, 8], [10, 12]]
A x B = [[19, 22], [43, 50]]
transpose = [[1, 4], [2, 5], [3, 6]]
spiral = 1 2 3 4 8 12 11 10 9 5 6 7
```



SPIRAL ORDER: OUTER RING (GOLD), THEN INNER RING (RED)

7.8 Command-line arguments array (String[] args)

The parameter of `main` is itself an array: `String[] args`. Each word typed after the class name becomes one element, so everything in this module applies to it: `args.length`, indexing, loops, parsing with `Integer.parseInt`. If no arguments are given, `args` is an **empty array** (length 0), never null.

TREATING ARGS LIKE ANY OTHER ARRAY

MaxOfArgs.java

```
public class MaxOfArgs {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Pass numbers, e.g. java MaxOfArgs 4 19 7");
            return;
        }
        int[] nums = new int[args.length];
        for (int i = 0; i < args.length; i++) nums[i] = Integer.parseInt(args[i]);
        int max = nums[0];
        for (int n : nums) max = Math.max(max, n);
        System.out.println("max of " + args.length + " numbers = " + max);
    }
}
```

OUTPUT

```
Pass numbers, e.g. java MaxOfArgs 4 19 7
```

Terminal

```
$ java MaxOfArgs 4 19 7 12
max of 4 numbers = 19
```

PRACTICE CHECKPOINT

Read **n** numbers into an array, sort them with **your own bubble sort**, then **binary-search** for a value.

SAMPLE SOLUTION

SortAndSearch.java

```
import java.util.Arrays;
import java.util.Scanner;

public class SortAndSearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many numbers? ");
        int n = sc.nextInt();
        int[] a = new int[n];
        System.out.print("Enter " + n + " numbers: ");
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();

        for (int pass = 0; pass < n - 1; pass++) {            // bubble sort
            boolean swapped = false;
            for (int i = 0; i < n - 1 - pass; i++) {
                if (a[i] > a[i + 1]) { int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t; swapped = true; }
            }
            if (!swapped) break;
        }
        System.out.println("Sorted: " + Arrays.toString(a));

        System.out.print("Value to find: ");
        int key = sc.nextInt();
        int low = 0, high = n - 1, found = -1;                // binary search
        while (low <= high) {
            int mid = (low + high) / 2;
            if (a[mid] == key) { found = mid; break; }
            if (a[mid] < key) low = mid + 1; else high = mid - 1;
        }
        System.out.println(found >= 0 ? key + " found at index " + found : key + " not found");
    }
}
```

CONSOLE (user input in gold)

```
How many numbers? 7
Enter 7 numbers: 42 7 19 3 25 11 30
Sorted: [3, 7, 11, 19, 25, 30, 42]
Value to find: 19
19 found at index 3
```

MODULE 08 · BEGINNER

Strings

Minimum duration **3 days** | Level **Beginner**

A String is an immutable sequence of characters; this module covers how strings are stored, compared and changed efficiently.

IN THIS MODULE

8.1 Introduction to String
8.2 Immutability
8.3 Comparing strings
8.4 String methods
8.5 StringBuilder and StringBuffer
8.6 Characters and the Character class
8.7 Advanced string features
8.8 Classic string problems

8.1 Introduction to String

8.1.1 String as a class in java.lang

Text in Java is represented by the class `java.lang.String`. Because it is in `java.lang`, no import is needed. A String is an **object** that holds a sequence of characters, each at an index starting from 0, much like a char array but with dozens of useful methods.



"JAVA 21": CHARACTERS AT INDEXES 0 TO 6, LENGTH() = 7

8.1.2 Creating strings: literal vs new String()

WAYS TO CREATE A STRING

CreateStrings.java

```
public class CreateStrings {
    public static void main(String[] args) {
        String s1 = "Learn";                  // literal (preferred)
        String s2 = new String("Learn");      // explicit object (rarely needed)
        char[] letters = {'J', 'a', 'v', 'a'};
        String s3 = new String(letters);      // from a char array
        String s4 = String.valueOf(2026);     // from a number
        String s5 = "";                       // empty string, length 0
        System.out.println(s1 + " " + s2 + " " + s3 + " " + s4 + " [" + s5 + "] " + s5.length());
    }
}
```

OUTPUT

```
Learn Learn Java 2026 [] 0
```

8.1.3 String Constant Pool (SCP)

Programs use the same text again and again ("OK", "Yes", "name"). To save memory, the JVM keeps a special area of the heap called the **String Constant Pool**. When you write a literal, the JVM first looks in the pool: if the same text is already there, it **reuses** that object; otherwise it creates one in the pool. `new String("…")` always creates a **new** object outside the pool.

8.1.4 Memory diagram of literal vs new

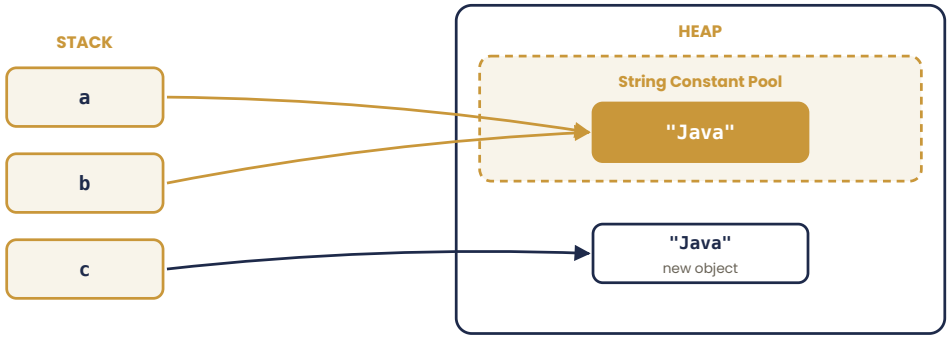
LITERALS SHARE ONE POOLED OBJECT

PoolDemo.java

```
public class PoolDemo {
    public static void main(String[] args) {
        String a = "Java";                // created in the pool
        String b = "Java";                // reuses the pooled object
        String c = new String("Java");    // new object outside the pool
        System.out.println("a == b : " + (a == b));       // same object
        System.out.println("a == c : " + (a == c));       // different objects
        System.out.println("a.equals(c) : " + a.equals(c)); // same text
    }
}
```

OUTPUT

```
a == b : true
a == c : false
a.equals(c) : true
```



A AND B POINT TO THE SAME POOLED "JAVA"; C POINTS TO A SEPARATE OBJECT

8.2 Immutability

8.2.1 What immutability means

A String object can **never be changed** after it is created. Methods that seem to modify a string, like `toUpperCase()` or `concat()`, actually **return a new String** and leave the original untouched.

8.2.2 Why strings are immutable (security, pooling, thread safety, hashing)

Security

File names, URLs, passwords and class names are strings. If code could change a string after it was checked, it could bypass security checks.

Pooling

The pool can safely share one object among many variables only because nobody can change it.

Thread safety

Immutable objects can be shared between threads with no locking.

Hashing

A String caches its hashCode, and it never changes, so strings make fast, reliable HashMap keys.

8.2.3 What happens on "modification"

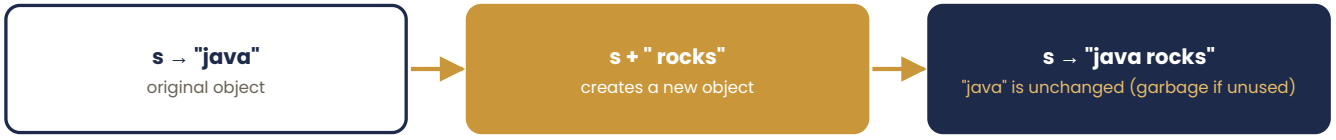
METHODS RETURN NEW STRINGS

Immutable.java

```
public class Immutable {
    public static void main(String[] args) {
        String s = "java";
        s.toUpperCase();                    // result thrown away!
        System.out.println(s);              // still "java"
        String t = s.toUpperCase();         // keep the NEW string
        System.out.println(t);
        s = s + " rocks";                   // s now refers to a NEW object
        System.out.println(s);
    }
}
```

OUTPUT

```
java
JAVA
java rocks
```



RE-ASSIGNMENT MOVES THE REFERENCE; THE OLD OBJECT IS NEVER MODIFIED

8.3 Comparing strings

8.3.1 == (reference comparison)

`==` checks whether two references point to the **same object**. It may be true for two literals (pooled) but false for strings built at runtime, even with identical text. **Never use == to compare string contents.**

8.3.2 equals() and equalsIgnoreCase()

`s1.equals(s2)` compares the **characters**. `equalsIgnoreCase()` ignores upper/lower case, useful for user input like "YES"/"yes". Write the known literal first, `"admin".equals(input)`, so a null input cannot cause a NullPointerException.

8.3.3 compareTo() and compareToIgnoreCase()

`a.compareTo(b)` compares **alphabetically** (lexicographically, by Unicode values). It returns a **negative** number if a comes before b, **0** if equal and a **positive** number if a comes after b. This is what sorting uses.

8.3.4 intern()

`s.intern()` returns the pooled copy of the string's text, adding it to the pool if needed. After interning, equal strings can be compared with ==. It is rarely needed in normal code.

EVERY WAY TO COMPARE STRINGS

CompareStrings.java

```
public class CompareStrings {
    public static void main(String[] args) {
        String a = "hello";
        String b = "hel" + "lo";                // compile-time constant: pooled
        String part = "hel";
        String c = part + "lo";                 // built at runtime: new object
        System.out.println("a == b          : " + (a == b));
        System.out.println("a == c          : " + (a == c));
        System.out.println("a.equals(c)     : " + a.equals(c));
        System.out.println("a == c.intern() : " + (a == c.intern()));
        System.out.println("equalsIgnoreCase: " + "YES".equalsIgnoreCase("yes"));
        System.out.println("apple vs banana : " + "apple".compareTo("banana"));
        System.out.println("pear vs pea     : " + "pear".compareTo("pea"));
        System.out.println("Zebra vs apple  : " + "Zebra".compareTo("apple") + " (uppercase sorts first)");
        System.out.println("ignore case     : " + "Zebra".compareToIgnoreCase("apple"));
        String input = null;
        System.out.println("safe check      : " + "admin".equals(input));
    }
}
```

OUTPUT

```
a == b          : true
a == c          : false
a.equals(c)     : true
a == c.intern() : true
equalsIgnoreCase: true
apple vs banana : -1
pear vs pea     : 1
Zebra vs apple  : -7 (uppercase sorts first)
ignore case     : 25
safe check      : false
```

Method	Compares	Returns
`==`	references (same object?)	boolean
`equals()`	characters, case-sensitive	boolean
`equalsIgnoreCase()`	characters, ignoring case	boolean
`compareTo()`	alphabetical order	int: <0, 0, >0
`compareToIgnoreCase()`	alphabetical, ignoring case	int

8.4 String methods

String has more than 60 methods. These are the ones you will use every day, grouped by purpose.

8.4.1 Length and characters: length(), charAt(), toCharArray(), chars()

GETTING THE CHARACTERS OF A STRING

LengthChars.java

```
public class LengthChars {
    public static void main(String[] args) {
        String s = "Hyderabad";
        System.out.println("length    : " + s.length());
        System.out.println("charAt(0) : " + s.charAt(0));
        System.out.println("last char : " + s.charAt(s.length() - 1));
        char[] arr = s.toCharArray();
        arr[0] = 'h';
        System.out.println("toCharArray changed copy: " + new String(arr) + ", original " + s);
        long vowels = s.chars().filter(ch -> "aeiouAEIOU".indexOf(ch) >= 0).count();
        System.out.println("vowels via chars(): " + vowels);
    }
}
```

OUTPUT

```
length    : 9
charAt(0) : H
last char : d
toCharArray changed copy: hyderabad, original Hyderabad
vowels via chars(): 3
```

8.4.2 Searching: indexOf(), lastIndexOf(), contains(), startsWith(), endsWith()

FINDING TEXT INSIDE A STRING

Searching.java

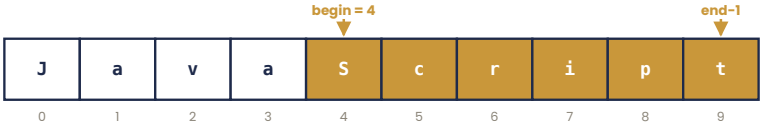
```
public class Searching {
    public static void main(String[] args) {
        String email = "aarav.sharma@learnbeingforward.in";
        System.out.println("indexOf('@')     : " + email.indexOf('@'));
        System.out.println("indexOf(\"a\")     : " + email.indexOf("a"));
        System.out.println("indexOf(\"a\", 5)  : " + email.indexOf("a", 5));
        System.out.println("lastIndexOf('.') : " + email.lastIndexOf('.'));
        System.out.println("indexOf(\"xyz\")   : " + email.indexOf("xyz") + " (not found)");
        System.out.println("contains(\"being\"): " + email.contains("being"));
        System.out.println("startsWith(\"aarav\"): " + email.startsWith("aarav"));
        System.out.println("endsWith(\".in\")  : " + email.endsWith(".in"));
    }
}
```

OUTPUT

```
indexOf('@')     : 12
indexOf("a")     : 0
indexOf("a", 5)  : 8
lastIndexOf('.') : 30
indexOf("xyz")   : -1 (not found)
contains("being"): true
startsWith("aarav"): true
endsWith(".in")  : true
```

8.4.3 Extracting: substring(), split()

`substring(begin)` returns from `begin` to the end; `substring(begin, end)` returns from begin **up to but not including** end, so its length is `end − begin`. `split(regex)` cuts a string into an array at every match of a delimiter.



"JAVASCRIPT".SUBSTRING(4) → "SCRIPT"; SUBSTRING(0, 4) → "JAVA"

SUBSTRING AND SPLIT

Extracting.java

```
import java.util.Arrays;

public class Extracting {
    public static void main(String[] args) {
        String s = "JavaScript";
        System.out.println(s.substring(4));          // Script
        System.out.println(s.substring(0, 4));       // Java
        String email = "diya@lbf.in";
        String user = email.substring(0, email.indexOf('@'));
        String domain = email.substring(email.indexOf('@') + 1);
        System.out.println(user + " | " + domain);

        String csv = "Aarav,21,Hyderabad,88";
        String[] parts = csv.split(",");
        System.out.println(Arrays.toString(parts) + " -> " + parts.length + " parts");
        String sentence = "  Java   is    fun ";
        System.out.println(Arrays.toString(sentence.trim().split("\\s+")));
        System.out.println(Arrays.toString("a.b.c".split("\\.")));   // . must be escaped
    }
}
```

OUTPUT

```
Script
Java
diya | lbf.in
[Aarav, 21, Hyderabad, 88] -> 4 parts
[Java, is, fun]
[a, b, c]
```

WATCH OUT

`split` takes a **regular expression**. Characters like `. | * + ?` have special meaning, so escape them: `split("\\.")`, `split("\\|")`.

8.4.4 Changing case and whitespace: toUpperCase(), toLowerCase(), trim(), strip()

`trim()` removes leading and trailing spaces (characters ≤ \u0020). `strip()` (Java 11) does the same but understands all Unicode whitespace; `stripLeading()` and `stripTrailing()` remove one side only.

CASE AND WHITESPACE

CaseAndSpace.java

```
public class CaseAndSpace {
    public static void main(String[] args) {
        String name = "  Rohan Verma  ";
        System.out.println("[" + name.toUpperCase() + "]");
        System.out.println("[" + name.toLowerCase() + "]");
        System.out.println("[" + name.trim() + "]");
        System.out.println("[" + name.strip() + "]");
        System.out.println("[" + name.stripLeading() + "]");
        System.out.println("[" + name.stripTrailing() + "]");
    }
}
```

OUTPUT

```
[  ROHAN VERMA  ]
[  rohan verma  ]
[Rohan Verma]
[Rohan Verma]
[Rohan Verma  ]
[  Rohan Verma]
```

8.4.5 Replacing: replace(), replaceAll(), replaceFirst()

`replace(old, new)` replaces **every** occurrence of a char or literal text. `replaceAll(regex, new)` and `replaceFirst(regex, new)` use regular expressions.

REPLACING TEXT

Replacing.java

```
public class Replacing {
    public static void main(String[] args) {
        String s = "I like tea. Tea is good. tea time!";
        System.out.println(s.replace("tea", "coffee"));
        System.out.println(s.replace('.', '!'));
        System.out.println(s.replaceFirst("[Tt]ea", "COFFEE"));
        System.out.println(s.replaceAll("[Tt]ea", "coffee"));
        System.out.println("Ph: 98480-111-11".replaceAll("[^0-9]", ""));   // keep digits only
    }
}
```

OUTPUT

```
I like coffee. Tea is good. coffee time!
I like tea! Tea is good! tea time!
I like COFFEE. Tea is good. tea time!
I like coffee. coffee is good. coffee time!
9848011111
```

8.4.6 Checking: isEmpty(), isBlank(), matches()

`isEmpty()` is true when the length is 0. `isBlank()` (Java 11) is also true for strings of only whitespace. `matches(regex)` checks whether the **whole** string matches a pattern, which is great for validation.

CHECKING CONTENT

Checking.java

```
public class Checking {
    public static void main(String[] args) {
        String[] tests = {"", "   ", "Java"};
        for (String t : tests) {
            System.out.println("[" + t + "] isEmpty=" + t.isEmpty() + " isBlank=" + t.isBlank());
        }
        System.out.println("9848011111 valid mobile? " + "9848011111".matches("[6-9]\\d{9}"));
        System.out.println("12345 valid mobile?      " + "12345".matches("[6-9]\\d{9}"));
        System.out.println("TS09AB1234 plate?        " + "TS09AB1234".matches("[A-Z]{2}\\d{2}[A-Z]{2}\\d{4}"));
    }
}
```

OUTPUT

```
[] isEmpty=true isBlank=true
[   ] isEmpty=false isBlank=true
[Java] isEmpty=false isBlank=false
9848011111 valid mobile? true
12345 valid mobile?      false
TS09AB1234 plate?        true
```

8.4.7 Joining and repeating: concat(), String.join(), repeat()

JOINING AND REPEATING

JoinRepeat.java

```
import java.util.List;

public class JoinRepeat {
    public static void main(String[] args) {
        System.out.println("Learn".concat(" Being").concat(" Forward"));
        System.out.println(String.join(" | ", "Java", "SQL", "Python"));
        System.out.println(String.join(", ", List.of("Aarav", "Diya", "Rohan")));
        System.out.println("=".repeat(20));
        System.out.println("ab".repeat(3));
    }
}
```

OUTPUT

```
Learn Being Forward
Java | SQL | Python
Aarav, Diya, Rohan
====================
ababab
```

8.4.8 Converting: valueOf(), toString()

`String.valueOf(x)` converts any value (int, double, char, boolean, char[], object) to a String, and returns "null" for a null object instead of crashing. Every object also has a `toString()` method.

TURNING VALUES INTO STRINGS Converting.java

```
public class Converting {
    public static void main(String[] args) {
        String a = String.valueOf(42);
        String b = String.valueOf(3.5);
        String c = String.valueOf(true);
        String d = String.valueOf(new char[] {'h', 'i'});
        Object nothing = null;
        String e = String.valueOf(nothing);
        Integer num = 7;
        System.out.println(a + " " + b + " " + c + " " + d + " " + e + " " + num.toString());
        System.out.println((a + 1) + " vs " + (Integer.parseInt(a) + 1));
    }
}
```

OUTPUT

```
42 3.5 true hi null 7
421 vs 43
```

8.5 StringBuilder and StringBuffer

8.5.1 Why mutable strings are needed (performance in loops)

Because Strings are immutable, `s = s + x` inside a loop creates a **brand-new** String every time and copies all the old characters into it. For n iterations this copies about n²/2 characters. `StringBuilder` keeps one growable character buffer and appends in place.



RELATIVE WORK FOR BUILDING A LONG STRING IN A LOOP (ILLUSTRATIVE)

8.5.2 StringBuilder methods: append(), insert(), delete(), deleteCharAt(), reverse(), setCharAt()

EDITING TEXT IN PLACE

BuilderMethods.java

```
public class BuilderMethods {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.append(" is").append(" fun");            // chaining
        System.out.println(sb);
        sb.insert(0, "Core ");
        System.out.println(sb);
        sb.delete(0, 5);                            // remove "Core "
        System.out.println(sb);
        sb.deleteCharAt(sb.length() - 1);           // remove last char
        System.out.println(sb);
        sb.setCharAt(0, 'j');
        System.out.println(sb);
        sb.replace(0, 4, "Python");
        System.out.println(sb);
        System.out.println(new StringBuilder("racecar").reverse());
        String result = sb.toString();              // back to an immutable String
        System.out.println(result.length());
    }
}
```

OUTPUT

```
Java is fun
Core Java is fun
Java is fun
Java is fu
java is fu
Python is fu
racecar
12
```

8.5.3 Capacity and growth

A StringBuilder has a **length** (characters used) and a **capacity** (space reserved). The default capacity is 16. When an append needs more space, it allocates a bigger buffer, about **(old capacity × 2) + 2**, and copies the characters. If you know the final size, pass it to the constructor to avoid repeated growth.

WATCHING THE CAPACITY GROW

Capacity.java

```
public class Capacity {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        System.out.println("length=" + sb.length() + " capacity=" + sb.capacity());
        sb.append("1234567890123456");          // 16 chars: still fits
        System.out.println("length=" + sb.length() + " capacity=" + sb.capacity());
        sb.append("x");                         // 17th char: grows
        System.out.println("length=" + sb.length() + " capacity=" + sb.capacity());
        StringBuilder big = new StringBuilder(1000);
        System.out.println("pre-sized capacity=" + big.capacity());
    }
}
```

OUTPUT

```
length=0 capacity=16
length=16 capacity=16
length=17 capacity=34
pre-sized capacity=1000
```

8.5.4 StringBuffer and synchronization

`StringBuffer` has exactly the same methods as StringBuilder, but every method is `synchronized`, so it is safe to use from several threads at once. That locking makes it slower. It dates from Java 1.0; StringBuilder was added in Java 5 as the faster single-threaded version.

8.5.5 String vs StringBuilder vs StringBuffer comparison

	String	StringBuilder	StringBuffer
Mutable?	no	yes	yes
Thread-safe?	yes (immutable)	no	yes (synchronized)
Speed for many edits	slow	**fastest**	slower than Builder
Since	Java 1.0	Java 5	Java 1.0
Use when	text does not change	building text in one thread (most cases)	shared between threads (rare)

TIMING 100,000 CONCATENATIONS (EXACT TIMES VARY BY MACHINE) ConcatTiming.java

```
public class ConcatTiming {
    public static void main(String[] args) {
        int n = 100_000;
        long t1 = System.nanoTime();
        String s = "";
        for (int i = 0; i < n; i++) s += "x";
        long t2 = System.nanoTime();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append("x");
        long t3 = System.nanoTime();
        StringBuffer sf = new StringBuffer();
        for (int i = 0; i < n; i++) sf.append("x");
        long t4 = System.nanoTime();
        long strMs = (t2 - t1) / 1_000_000, sbMs = (t3 - t2) / 1_000_000;
        System.out.println("lengths: " + s.length() + " " + sb.length() + " " + sf.length());
        System.out.println("String += was slower than StringBuilder? " + (strMs > sbMs));
        System.out.println("String += took more than 10x as long?   " + (strMs > 10 * Math.max(1, sbMs)));
    }
}
```

OUTPUT

```
lengths: 100000 100000 100000
String += was slower than StringBuilder? true
String += took more than 10x as long?   true
```

NOTE

On a typical laptop the String version takes hundreds of milliseconds while StringBuilder takes about 1 millisecond. Print `strMs` and `sbMs` yourself to see the numbers on your machine.

8.6 Characters and the Character class

8.6.1 isDigit(), isLetter(), isUpperCase(), isWhitespace(), toUpperCase()

Method	True for	Example
`Character.isDigit(c)`	0–9	`isDigit('7')` → true
`Character.isLetter(c)`	letters	`isLetter('x')` → true
`Character.isLetterOrDigit(c)`	letters and digits	`'_'` → false
`Character.isUpperCase(c)` / `isLowerCase`	case check	`isUpperCase('Q')` → true
`Character.isWhitespace(c)`	space, tab, newline	`' '` → true
`Character.toUpperCase(c)` / `toLowerCase`	(converts)	`'a'` → 'A'
`Character.getNumericValue(c)`	(digit value)	`'8'` → 8

CLASSIFYING CHARACTERS

CharClass.java

```
public class CharClass {
    public static void main(String[] args) {
        String pwd = "Lbf@2026 ok";
        int upper = 0, lower = 0, digit = 0, space = 0, special = 0;
        for (char c : pwd.toCharArray()) {
            if (Character.isUpperCase(c)) upper++;
            else if (Character.isLowerCase(c)) lower++;
            else if (Character.isDigit(c)) digit++;
            else if (Character.isWhitespace(c)) space++;
            else special++;
        }
        System.out.printf("upper=%d lower=%d digit=%d space=%d special=%d%n", upper, lower, digit, space, special);
        System.out.println(Character.toUpperCase('j') + "" + Character.toLowerCase('V'));
    }
}
```

OUTPUT

```
upper=1 lower=4 digit=4 space=1 special=1
Jv
```

8.6.2 ASCII arithmetic on chars

Characters are numbers, so arithmetic moves between them. Upper- and lowercase letters are exactly 32 apart ('a' 97 - 'A' 65). Subtracting '0' turns a digit character into its value. Subtracting 'a' turns a lowercase letter into a position 0–25, the basis of frequency arrays and the Caesar cipher.

ARITHMETIC WITH CHARACTERS

CharMath.java

```
public class CharMath {
    public static void main(String[] args) {
        System.out.println((char) ('a' - 32));          // A
        System.out.println('7' - '0');                  // 7
        System.out.println('d' - 'a');                  // 3 (position in alphabet)
        // Caesar cipher: shift each letter by 3
        String msg = "attack at dawn";
        StringBuilder enc = new StringBuilder();
        for (char c : msg.toCharArray()) {
            if (Character.isLowerCase(c)) enc.append((char) ('a' + (c - 'a' + 3) % 26));
            else enc.append(c);
        }
        System.out.println(enc);
    }
}
```

OUTPUT

```
A
7
3
dwwdfn dw gdzq
```

8.7 Advanced string features

8.7.1 Text blocks """ (Java 15+)

A **text block** is a multi-line string literal between triple quotes. You can write JSON, SQL or HTML exactly as it looks, without `\n` and escaped quotes. The common leading indentation is removed automatically.

MULTI-LINE TEXT WITHOUT ESCAPES

TextBlocks.java

```
public class TextBlocks {
    public static void main(String[] args) {
        String json = """
            {
              "name": "Aarav",
              "course": "Java",
              "marks": 88
            }
            """;
        String sql = """
            SELECT name, marks
            FROM students
            WHERE marks > 80""";
        System.out.print(json);
        System.out.println(sql);
    }
}
```

OUTPUT

```
{
  "name": "Aarav",
  "course": "Java",
  "marks": 88
}
SELECT name, marks
FROM students
WHERE marks > 80
```

8.7.2 Regular expressions basics (Pattern, Matcher)

A **regular expression (regex)** is a pattern that describes text. `Pattern.compile(regex)` compiles the pattern once; `matcher(text)` searches a string; `find()` moves to each match and `group()` returns it.

Regex	Matches
`\\d`	one digit
`\\w`	a letter, digit or _
`\\s`	whitespace
`[A-Z]`	one capital letter
`[^0-9]`	anything except a digit
`+ * ? {n} {n,m}`	one or more, zero or more, optional, exactly n, n to m times
`^ $`	start / end of the text
`( )`	a group you can extract

FINDING ALL MOBILE NUMBERS AND EMAILS RegexDemo.java

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class RegexDemo {
    public static void main(String[] args) {
        String text = "Call 9848011111 or 9000012345. Office: 040-2345678.";
        Pattern mobile = Pattern.compile("[6-9]\\d{9}");
        Matcher m = mobile.matcher(text);
        while (m.find()) {
            System.out.println("mobile: " + m.group() + " at index " + m.start());
        }
        Pattern email = Pattern.compile("(\\w+)@(\\w+)\\.in");
        Matcher e = email.matcher("contact: priya@lbf.in, rohan@college.in");
        while (e.find()) {
            System.out.println("user=" + e.group(1) + " domain=" + e.group(2));
        }
    }
}
```

OUTPUT

```
mobile: 9848011111 at index 5
mobile: 9000012345 at index 19
user=priya domain=lbf
user=rohan domain=college
```

8.7.3 StringTokenizer (legacy)

`java.util.StringTokenizer` splits a string into tokens using delimiter characters. It predates `split()` and regex; it is still found in old code and is fast, but new code should use `split()` or Scanner.

SPLITTING WITH STRINGTOKENIZER TokenizerDemo.java

```
import java.util.StringTokenizer;

public class TokenizerDemo {
    public static void main(String[] args) {
        StringTokenizer st = new StringTokenizer("Java,SQL;Python Spring", ",; ");
        System.out.println("tokens: " + st.countTokens());
        while (st.hasMoreTokens()) {
            System.out.println("  " + st.nextToken());
        }
    }
}
```

OUTPUT

```
tokens: 4
  Java
  SQL
  Python
  Spring
```

8.8 Classic string problems

8.8.1 Reverse a string and reverse words

FOUR KINDS OF REVERSAL

ReverseProblems.java

```
public class ReverseProblems {
    public static void main(String[] args) {
        String s = "Learn Being Forward";
        // 1. loop from the end
        String rev = "";
        for (int i = s.length() - 1; i >= 0; i--) rev += s.charAt(i);
        System.out.println(rev);
        // 2. StringBuilder
        System.out.println(new StringBuilder(s).reverse());
        // 3. reverse the order of words
        String[] words = s.split(" ");
        StringBuilder out = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            out.append(words[i]);
            if (i > 0) out.append(' ');
        }
        System.out.println(out);
        // 4. reverse each word, keep the order
        StringBuilder each = new StringBuilder();
        for (String w : words) each.append(new StringBuilder(w).reverse()).append(' ');
        System.out.println(each.toString().trim());
    }
}
```

OUTPUT

```
drawroF gnieB nraeL
drawroF gnieB nraeL
Forward Being Learn
nraeL gnieB drawroF
```

8.8.2 Palindrome and anagram check

A **palindrome** reads the same backwards ("madam", "Never odd or even" ignoring spaces and case). Two words are **anagrams** if they use the same letters the same number of times ("listen"/"silent").

TWO-POINTER PALINDROME AND COUNTING ANAGRAM CHECK

PalindromeAnagram.java

```
import java.util.Arrays;

public class PalindromeAnagram {
    static boolean isPalindrome(String s) {
        String clean = s.toLowerCase().replaceAll("[^a-z0-9]", "");
        int i = 0, j = clean.length() - 1;
        while (i < j) {
            if (clean.charAt(i++) != clean.charAt(j--)) return false;
        }
        return true;
    }
    static boolean isAnagram(String a, String b) {
        int[] count = new int[26];
        a = a.toLowerCase().replace(" ", "");
        b = b.toLowerCase().replace(" ", "");
        if (a.length() != b.length()) return false;
        for (int i = 0; i < a.length(); i++) {
            count[a.charAt(i) - 'a']++;
            count[b.charAt(i) - 'a']--;
        }
        for (int c : count) if (c != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        for (String s : new String[] {"Madam", "Never odd or even", "Java"})
            System.out.println("'" + s + "' palindrome? " + isPalindrome(s));
        System.out.println("listen/silent anagram? " + isAnagram("listen", "silent"));
        System.out.println("Dormitory/dirty room? " + isAnagram("Dormitory", "dirty room"));
        System.out.println("hello/world anagram? " + isAnagram("hello", "world"));
    }
}
```

OUTPUT

```
'Madam' palindrome? true
'Never odd or even' palindrome? true
'Java' palindrome? false
listen/silent anagram? true
Dormitory/dirty room? true
hello/world anagram? false
```

8.8.3 Count vowels, consonants, digits, spaces

COUNTING KINDS OF CHARACTERS

CountKinds.java

```
public class CountKinds {
    public static void main(String[] args) {
        String s = "Java 21 was released in 2023!";
        int v = 0, c = 0, d = 0, sp = 0, other = 0;
        for (char ch : s.toLowerCase().toCharArray()) {
            if ("aeiou".indexOf(ch) >= 0) v++;
            else if (ch >= 'a' && ch <= 'z') c++;
            else if (Character.isDigit(ch)) d++;
            else if (ch == ' ') sp++;
            else other++;
        }
        System.out.printf("vowels=%d consonants=%d digits=%d spaces=%d others=%d%n", v, c, d, sp, other);
    }
}
```

OUTPUT

```
vowels=8 consonants=9 digits=6 spaces=5 others=1
```

8.8.4 Character frequency and first non-repeating character

COUNTING WITH AN ARRAY AND WITH A MAP

Frequency.java

```
import java.util.LinkedHashMap;
import java.util.Map;

public class Frequency {
    public static void main(String[] args) {
        String s = "programming";
        int[] freq = new int[26];
        for (char ch : s.toCharArray()) freq[ch - 'a']++;
        for (int i = 0; i < 26; i++)
            if (freq[i] > 0) System.out.print((char) ('a' + i) + "=" + freq[i] + " ");
        System.out.println();
        // first non-repeating character: second pass in the original order
        for (char ch : s.toCharArray()) {
            if (freq[ch - 'a'] == 1) { System.out.println("first non-repeating: " + ch); break; }
        }
        // same with a LinkedHashMap (keeps insertion order, works for any characters)
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char ch : "swiss".toCharArray()) map.merge(ch, 1, Integer::sum);
        System.out.println("counts in swiss: " + map);
    }
}
```

OUTPUT

```
a=1 g=2 i=1 m=2 n=1 o=1 p=1 r=2
first non-repeating: p
counts in swiss: {s=3, w=1, i=1}
```

8.8.5 Remove duplicates, toggle case, string compression

THREE MORE CLASSICS

MoreStringProblems.java

```
public class MoreStringProblems {
    public static void main(String[] args) {
        // remove duplicate characters, keep first occurrence
        String s = "programming";
        StringBuilder uniq = new StringBuilder();
        for (char ch : s.toCharArray())
            if (uniq.indexOf(String.valueOf(ch)) == -1) uniq.append(ch);
        System.out.println("remove duplicates: " + uniq);

        // toggle case
        String t = "LeArN JaVa";
        StringBuilder tog = new StringBuilder();
        for (char ch : t.toCharArray())
            tog.append(Character.isUpperCase(ch) ? Character.toLowerCase(ch) : Character.toUpperCase(ch));
        System.out.println("toggle case: " + tog);

        // run-length compression: aaabbcccc -> a3b2c4
        String r = "aaabbcccca";
        StringBuilder comp = new StringBuilder();
        int count = 1;
        for (int i = 1; i <= r.length(); i++) {
            if (i < r.length() && r.charAt(i) == r.charAt(i - 1)) count++;
            else { comp.append(r.charAt(i - 1)).append(count); count = 1; }
        }
        System.out.println("compressed: " + comp);
    }
}
```

OUTPUT

```
remove duplicates: progamin
toggle case: lEaRn jAvA
compressed: a3b2c4a1
```

PRACTICE CHECKPOINT

An **anagram checker**, a **word counter**, and a **timing comparison** of String vs StringBuilder concatenation in a 100,000-iteration loop.

SAMPLE SOLUTION

StringsCheckpoint.java

```
import java.util.Arrays;

public class StringsCheckpoint {
    public static void main(String[] args) {
        // 1. anagram checker (sorting method)
        String a = "Triangle", b = "Integral";
        char[] x = a.toLowerCase().toCharArray(), y = b.toLowerCase().toCharArray();
        Arrays.sort(x); Arrays.sort(y);
        System.out.println(a + " / " + b + " anagrams? " + Arrays.equals(x, y));

        // 2. word counter
        String text = "  Java is simple and Java is powerful  ";
        String[] words = text.trim().split("\\s+");
        System.out.println("words: " + words.length);

        // 3. timing comparison
        long t1 = System.nanoTime();
        String s = "";
        for (int i = 0; i < 100_000; i++) s += "a";
        long t2 = System.nanoTime();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 100_000; i++) sb.append("a");
        long t3 = System.nanoTime();
        System.out.println("StringBuilder faster? " + ((t3 - t2) < (t2 - t1)));
    }
}
```

OUTPUT

```
Triangle / Integral anagrams? true
words: 7
StringBuilder faster? true
```

MODULE 09 · BEGINNER

Methods and recursion

Minimum duration **3 days** | Level **Beginner**

Methods package reusable logic under a name; recursion is a method solving a problem by calling itself on a smaller input.

IN THIS MODULE

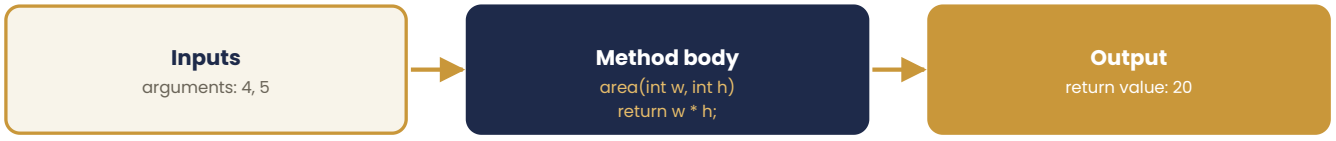
9.1 Introduction to methods
9.2 Defining and calling methods
9.3 Parameters and arguments
9.4 Return values
9.5 Types of methods
9.6 Method overloading
9.7 Recursion

9.1 Introduction to methods

9.1.1 What a method is and why (reuse, readability, testing)

A **method** is a named block of code that does one job. You write it once and **call** it whenever that job is needed. `main` is itself a method, and you have already called many library methods such as `System.out.println()`, `Math.max()` and `s.length()`.

Think of a method like a machine in a factory: you feed raw material in (the **arguments**), the machine does its work (the **body**) and something comes out (the **return value**).



A METHOD IS A SMALL MACHINE: INPUT → WORK → OUTPUT

Reuse
Write the logic once, call it from 10 places. A bug fix is made in one place only.

Readability
`if (isPrime(n))` reads like English. The details hide inside the method.

Testing
A small method with clear input and output can be tested on its own.

Decomposition
A big problem is broken into small named steps: readInput(), compute(), printReport().

ONE METHOD, REUSED THREE TIMES

WhyMethods.java

```
public class WhyMethods {
    // without methods we would repeat this loop three times
    static void printLine(char ch, int n) {
        for (int i = 0; i < n; i++) System.out.print(ch);
        System.out.println();
    }

    public static void main(String[] args) {
        printLine('=', 30);
        System.out.println("  LEARN BEING FORWARD REPORT");
        printLine('-', 30);
        System.out.println("  Students: 42   Passed: 39");
        printLine('=', 30);
    }
}
```

OUTPUT

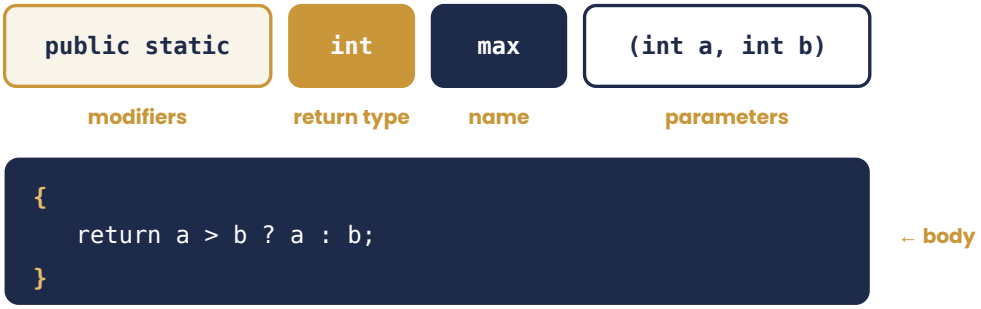
```
==============================
  LEARN BEING FORWARD REPORT
------------------------------
  Students: 42   Passed: 39
==============================
```

9.1.2 Method signature: name + parameter list

The **signature** of a method is its **name together with the types and order of its parameters**. The compiler uses the signature to tell methods apart. The return type, the parameter names and the modifiers are **not** part of the signature.

Method declaration	Signature
`static int add(int a, int b)`	`add(int, int)`
`static double add(double x, double y)`	`add(double, double)`
`public void print(String msg, int times)`	`print(String, int)`
`static long add(int p, int q)`	`add(int, int)` : same as the first, so it is a compile error

9.1.3 Method syntax: modifiers, return type, name, parameters, body



ANATOMY OF A METHOD DECLARATION

Part	Meaning	Examples
Modifiers	Who can call it (access) and how it belongs (static, final, abstract…)	`public`, `private`, `static`
Return type	Type of the value sent back, or `void` for nothing	`int`, `String`, `int[]`, `void`
Name	A verb in camelCase describing the job	`calculateTotal`, `isEven`
Parameters	Comma-separated *type name* pairs; may be empty `()`	`(int a, int b)`
Body	Statements inside `{ }`; must `return` a value if the type is not void	`{ return a + b; }`

TIP
Name methods with verbs (`print` , `calculate` , `find`) and boolean methods with `is` / `has` / `can` (`isValid` , `hasNext`). Keep each method short: if you cannot describe it in one sentence, split it.

9.2 Defining and calling methods

9.2.1 Calling a static method

A `static` method belongs to the **class**, so it can be called without creating an object. Inside the same class you call it by name; from another class you write `ClassName.method()` , as in `Math.sqrt(16)` .

CALLING STATIC METHODS StaticCall.java

```
public class StaticCall {
    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        int a = square(7);                          // same class: just the name
        int b = StaticCall.square(3);               // full form also works
        double r = Math.sqrt(a + b);                // another class: ClassName.method
        System.out.println("square(7) = " + a);
        System.out.println("square(3) = " + b);
        System.out.println("sqrt(58)  = " + r);
    }
}
```

OUTPUT
```
square(7) = 49
square(3) = 9
sqrt(58)  = 7.615773105863909
```

9.2.2 Calling an instance method through an object

A method **without** `static` is an **instance method**: it belongs to each object and can use that object's fields. You must first create an object with `new` , then call `object.method()` .

INSTANCE METHODS WORK ON ONE PARTICULAR OBJECT Counter.java

```
public class Counter {
    int count = 0;                      // each object has its own count

    void increment() { count++; }         // instance method
    int getCount()   { return count; }

    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        c1.increment(); c1.increment(); c1.increment();
        c2.increment();
        System.out.println("c1 = " + c1.getCount());
        System.out.println("c2 = " + c2.getCount());
    }
}
```

OUTPUT
```
c1 = 3
c2 = 1
```

WATCH OUT
Calling an instance method directly from `static main` without an object gives the error *"non-static method cannot be referenced from a static context"*. Either make the method `static` or create an object first.

9.2.3 Method call stack and stack frames

Each time a method is called, the JVM pushes a **stack frame** onto the thread's **call stack**. The frame holds that call's parameters, local variables and the return address (where to continue afterwards). When the method returns, its frame is **popped** and its local variables disappear. The method on top of the stack is the one currently running.

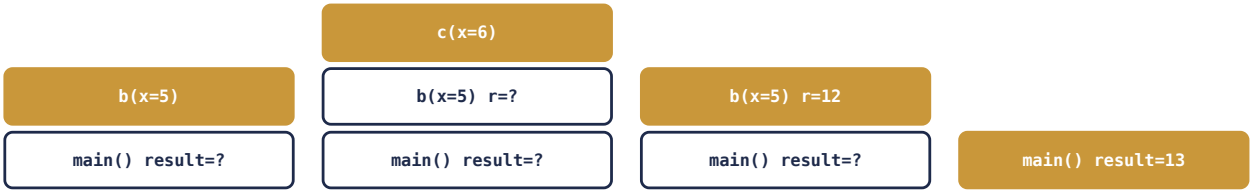
WATCHING CALLS GO IN AND COME BACK OUT

CallStackDemo.java

```
public class CallStackDemo {
    static int c(int x) {
        System.out.println("    in c(" + x + ")");
        return x * 2;
    }
    static int b(int x) {
        System.out.println("  in b(" + x + ") -> calling c");
        int r = c(x + 1);
        System.out.println("  back in b, c returned " + r);
        return r + 1;
    }
    public static void main(String[] args) {
        System.out.println("main -> calling b");
        int result = b(5);
        System.out.println("back in main, result = " + result);
    }
}
```

OUTPUT

```
main -> calling b
  in b(5) -> calling c
    in c(6)
  back in b, c returned 12
back in main, result = 13
```



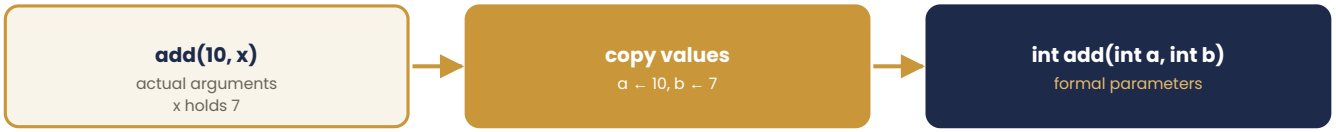
THE CALL STACK AT FOUR MOMENTS: FRAMES ARE PUSHED ON CALL AND POPPED ON RETURN

Frames are **last in, first out**: the last method called is the first to finish. An exception's stack trace is simply a printout of this stack at the moment of the error.

9.3 Parameters and arguments

9.3.1 Formal parameters vs actual arguments

Formal parameters are the variables in the method declaration: `int a, int b`. **Actual arguments** are the values you pass when calling: `add(10, x)`. At the call, each argument is evaluated and **copied** into the matching parameter, left to right. The number, order and (compatible) types must match.



ARGUMENTS ARE COPIED INTO PARAMETERS

9.3.2 Java is always pass-by-value

Java has exactly one rule: **the value of the argument is copied**. There is no pass-by-reference in Java. What looks different between primitives and objects is only *what* the copied value is.

9.3.2.1 Passing primitives

For a primitive, the copied value is the number itself. Changing the parameter inside the method changes only the copy; the caller's variable is untouched.

THE CALLER'S PRIMITIVE DOES NOT CHANGE PassPrimitive.java

```
public class PassPrimitive {
    static void tryToChange(int n) {
        n = n * 100;                    // changes the copy only
        System.out.println("inside method n = " + n);
    }
    public static void main(String[] args) {
        int marks = 5;
        tryToChange(marks);
        System.out.println("after call marks = " + marks);
    }
}
```

OUTPUT

```
inside method n = 500
after call marks = 5
```

9.3.2.2 Passing object references

For an object, the variable holds a **reference** (an address). The **reference is copied**, so the parameter points to the **same object**. Therefore the method **can change the object's contents**, but if it re-assigns the parameter to a new object, the caller does not see that.

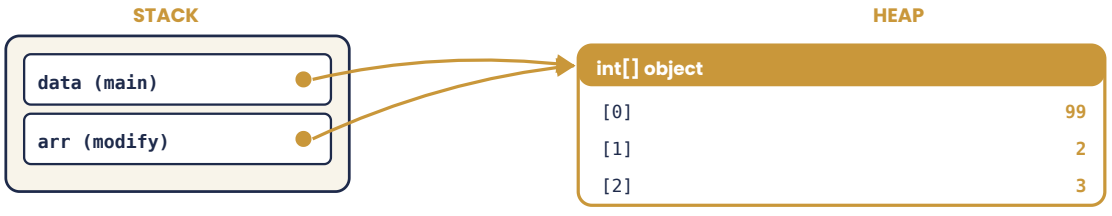
CONTENTS CHANGE, RE-ASSIGNMENT DOES NOT PassReference.java

```
import java.util.Arrays;

public class PassReference {
    static void modify(int[] arr) {
        arr[0] = 99;                    // changes the shared object: visible to caller
    }
    static void reassign(int[] arr) {
        arr = new int[] {7, 7, 7};      // parameter now points elsewhere: caller unaffected
        arr[0] = -1;
    }
    public static void main(String[] args) {
        int[] data = {1, 2, 3};
        modify(data);
        System.out.println("after modify   : " + Arrays.toString(data));
        reassign(data);
        System.out.println("after reassign : " + Arrays.toString(data));
    }
}
```

OUTPUT

```
after modify   : [99, 2, 3]
after reassign : [99, 2, 3]
```



BOTH REFERENCES POINT TO ONE ARRAY, SO ARR[0] = 99 IS SEEN THROUGH DATA

Pass-by-value of the reference: you can change the object through the copy, but you cannot make the caller's variable point to a different object.

9.3.3 Variable arguments (varargs) int... nums

Sometimes the number of arguments is not fixed: `sum(1, 2)`, `sum(4, 5, 6, 7)`. Declaring a parameter as `int... nums` lets the caller pass **zero or more** ints. Inside the method, `nums` is an ordinary `int[]`. `printf` and `String.format` use varargs.

- Only **one** varargs parameter is allowed, and it must be the **last** parameter.
- You can also pass an existing array: `sum(new int[]{1, 2, 3})`.
- A fixed-parameter overload is preferred over varargs when both match.

VARARGS: ANY NUMBER OF ARGUMENTS VarargsDemo.java

```
public class VarargsDemo {
    static int sum(int... nums) {
        int total = 0;
        for (int n : nums) total += n;
        return total;
    }
    static void greet(String greeting, String... names) {
        for (String n : names) System.out.println(greeting + ", " + n + "!");
    }
    public static void main(String[] args) {
        System.out.println("sum()           = " + sum());
        System.out.println("sum(5)          = " + sum(5));
        System.out.println("sum(1, 2, 3, 4) = " + sum(1, 2, 3, 4));
        System.out.println("sum(array)      = " + sum(new int[] {10, 20}));
        greet("Hello", "Asha", "Ravi");
    }
}
```

OUTPUT

```
sum()           = 0
sum(5)          = 5
sum(1, 2, 3, 4) = 10
sum(array)      = 30
Hello, Asha!
Hello, Ravi!
```

9.4 Return values

9.4.1 void methods

A `void` method performs an action (printing, changing an object, saving to a file) and sends **nothing** back. It ends when the closing brace is reached or at a bare `return;`. You cannot use its call in an expression: `int x = printLine();` does not compile.

9.4.2 Returning a value

A method with a return type must send a value of that type back with `return expression;` on **every** path. The call expression is then replaced by the returned value, so it can be stored, printed or used in another expression.

METHODS THAT RETURN VALUES

ReturnDemo.java

```java
public class ReturnDemo {
    static double average(int a, int b, int c) {
        return (a + b + c) / 3.0;
    }
    static boolean isEven(int n) {
        return n % 2 == 0;
    }
    static String grade(int marks) {
        if (marks >= 90) return "A";
        else if (marks >= 75) return "B";
        else if (marks >= 50) return "C";
        else return "F";                        // every path returns
    }
    public static void main(String[] args) {
        double avg = average(80, 92, 71);
        System.out.printf("average = %.2f%n", avg);
        System.out.println("isEven(14) = " + isEven(14));
        System.out.println("grade(" + (int) avg + ") = " + grade((int) avg));
        System.out.println("nested: " + grade((int) average(95, 91, 99)));
    }
}
```

OUTPUT

```
average = 81.00
isEven(14) = true
grade(81) = B
nested: A
```

WATCH OUT

"missing return statement" means some path through the method can reach the end without returning. Add a final `return` or an `else` branch.

9.4.3 Returning multiple values (arrays, objects, records)

A method returns **one** value. To send back several, wrap them in one container:

- an **array**, when all values have the same type (e.g. min and max);
- an **object** of a small class with named fields;
- a **record** (Java 16+), a one-line immutable data class: `record MinMax(int min, int max) {}`. This is the modern choice.

RETURNING SEVERAL VALUES IN AN ARRAY AND IN A RECORD

MultipleReturns.java

```
import java.util.Arrays;

public class MultipleReturns {
    record Stats(int min, int max, double avg) {}       // Java 16+

    static int[] minMaxArray(int[] a) {
        int min = a[0], max = a[0];
        for (int v : a) { min = Math.min(min, v); max = Math.max(max, v); }
        return new int[] {min, max};
    }

    static Stats stats(int[] a) {
        int[] mm = minMaxArray(a);
        double sum = 0;
        for (int v : a) sum += v;
        return new Stats(mm[0], mm[1], sum / a.length);
    }

    public static void main(String[] args) {
        int[] marks = {67, 92, 45, 88, 73};
        int[] mm = minMaxArray(marks);
        System.out.println("array  : " + Arrays.toString(mm));
        Stats s = stats(marks);
        System.out.println("record : " + s);
        System.out.println("max via accessor: " + s.max());
    }
}
```

OUTPUT

```
array  : [45, 92]
record : Stats[min=45, max=92, avg=73.0]
max via accessor: 92
```

9.4.4 Early return

`return` can appear anywhere, and the method stops immediately. Using it at the start to reject bad input (a **guard clause**) avoids deep nesting of if-else blocks.

LEAVING A METHOD AS SOON AS THE ANSWER IS KNOWN

EarlyReturn.java

```
public class EarlyReturn {
    static int indexOf(int[] a, int target) {
        for (int i = 0; i < a.length; i++)
            if (a[i] == target) return i;      // found: stop at once
        return -1;                              // not found
    }
    static String classify(int age) {
        if (age < 0) return "invalid";          // guard clause
        if (age < 13) return "child";
        if (age < 20) return "teen";
        return "adult";
    }
    public static void main(String[] args) {
        int[] a = {4, 8, 15, 16, 23, 42};
        System.out.println("indexOf 16 = " + indexOf(a, 16));
        System.out.println("indexOf 5  = " + indexOf(a, 5));
        System.out.println(classify(-3) + " " + classify(9) + " " + classify(17) + " " + classify(30));
    }
}
```

OUTPUT

```
indexOf 16 = 3
indexOf 5  = -1
invalid child teen adult
```

9.5 Types of methods

9.5.1 Predefined (library) methods

The JDK provides thousands of ready-made methods. Use them instead of rewriting the same logic: they are tested, fast and familiar to every Java developer.

Class	Useful methods	Example
`Math`	max, min, abs, pow, sqrt, round, ceil, floor, random	`Math.pow(2, 10)` → 1024.0
`String`	length, charAt, substring, indexOf, toUpperCase	`"abc".toUpperCase()`
`Integer`	parseInt, valueOf, toBinaryString, MAX_VALUE	`Integer.parseInt("42")`
`Arrays`	sort, toString, fill, copyOf, binarySearch	`Arrays.sort(a)`
`Character`	isDigit, isLetter, toUpperCase	`Character.isDigit('7')`

9.5.2 User-defined methods

Methods you write yourself for your program's own logic: `calculateFee()`, `isValidRollNo()`, `printReceipt()`. They follow the same syntax and can call library methods and each other.

9.5.3 Static methods vs instance methods

	static method	instance method
Belongs to	The class (one copy)	Each object
Called as	`ClassName.m()`	`object.m()`
Needs an object?	No	Yes
Can use instance fields / `this`?	No (only through an object reference)	Yes
Can use static fields?	Yes	Yes
Typical use	Utilities: `Math.max`, `Integer.parseInt`, `main`	Behaviour of an object: `account.deposit(500)`

STATIC AND INSTANCE METHODS SIDE BY SIDE

BankAccount.java

```
public class BankAccount {
    static double interestRate = 0.05;      // shared by all accounts
    double balance;                          // each account's own

    static double interestOn(double amount) {   // utility: no object needed
        return amount * interestRate;
    }
    void deposit(double amount) {                // works on THIS account
        balance += amount;
    }

    public static void main(String[] args) {
        System.out.println("interest on 1000 = " + BankAccount.interestOn(1000));
        BankAccount acc = new BankAccount();
        acc.deposit(2500);
        acc.deposit(500);
        System.out.println("balance = " + acc.balance);
        System.out.println("interest on balance = " + interestOn(acc.balance));
    }
}
```

OUTPUT

```
interest on 1000 = 50.0
balance = 3000.0
interest on balance = 150.0
```

9.6 Method overloading

Overloading means defining several methods with the **same name but different parameter lists** in the same class. The compiler picks the right one at **compile time** from the arguments (this is compile-time polymorphism). `System.out.println` is overloaded for int, double, char, String, Object and more.

9.6.1 Different number of parameters

9.6.2 Different types of parameters

9.6.3 Different order of parameters

THREE WAYS TO OVERLOAD

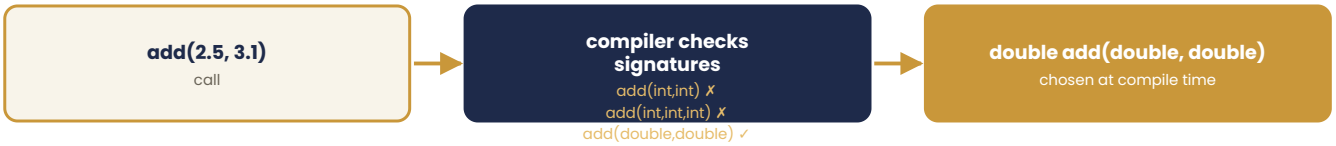
OverloadDemo.java

```
public class OverloadDemo {
    // 9.6.1 different number
    static int add(int a, int b)          { return a + b; }
    static int add(int a, int b, int c)   { return a + b + c; }
    // 9.6.2 different types
    static double add(double a, double b) { return a + b; }
    static String add(String a, String b) { return a + b; }
    // 9.6.3 different order
    static void show(String name, int age) { System.out.println("name first: " + name + ", " + age); }
    static void show(int age, String name) { System.out.println("age first : " + age + ", " + name); }

    public static void main(String[] args) {
        System.out.println(add(2, 3));
        System.out.println(add(2, 3, 4));
        System.out.println(add(2.5, 3.1));
        System.out.println(add("Java", "21"));
        show("Asha", 21);
        show(21, "Asha");
    }
}
```

OUTPUT

```
5
9
5.6
Java21
name first: Asha, 21
age first : 21, Asha
```



THE COMPILER MATCHES THE CALL TO ONE SIGNATURE

WATCH OUT

Overloading only on parameter **order** is legal but confusing for readers; use it sparingly.

9.6.4 Why return type alone cannot overload

The return type is not part of the signature. If two methods differed only in return type, a call like `get();` whose result is ignored, or `System.out.println(get())`, would give the compiler no way to decide which to call.

NOT ALLOWED

java

```
static int    get() { return 1; }
static double get() { return 1.0; }   // error: method get() is already defined
```

9.6.5 Overload resolution with type promotion

If there is no exact match, the compiler tries **widening** the arguments (byte → short → int → long → float → double, char → int). It follows these phases in order and stops at the first phase that finds a match:

1. **Exact match** of types.
2. **Widening** primitive conversion (e.g. int → long → double).
3. **Boxing/unboxing** (int ↔ Integer).

4. **Varargs**.

If two candidates are equally good in the same phase, the call is **ambiguous** and does not compile.

WHICH OVERLOAD IS CHOSEN?

Promotion.java

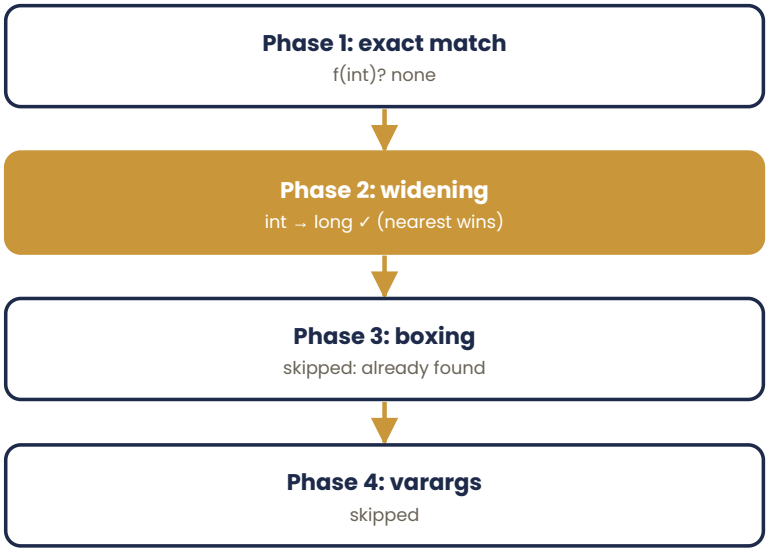
```
public class Promotion {
    static void f(long x)    { System.out.println("f(long)    <- " + x); }
    static void f(double x)  { System.out.println("f(double)  <- " + x); }
    static void f(Integer x) { System.out.println("f(Integer) <- " + x); }
    static void f(int... x)  { System.out.println("f(int...)  <- " + x.length + " values"); }

    static void g(Object o)  { System.out.println("g(Object)"); }
    static void g(String s)  { System.out.println("g(String)  most specific wins"); }

    public static void main(String[] args) {
        f(5);        // int: no f(int), widening to long wins (closest)
        f(5.0f);     // float: widened to double
        f('A');      // char -> int -> long
        f(Integer.valueOf(7));  // exact match
        f(1, 2, 3);  // only varargs fits
        g("hi");     // String is more specific than Object
    }
}
```

OUTPUT

```
f(long)    <- 5
f(double)  <- 5.0
f(long)    <- 65
f(Integer) <- 7
f(int...)  <- 3 values
g(String)  most specific wins
```



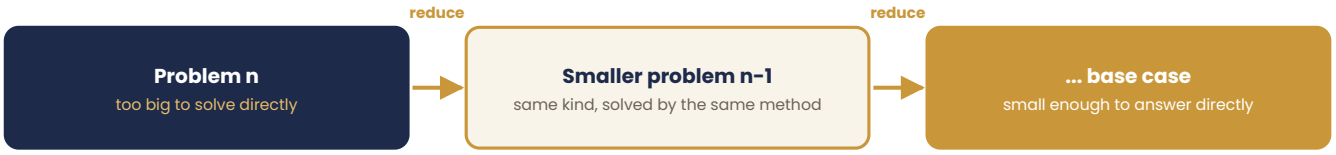
RESOLVING F(5)

9.7 Recursion

9.7.1 What recursion is

Recursion is when a method **calls itself** to solve a smaller version of the same problem. Many problems have a self-similar shape: the factorial of 5 is 5 × factorial of 4; a folder contains files and smaller folders; a tree is a node with smaller trees below it.

Picture a queue of people where you want to know your position. You ask the person ahead, "What is your position?" They ask the person ahead of them, and so on, until the first person says "1". The answers then come back down the line, each adding 1.



RECURSION REDUCES A PROBLEM UNTIL IT BECOMES TRIVIAL

9.7.2 Base case and recursive case

Every correct recursive method has two parts:

Base case

The simplest input, answered **without** another recursive call. It stops the recursion. Example: `if (n == 0) return 1;`

Recursive case

Calls the method on a **smaller** input and combines the result. Each call must move **closer** to the base case. Example: `return n * fact(n - 1);`

RECURSIVE FACTORIAL Factorial.java

```
public class Factorial {
    static long fact(int n) {
        if (n == 0) return 1;            // base case
        return n * fact(n - 1);          // recursive case: smaller input
    }
    public static void main(String[] args) {
        for (int i = 0; i <= 6; i++)
            System.out.println(i + "! = " + fact(i));
        System.out.println("20! = " + fact(20));
    }
}
```

OUTPUT

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
20! = 2432902008176640000
```

9.7.3 How the call stack grows and unwinds

Each recursive call gets its **own stack frame** with its own copy of `n`. The stack **grows** (winding) until the base case is reached; then the frames return one by one (**unwinding**), and each multiplies on its way back.

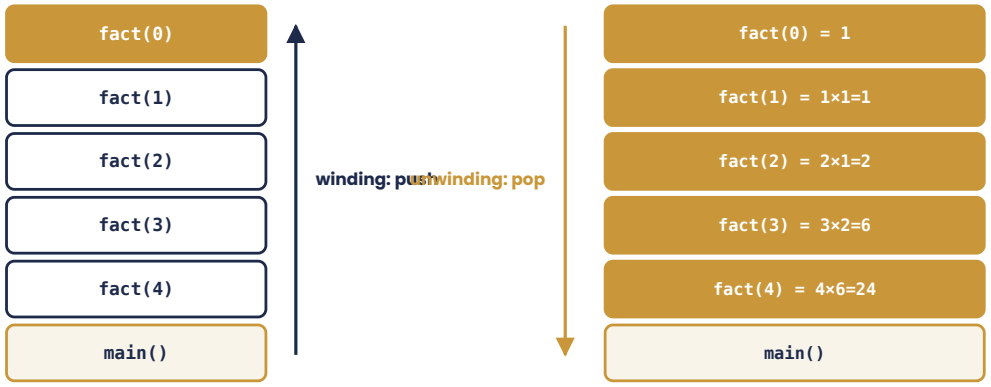
TRACING THE WINDING AND UNWINDING

TraceFactorial.java

```
public class TraceFactorial {
    static int fact(int n, String indent) {
        System.out.println(indent + "call fact(" + n + ")");
        if (n == 0) {
            System.out.println(indent + "base case -> 1");
            return 1;
        }
        int result = n * fact(n - 1, indent + "  ");
        System.out.println(indent + "fact(" + n + ") returns " + result);
        return result;
    }
    public static void main(String[] args) {
        fact(4, "");
    }
}
```

OUTPUT

```
call fact(4)
  call fact(3)
    call fact(2)
      call fact(1)
        call fact(0)
        base case -> 1
      fact(1) returns 1
    fact(2) returns 2
  fact(3) returns 6
fact(4) returns 24
```



THE STACK GROWS TO FACT(0), THEN UNWINDS, RETURNING VALUES UPWARD

9.7.4 StackOverflowError

The call stack has a limited size (typically 512 KB to 1 MB per thread, changeable with `-Xss`). If recursion never reaches a base case, or goes too deep (often around 10,000+ frames), the JVM throws `java.lang.StackOverflowError`.

- Missing base case: `static int f(int n) { return f(n - 1); }`
- Base case that is never reached: `if (n == 0)` with `f(n - 2)` starting from an odd number.
- Correct but too deep: summing a list of one million items recursively.

RECURSION WITHOUT A BASE CASE Overflow.java

```
public class Overflow {
    static int depth = 0;
    static void noBaseCase() {
        depth++;
        noBaseCase();          // never stops
    }
    public static void main(String[] args) {
        try {
            noBaseCase();
        } catch (StackOverflowError e) {
            System.out.println("StackOverflowError caught!");
            System.out.println("depth was more than 1000? " + (depth > 1000));
        }
    }
}
```

OUTPUT

```
StackOverflowError caught!
depth was more than 1000? true
```

WATCH OUT

StackOverflowError is an **Error**, not a normal exception. We catch it here only to demonstrate it; in real code you fix the recursion instead.

9.7.5 Types of recursion

9.7.5.1 Direct and indirect

Direct recursion: method `a()` calls `a()`. **Indirect** (mutual) recursion: `a()` calls `b()` and `b()` calls `a()`.

DIRECT AND INDIRECT RECURSION IndirectRecursion.java

```
public class IndirectRecursion {
    static boolean isEven(int n) { return n == 0 ? true  : isOdd(n - 1); }
    static boolean isOdd(int n)  { return n == 0 ? false : isEven(n - 1); }

    static int countDown(int n) {                    // direct recursion
        if (n == 0) return 0;
        System.out.print(n + " ");
        return countDown(n - 1);
    }
    public static void main(String[] args) {
        countDown(5);
        System.out.println();
        System.out.println("isEven(10) = " + isEven(10));
        System.out.println("isOdd(7)   = " + isOdd(7));
    }
}
```

OUTPUT

```
5 4 3 2 1
isEven(10) = true
isOdd(7)   = true
```



INDIRECT RECURSION: TWO METHODS CALLING EACH OTHER

9.7.5.2 Head and tail recursion

In **head recursion** the recursive call comes **first** and the work is done after it returns (on the way back up). In **tail recursion** the recursive call is the **last** action, and the work is done before it (on the way down).

SAME CALLS, OPPOSITE ORDER OF WORK

HeadTail.java

```
public class HeadTail {
    static void head(int n) {
        if (n == 0) return;
        head(n - 1);                // call first ...
        System.out.print(n + " ");  // ... work after: prints 1 2 3 4 5
    }
    static void tail(int n) {
        if (n == 0) return;
        System.out.print(n + " ");  // work first ...
        tail(n - 1);                // ... call last: prints 5 4 3 2 1
    }
    static long factTail(int n, long acc) {   // tail-recursive factorial with accumulator
        if (n == 0) return acc;
        return factTail(n - 1, acc * n);
    }
    public static void main(String[] args) {
        System.out.print("head: "); head(5); System.out.println();
        System.out.print("tail: "); tail(5); System.out.println();
        System.out.println("factTail(10) = " + factTail(10, 1));
    }
}
```

OUTPUT

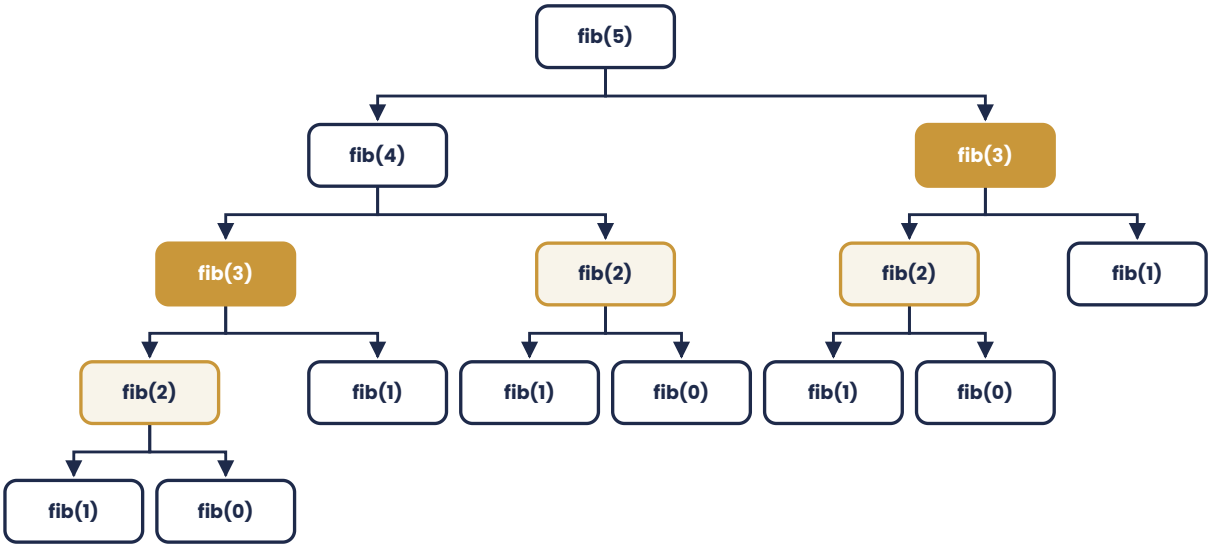
```
head: 1 2 3 4 5
tail: 5 4 3 2 1
factTail(10) = 3628800
```

NOTE

Some languages convert tail recursion into a loop (tail-call optimisation). **Java does not**, so a deep tail-recursive method can still overflow the stack. Tail-recursive code is, however, easy to rewrite as a loop by hand.

9.7.5.3 Tree (multiple) recursion

When a method makes **two or more** recursive calls, the calls form a tree. The naive Fibonacci method is the classic example: `fib(n) = fib(n-1) + fib(n-2)`.



RECURSION TREE OF FIB(5): FIB(3) (GOLD) IS COMPUTED TWICE AND FIB(2) (CREAM) THREE TIMES

The tree roughly doubles at every level, so the time is about $O(2^n)$. fib(40) makes over 300 million calls. Memoization (9.7.8) fixes this.

9.7.6 Recursion vs iteration

Anything that can be done with recursion can be done with a loop (sometimes with an explicit stack), and vice versa.

	Recursion	Iteration (loops)
Code	Often shorter and closer to the mathematical definition	Sometimes longer, but explicit
Memory	One stack frame per call: O(depth) extra memory	Usually O(1) extra memory
Speed	Call overhead; risk of repeated work	Generally faster
Risk	StackOverflowError when too deep	Infinite loop if the condition is wrong
Best for	Trees, divide and conquer, backtracking, nested structures	Simple counting, linear scans, very deep repetition

SAME RESULT, DIFFERENT LIMITS

RecVsIter.java

```
public class RecVsIter {
    static int sumRec(int n)  { return n == 0 ? 0 : n + sumRec(n - 1); }
    static int sumIter(int n) {
        int s = 0;
        for (int i = 1; i <= n; i++) s += i;
        return s;
    }
    public static void main(String[] args) {
        System.out.println("recursive 100 : " + sumRec(100));
        System.out.println("iterative 100 : " + sumIter(100));
        System.out.println("iterative 1_000_000 : " + sumIter(1_000_000) + " (overflowed int, but no crash)");
        try {
            sumRec(1_000_000);
        } catch (StackOverflowError e) {
            System.out.println("recursive 1_000_000 : StackOverflowError");
        }
    }
}
```

OUTPUT

```
recursive 100 : 5050
iterative 100 : 5050
iterative 1_000_000 : 1784293664 (overflowed int, but no crash)
recursive 1_000_000 : StackOverflowError
```

9.7.7 Classic recursive problems

To write any recursive method, answer three questions: **(1)** What is the smallest input I can answer directly? **(2)** How do I make the input smaller? **(3)** How do I build my answer from the smaller answer?

9.7.7.1 Factorial, Fibonacci, sum of digits, power

FOUR CLASSIC RECURSIVE FUNCTIONS

ClassicRec.java

```
public class ClassicRec {
    static long fact(int n)      { return n <= 1 ? 1 : n * fact(n - 1); }
    static long fib(int n)       { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
    static int sumDigits(int n)  { return n == 0 ? 0 : n % 10 + sumDigits(n / 10); }
    static long power(long b, int e) {        // fast power: O(log e)
        if (e == 0) return 1;
        long half = power(b, e / 2);
        return (e % 2 == 0) ? half * half : half * half * b;
    }
    public static void main(String[] args) {
        System.out.println("fact(10)       = " + fact(10));
        System.out.print("fib(0..10)     = ");
        for (int i = 0; i <= 10; i++) System.out.print(fib(i) + " ");
        System.out.println();
        System.out.println("sumDigits(9875)= " + sumDigits(9875));
        System.out.println("power(2, 30)   = " + power(2, 30));
        System.out.println("power(3, 13)   = " + power(3, 13));
    }
}
```

OUTPUT

```
fact(10)       = 3628800
fib(0..10)     = 0 1 1 2 3 5 8 13 21 34 55
sumDigits(9875)= 29
power(2, 30)   = 1073741824
power(3, 13)   = 1594323
```

Problem	Base case	Recursive case
factorial	$n \le 1 \to 1$	$n \times \text{fact}(n-1)$
Fibonacci	$n < 2 \to n$	$\text{fib}(n-1) + \text{fib}(n-2)$
sum of digits	$n = 0 \to 0$	last digit (n % 10) + sumDigits(n / 10)
power b^e	$e = 0 \to 1$	half = $b^{e/2}$; half² (× b if e is odd)

9.7.7.2 Reverse a string, palindrome check

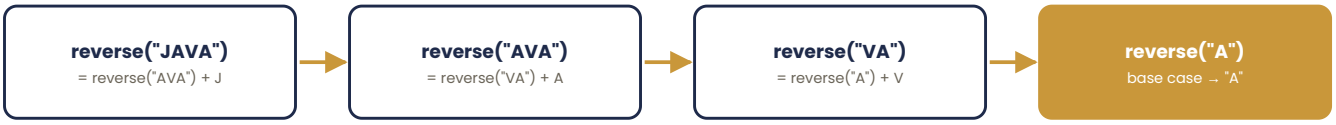
RECURSION ON STRINGS

StringRec.java

```
public class StringRec {
    static String reverse(String s) {
        if (s.length() <= 1) return s;                    // base case
        return reverse(s.substring(1)) + s.charAt(0);     // rest reversed + first char
    }
    static boolean isPal(String s, int lo, int hi) {
        if (lo >= hi) return true;                        // met in the middle
        if (s.charAt(lo) != s.charAt(hi)) return false;   // mismatch
        return isPal(s, lo + 1, hi - 1);                  // check the inner part
    }
    public static void main(String[] args) {
        System.out.println(reverse("FORWARD"));
        for (String w : new String[] {"madam", "racecar", "java", "abba"})
            System.out.println(w + " -> " + isPal(w, 0, w.length() - 1));
    }
}
```

OUTPUT

```
DRAWROF
madam -> true
racecar -> true
java -> false
abba -> true
```

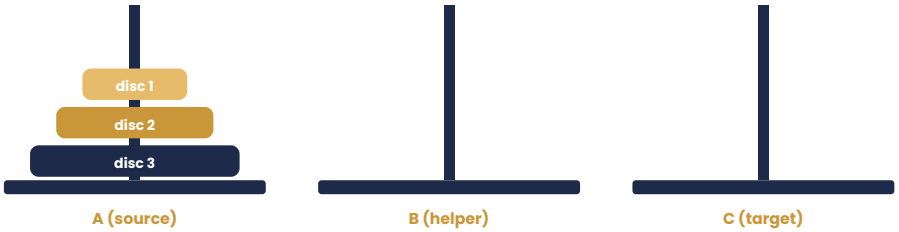


UNWINDING BUILDS "A" + "V" + "A" + "J" = "AVAJ"

9.7.7.3 Tower of Hanoi

Three rods A, B, C and n discs of different sizes stacked on A, largest at the bottom. Move all discs to C, one at a time, never placing a larger disc on a smaller one. The recursive insight:

1. Move the top **n-1** discs from A to B (using C as helper).
2. Move the largest disc from A to C.
3. Move the **n-1** discs from B to C (using A as helper).



TOWER OF HANOI WITH 3 DISCS: START POSITION

SOLVING TOWER OF HANOI Hanoi.java

```
public class Hanoi {
    static int moves = 0;
    static void hanoi(int n, char from, char helper, char to) {
        if (n == 0) return;                    // base case: nothing to move
        hanoi(n - 1, from, to, helper);        // 1. n-1 discs out of the way
        moves++;
        System.out.println("Move disc " + n + " from " + from + " to " + to);
        hanoi(n - 1, helper, from, to);        // 3. n-1 discs on top
    }
    public static void main(String[] args) {
        hanoi(3, 'A', 'B', 'C');
        System.out.println("Total moves = " + moves + " (2^3 - 1)");
    }
}
```

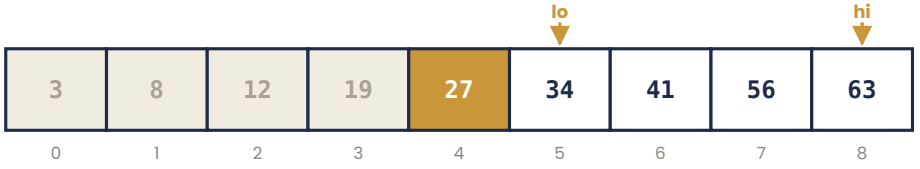
OUTPUT

```
Move disc 1 from A to C
Move disc 2 from A to B
Move disc 1 from C to B
Move disc 3 from A to C
Move disc 1 from B to A
Move disc 2 from B to C
Move disc 1 from A to C
Total moves = 7 (2^3 - 1)
```

Hanoi with n discs needs exactly $2^n - 1$ moves. 64 discs at one move per second would take about 585 billion years.

9.7.7.4 Binary search recursively

In a **sorted** array, compare the target with the middle element. If equal, done; if smaller, search the left half; if larger, search the right half. Each call halves the range, so it takes at most about $\log_2 n$ calls.



SEARCHING 41: MID = 27 IS TOO SMALL, SO LO MOVES TO INDEX 5 AND THE LEFT HALF IS DISCARDED

RECURSIVE BINARY SEARCH BinarySearchRec.java

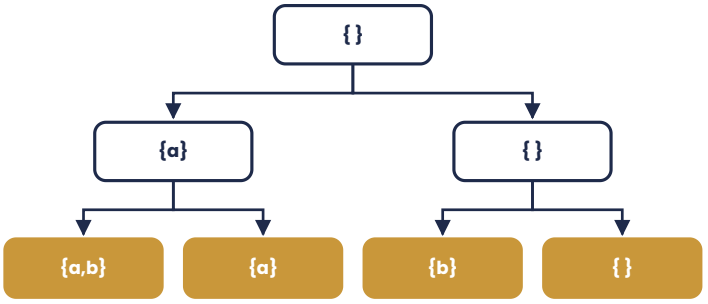
```
public class BinarySearchRec {
    static int search(int[] a, int target, int lo, int hi) {
        if (lo > hi) return -1;                    // empty range: not found
        int mid = lo + (hi - lo) / 2;              // avoids int overflow
        System.out.println("  checking range [" + lo + ".." + hi + "], mid=" + mid + " (" + a[mid] + ")");
        if (a[mid] == target) return mid;
        if (target < a[mid]) return search(a, target, lo, mid - 1);
        return search(a, target, mid + 1, hi);
    }
    public static void main(String[] args) {
        int[] a = {3, 8, 12, 19, 27, 34, 41, 56, 63};
        System.out.println("search 41:");
        System.out.println("found at index " + search(a, 41, 0, a.length - 1));
        System.out.println("search 20:");
        System.out.println("found at index " + search(a, 20, 0, a.length - 1));
    }
}
```

OUTPUT

```
search 41:
  checking range [0..8], mid=4 (27)
  checking range [5..8], mid=6 (41)
found at index 6
search 20:
  checking range [0..8], mid=4 (27)
  checking range [0..3], mid=1 (8)
  checking range [2..3], mid=2 (12)
  checking range [3..3], mid=3 (19)
found at index -1
```

9.7.7.5 Subsets and permutations (intro to backtracking)

Backtracking builds a solution step by step, explores each choice recursively and then **undoes** the choice (backtracks) to try the next one. For **subsets**, each element has two choices: include it or exclude it, giving 2^n subsets.



CHOICE TREE FOR SUBSETS OF [A, B]: LEFT = INCLUDE, RIGHT = EXCLUDE; THE LEAVES (GOLD) ARE THE 4 SUBSETS

GENERATING SUBSETS AND PERMUTATIONS

Backtracking.java

```
import java.util.ArrayList;
import java.util.List;

public class Backtracking {
    static void subsets(char[] items, int i, List<Character> current) {
        if (i == items.length) {                    // decided for every item
            System.out.print(current + " ");
            return;
        }
        current.add(items[i]);                      // choose: include
        subsets(items, i + 1, current);
        current.remove(current.size() - 1);         // un-choose (backtrack)
        subsets(items, i + 1, current);             // exclude
    }

    static void permute(String prefix, String rest) {
        if (rest.isEmpty()) {
            System.out.print(prefix + " ");
            return;
        }
        for (int i = 0; i < rest.length(); i++)     // pick each remaining char next
            permute(prefix + rest.charAt(i), rest.substring(0, i) + rest.substring(i + 1));
    }

    public static void main(String[] args) {
        System.out.print("subsets of abc: ");
        subsets(new char[] {'a', 'b', 'c'}, 0, new ArrayList<>());
        System.out.println();
        System.out.print("permutations of abc: ");
        permute("", "abc");
        System.out.println();
    }
}
```

OUTPUT

```
subsets of abc: [a, b, c] [a, b] [a, c] [a] [b, c] [b] [c] []
permutations of abc: abc acb bac bca cab cba
```

NOTE

n items have 2^n subsets and n! permutations. Backtracking is the basis of puzzles such as N-Queens and Sudoku solvers.

9.7.8 Memoization (intro)

Tree recursion often solves the **same sub-problem many times** (see the fib(5) tree). **Memoization** stores each answer the first time it is computed (in an array or map) and returns the stored value on later calls. This turns Fibonacci from $O(2^n)$ into $O(n)$. It is the first step towards **dynamic programming**.

THE SAME FUNCTION, MILLIONS OF CALLS FEWER

Memo.java

```
public class Memo {
    static long calls = 0;

    static long fibSlow(int n) {
        calls++;
        return n < 2 ? n : fibSlow(n - 1) + fibSlow(n - 2);
    }

    static long[] memo = new long[100];          // 0 means "not computed yet"
    static long fibMemo(int n) {
        calls++;
        if (n < 2) return n;
        if (memo[n] != 0) return memo[n];        // already known: reuse
        memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
        return memo[n];
    }

    public static void main(String[] args) {
        calls = 0;
        System.out.println("fibSlow(30) = " + fibSlow(30) + "  calls: " + calls);
        calls = 0;
        System.out.println("fibMemo(30) = " + fibMemo(30) + "  calls: " + calls);
        calls = 0;
        System.out.println("fibMemo(90) = " + fibMemo(90) + "  calls: " + calls);
    }
}
```

OUTPUT

```
fibSlow(30) = 832040  calls: 2692537
fibMemo(30) = 832040  calls: 59
fibMemo(90) = 2880067194370816120  calls: 121
```



NUMBER OF CALLS FOR N = 30

Memoization = recursion + a cache. Ask: "Does my recursion tree repeat the same arguments?" If yes, store the results.

PRACTICE CHECKPOINT

An **overloaded area()** method for circle, rectangle and triangle, plus recursive **factorial**, **Fibonacci** and **Tower of Hanoi**.

SAMPLE SOLUTION

MethodsCheckpoint.java

```
public class MethodsCheckpoint {
    // overloaded area()
    static double area(double radius)                { return Math.PI * radius * radius; }   // circle
    static double area(double length, double width)  { return length * width; }             // rectangle
    static double area(double a, double b, double c) {                                       // triangle (Heron)
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }

    static long factorial(int n) { return n <= 1 ? 1 : n * factorial(n - 1); }
    static long fibonacci(int n) { return n < 2 ? n : fibonacci(n - 1) + fibonacci(n - 2); }
    static void hanoi(int n, char from, char via, char to) {
        if (n == 0) return;
        hanoi(n - 1, from, to, via);
        System.out.println("  disc " + n + ": " + from + " -> " + to);
        hanoi(n - 1, via, from, to);
    }

    public static void main(String[] args) {
        System.out.printf("circle r=3        : %.2f%n", area(3));
        System.out.printf("rectangle 4 x 5   : %.2f%n", area(4, 5));
        System.out.printf("triangle 3, 4, 5  : %.2f%n", area(3, 4, 5));
        System.out.println("factorial(12)     : " + factorial(12));
        System.out.print("fibonacci(0..12)  : ");
        for (int i = 0; i <= 12; i++) System.out.print(fibonacci(i) + " ");
        System.out.println();
        System.out.println("hanoi(3):");
        hanoi(3, 'A', 'B', 'C');
    }
}
```

OUTPUT

```
circle r=3        : 28.27
rectangle 4 x 5   : 20.00
triangle 3, 4, 5  : 6.00
factorial(12)     : 479001600
fibonacci(0..12)  : 0 1 1 2 3 5 8 13 21 34 55 89 144
hanoi(3):
  disc 1: A -> C
  disc 2: A -> B
  disc 1: C -> B
  disc 3: A -> C
  disc 1: B -> A
  disc 2: B -> C
  disc 1: A -> C
```

LEVEL 2

Intermediate level

Outcome: design object-oriented programs, handle errors, use collections and files.

6 MODULES · 29 DAYS MINIMUM

Module 10 · Object-Oriented Programming (OOP)	**10 days**
Module 11 · Packages, access modifiers and wrapper classes	**3 days**
Module 12 · Exception handling	**3 days**
Module 13 · Generics	**2 days**
Module 14 · Collections Framework	**8 days**
Module 15 · File handling and I/O	**3 days**

MODULE 10 · INTERMEDIATE

Object-Oriented Programming (OOP)

Minimum duration **10 days** | Level **Intermediate**

OOP models a program as objects that combine data and behaviour. It rests on four pillars: encapsulation, inheritance, polymorphism and abstraction.

IN THIS MODULE

10.1 Introduction to OOP
10.2 Classes and objects
10.3 Constructors
10.4 Keywords inside classes
10.5 Initialization blocks and order
10.6 Encapsulation
10.7 Inheritance
10.8 Polymorphism
10.9 Abstraction
10.10 Object class methods
10.11 Relationships between classes
10.12 Inner (nested) classes
10.13 Enums
10.14 Records (Java 16+)
10.15 Sealed classes (Java 17+)

10.1 Introduction to OOP

10.1.1 Procedural vs object-oriented programming

In **procedural** programming (C, early BASIC) a program is a list of functions that work on separate data. As programs grow, any function can change any data, and it becomes hard to know who changed what. In **object-oriented** programming the data and the functions that work on it are packed together into **objects**, and each object protects its own data.



PROCEDURAL: EVERY FUNCTION REACHES INTO SHARED DATA. OOP: EACH OBJECT OWNS ITS DATA AND METHODS

Aspect	Procedural	Object-oriented
Building block	Functions	Objects (classes)
Data	Often global, shared by all functions	Hidden inside objects (encapsulation)
Reuse	Copying or calling functions	Inheritance, composition, polymorphism
Change	A data change can break many functions	Changes stay inside one class
Suited to	Small scripts, algorithms	Large systems with many interacting parts
Examples	C, Pascal	Java, C#, C++, Python

10.1.2 Real-world modelling with objects

OOP lets us describe software the way we describe the world. A college has **students**, **courses** and **teachers**. Each student has data (name, roll number, marks) and can do things (enroll, pay fees). In Java each of these nouns becomes

a **class**, each real student becomes an **object**, the data become **fields** and the actions become **methods**.

Real world	Java
Kind of thing ("Student")	class `Student`
A particular student "Asha"	object `new Student("Asha")`
Properties (name, marks)	fields / instance variables
Actions (enroll, study)	methods
"A student has a course"	a field of type `Course`
"A topper is a student"	`class Topper extends Student`

10.1.3 The four pillars overview

Encapsulation

Bundle data and methods together and **hide** the data behind methods. *A capsule protects the medicine inside.*

Inheritance

A new class **reuses** and extends an existing one (IS-A). *A Car is a Vehicle.*

Polymorphism

One name, **many forms**: the same call behaves differently for different objects. *speak() barks for a dog, meows for a cat.*

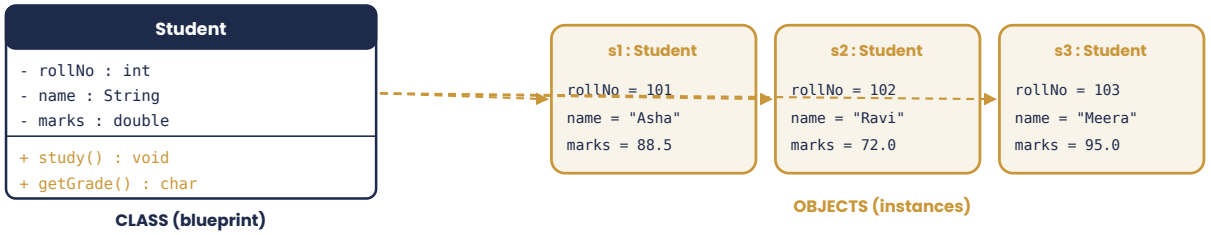
Abstraction

Show **what** an object does and hide **how**. *You drive a car with a steering wheel without knowing the engine.*

10.2 Classes and objects

10.2.1 Class as a blueprint

A **class** is a blueprint (template) that describes what data an object will hold and what it can do. It does not occupy memory for data by itself. From one blueprint we can build many houses; from one class we can create many objects.



ONE CLASS, MANY OBJECTS: EACH OBJECT HAS ITS OWN COPY OF THE FIELDS

A CLASS WITH FIELDS AND METHODS

ClassDemo.java

```
class Student {
    // fields (state)
    int rollNo;
    String name;
    double marks;

    // methods (behaviour)
    void study(int hours) {
        marks = Math.min(100, marks + hours * 0.5);
        System.out.println(name + " studied " + hours + "h, marks now " + marks);
    }
    char getGrade() {
        return marks >= 90 ? 'A' : marks >= 75 ? 'B' : marks >= 50 ? 'C' : 'F';
    }
}

public class ClassDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.rollNo = 101; s1.name = "Asha"; s1.marks = 88.5;
        s1.study(4);
        System.out.println(s1.name + " grade: " + s1.getGrade());
    }
}
```

OUTPUT

```
Asha studied 4h, marks now 90.5
Asha grade: A
```

10.2.2 Object as an instance: state, behaviour, identity

State

The current values of its fields. `s1.marks = 90.5` is part of s1's state.

Behaviour

What the object can do: its methods. `s1.study(4)`

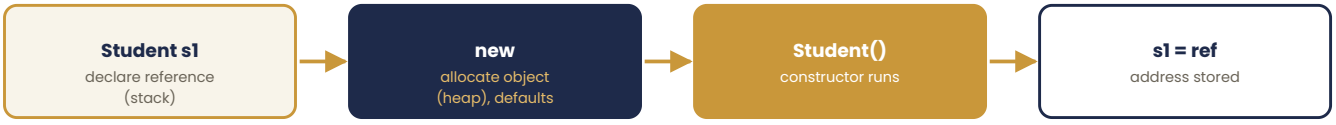
Identity

What makes it a distinct object even if another object has identical state: its own place in memory. Checked with `==`.

10.2.3 Creating objects with new

`Student s1 = new Student();` does three things:

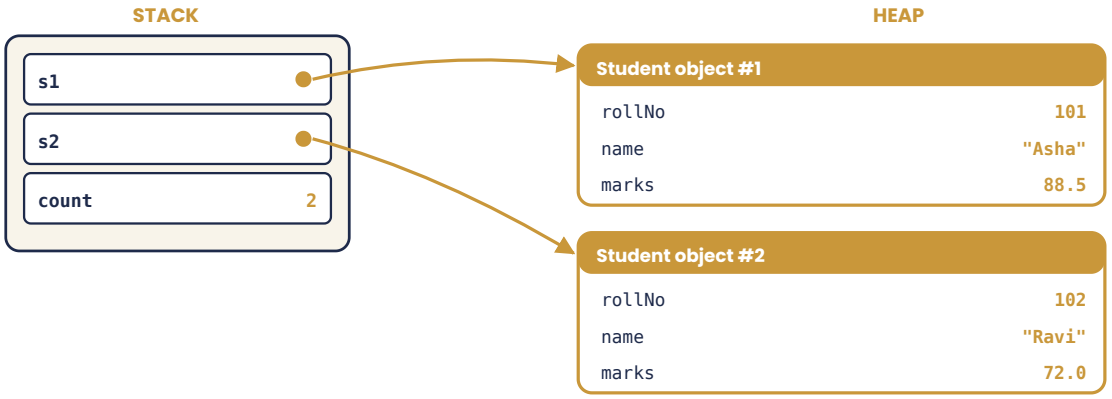
1. **Declaration**: `Student s1` creates a reference variable on the stack, able to point to a Student.
2. **Instantiation**: `new` allocates memory for a new Student object on the heap. Fields get default values (0, 0.0, false, null).
3. **Initialization**: the constructor `Student()` runs to set up the object, and `new` returns its reference, which is stored in s1.



STEPS OF OBJECT CREATION

10.2.4 Reference variables and memory (stack reference → heap object)

Objects always live on the **heap**. A local variable such as `s1` lives in the method's stack frame and holds only a **reference** (like a remote control that points to a TV). When no reference points to an object any more, the garbage collector can reclaim it.



REFERENCES ON THE STACK POINT TO OBJECTS ON THE HEAP; PRIMITIVES (COUNT) HOLD THE VALUE ITSELF

10.2.5 Multiple references to one object

Assigning one reference to another copies the **address**, not the object. Both variables then control the **same** object, so a change through one is visible through the other.

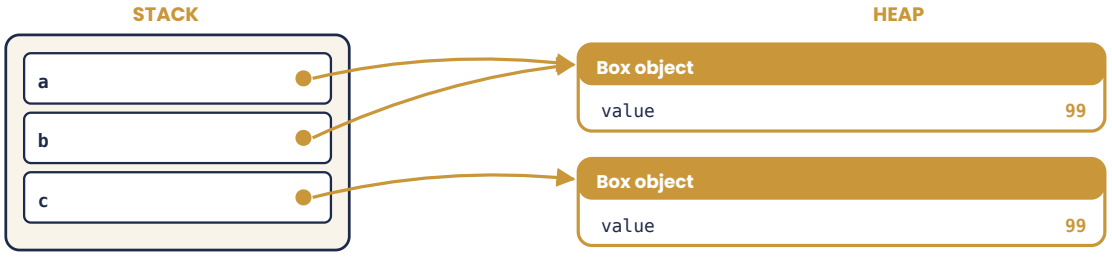
TWO REFERENCES, ONE OBJECT — SharedRef.java

```
class Box { int value; }

public class SharedRef {
    public static void main(String[] args) {
        Box a = new Box();
        a.value = 10;
        Box b = a;                // copies the reference: ONE object, two names
        b.value = 99;
        System.out.println("a.value = " + a.value);
        System.out.println("a == b  : " + (a == b));
        Box c = new Box();
        c.value = 99;
        System.out.println("a == c  : " + (a == c) + "  (same state, different identity)");
        a = null;                 // a no longer points anywhere
        System.out.println("b.value still = " + b.value);
    }
}
```

OUTPUT

```
a.value = 99
a == b  : true
a == c  : false  (same state, different identity)
b.value still = 99
```



A AND B SHARE ONE OBJECT; C IS A DIFFERENT OBJECT WITH EQUAL STATE

10.2.6 Ways to create objects (new, clone, reflection, deserialization)

Way	Example	Constructor runs?
`new` keyword	`Student s = new Student();`	Yes. The normal way (99% of cases)
`clone()`	`Student t = (Student) s.clone();`	No: copies field by field (10.10.4)

Way	Example	Constructor runs?
Reflection	`Student.class.getDeclaredConstructor().newInstance()`	Yes. Used by frameworks like Spring (Module 21)
Deserialization	`(Student) in.readObject()`	No (for the Serializable class itself). Rebuilds from bytes (Module 15)
Factory methods	`List.of(1,2)`, `LocalDate.now()`	Yes, internally uses new

THREE WAYS TO CREATE OBJECTS WaysToCreate.java

```
public class WaysToCreate implements Cloneable {
    String label;
    WaysToCreate() { label = "made by constructor"; }

    public static void main(String[] args) throws Exception {
        WaysToCreate a = new WaysToCreate();                                        // 1. new
        WaysToCreate b = (WaysToCreate) a.clone();                                  // 2. clone
        WaysToCreate c = WaysToCreate.class.getDeclaredConstructor().newInstance(); // 3. reflection
        System.out.println("new        : " + a.label);
        System.out.println("clone      : " + b.label + ", same object? " + (a == b));
        System.out.println("reflection : " + c.label);
    }
}
```

OUTPUT

```
new        : made by constructor
clone      : made by constructor, same object? false
reflection : made by constructor
```

10.2.7 Anonymous objects

An object created without storing its reference is an **anonymous object**. It is used once and then becomes eligible for garbage collection. Handy when you need an object for only one call.

OBJECTS USED ONCE WITHOUT A NAME AnonymousObject.java

```
class Calculator {
    int square(int n) { return n * n; }
}
public class AnonymousObject {
    public static void main(String[] args) {
        System.out.println(new Calculator().square(12));          // used once
        System.out.println(new StringBuilder("level").reverse()); // common with library classes
    }
}
```

OUTPUT

```
144
level
```

10.3 Constructors

10.3.1 What a constructor is and its rules

A **constructor** is a special block that runs automatically when an object is created with `new`. Its job is to put the new object into a valid starting state.

- Its name must be **exactly the class name**.
- It has **no return type**, not even `void`. (With a return type it becomes an ordinary method!)
- It can have any access modifier, but **cannot** be `static`, `final` or `abstract`.
- It is called once per object, only through `new` (or `this()` / `super()` from another constructor).
- Constructors are **not inherited**.

10.3.2 Default constructor (compiler-provided)

If you write **no constructor at all**, the compiler adds a public no-argument **default constructor** with an empty body (it only calls `super()`). As soon as you write **any** constructor, the default one is **not** added.

DEFAULT CONSTRUCTOR java

```java
class Point { int x, y; }
// the compiler silently adds:   Point() { super(); }
Point p = new Point();     // works: x = 0, y = 0
```

10.3.3 No-argument constructor

A constructor you write yourself with no parameters, to give sensible starting values instead of 0/null.

10.3.4 Parameterized constructor

A constructor with parameters lets the caller supply the starting values, so the object is fully set up in one line.

10.3.5 Constructor overloading

Like methods, a class can have several constructors with different parameter lists. The matching one is chosen by the arguments to `new`.

NO-ARG, PARAMETERIZED AND OVERLOADED CONSTRUCTORS Constructors.java

```java
class Product {
    String name;
    double price;
    int qty;

    Product() {                                  // 10.3.3 no-arg
        name = "Unnamed"; price = 0; qty = 1;
    }
    Product(String name, double price) {         // 10.3.4 parameterized
        this.name = name; this.price = price; this.qty = 1;
    }
    Product(String name, double price, int qty) {   // 10.3.5 overloaded
        this.name = name; this.price = price; this.qty = qty;
    }
    public String toString() { return name + " x" + qty + " @ " + price; }
}

public class Constructors {
    public static void main(String[] args) {
        System.out.println(new Product());
        System.out.println(new Product("Pen", 10));
        System.out.println(new Product("Notebook", 45.5, 3));
    }
}
```

OUTPUT

```
Unnamed x1 @ 0.0
Pen x1 @ 10.0
Notebook x3 @ 45.5
```

WATCH OUT

Once you add `Product(String, double)`, writing `new Product()` is a compile error unless you also write the no-arg constructor yourself.

10.3.6 Constructor chaining with this()

Overloaded constructors often repeat the same assignments. Instead, one constructor can call another of the same class with `this(...)`. The call must be the **first statement**. Usually the shorter constructors chain to the most complete one, so the real work is written once.

THIS() CHAINING: THE MOST COMPLETE CONSTRUCTOR RUNS FIRST

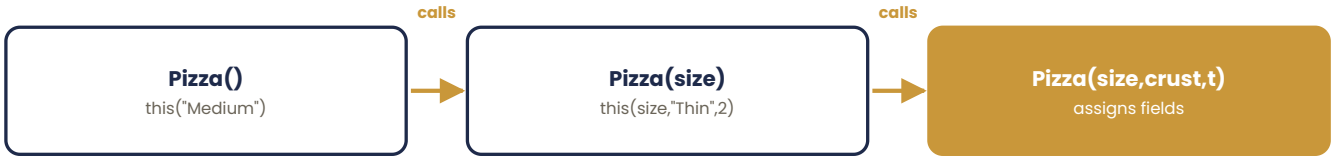
Chaining.java

```
class Pizza {
    String size, crust;
    int toppings;

    Pizza() {
        this("Medium");                         // chain
        System.out.println("  3. Pizza() body");
    }
    Pizza(String size) {
        this(size, "Thin", 2);                  // chain
        System.out.println("  2. Pizza(size) body");
    }
    Pizza(String size, String crust, int toppings) {
        this.size = size; this.crust = crust; this.toppings = toppings;
        System.out.println("  1. Pizza(size, crust, toppings) body");
    }
}
public class Chaining {
    public static void main(String[] args) {
        System.out.println("new Pizza():");
        Pizza p = new Pizza();
        System.out.println(p.size + ", " + p.crust + ", " + p.toppings + " toppings");
    }
}
```

OUTPUT

```
new Pizza():
  1. Pizza(size, crust, toppings) body
  2. Pizza(size) body
  3. Pizza() body
Medium, Thin, 2 toppings
```



CALLS GO RIGHT; BODIES FINISH RIGHT-TO-LEFT

10.3.7 Copy constructor

Java has no built-in copy constructor (unlike C++), but you can write one: a constructor that takes another object of the same class and copies its fields. For fields that are objects (like arrays), copy the contents too, otherwise both objects would share them.

INDEPENDENT COPY OF AN OBJECT CopyConstructor.java

```
import java.util.Arrays;

class Marks {
    String student;
    int[] scores;
    Marks(String student, int[] scores) { this.student = student; this.scores = scores; }

    Marks(Marks other) {                                  // copy constructor
        this.student = other.student;                     // String is immutable: safe to share
        this.scores = Arrays.copyOf(other.scores, other.scores.length);  // deep copy of array
    }
}
public class CopyConstructor {
    public static void main(String[] args) {
        Marks m1 = new Marks("Asha", new int[] {80, 90});
        Marks m2 = new Marks(m1);
        m2.scores[0] = 10;
        System.out.println("m1: " + Arrays.toString(m1.scores));
        System.out.println("m2: " + Arrays.toString(m2.scores));
    }
}
```

OUTPUT

```
m1: [80, 90]
m2: [10, 90]
```

10.3.8 Private constructors

A `private` constructor stops other classes from calling `new`. It is used for:

- **Utility classes** with only static methods (e.g. `Math`, `Collections`), which should never be instantiated;
- the **Singleton** pattern, where exactly one instance is allowed (Module 21);
- **static factory methods** such as `of(...)` that control how objects are created.

SINGLETON AND UTILITY CLASS PrivateConstructor.java

```
class AppConfig {
    private static AppConfig instance;              // the only instance
    private AppConfig() { System.out.println("AppConfig created"); }

    static AppConfig getInstance() {
        if (instance == null) instance = new AppConfig();
        return instance;
    }
}
final class MathUtil {
    private MathUtil() {}                           // no objects allowed
    static int cube(int n) { return n * n * n; }
}
public class PrivateConstructor {
    public static void main(String[] args) {
        AppConfig a = AppConfig.getInstance();
        AppConfig b = AppConfig.getInstance();
        System.out.println("same instance? " + (a == b));
        System.out.println("cube(4) = " + MathUtil.cube(4));
        // new AppConfig();   // error: AppConfig() has private access
    }
}
```

OUTPUT

```
AppConfig created
same instance? true
cube(4) = 64
```

10.3.9 Constructor vs method

	Constructor	Method
Purpose	Initialise a new object	Perform an action / compute a value
Name	Same as the class	Any valid name
Return type	None (not even void)	Required (void or a type)
Called	Automatically by `new`, once per object	Explicitly, any number of times
Inherited	No	Yes (if not private)
Default provided	Yes, when none is written	No
static / final / abstract	Not allowed	Allowed

10.4 Keywords inside classes

10.4.1 this keyword

`this` is a reference to the **current object**: the object whose method or constructor is running. It is available in every instance method and constructor (but not in static methods, since they have no current object).

10.4.1.1 Referring to current instance variables

When a parameter has the same name as a field, the parameter **shadows** the field. `this.name` means "the field", `name` means "the parameter".

WITHOUT THIS, THE FIELD STAYS NULL — ThisField.java

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        name = name;              // WRONG: assigns parameter to itself
        this.salary = salary;     // RIGHT: field = parameter
    }
}
public class ThisField {
    public static void main(String[] args) {
        Employee e = new Employee("Kiran", 50000);
        System.out.println("name = " + e.name + ", salary = " + e.salary);
    }
}
```

```
OUTPUT
name = null, salary = 50000.0
```

10.4.1.2 Calling current class methods and constructors

`this.method()` calls another method on the same object (writing just `method()` does the same). `this(...)` calls another constructor (10.3.6).

10.4.1.3 Passing and returning this

An object can pass **itself** to another method (`register(this)`) or **return itself** from a method. Returning `this` enables **method chaining** (a "fluent" style), used by `StringBuilder` and builders.

PASSING AND RETURNING THIS

ThisDemo.java

```
import java.util.ArrayList;
import java.util.List;

class Course {
    List<Learner> enrolled = new ArrayList<>();
    void register(Learner l) { enrolled.add(l); }
}
class Learner {
    String name; int age; String city;
    Learner name(String n) { this.name = n; return this; }   // returning this
    Learner age(int a)     { this.age = a;  return this; }
    Learner city(String c) { this.city = c; return this; }
    void join(Course c)    { c.register(this); }              // passing this
    void show() { this.print("Learner"); }                    // calling own method
    void print(String tag) { System.out.println(tag + ": " + name + ", " + age + ", " + city); }
}
public class ThisDemo {
    public static void main(String[] args) {
        Learner l = new Learner().name("Divya").age(22).city("Bengaluru");   // chaining
        l.show();
        Course java = new Course();
        l.join(java);
        System.out.println("enrolled: " + java.enrolled.size() + ", first = " + java.enrolled.get(0).name);
    }
}
```

OUTPUT

```
Learner: Divya, 22, Bengaluru
enrolled: 1, first = Divya
```

10.4.2 static keyword

`static` means "belongs to the **class**, not to any one object". There is exactly one copy, shared by all objects, and it exists even if no object has been created.

10.4.2.1 Static variables

A static field is stored once (in the class's area of memory), not inside each object. Use it for data shared by all instances: a counter of objects created, a common interest rate, a college name.

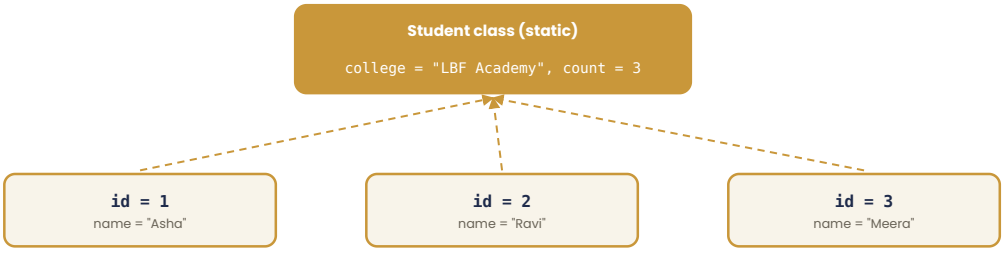
ONE SHARED COUNTER StaticVar.java

```
class Student {
    static String college = "LBF Institute";   // shared by all
    static int count = 0;                        // shared counter
    int id;                                      // each object's own
    String name;

    Student(String name) {
        this.name = name;
        count++;
        id = count;
    }
}
public class StaticVar {
    public static void main(String[] args) {
        Student a = new Student("Asha");
        Student b = new Student("Ravi");
        Student c = new Student("Meera");
        System.out.println(a.id + " " + a.name + " | " + b.id + " " + b.name + " | " + c.id + " " + c.name);
        System.out.println("total students: " + Student.count);
        Student.college = "LBF Academy";                // change once ...
        System.out.println(a.name + " @ " + a.college + ", " + c.name + " @ " + c.college);  // ... seen by all
    }
}
```

OUTPUT

```
1 Asha | 2 Ravi | 3 Meera
total students: 3
Asha @ LBF Academy, Meera @ LBF Academy
```



EACH OBJECT HAS ITS OWN ID AND NAME; ALL OBJECTS SHARE THE ONE STATIC COLLEGE AND COUNT

10.4.2.2 Static methods and their restrictions

A static method is called on the class: `Student.getCount()`. Because there is no current object:

- it **cannot** use instance fields or instance methods directly;
- it **cannot** use `this` or `super`;
- it **can** use static fields and other static methods, and it can work on objects passed to it as parameters.

WHAT A STATIC METHOD CANNOT TOUCH java

```
class Counter {
    int value;
    static int total;
    static void reset() {
        total = 0;          // OK: static field
        // value = 0;       // error: non-static variable value cannot be referenced from a static context
        // this.value = 0;  // error: non-static variable this cannot be referenced from a static context
    }
}
```

10.4.2.3 Static blocks

A `static { ... }` block runs **once**, when the class is first loaded, before `main` or any object creation. It is used to initialise complex static data (lookup tables, loading a driver). See 10.5.2 for the full order.

10.4.2.4 Static nested classes

A class declared `static` inside another class. It does not need an object of the outer class and is used to group a helper class with the class it belongs to, e.g. `Map.Entry`. Covered fully in 10.12.2.

10.4.3 final keyword

`final` means "cannot be changed later". Its exact meaning depends on where it is used.

10.4.3.1 Final variables (and blank final)

A final variable can be assigned **only once**. A final field that is not assigned at declaration is a **blank final**; it must be assigned in every constructor. `static final` fields are **constants**, named in UPPER_CASE.

FINAL FIELDS AND A FINAL REFERENCE FinalVars.java

```
class Circle {
    static final double PI = 3.14159;     // constant
    final int id;                         // blank final: set once in the constructor
    final double radius;

    Circle(int id, double radius) {
        this.id = id;
        this.radius = radius;
    }
    double area() { return PI * radius * radius; }
}
public class FinalVars {
    public static void main(String[] args) {
        Circle c = new Circle(1, 2.0);
        System.out.println("area = " + c.area());
        // c.radius = 5;       // error: cannot assign a value to final variable radius
        final int[] nums = {1, 2, 3};
        nums[0] = 100;         // allowed: the ARRAY can change ...
        // nums = new int[5];  // ... but the reference cannot
        System.out.println("nums[0] = " + nums[0]);
    }
}
```

OUTPUT

```
area = 12.56636
nums[0] = 100
```

A final **reference** cannot point to another object, but the object itself can still be modified. final ≠ immutable.

10.4.3.2 Final methods

A final method **cannot be overridden** by subclasses. Use it when a method's behaviour is critical and must not change (e.g. a security check).

10.4.3.3 Final classes

A final class **cannot be extended**. `String`, `Integer` and the other wrapper classes are final, so nobody can create a subclass that breaks their behaviour.

FINAL METHODS AND CLASSES java

```
final class Money { }
// class FakeMoney extends Money { }   // error: cannot inherit from final Money

class Bank {
    final boolean verifyPin(int pin) { return pin == 1234; }
}
class HackedBank extends Bank {
    // boolean verifyPin(int pin) { return true; }   // error: verifyPin(int) in HackedBank cannot override; overridden
method is final
}
```

Used on	Meaning
Variable / field	Value (or reference) assigned once
Parameter	Cannot be re-assigned inside the method
Method	Cannot be overridden
Class	Cannot be subclassed

10.5 Initialization blocks and order

10.5.1 Instance initializer blocks

A block `{ ... }` written directly inside a class (not in a method) is an **instance initializer**. It runs **every time an object is created**, before the constructor body (right after the `super()` call). It is useful for code shared by all constructors, and needed in anonymous classes that cannot have constructors.

10.5.2 Static initializer blocks

A `static { ... }` block runs **once** when the class is loaded. If there are several, they run top to bottom in the order written.

10.5.3 Order of execution: static block → instance block → constructor

WATCHING THE ORDER OF INITIALISATION

InitOrder.java

```
class Demo {
    static int s = initStatic();
    int x = initField();

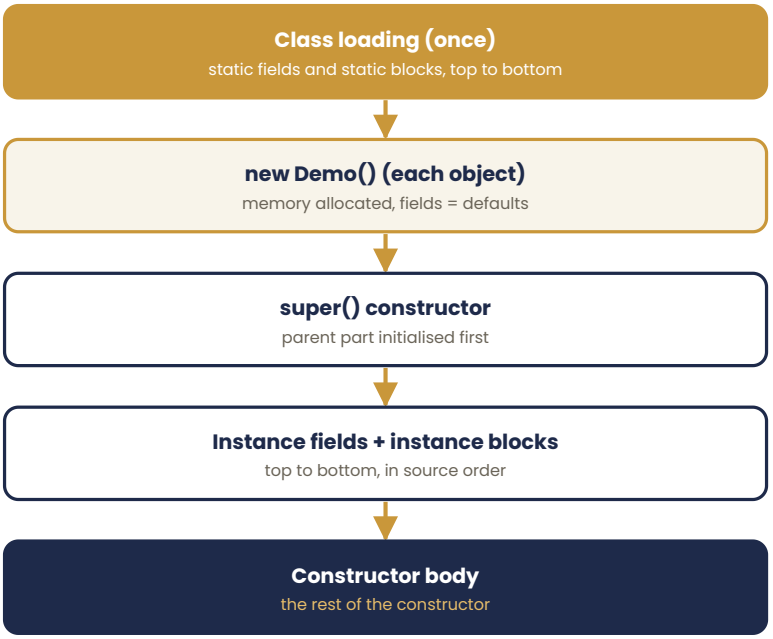
    static { System.out.println("2. static block"); }
    { System.out.println("4. instance block"); }

    Demo() { System.out.println("5. constructor"); }

    static int initStatic() { System.out.println("1. static field initializer"); return 1; }
    int initField()         { System.out.println("3. instance field initializer"); return 1; }
}
public class InitOrder {
    public static void main(String[] args) {
        System.out.println("main starts");
        System.out.println("-- first object --");
        new Demo();
        System.out.println("-- second object --");
        new Demo();
    }
}
```

OUTPUT

```
main starts
-- first object --
1. static field initializer
2. static block
3. instance field initializer
4. instance block
5. constructor
-- second object --
3. instance field initializer
4. instance block
5. constructor
```



ORDER OF INITIALISATION

NOTE

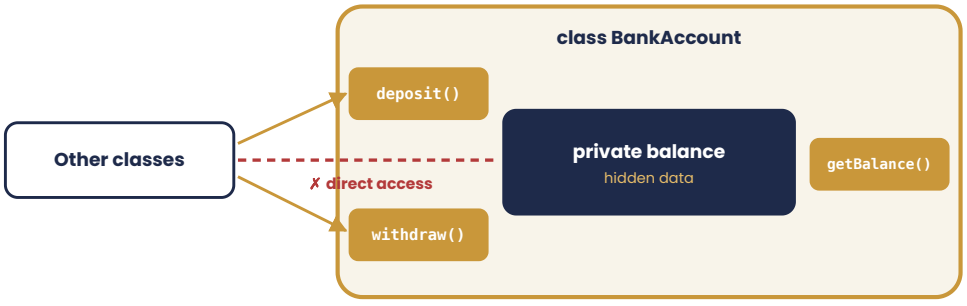
Static parts run before `main` only if the class is loaded then. Here `Demo` is loaded on first use, so "main starts" prints first.

10.6 Encapsulation

10.6.1 Data hiding with private fields

Encapsulation means wrapping data and the code that works on it into one unit (the class) and **hiding the data** from the outside. Fields are made `private` so other classes cannot read or change them directly. They must go through public methods, which can check every request.

Without encapsulation anyone can write `account.balance = -50000;`. With it, the only way to change the balance is `deposit()` or `withdraw()`, which enforce the rules.



OUTSIDERS REACH THE PRIVATE DATA ONLY THROUGH PUBLIC METHODS

10.6.2 Getters and setters

A **getter** (`getX()`, or `isX()` for booleans) returns a field's value; a **setter** (`setX(value)`) changes it. You decide which fields get which: a read-only field has a getter but no setter.

10.6.3 Validation inside setters

AN ENCAPSULATED BANK ACCOUNT WITH VALIDATION

Encapsulation.java

```
class BankAccount {
    private final String accountNo;      // read-only: getter only
    private String holder;
    private double balance;              // changed only through methods

    BankAccount(String accountNo, String holder) {
        this.accountNo = accountNo;
        setHolder(holder);
    }
    public String getAccountNo() { return accountNo; }
    public String getHolder()    { return holder; }
    public double getBalance()   { return balance; }

    public void setHolder(String holder) {
        if (holder == null || holder.isBlank())
            throw new IllegalArgumentException("holder name required");
        this.holder = holder.trim();
    }
    public void deposit(double amount) {
        if (amount <= 0) { System.out.println("  rejected: deposit must be positive"); return; }
        balance += amount;
    }
    public boolean withdraw(double amount) {
        if (amount <= 0 || amount > balance) { System.out.println("  rejected: invalid or insufficient funds"); return
false; }
        balance -= amount;
        return true;
    }
}
public class Encapsulation {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("SB-1001", "  Lohit  ");
        acc.deposit(5000);
        acc.deposit(-200);
        acc.withdraw(8000);
        acc.withdraw(1500);
        System.out.println(acc.getAccountNo() + " | " + acc.getHolder() + " | balance " + acc.getBalance());
        // acc.balance = 1_000_000;   // error: balance has private access in BankAccount
    }
}
```

OUTPUT

```
  rejected: deposit must be positive
  rejected: invalid or insufficient funds
SB-1001 | Lohit | balance 3500.0
```

10.6.4 JavaBeans / POJO conventions

A **POJO** (Plain Old Java Object) is a simple class with fields and methods, not tied to any framework. A **JavaBean** is a POJO that follows conventions so that tools (Spring, Hibernate, JSP, JSON libraries) can work with it automatically:

- all fields `private`;
- a **public no-argument constructor**;
- public getters/setters named `getName()` / `setName()` (`isActive()` for boolean);
- usually `implements Serializable`.

A JAVABEAN java

```java
public class StudentBean implements java.io.Serializable {
    private String name;
    private boolean active;

    public StudentBean() { }                        // required no-arg constructor
    public String getName()          { return name; }
    public void setName(String name) { this.name = name; }
    public boolean isActive()        { return active; }
    public void setActive(boolean a) { this.active = a; }
}
```

10.6.5 Immutable classes

An **immutable** object cannot change after creation (like String). Immutable objects are simple, thread-safe and safe to share or use as map keys. Recipe:

1. Make the class `final` (no subclass can add mutability).
2. Make all fields `private final`.
3. Set all fields in the constructor; provide **no setters**.
4. For mutable fields (arrays, lists, dates), make **defensive copies** in the constructor and in getters.
5. Methods that "change" the object return a **new** object instead.

A PROPERLY IMMUTABLE CLASS ImmutableDemo.java

```java
import java.util.ArrayList;
import java.util.List;

final class Order {
    private final String id;
    private final List<String> items;

    Order(String id, List<String> items) {
        this.id = id;
        this.items = new ArrayList<>(items);          // defensive copy IN
    }
    public String getId() { return id; }
    public List<String> getItems() { return List.copyOf(items); }   // read-only copy OUT
    public Order withItem(String item) {                              // "change" -> new object
        List<String> copy = new ArrayList<>(items);
        copy.add(item);
        return new Order(id, copy);
    }
    public String toString() { return id + items; }
}
public class ImmutableDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("pen", "book"));
        Order o1 = new Order("ORD-1", list);
        list.add("hacked");                           // does not affect o1
        Order o2 = o1.withItem("bag");
        System.out.println("o1 = " + o1);
        System.out.println("o2 = " + o2);
        try {
            o1.getItems().add("x");
        } catch (UnsupportedOperationException e) {
            System.out.println("cannot modify items: " + e.getClass().getSimpleName());
        }
    }
}
```

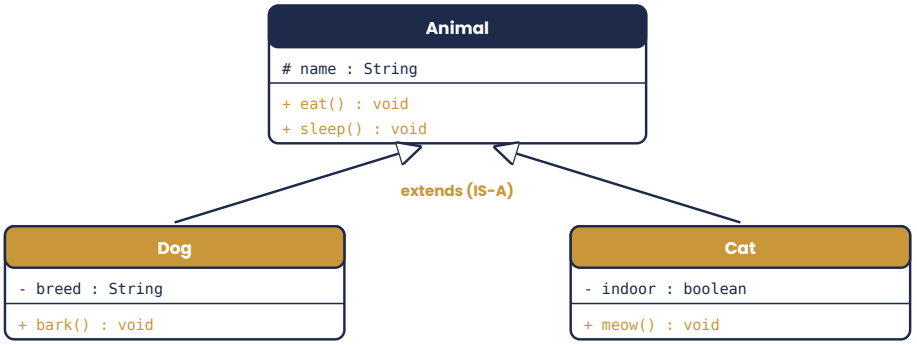
OUTPUT

```
o1 = ORD-1[pen, book]
o2 = ORD-1[pen, book, bag]
cannot modify items: UnsupportedOperationException
```

10.7 Inheritance

10.7.1 What inheritance is (IS-A relationship), extends

Inheritance lets a new class (the **subclass**, child) acquire the fields and methods of an existing class (the **superclass**, parent) using `extends`, and then add or change behaviour. It models an **IS-A** relationship: a Dog *is an* Animal, a SavingsAccount *is an* Account. Benefits: code reuse, a natural hierarchy, and it enables polymorphism.



UML: DOG AND CAT INHERIT NAME, EAT() AND SLEEP() FROM ANIMAL (HOLLOW TRIANGLE = INHERITANCE)

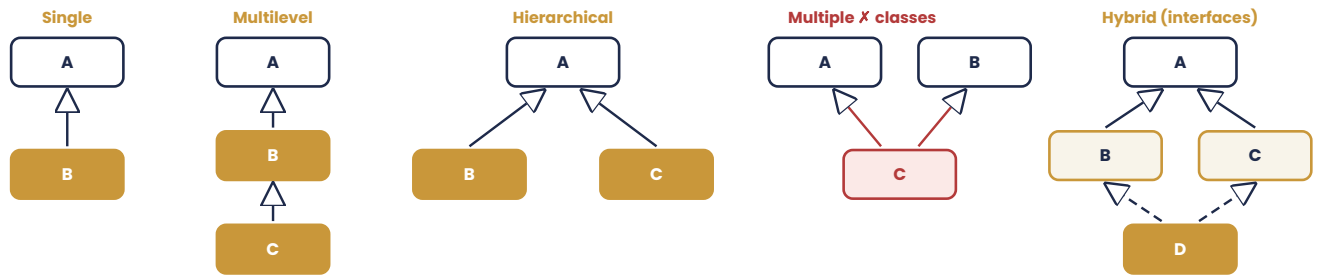
DOG REUSES EVERYTHING ANIMAL HAS — InheritanceDemo.java

```
class Animal {
    protected String name;
    void eat()   { System.out.println(name + " is eating"); }
    void sleep() { System.out.println(name + " is sleeping"); }
}
class Dog extends Animal {
    String breed;
    void bark() { System.out.println(name + " the " + breed + " says Woof!"); }
}
public class InheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.name = "Bruno";          // inherited field
        d.breed = "Labrador";      // own field
        d.eat();                   // inherited method
        d.bark();                  // own method
        d.sleep();
    }
}
```

OUTPUT

```
Bruno is eating
Bruno the Labrador says Woof!
Bruno is sleeping
```

10.7.2 Types of inheritance



THE FIVE SHAPES OF INHERITANCE; ARROWS POINT FROM CHILD TO PARENT

10.7.2.1 Single

One child, one parent: `class Dog extends Animal`.

10.7.2.2 Multilevel

A chain: `C extends B`, `B extends A`. C gets everything from both B and A.

10.7.2.3 Hierarchical

Several children share one parent: Dog, Cat and Cow all extend Animal.

SINGLE, MULTILEVEL AND HIERARCHICAL IN ONE PROGRAM — InheritanceTypes.java

```
class Vehicle { void start() { System.out.println(getClass().getSimpleName() + ": engine started"); } }
class Car extends Vehicle { void openBoot() { System.out.println("Car: boot opened"); } }          // single
class SportsCar extends Car { void turbo() { System.out.println("SportsCar: turbo on!"); } }        // multilevel
class Bike extends Vehicle { void kick() { System.out.println("Bike: kick start"); } }             // hierarchical

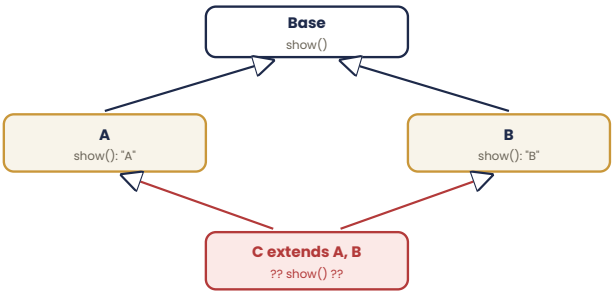
public class InheritanceTypes {
    public static void main(String[] args) {
        SportsCar s = new SportsCar();
        s.start(); s.openBoot(); s.turbo();          // methods from 3 levels
        Bike b = new Bike();
        b.start(); b.kick();
    }
}
```

OUTPUT

```
SportsCar: engine started
Car: boot opened
SportsCar: turbo on!
Bike: engine started
Bike: kick start
```

10.7.2.4 Multiple (not with classes; why: the diamond problem)

A class **cannot** extend two classes in Java: `class C extends A, B` is a compile error. Reason: the **diamond problem**. If A and B both inherit `show()` from a common parent and both override it differently, which version should C inherit? C++ allows this and needs complex rules; Java avoids the ambiguity entirely for classes.



THE DIAMOND PROBLEM: C CANNOT KNOW WHICH SHOW() TO INHERIT

10.7.2.5 Hybrid (through interfaces)

A class can extend **one** class and implement **many** interfaces, giving a hybrid of single and multiple inheritance. Interfaces avoid the diamond problem because they have no instance state, and if two default methods clash the class **must** override and choose (10.9.3.3).

EXTENDS ONE CLASS, IMPLEMENTS TWO INTERFACES

Hybrid.java

```
interface Flyable  { default String move() { return "fly"; } }
interface Swimmable { default String move() { return "swim"; } }
class Bird { String name = "Duck"; }

class Duck extends Bird implements Flyable, Swimmable {
    @Override
    public String move() {            // must resolve the clash explicitly
        return Flyable.super.move() + " and " + Swimmable.super.move();
    }
}
public class Hybrid {
    public static void main(String[] args) {
        Duck d = new Duck();
        System.out.println(d.name + " can " + d.move());
    }
}
```

OUTPUT

```
Duck can fly and swim
```

10.7.3 super keyword

`super` refers to the **parent part** of the current object. It is used to reach parent members that the child has hidden or overridden, and to call the parent constructor.

10.7.3.1 Accessing parent variables

10.7.3.2 Calling parent methods

10.7.3.3 Calling parent constructors with super()

THREE USES OF SUPER

SuperDemo.java

```
class Person {
    String type = "Person";
    String name;
    Person(String name) {
        this.name = name;
        System.out.println("Person constructor for " + name);
    }
    void describe() { System.out.println("I am " + name); }
}
class Teacher extends Person {
    String type = "Teacher";        // hides the parent field
    String subject;
    Teacher(String name, String subject) {
        super(name);                // 10.7.3.3 must be the first statement
        this.subject = subject;
        System.out.println("Teacher constructor");
    }
    @Override
    void describe() {
        super.describe();           // 10.7.3.2 run the parent version first
        System.out.println("I teach " + subject);
    }
    void types() {
        System.out.println("this.type = " + this.type + ", super.type = " + super.type);  // 10.7.3.1
    }
}
public class SuperDemo {
    public static void main(String[] args) {
        Teacher t = new Teacher("Mr. Rao", "Java");
        t.describe();
        t.types();
    }
}
```

OUTPUT

```
Person constructor for Mr. Rao
Teacher constructor
I am Mr. Rao
I teach Java
this.type = Teacher, super.type = Person
```

	`this`	`super`
Refers to	Current object (child view)	Parent part of the current object
Field	`this.x` : own field	`super.x` : parent's field
Method	`this.m()`	`super.m()` : parent version
Constructor	`this(...)` : same class	`super(...)` : parent class
Rule	First statement; cannot use both `this()` and `super()` in one constructor	

10.7.4 Constructor execution order in inheritance

Every constructor's first action is to call a parent constructor: either an explicit `super(...)` or an **implicit** `super()` added by the compiler. So construction runs from the **top of the hierarchy down**: Object → grandparent → parent → child. (If the parent has no no-arg constructor, the child **must** call `super(args)` explicitly.)

PARENTS ARE BUILT FIRST

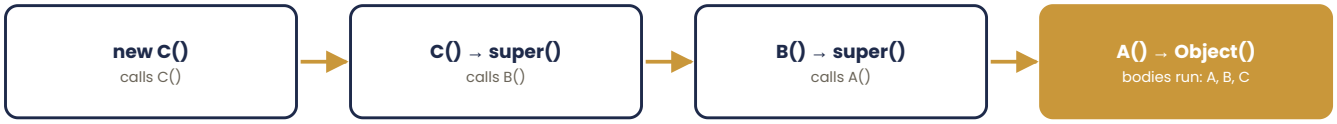
ConstructorOrder.java

```
class A { A() { System.out.println("1. A constructor"); } }
class B extends A { B() { System.out.println("2. B constructor"); } }       // implicit super()
class C extends B { C() { super(); System.out.println("3. C constructor"); } }

public class ConstructorOrder {
    public static void main(String[] args) {
        new C();
    }
}
```

OUTPUT

```
1. A constructor
2. B constructor
3. C constructor
```



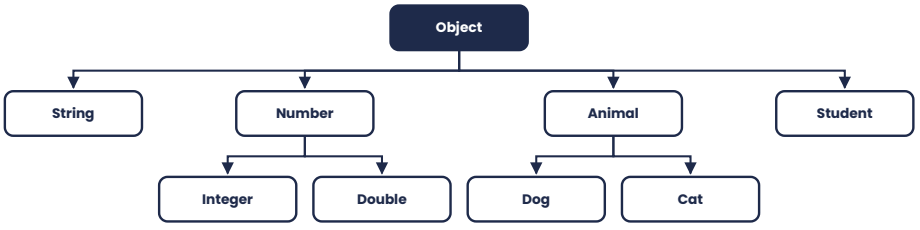
CALLS TRAVEL UP; BODIES EXECUTE TOP-DOWN

10.7.5 What is and is not inherited (private members, constructors)

Member	Inherited?	Notes
public / protected fields and methods	Yes	Usable directly in the child
default (package) members	Yes, if the child is in the same package	
private fields	Present in the object but **not accessible**	Reach them via public/protected getters
private methods	No	Cannot be overridden
Constructors	No	Called with `super(...)`
static members	Accessible through the child	Not overridden, only hidden (10.8.6)
Initializer blocks	No	Run as part of the parent constructor

10.7.6 The Object class as the root of all classes

Every class that does not say `extends` automatically extends `java.lang.Object`. So every object in Java has Object's methods: `toString()`, `equals()`, `hashCode()`, `getClass()`, `clone()`, `wait()/notify()` (10.10). An `Object` variable can refer to **any** object.



OBJECT SITS AT THE TOP OF EVERY CLASS HIERARCHY

10.8 Polymorphism

10.8.1 Meaning: one interface, many forms

Polymorphism (Greek: "many forms") means one name or one call can behave differently depending on the object or the arguments. A person is a student in class, a customer in a shop and a son at home: the same person, different

behaviour in different roles. In Java the same method call `shape.area()` computes a circle's area for a Circle and a rectangle's area for a Rectangle.

Kind	Mechanism	Decided when	Also called
Compile-time	Method overloading	At compile time, from the argument types	Static binding, early binding
Runtime	Method overriding	At run time, from the actual object	Dynamic binding, late binding, dynamic method dispatch

10.8.2 Compile-time polymorphism (method overloading)

Covered in 9.6: several methods with the same name but different parameter lists. The compiler chooses one by looking at the **declared types** of the arguments. Constructors can be overloaded the same way (10.3.5).

ONE NAME, CHOSEN BY ARGUMENT TYPE CompileTimePoly.java

```
class Printer {
    void print(int n)         { System.out.println("int: " + n); }
    void print(String s)      { System.out.println("String: " + s); }
    void print(int[] a)       { System.out.println("array of " + a.length); }
}
public class CompileTimePoly {
    public static void main(String[] args) {
        Printer p = new Printer();
        p.print(42);
        p.print("Java");
        p.print(new int[] {1, 2, 3});
    }
}
```

OUTPUT

```
int: 42
String: Java
array of 3
```

10.8.3 Runtime polymorphism (method overriding)

Overriding: a subclass provides its **own version** of a method it inherited, with the **same signature**. When the method is called through a parent-type reference, the JVM runs the version belonging to the **actual object**.

ONE LOOP, FOUR BEHAVIOURS RuntimePoly.java

```
class Shape {
    double area() { return 0; }
    String name() { return "Shape"; }
}
class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    @Override double area() { return Math.PI * r * r; }
    @Override String name() { return "Circle"; }
}
class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) { this.w = w; this.h = h; }
    @Override double area() { return w * h; }
    @Override String name() { return "Rectangle"; }
}
class Square extends Rectangle {
    Square(double s) { super(s, s); }
    @Override String name() { return "Square"; }      // area() inherited from Rectangle
}
public class RuntimePoly {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(1), new Rectangle(3, 4), new Square(5), new Shape() };
        for (Shape s : shapes)                                  // same call ...
            System.out.printf("%-10s area = %.2f%n", s.name(), s.area());   // ... different behaviour
    }
}
```

OUTPUT

```
Circle     area = 3.14
Rectangle  area = 12.00
Square     area = 25.00
Shape      area = 0.00
```

10.8.3.1 Rules of overriding

- Same method **name** and same **parameter list** (otherwise it is overloading, not overriding).
- Return type: same, or a **subtype** (covariant return, 10.8.3.3). For primitives it must be identical.
- Access: same or **wider** (cannot reduce visibility).
- Checked exceptions: same, narrower or none; never new or broader checked exceptions.
- `final`, `static` and `private` methods cannot be overridden.
- Constructors are not inherited, so they cannot be overridden.
- There must be an IS-A relationship (inheritance or interface implementation).

10.8.3.2 @Override annotation

Put `@Override` on every overriding method. It tells the compiler "I intend to override", so a typo in the name or a wrong parameter type becomes a **compile error** instead of a silent new method.

@OVERRIDE CATCHES SPELLING MISTAKES java

```
class Animal { void makeSound() { } }
class Dog extends Animal {
    @Override
    void makesound() { }    // error: method does not override or implement a method from a supertype
}
```

10.8.3.3 Covariant return types

Since Java 5, an overriding method may return a **more specific** (sub)type than the parent method. This saves the caller from casting.

RETURNING A SUBTYPE Covariant.java

```
class Animal {
    Animal reproduce() { return new Animal(); }
}
class Cat extends Animal {
    @Override
    Cat reproduce() { return new Cat(); }     // Cat is a subtype of Animal: allowed
    void meow() { System.out.println("Meow from a new kitten"); }
}
public class Covariant {
    public static void main(String[] args) {
        Cat c = new Cat();
        c.reproduce().meow();                 // no cast needed
    }
}
```

OUTPUT

```
Meow from a new kitten
```

10.8.3.4 Access modifier and exception rules

Parent method is	Overriding method may be
`public`	`public` only
`protected`	`protected` or `public`
default (package)	default, `protected` or `public`
`private`	not inherited, so cannot be overridden

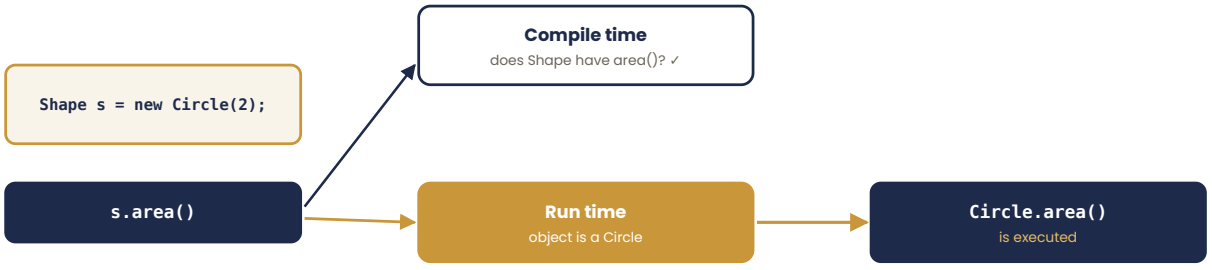
ACCESS AND EXCEPTION RULES java

```
import java.io.*;
class Reader {
    public void read() throws IOException { }
}
class FileReader2 extends Reader {
    // void read() { }                                  // error: weaker access (public -> default)
    // public void read() throws Exception { }          // error: broader checked exception
    @Override public void read() throws FileNotFoundException { }  // OK: narrower exception
}
```

Why? Code written against the parent type (`Reader r`) must keep working with any subclass. A subclass that was less accessible, or that threw new checked exceptions, would break that code.

10.8.3.5 Dynamic method dispatch

For a call `ref.method()`, the compiler only checks that the **reference type** has such a method. At run time the JVM looks at the **actual object** and runs that class's version (searching up the hierarchy if the class does not override it). This is **dynamic method dispatch**, implemented with a per-class method table (vtable).



THE REFERENCE TYPE IS CHECKED AT COMPILE TIME; THE OBJECT TYPE PICKS THE METHOD AT RUN TIME

PAY() NEVER NEEDS TO CHANGE WHEN A NEW TYPE OF EMPLOYEE IS ADDED

Dispatch.java

```
class Employee {
    String name;
    Employee(String name) { this.name = name; }
    double salary() { return 30000; }
}
class Manager extends Employee {
    Manager(String n) { super(n); }
    @Override double salary() { return super.salary() + 20000; }
}
class Intern extends Employee {
    Intern(String n) { super(n); }
    @Override double salary() { return 10000; }
}
public class Dispatch {
    static void pay(Employee e) {                    // works for ALL present and future subclasses
        System.out.println(e.name + " (" + e.getClass().getSimpleName() + ") paid " + e.salary());
    }
    public static void main(String[] args) {
        pay(new Employee("Anil"));
        pay(new Manager("Bhavna"));
        pay(new Intern("Chirag"));
    }
}
```

OUTPUT

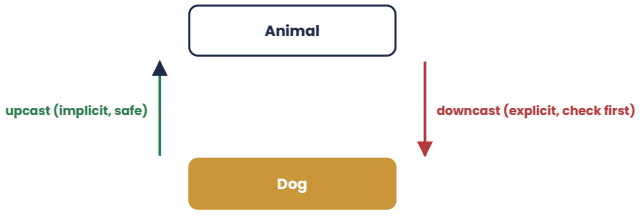
```
Anil (Employee) paid 30000.0
Bhavna (Manager) paid 50000.0
Chirag (Intern) paid 10000.0
```

Write code against the **parent type** and let each subclass supply its own behaviour. Adding a new subclass then requires **no change** to existing code (the Open/Closed Principle, Module 21).

10.8.4 Upcasting and downcasting

Upcasting: treating a child object as its parent type, `Animal a = new Dog();`. It is always safe and automatic. Through `a` you can call only methods declared in Animal (overridden versions still run).

Downcasting: converting a parent reference back to a child type, `Dog d = (Dog) a;`. It needs an explicit cast and is safe only if the object really is a Dog; otherwise a `ClassCastException` is thrown at run time.



UP THE HIERARCHY IS AUTOMATIC; DOWN NEEDS A CAST

10.8.5 instanceof and ClassCastException

`obj instanceof Type` returns true if obj is a Type (or a subtype) and not null. Always check before downcasting. Since Java 16, **pattern matching** combines the test and the cast: `if (a instanceof Dog d) d.bark();`

SAFE AND UNSAFE DOWNCASTING Casting.java

```
class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Woof"); }
    void fetch() { System.out.println("Dog fetches the ball"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}
public class Casting {
    public static void main(String[] args) {
        Animal a = new Dog();          // upcast
        a.sound();                     // Woof (overridden version)
        // a.fetch();                  // error: Animal has no fetch()

        if (a instanceof Dog) {        // classic check + cast
            Dog d = (Dog) a;
            d.fetch();
        }
        Animal b = new Cat();
        if (b instanceof Dog d2) d2.fetch();          // Java 16 pattern: false, so skipped
        else System.out.println("b is not a Dog");

        System.out.println("null instanceof Animal: " + (null instanceof Animal));
        Dog wrong = (Dog) b;           // compiles, fails at run time
    }
}
```

OUTPUT

```
Woof
Dog fetches the ball
b is not a Dog
null instanceof Animal: false
Exception in thread "main" java.lang.ClassCastException: class Cat cannot be cast to class Dog (Cat and Dog are in
unnamed module of loader 'app')
        at Casting.main(Casting.java:24)
```

10.8.6 Static methods are hidden, not overridden

A static method belongs to the class, not the object, so it is **not** dispatched dynamically. If a child declares a static method with the same signature, it **hides** the parent's method: the version called depends on the **reference type**, not the object.

HIDING VS OVERRIDING Hiding.java

```
class Parent {
    static void greet()  { System.out.println("static Parent.greet"); }
    void hello()         { System.out.println("instance Parent.hello"); }
}
class Child extends Parent {
    static void greet()  { System.out.println("static Child.greet"); }      // hides
    @Override void hello() { System.out.println("instance Child.hello"); }   // overrides
}
public class Hiding {
    public static void main(String[] args) {
        Parent p = new Child();
        p.greet();     // Parent version: chosen by the reference type (compile time)
        p.hello();     // Child version: chosen by the object (run time)
        Child.greet();
    }
}
```

OUTPUT

```
static Parent.greet
instance Child.hello
static Child.greet
```

WATCH OUT
Calling a static method through an object reference compiles, but is misleading. Always call static methods through the class name: `Parent.greet()`.

10.8.7 Overloading vs overriding comparison

	Overloading	Overriding
Where	Same class (or child adds a new overload)	Child class redefines parent method
Signature	Same name, **different** parameters	Same name, **same** parameters
Return type	Can be anything	Same or covariant subtype
Access modifier	Any	Same or wider
Exceptions	Any	Same, narrower or none (checked)
Binding	Compile time (static)	Run time (dynamic)
static / private / final methods	Can be overloaded	Cannot be overridden
Purpose	Convenience: same action, different inputs	Specialise behaviour in a subclass

10.9 Abstraction

10.9.1 What abstraction is (what, not how)

Abstraction means exposing only the essential operations of something and hiding the implementation details. An ATM shows "Withdraw", "Balance", "Deposit". You do not see how it talks to the bank server. In Java, abstraction is achieved with **abstract classes** (partial abstraction) and **interfaces** (full abstraction of behaviour).

Encapsulation	Abstraction
Hides **data** (private fields) to protect it	Hides **complexity** (implementation) to simplify use
Achieved with access modifiers, getters/setters	Achieved with abstract classes and interfaces
Focus: *how to protect*	Focus: *what to expose*

10.9.2 Abstract classes

An **abstract class** is declared with `abstract`. It represents a general idea (Shape, Account, Employee) that is not meant to exist on its own, and it can mix finished code with "to be completed" methods.

10.9.2.1 Abstract methods

An abstract method has a signature but **no body**: `abstract double area();`. Every concrete (non-abstract) subclass **must** override it; a subclass that does not becomes abstract itself. Any class with an abstract method must be declared abstract.

10.9.2.2 Constructors and concrete methods in abstract classes

An abstract class can have fields, constructors, normal methods, static methods and final methods. The constructor runs when a subclass object is created (through `super(...)`), to initialise the common fields.

10.9.2.3 Cannot be instantiated

`new Shape()` is a compile error for an abstract Shape. You can, however, declare **references** of the abstract type: `Shape s = new Circle(2);`.

AN ABSTRACT CLASS WITH A CONSTRUCTOR, ABSTRACT METHODS AND A CONCRETE METHOD

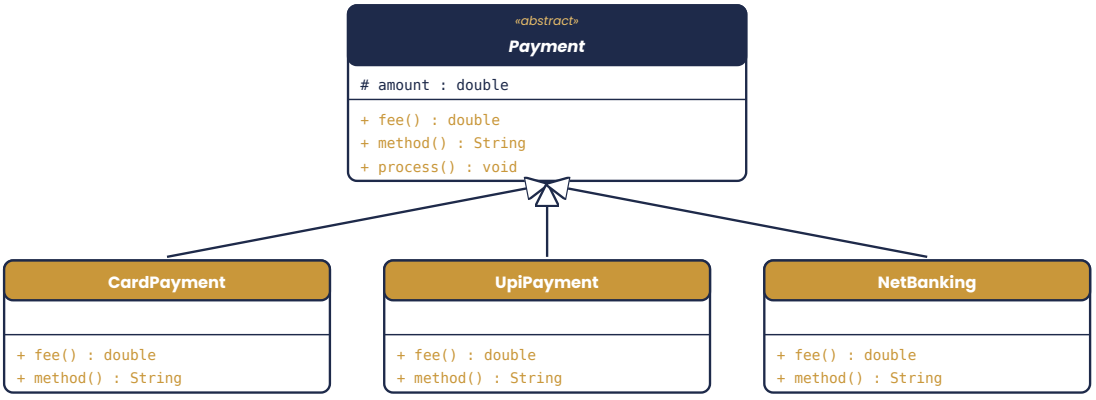
AbstractDemo.java

```
abstract class Payment {
    protected final double amount;
    Payment(double amount) {                          // constructor in abstract class
        this.amount = amount;
    }
    abstract double fee();                            // abstract: each type decides
    abstract String method();

    final void process() {                            // concrete "template" method
        System.out.printf("%-12s amount %8.2f  fee %6.2f  total %8.2f%n",
                method(), amount, fee(), amount + fee());
    }
}
class CardPayment extends Payment {
    CardPayment(double a) { super(a); }
    double fee() { return amount * 0.02; }
    String method() { return "Card"; }
}
class UpiPayment extends Payment {
    UpiPayment(double a) { super(a); }
    double fee() { return 0; }
    String method() { return "UPI"; }
}
class NetBanking extends Payment {
    NetBanking(double a) { super(a); }
    double fee() { return 15; }
    String method() { return "NetBanking"; }
}
public class AbstractDemo {
    public static void main(String[] args) {
        Payment[] payments = { new CardPayment(2500), new UpiPayment(2500), new NetBanking(2500) };
        for (Payment p : payments) p.process();
        // Payment p = new Payment(100);   // error: Payment is abstract; cannot be instantiated
    }
}
```

OUTPUT

```
Card         amount  2500.00  fee  50.00  total  2550.00
UPI          amount  2500.00  fee   0.00  total  2500.00
NetBanking   amount  2500.00  fee  15.00  total  2515.00
```



UML: ONE ABSTRACT PARENT, THREE CONCRETE CHILDREN

10.9.3 Interfaces

An **interface** is a pure contract: a list of method signatures that a class promises to provide. It says **what** a type can do (Comparable: "can be compared", Runnable: "can be run"), not what it is. A class **implements** an interface and must provide all its abstract methods.

10.9.3.1 Declaring and implementing an interface

IMPLEMENTING INTERFACES InterfaceDemo.java

```
interface Drawable {
    void draw();                    // implicitly public abstract
}
interface Resizable {
    void resize(double factor);
}
class Picture implements Drawable, Resizable {
    double size = 10;
    public void draw()                  { System.out.println("Drawing picture of size " + size); }
    public void resize(double factor)   { size *= factor; }
}
class Text implements Drawable {
    public void draw() { System.out.println("Drawing text"); }
}
public class InterfaceDemo {
    public static void main(String[] args) {
        Drawable[] items = { new Picture(), new Text() };
        for (Drawable d : items) d.draw();       // polymorphism through an interface
        Picture p = new Picture();
        p.resize(2.5);
        p.draw();
    }
}
```

OUTPUT

```
Drawing picture of size 10.0
Drawing text
Drawing picture of size 25.0
```

NOTE

Interface methods are implicitly `public`, so implementing methods must be declared `public` too. Forgetting this gives "attempting to assign weaker access privileges".

10.9.3.2 Interface variables (public static final)

Every field in an interface is automatically `public static final`: a constant. Interfaces cannot have instance fields, so they hold no object state.

INTERFACE CONSTANTS java

```
interface Limits {
    int MAX_USERS = 100;          // really: public static final int MAX_USERS = 100;
}
// Limits.MAX_USERS = 200;       // error: cannot assign a value to final variable
```

10.9.3.3 Multiple inheritance through interfaces

A class can implement any number of interfaces, and an interface can extend several interfaces. This gives multiple inheritance of **type** without the diamond problem for state. If two interfaces provide the same default method, the class must override it (see 10.7.2.5 for `X.super.method()`).

10.9.3.4 Default methods (Java 8)

A `default` method has a body inside the interface. Implementing classes inherit it and may override it. Default methods let library designers add new methods to an interface without breaking every existing class; for example, Java 8 added `forEach` to `Iterable` and `sort` to `List`.

10.9.3.5 Static methods in interfaces (Java 8)

A static method in an interface is a helper that belongs to the interface. It is called as `InterfaceName.method()` and is **not** inherited by implementing classes. Example: `Comparator.naturalOrder()`.

10.9.3.6 Private methods in interfaces (Java 9)

Private (and private static) methods let several default methods share common code without exposing it to implementing classes.

ABSTRACT, DEFAULT, PRIVATE AND STATIC INTERFACE METHODS TOGETHER InterfaceMethods.java

```
interface Logger {
    void write(String msg);                                  // abstract

    default void info(String msg)  { write(format("INFO", msg)); }     // default (Java 8)
    default void error(String msg) { write(format("ERROR", msg)); }

    private String format(String level, String msg) {        // private helper (Java 9)
        return "[" + level + "] " + msg;
    }
    static Logger console() {                                 // static factory (Java 8)
        return msg -> System.out.println(msg);
    }
}
class FileLogger implements Logger {
    public void write(String msg) { System.out.println("file <- " + msg); }
    @Override public void error(String msg) { write("!!! " + msg.toUpperCase()); }   // override a default
}
public class InterfaceMethods {
    public static void main(String[] args) {
        Logger c = Logger.console();
        c.info("server started");
        c.error("disk almost full");
        Logger f = new FileLogger();
        f.info("backup done");
        f.error("backup failed");
    }
}
```

OUTPUT

```
[INFO] server started
[ERROR] disk almost full
file <- [INFO] backup done
file <- !!! BACKUP FAILED
```

10.9.3.7 Functional interfaces

An interface with **exactly one abstract method** (default and static methods do not count) is a **functional interface**. It can be implemented with a short **lambda expression** instead of a whole class. Mark it with `@FunctionalInterface` so the compiler checks it. Examples: `Runnable`, `Comparator`, `Predicate`. Module 17 covers this in depth.

A FUNCTIONAL INTERFACE IMPLEMENTED BY LAMBDAS FunctionalDemo.java

```
@FunctionalInterface
interface Operation {
    int apply(int a, int b);
}
public class FunctionalDemo {
    public static void main(String[] args) {
        Operation add = (a, b) -> a + b;          // lambda implements apply()
        Operation mul = (a, b) -> a * b;
        Operation max = Math::max;                // method reference
        System.out.println("add: " + add.apply(6, 7));
        System.out.println("mul: " + mul.apply(6, 7));
        System.out.println("max: " + max.apply(6, 7));
    }
}
```

OUTPUT

```
add: 13
mul: 42
max: 7
```

10.9.3.8 Marker interfaces (Serializable, Cloneable)

A **marker interface** has **no methods at all**. Implementing it simply "tags" a class so that the JVM or a library treats it specially: `Serializable` allows object serialization (Module 15); `Cloneable` allows `Object.clone()` (otherwise it throws CloneNotSupportedException). Today annotations (Module 21) are often used instead.

10.9.3.9 Interface extending interfaces

COMBINING INTERFACES

InterfaceExtends.java

```
interface Readable2 { String read(); }
interface Writable2 { void write(String s); }
interface Storage extends Readable2, Writable2 {      // an interface may extend MANY interfaces
    default void copyTo(Storage other) { other.write(read()); }
}
class MemoryStorage implements Storage {
    private String data = "";
    public String read() { return data; }
    public void write(String s) { data = s; }
}
public class InterfaceExtends {
    public static void main(String[] args) {
        MemoryStorage a = new MemoryStorage(), b = new MemoryStorage();
        a.write("LBF notes");
        a.copyTo(b);
        System.out.println("b contains: " + b.read());
    }
}
```

OUTPUT

```
b contains: LBF notes
```

10.9.4 Abstract class vs interface: when to use which

	Abstract class	Interface
Keyword	`abstract class` / `extends`	`interface` / `implements`
Methods	Abstract and concrete	Abstract, default, static, private
Fields	Any kind (instance state)	Only `public static final` constants
Constructors	Yes	No
Inheritance	A class extends only **one**	A class implements **many**
Access of members	Any	Public (except private helpers)
Models	IS-A: a shared base with common state and code	CAN-DO: a capability that unrelated classes can share
Example	`Account` → Savings, Current	`Comparable`, `Runnable`, `Payable`

Rule of thumb: start with an **interface** for the contract. Add an **abstract class** when several implementations share real state or code. Many designs use both: `List` (interface) ← `AbstractList` (abstract class) ← `ArrayList`.

10.10 Object class methods

Every class inherits these methods from `java.lang.Object`. The first three are overridden in almost every real class.

10.10.1 toString()

Returns a text description of the object. It is called automatically by `println` and string concatenation. The default prints `ClassName@hexHashCode`, which is rarely useful, so override it.

10.10.2 equals() and hashCode() and their contract

`equals()` decides whether two objects are **meaningfully equal**. The default from Object is the same as `==` (same object). Override it to compare fields. Whenever you override equals you **must** override `hashCode()`, because HashMap and HashSet first use the hash to find a bucket and only then call equals.

Reflexive
`x.equals(x)` is true.

Symmetric
x.equals(y) ⇔ y.equals(x).

Transitive
x = y and y = z ⇒ x = z.

Consistent
Same answer each call while the objects are unchanged.

Null
`x.equals(null)` is false.

hashCode rule
Equal objects **must** have equal hash codes. (Unequal objects may collide.)

OVERRIDING TOSTRING, EQUALS AND HASHCODE

ObjectMethods.java

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

class PointNoEq {
    int x, y;
    PointNoEq(int x, int y) { this.x = x; this.y = y; }
}
class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override public String toString() { return "Point(" + x + ", " + y + ")"; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;                      // same object
        if (!(o instanceof Point p)) return false;       // null or other type
        return x == p.x && y == p.y;                     // compare fields
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
}
public class ObjectMethods {
    public static void main(String[] args) {
        PointNoEq a = new PointNoEq(1, 2), b = new PointNoEq(1, 2);
        System.out.println("default toString : " + (a.toString().startsWith("PointNoEq@") ? "PointNoEq@<hash>" : a));
        System.out.println("default equals   : " + a.equals(b));

        Point p = new Point(1, 2), q = new Point(1, 2);
        System.out.println("toString         : " + p);
        System.out.println("p == q           : " + (p == q));
        System.out.println("p.equals(q)      : " + p.equals(q));
        System.out.println("same hashCode    : " + (p.hashCode() == q.hashCode()));
        Set<Point> set = new HashSet<>();
        set.add(p); set.add(q);
        System.out.println("set size         : " + set.size() + " (duplicates removed)");
    }
}
```

OUTPUT

```
default toString : PointNoEq@<hash>
default equals   : false
toString         : Point(1, 2)
p == q           : false
p.equals(q)      : true
same hashCode    : true
set size         : 1 (duplicates removed)
```

WATCH OUT
Overriding equals but **not** hashCode breaks hash-based collections: two equal points could land in different buckets and both be stored in a HashSet.

10.10.3 getClass()

`obj.getClass()` returns the **runtime class** of the object as a `Class` object. It is final and gives the actual type even through a parent reference: `Animal a = new Dog(); a.getClass().getSimpleName()` is "Dog". It is the entry point to reflection (Module 21).

10.10.4 clone(): shallow vs deep cloning

`clone()` creates a field-by-field copy of an object. The class must implement `Cloneable` and usually overrides `clone()` as public. The default copy is **shallow**: primitive fields are copied, but reference fields are copied as references, so the copy **shares** inner objects with the original. A **deep** clone also copies those inner objects.

SHALLOW COPY SHARES INNER OBJECTS; DEEP COPY DOES NOT

CloneDemo.java

```
import java.util.Arrays;

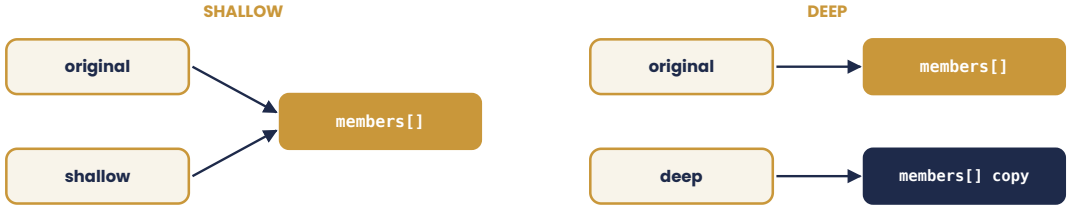
class Team implements Cloneable {
    String name;
    String[] members;
    Team(String name, String... members) { this.name = name; this.members = members; }

    Team shallowCopy() throws CloneNotSupportedException {
        return (Team) super.clone();                // members array is SHARED
    }
    Team deepCopy() throws CloneNotSupportedException {
        Team t = (Team) super.clone();
        t.members = members.clone();                // copy the array too
        return t;
    }
}
public class CloneDemo {
    public static void main(String[] args) throws CloneNotSupportedException {
        Team original = new Team("Alpha", "Asha", "Ravi");
        Team shallow = original.shallowCopy();
        Team deep = original.deepCopy();

        shallow.members[0] = "CHANGED";
        System.out.println("original after shallow change: " + Arrays.toString(original.members));
        deep.members[1] = "DEEP";
        System.out.println("original after deep change   : " + Arrays.toString(original.members));
        System.out.println("deep copy                    : " + Arrays.toString(deep.members));
    }
}
```

OUTPUT

```
original after shallow change: [CHANGED, Ravi]
original after deep change   : [CHANGED, Ravi]
deep copy                    : [Asha, DEEP]
```



SHALLOW: ONE SHARED ARRAY. DEEP: EACH OBJECT OWNS ITS ARRAY

TIP

Many developers prefer a **copy constructor** (10.3.7) or a static `copyOf` method over clone(), because they are clearer and do not need Cloneable or casts.

10.10.5 finalize() (deprecated) and why to avoid it

`finalize()` was meant to run just before the garbage collector reclaims an object. It is **deprecated** (Java 9) and marked for removal (Java 18): there is no guarantee when or even **whether** it runs, it slows down garbage collection, and it can accidentally bring objects back to life. To release resources (files, connections), use **try-with-resources** and `AutoCloseable` (Module 12) or `java.lang.ref.Cleaner`.

10.10.6 wait(), notify(), notifyAll() (covered in Module 16)

These final methods let threads coordinate through an object's monitor lock: `wait()` releases the lock and sleeps until another thread calls `notify()` / `notifyAll()` on the same object. They must be called inside a `synchronized` block. Details

and examples are in Module 16.

Method	Purpose	Override?
`toString()`	Text form	Almost always
`equals(Object)`	Logical equality	For value-like classes
`hashCode()`	Hash for hash tables	Whenever equals is overridden
`getClass()`	Runtime class	Cannot (final)
`clone()`	Field-by-field copy	Optional, with Cloneable
`finalize()`	Cleanup before GC	No: deprecated
`wait/notify/notifyAll`	Thread coordination	Cannot (final)

10.11 Relationships between classes

10.11.1 IS-A (inheritance)

IS-A is expressed with `extends` or `implements`: *a Car IS-A Vehicle*. Test it by saying the sentence aloud; if it sounds wrong ("a Car is an Engine"), inheritance is the wrong tool.

10.11.2 HAS-A: association

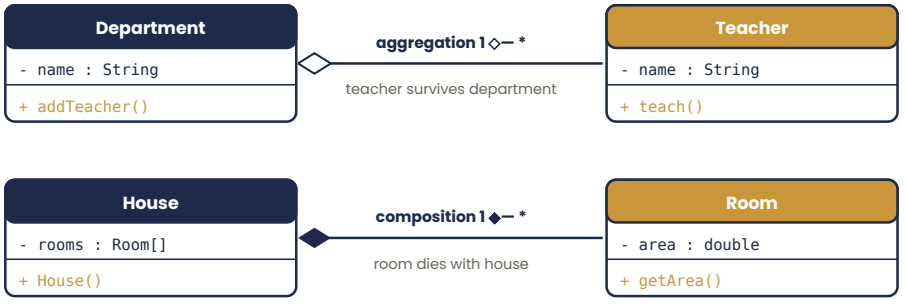
Association means one class uses or holds a reference to another: a Car HAS-A Engine, a Teacher HAS Students. It is implemented with a field. Associations differ in how strongly the objects' lifetimes are tied.

10.11.2.1 Aggregation (weak HAS-A)

The part can **exist independently** of the whole and may be shared. A Department has Teachers, but if the department closes, the teachers still exist. Typically the part is created outside and passed in. UML: hollow diamond.

10.11.2.2 Composition (strong HAS-A)

The part **cannot exist without** the whole, and is owned by it alone. A House has Rooms; destroy the house and its rooms are gone. Typically the whole creates its parts inside its constructor. UML: filled diamond.



AGGREGATION (HOLLOW DIAMOND) VS COMPOSITION (FILLED DIAMOND)

AGGREGATION AND COMPOSITION IN CODE

Relationships.java

```
import java.util.ArrayList;
import java.util.List;

class Teacher {
    String name;
    Teacher(String name) { this.name = name; }
}
class Department {                                  // AGGREGATION
    String name;
    List<Teacher> teachers = new ArrayList<>();
    Department(String name) { this.name = name; }
    void addTeacher(Teacher t) { teachers.add(t); }   // part created outside, passed in
}
class Room {
    String type;
    Room(String type) { this.type = type; }
}
class House {                                       // COMPOSITION
    private final List<Room> rooms = new ArrayList<>();
    House() {                                       // parts created inside, owned only here
        rooms.add(new Room("Hall"));
        rooms.add(new Room("Kitchen"));
        rooms.add(new Room("Bedroom"));
    }
    int roomCount() { return rooms.size(); }
}
public class Relationships {
    public static void main(String[] args) {
        Teacher rao = new Teacher("Mr. Rao");
        Department cs = new Department("CS");
        Department it = new Department("IT");
        cs.addTeacher(rao);
        it.addTeacher(rao);                         // shared by two departments
        cs = null;                                  // department gone ...
        System.out.println(rao.name + " still exists and teaches in IT: " + it.teachers.get(0).name);

        House h = new House();
        System.out.println("house has " + h.roomCount() + " rooms (they live and die with it)");
    }
}
```

OUTPUT

```
Mr. Rao still exists and teaches in IT: Mr. Rao
house has 3 rooms (they live and die with it)
```

10.11.3 Composition over inheritance

A well-known design guideline (from the "Gang of Four" book): prefer building classes by **combining** objects (HAS-A) over **extending** them (IS-A), unless there is a true IS-A relationship. Inheritance exposes the parent's internals, ties the child to every future parent change, and is fixed at compile time. Composition is flexible: parts can be swapped even at run time.

SWAPPING BEHAVIOUR THROUGH COMPOSITION CompositionOverInheritance.java

```
interface Engine { String start(); }
class PetrolEngine implements Engine { public String start() { return "petrol engine: vroom"; } }
class ElectricEngine implements Engine { public String start() { return "electric motor: hmmm"; } }

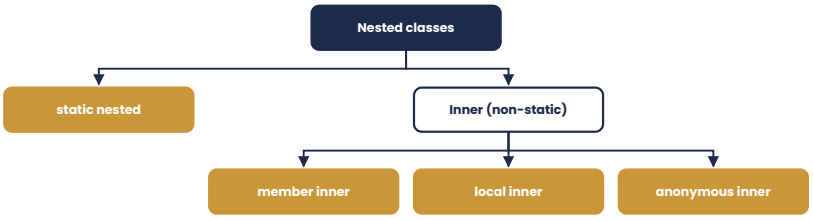
class Car {
    private Engine engine;                              // HAS-A, not IS-A
    Car(Engine engine) { this.engine = engine; }
    void setEngine(Engine e) { this.engine = e; }       // swap at run time
    void drive() { System.out.println("Car starts with " + engine.start()); }
}
public class CompositionOverInheritance {
    public static void main(String[] args) {
        Car car = new Car(new PetrolEngine());
        car.drive();
        car.setEngine(new ElectricEngine());             // impossible with "Car extends PetrolEngine"
        car.drive();
    }
}
```

OUTPUT

```
Car starts with petrol engine: vroom
Car starts with electric motor: hmmm
```

10.12 Inner (nested) classes

A class can be declared **inside** another class. Nesting groups classes that belong together, increases encapsulation (the inner class can be private) and gives the inner class access to the outer class's members.



KINDS OF NESTED CLASSES

10.12.1 Member inner class

A non-static class declared at member level. Each inner object is tied to an **outer object** and can access all its fields, including private ones. Create it with `outer.new Inner()`.

10.12.2 Static nested class

Declared with `static`. It does not belong to any outer object, so it can only access the outer class's **static** members directly. Create it with `new Outer.Nested()`. Common for builders and helper types (`Map.Entry`).

10.12.3 Local inner class

A class declared **inside a method**. It is visible only in that method and can use the method's local variables if they are **effectively final** (never re-assigned).

10.12.4 Anonymous inner class

A class with **no name**, declared and instantiated in one expression, usually to implement an interface or extend a class for one-time use. Before Java 8 this was the way to pass behaviour; for functional interfaces, a lambda is now shorter.

ALL FOUR KINDS OF NESTED CLASS NestedClasses.java

```
import java.util.Arrays;
import java.util.Comparator;

class University {
    private String name = "LBF University";
    private static int founded = 2020;
```

```
    class Department {                                  // 10.12.1 member inner
        String dept;
        Department(String dept) { this.dept = dept; }
        void show() { System.out.println(dept + " belongs to " + name); }   // outer private field
    }
    static class Stats {                                // 10.12.2 static nested
        void show() { System.out.println("founded " + founded); }         // static only
    }
    void ranking(int year) {
        int base = 50;                                  // effectively final
        class Rank {                                    // 10.12.3 local inner
            int compute() { return base - (year - founded); }
        }
        System.out.println("rank in " + year + ": " + new Rank().compute());
    }
}
public class NestedClasses {
    public static void main(String[] args) {
        University u = new University();
        University.Department d = u.new Department("Computer Science");
        d.show();
        University.Stats st = new University.Stats();
        st.show();
        u.ranking(2026);

        String[] names = {"Ravi", "Asha", "Meenakshi", "Om"};
        Arrays.sort(names, new Comparator<String>() {        // 10.12.4 anonymous inner
            @Override public int compare(String a, String b) { return a.length() - b.length(); }
        });
        System.out.println("by length (anonymous): " + Arrays.toString(names));
        Arrays.sort(names, (a, b) -> a.compareTo(b));         // same idea as a lambda
        System.out.println("alphabetical (lambda): " + Arrays.toString(names));
    }
}
```

OUTPUT

```
Computer Science belongs to LBF University
founded 2020
rank in 2026: 44
by length (anonymous): [Om, Ravi, Asha, Meenakshi]
alphabetical (lambda): [Asha, Meenakshi, Om, Ravi]
```

Kind	Needs outer object?	Accesses outer instance members?	Typical use
Member inner	Yes	Yes	Iterator over a collection's private data
Static nested	No	No (static only)	Builders, Node in a LinkedList, Map.Entry
Local inner	Yes (if in instance method)	Yes + effectively final locals	Helper used in one method
Anonymous	Depends where declared	Yes + effectively final locals	One-off listeners, comparators

10.13 Enums

10.13.1 Declaring enums

An **enum** is a special class with a **fixed set of named constants**: days of the week, order statuses, traffic-light colours. Using an enum instead of int or String constants gives type safety: a method taking `Level` cannot receive "MEDUIM" or 7.

AN ENUM

java

```
enum Level { LOW, MEDIUM, HIGH }     // each constant is a public static final Level object
Level l = Level.HIGH;
```

10.13.2 values(), valueOf(), ordinal(), name()

Method	Returns
`Level.values()`	Array of all constants in declaration order
`Level.valueOf("HIGH")`	The constant with that exact name (IllegalArgumentException if none)
`l.ordinal()`	Position, starting at 0
`l.name()` / `toString()`	The constant's name as a String
`l.compareTo(other)`	Compares by ordinal

10.13.3 Enums with fields, constructors and methods

Enums are full classes: they can have fields, a constructor (always private), methods, and even per-constant method bodies. Each constant calls the constructor with its arguments.

10.13.4 Enums in switch

ENUM METHODS, FIELDS AND SWITCH

EnumDemo.java

```
enum Planet {
    MERCURY(3.303e+23, 2.4397e6), EARTH(5.976e+24, 6.37814e6), JUPITER(1.9e+27, 7.1492e7);

    private final double mass, radius;                      // fields
    Planet(double mass, double radius) {                    // private constructor
        this.mass = mass; this.radius = radius;
    }
    double surfaceGravity() { return 6.67300E-11 * mass / (radius * radius); }
}
enum Status { PLACED, SHIPPED, DELIVERED, CANCELLED }

public class EnumDemo {
    static String message(Status s) {
        return switch (s) {                                 // enum in a switch expression
            case PLACED -> "We received your order";
            case SHIPPED -> "On the way";
            case DELIVERED -> "Enjoy!";
            case CANCELLED -> "Refund initiated";
        };
    }
    public static void main(String[] args) {
        for (Status s : Status.values())
            System.out.println(s.ordinal() + " " + s.name() + " -> " + message(s));
        Status s = Status.valueOf("SHIPPED");
        System.out.println("valueOf: " + s + ", after PLACED? " + (s.compareTo(Status.PLACED) > 0));
        for (Planet p : Planet.values())
            System.out.printf("%-8s gravity %.2f m/s2%n", p, p.surfaceGravity());
    }
}
```

OUTPUT

```
0 PLACED -> We received your order
1 SHIPPED -> On the way
2 DELIVERED -> Enjoy!
3 CANCELLED -> Refund initiated
valueOf: SHIPPED, after PLACED? true
MERCURY  gravity 3.70 m/s2
EARTH    gravity 9.80 m/s2
JUPITER  gravity 24.81 m/s2
```

10.13.5 EnumMap and EnumSet

`EnumMap` and `EnumSet` are Map and Set implementations specialised for enum keys. Internally they use arrays/bit-vectors indexed by ordinal, so they are very fast and compact, and they keep keys in declaration order.

ENUMSET AND ENUMMAP

EnumCollections.java

```
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.Map;

enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

public class EnumCollections {
    public static void main(String[] args) {
        EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
        EnumSet<Day> weekdays = EnumSet.complementOf(weekend);
        EnumSet<Day> midweek = EnumSet.range(Day.TUE, Day.THU);
        System.out.println("weekend : " + weekend);
        System.out.println("weekdays: " + weekdays);
        System.out.println("midweek : " + midweek);

        Map<Day, String> classes = new EnumMap<>(Day.class);
        classes.put(Day.WED, "OOP");
        classes.put(Day.MON, "Arrays");
        classes.put(Day.FRI, "Collections");
        System.out.println("timetable (in enum order): " + classes);
    }
}
```

OUTPUT

```
weekend : [SAT, SUN]
weekdays: [MON, TUE, WED, THU, FRI]
midweek : [TUE, WED, THU]
timetable (in enum order): {MON=Arrays, WED=OOP, FRI=Collections}
```

10.14 Records (Java 16+)

10.14.1 Declaring a record

Many classes exist only to **carry data**: a Point, a StudentDTO, a coordinate. Writing fields, constructor, getters, equals, hashCode and toString for each one is long and error-prone. A **record** declares all of that in one line. Records are implicitly **final**, their fields are **private final**, and they are therefore immutable (shallowly).

ONE LINE REPLACES ~50 LINES OF BOILERPLATE

java

```
record Student(int rollNo, String name, double marks) { }
```

10.14.2 Auto-generated constructor, accessors, equals, hashCode, toString

Generated member	For record Student(int rollNo, String name, double marks)
Canonical constructor	`Student(int rollNo, String name, double marks)`
Accessors (no "get")	`rollNo()`, `name()`, `marks()`
`equals()` / `hashCode()`	Based on all components
`toString()`	`Student[rollNo=1, name=Asha, marks=91.5]`

10.14.3 Compact constructors and validation

A **compact constructor** has no parameter list. It runs before the fields are assigned and is used to validate or normalise the arguments. Records may also have extra methods, static fields and static methods, but no extra instance fields.

A RECORD WITH VALIDATION, AN EXTRA METHOD AND A FACTORY

RecordDemo.java

```
import java.util.HashSet;
import java.util.Set;

record Student(int rollNo, String name, double marks) {
    Student {                                              // compact constructor
        if (marks < 0 || marks > 100) throw new IllegalArgumentException("marks out of range: " + marks);
        name = name.trim();                                // normalise before assignment
    }
    char grade() { return marks >= 90 ? 'A' : marks >= 75 ? 'B' : 'C'; }   // extra method
    static Student of(String csv) {                        // static factory
        String[] p = csv.split(",");
        return new Student(Integer.parseInt(p[0]), p[1], Double.parseDouble(p[2]));
    }
}
public class RecordDemo {
    public static void main(String[] args) {
        Student a = new Student(1, "  Asha ", 91.5);
        Student b = Student.of("1,Asha,91.5");
        System.out.println(a);
        System.out.println("name = " + a.name() + ", grade = " + a.grade());
        System.out.println("a.equals(b) = " + a.equals(b));
        Set<Student> set = new HashSet<>(java.util.List.of(a, b));
        System.out.println("set size = " + set.size());
        try {
            new Student(2, "Ravi", 120);
        } catch (IllegalArgumentException e) {
            System.out.println("rejected: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
Student[rollNo=1, name=Asha, marks=91.5]
name = Asha, grade = A
a.equals(b) = true
set size = 1
rejected: marks out of range: 120.0
```

10.15 Sealed classes (Java 17+)

10.15.1 sealed, permits, non-sealed, final

Normally any class can extend a non-final class. A **sealed** class or interface lists **exactly which classes may extend it** using `permits`. This lets you model a closed set of alternatives ("a Shape is a Circle, a Square or a Rectangle, nothing else"), and the compiler can then check that a switch covers every case.

Each permitted subclass must declare one of:

Modifier	Meaning
`final`	Cannot be extended further
`sealed`	Can be extended only by its own permitted list
`non-sealed`	Opens the hierarchy again: anyone may extend it

A SEALED HIERARCHY WITH AN EXHAUSTIVE SWITCH

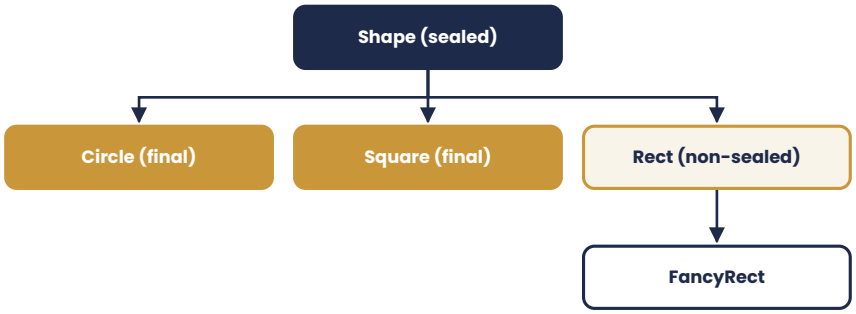
SealedDemo.java

```
sealed interface Shape permits Circle, Square, Rect { }
record Circle(double r) implements Shape { }                // records are implicitly final
record Square(double side) implements Shape { }
non-sealed class Rect implements Shape {                    // open again
    final double w, h;
    Rect(double w, double h) { this.w = w; this.h = h; }
}
class FancyRect extends Rect { FancyRect() { super(2, 3); } }   // allowed because Rect is non-sealed

public class SealedDemo {
    static double area(Shape s) {
        return switch (s) {                                 // exhaustive: no default needed
            case Circle c -> Math.PI * c.r() * c.r();
            case Square q -> q.side() * q.side();
            case Rect r -> r.w * r.h;
        };
    }
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(1), new Square(3), new Rect(2, 5), new FancyRect() };
        for (Shape s : shapes)
            System.out.printf("%-10s %.2f%n", s.getClass().getSimpleName(), area(s));
    }
}
```

OUTPUT

```
Circle     3.14
Square     9.00
Rect       10.00
FancyRect  6.00
```



ONLY THE PERMITTED CLASSES MAY EXTEND SHAPE

PRACTICE CHECKPOINT

A **Bank Account system** with an abstract `Account`, `SavingsAccount` and `CurrentAccount` subclasses, an interface for interest, encapsulated balance, and overridden `toString()`.

SAMPLE SOLUTION

BankSystem.java

```
import java.util.ArrayList;
import java.util.List;

interface InterestBearing {
    double RATE = 0.04;                                     // public static final
    double calculateInterest();
}

abstract class Account {
    private static int counter = 1000;
    private final String accountNo;
    private final String holder;
    private double balance;                                 // encapsulated

    protected Account(String holder, double opening) {
        this.accountNo = "AC" + (++counter);
        this.holder = holder;
```

```
        this.balance = opening;
    }
    public String getAccountNo() { return accountNo; }
    public double getBalance()   { return balance; }

    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Deposit must be positive");
        balance += amt;
    }
    public boolean withdraw(double amt) {
        if (amt <= 0 || !canWithdraw(amt)) {
            System.out.println("  " + accountNo + ": withdrawal of " + amt + " DECLINED");
            return false;
        }
        balance -= amt;
        return true;
    }
    protected abstract boolean canWithdraw(double amt);   // each type has its own rule
    protected abstract String type();

    @Override
    public String toString() {
        return String.format("%-8s %s | %-6s | balance %10.2f", type(), accountNo, holder, balance);
    }
}

class SavingsAccount extends Account implements InterestBearing {
    private static final double MIN_BALANCE = 1000;
    SavingsAccount(String holder, double opening) { super(holder, opening); }
    protected boolean canWithdraw(double amt) { return getBalance() - amt >= MIN_BALANCE; }
    protected String type() { return "Savings"; }
    public double calculateInterest() { return getBalance() * RATE; }
}

class CurrentAccount extends Account {
    private final double overdraft;
    CurrentAccount(String holder, double opening, double overdraft) {
        super(holder, opening);
        this.overdraft = overdraft;
    }
    protected boolean canWithdraw(double amt) { return getBalance() + overdraft >= amt; }
    protected String type() { return "Current"; }
    @Override public String toString() { return super.toString() + " | overdraft " + overdraft; }
}

public class BankSystem {
    public static void main(String[] args) {
        List<Account> accounts = new ArrayList<>();
        accounts.add(new SavingsAccount("Asha", 5000));
        accounts.add(new CurrentAccount("Ravi", 2000, 5000));

        accounts.get(0).deposit(2500);
        accounts.get(0).withdraw(6800);          // would go below minimum balance
        accounts.get(0).withdraw(1500);
        accounts.get(1).withdraw(6000);          // uses overdraft
        accounts.get(1).withdraw(2000);          // beyond overdraft

        System.out.println("--- Accounts ---");
        for (Account a : accounts) {
            System.out.println(a);                                  // overridden toString
            if (a instanceof InterestBearing ib)
                System.out.printf("         yearly interest: %.2f%n", ib.calculateInterest());
        }
    }
}
```

OUTPUT

```
  AC1001: withdrawal of 6800.0 DECLINED
  AC1002: withdrawal of 2000.0 DECLINED
--- Accounts ---
Savings  AC1001 | Asha   | balance    6000.00
         yearly interest: 240.00
Current  AC1002 | Ravi   | balance   -4000.00 | overdraft 5000.0
```

MODULE 11 · INTERMEDIATE

Packages, access modifiers and wrapper classes

Minimum duration **3 days** | Level **Intermediate**

Packages organize code, access modifiers control who can see it, and wrapper classes let primitives behave as objects.

IN THIS MODULE

11.1 Packages
11.2 Access modifiers
11.3 Non-access modifiers summary
11.4 Wrapper classes
11.5 Useful java.lang and java.util classes
11.6 Date and Time API (java.time)

11.1 Packages

11.1.1 What a package is and why (organization, name clashes, access control)

A **package** is a named group of related classes and interfaces, just as a folder groups related files on your computer. Every Java class belongs to a package (if you do not declare one, it goes into the nameless *default package*, which is fine only for tiny experiments).

Organisation

Hundreds of classes are grouped by purpose: `model`, `service`, `util`. Easy to find and maintain.

Name clashes

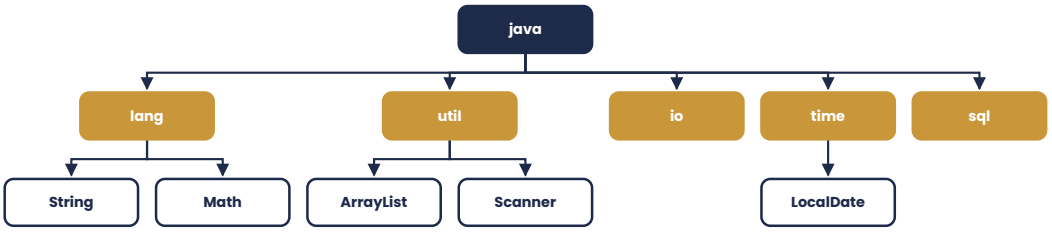
Two classes may share a simple name in different packages: `java.util.Date` and `java.sql.Date`. The **fully qualified name** keeps them apart.

Access control

Package-private (default) and protected members are visible only within the package (and subclasses), so a package can hide its internals.

11.1.2 Built-in packages: java.lang, java.util, java.io, java.time, java.sql

Package	Contains	Import needed?
`java.lang`	String, Math, Object, System, wrapper classes, Thread, exceptions	No: imported automatically
`java.util`	Collections (ArrayList, HashMap…), Scanner, Random, Arrays, Objects	Yes
`java.io`	File, streams, readers and writers, Serializable	Yes
`java.nio.file`	Path, Files: modern file API	Yes
`java.time`	LocalDate, LocalDateTime, Duration, DateTimeFormatter	Yes
`java.sql`	JDBC: Connection, PreparedStatement, ResultSet	Yes
`java.math`	BigInteger, BigDecimal	Yes



PACKAGES ARE HIERARCHICAL, LIKE FOLDERS

11.1.3 Creating user-defined packages

Put a `package` statement as the **very first line** of the source file (only comments may come before it). The file must then live in a folder path that matches the package name.

DECLARING A PACKAGE

java

```java
package com.lbf.college.model;      // first statement in the file

public class Student {
    // ...
}
```

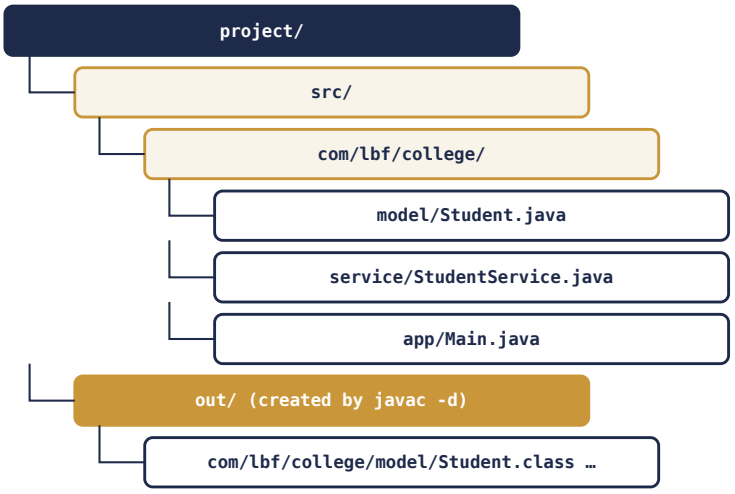
11.1.4 Package naming convention (reverse domain: com.company.project)

Package names are all **lowercase** and start with your organisation's internet domain **reversed**, which makes them globally unique. For learnbeingforward.in: `in.learnbeingforward.college.model`. Common sub-packages are `model` (or `entity`), `service`, `dao` / `repository`, `util`, `app` / `controller`.

Organisation	Domain	Package prefix
Learn Being Forward	learnbeingforward.in	`in.learnbeingforward`
Google	google.com	`com.google`
Apache	apache.org	`org.apache`
Your project (examples here)	lbf.com	`com.lbf.college`

11.1.5 Folder structure and compiling with -d

The package name maps directly to folders: `com.lbf.college.model.Student` lives in `com/lbf/college/model/Student.java`. The compiler option `-d out` tells javac to write the .class files into an output folder, creating the package folders automatically. You then run the class by its **fully qualified name**.



SOURCE FOLDERS MIRROR THE PACKAGE NAMES

Compile into out/ and run by fully qualified name

```
$ javac -d out src/com/lbf/college/model/Student.java src/com/lbf/college/service/StudentService.java
src/com/lbf/college/app/Main.java
$ java -cp out com.lbf.college.app.Main
```

11.1.6 Importing

To use a class from another package you either write its full name every time or **import** it once at the top of the file (after the package line, before the class).

11.1.6.1 Single-type import

`import java.util.ArrayList;` imports exactly one class. Preferred: it documents exactly which classes the file uses.

11.1.6.2 Wildcard import *

`import java.util.*;` makes all public types of that package available. It does **not** import sub-packages (`java.util.*` does not include `java.util.function.Function`) and does not slow the program down. If two wildcards contain the same simple name (`java.util.*` and `java.sql.*` both have Date), using `Date` becomes ambiguous.

11.1.6.3 Fully qualified names

Writing `java.time.LocalDate.of(2026, 6, 1)` needs no import. It is used when two classes with the same name are needed in one file.

11.1.6.4 Static import

`import static java.lang.Math.round;` imports a **static member**, so you can write `round(x)` instead of `Math.round(x)`. `import static java.lang.Math.*;` imports all static members. Use sparingly: too many make it unclear where a method comes from.

The three files of the small project below use all four forms:

SRC/COM/LBF/COLLEGE/MODEL/STUDENT.JAVA Student.java

```java
package com.lbf.college.model;

public class Student {
    private final int rollNo;          // private: only this class
    String name;                       // default: model package only
    protected double marks;            // protected: package + subclasses
    public static final String COLLEGE = "LBF College";   // public: everyone

    public Student(int rollNo, String name, double marks) {
        this.rollNo = rollNo; this.name = name; this.marks = marks;
    }
    public int getRollNo() { return rollNo; }
    public String getName() { return name; }
    public double getMarks() { return marks; }
}
```

SRC/COM/LBF/COLLEGE/SERVICE/STUDENTSERVICE.JAVA StudentService.java

```java
package com.lbf.college.service;

import com.lbf.college.model.Student;          // single-type import
import java.util.*;                            // wildcard import

public class StudentService {
    private final List<Student> students = new ArrayList<>();

    public void add(Student s) { students.add(s); }

    public double average() {
        double sum = 0;
        for (Student s : students) sum += s.getMarks();   // s.marks would NOT compile here
        return students.isEmpty() ? 0 : sum / students.size();
    }
    public List<Student> all() { return Collections.unmodifiableList(students); }
}
```

SRC/COM/LBF/COLLEGE/APP/MAIN.JAVA

Main.java

```
package com.lbf.college.app;

import com.lbf.college.model.Student;
import com.lbf.college.service.StudentService;
import static java.lang.Math.round;            // static import

public class Main {
    public static void main(String[] args) {
        StudentService service = new StudentService();
        service.add(new Student(1, "Asha", 91));
        service.add(new Student(2, "Ravi", 78));
        java.time.LocalDate day = java.time.LocalDate.of(2026, 6, 1);   // fully qualified name
        System.out.println(Student.COLLEGE + " report on " + day);
        for (Student s : service.all())
            System.out.println(s.getRollNo() + " " + s.getName() + " " + s.getMarks());
        System.out.println("average (rounded) = " + round(service.average()));
    }
}
```

OUTPUT

```
LBF College report on 2026-06-01
1 Asha 91.0
2 Ravi 78.0
average (rounded) = 85
```

The compiled package folders

```
$ javac -d out $(find src -name "*.java")
$ find out -type f
out/com/lbf/college/app/Main.class
out/com/lbf/college/model/Student.class
out/com/lbf/college/service/StudentService.class
```

11.1.7 Classpath

The **classpath** is the list of folders and JAR files where the JVM and javac look for `.class` files. To find `com.lbf.college.model.Student`, the JVM looks for `com/lbf/college/model/Student.class` inside each classpath entry.

- Set it per command with `-cp` (or `-classpath`): `java -cp out com.lbf.college.app.Main`.
- Several entries are separated by `:` on macOS/Linux and `;` on Windows: `java -cp "out;lib/mysql-connector-j.jar" ...`.
- If not set, the default classpath is the current directory `.`.
- The `CLASSPATH` environment variable also works but is discouraged. IDEs and build tools (Maven, Gradle: Module 22) manage the classpath for you.
- A missing entry causes `ClassNotFoundException` or `NoClassDefFoundError`; this is exactly the "MySQL driver not found" problem from JDBC.

11.2 Access modifiers

Access modifiers decide **who may use** a class, field, method or constructor. Choosing the most restrictive level that works is the heart of encapsulation.

11.2.1 private: same class only

Visible only inside the class that declares it. Use for fields and for helper methods that are internal details.

11.2.2 default (package-private): same package

When you write **no modifier**, the member is visible to every class in the **same package**, but not outside. Good for classes that cooperate closely inside one package.

11.2.3 protected: same package + subclasses

Visible in the same package **and** in subclasses in other packages (through inheritance). Use for members that subclasses are meant to use or override.

11.2.4 public: everywhere

Visible to every class in every package. Use for the class's intended API: public methods and constants.

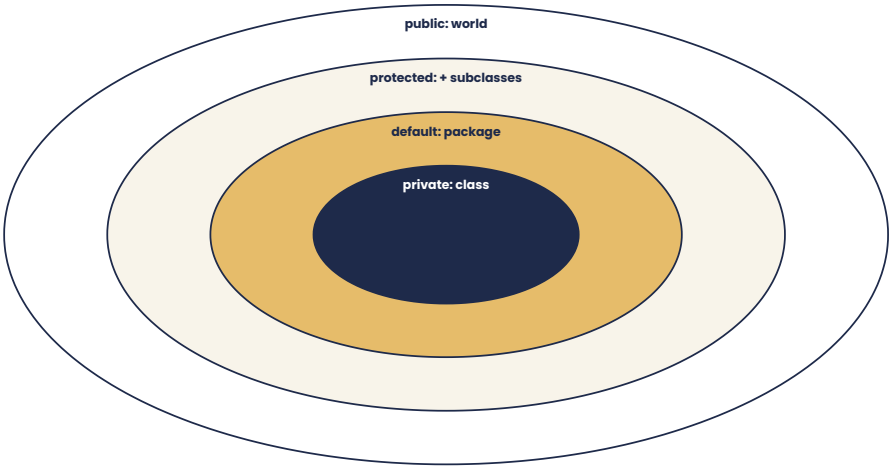
In the project above, `StudentService` (package `service`) is not a subclass of Student, so it cannot touch `marks`:

Compiler output if StudentService used s.marks

```
src/com/lbf/college/service/StudentService.java:13: error: marks has protected access in Student
        for (Student s : students) sum += s.marks;
                                          ^
1 error
```

11.2.5 Access table (class / package / subclass / world)

Modifier	Same class	Same package	Subclass (other package)	World (other package)
`private`	✓	✗	✗	✗
default (none)	✓	✓	✗	✗
`protected`	✓	✓	✓ (via inheritance)	✗
`public`	✓	✓	✓	✓



EACH LEVEL INCLUDES EVERYTHING INSIDE IT

ACCESS LEVELS INSIDE ONE PACKAGE AccessDemo.java

```
class Account {
    private double balance = 500;           // only Account
    String branch = "Bengaluru";            // same package
    protected String type = "Savings";      // package + subclasses
    public String bank = "LBF Bank";        // everyone

    private void audit() { System.out.println("  audit: balance " + balance); }
    public void show() { audit(); }         // private used inside its own class: OK
}
class Savings extends Account {
    void info() {
        System.out.println("  subclass sees: " + type + ", " + branch + ", " + bank);
        // System.out.println(balance);     // error: balance has private access in Account
    }
}
public class AccessDemo {
    public static void main(String[] args) {
        Account a = new Account();
        System.out.println("same package sees: " + a.branch + ", " + a.type + ", " + a.bank);
        a.show();
        new Savings().info();
    }
}
```

OUTPUT

```
same package sees: Bengaluru, Savings, LBF Bank
  audit: balance 500.0
  subclass sees: Savings, Bengaluru, LBF Bank
```

11.2.6 Modifiers allowed on classes vs members

Modifier	Top-level class	Nested class	Field	Method	Constructor
`public`	✓	✓	✓	✓	✓
`protected`	✗	✓	✓	✓	✓
default	✓	✓	✓	✓	✓
`private`	✗	✓	✓	✓	✓
`static`	✗	✓	✓	✓	✗
`final`	✓	✓	✓	✓	✗
`abstract`	✓	✓	✗	✓	✗

NOTE

A source file may contain only **one public top-level class**, and the file must be named after it. Top-level classes can only be public or default.

11.3 Non-access modifiers summary

11.3.1 static, final, abstract

Modifier	On a class	On a method	On a variable
`static`	Nested class not tied to an outer object	Belongs to the class; no `this`	One shared copy per class
`final`	Cannot be extended	Cannot be overridden	Assigned once (constant)
`abstract`	Cannot be instantiated; may have abstract methods	No body; subclass must implement	Not allowed

WATCH OUT
Illegal combinations: `abstract final` (must be overridden vs cannot be), `abstract static` and `abstract private` methods.

11.3.2 synchronized, volatile, transient, native, strictfp

Modifier	Used on	Meaning	Covered in
`synchronized`	Methods, blocks	Only one thread at a time may run it on the same lock	Module 16
`volatile`	Fields	Every read/write goes to main memory: changes are visible to all threads immediately	Module 16
`transient`	Fields	Skipped during serialization (e.g. passwords, caches)	Module 15
`native`	Methods	Implemented in C/C++ through JNI; no Java body	Module 19
`strictfp`	Classes, methods	Forced IEEE floating-point. Since Java 17 all floating point is strict, so it is obsolete	n/a

TRANSIENT FIELDS ARE NOT SERIALIZED

TransientDemo.java

```
import java.io.*;

class User implements Serializable {
    String name;
    transient String password;                    // not saved
    User(String n, String p) { name = n; password = p; }
}
public class TransientDemo {
    public static void main(String[] args) throws Exception {
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(bytes)) {
            out.writeObject(new User("lohit", "secret123"));
        }
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
            User u = (User) in.readObject();
            System.out.println("name = " + u.name + ", password = " + u.password);
        }
    }
}
```

OUTPUT
```
name = lohit, password = null
```

11.4 Wrapper classes

11.4.1 Why wrappers (collections need objects, utility methods, null)

Primitives (int, double, …) are fast but are **not objects**. A **wrapper class** wraps one primitive value inside an object. We need them because:

Collections need objects
`ArrayList<int>` is illegal; generics work only with reference types, so we write `ArrayList<Integer>`.

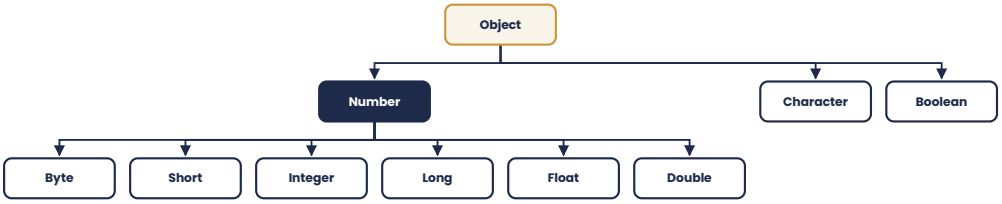
Utility methods
Conversions and constants live in the wrappers: `Integer.parseInt("42")`, `Character.isDigit(c)`, `Integer.MAX_VALUE`.

null
A wrapper can be `null` to mean "no value" (e.g. a missing database column), which a primitive cannot express.

11.4.2 The 8 wrappers: Byte, Short, Integer, Long, Float, Double, Character, Boolean

Primitive	Wrapper	Parse from String	Size
`byte`	`Byte`	`Byte.parseByte("12")`	8 bits

Primitive	Wrapper	Parse from String	Size
`short`	`Short`	`Short.parseShort("12")`	16 bits
`int`	`Integer`	`Integer.parseInt("12")`	32 bits
`long`	`Long`	`Long.parseLong("12")`	64 bits
`float`	`Float`	`Float.parseFloat("1.5")`	32 bits
`double`	`Double`	`Double.parseDouble("1.5")`	64 bits
`char`	`Character`	n/a (`s.charAt(0)`)	16 bits
`boolean`	`Boolean`	`Boolean.parseBoolean("true")`	JVM-dependent



THE SIX NUMERIC WRAPPERS EXTEND NUMBER; ALL WRAPPERS ARE FINAL AND IMMUTABLE

11.4.3 Autoboxing and unboxing

Since Java 5 the compiler converts automatically: **autoboxing** turns a primitive into its wrapper (`Integer x = 5;` becomes `Integer.valueOf(5)`) and **unboxing** turns a wrapper back into a primitive (`int y = x;` becomes `x.intValue()`).



THE COMPILER INSERTS THESE CALLS FOR YOU

AUTOBOXING IN PRACTICE, AND ITS HIDDEN COST IN LOOPS

Boxing.java

```
import java.util.ArrayList;
import java.util.List;

public class Boxing {
    public static void main(String[] args) {
        List<Integer> marks = new ArrayList<>();
        marks.add(85);                    // autoboxing: int -> Integer
        marks.add(92);
        int first = marks.get(0);         // unboxing: Integer -> int
        int total = 0;
        for (Integer m : marks) total += m;     // unboxing in arithmetic
        System.out.println("first = " + first + ", total = " + total);

        Integer a = 10;                   // boxing
        Integer b = a + 5;                // unbox, add, box again
        System.out.println("b = " + b);

        long start = System.nanoTime();
        Long sumBoxed = 0L;
        for (int i = 0; i < 1_000_000; i++) sumBoxed += i;      // creates many Long objects!
        long t1 = System.nanoTime() - start;
        start = System.nanoTime();
        long sum = 0;
        for (int i = 0; i < 1_000_000; i++) sum += i;
        long t2 = System.nanoTime() - start;
        System.out.println("same sum: " + (sum == sumBoxed) + ", primitive loop faster: " + (t2 < t1));
    }
}
```

OUTPUT

```
first = 85, total = 177
b = 15
same sum: true, primitive loop faster: true
```

11.4.4 Useful methods: parseXxx(), valueOf(), xxxValue(), MAX_VALUE, MIN_VALUE

Method / constant	Returns	Example
`Integer.parseInt(s)`	primitive int	`parseInt("123")` → 123
`Integer.parseInt(s, radix)`	int from another base	`parseInt("ff", 16)` → 255
`Integer.valueOf(s or int)`	Integer object (cached)	`valueOf("7")`
`x.intValue()`, `doubleValue()` …	primitive of the chosen type	`d.intValue()`
`Integer.MAX_VALUE / MIN_VALUE`	limits of the type	2147483647 / -2147483648
`Integer.toBinaryString(n)`, `toHexString`	String in base 2 / 16	`toBinaryString(10)` → "1010"
`Integer.compare(a, b)`, `Integer.sum`, `max`	helpers for sorting/streams	`compare(3, 9)` → -1
`Character.isDigit / isLetter / toUpperCase`	char tests	see 8.6

PARSING, CONVERTING AND LIMITS

WrapperMethods.java

```
public class WrapperMethods {
    public static void main(String[] args) {
        int n = Integer.parseInt("2026");
        double d = Double.parseDouble("3.75");
        boolean ok = Boolean.parseBoolean("TRUE");          // case-insensitive
        System.out.println(n + 1 + " | " + (d * 2) + " | " + ok);

        Integer boxed = Integer.valueOf(255);
        System.out.println("binary " + Integer.toBinaryString(boxed) + ", hex " + Integer.toHexString(boxed)
                + ", from base 2: " + Integer.parseInt("1010", 2));
        Double price = 99.99;
        System.out.println("intValue of 99.99 = " + price.intValue());

        System.out.println("int range : " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
        System.out.println("long max  : " + Long.MAX_VALUE);
        System.out.println("double max: " + Double.MAX_VALUE + ", byte range " + Byte.MIN_VALUE + ".." +
Byte.MAX_VALUE);
        System.out.println("overflow  : " + (Integer.MAX_VALUE + 1));
        try {
            Integer.parseInt("12a");
        } catch (NumberFormatException e) {
            System.out.println("bad input -> " + e.getMessage());
        }
    }
}
```

OUTPUT

```
2027 | 7.5 | true
binary 11111111, hex ff, from base 2: 10
intValue of 99.99 = 99
int range : -2147483648 .. 2147483647
long max  : 9223372036854775807
double max: 1.7976931348623157E308, byte range -128..127
overflow  : -2147483648
bad input -> For input string: "12a"
```

11.4.5 Integer cache (-128 to 127) and the == trap

`Integer.valueOf` (and therefore autoboxing) returns **cached** objects for values from **-128 to 127**. Two boxed 100s are the same object, but two boxed 1000s are different objects. Because `==` on objects compares references, the result depends on the value: a classic interview trap. **Always compare wrappers with `equals()`** (or unbox first).

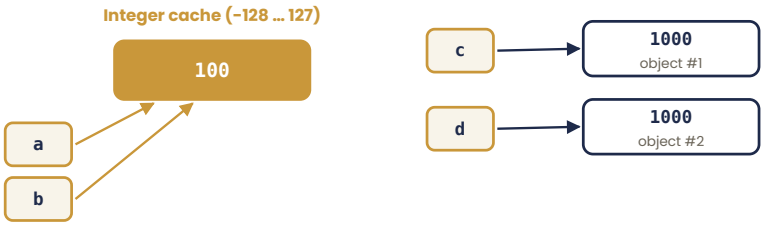
WHY == ON INTEGERS IS UNRELIABLE

IntegerCache.java

```
public class IntegerCache {
    public static void main(String[] args) {
        Integer a = 100, b = 100;
        Integer c = 1000, d = 1000;
        System.out.println("100 == 100     : " + (a == b));        // same cached object
        System.out.println("1000 == 1000   : " + (c == d));        // two different objects
        System.out.println("1000 equals    : " + c.equals(d));     // correct way
        System.out.println("unboxed ==     : " + (c.intValue() == d.intValue()));
        Long big = 1000L;
        System.out.println("c.equals(1000L): " + c.equals(big) + "  (Integer vs Long: different types)");
    }
}
```

OUTPUT

```
100 == 100     : true
1000 == 1000   : false
1000 equals    : true
unboxed ==     : true
c.equals(1000L): false  (Integer vs Long: different types)
```



A AND B SHARE A CACHED OBJECT; C AND D ARE SEPARATE OBJECTS

11.4.6 NullPointerException from unboxing null

A wrapper can be null, but a primitive cannot. Unboxing a null wrapper (assigning it to a primitive, or using it in arithmetic or a condition) throws `NullPointerException`. This often happens with `Map.get()`, which returns null for a missing key.

UNBOXING A NULL UnboxNull.java

```
import java.util.HashMap;
import java.util.Map;

public class UnboxNull {
    public static void main(String[] args) {
        Map<String, Integer> stock = new HashMap<>();
        stock.put("pen", 40);
        Integer pencils = stock.get("pencil");                    // null: key missing
        System.out.println("pencils = " + pencils);
        int safe = stock.getOrDefault("pencil", 0);               // safe alternative
        System.out.println("safe = " + safe);
        int count = stock.get("pencil");                          // unboxing null -> NPE
        System.out.println(count);
    }
}
```

OUTPUT

```
pencils = null
safe = 0
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "java.lang.Integer.intValue()" because the
return value of "java.util.Map.get(Object)" is null
        at UnboxNull.main(UnboxNull.java:12)
```

11.5 Useful java.lang and java.util classes

11.5.1 Math

`java.lang.Math` is a utility class (private constructor, all methods static).

COMMON MATH METHODS MathDemo.java

```
public class MathDemo {
    public static void main(String[] args) {
        System.out.println("abs(-7)        = " + Math.abs(-7));
        System.out.println("max / min      = " + Math.max(4, 9) + " / " + Math.min(4, 9));
        System.out.println("pow(2, 10)     = " + Math.pow(2, 10));
        System.out.println("sqrt / cbrt    = " + Math.sqrt(144) + " / " + Math.cbrt(27));
        System.out.println("ceil / floor   = " + Math.ceil(4.1) + " / " + Math.floor(4.9));
        System.out.println("round(2.5)     = " + Math.round(2.5) + ", round(-2.5) = " + Math.round(-2.5));
        System.out.println("hypot(3, 4)    = " + Math.hypot(3, 4));
        System.out.println("PI, E          = " + Math.PI + ", " + Math.E);
        System.out.printf ("sin(30 deg)    = %.2f%n", Math.sin(Math.toRadians(30)));
        System.out.println("floorMod(-7,3) = " + Math.floorMod(-7, 3) + "  vs  -7 % 3 = " + (-7 % 3));
        try {
            Math.addExact(Integer.MAX_VALUE, 1);
        } catch (ArithmeticException e) {
            System.out.println("addExact       : " + e.getMessage());
        }
    }
}
```

OUTPUT

```
abs(-7)        = 7
max / min      = 9 / 4
pow(2, 10)     = 1024.0
sqrt / cbrt    = 12.0 / 3.0
ceil / floor   = 5.0 / 4.0
round(2.5)     = 3, round(-2.5) = -2
hypot(3, 4)    = 5.0
PI, E          = 3.141592653589793, 2.718281828459045
sin(30 deg)    = 0.50
floorMod(-7,3) = 2  vs  -7 % 3 = -1
addExact       : integer overflow
```

11.5.2 Random and ThreadLocalRandom

`java.util.Random` generates pseudo-random numbers. Given the same **seed**, it produces the same sequence, which is useful for repeatable tests and simulations. `ThreadLocalRandom.current()` is the faster choice in multithreaded code (each thread has its own generator). For security (passwords, OTPs) use `java.security.SecureRandom`. `Math.random()` returns a double in [0.0, 1.0).

SEEDED AND THREAD-LOCAL RANDOM NUMBERS RandomDemo.java

```
import java.util.Random;
import java.util.concurrent.ThreadLocalRandom;

public class RandomDemo {
    public static void main(String[] args) {
        Random r = new Random(42);                      // fixed seed: repeatable
        System.out.print("dice (seed 42): ");
        for (int i = 0; i < 8; i++) System.out.print((r.nextInt(6) + 1) + " ");   // 1..6
        System.out.println();
        Random again = new Random(42);
        System.out.println("same first roll again? " + (again.nextInt(6) + 1));

        int otp = ThreadLocalRandom.current().nextInt(100000, 1000000);   // 6 digits, upper bound exclusive
        System.out.println("OTP has 6 digits: " + (String.valueOf(otp).length() == 6));
        double m = Math.random();
        System.out.println("Math.random in [0,1): " + (m >= 0 && m < 1));
    }
}
```

OUTPUT

```
dice (seed 42): 3 4 1 3 1 2 6 3
same first roll again? 3
OTP has 6 digits: true
Math.random in [0,1): true
```

11.5.3 BigInteger and BigDecimal

`long` overflows after about 9.2×10^{18}, and `double` cannot represent most decimal fractions exactly (0.1 + 0.2 ≠ 0.3). `java.math.BigInteger` handles integers of **any size**; `BigDecimal` stores decimal numbers **exactly** and is the standard choice for **money**. Both are immutable: operations return new objects.

EXACT AND UNLIMITED ARITHMETIC

BigNumbers.java

```
import java.math.BigDecimal;
import java.math.BigInteger;
import java.math.RoundingMode;

public class BigNumbers {
    public static void main(String[] args) {
        BigInteger fact = BigInteger.ONE;
        for (int i = 2; i <= 30; i++) fact = fact.multiply(BigInteger.valueOf(i));
        System.out.println("30! = " + fact);
        System.out.println("2^100 = " + BigInteger.TWO.pow(100));

        System.out.println("double: 0.1 + 0.2 = " + (0.1 + 0.2));
        BigDecimal x = new BigDecimal("0.1"), y = new BigDecimal("0.2");   // use the String constructor!
        System.out.println("BigDecimal: 0.1 + 0.2 = " + x.add(y));

        BigDecimal price = new BigDecimal("1999.99");
        BigDecimal gst = price.multiply(new BigDecimal("0.18")).setScale(2, RoundingMode.HALF_UP);
        System.out.println("GST on 1999.99 = " + gst + ", total = " + price.add(gst));
        System.out.println("10 / 3 = " + BigDecimal.TEN.divide(new BigDecimal("3"), 4, RoundingMode.HALF_UP));
        System.out.println("new BigDecimal(0.1) = " + new BigDecimal(0.1));   // shows double's error
    }
}
```

OUTPUT

```
30! = 265252859812191058636308480000000
2^100 = 1267650600228229401496703205376
double: 0.1 + 0.2 = 0.30000000000000004
BigDecimal: 0.1 + 0.2 = 0.3
GST on 1999.99 = 360.00, total = 2359.99
10 / 3 = 3.3333
new BigDecimal(0.1) = 0.1000000000000000055511151231257827021181583404541015625
```

WATCH OUT

Create BigDecimal from a **String** (`new BigDecimal("0.1")`) or `BigDecimal.valueOf(0.1)`, never `new BigDecimal(0.1)`, which copies the double's binary error. Compare with `compareTo()`, because `equals()` treats 2.0 and 2.00 as different.

11.5.4 Objects utility class (requireNonNull, equals, hash)

NULL-SAFE HELPERS FROM JAVA.UTIL.OBJECTS

ObjectsDemo.java

```
import java.util.Objects;

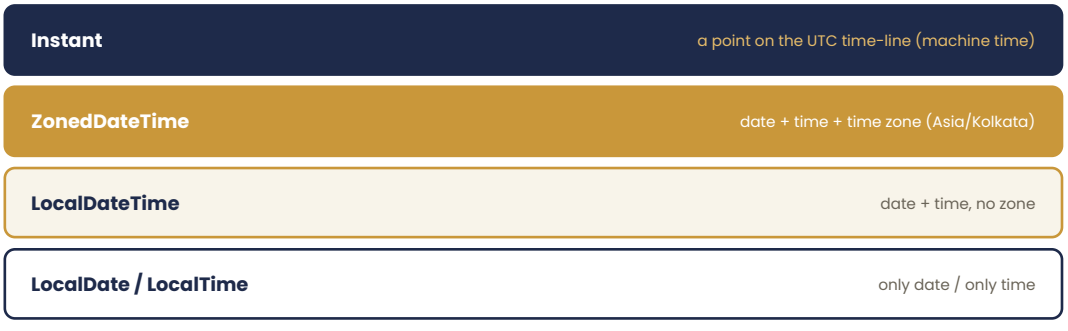
class Customer {
    private final String name;
    private final String email;
    Customer(String name, String email) {
        this.name = Objects.requireNonNull(name, "name must not be null");   // fail fast
        this.email = email;                                                  // may be null
    }
    @Override public boolean equals(Object o) {
        return o instanceof Customer c && name.equals(c.name) && Objects.equals(email, c.email);  // null-safe
    }
    @Override public int hashCode() { return Objects.hash(name, email); }
    @Override public String toString() { return name + " <" + Objects.toString(email, "no email") + ">"; }
}
public class ObjectsDemo {
    public static void main(String[] args) {
        Customer a = new Customer("Asha", null), b = new Customer("Asha", null);
        System.out.println(a + " | equal? " + a.equals(b) + " | same hash? " + (a.hashCode() == b.hashCode()));
        System.out.println("isNull: " + Objects.isNull(null) + ", requireNonNullElse: " +
Objects.requireNonNullElse(null, "default"));
        try {
            new Customer(null, "x@y.com");
        } catch (NullPointerException e) {
            System.out.println("NPE: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
Asha <no email> | equal? true | same hash? true
isNull: true, requireNonNullElse: default
NPE: name must not be null
```

11.6 Date and Time API (java.time)

Java 8 introduced `java.time` (based on the Joda-Time library). Its classes are **immutable**, thread-safe and clearly named. Month numbers start at **1** (January = 1), unlike the legacy API.



FROM MACHINE TIME DOWN TO SIMPLE HUMAN DATES

11.6.1 LocalDate, LocalTime, LocalDateTime

WORKING WITH DATES AND TIMES

LocalDemo.java

```
import java.time.*;

public class LocalDemo {
    public static void main(String[] args) {
        LocalDate join = LocalDate.of(2026, 1, 15);                // fixed date (LocalDate.now() for today)
        LocalTime classTime = LocalTime.of(10, 30);
        LocalDateTime exam = LocalDateTime.of(2026, Month.MARCH, 20, 9, 0);

        System.out.println("join date       : " + join);
        System.out.println("day of week     : " + join.getDayOfWeek());
        System.out.println("+90 days        : " + join.plusDays(90));
        System.out.println("-1 month        : " + join.minusMonths(1));
        System.out.println("leap year?      : " + join.isLeapYear() + ", days in month: " + join.lengthOfMonth());
        System.out.println("class ends      : " + classTime.plusHours(2));
        System.out.println("exam            : " + exam + " (" + exam.getMonth() + ")");
        System.out.println("exam after join : " + exam.toLocalDate().isAfter(join));
        LocalDate fixed = join.withDayOfMonth(1);                  // immutable: returns a new object
        System.out.println("first of month  : " + fixed + ", original unchanged: " + join);
    }
}
```

OUTPUT

```
join date       : 2026-01-15
day of week     : THURSDAY
+90 days        : 2026-04-15
-1 month        : 2025-12-15
leap year?      : false, days in month: 31
class ends      : 12:30
exam            : 2026-03-20T09:00 (MARCH)
exam after join : true
first of month  : 2026-01-01, original unchanged: 2026-01-15
```

11.6.2 ZonedDateTime and Instant

`ZonedDateTime` adds a time zone (`ZoneId.of("Asia/Kolkata")`), handling offsets and daylight-saving rules. `Instant` is a timestamp: nanoseconds since 1970-01-01T00:00Z (the Unix epoch), ideal for logging and storing in databases.

ONE MOMENT, THREE TIME ZONES

ZonedDemo.java

```
import java.time.*;

public class ZonedDemo {
    public static void main(String[] args) {
        ZonedDateTime webinarIst = ZonedDateTime.of(2026, 7, 10, 19, 0, 0, 0, ZoneId.of("Asia/Kolkata"));
        ZonedDateTime inNewYork  = webinarIst.withZoneSameInstant(ZoneId.of("America/New_York"));
        ZonedDateTime inLondon   = webinarIst.withZoneSameInstant(ZoneId.of("Europe/London"));
        System.out.println("India    : " + webinarIst);
        System.out.println("New York : " + inNewYork);
        System.out.println("London   : " + inLondon);

        Instant instant = webinarIst.toInstant();
        System.out.println("Instant (UTC)   : " + instant);
        System.out.println("epoch seconds   : " + instant.getEpochSecond());
        System.out.println("same instant?   : " + instant.equals(inNewYork.toInstant()));
    }
}
```

OUTPUT

```
India    : 2026-07-10T19:00+05:30[Asia/Kolkata]
New York : 2026-07-10T09:30-04:00[America/New_York]
London   : 2026-07-10T14:30+01:00[Europe/London]
Instant (UTC)   : 2026-07-10T13:30:00Z
epoch seconds   : 1783690200
same instant?   : true
```

11.6.3 Period and Duration

`Period` measures a date-based amount (years, months, days): *age, course length*. `Duration` measures a time-based amount (hours, minutes, seconds, nanos): *exam duration, elapsed time*. `ChronoUnit.DAYS.between(a, b)` gives a single number.

MEASURING AMOUNTS OF TIME

PeriodDuration.java

```
import java.time.*;
import java.time.temporal.ChronoUnit;

public class PeriodDuration {
    public static void main(String[] args) {
        LocalDate birth = LocalDate.of(2001, 8, 25);
        LocalDate today = LocalDate.of(2026, 6, 1);
        Period age = Period.between(birth, today);
        System.out.println("age: " + age.getYears() + " years " + age.getMonths() + " months " + age.getDays() + "
days");
        System.out.println("total days lived: " + ChronoUnit.DAYS.between(birth, today));

        Period course = Period.ofMonths(3).plusDays(10);
        System.out.println("course " + course + " ends on " + today.plus(course));

        LocalTime start = LocalTime.of(9, 15), end = LocalTime.of(12, 45);
        Duration exam = Duration.between(start, end);
        System.out.println("exam duration: " + exam + " = " + exam.toMinutes() + " minutes");
        System.out.println("plus 30 min break: " + exam.plusMinutes(30).toHours() + "h " +
exam.plusMinutes(30).toMinutesPart() + "m");
    }
}
```

OUTPUT

```
age: 24 years 9 months 7 days
total days lived: 9046
course P3M10D ends on 2026-09-11
exam duration: PT3H30M = 210 minutes
plus 30 min break: 4h 0m
```

11.6.4 DateTimeFormatter (formatting and parsing)

Pattern letter	Meaning	Example
`dd` / `d`	day of month	05 / 5
`MM` / `MMM` / `MMMM`	month	06 / Jun / June
`yyyy` / `yy`	year	2026 / 26
`EEE` / `EEEE`	day name	Mon / Monday
`HH` / `hh`	hour 0-23 / 1-12	19 / 07
`mm`, `ss`	minutes, seconds	30, 05
`a`	AM/PM	PM

FORMATTING AND PARSING DATES

FormatDemo.java

```java
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.Locale;

public class FormatDemo {
    public static void main(String[] args) {
        LocalDateTime t = LocalDateTime.of(2026, 6, 5, 19, 30);
        DateTimeFormatter indian = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        DateTimeFormatter pretty = DateTimeFormatter.ofPattern("EEEE, d MMMM yyyy 'at' hh:mm a", Locale.ENGLISH);
        System.out.println("ISO default : " + t);
        System.out.println("dd/MM/yyyy  : " + t.format(indian));
        System.out.println("pretty      : " + t.format(pretty));
        System.out.println("ISO_DATE    : " + t.format(DateTimeFormatter.ISO_DATE));

        LocalDate parsed = LocalDate.parse("15/08/2026", indian);      // String -> date
        System.out.println("parsed      : " + parsed + " (" + parsed.getDayOfWeek() + ")");
        try {
            LocalDate.parse("31/02/2026", indian);
        } catch (DateTimeParseException e) {
            System.out.println("invalid     : " + e.getMessage());
        }
    }
}
```

OUTPUT

```
ISO default : 2026-06-05T19:30
dd/MM/yyyy  : 05/06/2026
pretty      : Friday, 5 June 2026 at 07:30 PM
ISO_DATE    : 2026-06-05
parsed      : 2026-08-15 (SATURDAY)
```

11.6.5 Legacy Date and Calendar, and why they were replaced

Before Java 8 there were `java.util.Date`, `java.util.Calendar` and `java.text.SimpleDateFormat`. You will still see them in old code and in some JDBC APIs. Their problems:

Problem	Legacy API	java.time
Mutability	`Date` has setters; shared dates can be changed by anyone	All classes immutable
Thread safety	`SimpleDateFormat` is not thread-safe	`DateTimeFormatter` is thread-safe
Confusing values	Months 0–11; `Date` year = year - 1900	Months 1–12, real years
Poor design	`Date` is really an instant, yet prints in local zone	Separate types for each concept
Zones	Awkward	`ZoneId`, `ZonedDateTime`

THE 0-BASED MONTH TRAP AND CONVERTING TO JAVA.TIME

LegacyDates.java

```
import java.util.Calendar;
import java.util.Date;
import java.time.*;

public class LegacyDates {
    public static void main(String[] args) {
        Calendar cal = Calendar.getInstance();
        cal.clear();
        cal.set(2026, 5, 1);                        // 5 means JUNE: months start at 0!
        System.out.println("Calendar month field for June = " + cal.get(Calendar.MONTH));
        Date d = cal.getTime();
        // converting legacy -> modern
        LocalDate ld = d.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();
        System.out.println("converted to LocalDate: " + ld);
        // and back
        Date back = Date.from(ld.atStartOfDay(ZoneId.systemDefault()).toInstant());
        System.out.println("round trip equal: " + back.equals(d));
    }
}
```

OUTPUT

```
Calendar month field for June = 5
converted to LocalDate: 2026-06-01
round trip equal: true
```

PRACTICE CHECKPOINT

Split a small project into **model**, **service** and **app** packages and demonstrate each access modifier.

Sample solution: the three-file project in 11.1.6 is exactly this structure. `Student` (model) uses all four modifiers: private `rollNo`, default `name`, protected `marks` and public `COLLEGE`; `StudentService` (service) can reach only the public members (the protected access error is shown in 11.2.4); `Main` (app) wires them together. As an extension, add `com.lbf.college.app.Topper extends Student` in a **different** package and show that it *can* read `marks` (protected, via inheritance) but not `name` (default).

MODULE 12 · INTERMEDIATE

Exception handling

Minimum duration **3 days** | Level **Intermediate**

An exception is an event that disrupts normal flow; handling it lets the program recover or fail cleanly instead of crashing.

IN THIS MODULE

12.1 Introduction
12.2 Exception hierarchy
12.3 Handling exceptions
12.4 Throwing exceptions
12.5 Custom exceptions
12.6 Exception methods
12.7 Best practices
12.8 Assertions

12.1 Introduction

12.1.1 What an exception is

An **exception** is an unexpected event during execution that interrupts the normal flow of statements: dividing by zero, reading a file that does not exist, converting "abc" to a number, calling a method on `null`. When this happens, the JVM creates an **exception object** describing the problem (its type, a message and the stack trace) and **throws** it. Execution then jumps to the nearest code that can **catch** it.



WHAT HAPPENS WHEN SOMETHING GOES WRONG

Exception handling separates the **normal logic** from the **error-handling logic**, and lets the program recover (ask the user again, use a default, retry) or fail cleanly with a clear message instead of crashing halfway through.

12.1.2 Error vs exception

	Error	Exception
Meaning	Serious problem in the JVM or environment	Problem in the program or its inputs
Examples	`OutOfMemoryError`, `StackOverflowError`	`IOException`, `NullPointerException`
Recoverable?	Usually not	Usually yes
Should you catch it?	No (let the program end, fix the cause)	Yes, where you can do something useful
Package	`java.lang.Error` subclasses	`java.lang.Exception` subclasses

12.1.3 What happens when an exception is not handled (stack trace)

If no method catches the exception, it travels up the call stack, frame by frame, all the way out of `main`. The JVM's **default handler** then prints the exception's type, message and **stack trace** to stderr and terminates the thread. Statements after the failing line never run.

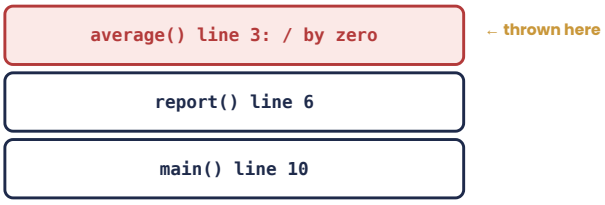
AN EXCEPTION WITH NO HANDLER

Unhandled.java

```
public class Unhandled {
    static int average(int total, int count) {
        return total / count;                    // line 3: fails when count = 0
    }
    static void report(int total, int count) {
        System.out.println("Average = " + average(total, count));   // line 6
    }
    public static void main(String[] args) {
        report(300, 3);
        report(300, 0);                          // line 10
        System.out.println("This line never runs");
    }
}
```

OUTPUT

```
Average = 100
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Unhandled.average(Unhandled.java:3)
        at Unhandled.report(Unhandled.java:6)
        at Unhandled.main(Unhandled.java:10)
```



READ THE STACK TRACE FROM THE TOP: THE FIRST "AT" LINE IS WHERE IT HAPPENED; LOWER LINES SHOW WHO CALLED IT

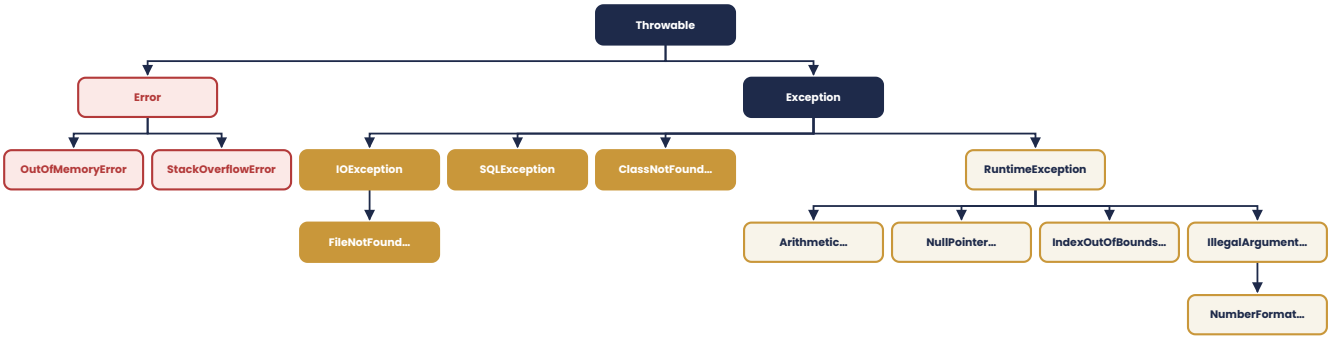
Read a stack trace like this: **first line** = exception type and message; **first "at" line** = the exact file and line where it was thrown; following lines = the chain of callers. Look for the first line that belongs to *your* code.

12.2 Exception hierarchy

12.2.1 Throwable → Error and Exception

12.2.2 Exception → RuntimeException and others

All exceptions are objects of classes that extend `java.lang.Throwable`. Only Throwables can be thrown and caught. Throwable has two branches: `Error` and `Exception`. Exception in turn has a special child, `RuntimeException`, which is the root of all **unchecked** exceptions.



THE EXCEPTION HIERARCHY: RED = ERRORS, GOLD = CHECKED EXCEPTIONS, CREAM = UNCHECKED (RUNTIMEEXCEPTION)

12.2.3 Checked exceptions (IOException, SQLException, ClassNotFoundException)

Checked exceptions are all subclasses of Exception that are **not** RuntimeException. They represent problems outside the program's control that a well-written program should expect: a missing file, a lost database connection. The compiler

forces you to deal with them: either **catch** them or **declare** them with `throws`. Otherwise: *"unreported exception … must be caught or declared to be thrown"*.

Checked exception	Typical cause
`IOException` / `FileNotFoundException`	File missing, disk error, network stream closed
`SQLException`	Bad SQL, wrong password, database down (Module 20)
`ClassNotFoundException`	Class.forName() cannot find a class (e.g. JDBC driver jar missing)
`InterruptedException`	A sleeping/waiting thread was interrupted (Module 16)

THE COMPILER INSISTS ON CHECKED EXCEPTIONS

java

```java
import java.io.FileReader;
public class NeedsHandling {
    public static void main(String[] args) {
        FileReader fr = new FileReader("data.txt");
        // error: unreported exception FileNotFoundException; must be caught or declared to be thrown
    }
}
```

12.2.4 Unchecked exceptions (ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, ClassCastException, IllegalArgumentException)

Unchecked exceptions extend `RuntimeException`. They usually indicate **programming bugs** and the compiler does not force you to handle them. The right fix is usually to correct the code (check for null, validate the index), not to wrap everything in try-catch.

THE SIX MOST COMMON UNCHECKED EXCEPTIONS

UncheckedTour.java

```java
public class UncheckedTour {
    static void attempt(String label, Runnable r) {
        try {
            r.run();
        } catch (RuntimeException e) {
            System.out.printf("%-16s -> %s: %s%n", label, e.getClass().getSimpleName(), e.getMessage());
        }
    }
    public static void main(String[] args) {
        attempt("10 / 0", () -> System.out.println(10 / 0));
        attempt("null.length()", () -> { String s = null; s.length(); });
        attempt("arr[5]", () -> { int[] a = new int[3]; a[5] = 1; });
        attempt("parseInt(\"12a\")", () -> Integer.parseInt("12a"));
        attempt("(String) 42", () -> { Object o = 42; String s = (String) o; });
        attempt("negative age", () -> { throw new IllegalArgumentException("age must be >= 0"); });
        attempt("\"abc\".charAt(9)", () -> "abc".charAt(9));
    }
}
```

OUTPUT

```
10 / 0           -> ArithmeticException: / by zero
null.length()    -> NullPointerException: Cannot invoke "String.length()" because "s" is null
arr[5]           -> ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
parseInt("12a")  -> NumberFormatException: For input string: "12a"
(String) 42      -> ClassCastException: class java.lang.Integer cannot be cast to class java.lang.String
(java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
negative age     -> IllegalArgumentException: age must be >= 0
"abc".charAt(9)  -> StringIndexOutOfBoundsException: Index 9 out of bounds for length 3
```

Exception	Cause	Prevent it by
`ArithmeticException`	Integer division by zero	Checking the divisor
`NullPointerException`	Using a member of a null reference	Null checks, `Objects.requireNonNull`, Optional

Exception	Cause	Prevent it by
`ArrayIndexOutOfBoundsException`	Index < 0 or ≥ length	Loop with `i < arr.length`
`NumberFormatException`	Parsing a non-numeric string	Validating input first
`ClassCastException`	Invalid downcast	`instanceof` check
`IllegalArgumentException`	Thrown by your code for a bad argument	Validating arguments (fail fast)

12.2.5 Errors (OutOfMemoryError, StackOverflowError)

`OutOfMemoryError`: the heap is full and the garbage collector cannot free enough space (e.g. adding to a list forever, or loading a huge file into memory). `StackOverflowError`: the call stack is full, almost always due to runaway recursion (9.7.4). Do not catch errors to "fix" them; fix the cause (bounded data, a base case) or give the JVM more memory (`-Xmx`, Module 19).

FILLING THE HEAP (RUN WITH A SMALL HEAP) OomDemo.java

```
import java.util.ArrayList;
import java.util.List;

public class OomDemo {
    public static void main(String[] args) {
        List<long[]> hog = new ArrayList<>();
        try {
            while (true) hog.add(new long[10_000_000]);     // 80 MB each
        } catch (OutOfMemoryError e) {
            int blocks = hog.size();
            hog = null;                                     // release memory before printing
            System.out.println("OutOfMemoryError after " + (blocks > 0 ? "some" : "no") + " blocks: " +
e.getMessage());
        }
    }
}
```

```
OUTPUT
OutOfMemoryError after some blocks: Java heap space
```

12.3 Handling exceptions

12.3.1 try-catch

Put the code that might fail inside `try`. If an exception of the matching type is thrown, the rest of the try block is **skipped** and control jumps into the `catch` block. After the catch, execution continues normally after the whole try-catch. If nothing is thrown, the catch block is skipped.

TRY-CATCH KEEPS THE PROGRAM ALIVE

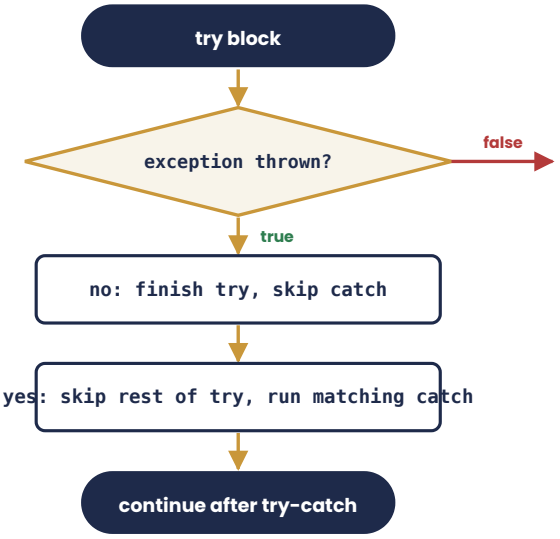
TryCatch.java

```
import java.util.Scanner;

public class TryCatch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter total marks: ");
        int total = sc.nextInt();
        System.out.print("Enter number of subjects: ");
        int n = sc.nextInt();
        try {
            int avg = total / n;                        // may throw
            System.out.println("Average = " + avg);      // skipped if it throws
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero subjects! (" + e.getMessage() + ")");
        }
        System.out.println("Program continues normally.");
    }
}
```

CONSOLE (user input in gold)

```
Enter total marks: 450
Enter number of subjects: 0
Cannot divide by zero subjects! (/ by zero)
Program continues normally.
```



FLOW THROUGH TRY-CATCH

12.3.2 Multiple catch blocks and their order

A try can be followed by several catch blocks. The JVM checks them **top to bottom** and runs only the **first** one whose type matches (the exception is that class or a subclass). Therefore catch blocks must go from **most specific to most general**; putting `Exception` first makes the later ones unreachable, which is a compile error.

SPECIFIC CATCHES FIRST, GENERAL LAST MultipleCatch.java

```
public class MultipleCatch {
    static void process(String input, int index) {
        try {
            int[] data = {10, 20, 30};
            int divisor = Integer.parseInt(input);       // NumberFormatException
            System.out.println("result = " + data[index] / divisor);   // AIOOBE or ArithmeticException
        } catch (NumberFormatException e) {
            System.out.println("  not a number: " + input);
        } catch (ArithmeticException e) {
            System.out.println("  division by zero");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("  bad index " + index);
        } catch (Exception e) {                          // most general LAST
            System.out.println("  something else: " + e);
        }
    }
    public static void main(String[] args) {
        process("5", 1);
        process("five", 1);
        process("0", 1);
        process("5", 7);
    }
}
```

OUTPUT

```
result = 4
  not a number: five
  division by zero
  bad index 7
```

WRONG ORDER DOES NOT COMPILE java

```
try { ... }
catch (Exception e) { }
catch (ArithmeticException e) { }   // error: exception ArithmeticException has already been caught
```

12.3.3 Multi-catch with | (Java 7)

When several exception types need the **same** handling, list them in one catch with `|`. The types must not be subclasses of each other, and the variable `e` is implicitly final.

ONE HANDLER FOR TWO EXCEPTION TYPES MultiCatch.java

```
public class MultiCatch {
    public static void main(String[] args) {
        String[] inputs = {"42", "abc", null};
        for (String in : inputs) {
            try {
                System.out.println("parsed: " + Integer.parseInt(in.trim()));
            } catch (NumberFormatException | NullPointerException e) {
                System.out.println("invalid input (" + e.getClass().getSimpleName() + ")");
            }
        }
    }
}
```

OUTPUT

```
parsed: 42
invalid input (NumberFormatException)
invalid input (NullPointerException)
```

12.3.4 Nested try blocks

A try-catch can appear inside another try. An exception is first offered to the inner catches; if none matches, it propagates to the outer catches. Useful when one step can fail independently of the larger operation.

INNER CATCHES HANDLE ROW ERRORS; THE OUTER CATCH HANDLES FATAL ONES

NestedTry.java

```
public class NestedTry {
    public static void main(String[] args) {
        String[] rows = {"10,2", "8,0", "x,1", "7"};
        int processed = 0;
        try {
            for (String row : rows) {
                String[] parts = row.split(",");
                try {                                          // inner: per-row problems
                    int a = Integer.parseInt(parts[0]);
                    int b = Integer.parseInt(parts[1]);
                    System.out.println(row + " -> " + (a / b));
                } catch (NumberFormatException | ArithmeticException e) {
                    System.out.println(row + " -> skipped (" + e.getMessage() + ")");
                }
                processed++;
            }
        } catch (ArrayIndexOutOfBoundsException e) {           // outer: not handled inside
            System.out.println("malformed row stops the import: " + e.getMessage());
        }
        System.out.println("rows processed: " + processed);
    }
}
```

OUTPUT

```
10,2 -> 5
8,0 -> skipped (/ by zero)
x,1 -> skipped (For input string: "x")
malformed row stops the import: Index 1 out of bounds for length 1
rows processed: 3
```

12.3.5 finally block

A `finally` block runs **after** the try (and catch, if any) **whether or not an exception occurred**, and even if the try or catch executes `return`, `break` or `continue`. It is the place for cleanup: closing files, connections, releasing locks.

FINALLY RUNS IN BOTH CASES

FinallyDemo.java

```
public class FinallyDemo {
    static void run(int divisor) {
        System.out.println("divisor " + divisor + ":");
        try {
            System.out.println("  try: open resource, result = " + (100 / divisor));
        } catch (ArithmeticException e) {
            System.out.println("  catch: " + e.getMessage());
        } finally {
            System.out.println("  finally: close resource");      // always
        }
    }
    public static void main(String[] args) {
        run(5);
        run(0);
    }
}
```

OUTPUT

```
divisor 5:
  try: open resource, result = 20
  finally: close resource
divisor 0:
  catch: / by zero
  finally: close resource
```

12.3.5.1 When finally runs and when it does not (System.exit)

Situation	Does finally run?
try completes normally	Yes

Situation	Does finally run?
Exception thrown and caught	Yes, after the catch
Exception thrown and NOT caught	Yes, then the exception continues upward
`return` / `break` / `continue` in try or catch	Yes, before control leaves
`System.exit()` called in try/catch	**No**: the JVM stops immediately
JVM crash, power loss, thread killed	No

SYSTEM.EXIT SKIPS FINALLY FinallyExit.java

```
public class FinallyExit {
    public static void main(String[] args) {
        try {
            System.out.println("in try, calling System.exit(0)");
            System.exit(0);
        } finally {
            System.out.println("finally (never printed)");
        }
    }
}
```

OUTPUT
```
in try, calling System.exit(0)
```

12.3.5.2 return inside try and finally

If try returns a value, the value is computed and saved, then finally runs, then the method returns the saved value. Changing a variable in finally does not change an already-computed primitive result. But if **finally itself returns**, its value replaces the try's value and even **swallows any exception**, so never return from finally.

RETURN VALUES AND FINALLY ReturnFinally.java

```
public class ReturnFinally {
    static int a() {
        int x = 10;
        try {
            return x;                    // value 10 saved
        } finally {
            x = 99;                      // too late: does not change the result
            System.out.println("  a(): finally ran, x = " + x);
        }
    }
    @SuppressWarnings("finally")
    static int b() {
        try {
            throw new RuntimeException("lost!");
        } finally {
            return -1;                   // BAD: replaces the exception
        }
    }
    public static void main(String[] args) {
        System.out.println("a() returned " + a());
        System.out.println("b() returned " + b() + " (the exception disappeared)");
    }
}
```

OUTPUT
```
  a(): finally ran, x = 99
a() returned 10
b() returned -1 (the exception disappeared)
```

12.3.6 try-with-resources (Java 7) and AutoCloseable

Resources such as files, scanners, sockets and database connections must be **closed**. Doing it in finally is long and error-prone (close() itself can throw). With **try-with-resources** you declare the resources in parentheses after `try`; Java

closes them **automatically**, in **reverse order** of opening, when the block ends, normally or by exception. Any class implementing `java.lang.AutoCloseable` (single method `close()`) can be used.

AUTOMATIC CLOSING IN REVERSE ORDER

TryWithResources.java

```
class Resource implements AutoCloseable {
    private final String name;
    Resource(String name) { this.name = name; System.out.println("open  " + name); }
    void use(boolean fail) {
        System.out.println("use   " + name);
        if (fail) throw new IllegalStateException(name + " failed");
    }
    @Override public void close() { System.out.println("close " + name); }
}
public class TryWithResources {
    public static void main(String[] args) {
        try (Resource db = new Resource("DB"); Resource file = new Resource("File")) {
            db.use(false);
            file.use(true);
            System.out.println("not reached");
        } catch (IllegalStateException e) {
            System.out.println("caught: " + e.getMessage() + " (resources already closed)");
        }
    }
}
```

OUTPUT

```
open  DB
open  File
use   DB
use   File
close File
close DB
caught: File failed (resources already closed)
```

BEFORE AND AFTER

java

```
// old style (before Java 7)
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("a.txt"));
    System.out.println(br.readLine());
} finally {
    if (br != null) br.close();         // close() can throw too ...
}

// modern style
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {
    System.out.println(br.readLine());
}                                       // closed automatically
```

12.3.7 Suppressed exceptions

What if the body throws an exception **and** close() also throws? With try-with-resources, the body's exception is the one that propagates (it is the real problem); the close() exception is attached to it as a **suppressed** exception, available through `getSuppressed()`. (With the old finally style, the close() exception would replace and hide the original one.)

BOTH EXCEPTIONS ARE KEPT

Suppressed.java

```
class FaultyResource implements AutoCloseable {
    @Override public void close() throws Exception {
        throw new Exception("close() failed");
    }
}
public class Suppressed {
    public static void main(String[] args) {
        try (FaultyResource r = new FaultyResource()) {
            throw new IllegalStateException("body failed");
        } catch (Exception e) {
            System.out.println("main exception : " + e.getMessage());
            for (Throwable s : e.getSuppressed())
                System.out.println("suppressed     : " + s.getMessage());
        }
    }
}
```

OUTPUT

```
main exception : body failed
suppressed     : close() failed
```

12.4 Throwing exceptions

12.4.1 throw keyword

`throw` lets your own code raise an exception: `throw new IllegalArgumentException("message");`. It is used to reject invalid input or impossible states. `throw` immediately ends the current block, like return.

VALIDATING WITH THROW

ThrowDemo.java

```
public class ThrowDemo {
    static void setAge(int age) {
        if (age < 0 || age > 150)
            throw new IllegalArgumentException("Invalid age: " + age);
        System.out.println("age set to " + age);
    }
    public static void main(String[] args) {
        setAge(25);
        try {
            setAge(-4);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
age set to 25
Error: Invalid age: -4
```

12.4.2 throws keyword in method signatures

`throws` in a method header **declares** that the method may throw certain (usually checked) exceptions and leaves the handling to the **caller**. The caller must then catch them or declare them in turn.

THE CALLER DECIDES HOW TO HANDLE

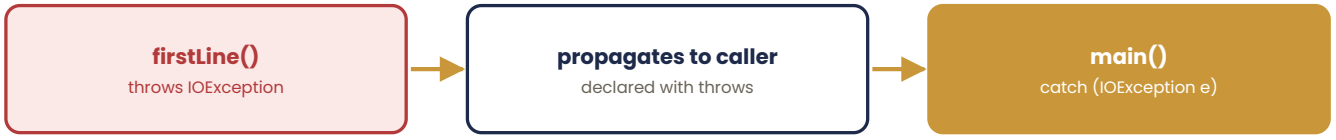
ThrowsDemo.java

```
import java.io.*;

public class ThrowsDemo {
    static String firstLine(String path) throws IOException {        // declares, does not handle
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            return br.readLine();
        }
    }
    public static void main(String[] args) {
        try {
            System.out.println(firstLine("missing-file.txt"));
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
File not found: missing-file.txt (No such file or directory)
```



THROWS PASSES RESPONSIBILITY UP THE CALL STACK

12.4.3 throw vs throws

	`throw`	`throws`
What	A statement that throws one exception object	A clause in a method header
Where	Inside a method body	After the parameter list
Followed by	An instance: `throw new X("…")`	Class names: `throws X, Y`
Count	One exception at a time	Can list several
Purpose	Raise an exception now	Warn callers what may be thrown

12.4.4 Re-throwing and exception chaining (cause)

Often a low-level exception (SQLException) should be turned into a meaningful higher-level one (StudentNotFoundException) as it moves up through the layers. **Chaining** keeps the original as the **cause**: `throw new ServiceException("Could not load student", e);`. Never lose the cause; it holds the real root of the problem.

WRAPPING, RE-THROWING AND THE CAUSE CHAIN Chaining.java

```
class DataAccessException extends RuntimeException {
    DataAccessException(String msg, Throwable cause) { super(msg, cause); }
}
public class Chaining {
    static int readPort(String text) {
        try {
            return Integer.parseInt(text);
        } catch (NumberFormatException e) {
            throw new DataAccessException("Invalid port in config: '" + text + "'", e);   // wrap + keep cause
        }
    }
    static void logAndRethrow() {
        try {
            readPort("80a");
        } catch (DataAccessException e) {
            System.out.println("log: " + e.getMessage());
            throw e;                                        // re-throw the same exception
        }
    }
    public static void main(String[] args) {
        try {
            logAndRethrow();
        } catch (DataAccessException e) {
            System.out.println("top-level: " + e.getMessage());
            System.out.println("root cause: " + e.getCause());
        }
    }
}
```

```
OUTPUT
log: Invalid port in config: '80a'
top-level: Invalid port in config: '80a'
root cause: java.lang.NumberFormatException: For input string: "80a"
```

12.4.5 Exceptions and method overriding rules

An overriding method may throw **fewer** or **narrower** checked exceptions than the parent method, never new or broader checked ones (see 10.8.3.4). Unchecked exceptions are not restricted.

Parent method declares	Overriding method may declare
nothing	no checked exceptions (any unchecked)
`throws IOException`	IOException, FileNotFoundException, nothing, any unchecked
`throws Exception`	any exception or none

12.5 Custom exceptions

Built-in exceptions describe technical problems. For **business rules** (insufficient balance, seat already booked, invalid roll number) create your own exception classes. Their names make the code self-documenting, and callers can catch them specifically.

12.5.1 Creating a checked custom exception

Extend `Exception`. Callers are forced to handle it, which is suitable when the caller can reasonably recover (e.g. ask for a smaller amount).

12.5.2 Creating an unchecked custom exception

Extend `RuntimeException`. Suitable for programming errors or when callers usually cannot recover. Modern frameworks (Spring) mostly use unchecked exceptions.

12.5.3 Adding messages and fields

Provide constructors that pass a message (and optionally a cause) to `super`, and add fields that carry useful details for the handler.

A CHECKED AND AN UNCHECKED CUSTOM EXCEPTION CustomExceptions.java

```
class InsufficientFundsException extends Exception {                // CHECKED
    private final double shortBy;
    InsufficientFundsException(String msg, double shortBy) {
        super(msg);
        this.shortBy = shortBy;
    }
    double getShortBy() { return shortBy; }
}
class InvalidAmountException extends RuntimeException {               // UNCHECKED
    InvalidAmountException(double amount) {
        super("Amount must be positive, got " + amount);
    }
}
class Wallet {
    private double balance = 1000;
    void pay(double amount) throws InsufficientFundsException {
        if (amount <= 0) throw new InvalidAmountException(amount);
        if (amount > balance)
            throw new InsufficientFundsException("Balance " + balance + " is less than " + amount, amount - balance);
        balance -= amount;
        System.out.println("Paid " + amount + ", balance " + balance);
    }
}
public class CustomExceptions {
    public static void main(String[] args) {
        Wallet w = new Wallet();
        double[] payments = {300, 900, -5};
        for (double p : payments) {
            try {
                w.pay(p);
            } catch (InsufficientFundsException e) {
                System.out.println("Declined: " + e.getMessage() + " (short by " + e.getShortBy() + ")");
            } catch (InvalidAmountException e) {
                System.out.println("Rejected: " + e.getMessage());
            }
        }
    }
}
```

OUTPUT

```
Paid 300.0, balance 700.0
Declined: Balance 700.0 is less than 900.0 (short by 200.0)
Rejected: Amount must be positive, got -5.0
```

TIP

Name custom exceptions with the suffix `Exception`, provide at least `(String message)` and `(String message, Throwable cause)` constructors, and make extra fields final.

12.6 Exception methods

12.6.1 getMessage(), toString(), printStackTrace(), getCause()

Method	Returns / does	Example output
`getMessage()`	The detail message only	`/ by zero`
`toString()`	Class name + ": " + message	`java.lang.ArithmeticException: / by zero`
`printStackTrace()`	Prints toString and the stack trace to stderr	full trace with "at …" lines
`getCause()`	The wrapped original exception, or null	`java.lang.NumberFormatException: …`

Method	Returns / does	Example output
`getStackTrace()`	Array of StackTraceElement (class, method, line)	
`getSuppressed()`	Exceptions suppressed during try-with-resources	

WHAT EACH METHOD SHOWS

ExceptionMethods.java

```
public class ExceptionMethods {
    static void level2() { int[] a = new int[2]; a[3] = 7; }
    static void level1() {
        try { level2(); }
        catch (ArrayIndexOutOfBoundsException e) { throw new IllegalStateException("level1 failed", e); }
    }
    public static void main(String[] args) {
        try {
            level1();
        } catch (IllegalStateException e) {
            System.out.println("getMessage() : " + e.getMessage());
            System.out.println("toString()   : " + e);
            System.out.println("getCause()   : " + e.getCause());
            StackTraceElement top = e.getStackTrace()[0];
            System.out.println("thrown in    : " + top.getMethodName() + " line " + top.getLineNumber());
            System.out.println("printStackTrace():");
            e.printStackTrace(System.out);
        }
    }
}
```

OUTPUT

```
getMessage() : level1 failed
toString()   : java.lang.IllegalStateException: level1 failed
getCause()   : java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 2
thrown in    : level1 line 5
printStackTrace():
java.lang.IllegalStateException: level1 failed
        at ExceptionMethods.level1(ExceptionMethods.java:5)
        at ExceptionMethods.main(ExceptionMethods.java:9)
Caused by: java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 2
        at ExceptionMethods.level2(ExceptionMethods.java:2)
        at ExceptionMethods.level1(ExceptionMethods.java:4)
        ... 1 more
```

NOTE

In the trace, `Caused by:` introduces the chained cause and `... 1 more` means the remaining frames are the same as in the enclosing trace.

12.7 Best practices

12.7.1 Catch specific exceptions, not Exception

`catch (Exception e)` also catches bugs you did not expect (NullPointerException, ClassCastException) and hides them. Catch the most specific types you can actually handle and let the rest propagate.

12.7.2 Do not swallow exceptions

An empty catch block **swallows** the exception: the program silently continues in a broken state and nobody knows why. At minimum log it; better, handle it or re-throw it wrapped.

SWALLOWING VS HANDLING

java

```java
try { saveOrder(order); }
catch (SQLException e) { }                                  // BAD: silently ignored

try { saveOrder(order); }
catch (SQLException e) {
    log.error("Could not save order " + order.getId(), e);   // GOOD: logged with cause
    throw new OrderException("Saving order failed", e);      // and reported upward
}
```

12.7.3 Use exceptions for exceptional cases, not control flow

Throwing and catching is slow (building the stack trace) and makes code hard to read. Do not use exceptions where a simple check works.

NORMAL CONDITIONS SHOULD NOT NEED EXCEPTIONS

java

```java
// BAD: using an exception to end a loop
try { int i = 0; while (true) System.out.println(arr[i++]); }
catch (ArrayIndexOutOfBoundsException e) { }

// GOOD
for (int x : arr) System.out.println(x);

// BAD: parse to test                          // GOOD: test first
try { Integer.parseInt(s); valid = true; }     valid = s.matches("-?\\d+");
catch (NumberFormatException e) { valid = false; }
```

12.7.4 Clean up resources

Always release files, streams, connections and locks: use **try-with-resources** for anything AutoCloseable, and `finally` for other cleanup (such as `lock.unlock()`).

12.7.5 Logging exceptions

In real applications, do not use `printStackTrace()` or `System.out`; use a **logger** (java.util.logging built in, or SLF4J/Logback, Log4j2 in industry). Loggers add timestamps and levels (SEVERE/ERROR, WARNING, INFO), write to files, and can be configured without changing code. Pass the exception object itself so the full stack trace is recorded.

LOGGING AN EXCEPTION WITH JAVA.UTIL.LOGGING LoggingDemo.java

```java
import java.util.logging.*;

public class LoggingDemo {
    private static final Logger log = Logger.getLogger(LoggingDemo.class.getName());

    public static void main(String[] args) {
        Logger root = Logger.getLogger("");
        for (Handler h : root.getHandlers()) root.removeHandler(h);
        ConsoleHandler h = new ConsoleHandler();
        h.setFormatter(new SimpleFormatter() {                      // short, predictable format
            @Override public String format(LogRecord r) {
                return r.getLevel() + ": " + r.getMessage()
                        + (r.getThrown() != null ? " | " + r.getThrown() : "") + System.lineSeparator();
            }
        });
        root.addHandler(h);

        log.info("Import started");
        try {
            Integer.parseInt("N/A");
        } catch (NumberFormatException e) {
            log.log(Level.WARNING, "Skipping bad record", e);         // message + exception
        }
        log.severe("Import finished with errors");
    }
}
```

OUTPUT

```
INFO: Import started
WARNING: Skipping bad record | java.lang.NumberFormatException: For input string: "N/A"
SEVERE: Import finished with errors
```

Do

Catch specific types · add context in the message · keep the cause · close resources · log once at the boundary · fail fast on bad input

Do not

Swallow exceptions · catch Throwable/Error · use exceptions for flow control · return from finally · log and re-throw at every layer

12.8 Assertions

12.8.1 assert keyword and enabling with -ea

An **assertion** states something the programmer believes is **always true** at that point. If it is false, an `AssertionError` is thrown. Assertions are for catching **bugs during development and testing**, not for validating user input. They are **disabled by default**; enable them with `java -ea ClassName` (enable assertions).

SYNTAX java

```java
assert condition;
assert condition : "message shown if it fails";
```

RUN WITH: JAVA -EA ASSERTDEMO

AssertDemo.java

```
public class AssertDemo {
    static int discountPercent(int amount) {
        int pct = amount > 5000 ? 20 : amount > 1000 ? 10 : 0;
        assert pct >= 0 && pct <= 20 : "discount out of range: " + pct;   // internal invariant
        return pct;
    }
    public static void main(String[] args) {
        System.out.println("discount for 6000 = " + discountPercent(6000) + "%");
        int balance = -50;                      // imagine a bug produced this
        assert balance >= 0 : "balance went negative: " + balance;
        System.out.println("only printed when assertions are disabled");
    }
}
```

OUTPUT

```
discount for 6000 = 20%
Exception in thread "main" java.lang.AssertionError: balance went negative: -50
        at AssertDemo.main(AssertDemo.java:10)
```

The same program without -ea

```
$ java AssertDemo            # assertions disabled (default)
discount for 6000 = 20%
only printed when assertions are disabled
```

WATCH OUT

Never put code with side effects inside an assert (`assert list.remove(x);`): with assertions disabled the code would not run at all. For public method arguments, throw IllegalArgumentException instead.

PRACTICE CHECKPOINT

An **ATM withdrawal program** that throws a custom `InsufficientBalanceException` and uses **try-with-resources** for input.

SAMPLE SOLUTION

AtmApp.java

```
import java.util.InputMismatchException;
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    private final double balance, requested;
    InsufficientBalanceException(double balance, double requested) {
        super(String.format("Insufficient balance: available %.2f, requested %.2f", balance, requested));
        this.balance = balance;
        this.requested = requested;
    }
    double shortfall() { return requested - balance; }
}

class Atm {
    private double balance;
    Atm(double opening) { balance = opening; }

    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount <= 0 || amount % 100 != 0)
            throw new IllegalArgumentException("Amount must be a positive multiple of 100");
        if (amount > balance) throw new InsufficientBalanceException(balance, amount);
        balance -= amount;
        System.out.printf("Please collect %.0f. Remaining balance: %.2f%n", amount, balance);
    }
}

public class AtmApp {
    public static void main(String[] args) {
        Atm atm = new Atm(5000);
        try (Scanner sc = new Scanner(System.in)) {             // closed automatically
            for (int attempt = 1; attempt <= 4; attempt++) {
```

```
                System.out.print("Enter amount to withdraw: ");
                try {
                    double amt = sc.nextDouble();
                    atm.withdraw(amt);
                } catch (InsufficientBalanceException e) {
                    System.out.println(e.getMessage() + " (short by " + e.shortfall() + ")");
                } catch (IllegalArgumentException e) {
                    System.out.println("Invalid amount: " + e.getMessage());
                } catch (InputMismatchException e) {
                    System.out.println("Please enter digits only.");
                    sc.next();                                  // discard the bad token
                } finally {
                    System.out.println("-- transaction " + attempt + " ended --");
                }
            }
        }
        System.out.println("Thank you for banking with LBF.");
    }
}
```

CONSOLE (user input in gold)

```
Enter amount to withdraw: 2000
Please collect 2000. Remaining balance: 3000.00
-- transaction 1 ended --
Enter amount to withdraw: 350
Invalid amount: Amount must be a positive multiple of 100
-- transaction 2 ended --
Enter amount to withdraw: abc
Please enter digits only.
-- transaction 3 ended --
Enter amount to withdraw: 4000
Insufficient balance: available 3000.00, requested 4000.00 (short by 1000.0)
-- transaction 4 ended --
Thank you for banking with LBF.
```

MODULE 13 · INTERMEDIATE

Generics

Minimum duration **2 days** | Level **Intermediate**

Generics let classes and methods work with any type while keeping compile-time type safety. They are the foundation for the Collections module that follows.

IN THIS MODULE

13.1 Why generics
13.2 Generic classes
13.3 Generic methods
13.4 Generic interfaces
13.5 Bounded type parameters
13.6 Wildcards
13.7 Type erasure
13.8 Generics and inheritance

13.1 Why generics

13.1.1 Problems without generics (raw types, casting, ClassCastException)

Before Java 5, collections stored everything as `Object`. You could put **anything** into a list, and you had to **cast** every element you took out. A wrong element was not detected by the compiler; it crashed at run time, often far from where it was added. A collection used without a type argument (`List list`) is called a **raw type**.

THE PRE-GENERICS PROBLEM: A RUN-TIME CRASH

RawTypes.java

```
import java.util.ArrayList;
import java.util.List;

public class RawTypes {
    @SuppressWarnings({"rawtypes", "unchecked"})
    public static void main(String[] args) {
        List names = new ArrayList();          // raw type: holds Objects
        names.add("Asha");
        names.add("Ravi");
        names.add(42);                         // compiles! nobody stops this mistake
        for (Object o : names) {
            String s = (String) o;             // manual cast on every read
            System.out.println(s.toUpperCase());
        }
    }
}
```

OUTPUT

```
ASHA
RAVI
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot be cast to class
java.lang.String (java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
        at RawTypes.main(RawTypes.java:12)
```

13.1.2 Type safety and code reuse

With generics you state the element type once, `List<String>`. The compiler then **rejects** wrong elements and inserts the casts for you. One generic class (`ArrayList<E>`) is **reused** for every type: strings, integers, students, without writing StringList, IntegerList, StudentList.

THE SAME CODE, MADE TYPE-SAFE

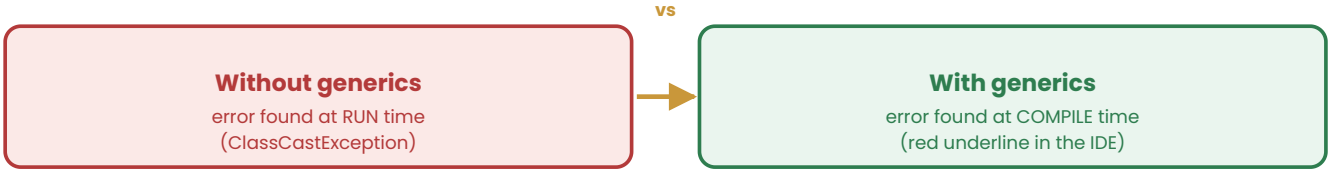
WithGenerics.java

```
import java.util.ArrayList;
import java.util.List;

public class WithGenerics {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Asha");
        names.add("Ravi");
        // names.add(42);                  // compile error: incompatible types: int cannot be converted to String
        for (String s : names)             // no cast needed
            System.out.println(s.toUpperCase());
    }
}
```

OUTPUT

```
ASHA
RAVI
```



GENERICS MOVE ERRORS FROM RUN TIME TO COMPILE TIME

Type safety
Wrong types are compile errors.

No casts
The compiler inserts them, so code is shorter and clearer.

Reuse
One generic algorithm or container works for every type.

13.2 Generic classes

13.2.1 Type parameters <T>

A generic class declares one or more **type parameters** in angle brackets after its name: `class Box<T>`. Inside the class, `T` is used like a real type for fields, parameters and return types. When you use the class you supply a **type argument**: `Box<String>`, and every T becomes String for that object.

ONE BOX CLASS FOR ANY TYPE

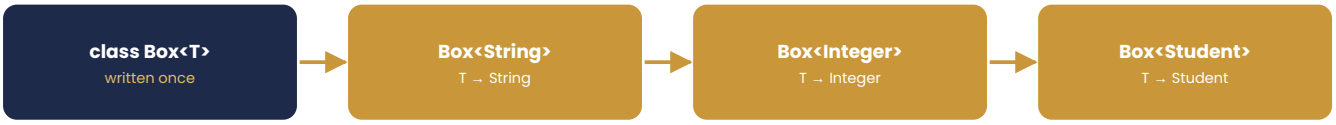
GenericBox.java

```
class Box<T> {                        // T = type parameter
    private T content;
    void put(T item) { content = item; }
    T get() { return content; }
    boolean isEmpty() { return content == null; }
}
public class GenericBox {
    public static void main(String[] args) {
        Box<String> textBox = new Box<>();          // T = String
        textBox.put("Learn Being Forward");
        String s = textBox.get();                   // no cast
        System.out.println(s + " (" + s.length() + " chars)");

        Box<Integer> numBox = new Box<>();          // T = Integer
        numBox.put(2026);
        int year = numBox.get() + 1;
        System.out.println("next year: " + year);
        // numBox.put("hello");                     // compile error
        System.out.println("empty? " + new Box<Double>().isEmpty());
    }
}
```

OUTPUT

```
Learn Being Forward (19 chars)
next year: 2027
empty? true
```



ONE GENERIC CLASS, MANY PARAMETERISED TYPES

13.2.2 Naming conventions: T, E, K, V, N, R

Letter	Stands for	Used in
`T`	Type (general)	`Box<T>`, `Optional<T>`
`E`	Element	`List<E>`, `Set<E>`
`K`	Key	`Map<K, V>`
`V`	Value	`Map<K, V>`
`N`	Number	`<N extends Number>`
`R`	Result / return type	`Function<T, R>`
`S, U`	2nd, 3rd types	`BiFunction<T, U, R>`

13.2.3 Multiple type parameters <K, V>

A PAIR WITH TWO TYPE PARAMETERS — PairDemo.java

```
class Pair<K, V> {
    private final K key;
    private final V value;
    Pair(K key, V value) { this.key = key; this.value = value; }
    K getKey()   { return key; }
    V getValue() { return value; }
    @Override public String toString() { return "(" + key + " -> " + value + ")"; }
}
public class PairDemo {
    public static void main(String[] args) {
        Pair<String, Integer> marks = new Pair<>("Asha", 91);
        Pair<Integer, String> roll  = new Pair<>(101, "Ravi");
        Pair<String, Pair<String, Double>> nested = new Pair<>("topper", new Pair<>("Meera", 98.5));
        System.out.println(marks + " " + roll + " " + nested);
        int m = marks.getValue();
        System.out.println(marks.getKey() + " scored " + m + ", topper is " + nested.getValue().getKey());
    }
}
```

OUTPUT

```
(Asha -> 91) (101 -> Ravi) (topper -> (Meera -> 98.5))
Asha scored 91, topper is Meera
```

13.2.4 Diamond operator <>

Since Java 7 you can leave the type arguments on the right-hand side empty: `Map<String, List<Integer>> m = new HashMap<>();`. The compiler **infers** them from the left side. Without the diamond you would write the full types twice. (Writing `new HashMap()` without <> creates a **raw type**, which is a different and unsafe thing.)

WITH AND WITHOUT THE DIAMOND — java

```
Map<String, List<Integer>> a = new HashMap<String, List<Integer>>();   // Java 5/6
Map<String, List<Integer>> b = new HashMap<>();                         // Java 7+ diamond
var c = new HashMap<String, List<Integer>>();                           // Java 10+ var: type on the right
```

13.3 Generic methods

13.3.1 Declaring <T> void print(T item)

A single **method** can be generic even if its class is not. The type parameter list goes **before the return type**: `<T> void print(T item)`. The type parameter belongs to that method only.

13.3.2 Static generic methods

Utility methods are often static generic methods: `Collections.max`, `Arrays.asList`, `List.of`. The syntax is `static <T> T first(List<T> list)`.

13.3.3 Type inference

Normally you do not specify T: the compiler **infers** it from the arguments (and the target type). You can still give it explicitly: `Util.<String>first(list)`.

GENERIC METHODS WITH INFERENCE

GenericMethods.java

```
import java.util.Arrays;
import java.util.List;

public class GenericMethods {
    static <T> void print(T item) {                          // 13.3.1
        System.out.println("[" + item.getClass().getSimpleName() + "] " + item);
    }
    static <T> T first(List<T> list) {                       // 13.3.2 static generic
        return list.isEmpty() ? null : list.get(0);
    }
    static <T> void swap(T[] arr, int i, int j) {
        T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
    }
    static <K, V> String describe(K key, V value) {
        return key + "=" + value;
    }
    public static void main(String[] args) {
        print("hello");                                      // T inferred as String
        print(3.14);                                         // Double
        print(List.of(1, 2));                                // an internal immutable List class
        String f = first(List.of("x", "y"));                 // 13.3.3 inferred from argument
        Integer n = GenericMethods.<Integer>first(List.of(7, 8));   // explicit type argument
        System.out.println(f + " " + n);
        Integer[] nums = {1, 2, 3};
        swap(nums, 0, 2);
        System.out.println(Arrays.toString(nums));
        System.out.println(describe("year", 2026));
    }
}
```

OUTPUT

```
[String] hello
[Double] 3.14
[List12] [1, 2]
x 7
[3, 2, 1]
year=2026
```

13.4 Generic interfaces

Interfaces can be generic too: `Comparable<T>`, `Comparator<T>`, `Iterable<T>`, `Function<T, R>`. A class implementing one can either **fix** the type (`class Student implements Comparable<Student>`) or stay generic itself (`class MemoryRepo<T> implements Repository<T>`).

A GENERIC REPOSITORY INTERFACE GenericInterface.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

interface Repository<T, ID> {                       // generic interface
    void save(T item);
    Optional<T> findById(ID id);
    List<T> findAll();
}
record Course(String code, String title) { }

class CourseRepo implements Repository<Course, String> {   // type arguments fixed
    private final List<Course> data = new ArrayList<>();
    public void save(Course c) { data.add(c); }
    public Optional<Course> findById(String code) {
        for (Course c : data) if (c.code().equals(code)) return Optional.of(c);
        return Optional.empty();
    }
    public List<Course> findAll() { return data; }
}
class Student implements Comparable<Student> {              // Comparable<T>
    String name; int marks;
    Student(String n, int m) { name = n; marks = m; }
    public int compareTo(Student o) { return Integer.compare(marks, o.marks); }
}
public class GenericInterface {
    public static void main(String[] args) {
        Repository<Course, String> repo = new CourseRepo();
        repo.save(new Course("J101", "Core Java"));
        repo.save(new Course("S201", "SQL"));
        System.out.println(repo.findById("S201").map(Course::title).orElse("none"));
        System.out.println(repo.findById("X999").isPresent());
        System.out.println("all: " + repo.findAll().size());
        System.out.println("compare: " + new Student("A", 90).compareTo(new Student("B", 75)));
    }
}
```

OUTPUT

```
SQL
false
all: 2
compare: 1
```

13.5 Bounded type parameters

13.5.1 Upper bound <T extends Number>

Sometimes a generic method needs to call methods that only certain types have. `<T extends Number>` restricts T to Number **or its subclasses** (Integer, Double, Long…). Inside, you can call Number methods such as `doubleValue()`. (In generics `extends` is used for both classes and interfaces.)

UPPER-BOUNDED TYPE PARAMETERS UpperBound.java

```
import java.util.List;

public class UpperBound {
    static <T extends Number> double average(List<T> nums) {
        double sum = 0;
        for (T n : nums) sum += n.doubleValue();          // allowed: T is a Number
        return nums.isEmpty() ? 0 : sum / nums.size();
    }
    static <T extends Comparable<T>> T maxOf(List<T> items) {
        T best = items.get(0);
        for (T x : items) if (x.compareTo(best) > 0) best = x;
        return best;
    }
    public static void main(String[] args) {
        System.out.println(average(List.of(80, 90, 100)));        // Integer
        System.out.println(average(List.of(2.5, 3.5)));           // Double
        // average(List.of("a", "b"));                            // compile error: String is not a Number
        System.out.println(maxOf(List.of(4, 19, 7)));
        System.out.println(maxOf(List.of("pear", "apple", "zebra", "mango")));
    }
}
```

OUTPUT

```
90.0
3.0
19
zebra
```

13.5.2 Multiple bounds <T extends A & B>

A type parameter can have several bounds joined by `&`: one class (which must come first) plus any number of interfaces. T must satisfy **all** of them.

T MUST BE A NUMBER AND COMPARABLE MultipleBounds.java

```
import java.io.Serializable;

public class MultipleBounds {
    static <T extends Number & Comparable<T>> T clamp(T value, T min, T max) {
        if (value.compareTo(min) < 0) return min;     // Comparable method
        if (value.compareTo(max) > 0) return max;
        return value;
    }
    static <T extends CharSequence & Serializable> int len(T text) { return text.length(); }

    public static void main(String[] args) {
        System.out.println(clamp(150, 0, 100));
        System.out.println(clamp(-3.5, 0.0, 10.0));
        System.out.println(clamp(42L, 0L, 100L));
        System.out.println(len("generics"));
    }
}
```

OUTPUT

```
100
0.0
42
8
```

13.6 Wildcards

A **wildcard** `?` means "some unknown type". Wildcards are used in parameter types (and fields/variables) to make methods accept a **family** of parameterised types.

13.6.1 Unbounded <?>

`List<?>` is a list of *some* type. You can read elements as `Object`, but you cannot add anything (except null), because the compiler does not know the real type. Use it when the code does not care about the element type: printing, counting, clearing.

13.6.2 Upper-bounded <? extends T>

`List<? extends Number>` accepts `List<Integer>`, `List<Double>`, `List<Number>`. You can safely **read** elements as Number. You cannot **add** (the list might be a List<Double>, so adding an Integer would corrupt it).

13.6.3 Lower-bounded <? super T>

`List<? super Integer>` accepts `List<Integer>`, `List<Number>`, `List<Object>`. You can safely **add** Integers (every possible list can hold them), but reading gives only `Object`.

THE THREE KINDS OF WILDCARD — Wildcards.java

```
import java.util.ArrayList;
import java.util.List;

public class Wildcards {
    static void printAll(List<?> list) {                        // 13.6.1 any list
        System.out.println("  " + list.size() + " items: " + list);
    }
    static double sum(List<? extends Number> nums) {            // 13.6.2 read Numbers
        double s = 0;
        for (Number n : nums) s += n.doubleValue();
        // nums.add(5);                                          // compile error
        return s;
    }
    static void addSquares(List<? super Integer> dest, int n) { // 13.6.3 write Integers
        for (int i = 1; i <= n; i++) dest.add(i * i);
        Object first = dest.get(0);                             // reading gives only Object
    }
    public static void main(String[] args) {
        List<Integer> ints = List.of(1, 2, 3);
        List<Double> dbls = List.of(1.5, 2.5);
        printAll(ints);
        printAll(List.of("a", "b"));
        System.out.println("sum ints = " + sum(ints) + ", sum doubles = " + sum(dbls));

        List<Number> numbers = new ArrayList<>();
        List<Object> objects = new ArrayList<>();
        addSquares(numbers, 4);
        addSquares(objects, 3);
        System.out.println("numbers = " + numbers + ", objects = " + objects);
    }
}
```

OUTPUT

```
  3 items: [1, 2, 3]
  2 items: [a, b]
sum ints = 6.0, sum doubles = 4.0
numbers = [1, 4, 9, 16], objects = [1, 4, 9]
```

13.6.4 PECS rule (Producer Extends, Consumer Super)

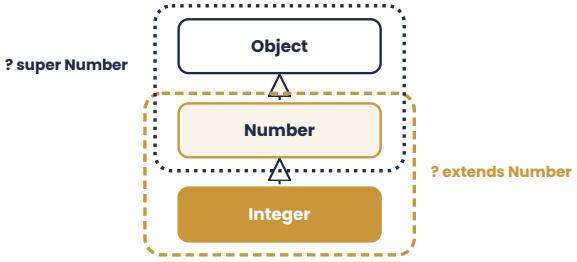
How do you choose? Joshua Bloch's rule **PECS**: if a parameter **produces** values that your method reads, use `? extends T`; if it **consumes** values that your method writes into it, use `? super T`; if it does both, use exact `T`. The JDK's `Collections.copy` is the textbook example.

JAVA.UTIL.COLLECTIONS.COPY — java

```
public static <T> void copy(List<? super T> dest,       // CONSUMER: we write into it
                            List<? extends T> src)       // PRODUCER: we read from it
```



PECS: DATA FLOWS OUT OF THE PRODUCER (EXTENDS) AND INTO THE CONSUMER (SUPER)



EXTENDS COVERS A TYPE AND EVERYTHING BELOW IT; SUPER COVERS A TYPE AND EVERYTHING ABOVE IT

13.7 Type erasure

13.7.1 What the compiler does to generics

Generics exist only at **compile time**. After checking the types, the compiler **erases** them: each type parameter is replaced by its bound (`Object` if unbounded, `Number` for `T extends Number`), and casts are inserted where values are read. The bytecode for `Box<String>` and `Box<Integer>` is the **same class**. This kept new generic code compatible with old pre-Java 5 code.

BEFORE AND AFTER ERASURE java

```java
// what you write
class Box<T> {
    T content;
    T get() { return content; }
}
String s = box.get();
```

```java
// what the compiler generates (roughly)
class Box {
    Object content;
    Object get() { return content; }
}
String s = (String) box.get();
```

AT RUN TIME, LIST AND LIST ARE THE SAME CLASS Erasure.java

```java
import java.util.ArrayList;
import java.util.List;

public class Erasure {
    public static void main(String[] args) {
        List<String> a = new ArrayList<>();
        List<Integer> b = new ArrayList<>();
        System.out.println("same runtime class? " + (a.getClass() == b.getClass()));
        System.out.println("runtime class name : " + a.getClass().getName());
        // if (a instanceof List<String>) { }   // error in older Java: type info not available at run time
        System.out.println("a instanceof List<?>: " + (a instanceof List<?>));
    }
}
```

OUTPUT

```
same runtime class? true
runtime class name : java.util.ArrayList
a instanceof List<?>: true
```

13.7.2 Restrictions: no primitives, no new T(), no generic arrays, no static T fields

Not allowed	Why	Workaround
`List<int>`	After erasure T becomes Object; a primitive is not an Object	Use wrappers: `List<Integer>`

Not allowed	Why	Workaround
`new T()`	At run time T is unknown, so the JVM cannot know which class to create	Pass a `Supplier<T>` or `Class<T>`
`new T[10]`, `new List<String>[10]`	Arrays check element types at run time, but T has been erased	Use `List<T>`, or `(T[]) new Object[n]` internally
`static T field;`	Static members are shared by Box<String> and Box<Integer>; T would differ	Make it an instance field or a generic static method
`instanceof Box<String>`	No type argument at run time	Use `instanceof Box<?>`
catching `T` / generic exceptions	catch needs a runtime type	Not possible

CREATING T OBJECTS THROUGH A SUPPLIER ErasureWorkarounds.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Supplier;

class Factory<T> {
    private final Supplier<T> maker;                // workaround for new T()
    Factory(Supplier<T> maker) { this.maker = maker; }
    List<T> make(int n) {
        List<T> list = new ArrayList<>();
        for (int i = 0; i < n; i++) list.add(maker.get());
        return list;
    }
}
public class ErasureWorkarounds {
    public static void main(String[] args) {
        Factory<StringBuilder> f = new Factory<>(StringBuilder::new);
        List<StringBuilder> sbs = f.make(3);
        sbs.get(0).append("first");
        System.out.println(sbs.size() + " objects created, first = " + sbs.get(0));
        List<Integer> ints = new ArrayList<>();          // wrapper instead of int
        ints.add(5);
        System.out.println(ints);
    }
}
```

OUTPUT

```
3 objects created, first = first
[5]
```

13.8 Generics and inheritance

13.8.1 Why List<Integer> is not a List<Number>

Integer IS-A Number, so it is natural to expect that `List<Integer>` IS-A `List<Number>`. It is **not**: generic types are **invariant**. If it were allowed, this would compile and corrupt the list:

WHY INVARIANCE IS NECESSARY java

```
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;        // suppose this were allowed (it is a compile error)
nums.add(3.14);                  // a Double inside a List<Integer>!
Integer x = ints.get(0);         // ClassCastException at run time
```

Arrays, by contrast, **are** covariant (`Integer[]` IS-A `Number[]`), and pay for it with a run-time check:

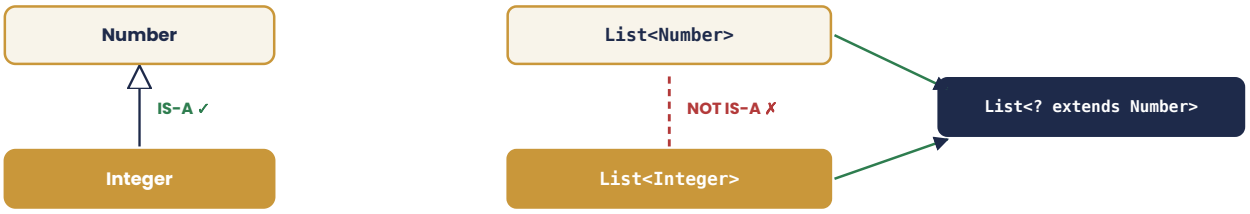
ARRAYS CATCH THE MISTAKE ONLY AT RUN TIME

ArrayCovariance.java

```
public class ArrayCovariance {
    public static void main(String[] args) {
        Integer[] ints = {1, 2, 3};
        Number[] nums = ints;                    // allowed for arrays
        try {
            nums[0] = 3.14;                      // compiles, fails at run time
        } catch (ArrayStoreException e) {
            System.out.println("ArrayStoreException: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
ArrayStoreException: java.lang.Double
```



THE ELEMENT TYPES ARE RELATED, THE LIST TYPES ARE NOT; A WILDCARD TYPE IS THE COMMON SUPERTYPE

Use a wildcard when you need a method that accepts lists of several related types: `double sum(List<? extends Number> list)` (13.6.2). Note that normal inheritance still works on the generic class itself: `ArrayList<Integer>` IS-A `List<Integer>` IS-A `Collection<Integer>`.

PRACTICE CHECKPOINT

A generic `Box<T>`, a generic `Pair<K, V>`, and a bounded method that sums any list of numbers.

SAMPLE SOLUTION

GenericsCheckpoint.java

```
import java.math.BigDecimal;
import java.util.List;

class Box<T> {
    private T item;
    Box(T item) { this.item = item; }
    T get() { return item; }
    void set(T item) { this.item = item; }
    <R> Box<R> map(java.util.function.Function<T, R> f) { return new Box<>(f.apply(item)); }
    @Override public String toString() { return "Box[" + item + "]"; }
}

class Pair<K, V> {
    private final K key;
    private final V value;
    Pair(K key, V value) { this.key = key; this.value = value; }
    K key() { return key; }
    V value() { return value; }
    Pair<V, K> swap() { return new Pair<>(value, key); }
    @Override public String toString() { return "Pair(" + key + ", " + value + ")"; }
}

public class GenericsCheckpoint {
    static double sum(List<? extends Number> numbers) {         // bounded wildcard
        double total = 0;
        for (Number n : numbers) total += n.doubleValue();
        return total;
    }
    static <T extends Number> T largest(List<T> numbers) {      // bounded type parameter
        T best = numbers.get(0);
        for (T n : numbers) if (n.doubleValue() > best.doubleValue()) best = n;
        return best;
```

```
    }
    public static void main(String[] args) {
        Box<String> name = new Box<>("generics");
        Box<Integer> len = name.map(String::length);
        System.out.println(name + " -> " + len);

        Pair<String, Integer> p = new Pair<>("Asha", 91);
        System.out.println(p + " swapped " + p.swap());

        System.out.println("sum ints    = " + sum(List.of(10, 20, 30)));
        System.out.println("sum doubles = " + sum(List.of(1.25, 2.75)));
        System.out.println("sum mixed   = " + sum(List.of(1, 2.5, 3L, new BigDecimal("0.5"))));
        System.out.println("largest     = " + largest(List.of(4, 42, 17)));
    }
}
```

OUTPUT

```
Box[generics] -> Box[8]
Pair(Asha, 91) swapped Pair(91, Asha)
sum ints    = 60.0
sum doubles = 4.0
sum mixed   = 7.0
largest     = 42
```

MODULE 14 · INTERMEDIATE

Collections Framework

Minimum duration **8 days** | Level **Intermediate**

The Collections Framework provides ready-made data structures (lists, sets, queues, maps) and algorithms to store and process groups of objects.

IN THIS MODULE

14.1 Introduction
14.2 Iterating over collections
14.3 List interface
14.4 Set interface
14.5 Queue and Deque
14.6 Map interface
14.7 Sorting and comparing objects
14.8 Collections utility class
14.9 Choosing the right collection
14.10 Data structures with collections

14.1 Introduction

14.1.1 Limitations of arrays

Arrays are fast and simple, but real programs quickly hit their limits:

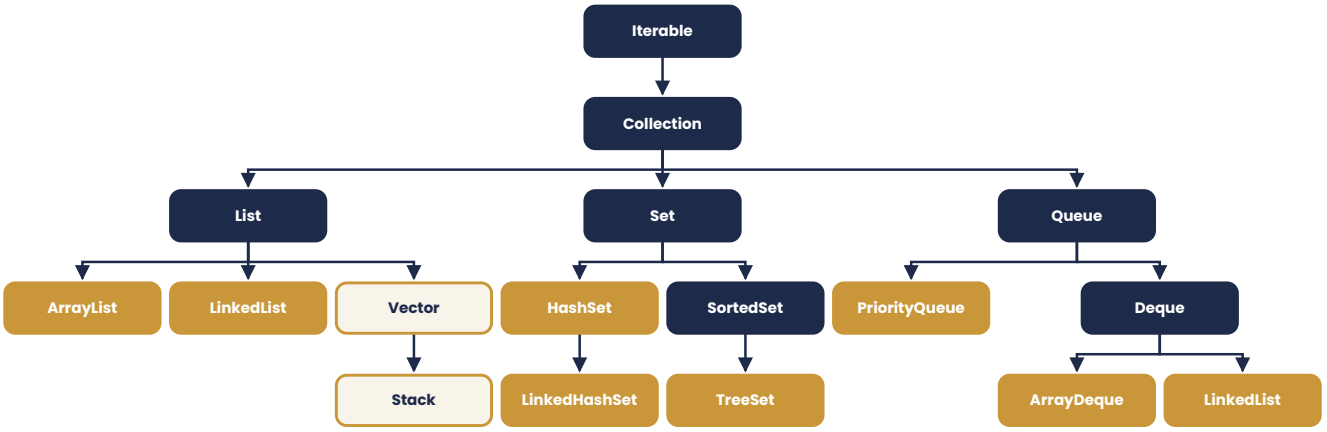
Limitation of arrays	How collections solve it
Fixed size: decided at creation; adding a 101st student to a 100-element array means creating a new array and copying	Grow and shrink automatically (ArrayList, HashMap…)
No built-in insert/delete in the middle: you shift elements by hand	`add(index, x)`, `remove(x)`
No fast search by value or key: you write a loop	`contains()`, `HashMap.get(key)` in O(1)
No uniqueness or sorting guarantees	Set (no duplicates), TreeSet/TreeMap (always sorted)
Only `length`, few methods; printing needs Arrays.toString	Rich API, readable `toString()`, streams
Can hold primitives directly (an advantage)	Need wrappers (Integer), slightly more memory

The **Java Collections Framework** (package `java.util`) is a unified set of **interfaces** (List, Set, Queue, Map), **implementations** (ArrayList, HashSet, HashMap…) and **algorithms** (sort, search, shuffle) for storing and processing groups of objects.

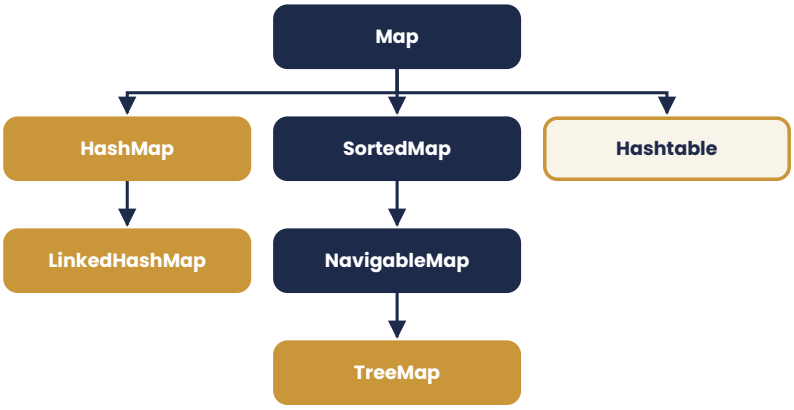
14.1.2 Collection vs Collections (interface vs utility class)

	`Collection`	`Collections`
What	An **interface**: the root of List, Set and Queue	A **utility class** of static helper methods
Used as	`Collection<String> c = new ArrayList<>();`	`Collections.sort(list)`
Contains	add, remove, size, contains, iterator, stream…	sort, reverse, shuffle, max, min, frequency, unmodifiableList…
Similar to	-	`Arrays` (helpers for arrays)

14.1.3 Collection hierarchy: Iterable → Collection → List, Set, Queue; Map as a separate hierarchy



THE COLLECTION HIERARCHY: NAVY = INTERFACES, GOLD = MAIN CLASSES, CREAM = LEGACY CLASSES (LINKEDLIST IMPLEMENTS BOTH LIST AND DEQUE)



MAP IS A SEPARATE HIERARCHY: IT STORES KEY-VALUE PAIRS, NOT SINGLE ELEMENTS, SO IT DOES NOT EXTEND COLLECTION

Interface	Order	Duplicates	Access	Main classes
`List`	Insertion order, by index	Allowed	By index	ArrayList, LinkedList
`Set`	None / insertion / sorted	Not allowed	By value	HashSet, LinkedHashSet, TreeSet
`Queue` / `Deque`	FIFO, priority, or both ends	Allowed	Head (and tail)	ArrayDeque, PriorityQueue, LinkedList
`Map`	None / insertion / sorted keys	Keys unique, values may repeat	By key	HashMap, LinkedHashMap, TreeMap

Program to the **interface**: `List<String> names = new ArrayList<>();`. If you later need a LinkedList, only the right-hand side changes.

14.2 Iterating over collections

14.2.1 for-each loop

The enhanced for loop works with any `Iterable` (every Collection) and arrays. It is the simplest way to read every element. It cannot give you the index or remove elements.

14.2.2 Iterator: hasNext(), next(), remove()

Every collection provides an `Iterator`, a cursor that walks the elements: `hasNext()` asks whether more elements exist, `next()` returns the next one and advances, `remove()` deletes the element last returned by next(). The for-each loop is compiled into exactly this. Use an explicit iterator when you need to **remove while iterating**.

ITERATING AND REMOVING SAFELY

IteratorDemo.java

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorDemo {
    public static void main(String[] args) {
        List<Integer> marks = new ArrayList<>(List.of(78, 34, 91, 28, 66, 45));

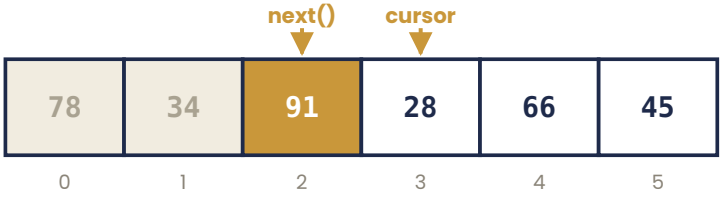
        System.out.print("for-each : ");
        for (int m : marks) System.out.print(m + " ");
        System.out.println();

        Iterator<Integer> it = marks.iterator();
        while (it.hasNext()) {
            int m = it.next();
            if (m < 40) it.remove();                // safe removal of failed marks
        }
        System.out.println("after removing < 40: " + marks);

        marks.removeIf(m -> m < 50);                // Java 8 shortcut for the same idea
        System.out.println("after removeIf < 50 : " + marks);
    }
}
```

OUTPUT

```
for-each : 78 34 91 28 66 45
after removing < 40: [78, 91, 66, 45]
after removeIf < 50 : [78, 91, 66]
```



THE ITERATOR'S CURSOR SITS BETWEEN ELEMENTS; NEXT() RETURNS THE ELEMENT IT JUMPS OVER

14.2.3 ListIterator (bidirectional)

`ListIterator` (only for Lists) can move **both ways** (`hasPrevious()`, `previous()`), knows the index (`nextIndex()`), and can `set()` or `add()` elements during iteration.

MODIFYING AND WALKING BACKWARDS ListIteratorDemo.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class ListIteratorDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(List.of("asha", "ravi", "meera"));
        ListIterator<String> it = names.listIterator();
        while (it.hasNext()) {
            int i = it.nextIndex();
            String n = it.next();
            it.set(n.toUpperCase());                    // replace current element
            if (i == 0) it.add("NEW");                  // insert after it
        }
        System.out.println("forward : " + names);
        System.out.print("backward: ");
        while (it.hasPrevious()) System.out.print(it.previous() + " ");
        System.out.println();
    }
}
```

```
OUTPUT
forward : [ASHA, NEW, RAVI, MEERA]
backward: MEERA RAVI NEW ASHA
```

14.2.4 forEach() with lambdas

Since Java 8, every Iterable has `forEach(action)`, and Map has `forEach((k, v) -> …)`. The action is a lambda or method reference (Module 17).

FOREACH ON A LIST AND A MAP ForEachLambda.java

```
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class ForEachLambda {
    public static void main(String[] args) {
        List<String> courses = List.of("Java", "SQL", "DSA");
        courses.forEach(c -> System.out.println("Course: " + c));
        courses.forEach(System.out::println);                       // method reference

        Map<String, Integer> fees = new TreeMap<>(Map.of("Java", 15000, "SQL", 8000));
        fees.forEach((course, fee) -> System.out.println(course + " costs " + fee));
    }
}
```

```
OUTPUT
Course: Java
Course: SQL
Course: DSA
Java
SQL
DSA
Java costs 15000
SQL costs 8000
```

14.2.5 ConcurrentModificationException; fail-fast vs fail-safe iterators

If a collection is structurally modified (add/remove) **while** you iterate over it with for-each or an iterator, other than through the iterator's own remove(), the iterator of ArrayList, HashMap etc. throws `ConcurrentModificationException` on the next step. Such iterators are **fail-fast**: they keep a modification counter (`modCount`) and stop as soon as it changes unexpectedly, rather than returning wrong results.

The concurrent collections (`CopyOnWriteArrayList`, `ConcurrentHashMap`, Module 16) have **fail-safe** (weakly consistent) iterators: they work on a snapshot or tolerate changes, and never throw CME.

FAIL-FAST VS FAIL-SAFE

FailFast.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class FailFast {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String s : list)
                if (s.equals("a")) list.remove(s);          // modifying during for-each
        } catch (java.util.ConcurrentModificationException e) {
            System.out.println("ArrayList (fail-fast): ConcurrentModificationException");
        }

        List<String> safe = new CopyOnWriteArrayList<>(List.of("a", "b", "c"));
        for (String s : safe)
            if (s.equals("a")) safe.add("d");               // iterates over a snapshot
        System.out.println("CopyOnWriteArrayList (fail-safe): " + safe);
    }
}
```

OUTPUT

```
ArrayList (fail-fast): ConcurrentModificationException
CopyOnWriteArrayList (fail-safe): [a, b, c, d]
```

	Fail-fast	Fail-safe
Collections	ArrayList, HashSet, HashMap, LinkedList…	CopyOnWriteArrayList, ConcurrentHashMap…
On modification	Throws ConcurrentModificationException	Continues (snapshot / weakly consistent)
Memory	No extra copy	May copy the data
Fix in normal code	Use `iterator.remove()` or `removeIf()`	-

14.3 List interface (ordered, allows duplicates)

A **List** keeps elements in a **sequence**, each at an index starting from 0, and allows **duplicates** and nulls. It adds index-based methods to Collection: `get(i)`, `set(i, x)`, `add(i, x)`, `remove(i)`, `indexOf(x)`, `subList(a, b)`, `sort(cmp)`.

14.3.1 ArrayList

14.3.1.1 Dynamic array internals and resizing

`ArrayList` stores its elements in an ordinary **array** (`Object[] elementData`) that is usually bigger than the number of elements (**size** vs **capacity**). When the array is full and you add one more element, ArrayList creates a **new array about 1.5× larger**, copies everything over, and discards the old one. The default capacity is 10 (allocated on the first add).



GROWING AN ARRAYLIST: ALLOCATE A LARGER ARRAY, COPY, THEN ADD

NOTE

Because each resize copies everything, but the array grows by 50% each time, adding n elements costs O(n) in total, so `add()` is **amortised O(1)**. If you know the size in advance, use `new ArrayList<>(expectedSize)` to avoid the copies.

14.3.1.2 Methods: add, get, set, remove, size, contains, indexOf, clear

THE EVERYDAY ARRAYLIST API

ArrayListMethods.java

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListMethods {
    public static void main(String[] args) {
        List<String> cities = new ArrayList<>();
        cities.add("Delhi");                        // append
        cities.add("Mumbai");
        cities.add("Chennai");
        cities.add(1, "Pune");                      // insert at index 1 (shifts right)
        cities.add("Mumbai");                       // duplicates allowed
        System.out.println("list        : " + cities + "  size " + cities.size());
        System.out.println("get(2)      : " + cities.get(2));
        cities.set(0, "New Delhi");                 // replace
        System.out.println("after set   : " + cities);
        System.out.println("contains    : " + cities.contains("Pune") + ", indexOf Mumbai " + cities.indexOf("Mumbai")
                + ", lastIndexOf " + cities.lastIndexOf("Mumbai"));
        cities.remove("Mumbai");                    // removes first occurrence (by value)
        cities.remove(0);                           // removes by index
        System.out.println("after remove: " + cities);
        System.out.println("subList(0,2): " + cities.subList(0, 2));

        List<Integer> nums = new ArrayList<>(List.of(10, 20, 30));
        nums.remove(Integer.valueOf(10));           // remove the VALUE 10
        nums.remove(0);                             // remove INDEX 0
        System.out.println("nums        : " + nums);
        cities.clear();
        System.out.println("after clear : " + cities + ", isEmpty " + cities.isEmpty());
    }
}
```

OUTPUT

```
list        : [Delhi, Pune, Mumbai, Chennai, Mumbai]  size 5
get(2)      : Mumbai
after set   : [New Delhi, Pune, Mumbai, Chennai, Mumbai]
contains    : true, indexOf Mumbai 2, lastIndexOf 4
after remove: [Pune, Chennai, Mumbai]
subList(0,2): [Pune, Chennai]
nums        : [30]
after clear : [], isEmpty true
```

WATCH OUT

`list.remove(1)` on a `List<Integer>` removes **index** 1, not the value 1. Use `remove(Integer.valueOf(1))` to remove a value.

14.3.1.3 Time complexity of operations

Operation	ArrayList	Why
`get(i)`, `set(i, x)`	**O(1)**	Direct array index
`add(x)` at end	**O(1)** amortised	Occasional resize
`add(i, x)`, `remove(i)` in middle	O(n)	Elements after i are shifted
`contains(x)`, `indexOf(x)`	O(n)	Linear search with equals()
`remove(last)`	O(1)	No shift needed

14.3.2 LinkedList

14.3.2.1 Doubly linked list internals

`LinkedList` stores each element in a separate **node** object that holds the element plus references to the **previous** and **next** nodes. The list keeps references to the `first` and `last` nodes. Inserting or removing at either end, or at a position you already hold an iterator to, just re-links a few references: O(1). But reaching index i means walking node by node: O(n).



EACH NODE LINKS TO ITS NEIGHBOURS IN BOTH DIRECTIONS

14.3.2.2 Used as List, Queue and Deque

LinkedList implements **both** `List` and `Deque`, so the same class can act as a list, a FIFO queue and a stack.

ONE LINKEDLIST, THREE ROLES LinkedListDemo.java

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> ll = new LinkedList<>();
        ll.add("B"); ll.add("C");
        ll.addFirst("A");                  // O(1) at the front
        ll.addLast("D");                   // O(1) at the back
        ll.add(2, "X");                    // List method (walks to index 2)
        System.out.println("as list : " + ll + ", get(2) = " + ll.get(2));

        System.out.println("peekFirst " + ll.peekFirst() + ", peekLast " + ll.peekLast());
        ll.removeFirst(); ll.removeLast();
        System.out.println("after removing both ends: " + ll);

        ll.offer("E");                     // Queue: add at tail
        System.out.println("queue poll -> " + ll.poll() + ", now " + ll);
        ll.push("TOP");                    // Stack: add at head
        System.out.println("stack pop  -> " + ll.pop() + ", now " + ll);
    }
}
```

OUTPUT

```
as list : [A, B, X, C, D], get(2) = X
peekFirst A, peekLast D
after removing both ends: [B, X, C]
queue poll -> B, now [X, C, E]
stack pop  -> TOP, now [X, C, E]
```

14.3.3 ArrayList vs LinkedList

	ArrayList	LinkedList
Internal structure	Resizable array	Doubly linked nodes
`get(i)`	**O(1)**	O(n)
Add/remove at end	O(1) amortised	O(1)
Add/remove at front	O(n) (shift)	**O(1)**
Add/remove in middle	O(n) shift	O(n) to find + O(1) to link
Memory per element	Small (one reference)	Large (node with 3 references)
CPU cache friendliness	Excellent (contiguous)	Poor (scattered nodes)

	ArrayList	LinkedList
Use when	Almost always: reading by index, appending	Many inserts/removes at the ends (but ArrayDeque is usually better)

MEASURING THE DIFFERENCE (TRUE/FALSE ONLY, TIMINGS VARY BY MACHINE)

ListBenchmark.java

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;

public class ListBenchmark {
    static long time(Runnable r) { long t = System.nanoTime(); r.run(); return (System.nanoTime() - t) / 1_000_000; }
    public static void main(String[] args) {
        int n = 50_000;
        List<Integer> al = new ArrayList<>(), ll = new LinkedList<>();
        long a1 = time(() -> { for (int i = 0; i < n; i++) al.add(0, i); });    // insert at front
        long l1 = time(() -> { for (int i = 0; i < n; i++) ll.add(0, i); });
        long a2 = time(() -> { long s = 0; for (int i = 0; i < n; i++) s += al.get(i); });   // random access
        long l2 = time(() -> { long s = 0; for (int i = 0; i < n; i++) s += ll.get(i); });
        System.out.println("insert at front : LinkedList faster? " + (l1 < a1));
        System.out.println("get(i) in loop  : ArrayList faster?  " + (a2 < l2));
    }
}
```

OUTPUT

```
insert at front : LinkedList faster? true
get(i) in loop  : ArrayList faster?  true
```

14.3.4 Vector and Stack (legacy)

`Vector` (Java 1.0) is like ArrayList but every method is `synchronized`, which makes it slower in single-threaded code; it grows by 2×. `Stack` extends Vector and adds push/pop/peek. Both are kept for backward compatibility. Use **ArrayList** instead of Vector, and **ArrayDeque** instead of Stack. For thread safety, use the concurrent collections (Module 16).

LEGACY CLASSES AND THEIR MODERN REPLACEMENT

LegacyLists.java

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Stack;
import java.util.Vector;

public class LegacyLists {
    public static void main(String[] args) {
        Vector<Integer> v = new Vector<>(2);
        v.add(1); v.add(2); v.add(3);
        System.out.println("Vector " + v + " capacity " + v.capacity() + " (doubled from 2)");

        Stack<String> st = new Stack<>();
        st.push("a"); st.push("b"); st.push("c");
        System.out.println("Stack pop " + st.pop() + ", peek " + st.peek() + ", " + st);

        Deque<String> modern = new ArrayDeque<>();          // preferred replacement
        modern.push("a"); modern.push("b"); modern.push("c");
        System.out.println("ArrayDeque pop " + modern.pop() + ", peek " + modern.peek() + ", " + modern);
    }
}
```

OUTPUT

```
Vector [1, 2, 3] capacity 4 (doubled from 2)
Stack pop c, peek b, [a, b]
ArrayDeque pop c, peek b, [b, a]
```

14.3.5 Immutable lists: List.of(), Collections.unmodifiableList()

	`List.of(...)` (Java 9)	`Collections.unmodifiableList(list)`	`List.copyOf(list)` (Java 10)
Modifiable?	No (any change throws UnsupportedOperationException)	No through the wrapper	No
Reflects changes to the original?	n/a	**Yes**: it is a read-only *view*	No: independent copy
Nulls allowed?	No (NPE)	Yes	No

THREE WAYS TO GET A LIST NOBODY CAN MODIFY — ImmutableLists.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ImmutableLists {
    public static void main(String[] args) {
        List<String> fixed = List.of("Mon", "Tue", "Wed");
        try { fixed.add("Thu"); } catch (UnsupportedOperationException e) { System.out.println("List.of is immutable");
}

        List<String> source = new ArrayList<>(List.of("x", "y"));
        List<String> view = Collections.unmodifiableList(source);
        List<String> copy = List.copyOf(source);
        source.add("z");                                  // change the original
        System.out.println("view sees change : " + view);
        System.out.println("copy does not    : " + copy);
        try { view.set(0, "q"); } catch (UnsupportedOperationException e) { System.out.println("view is read-only"); }
    }
}
```

OUTPUT

```
List.of is immutable
view sees change : [x, y, z]
copy does not    : [x, y]
view is read-only
```

14.4 Set interface (no duplicates)

A **Set** is a collection with **no duplicate elements**, modelled on the mathematical set. `add()` returns false if the element is already present. Typical uses: unique visitors, removing duplicates, fast membership tests.

14.4.1 HashSet: hashing, no order

`HashSet` is backed by a **HashMap** (elements are the keys). It uses the element's `hashCode()` to choose a bucket, giving **O(1)** average add, remove and contains. The iteration order is **not defined** and can change as the set grows. One null is allowed.

14.4.2 LinkedHashSet: insertion order

`LinkedHashSet` is a HashSet that also keeps a doubly linked list through its entries, so it iterates in **insertion order**, with almost the same O(1) speed. Perfect for "remove duplicates but keep the original order".

14.4.3 TreeSet: sorted order, NavigableSet methods

`TreeSet` keeps elements **sorted** (natural order or a Comparator) in a red-black tree: add/remove/contains are **O(log n)**. It implements `NavigableSet`, giving range and nearest-match queries. No nulls.

HASHSET, LINKEDHASHSET AND TREESET SIDE BY SIDE

SetTypes.java

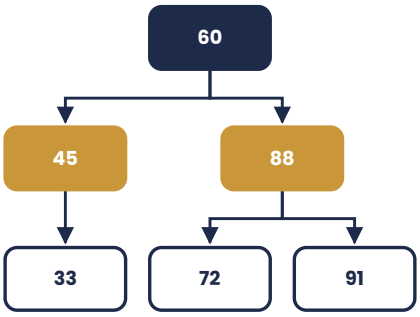
```
import java.util.*;

public class SetTypes {
    public static void main(String[] args) {
        List<String> visits = List.of("Pune", "Delhi", "Agra", "Pune", "Mysore", "Delhi", "Chennai");
        Set<String> hash = new HashSet<>(visits);
        Set<String> linked = new LinkedHashSet<>(visits);
        TreeSet<String> tree = new TreeSet<>(visits);
        System.out.println("HashSet       : " + hash + "  (order not guaranteed)");
        System.out.println("LinkedHashSet : " + linked + "  (insertion order)");
        System.out.println("TreeSet       : " + tree + "  (sorted)");
        System.out.println("add duplicate returns " + hash.add("Pune") + ", size " + hash.size());

        TreeSet<Integer> scores = new TreeSet<>(List.of(45, 72, 88, 91, 60, 33));
        System.out.println("scores        : " + scores);
        System.out.println("first / last  : " + scores.first() + " / " + scores.last());
        System.out.println("floor(70)     : " + scores.floor(70) + "   (largest <= 70)");
        System.out.println("ceiling(70)   : " + scores.ceiling(70) + "   (smallest >= 70)");
        System.out.println("higher(88)    : " + scores.higher(88) + ", lower(45) " + scores.lower(45));
        System.out.println("headSet(60)   : " + scores.headSet(60) + ", tailSet(88) " + scores.tailSet(88));
        System.out.println("subSet(40,90) : " + scores.subSet(40, 90));
        System.out.println("descending    : " + scores.descendingSet());
        System.out.println("pollFirst     : " + scores.pollFirst() + " -> " + scores);
    }
}
```

OUTPUT

```
HashSet       : [Delhi, Chennai, Pune, Agra, Mysore]  (order not guaranteed)
LinkedHashSet : [Pune, Delhi, Agra, Mysore, Chennai]  (insertion order)
TreeSet       : [Agra, Chennai, Delhi, Mysore, Pune]  (sorted)
add duplicate returns false, size 5
scores        : [33, 45, 60, 72, 88, 91]
first / last  : 33 / 91
floor(70)     : 60   (largest <= 70)
ceiling(70)   : 72   (smallest >= 70)
higher(88)    : 91, lower(45) 33
headSet(60)   : [33, 45], tailSet(88) [88, 91]
subSet(40,90) : [45, 60, 72, 88]
descending    : [91, 88, 72, 60, 45, 33]
pollFirst     : 33 -> [45, 60, 72, 88, 91]
```



A TREESET IS A BALANCED BINARY SEARCH TREE: SMALLER VALUES TO THE LEFT, LARGER TO THE RIGHT

14.4.4 Role of equals() and hashCode() in sets

HashSet decides "is this a duplicate?" in two steps: (1) compute `hashCode()` to find the bucket; (2) compare with `equals()` against elements in that bucket. If your class does not override **both** consistently (10.10.2), two "equal" objects will both be stored. TreeSet does not use equals/hashCode at all: it uses `compareTo()` / `compare()`; a result of 0 means duplicate.

DUPLICATES ARE DECIDED BY EQUALS/HASHCODE (HASH SETS) OR COMPARE (TREE SETS)

SetEquality.java

```
import java.util.*;

class BadStudent {
    int roll; String name;
    BadStudent(int r, String n) { roll = r; name = n; }
}
class GoodStudent {
    int roll; String name;
    GoodStudent(int r, String n) { roll = r; name = n; }
    @Override public boolean equals(Object o) { return o instanceof GoodStudent g && g.roll == roll; }
    @Override public int hashCode() { return Integer.hashCode(roll); }
}
public class SetEquality {
    public static void main(String[] args) {
        Set<BadStudent> bad = new HashSet<>();
        bad.add(new BadStudent(1, "Asha"));
        bad.add(new BadStudent(1, "Asha"));
        System.out.println("without equals/hashCode: size " + bad.size());

        Set<GoodStudent> good = new HashSet<>();
        good.add(new GoodStudent(1, "Asha"));
        good.add(new GoodStudent(1, "Asha (dup)"));
        System.out.println("with equals/hashCode   : size " + good.size());

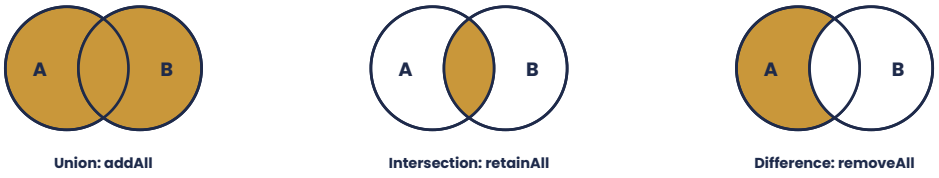
        TreeSet<String> ci = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
        ci.addAll(List.of("java", "Java", "JAVA", "sql"));
        System.out.println("TreeSet with case-insensitive comparator: " + ci);
    }
}
```

OUTPUT

```
without equals/hashCode: size 2
with equals/hashCode   : size 1
TreeSet with case-insensitive comparator: [java, sql]
```

14.4.5 Set operations: union, intersection, difference

Bulk methods turn sets into mathematical sets. They **modify** the set they are called on, so work on a copy to keep the original.



SET OPERATIONS (GOLD = RESULT)

UNION, INTERSECTION, DIFFERENCE AND SUBSET SetOperations.java

```
import java.util.*;

public class SetOperations {
    public static void main(String[] args) {
        Set<String> java = new TreeSet<>(List.of("Asha", "Ravi", "Meera", "Kiran"));
        Set<String> sql  = new TreeSet<>(List.of("Ravi", "Kiran", "Dev"));

        Set<String> union = new TreeSet<>(java);   union.addAll(sql);
        Set<String> both  = new TreeSet<>(java);   both.retainAll(sql);
        Set<String> onlyJava = new TreeSet<>(java); onlyJava.removeAll(sql);
        Set<String> symDiff = new TreeSet<>(union); symDiff.removeAll(both);

        System.out.println("Java students      : " + java);
        System.out.println("SQL students       : " + sql);
        System.out.println("either (union)     : " + union);
        System.out.println("both (intersection): " + both);
        System.out.println("only Java (diff)   : " + onlyJava);
        System.out.println("exactly one course : " + symDiff);
        System.out.println("is subset?         : " + union.containsAll(java));
    }
}
```

OUTPUT

```
Java students      : [Asha, Kiran, Meera, Ravi]
SQL students       : [Dev, Kiran, Ravi]
either (union)     : [Asha, Dev, Kiran, Meera, Ravi]
both (intersection): [Kiran, Ravi]
only Java (diff)   : [Asha, Meera]
exactly one course : [Asha, Dev, Meera]
is subset?         : true
```

14.5 Queue and Deque

14.5.1 Queue methods: offer, poll, peek (vs add, remove, element)

A **Queue** holds elements waiting to be processed, normally **FIFO** (first in, first out), like a ticket counter line. Each operation exists in two forms: one **throws** an exception on failure, the other **returns a special value** (false/null). Prefer the second form.

Operation	Throws exception	Returns special value
Insert at tail	`add(e)` (IllegalStateException if full)	`offer(e)` → false
Remove head	`remove()` (NoSuchElementException if empty)	`poll()` → null
Look at head	`element()` (NoSuchElementException)	`peek()` → null

FIFO WITH THE NULL-RETURNING METHODS

QueueDemo.java

```
import java.util.ArrayDeque;
import java.util.Queue;

public class QueueDemo {
    public static void main(String[] args) {
        Queue<String> counter = new ArrayDeque<>();
        counter.offer("Customer-1");
        counter.offer("Customer-2");
        counter.offer("Customer-3");
        System.out.println("queue: " + counter + ", next: " + counter.peek());
        while (!counter.isEmpty())
            System.out.println("serving " + counter.poll());
        System.out.println("poll on empty -> " + counter.poll());
        try {
            counter.remove();
        } catch (java.util.NoSuchElementException e) {
            System.out.println("remove on empty -> NoSuchElementException");
        }
    }
}
```

OUTPUT

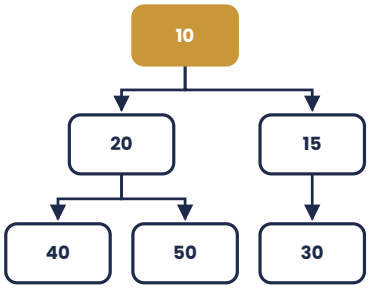
```
queue: [Customer-1, Customer-2, Customer-3], next: Customer-1
serving Customer-1
serving Customer-2
serving Customer-3
poll on empty -> null
remove on empty -> NoSuchElementException
```



FIRST IN, FIRST OUT

14.5.2 PriorityQueue (min-heap, custom comparator for max-heap)

A `PriorityQueue` always gives you the **smallest** element first (by natural order or a Comparator), regardless of insertion order. Internally it is a **binary min-heap** stored in an array: `offer` / `poll` are O(log n), `peek` is O(1). Iterating or printing it does **not** show sorted order (only the heap layout); only repeated poll() gives sorted output. Pass `Comparator.reverseOrder()` for a **max-heap**.



MIN-HEAP: EVERY PARENT ≤ ITS CHILDREN, SO THE MINIMUM IS ALWAYS AT THE ROOT. ARRAY FORM: [10, 20, 15, 40, 50, 30]

MIN-HEAP, MAX-HEAP AND A TASK SCHEDULER

PriorityQueueDemo.java

```
import java.util.*;

public class PriorityQueueDemo {
    record Task(String name, int priority) { }

    public static void main(String[] args) {
        PriorityQueue<Integer> min = new PriorityQueue<>(List.of(40, 10, 50, 20, 15, 30));
        System.out.println("internal heap array: " + min);
        System.out.print("poll order (min-heap): ");
        while (!min.isEmpty()) System.out.print(min.poll() + " ");
        System.out.println();

        PriorityQueue<Integer> max = new PriorityQueue<>(Comparator.reverseOrder());
        max.addAll(List.of(40, 10, 50, 20));
        System.out.println("max-heap peek: " + max.peek());

        PriorityQueue<Task> tasks = new PriorityQueue<>(Comparator.comparingInt(Task::priority));
        tasks.offer(new Task("write report", 3));
        tasks.offer(new Task("fix server", 1));
        tasks.offer(new Task("reply email", 2));
        while (!tasks.isEmpty()) System.out.println("  do: " + tasks.poll().name());
    }
}
```

OUTPUT

```
internal heap array: [10, 15, 30, 20, 40, 50]
poll order (min-heap): 10 15 20 30 40 50
max-heap peek: 50
  do: fix server
  do: reply email
  do: write report
```

14.5.3 Deque and ArrayDeque (as stack and queue)

A **Deque** ("deck", double-ended queue) allows insertion and removal at **both ends**: `offerFirst/offerLast`, `pollFirst/pollLast`, `peekFirst/peekLast`. `ArrayDeque` is a resizable circular array: faster than Stack (no synchronization) and than LinkedList (no node objects). It is the recommended class for both stacks and queues. No nulls.

ARRAYDEQUE AS DEQUE, STACK AND QUEUE

DequeDemo.java

```
import java.util.ArrayDeque;
import java.util.Deque;

public class DequeDemo {
    public static void main(String[] args) {
        Deque<Integer> dq = new ArrayDeque<>();
        dq.offerLast(2); dq.offerLast(3); dq.offerFirst(1); dq.offerFirst(0);
        System.out.println("deque           : " + dq);
        System.out.println("pollFirst/Last  : " + dq.pollFirst() + " / " + dq.pollLast() + " -> " + dq);

        Deque<String> stack = new ArrayDeque<>();     // LIFO: push/pop/peek work on the head
        stack.push("page1"); stack.push("page2"); stack.push("page3");
        System.out.println("back button pops: " + stack.pop() + ", then " + stack.pop());

        Deque<String> queue = new ArrayDeque<>();     // FIFO: offer at tail, poll from head
        queue.offer("job1"); queue.offer("job2");
        System.out.println("queue serves    : " + queue.poll());

        // palindrome check using both ends
        Deque<Character> chars = new ArrayDeque<>();
        for (char c : "racecar".toCharArray()) chars.add(c);
        boolean pal = true;
        while (chars.size() > 1) if (chars.pollFirst() != chars.pollLast()) pal = false;
        System.out.println("racecar palindrome? " + pal);
    }
}
```

OUTPUT

```
deque           : [0, 1, 2, 3]
pollFirst/Last  : 0 / 3 -> [1, 2]
back button pops: page3, then page2
queue serves    : job1
racecar palindrome? true
```



A DEQUE WORKS AT BOTH ENDS: STACK = PUSH/POP AT THE HEAD; QUEUE = OFFER AT TAIL, POLL AT HEAD

14.6 Map interface (key-value pairs)

A **Map** stores **key → value** pairs, like a dictionary (word → meaning) or a phone book (name → number). Each key is **unique** and maps to exactly one value; values may repeat. You look a value up by its key instead of by an index. Map does not extend Collection, but provides three collection **views**: `keySet()`, `values()` and `entrySet()`.

14.6.1 HashMap

14.6.1.1 Methods: put, get, getOrDefault, containsKey, remove, putIfAbsent, merge, compute

Method	Does	Returns
`put(k, v)`	Adds, or **replaces** the value of an existing key	previous value or null
`get(k)`	Looks up a key	value, or null if absent
`getOrDefault(k, d)`	Like get, with a fallback	value or d
`containsKey(k)` / `containsValue(v)`	Membership test (key O(1), value O(n))	boolean
`remove(k)`	Deletes the pair	removed value or null

Method	Does	Returns
`putIfAbsent(k, v)`	Puts only if the key is missing	existing value or null
`merge(k, v, fn)`	If absent put v, else put fn(old, v): ideal for counting	new value
`compute(k, fn)`, `computeIfAbsent(k, fn)`	Compute a new value from key and old value; create on first use	new value

THE HASHMAP TOOLBOX

HashMapMethods.java

```
import java.util.*;

public class HashMapMethods {
    public static void main(String[] args) {
        Map<String, Integer> stock = new HashMap<>();
        stock.put("pen", 50);
        stock.put("pencil", 80);
        stock.put("eraser", 20);
        System.out.println("put again returns old value: " + stock.put("pen", 45));
        System.out.println("get(pen)            : " + stock.get("pen"));
        System.out.println("get(stapler)        : " + stock.get("stapler"));
        System.out.println("getOrDefault        : " + stock.getOrDefault("stapler", 0));
        System.out.println("containsKey(eraser) : " + stock.containsKey("eraser") + ", containsValue(80) " +
stock.containsValue(80));
        stock.putIfAbsent("pen", 999);              // ignored: pen exists
        stock.putIfAbsent("marker", 10);            // added
        stock.merge("pen", 5, Integer::sum);        // 45 + 5
        stock.merge("ruler", 7, Integer::sum);      // absent -> 7
        stock.compute("eraser", (k, v) -> v == null ? 1 : v * 2);
        stock.remove("pencil");
        System.out.println("final (sorted view) : " + new TreeMap<>(stock));

        Map<String, List<String>> byCity = new HashMap<>();
        byCity.computeIfAbsent("Pune", k -> new ArrayList<>()).add("Asha");   // create list on first use
        byCity.computeIfAbsent("Pune", k -> new ArrayList<>()).add("Ravi");
        byCity.computeIfAbsent("Delhi", k -> new ArrayList<>()).add("Meera");
        System.out.println("grouping            : " + new TreeMap<>(byCity));
    }
}
```

OUTPUT

```
put again returns old value: 50
get(pen)            : 45
get(stapler)        : null
getOrDefault        : 0
containsKey(eraser) : true, containsValue(80) true
final (sorted view) : {eraser=40, marker=10, pen=50, ruler=7}
grouping            : {Delhi=[Meera], Pune=[Asha, Ravi]}
```

14.6.1.2 Iterating with keySet, values, entrySet

THREE VIEWS OF A MAP

MapIteration.java

```
import java.util.LinkedHashMap;
import java.util.Map;

public class MapIteration {
    public static void main(String[] args) {
        Map<String, Double> marks = new LinkedHashMap<>();
        marks.put("Asha", 91.5); marks.put("Ravi", 78.0); marks.put("Meera", 85.0);

        System.out.println("keys   : " + marks.keySet());
        System.out.println("values : " + marks.values());
        for (Map.Entry<String, Double> e : marks.entrySet())       // best: key and value together
            System.out.println("  " + e.getKey() + " -> " + e.getValue());

        for (Map.Entry<String, Double> e : marks.entrySet())
            e.setValue(e.getValue() + 2);                          // update through the entry
        marks.forEach((name, m) -> System.out.print(name + "=" + m + " "));
        System.out.println();
        marks.entrySet().removeIf(e -> e.getValue() < 85);         // remove via the view
        System.out.println("after removeIf: " + marks);
    }
}
```

OUTPUT

```
keys   : [Asha, Ravi, Meera]
values : [91.5, 78.0, 85.0]
  Asha -> 91.5
  Ravi -> 78.0
  Meera -> 85.0
Asha=93.5 Ravi=80.0 Meera=87.0
after removeIf: {Asha=93.5, Meera=87.0}
```

TIP

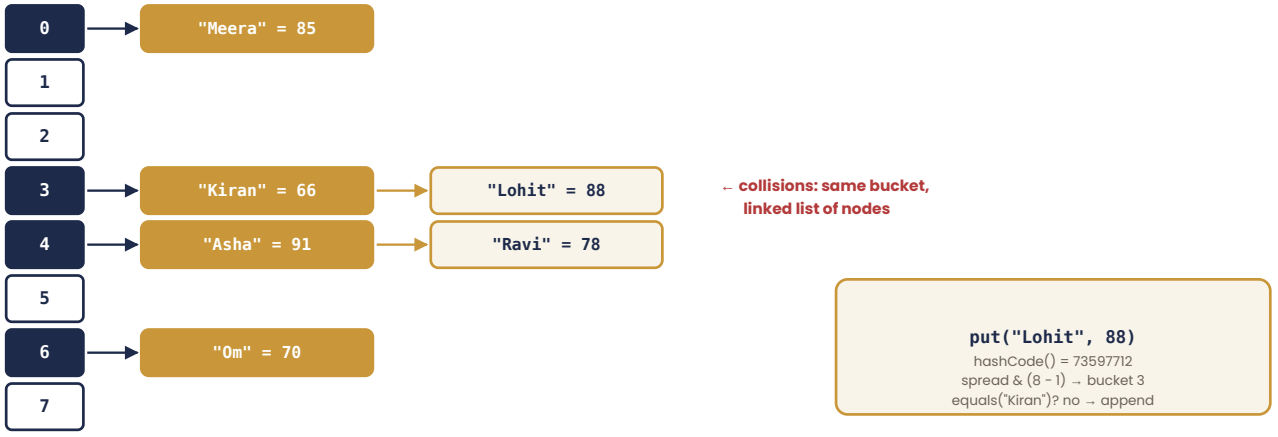
Iterate over `entrySet()` when you need both key and value. Looping over `keySet()` and calling `get(key)` each time does a second lookup per element.

14.6.1.3 Internal working: buckets, hashing, collisions, load factor, rehashing

A HashMap is an **array of buckets** (the "table", default size 16). To store a pair:

1. Compute `key.hashCode()`, then spread its bits: `h ^ (h >>> 16)`.
2. Find the bucket index: `index = hash & (n - 1)`, where n is the table size (always a power of two, so this equals hash % n).
3. If the bucket is empty, place a new **node** (hash, key, value, next) there.
4. If not, it is a **collision**: walk the bucket's linked list comparing keys with `equals()`. If an equal key is found, **replace** its value; otherwise **append** a new node.

`get(key)` repeats steps 1–2 and searches only that one bucket, which is why lookups are O(1) on average. When the number of entries exceeds **capacity × load factor** (16 × 0.75 = 12 by default), the table **doubles** and every entry is redistributed (**rehashing**), keeping the buckets short.



A HASHMAP WITH 8 BUCKETS: KEYS SPREAD ACROSS BUCKETS; TWO KEYS LANDING IN BUCKET 3 FORM A CHAIN

COMPUTING BUCKET INDEXES LIKE HASHMAP DOES

HashingSteps.java

```
import java.util.HashMap;
import java.util.Map;

public class HashingSteps {
    static int bucket(Object key, int n) {
        int h = key.hashCode();
        int spread = h ^ (h >>> 16);              // same as HashMap.hash()
        return spread & (n - 1);
    }
    public static void main(String[] args) {
        String[] keys = {"Asha", "Ravi", "Meera", "Kiran", "Aa", "BB"};
        for (String k : keys)
            System.out.printf("%-6s hashCode %10d -> bucket %2d of 16%n", k, k.hashCode(), bucket(k, 16));
        System.out.println("\"Aa\" and \"BB\" collide: same hashCode " + "Aa".hashCode() + " but equals() = " +
"Aa".equals("BB"));

        Map<String, Integer> m = new HashMap<>();
        m.put("Aa", 1); m.put("BB", 2);           // both stored in the same bucket
        System.out.println("both kept: " + m.get("Aa") + ", " + m.get("BB"));
        System.out.println("resize happens when size > 16 * 0.75 = " + (int) (16 * 0.75) + " entries");
    }
}
```

OUTPUT

```
Asha   hashCode    2050251 -> bucket  4 of 16
Ravi   hashCode    2539842 -> bucket  4 of 16
Meera  hashCode   74220700 -> bucket  0 of 16
Kiran  hashCode   72504801 -> bucket  3 of 16
Aa     hashCode       2112 -> bucket  0 of 16
BB     hashCode       2112 -> bucket  0 of 16
"Aa" and "BB" collide: same hashCode 2112 but equals() = false
both kept: 1, 2
resize happens when size > 16 * 0.75 = 12 entries
```

Term	Meaning	Default
Capacity	Number of buckets (power of 2)	16
Load factor	How full the table may get before resizing	0.75
Threshold	capacity × load factor	12
Rehashing	Doubling the table and re-placing every entry	When size > threshold
TREEIFY_THRESHOLD	Chain length that turns a bucket into a tree	8 (and table ≥ 64)

14.6.1.4 Treeification of buckets (Java 8)

If many keys collide into one bucket (poor hashCode, or a deliberate attack), the chain becomes long and lookups degrade to O(n). Since Java 8, when one bucket holds more than **8** nodes (and the table has at least 64 buckets), that

bucket is converted into a **red-black tree**, making worst-case lookup **O(log n)**. If it shrinks below 6 nodes it turns back into a list.



TREEIFICATION PROTECTS HASHMAP AGAINST LONG COLLISION CHAINS

Average HashMap complexity: put/get/remove **O(1)**; worst case O(log n) since Java 8. Requirements: keys must have consistent `equals()` / `hashCode()`, and should be **immutable**. If a key's fields change after insertion, its hash changes and the entry becomes unreachable.

14.6.2 LinkedHashMap (insertion/access order, LRU cache)

`LinkedHashMap` is a HashMap with a doubly linked list through its entries. By default it iterates in **insertion order**. Constructed with `accessOrder = true`, it moves each entry to the end whenever it is read, so the **least recently used** entry is first. Overriding `removeEldestEntry` turns it into an **LRU cache** in a few lines.

INSERTION ORDER AND AN LRU CACHE — LinkedHashMapDemo.java

```
import java.util.LinkedHashMap;
import java.util.Map;

class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LRUCache(int capacity) {
        super(16, 0.75f, true);                  // true = access order
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;                // evict least recently used
    }
}
public class LinkedHashMapDemo {
    public static void main(String[] args) {
        Map<String, Integer> ordered = new LinkedHashMap<>();
        ordered.put("zeta", 1); ordered.put("alpha", 2); ordered.put("mid", 3);
        System.out.println("insertion order: " + ordered);

        LRUCache<String, String> cache = new LRUCache<>(3);
        cache.put("A", "page A"); cache.put("B", "page B"); cache.put("C", "page C");
        cache.get("A");                           // A becomes most recently used
        cache.put("D", "page D");                 // evicts B (least recently used)
        System.out.println("LRU cache keys : " + cache.keySet());
    }
}
```

OUTPUT

```
insertion order: {zeta=1, alpha=2, mid=3}
LRU cache keys : [C, A, D]
```

14.6.3 TreeMap (sorted keys, NavigableMap methods)

`TreeMap` keeps keys **sorted** in a red-black tree: O(log n) operations. Like TreeSet, it offers navigation: `firstKey`, `lastKey`, `floorKey`, `ceilingKey`, `headMap`, `tailMap`, `subMap`, `descendingMap`, `pollFirstEntry`. Great for ranges: grade boundaries, time slots, leaderboards.

RANGE LOOKUPS WITH FLOORENTRY

TreeMapDemo.java

```
import java.util.Map;
import java.util.TreeMap;

public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap<Integer, String> grades = new TreeMap<>();
        grades.put(90, "A"); grades.put(75, "B"); grades.put(60, "C"); grades.put(40, "D"); grades.put(0, "F");
        int[] scores = {95, 76, 59, 40, 12};
        for (int s : scores)
            System.out.println(s + " -> grade " + grades.floorEntry(s).getValue());   // largest key <= s

        TreeMap<String, Integer> pop = new TreeMap<>(Map.of("Pune", 7, "Agra", 2, "Delhi", 32, "Mysore", 1, "Chennai",
11));
        System.out.println("sorted     : " + pop);
        System.out.println("first/last : " + pop.firstKey() + " / " + pop.lastKey());
        System.out.println("headMap(D) : " + pop.headMap("D"));
        System.out.println("tailMap(M) : " + pop.tailMap("M"));
        System.out.println("ceilingKey(E): " + pop.ceilingKey("E"));
        System.out.println("descending : " + pop.descendingMap());
    }
}
```

OUTPUT

```
95 -> grade A
76 -> grade B
59 -> grade D
40 -> grade D
12 -> grade F
sorted     : {Agra=2, Chennai=11, Delhi=32, Mysore=1, Pune=7}
first/last : Agra / Pune
headMap(D) : {Agra=2, Chennai=11}
tailMap(M) : {Mysore=1, Pune=7}
ceilingKey(E): Mysore
descending : {Pune=7, Mysore=1, Delhi=32, Chennai=11, Agra=2}
```

14.6.4 Hashtable (legacy)

`Hashtable` (Java 1.0) is a synchronized hash map: every method locks the whole table, which is slow. It allows **no null keys or values**. Like Vector, it is legacy: use `HashMap` for single-threaded code and `ConcurrentHashMap` for multithreaded code.

14.6.5 Comparison of HashMap, LinkedHashMap, TreeMap, Hashtable

	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	None	Insertion (or access)	Sorted by key	None
get/put	O(1)	O(1)	O(log n)	O(1), synchronized
null key	One allowed	One allowed	No (needs compare)	No
null values	Yes	Yes	Yes	No
Thread-safe	No	No	No	Yes (slow)
Internals	Bucket array	Buckets + linked list	Red-black tree	Bucket array
Use for	Default choice	Predictable order, LRU caches	Sorted keys, ranges	Legacy only

SAME DATA, DIFFERENT ORDERS AND NULL RULES MapComparison.java

```
import java.util.*;

public class MapComparison {
    public static void main(String[] args) {
        String[] keys = {"banana", "apple", "cherry", "date"};
        Map<String, Integer> hm = new HashMap<>(), lhm = new LinkedHashMap<>(), tm = new TreeMap<>();
        for (int i = 0; i < keys.length; i++) { hm.put(keys[i], i); lhm.put(keys[i], i); tm.put(keys[i], i); }
        System.out.println("HashMap       : " + hm);
        System.out.println("LinkedHashMap : " + lhm);
        System.out.println("TreeMap       : " + tm);
        hm.put(null, -1);
        System.out.println("HashMap null key ok: " + hm.get(null));
        try { new Hashtable<String, Integer>().put(null, 1); }
        catch (NullPointerException e) { System.out.println("Hashtable null key: NullPointerException"); }
        try { new TreeMap<String, Integer>().put(null, 1); }
        catch (NullPointerException e) { System.out.println("TreeMap null key  : NullPointerException"); }
    }
}
```

OUTPUT

```
HashMap       : {banana=0, date=3, apple=1, cherry=2}
LinkedHashMap : {banana=0, apple=1, cherry=2, date=3}
TreeMap       : {apple=1, banana=0, cherry=2, date=3}
HashMap null key ok: -1
Hashtable null key: NullPointerException
TreeMap null key  : NullPointerException
```

14.6.6 Immutable maps: Map.of()

`Map.of(k1, v1, …)` creates an immutable map of up to 10 pairs; `Map.ofEntries(Map.entry(k, v), …)` has no limit; `Map.copyOf(map)` makes an immutable copy. They reject nulls and duplicate keys, and their iteration order is unspecified.

CREATING CONSTANT MAPS ImmutableMaps.java

```
import java.util.Map;

public class ImmutableMaps {
    public static void main(String[] args) {
        Map<String, Integer> days = Map.of("Mon", 1, "Tue", 2, "Wed", 3);
        Map<String, String> capitals = Map.ofEntries(Map.entry("India", "New Delhi"), Map.entry("Japan", "Tokyo"));
        System.out.println(days.get("Tue") + ", " + capitals.get("Japan"));
        try { days.put("Thu", 4); } catch (UnsupportedOperationException e) { System.out.println("immutable: cannot
put"); }
        try { Map.of("A", 1, "A", 2); } catch (IllegalArgumentException e) { System.out.println("duplicate key: " +
e.getMessage()); }
    }
}
```

OUTPUT

```
2, Tokyo
immutable: cannot put
duplicate key: duplicate key: A
```

14.7 Sorting and comparing objects

`Collections.sort(list)`, `list.sort(…)`, TreeSet, TreeMap and PriorityQueue all need to know how to **order** two objects. Java provides two ways to define that order.

14.7.1 Comparable and compareTo() (natural ordering)

A class implements `Comparable<T>` to define its **natural order**, the one obvious default ordering (String alphabetical, Integer numeric, LocalDate chronological). `a.compareTo(b)` returns a **negative** number if a comes first, **0** if equal, **positive** if a comes after b. Use `Integer.compare` / `Double.compare` instead of subtraction, which can overflow.

14.7.2 Comparator and compare() (custom ordering)

A `Comparator<T>` is a separate object that defines an **alternative** ordering, without changing the class: sort students by name, by marks descending, by city then name. You can have as many comparators as you like, and you can sort classes you cannot modify. It is a functional interface, so a lambda works.

14.7.3 Comparator.comparing(), thenComparing(), reversed()

Java 8 added factory and combinator methods, so comparators read like English: `Comparator.comparing(Student::getCity).thenComparing(Student::getName)`, `comparingDouble(Student::getMarks).reversed()`, `nullsFirst(...)`, `naturalOrder()`, `reverseOrder()`.

NATURAL ORDER, CUSTOM COMPARATORS AND CHAINED COMPARATORS SortingDemo.java

```
import java.util.*;

class Student implements Comparable<Student> {
    final int id; final String name; final String city; final double marks;
    Student(int id, String name, String city, double marks) { this.id = id; this.name = name; this.city = city;
this.marks = marks; }
    int getId() { return id; } String getName() { return name; } String getCity() { return city; } double getMarks() {
return marks; }

    @Override public int compareTo(Student o) { return Integer.compare(id, o.id); }   // natural order: by id
    @Override public String toString() { return id + ":" + name + "(" + city + "," + marks + ")"; }
}
public class SortingDemo {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>(List.of(
            new Student(3, "Ravi", "Pune", 78), new Student(1, "Meera", "Delhi", 92),
            new Student(4, "Asha", "Pune", 92), new Student(2, "Kiran", "Delhi", 65)));

        Collections.sort(list);                                          // Comparable
        System.out.println("by id (natural): " + list);

        list.sort(new Comparator<Student>() {                            // Comparator, old style
            public int compare(Student a, Student b) { return a.name.compareTo(b.name); }
        });
        System.out.println("by name        : " + list);

        list.sort(Comparator.comparingDouble(Student::getMarks).reversed()      // marks high -> low
                            .thenComparing(Student::getName));                  // tie: by name
        System.out.println("by marks desc  : " + list);

        list.sort(Comparator.comparing(Student::getCity).thenComparing(Student::getMarks, Comparator.reverseOrder()));
        System.out.println("city, marks    : " + list);

        TreeSet<Student> byMarks = new TreeSet<>(Comparator.comparingDouble(Student::getMarks));
        byMarks.addAll(list);
        System.out.println("TreeSet by marks (92 duplicate dropped!): " + byMarks);
    }
}
```

OUTPUT

```
by id (natural): [1:Meera(Delhi,92.0), 2:Kiran(Delhi,65.0), 3:Ravi(Pune,78.0), 4:Asha(Pune,92.0)]
by name        : [4:Asha(Pune,92.0), 2:Kiran(Delhi,65.0), 1:Meera(Delhi,92.0), 3:Ravi(Pune,78.0)]
by marks desc  : [4:Asha(Pune,92.0), 1:Meera(Delhi,92.0), 3:Ravi(Pune,78.0), 2:Kiran(Delhi,65.0)]
city, marks    : [1:Meera(Delhi,92.0), 2:Kiran(Delhi,65.0), 4:Asha(Pune,92.0), 3:Ravi(Pune,78.0)]
TreeSet by marks (92 duplicate dropped!): [2:Kiran(Delhi,65.0), 3:Ravi(Pune,78.0), 1:Meera(Delhi,92.0)]
```

WATCH OUT

A TreeSet/TreeMap treats compare() == 0 as "the same element". In the last line Meera and Asha both have 92, so one of them is silently dropped. Make comparators used by sorted collections break ties (e.g. `.thenComparing(Student::getId)`).

14.7.4 Comparable vs Comparator

	Comparable	Comparator
Package	`java.lang`	`java.util`

	Comparable	Comparator
Method	`int compareTo(T other)`	`int compare(T a, T b)`
Where defined	Inside the class being sorted	In a separate class / lambda
How many orders	One (natural)	Any number
Modify the class?	Yes	No, so works with library classes
Used by	`Collections.sort(list)`, TreeSet()	`list.sort(cmp)`, `new TreeSet<>(cmp)`

14.8 Collections utility class

14.8.1 sort, reverse, shuffle, max, min, frequency, binarySearch

ALGORITHMS IN JAVA.UTIL.COLLECTIONS

CollectionsUtility.java

```
import java.util.*;

public class CollectionsUtility {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>(List.of(42, 7, 19, 7, 88, 3, 7));
        Collections.sort(nums);
        System.out.println("sort          : " + nums);
        System.out.println("binarySearch 19: index " + Collections.binarySearch(nums, 19) + " (list must be sorted)");
        Collections.reverse(nums);
        System.out.println("reverse       : " + nums);
        Collections.shuffle(nums, new Random(1));               // seeded for repeatable output
        System.out.println("shuffle       : " + nums);
        System.out.println("max / min     : " + Collections.max(nums) + " / " + Collections.min(nums));
        System.out.println("frequency(7)  : " + Collections.frequency(nums, 7));
        Collections.swap(nums, 0, 1);
        Collections.rotate(nums, 2);
        System.out.println("swap + rotate : " + nums);
        System.out.println("nCopies       : " + Collections.nCopies(3, "ab"));
        List<String> names = new ArrayList<>(List.of("ravi", "Asha", "meera"));
        Collections.sort(names, String.CASE_INSENSITIVE_ORDER);
        System.out.println("case-insens.  : " + names + ", max by length " + Collections.max(names,
Comparator.comparingInt(String::length)));
        Collections.addAll(names, "x", "y");
        System.out.println("addAll        : " + names + ", disjoint with [z]? " + Collections.disjoint(names,
List.of("z")));
    }
}
```

OUTPUT

```
sort          : [3, 7, 7, 7, 19, 42, 88]
binarySearch 19: index 4 (list must be sorted)
reverse       : [88, 42, 19, 7, 7, 7, 3]
shuffle       : [7, 88, 7, 42, 19, 3, 7]
max / min     : 88 / 3
frequency(7)  : 3
swap + rotate : [3, 7, 88, 7, 7, 42, 19]
nCopies       : [ab, ab, ab]
case-insens.  : [Asha, meera, ravi], max by length meera
addAll        : [Asha, meera, ravi, x, y], disjoint with [z]? true
```

14.8.2 Synchronized and unmodifiable wrappers

`Collections.unmodifiableList/Set/Map(c)` return a **read-only view**: any modification through it throws UnsupportedOperationException. Use it to expose internal collections from a class safely. `Collections.synchronizedList/Set/Map(c)` return a **thread-safe view** where every method locks the collection; you must still lock manually when iterating. Modern code generally prefers the concurrent collections (Module 16).

READ-ONLY AND SYNCHRONIZED VIEWS

Wrappers.java

```
import java.util.*;

class Classroom {
    private final List<String> students = new ArrayList<>();
    void enroll(String s) { students.add(s); }
    List<String> getStudents() { return Collections.unmodifiableList(students); }   // safe exposure
}
public class Wrappers {
    public static void main(String[] args) {
        Classroom c = new Classroom();
        c.enroll("Asha"); c.enroll("Ravi");
        List<String> view = c.getStudents();
        try { view.clear(); } catch (UnsupportedOperationException e) { System.out.println("outsiders cannot modify: "
+ view); }

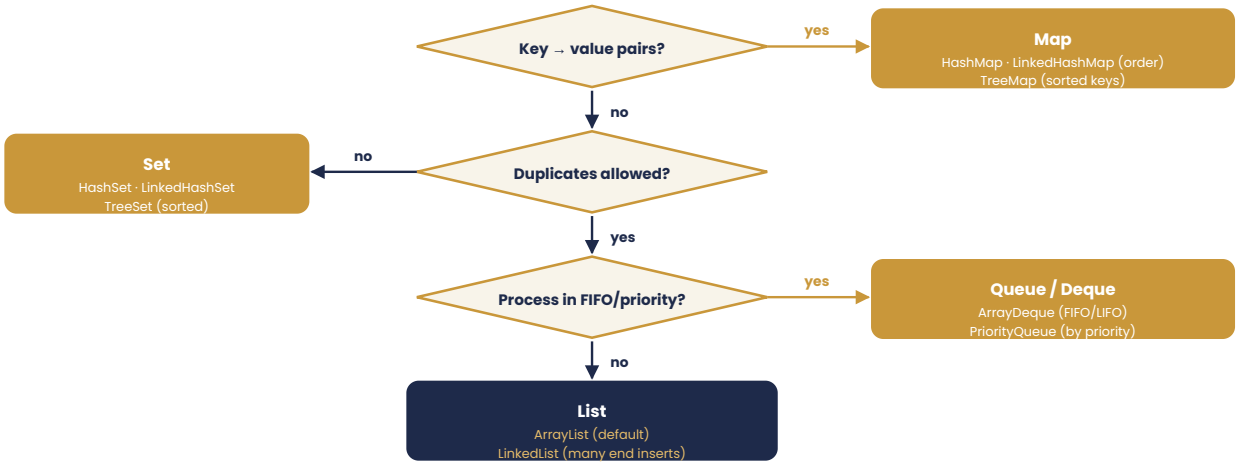
        List<Integer> shared = Collections.synchronizedList(new ArrayList<>());
        shared.add(1); shared.add(2);
        synchronized (shared) {                          // required while iterating
            for (int x : shared) System.out.print(x + " ");
        }
        System.out.println("<- iterated under lock");
    }
}
```

OUTPUT

```
outsiders cannot modify: [Asha, Ravi]
1 2 <- iterated under lock
```

14.9 Choosing the right collection

14.9.1 Decision guide by ordering, duplicates, lookup speed



CHOOSING A COLLECTION IN THREE QUESTIONS

Need	Use
Ordered list, index access, duplicates (default)	`ArrayList`
Unique elements, fastest membership test	`HashSet`
Unique elements in insertion order	`LinkedHashSet`
Unique elements always sorted / range queries	`TreeSet`
Key lookup, fastest	`HashMap`
Key lookup with insertion order or LRU	`LinkedHashMap`
Sorted keys, floor/ceiling lookups	`TreeMap`

Need	Use
Stack or FIFO queue	`ArrayDeque`
Always take the smallest/largest/most urgent next	`PriorityQueue`
Enum keys / enum elements	`EnumMap` / `EnumSet`
Many threads	`ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`

14.9.2 Big-O summary table

Collection	add	remove	get / contains	Iteration order
ArrayList	O(1)* end, O(n) middle	O(n)	get O(1), contains O(n)	Index
LinkedList	O(1) ends	O(1) ends	get O(n), contains O(n)	Index
ArrayDeque	O(1)* ends	O(1) ends	peek O(1)	Head → tail
HashSet / HashMap	O(1) avg	O(1) avg	O(1) avg	Unspecified
LinkedHashSet / LinkedHashMap	O(1) avg	O(1) avg	O(1) avg	Insertion (or access)
TreeSet / TreeMap	O(log n)	O(log n)	O(log n)	Sorted
PriorityQueue	O(log n)	poll O(log n)	peek O(1), contains O(n)	Heap (poll gives sorted)

> **NOTE**
> * amortised: occasionally O(n) when the internal array is resized.

14.10 Data structures with collections

14.10.1 Stack, queue and linked list implementations from scratch

Building the structures yourself shows what the library classes do inside. Below: a generic **stack** on an array that grows, a **queue** as a circular array, and a singly **linked list**.

A STACK, A CIRCULAR QUEUE AND A LINKED LIST WRITTEN BY HAND — FromScratch.java

```
import java.util.NoSuchElementException;

class MyStack<T> {
    private Object[] data = new Object[2];
    private int size;
    void push(T x) {
        if (size == data.length) data = java.util.Arrays.copyOf(data, size * 2);   // grow
        data[size++] = x;
    }
    @SuppressWarnings("unchecked")
    T pop() {
        if (size == 0) throw new RuntimeException("stack underflow");
        T x = (T) data[--size];
        data[size] = null;                          // help GC
        return x;
    }
    boolean isEmpty() { return size == 0; }
}

class MyQueue<T> {                                  // circular array
    private final Object[] data;
    private int head, tail, size;
    MyQueue(int capacity) { data = new Object[capacity]; }
    boolean offer(T x) {
        if (size == data.length) return false;
        data[tail] = x; tail = (tail + 1) % data.length; size++;
        return true;
    }
    @SuppressWarnings("unchecked")
```

```
    T poll() {
        if (size == 0) return null;
        T x = (T) data[head]; data[head] = null; head = (head + 1) % data.length; size--;
        return x;
    }
}

class MyLinkedList<T> {
    private static class Node<T> { T val; Node<T> next; Node(T v) { val = v; } }
    private Node<T> head;
    private int size;
    void addFirst(T x) { Node<T> n = new Node<>(x); n.next = head; head = n; size++; }
    void addLast(T x) {
        Node<T> n = new Node<>(x);
        if (head == null) { head = n; } else { Node<T> c = head; while (c.next != null) c = c.next; c.next = n; }
        size++;
    }
    boolean remove(T x) {
        if (head == null) return false;
        if (head.val.equals(x)) { head = head.next; size--; return true; }
        for (Node<T> c = head; c.next != null; c = c.next)
            if (c.next.val.equals(x)) { c.next = c.next.next; size--; return true; }
        return false;
    }
    void reverse() {
        Node<T> prev = null, cur = head;
        while (cur != null) { Node<T> nx = cur.next; cur.next = prev; prev = cur; cur = nx; }
        head = prev;
    }
    @Override public String toString() {
        StringBuilder sb = new StringBuilder("[");
        for (Node<T> c = head; c != null; c = c.next) sb.append(c.val).append(c.next != null ? " -> " : "");
        return sb.append("] size=").append(size).toString();
    }
}

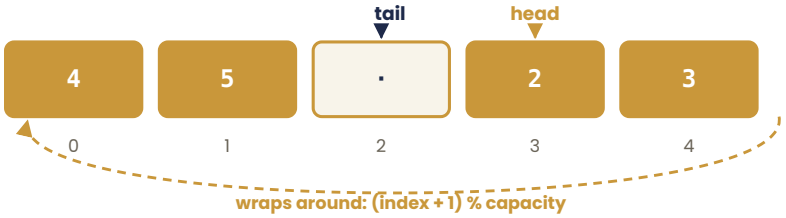
public class FromScratch {
    public static void main(String[] args) {
        MyStack<String> st = new MyStack<>();
        for (String s : "to be or not".split(" ")) st.push(s);
        System.out.print("stack pops: ");
        while (!st.isEmpty()) System.out.print(st.pop() + " ");
        System.out.println();

        MyQueue<Integer> q = new MyQueue<>(3);
        System.out.println("offer 1,2,3,4: " + q.offer(1) + " " + q.offer(2) + " " + q.offer(3) + " " + q.offer(4));
        System.out.println("poll " + q.poll() + ", offer 5 " + q.offer(5) + ", poll " + q.poll() + " " + q.poll() + " "
+ q.poll());

        MyLinkedList<Integer> ll = new MyLinkedList<>();
        ll.addLast(2); ll.addLast(3); ll.addFirst(1); ll.addLast(4);
        System.out.println("list     : " + ll);
        ll.remove(3);
        ll.reverse();
        System.out.println("reversed : " + ll);
    }
}
```

OUTPUT

```
stack pops: not or be to
offer 1,2,3,4: true true true false
poll 1, offer 5 true, poll 2 3 5
list     : [1 -> 2 -> 3 -> 4] size=4
reversed : [4 -> 2 -> 1] size=3
```



CIRCULAR QUEUE: HEAD AND TAIL MOVE FORWARD AND WRAP TO THE START, SO NO ELEMENT IS EVER SHIFTED

14.10.2 Problems: frequency count, two-sum, duplicate detection, LRU cache, top-K elements

Most interview problems on collections are solved by choosing the right structure. Each solution below is O(n) or O(n log k), compared with O(n²) brute force.

Problem	Key idea	Structure	Time
Frequency count	`map.merge(x, 1, Integer::sum)`	HashMap / TreeMap	O(n)
Two-sum	For each x, check if target - x was seen before	HashMap value → index	O(n)
First duplicate	`if (!seen.add(x))` → duplicate	HashSet	O(n)
LRU cache	Access-ordered map with eviction	LinkedHashMap	O(1) per op
Top-K largest	Keep a min-heap of size k; pop when it exceeds k	PriorityQueue	O(n log k)

FIVE CLASSIC COLLECTION PROBLEMS CollectionProblems.java

```
import java.util.*;

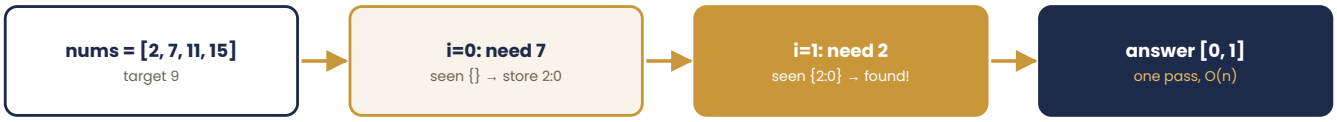
public class CollectionProblems {
    static Map<String, Integer> wordFrequency(String text) {
        Map<String, Integer> freq = new TreeMap<>();
        for (String w : text.toLowerCase().split("\\W+")) freq.merge(w, 1, Integer::sum);
        return freq;
    }
    static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();              // value -> index
        for (int i = 0; i < nums.length; i++) {
            Integer j = seen.get(target - nums[i]);
            if (j != null) return new int[] {j, i};
            seen.put(nums[i], i);
        }
        return new int[0];
    }
    static Integer firstDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) if (!seen.add(x)) return x;           // add() false = already present
        return null;
    }
    static List<Integer> topK(int[] nums, int k) {
        PriorityQueue<Integer> heap = new PriorityQueue<>();      // min-heap of the k largest so far
        for (int x : nums) {
            heap.offer(x);
            if (heap.size() > k) heap.poll();                     // drop the smallest
        }
        List<Integer> result = new ArrayList<>(heap);
        result.sort(Comparator.reverseOrder());
        return result;
    }
    public static void main(String[] args) {
        System.out.println("frequency  : " + wordFrequency("Java is fun and Java is powerful, fun fun!"));
        System.out.println("two-sum 9  : " + Arrays.toString(twoSum(new int[] {2, 7, 11, 15}, 9)));
        System.out.println("two-sum 26 : " + Arrays.toString(twoSum(new int[] {2, 7, 11, 15}, 26)));
        System.out.println("first dup  : " + firstDuplicate(new int[] {3, 8, 1, 9, 8, 3}));
        System.out.println("top-3      : " + topK(new int[] {5, 12, 3, 99, 41, 7, 64, 8}, 3));

        LinkedHashMap<Integer, String> lru = new LinkedHashMap<>(16, 0.75f, true) {
            protected boolean removeEldestEntry(Map.Entry<Integer, String> e) { return size() > 2; }
        };
```

```
        lru.put(1, "one"); lru.put(2, "two"); lru.get(1); lru.put(3, "three");
        System.out.println("LRU (cap 2): " + lru.keySet() + "  (2 was evicted)");
    }
}
```

OUTPUT

```
frequency  : {and=1, fun=3, is=2, java=2, powerful=1}
two-sum 9  : [0, 1]
two-sum 26 : [2, 3]
first dup  : 8
top-3      : [99, 64, 41]
LRU (cap 2): [1, 3]  (2 was evicted)
```



TWO-SUM WITH A HASHMAP

PRACTICE CHECKPOINT

A **Student Management System** that stores students in an ArrayList, indexes them by ID in a HashMap, and sorts them by marks and by name.

SAMPLE SOLUTION StudentManagementSystem.java

```
import java.util.*;

class Student {
    private final int id;
    private final String name;
    private double marks;
    Student(int id, String name, double marks) { this.id = id; this.name = name; this.marks = marks; }
    int getId() { return id; }
    String getName() { return name; }
    double getMarks() { return marks; }
    void setMarks(double m) { marks = m; }
    @Override public String toString() { return String.format("%-4d %-8s %6.1f", id, name, marks); }
}

class StudentManager {
    private final List<Student> students = new ArrayList<>();       // ordered storage
    private final Map<Integer, Student> byId = new HashMap<>();     // O(1) lookup index

    boolean add(Student s) {
        if (byId.containsKey(s.getId())) return false;              // reject duplicate id
        students.add(s);
        byId.put(s.getId(), s);
        return true;
    }
    Optional<Student> find(int id) { return Optional.ofNullable(byId.get(id)); }
    boolean remove(int id) {
        Student s = byId.remove(id);
        return s != null && students.remove(s);
    }
    List<Student> sortedByMarks() {
        List<Student> copy = new ArrayList<>(students);
        copy.sort(Comparator.comparingDouble(Student::getMarks).reversed().thenComparing(Student::getName));
        return copy;
    }
    List<Student> sortedByName() {
        List<Student> copy = new ArrayList<>(students);
        copy.sort(Comparator.comparing(Student::getName));
        return copy;
    }
    void print(String title, List<Student> list) {
        System.out.println("== " + title + " ==");
        list.forEach(System.out::println);
    }
```

```
    List<Student> all() { return Collections.unmodifiableList(students); }
}

public class StudentManagementSystem {
    public static void main(String[] args) {
        StudentManager sm = new StudentManager();
        sm.add(new Student(104, "Ravi", 78.5));
        sm.add(new Student(101, "Meera", 92.0));
        sm.add(new Student(103, "Asha", 92.0));
        sm.add(new Student(102, "Kiran", 65.0));
        System.out.println("duplicate id 101 added? " + sm.add(new Student(101, "Fake", 10)));

        sm.print("Insertion order", sm.all());
        sm.print("Sorted by marks (high to low)", sm.sortedByMarks());
        sm.print("Sorted by name", sm.sortedByName());

        System.out.println("find 103 -> " + sm.find(103).map(Student::getName).orElse("not found"));
        System.out.println("find 999 -> " + sm.find(999).map(Student::getName).orElse("not found"));
        sm.find(102).ifPresent(s -> s.setMarks(70));
        System.out.println("remove 104 -> " + sm.remove(104));
        sm.print("After update and remove", sm.all());
    }
}
```

OUTPUT

```
duplicate id 101 added? false
== Insertion order ==
104  Ravi       78.5
101  Meera      92.0
103  Asha       92.0
102  Kiran      65.0
== Sorted by marks (high to low) ==
103  Asha       92.0
101  Meera      92.0
104  Ravi       78.5
102  Kiran      65.0
== Sorted by name ==
103  Asha       92.0
102  Kiran      65.0
101  Meera      92.0
104  Ravi       78.5
find 103 -> Asha
find 999 -> not found
remove 104 -> true
== After update and remove ==
101  Meera      92.0
103  Asha       92.0
102  Kiran      70.0
```

MODULE 15 · INTERMEDIATE

File handling and I/O

Minimum duration **3 days** | Level **Intermediate**

Java I/O reads and writes data through streams; this module covers files, streams, NIO and saving objects to disk.

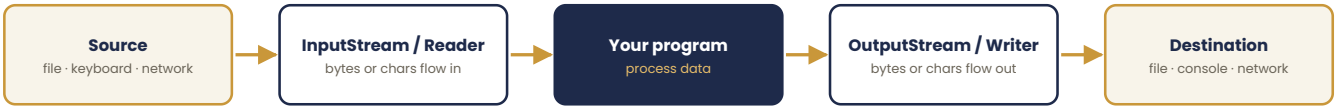
IN THIS MODULE

15.1 I/O fundamentals
15.2 The File class
15.3 Byte streams
15.4 Character streams
15.5 Serialization
15.6 NIO and NIO.2 (java.nio.file)
15.7 Working with common formats

15.1 I/O fundamentals

15.1.1 What a stream is

A **stream** is a one-way flow of data between your program and a **source** or **destination**: a file, the keyboard, the console, a network socket, or a byte array in memory. You read from an **input stream** one piece after another, and you write to an **output stream** one piece after another, without caring where the data physically lives. (These I/O streams are unrelated to the Stream API of Module 17.)



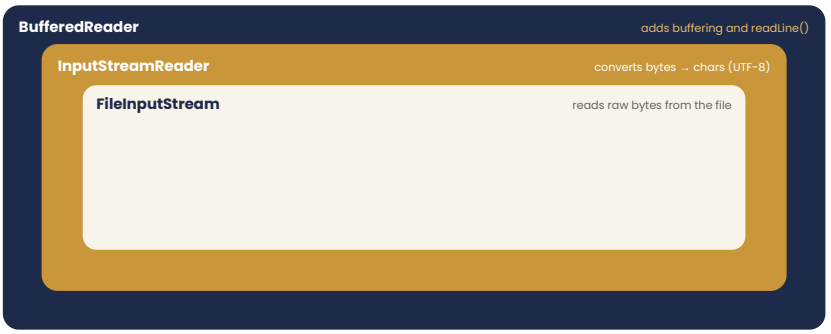
DATA FLOWS IN THROUGH INPUT STREAMS AND OUT THROUGH OUTPUT STREAMS

15.1.2 Byte streams vs character streams

	Byte streams	Character streams
Unit	8-bit bytes	16-bit chars (text)
Base classes	`InputStream`, `OutputStream`	`Reader`, `Writer`
Encoding	None: raw data	Converts bytes ↔ characters using a charset (UTF-8)
Use for	Images, audio, video, PDFs, zip, any binary file	Text files: .txt, .csv, .java, .json, .html
Examples	FileInputStream, BufferedOutputStream, DataInputStream, ObjectOutputStream	FileReader, BufferedReader, PrintWriter, InputStreamReader

15.1.3 Input vs output streams

Input classes (`…InputStream`, `…Reader`) **read** data into the program; output classes (`…OutputStream`, `…Writer`) **write** data out of it. Streams are designed to be **wrapped** (the Decorator pattern): a basic stream that talks to the file is wrapped by a buffered stream for speed, which may be wrapped again by a stream that adds features (reading lines, numbers or objects).



STREAMS WRAP OTHER STREAMS, EACH LAYER ADDING ONE FEATURE

	Byte input	Byte output	Char input	Char output
Base	InputStream	OutputStream	Reader	Writer
File	FileInputStream	FileOutputStream	FileReader	FileWriter
Buffered	BufferedInputStream	BufferedOutputStream	BufferedReader	BufferedWriter
Primitives / objects	DataInputStream, ObjectInputStream	DataOutputStream, ObjectOutputStream	-	PrintWriter
Memory	ByteArrayInputStream	ByteArrayOutputStream	StringReader	StringWriter

15.2 The File class

15.2.1 Creating File objects

`java.io.File` represents a **path** to a file or directory. Creating a File object does **not** create anything on disk; it is just a name you can then ask questions about or act on. Paths may be absolute (`C:\data\a.txt` , `/home/asha/a.txt`) or relative to the program's working directory. Use `File.separator` or the two-argument constructor for portability.

CONSTRUCTING FILE OBJECTS java

```
File f1 = new File("notes.txt");                  // relative to the working directory
File f2 = new File("data", "marks.csv");           // parent folder + child name
File f3 = new File("C:\\Users\\Lohit\\report.pdf");   // Windows path: write \\ for each \
```

15.2.2 Methods: exists, createNewFile, delete, mkdir, mkdirs, list, length, isFile, isDirectory

WORKING WITH FILES AND FOLDERS

FileClassDemo.java

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Arrays;

public class FileClassDemo {
    public static void main(String[] args) throws IOException {
        File dir = new File("college/reports/2026");
        System.out.println("exists before?   " + dir.exists());
        System.out.println("mkdir (one level)? " + new File("college").mkdir());
        System.out.println("mkdirs (all levels)? " + dir.mkdirs());

        File f = new File(dir, "marks.txt");
        System.out.println("createNewFile?   " + f.createNewFile());
        System.out.println("again?           " + f.createNewFile() + " (already exists)");
        try (FileWriter w = new FileWriter(f)) { w.write("Asha,91\nRavi,78\n"); }

        System.out.println("name             " + f.getName());
        System.out.println("path             " + f.getPath());
        System.out.println("parent           " + f.getParent());
        System.out.println("isFile / isDir   " + f.isFile() + " / " + f.isDirectory());
        System.out.println("length (bytes)   " + f.length());
        System.out.println("canRead/Write    " + f.canRead() + " / " + f.canWrite());
        new File(dir, "attendance.txt").createNewFile();
        String[] names = dir.list();
        Arrays.sort(names);
        System.out.println("list()           " + Arrays.toString(names));

        System.out.println("delete file      " + f.delete());
        System.out.println("delete non-empty dir? " + new File("college").delete());
    }
}
```

OUTPUT

```
exists before?   false
mkdir (one level)? true
mkdirs (all levels)? true
createNewFile?   true
again?           false (already exists)
name             marks.txt
path             college/reports/2026/marks.txt
parent           college/reports/2026
isFile / isDir   true / false
length (bytes)   16
canRead/Write    true / true
list()           [attendance.txt, marks.txt]
delete file      true
delete non-empty dir? false
```

NOTE

`delete()` removes a directory only if it is empty, and returns false instead of throwing on failure, which makes errors easy to miss. The newer `Files` class (15.6) throws informative exceptions instead.

15.3 Byte streams

15.3.1 InputStream and OutputStream

The two abstract base classes of all byte streams. Key methods:

InputStream	OutputStream
`int read()` : next byte (0–255), or **-1** at end of stream	`void write(int b)` : writes one byte
`int read(byte[] buf)` : fills buf, returns count or -1	`void write(byte[] buf, int off, int len)`

InputStream	OutputStream
`byte[] readAllBytes()` (Java 9)	`void flush()`: push buffered bytes out
`void close()`	`void close()`

15.3.2 FileInputStream and FileOutputStream

WRITING AND READING RAW BYTES

ByteFileDemo.java

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class ByteFileDemo {
    public static void main(String[] args) throws IOException {
        try (FileOutputStream out = new FileOutputStream("bytes.bin")) {
            out.write(65);                          // 'A'
            out.write(new byte[] {66, 67, 10});     // 'B', 'C', newline
            out.write("Hello".getBytes());
        }
        try (FileOutputStream out = new FileOutputStream("bytes.bin", true)) {   // true = append
            out.write(" World".getBytes());
        }
        try (FileInputStream in = new FileInputStream("bytes.bin")) {
            int b, count = 0;
            StringBuilder raw = new StringBuilder();
            while ((b = in.read()) != -1) {         // -1 means end of file
                raw.append(b).append(' ');
                count++;
            }
            System.out.println("bytes read: " + count);
            System.out.println("values    : " + raw.toString().trim());
        }
        try (FileInputStream in = new FileInputStream("bytes.bin")) {
            System.out.println("as text   : " + new String(in.readAllBytes()).replace("\n", "\\n"));
        }
    }
}
```

OUTPUT

```
bytes read: 15
values    : 65 66 67 10 72 101 108 108 111 32 87 111 114 108 100
as text   : ABC\nHello World
```

15.3.3 BufferedInputStream and BufferedOutputStream

Each `read()` on a raw FileInputStream may cause a slow system call to the operating system. A **buffered** stream reads a large block (8 KB by default) into memory at once and serves later reads from that buffer; a buffered output stream collects writes and sends them in blocks. Always wrap file streams in buffers. Call `flush()` (or close) to make sure buffered output reaches the file.

BUFFERING MAKES BYTE-BY-BYTE I/O FAST

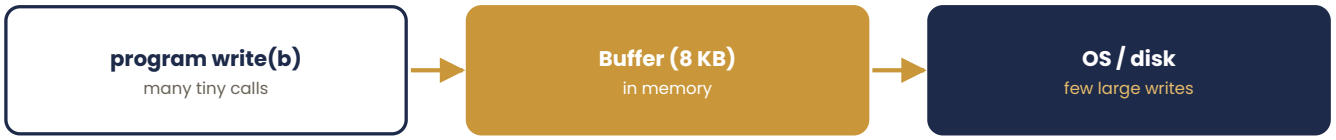
BufferedSpeed.java

```
import java.io.*;

public class BufferedSpeed {
    public static void main(String[] args) throws IOException {
        int n = 300_000;
        long t0 = System.nanoTime();
        try (OutputStream out = new FileOutputStream("raw.dat")) {
            for (int i = 0; i < n; i++) out.write(i % 256);                // one system call per byte
        }
        long t1 = System.nanoTime();
        try (OutputStream out = new BufferedOutputStream(new FileOutputStream("buf.dat"))) {
            for (int i = 0; i < n; i++) out.write(i % 256);                // collected in an 8 KB buffer
        }
        long t2 = System.nanoTime();
        System.out.println("same size: " + (new File("raw.dat").length() == new File("buf.dat").length()));
        System.out.println("buffered at least 5x faster: " + ((t1 - t0) > 5 * (t2 - t1)));
    }
}
```

OUTPUT

```
same size: true
buffered at least 5x faster: true
```



A BUFFER TURNS THOUSANDS OF SMALL OPERATIONS INTO A FEW LARGE ONES

15.3.4 DataInputStream and DataOutputStream

These read and write **primitive values** (int, double, boolean, UTF strings) in a portable binary format. Values must be read back in **exactly the same order and types** they were written.

TYPED BINARY DATA

DataStreams.java

```
import java.io.*;

public class DataStreams {
    public static void main(String[] args) throws IOException {
        try (DataOutputStream out = new DataOutputStream(new BufferedOutputStream(new
FileOutputStream("student.dat")))) {
            out.writeInt(101);
            out.writeUTF("Asha");
            out.writeDouble(91.5);
            out.writeBoolean(true);
        }
        System.out.println("file size: " + new File("student.dat").length() + " bytes (4 + 2+4 + 8 + 1)");
        try (DataInputStream in = new DataInputStream(new BufferedInputStream(new FileInputStream("student.dat")))) {
            int id = in.readInt();
            String name = in.readUTF();
            double marks = in.readDouble();
            boolean active = in.readBoolean();
            System.out.println(id + " " + name + " " + marks + " " + active);
            try {
                in.readInt();
            } catch (EOFException e) {
                System.out.println("EOFException: no more data");
            }
        }
    }
}
```

OUTPUT

```
file size: 19 bytes (4 + 2+4 + 8 + 1)
101 Asha 91.5 true
EOFException: no more data
```

15.3.5 Copying binary files (images)

To copy any binary file (an image, a PDF), read a chunk of bytes into a buffer array and write exactly the number of bytes read, until read returns -1. Never use a Reader/Writer for binary files: the character conversion corrupts them.

CHUNKED COPY OF A BINARY FILE CopyBinary.java

```
import java.io.*;
import java.util.Arrays;

public class CopyBinary {
    static long copy(String src, String dest) throws IOException {
        long total = 0;
        try (InputStream in = new BufferedInputStream(new FileInputStream(src));
             OutputStream out = new BufferedOutputStream(new FileOutputStream(dest))) {
            byte[] buffer = new byte[4096];
            int n;
            while ((n = in.read(buffer)) != -1) {        // n = bytes actually read
                out.write(buffer, 0, n);                  // write only those n bytes
                total += n;
            }
        }
        return total;
    }
    public static void main(String[] args) throws IOException {
        byte[] fakeImage = new byte[10_000];              // stand-in for logo.png
        for (int i = 0; i < fakeImage.length; i++) fakeImage[i] = (byte) (i * 31);
        try (FileOutputStream f = new FileOutputStream("logo.png")) { f.write(fakeImage); }

        long copied = copy("logo.png", "logo_copy.png");
        byte[] a, b;
        try (FileInputStream x = new FileInputStream("logo.png"); FileInputStream y = new
FileInputStream("logo_copy.png")) {
            a = x.readAllBytes(); b = y.readAllBytes();
        }
        System.out.println("copied " + copied + " bytes, identical: " + Arrays.equals(a, b));
    }
}
```

OUTPUT

```
copied 10000 bytes, identical: true
```

15.4 Character streams

15.4.1 Reader and Writer

The abstract bases for text. `Reader.read()` returns one **char** (or -1), `Writer.write(String)` writes text. Character streams handle the **encoding**: they convert between the bytes stored in the file and Java's Unicode characters, so "₹", "é" or "हिन्दी" are read correctly when the right charset (normally UTF-8) is used.

15.4.2 FileReader and FileWriter (append mode)

`FileWriter(name)` **overwrites** the file; `FileWriter(name, true)` **appends** to it. Since Java 11 both classes accept a `Charset`; always pass `StandardCharsets.UTF_8` so the program behaves the same on Windows and macOS.

WRITING, APPENDING AND READING TEXT

FileReaderWriter.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class FileReaderWriter {
    public static void main(String[] args) throws IOException {
        try (FileWriter w = new FileWriter("diary.txt", StandardCharsets.UTF_8)) {          // overwrite
            w.write("Day 1: learned Java I/O\n");
        }
        try (FileWriter w = new FileWriter("diary.txt", StandardCharsets.UTF_8, true)) {    // append
            w.write("Day 2: fees paid ₹15,000\n");
        }
        try (FileReader r = new FileReader("diary.txt", StandardCharsets.UTF_8)) {
            int c, chars = 0;
            StringBuilder sb = new StringBuilder();
            while ((c = r.read()) != -1) { sb.append((char) c); chars++; }
            System.out.print(sb);
            System.out.println("chars: " + chars + ", bytes on disk: " + new File("diary.txt").length() + " (₹ takes 3
bytes in UTF-8)");
        }
    }
}
```

OUTPUT

```
Day 1: learned Java I/O
Day 2: fees paid ₹15,000
chars: 49, bytes on disk: 51 (₹ takes 3 bytes in UTF-8)
```

15.4.3 BufferedReader (readLine()) and BufferedWriter (newLine())

`BufferedReader` adds a buffer and the very useful `readLine()`, which returns one line without the line terminator, or **null** at end of file. `BufferedWriter` adds a buffer and `newLine()`, which writes the platform's line separator.

THE STANDARD LINE-BY-LINE PATTERN

BufferedText.java

```
import java.io.*;

public class BufferedText {
    public static void main(String[] args) throws IOException {
        String[] students = {"Asha,91", "Ravi,78", "Meera,85", "Kiran,62"};
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("students.txt"))) {
            for (String s : students) {
                bw.write(s);
                bw.newLine();
            }
        }
        int lineNo = 0, total = 0;
        try (BufferedReader br = new BufferedReader(new FileReader("students.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {          // null = end of file
                lineNo++;
                String[] parts = line.split(",");
                total += Integer.parseInt(parts[1]);
                System.out.println(lineNo + ". " + parts[0] + " scored " + parts[1]);
            }
        }
        System.out.println("average = " + (double) total / lineNo);
    }
}
```

OUTPUT

```
1. Asha scored 91
2. Ravi scored 78
3. Meera scored 85
4. Kiran scored 62
average = 79.0
```

15.4.4 PrintWriter

`PrintWriter` offers the familiar `print`, `println` and `printf` methods for writing text to a file (or any Writer/OutputStream). Its methods **never throw IOException**; call `checkError()` to detect problems. Pass `true` as the autoflush argument to flush on every println.

FORMATTED TEXT OUTPUT TO A FILE

PrintWriterDemo.java

```
import java.io.*;

public class PrintWriterDemo {
    public static void main(String[] args) throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("report.txt"))) {
            pw.println("LBF MARKS REPORT");
            pw.println("----------------");
            pw.printf("%-8s %6s%n", "Name", "Marks");
            pw.printf("%-8s %6.1f%n", "Asha", 91.5);
            pw.printf("%-8s %6.1f%n", "Ravi", 78.0);
            System.out.println("errors? " + pw.checkError());
        }
        try (BufferedReader br = new BufferedReader(new FileReader("report.txt"))) {
            br.lines().forEach(System.out::println);        // Java 8: lines as a stream
        }
    }
}
```

OUTPUT

```
errors? false
LBF MARKS REPORT
----------------
Name      Marks
Asha       91.5
Ravi       78.0
```

15.4.5 InputStreamReader and OutputStreamWriter (charset bridges)

These two classes are the **bridges** between the byte world and the character world. `InputStreamReader` wraps any InputStream (a file, `System.in`, a network socket) and decodes bytes into chars using a charset; `OutputStreamWriter` encodes chars into bytes. Choosing the wrong charset produces garbled text ("mojibake").

ENCODING MATTERS

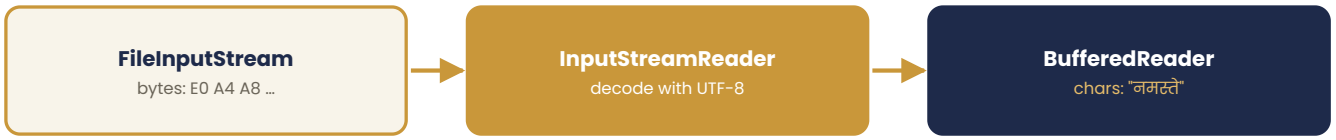
CharsetBridge.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class CharsetBridge {
    public static void main(String[] args) throws IOException {
        try (Writer w = new OutputStreamWriter(new FileOutputStream("hindi.txt"), StandardCharsets.UTF_8)) {
            w.write("नमस्ते Java");
        }
        try (BufferedReader ok = new BufferedReader(new InputStreamReader(new FileInputStream("hindi.txt"),
StandardCharsets.UTF_8))) {
            System.out.println("read as UTF-8     : " + ok.readLine());
        }
        try (BufferedReader bad = new BufferedReader(new InputStreamReader(new FileInputStream("hindi.txt"),
StandardCharsets.ISO_8859_1))) {
            String s = bad.readLine();
            System.out.println("read as ISO-8859-1: garbled? " + !s.startsWith("नमस्ते") + " (" + s.length() + " chars
instead of 11)");
        }
    }
}
```

OUTPUT

```
read as UTF-8     : नमस्ते Java
read as ISO-8859-1: garbled? true (23 chars instead of 11)
```

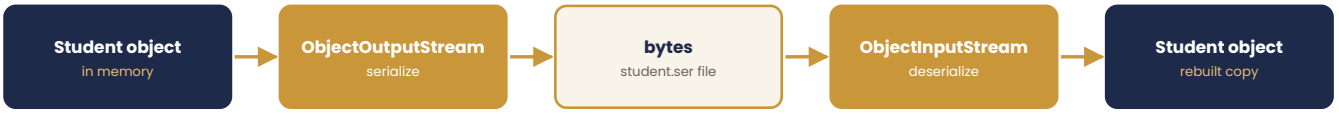


INPUTSTREAMREADER TURNS BYTES INTO CHARACTERS

15.5 Serialization

15.5.1 What serialization and deserialization are

Serialization converts an object (and every object it references) into a stream of bytes that can be saved to a file, sent over a network, or cached. **Deserialization** rebuilds an identical object from those bytes. It lets you "freeze" an object's state and restore it later.



OBJECT → BYTES → OBJECT

15.5.2 Serializable interface

A class must implement the **marker interface** `java.io.Serializable` (no methods) to be serializable. All its fields must be serializable too (primitives, String, collections and other Serializable classes are); otherwise `NotSerializableException` is thrown.

15.5.3 ObjectOutputStream and ObjectInputStream

15.5.4 transient fields

A field marked `transient` is **skipped** during serialization; after deserialization it has its default value (null, 0, false). Use it for passwords, derived/cached values, and fields that are not serializable (threads, connections).

SAVING AND RESTORING AN OBJECT; TRANSIENT FIELDS ARE NOT SAVED SerializationDemo.java

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    int id;
    String name;
    List<String> courses = new ArrayList<>();
    transient String password;                    // not saved
    transient int cachedRank = -1;                // recomputed later
    Student(int id, String name, String pw) { this.id = id; this.name = name; this.password = pw; }
    @Override public String toString() {
        return id + " " + name + " " + courses + " pw=" + password + " rank=" + cachedRank;
    }
}
public class SerializationDemo {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        Student s = new Student(101, "Asha", "secret");
        s.courses.add("Java"); s.courses.add("SQL");
        s.cachedRank = 3;
        System.out.println("before : " + s);

        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("student.ser"))) {
            out.writeObject(s);
        }
        System.out.println("saved " + new File("student.ser").length() + " bytes");

        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("student.ser"))) {
            Student copy = (Student) in.readObject();
            System.out.println("after  : " + copy);
            System.out.println("same object? " + (copy == s));
        }
    }
}
```

OUTPUT

```
before : 101 Asha [Java, SQL] pw=secret rank=3
saved 168 bytes
after  : 101 Asha [Java, SQL] pw=null rank=0
same object? false
```

NOTE

During deserialization the constructor of a Serializable class is **not called** and field initializers do not run; that is why `cachedRank` is 0 (the default), not -1.

15.5.5 serialVersionUID

Each Serializable class has a version number, `serialVersionUID`. It is written into the bytes, and on reading it must match the class's current value, or `InvalidClassException` is thrown. If you do not declare one, the compiler computes it from the class structure, so **any** change (adding a field or method) breaks old files. Always declare it: `private static final long serialVersionUID = 1L;` and change it only when you intentionally make old data incompatible.

15.5.6 Serialization with inheritance

If a parent class is Serializable, all subclasses are too. If the **parent is not** Serializable but the child is, the parent's fields are **not saved**; during deserialization the parent's **no-arg constructor is called** to initialise them (and it must exist).

A NON-SERIALIZABLE PARENT LOSES ITS FIELDS SerialInheritance.java

```
import java.io.*;

class Person {                                    // NOT Serializable
    String city = "unknown";
    Person() { System.out.println("  Person() no-arg constructor called"); }
}
class Learner extends Person implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    Learner(String name, String city) { this.name = name; this.city = city; }
}
public class SerialInheritance {
    public static void main(String[] args) throws Exception {
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        System.out.println("creating:");
        Learner l = new Learner("Ravi", "Pune");
        try (ObjectOutputStream out = new ObjectOutputStream(bytes)) { out.writeObject(l); }
        System.out.println("deserializing:");
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
            Learner copy = (Learner) in.readObject();
            System.out.println("name = " + copy.name + " (saved), city = " + copy.city + " (parent field reset)");
        }
    }
}
```

OUTPUT

```
creating:
  Person() no-arg constructor called
deserializing:
  Person() no-arg constructor called
name = Ravi (saved), city = unknown (parent field reset)
```

15.5.7 Externalizable

`Externalizable` extends Serializable and gives you **full control**: you implement `writeExternal(ObjectOutput)` and `readExternal(ObjectInput)` and decide exactly what is written. It requires a **public no-arg constructor**, which is called before readExternal. It can be smaller and faster, at the cost of more code.

CUSTOM SERIALIZED FORM ExternalizableDemo.java

```
import java.io.*;

class Product implements Externalizable {
    String code; double price; int stock;
    public Product() { }                                  // required
    Product(String c, double p, int s) { code = c; price = p; stock = s; }
    @Override public void writeExternal(ObjectOutput out) throws IOException {
        out.writeUTF(code);
        out.writeDouble(price);                           // stock deliberately not saved
    }
    @Override public void readExternal(ObjectInput in) throws IOException {
        code = in.readUTF();
        price = in.readDouble();
    }
}
public class ExternalizableDemo {
    public static void main(String[] args) throws Exception {
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(bytes)) { out.writeObject(new Product("PEN-01", 25.0,
400)); }
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
            Product p = (Product) in.readObject();
            System.out.println(p.code + " " + p.price + " stock=" + p.stock);
        }
    }
}
```

OUTPUT

```
PEN-01 25.0 stock=0
```

15.5.8 Security risks and modern alternatives (JSON with Jackson or Gson)

Java serialization has serious drawbacks, and its own designers now call it a mistake:

- **Security**: deserializing untrusted bytes can create arbitrary objects and trigger "gadget chains" that execute code. Many critical vulnerabilities came from this. **Never deserialize data from untrusted sources.** If you must, use an `ObjectInputFilter` (Java 9) to allow-list classes.
- **Fragility**: tied to class structure and serialVersionUID; hard to evolve.
- **Java-only**: other languages cannot read the format, and it is not human-readable.

Modern applications store and exchange data as **JSON** (or XML, Protocol Buffers, Avro). The most popular Java JSON libraries are **Jackson** (used by Spring Boot) and **Gson** (Google). They are added via Maven/Gradle (Module 22):

JSON WITH JACKSON AND GSON (EXTERNAL LIBRARIES) java

```java
// Jackson: com.fasterxml.jackson.core:jackson-databind
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(student);          // object -> JSON text
Student s = mapper.readValue(json, Student.class);         // JSON text -> object

// Gson: com.google.code.gson:gson
Gson gson = new Gson();
String json2 = gson.toJson(student);
Student s2 = gson.fromJson(json2, Student.class);
```

15.6 NIO and NIO.2 (java.nio.file)

Java 1.4 introduced **NIO** (New I/O: buffers, channels, non-blocking I/O) and Java 7 added **NIO.2**: the `java.nio.file` package with `Path` and `Files`. For everyday file work, NIO.2 is the modern, recommended API: shorter code, clear exceptions, and support for symbolic links, attributes and directory walking.

15.6.1 Path and Paths

A `Path` represents a file-system path (the modern replacement for File). Create one with `Path.of("a", "b.txt")` (Java 11) or `Paths.get(...)` (Java 7). Paths can be combined and inspected without touching the disk.

BUILDING AND INSPECTING PATHS PathDemo.java

```java
import java.nio.file.Path;
import java.nio.file.Paths;

public class PathDemo {
    public static void main(String[] args) {
        Path p = Path.of("college", "reports", "2026", "marks.csv");
        Path q = Paths.get("college/reports/../notes/./todo.txt");
        System.out.println("path        : " + p);
        System.out.println("file name   : " + p.getFileName());
        System.out.println("parent      : " + p.getParent());
        System.out.println("root part   : " + p.getName(0) + ", name count " + p.getNameCount());
        System.out.println("resolve     : " + p.getParent().resolve("attendance.csv"));
        System.out.println("normalize   : " + q.normalize());
        System.out.println("relativize  : " + Path.of("college").relativize(p));
        System.out.println("is absolute : " + p.isAbsolute() + ", ends with .csv: " + p.toString().endsWith(".csv"));
    }
}
```

OUTPUT

```
path        : college/reports/2026/marks.csv
file name   : marks.csv
parent      : college/reports/2026
root part   : college, name count 4
resolve     : college/reports/2026/attendance.csv
normalize   : college/notes/todo.txt
relativize  : reports/2026/marks.csv
is absolute : false, ends with .csv: true
```

15.6.2 Files utility: readAllLines, readString, write, writeString, copy, move, delete, exists

COMMON FILES OPERATIONS IN A FEW LINES EACH

FilesDemo.java

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;

public class FilesDemo {
    public static void main(String[] args) throws IOException {
        Path dir = Path.of("lbf");
        Files.createDirectories(dir.resolve("backup"));
        Path f = dir.resolve("courses.txt");

        Files.writeString(f, "Java\nSQL\n");                                    // Java 11
        Files.write(f, List.of("DSA", "Spring"), StandardOpenOption.APPEND);     // lines, append
        System.out.println("exists      : " + Files.exists(f) + ", size " + Files.size(f) + " bytes");
        List<String> lines = Files.readAllLines(f);
        System.out.println("readAllLines: " + lines);
        System.out.println("readString  : " + Files.readString(f).replace("\n", " | "));

        Path copy = Files.copy(f, dir.resolve("backup/courses.bak"), StandardCopyOption.REPLACE_EXISTING);
        Path moved = Files.move(copy, dir.resolve("backup/courses-old.txt"), StandardCopyOption.REPLACE_EXISTING);
        System.out.println("copied+moved: " + moved + ", old name exists? " + Files.exists(copy));

        Files.delete(moved);
        System.out.println("deleteIfExists again: " + Files.deleteIfExists(moved));
        try {
            Files.delete(dir.resolve("nothing.txt"));
        } catch (NoSuchFileException e) {
            System.out.println("delete missing -> NoSuchFileException: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
exists      : true, size 20 bytes
readAllLines: [Java, SQL, DSA, Spring]
readString  : Java | SQL | DSA | Spring |
copied+moved: lbf/backup/courses-old.txt, old name exists? false
deleteIfExists again: false
delete missing -> NoSuchFileException: lbf/nothing.txt
```

Method	Does
`Files.readString(p)` / `writeString(p, s)`	Whole file as one String (Java 11)
`Files.readAllLines(p)` / `write(p, lines)`	Whole file as a List of lines
`Files.newBufferedReader(p)` / `newBufferedWriter(p)`	Buffered reader/writer for large files
`Files.copy`, `move`, `delete`, `deleteIfExists`	File management (options: REPLACE_EXISTING, ATOMIC_MOVE)
`Files.exists`, `isDirectory`, `size`, `getLastModifiedTime`	Queries
`Files.createDirectories`, `createFile`, `createTempFile`	Creation

WATCH OUT

`readAllLines` and `readString` load the **entire** file into memory. For large files (logs of many GB) use `Files.lines()` or a BufferedReader to process one line at a time.

15.6.3 Walking directories: Files.walk(), Files.list()

`Files.list(dir)` returns a stream of the **direct** children; `Files.walk(dir)` returns **all** descendants recursively (optionally to a max depth). Both return streams that hold open directory handles, so use them in try-with-resources.

LISTING AND WALKING A FOLDER TREE WalkDemo.java

```java
import java.io.IOException;
import java.nio.file.*;
import java.util.stream.Stream;

public class WalkDemo {
    public static void main(String[] args) throws IOException {
        Path root = Path.of("project");
        Files.createDirectories(root.resolve("src/model"));
        Files.createDirectories(root.resolve("src/service"));
        Files.writeString(root.resolve("README.md"), "# LBF");
        Files.writeString(root.resolve("src/model/Student.java"), "class Student {}");
        Files.writeString(root.resolve("src/service/StudentService.java"), "class StudentService {}");

        try (Stream<Path> children = Files.list(root)) {
            System.out.println("list (direct children):");
            children.sorted().forEach(p -> System.out.println("  " + root.relativize(p)));
        }
        try (Stream<Path> all = Files.walk(root)) {
            System.out.println("walk (all .java files):");
            all.filter(p -> p.toString().endsWith(".java")).sorted()
               .forEach(p -> System.out.println("  " + root.relativize(p)));
        }
        try (Stream<Path> all = Files.walk(root)) {
            long dirs = all.filter(Files::isDirectory).count();
            System.out.println("directories including root: " + dirs);
        }
    }
}
```

OUTPUT

```
list (direct children):
  README.md
  src
walk (all .java files):
  src/model/Student.java
  src/service/StudentService.java
directories including root: 4
```

15.6.4 Reading lines as a stream: Files.lines()

`Files.lines(path)` reads lines **lazily**, one at a time, as a `Stream<String>`, so even huge files can be filtered, counted and transformed with little memory, using the Stream API (Module 17).

PROCESSING A LOG FILE LAZILY

LinesStream.java

```java
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.stream.Stream;

public class LinesStream {
    public static void main(String[] args) throws IOException {
        Path log = Path.of("server.log");
        Files.write(log, List.of(
            "2026-06-01 INFO  started",
            "2026-06-01 ERROR db timeout",
            "2026-06-01 INFO  request /home",
            "2026-06-02 ERROR disk full",
            "2026-06-02 WARN  slow query"));
        try (Stream<String> lines = Files.lines(log)) {
            lines.filter(l -> l.contains("ERROR"))
                 .map(l -> l.substring(17))
                 .forEach(l -> System.out.println("error: " + l));
        }
        try (Stream<String> lines = Files.lines(log)) {
            System.out.println("lines on 2026-06-02: " + lines.filter(l -> l.startsWith("2026-06-02")).count());
        }
    }
}
```

OUTPUT

```
error: db timeout
error: disk full
lines on 2026-06-02: 2
```

15.6.5 Buffers and channels (overview)

Underneath, NIO works with **channels** and **buffers**. A **channel** (`FileChannel`, `SocketChannel`) is a connection to an I/O device that can read and write. A **buffer** (`ByteBuffer`) is a fixed-size block of memory with a **position**, **limit** and **capacity**. You fill a buffer from a channel, `flip()` it to switch from writing to reading mode, and drain it. Channels also support memory-mapped files, file locking, fast transfers (`transferTo`) and, with a `Selector`, **non-blocking** I/O, where one thread serves thousands of network connections (the basis of servers such as Netty).

READING A FILE THROUGH A CHANNEL AND A SMALL BUFFER

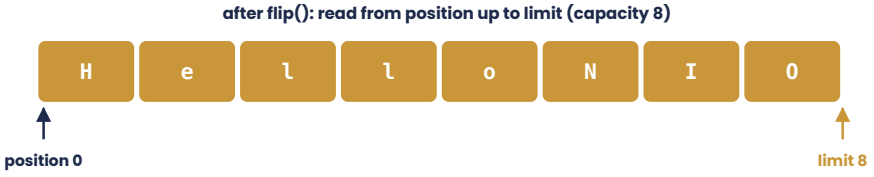
ChannelDemo.java

```
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;

public class ChannelDemo {
    public static void main(String[] args) throws IOException {
        Path p = Path.of("channel.txt");
        try (FileChannel ch = FileChannel.open(p, StandardOpenOption.CREATE, StandardOpenOption.WRITE,
StandardOpenOption.TRUNCATE_EXISTING)) {
            ByteBuffer buf = ByteBuffer.wrap("Hello NIO channels".getBytes(StandardCharsets.UTF_8));
            System.out.println("written bytes: " + ch.write(buf));
        }
        try (FileChannel ch = FileChannel.open(p, StandardOpenOption.READ)) {
            ByteBuffer buf = ByteBuffer.allocate(8);                        // small on purpose
            StringBuilder sb = new StringBuilder();
            while (ch.read(buf) != -1) {
                buf.flip();                                                 // write mode -> read mode
                System.out.println("  chunk: position=" + buf.position() + " limit=" + buf.limit() + " capacity=" +
buf.capacity());
                sb.append(StandardCharsets.UTF_8.decode(buf));
                buf.clear();                                                // ready to fill again
            }
            System.out.println("read back: " + sb);
        }
    }
}
```

OUTPUT

```
written bytes: 18
  chunk: position=0 limit=8 capacity=8
  chunk: position=0 limit=8 capacity=8
  chunk: position=0 limit=2 capacity=8
read back: Hello NIO channels
```



A BYTEBUFFER AFTER FLIP()

15.7 Working with common formats

15.7.1 Reading and writing CSV files

CSV (comma-separated values) is plain text: one record per line, fields separated by commas, usually with a header line. Excel and every database can import and export it. For simple data, `split(",")` works; real-world CSV may contain quoted fields with commas inside (`"Mumbai, MH"`), for which libraries such as **OpenCSV** or **Apache Commons CSV** are used.

WRITING AND PARSING A CSV FILE

CsvDemo.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class CsvDemo {
    record Course(String code, String title, int fee) { }

    public static void main(String[] args) throws IOException {
        List<Course> courses = List.of(new Course("J101", "Core Java", 15000), new Course("S201", "SQL", 8000), new
Course("D301", "DSA", 12000));
        Path csv = Path.of("courses.csv");
        try (BufferedWriter w = Files.newBufferedWriter(csv)) {
            w.write("code,title,fee");                                  // header
            w.newLine();
            for (Course c : courses) {
                w.write(String.join(",", c.code(), c.title(), String.valueOf(c.fee())));
                w.newLine();
            }
        }
        System.out.println("file:\n" + Files.readString(csv));

        List<Course> loaded = new ArrayList<>();
        try (BufferedReader r = Files.newBufferedReader(csv)) {
            String header = r.readLine();                               // skip header
            String line;
            while ((line = r.readLine()) != null) {
                if (line.isBlank()) continue;
                String[] f = line.split(",");
                loaded.add(new Course(f[0], f[1], Integer.parseInt(f[2].trim())));
            }
        }
        int total = loaded.stream().mapToInt(Course::fee).sum();
        System.out.println("loaded " + loaded.size() + " courses, total fees " + total);
    }
}
```

OUTPUT

```
file:
code,title,fee
J101,Core Java,15000
S201,SQL,8000
D301,DSA,12000

loaded 3 courses, total fees 35000
```

15.7.2 Properties files

A `.properties` file stores configuration as `key=value` lines (with `#` comments): database URLs, usernames, feature flags. `java.util.Properties` loads and saves them, so settings can change without recompiling. Never hard-code passwords in source code.

STORING AND LOADING CONFIGURATION PropertiesDemo.java

```
import java.io.*;
import java.util.Properties;

public class PropertiesDemo {
    public static void main(String[] args) throws IOException {
        Properties out = new Properties();
        out.setProperty("db.url", "jdbc:mysql://localhost:3306/college");
        out.setProperty("db.user", "root");
        out.setProperty("app.pageSize", "20");
        try (Writer w = new FileWriter("app.properties")) {
            out.store(w, "LBF application settings");
        }

        Properties config = new Properties();
        try (Reader r = new FileReader("app.properties")) {
            config.load(r);
        }
        System.out.println("url      = " + config.getProperty("db.url"));
        System.out.println("user     = " + config.getProperty("db.user"));
        int pageSize = Integer.parseInt(config.getProperty("app.pageSize"));
        System.out.println("pageSize = " + pageSize);
        System.out.println("timeout  = " + config.getProperty("db.timeout", "30") + " (default used)");
    }
}
```

OUTPUT

```
url      = jdbc:mysql://localhost:3306/college
user     = root
pageSize = 20
timeout  = 30 (default used)
```

APP.PROPERTIES (STORE() ESCAPES ":" AND ADDS A TIMESTAMP COMMENT) text

```
#LBF application settings
app.pageSize=20
db.url=jdbc\:mysql\://localhost\:3306/college
db.user=root
```

15.7.3 JSON overview

JSON (JavaScript Object Notation) is the standard text format for web APIs and configuration. It has objects `{ }` with "key": value pairs, arrays `[ ]`, strings, numbers, booleans and null. It maps naturally to Java objects, lists and maps.

STUDENT.JSON text

```
{
  "id": 101,
  "name": "Asha",
  "active": true,
  "courses": ["Java", "SQL"],
  "address": { "city": "Pune", "pin": "411001" }
}
```

JSON	Java
object `{ }`	class / record / `Map<String, Object>`
array `[ ]`	`List` / array
string	`String`
number	`int`, `long`, `double`, `BigDecimal`
true / false	`boolean`
null	`null`

The JDK has no built-in JSON library, so projects add **Jackson** or **Gson** (15.5.8). For learning, a small JSON string can be built by hand:

PRODUCING JSON TEXT MANUALLY (REAL PROJECTS USE JACKSON/GSON) JsonByHand.java

```
import java.util.List;
import java.util.stream.Collectors;

public class JsonByHand {
    record Student(int id, String name, boolean active, List<String> courses) {
        String toJson() {
            String arr = courses.stream().map(c -> "\"" + c + "\"").collect(Collectors.joining(", ", "[", "]"));
            return String.format("{\"id\": %d, \"name\": \"%s\", \"active\": %b, \"courses\": %s}", id, name, active,
arr);
        }
    }
    public static void main(String[] args) {
        Student s = new Student(101, "Asha", true, List.of("Java", "SQL"));
        System.out.println(s.toJson());
    }
}
```

OUTPUT

```
{"id": 101, "name": "Asha", "active": true, "courses": ["Java", "SQL"]}
```

PRACTICE CHECKPOINT

Extend the **Student Management System** to save records to a CSV file and load them on startup.

SAMPLE SOLUTION StudentFileSystem.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class StudentFileSystem {
    record Student(int id, String name, double marks) {
        String toCsv() { return id + "," + name + "," + marks; }
        static Student fromCsv(String line) {
            String[] f = line.split(",");
            return new Student(Integer.parseInt(f[0].trim()), f[1].trim(), Double.parseDouble(f[2].trim()));
        }
    }

    private static final Path FILE = Path.of("students.csv");
    private final Map<Integer, Student> byId = new LinkedHashMap<>();

    void load() throws IOException {
        if (!Files.exists(FILE)) {
            System.out.println("No data file yet, starting empty.");
            return;
        }
        try (BufferedReader r = Files.newBufferedReader(FILE)) {
            r.readLine();                                          // header
            String line;
            int bad = 0;
            while ((line = r.readLine()) != null) {
                try {
                    Student s = Student.fromCsv(line);
                    byId.put(s.id(), s);
                } catch (RuntimeException e) {
                    bad++;                                         // skip corrupt line, keep going
                }
            }
            System.out.println("Loaded " + byId.size() + " students (" + bad + " bad lines skipped).");
        }
    }
    void save() throws IOException {
        Path tmp = Path.of("students.csv.tmp");
        try (BufferedWriter w = Files.newBufferedWriter(tmp)) {
            w.write("id,name,marks"); w.newLine();
```

```
            for (Student s : byId.values()) { w.write(s.toCsv()); w.newLine(); }
        }
        Files.move(tmp, FILE, StandardCopyOption.REPLACE_EXISTING);   // replace in one step
        System.out.println("Saved " + byId.size() + " students to " + FILE);
    }
    void add(Student s) { byId.put(s.id(), s); }
    void printSortedByMarks() {
        byId.values().stream()
            .sorted(Comparator.comparingDouble(Student::marks).reversed())
            .forEach(s -> System.out.printf("  %d %-6s %5.1f%n", s.id(), s.name(), s.marks()));
    }

    public static void main(String[] args) throws IOException {
        Files.deleteIfExists(FILE);                              // fresh demo

        System.out.println("--- first run ---");
        StudentFileSystem app = new StudentFileSystem();
        app.load();
        app.add(new Student(101, "Asha", 91.5));
        app.add(new Student(102, "Ravi", 78.0));
        app.add(new Student(103, "Meera", 85.0));
        app.save();

        Files.writeString(FILE, "oops,this,is-not-valid\n", StandardOpenOption.APPEND);   // simulate a corrupt line

        System.out.println("--- second run (program restarted) ---");
        StudentFileSystem again = new StudentFileSystem();
        again.load();
        again.add(new Student(104, "Kiran", 88.0));
        again.printSortedByMarks();
        again.save();
        System.out.println("--- file contents ---");
        System.out.print(Files.readString(FILE));
    }
}
```

OUTPUT

```
--- first run ---
No data file yet, starting empty.
Saved 3 students to students.csv
--- second run (program restarted) ---
Loaded 3 students (1 bad lines skipped).
  101 Asha    91.5
  104 Kiran   88.0
  103 Meera   85.0
  102 Ravi    78.0
Saved 4 students to students.csv
--- file contents ---
id,name,marks
101,Asha,91.5
102,Ravi,78.0
103,Meera,85.0
104,Kiran,88.0
```

LEVEL 3

Advanced level

Outcome: concurrency, functional style, modern Java, JVM internals, databases and real-world tooling.

7 MODULES · 35 DAYS MINIMUM

Module 16 · Multithreading and concurrency	**7 days**
Module 17 · Functional programming in Java (Java 8+)	**5 days**
Module 18 · Modern Java features (Java 9 to 25)	**3 days**
Module 19 · JVM internals and memory management	**3 days**
Module 20 · JDBC (Java Database Connectivity)	**5 days**
Module 21 · Reflection, annotations, design principles and patterns	**5 days**
Module 22 · Build tools, testing and the Java ecosystem	**7 days**

MODULE 16 · ADVANCED

Multithreading and concurrency

Minimum duration **7 days** | Level **Advanced**

Multithreading runs several tasks at the same time inside one program; concurrency tools keep shared data correct while they do.

IN THIS MODULE

16.1 Basics
16.2 Creating threads
16.3 Thread lifecycle
16.4 Thread methods
16.5 Synchronization
16.6 Inter-thread communication
16.7 Concurrency problems
16.8 java.util.concurrent
16.9 Asynchronous programming
16.10 Virtual threads (Java 21+)

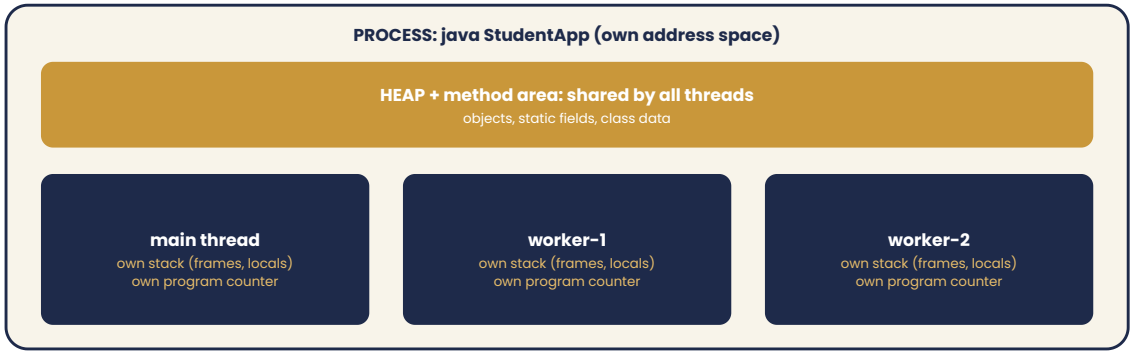
NOTE

Threads run in an order chosen by the operating system, so output that interleaves several threads can differ from run to run. Such boxes are labelled **ONE POSSIBLE OUTPUT**. Where the result must be exact, the examples wait for all threads (join) before printing.

16.1 Basics

16.1.1 Process vs thread

A **process** is a running program with its **own memory space**: Chrome, VS Code and your Java program are separate processes, and one cannot read another's memory. A **thread** is a single path of execution **inside** a process. A process can have many threads; they **share** the process's heap (objects, static fields) but each has its **own stack** (local variables, call frames) and program counter.



THREADS OF ONE PROCESS SHARE THE HEAP BUT EACH HAS ITS OWN STACK

	Process	Thread
Memory	Separate address space	Shares the heap of its process
Creation cost	Heavy (OS allocates resources)	Light
Communication	Inter-process: files, sockets, pipes	Directly through shared objects
Crash impact	Isolated from other processes	An uncaught error can affect the whole process
Context switch	Expensive	Cheaper

16.1.2 Multitasking, multiprocessing, multithreading

Multitasking

The OS runs several programs **apparently** at once (music + browser + IDE) by switching quickly between them.

Multiprocessing

A machine with **several CPU cores** runs several processes or threads **truly** at the same time.

Multithreading

One program splits its work into several **threads**, e.g. a server handling each request in a thread, or a UI thread plus a background download thread.

16.1.3 Concurrency vs parallelism

Concurrency is about *dealing with* many tasks at once: tasks make progress in overlapping time periods, even on one core, by interleaving. **Parallelism** is about *doing* many tasks at once: tasks literally run at the same instant on different cores. Concurrency is a way of structuring a program; parallelism is a way of executing it faster.



TOP: ONE CORE ALTERNATES A AND B. BOTTOM: TWO CORES RUN A AND B AT THE SAME TIME

16.1.4 The main thread

When the JVM starts your program it creates the **main thread**, which runs `main()`. Every thread you create starts from it. The JVM also runs its own background threads (garbage collector, JIT compiler). The program ends when all **non-daemon** threads have finished, not necessarily when main returns.

INSPECTING THE MAIN THREAD

MainThread.java

```
public class MainThread {
    public static void main(String[] args) {
        Thread t = Thread.currentThread();
        System.out.println("name     : " + t.getName());
        System.out.println("id       : " + t.threadId());
        System.out.println("priority : " + t.getPriority());
        System.out.println("daemon?  : " + t.isDaemon());
        System.out.println("group    : " + t.getThreadGroup().getName());
        System.out.println("cores    : " + (Runtime.getRuntime().availableProcessors() > 0 ? "available" : "none"));
    }
}
```

OUTPUT

```
name     : main
id       : 1
priority : 5
daemon?  : false
group    : main
cores    : available
```

16.2 Creating threads

16.2.1 Extending Thread

Subclass `java.lang.Thread`, override `run()` with the work to do, create an object and call `start()`. Simple, but your class can then not extend anything else, and it mixes the task with the thread mechanism.

TWO THREADS BY SUBCLASSING THREAD ExtendThread.java

```
class Downloader extends Thread {
    private final String file;
    Downloader(String file) { this.file = file; }
    @Override public void run() {
        for (int pct = 0; pct <= 100; pct += 50)
            System.out.println(getName() + " downloading " + file + " " + pct + "%");
    }
}
public class ExtendThread {
    public static void main(String[] args) throws InterruptedException {
        Downloader d1 = new Downloader("notes.pdf");
        Downloader d2 = new Downloader("video.mp4");
        d1.start();                              // new thread runs d1.run()
        d2.start();
        d1.join(); d2.join();                    // wait for both
        System.out.println("main: all downloads finished");
    }
}
```

ONE POSSIBLE OUTPUT

```
Thread-1 downloading video.mp4 0%
Thread-1 downloading video.mp4 50%
Thread-1 downloading video.mp4 100%
Thread-0 downloading notes.pdf 0%
Thread-0 downloading notes.pdf 50%
Thread-0 downloading notes.pdf 100%
main: all downloads finished
```

16.2.2 Implementing Runnable

Implement the `Runnable` interface (single method `void run()`) and pass the object to a `Thread` constructor. This separates **the task** (Runnable) from **the worker** (Thread), leaves your class free to extend another class, and lets the same task be given to thread pools. This is the preferred way.

16.2.3 Runnable with lambda

Runnable is a functional interface, so a lambda is the shortest way to write a task.

RUNNABLE AS A CLASS, A LAMBDA AND WITH THE JAVA 21 BUILDER RunnableDemo.java

```
class PrintTask implements Runnable {
    private final String msg;
    PrintTask(String msg) { this.msg = msg; }
    public void run() {
        System.out.println(Thread.currentThread().getName() + ": " + msg);
    }
}
public class RunnableDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(new PrintTask("task as a class"), "worker-1");   // 16.2.2
        Runnable r = () -> System.out.println(Thread.currentThread().getName() + ": task as a lambda");
        Thread t2 = new Thread(r, "worker-2");                                  // 16.2.3
        t1.start(); t1.join();
        t2.start(); t2.join();
        Thread t3 = Thread.ofPlatform().name("worker-3").start(() ->              // Java 21 builder
                System.out.println(Thread.currentThread().getName() + ": started with a builder"));
        t3.join();
    }
}
```

OUTPUT

```
worker-1: task as a class
worker-2: task as a lambda
worker-3: started with a builder
```

16.2.4 Callable and Future (returning results)

`Runnable.run()` returns nothing and cannot throw checked exceptions. `Callable<V>` has `V call() throws Exception`, so it can **return a result**. You submit a Callable to an `ExecutorService` (16.8) and get back a `Future<V>`, a placeholder for a result that will exist later. `future.get()` waits until the result is ready.

GETTING RESULTS BACK FROM THREADS

CallableDemo.java

```
import java.util.concurrent.*;

public class CallableDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(2);
        Callable<Long> sumTask = () -> {
            long s = 0;
            for (int i = 1; i <= 1_000_000; i++) s += i;
            return s;
        };
        Callable<String> slowTask = () -> { Thread.sleep(200); return "report ready"; };

        Future<Long> f1 = pool.submit(sumTask);
        Future<String> f2 = pool.submit(slowTask);
        System.out.println("tasks submitted, main continues...");
        System.out.println("f2 done yet? " + f2.isDone());
        System.out.println("sum   = " + f1.get());                    // blocks until ready
        System.out.println("other = " + f2.get(2, TimeUnit.SECONDS));  // wait at most 2 s
        pool.shutdown();
    }
}
```

OUTPUT

```
tasks submitted, main continues...
f2 done yet? false
sum   = 500000500000
other = report ready
```

16.2.5 start() vs run()

`start()` asks the JVM to create a **new thread**, which then calls `run()`. Calling `run()` yourself is just an ordinary method call: it executes in the **current** thread, and no concurrency happens. Calling `start()` twice on the same Thread throws `IllegalThreadStateException`.

RUN() DOES NOT CREATE A THREAD

StartVsRun.java

```
public class StartVsRun {
    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> System.out.println("running in: " + Thread.currentThread().getName());
        Thread t = new Thread(task, "new-thread");
        t.run();                // WRONG: runs in main
        t.start();              // RIGHT: runs in new-thread
        t.join();
        try {
            t.start();          // a thread can be started only once
        } catch (IllegalThreadStateException e) {
            System.out.println("second start(): IllegalThreadStateException");
        }
    }
}
```

OUTPUT

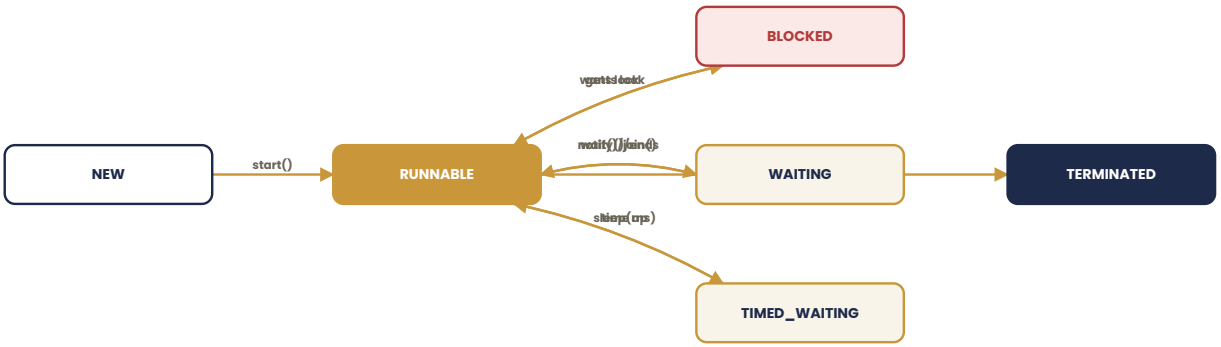
```
running in: main
running in: new-thread
second start(): IllegalThreadStateException
```

16.3 Thread lifecycle

16.3.1 States: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

State	Meaning
`NEW`	Created with `new Thread()`, not yet started
`RUNNABLE`	Started; running on a CPU or ready and waiting for the scheduler to give it one
`BLOCKED`	Waiting to acquire a monitor lock to enter a `synchronized` block
`WAITING`	Waiting indefinitely for another thread: `wait()`, `join()`, `LockSupport.park()`
`TIMED_WAITING`	Waiting with a timeout: `sleep(ms)`, `wait(ms)`, `join(ms)`
`TERMINATED`	run() has finished (normally or with an exception); cannot be restarted

16.3.2 Transitions between states



THREAD STATE TRANSITIONS

OBSERVING EVERY STATE LifecycleDemo.java

```
public class LifecycleDemo {
    public static void main(String[] args) throws InterruptedException {
        Object lock = new Object();
        Thread sleeper = new Thread(() -> {
            try { Thread.sleep(300); } catch (InterruptedException e) { }
        });
        Thread blocked = new Thread(() -> { synchronized (lock) { } });
        Thread waiter = new Thread(() -> {
            synchronized (lock) { try { lock.wait(); } catch (InterruptedException e) { } }
        });

        System.out.println("after new      : " + sleeper.getState());
        sleeper.start();
        Thread.sleep(50);
        System.out.println("during sleep() : " + sleeper.getState());

        waiter.start();
        Thread.sleep(50);
        System.out.println("during wait()  : " + waiter.getState());

        synchronized (lock) {                   // main holds the lock ...
            blocked.start();
            Thread.sleep(50);
            System.out.println("lock is taken  : " + blocked.getState());
            lock.notify();                      // wake the waiter
        }
        sleeper.join(); waiter.join(); blocked.join();
        System.out.println("after finishing: " + sleeper.getState());
        System.out.println("current thread : " + Thread.currentThread().getState());
    }
}
```

OUTPUT

```
after new      : NEW
during sleep() : TIMED_WAITING
during wait()  : WAITING
lock is taken  : BLOCKED
after finishing: TERMINATED
current thread : RUNNABLE
```

16.4 Thread methods

16.4.1 sleep(), join(), yield(), interrupt(), isAlive()

Method	Does
`Thread.sleep(ms)`	Pauses the **current** thread for at least ms milliseconds (keeps any locks it holds)
`t.join()` / `join(ms)`	Current thread waits until t finishes (or the timeout passes)
`Thread.yield()`	Hint to the scheduler that the current thread is willing to give up the CPU; rarely useful
`t.interrupt()`	Politely asks t to stop: sets its interrupt flag; if t is sleeping/waiting it gets `InterruptedException`
`t.isInterrupted()` / `Thread.interrupted()`	Check the flag (the static version also clears it)
`t.isAlive()`	True if started and not yet terminated

INTERRUPTING A SLEEPING WORKER AND JOINING IT

ThreadMethods.java

```
public class ThreadMethods {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            int count = 0;
            try {
                while (true) {
                    count++;
                    Thread.sleep(100);              // interruptible
                }
            } catch (InterruptedException e) {
                System.out.println("worker: interrupted after about " + (count >= 2 ? "several" : "few") + " cycles,
cleaning up");
            }
        }, "worker");

        worker.start();
        System.out.println("isAlive after start: " + worker.isAlive());
        Thread.sleep(350);
        worker.interrupt();                         // ask it to stop
        worker.join();                              // wait for it to finish
        System.out.println("isAlive after join : " + worker.isAlive());

        Thread quick = new Thread(() -> System.out.println("quick: done"));
        quick.start();
        quick.join(1000);                           // wait at most 1 s
        System.out.println("main: finished");
    }
}
```

OUTPUT

```
isAlive after start: true
worker: interrupted after about several cycles, cleaning up
isAlive after join : false
quick: done
main: finished
```

Java has no safe way to *force* a thread to stop (`stop()` is deprecated and removed). Use **interrupt()**, and write tasks that check `Thread.currentThread().isInterrupted()` or catch InterruptedException and exit cleanly.

16.4.2 Names and priorities

Give threads meaningful names (`setName` or the constructor) so logs and thread dumps are readable. Priorities range from `MIN_PRIORITY` (1) to `MAX_PRIORITY` (10), default `NORM_PRIORITY` (5). They are only **hints** to the OS scheduler and behave differently on each platform, so never rely on them for correctness.

NAMING A THREAD AND SETTING A PRIORITY

NamesPriorities.java

```
public class NamesPriorities {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            Thread me = Thread.currentThread();
            System.out.println("I am " + me.getName() + " with priority " + me.getPriority());
        });
        t.setName("report-generator");
        t.setPriority(Thread.MAX_PRIORITY);
        System.out.println("MIN/NORM/MAX = " + Thread.MIN_PRIORITY + "/" + Thread.NORM_PRIORITY + "/" +
Thread.MAX_PRIORITY);
        t.start();
        t.join();
    }
}
```

OUTPUT

```
MIN/NORM/MAX = 1/5/10
I am report-generator with priority 10
```

16.4.3 Daemon threads

A **daemon** thread is a background service thread (garbage collection, auto-save, heartbeat). The JVM does **not** wait for daemon threads: when the last non-daemon thread ends, the JVM exits and daemon threads are stopped abruptly, without running finally blocks. Call `setDaemon(true)` **before** `start()`.

THE JVM DOES NOT WAIT FOR DAEMON THREADS

DaemonDemo.java

```
public class DaemonDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread autosave = new Thread(() -> {
            int n = 0;
            while (true) {
                n++;
                try { Thread.sleep(100); } catch (InterruptedException e) { return; }
            }
        }, "autosave");
        autosave.setDaemon(true);                // must be set before start()
        autosave.start();
        System.out.println("autosave is daemon: " + autosave.isDaemon());
        Thread.sleep(250);
        System.out.println("main ends now; the JVM exits without waiting for autosave");
    }
}
```

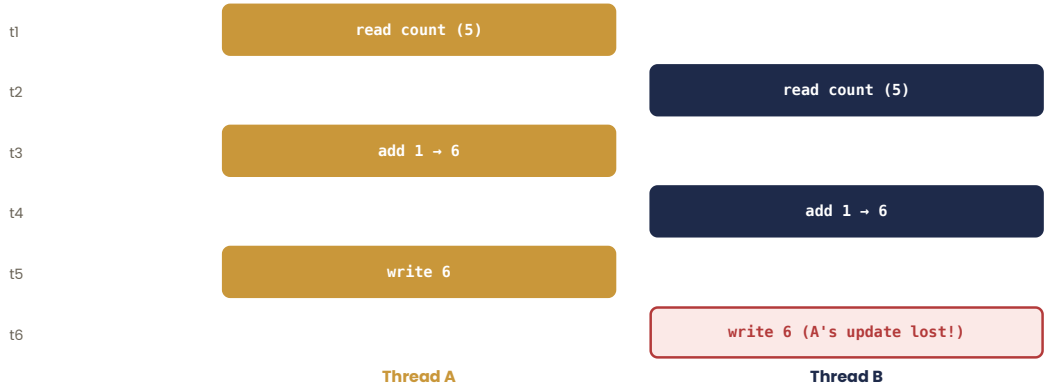
OUTPUT

```
autosave is daemon: true
main ends now; the JVM exits without waiting for autosave
```

16.5 Synchronization

16.5.1 Race conditions and shared mutable state

Trouble starts when several threads **read and write the same data**. `count++` looks like one step, but the CPU does three: **read** count, **add** 1, **write** it back. If two threads interleave these steps, one update overwrites the other and is **lost**. When the result depends on the timing of threads, it is a **race condition**. The section of code that touches the shared data is the **critical section**.



TWO INCREMENTS, BUT COUNT GOES FROM 5 TO 6: A LOST UPDATE

A RACE CONDITION IN ACTION (THE WRONG NUMBER CHANGES EVERY RUN)

RaceCondition.java

```
public class RaceCondition {
    static int count = 0;

    static void increment() {          // what count++ really does, step by step
        int value = count;             // 1. read
        Thread.yield();                // let another thread run here (makes the race easy to see)
        count = value + 1;             // 2. add  3. write
    }
    public static void main(String[] args) throws InterruptedException {
        Runnable inc = () -> { for (int i = 0; i < 10_000; i++) increment(); };
        Thread t1 = new Thread(inc), t2 = new Thread(inc);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println("expected 20000, got " + count);
    }
}
```

ONE POSSIBLE OUTPUT

```
expected 20000, got 10000
```

NOTE

A plain `count++` loop has exactly the same bug, but on a fast machine the unlucky interleaving may be rare, so the program can *appear* to work in testing and then fail in production. That is what makes race conditions dangerous.

16.5.2 synchronized methods

Marking a method `synchronized` means a thread must obtain the object's **lock** (monitor) before entering it. Only one thread at a time can hold that lock, so only one thread at a time can run **any** synchronized instance method of that same object. Others trying to enter are BLOCKED until the lock is released (when the method returns or throws).

16.5.3 synchronized blocks

A `synchronized (lockObject) { … }` block locks only the **critical section** instead of the whole method, and lets you choose the lock object. Smaller critical sections mean less waiting and better performance. A private final lock object prevents outside code from interfering.

FIXING THE RACE WITH SYNCHRONIZED SynchronizedDemo.java

```
class Counter {
    private int count = 0;
    private int other = 0;
    private final Object lock = new Object();

    synchronized void increment() { count++; }          // whole method locked on "this"

    void incrementOther() {
        // ... code that needs no lock could run here in parallel ...
        synchronized (lock) {                           // only the critical section
            other++;
        }
    }
    synchronized int getCount() { return count; }
    int getOther() { synchronized (lock) { return other; } }
}
public class SynchronizedDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Runnable work = () -> {
            for (int i = 0; i < 100_000; i++) { c.increment(); c.incrementOther(); }
        };
        Thread[] ts = new Thread[4];
        for (int i = 0; i < ts.length; i++) { ts[i] = new Thread(work); ts[i].start(); }
        for (Thread t : ts) t.join();
        System.out.println("count = " + c.getCount() + ", other = " + c.getOther() + " (always 400000)");
    }
}
```

OUTPUT

```
count = 400000, other = 400000 (always 400000)
```

16.5.4 Static synchronization (class-level lock)

A `static synchronized` method locks on the **Class object** (`Counter.class`), not on an instance. It protects static data shared by all objects. An instance lock and the class lock are **different** locks, so a static synchronized method and an instance synchronized method can run at the same time.

A CLASS-LEVEL LOCK FOR STATIC DATA StaticSync.java

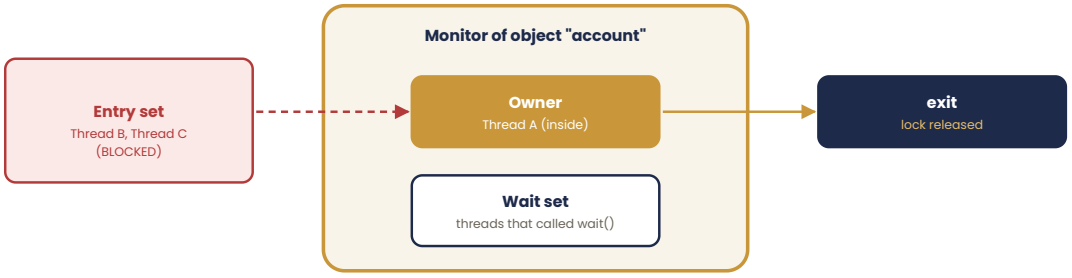
```
class Ticketing {
    private static int nextTicket = 1;                    // shared by all counters
    static synchronized int issue() { return nextTicket++; } // locks Ticketing.class
    // same as: static int issue() { synchronized (Ticketing.class) { return nextTicket++; } }
}
public class StaticSync {
    public static void main(String[] args) throws InterruptedException {
        java.util.Set<Integer> issued = java.util.concurrent.ConcurrentHashMap.newKeySet();
        Runnable counter = () -> { for (int i = 0; i < 1000; i++) issued.add(Ticketing.issue()); };
        Thread a = new Thread(counter), b = new Thread(counter), c = new Thread(counter);
        a.start(); b.start(); c.start();
        a.join(); b.join(); c.join();
        System.out.println("tickets issued: " + issued.size() + ", all unique: " + (issued.size() == 3000));
    }
}
```

OUTPUT

```
tickets issued: 3000, all unique: true
```

16.5.5 Intrinsic locks and monitors

Every Java object has a built-in lock called its **intrinsic lock** or **monitor**. `synchronized` acquires it on entry and releases it on exit. A monitor also has a **wait set** used by wait/notify (16.6). Intrinsic locks are **reentrant**: a thread that already holds a lock can enter another synchronized block on the same object again (for example, one synchronized method calling another) without deadlocking itself.



A MONITOR: ONE OWNER AT A TIME, AN ENTRY QUEUE OF BLOCKED THREADS, AND A WAIT SET

16.5.6 volatile and memory visibility

Besides atomicity there is a second problem: **visibility**. For speed, each CPU core caches values, and the JIT compiler may keep a field in a register. A thread may therefore never see another thread's write to a normal field. Declaring the field `volatile` guarantees that every read sees the latest write (reads and writes go to main memory and are not reordered). volatile does **not** make `count++` atomic; it is ideal for simple flags written by one thread and read by others.

A VOLATILE STOP FLAG VolatileFlag.java

```
public class VolatileFlag {
    private static volatile boolean running = true;     // try removing volatile: the loop may never end

    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            long loops = 0;
            while (running) loops++;                    // busy loop reading the flag
            System.out.println("worker saw running = false and stopped");
        });
        worker.start();
        Thread.sleep(200);
        running = false;                                // visible to the worker immediately
        worker.join(2000);
        System.out.println("worker alive? " + worker.isAlive());
    }
}
```

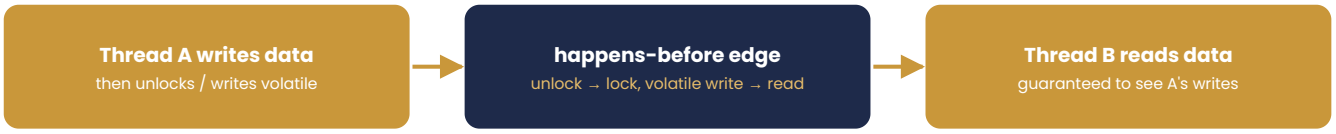
```
OUTPUT
worker saw running = false and stopped
worker alive? false
```

	`volatile`	`synchronized`
Visibility	Yes	Yes
Atomic compound actions (count++)	No	Yes
Blocking	Never blocks	Threads may block
Use for	Status flags, one writer	Read-modify-write of shared state

16.5.7 Java Memory Model and happens-before (overview)

The **Java Memory Model** (JMM) defines when a write by one thread is guaranteed to be visible to a read by another. Its central idea is the **happens-before** relationship: if action X happens-before action Y, then Y sees X's effects. Without such a relationship the compiler and CPU may reorder instructions, and other threads may see stale or surprising values. Key happens-before rules:

- **Program order**: within one thread, each action happens-before the next.
- **Monitor lock**: unlocking a lock happens-before every later lock of the same lock.
- **volatile**: a write to a volatile field happens-before every later read of that field.
- **Thread start**: `t.start()` happens-before any action in t.
- **Thread join**: all actions in t happen-before another thread returns from `t.join()`.
- **Transitivity**: if A hb B and B hb C, then A hb C.



LOCKS, VOLATILE, START AND JOIN CREATE HAPPENS-BEFORE EDGES BETWEEN THREADS

16.6 Inter-thread communication

16.6.1 wait(), notify(), notifyAll()

Sometimes a thread must **wait for a condition** that another thread will make true: a consumer waits until a queue is not empty. Busy-looping wastes CPU. Instead, Object provides:

Method	Effect
`wait()`	Releases the lock and sleeps in the object's wait set until notified (or interrupted)
`notify()`	Wakes **one** waiting thread (which must then re-acquire the lock)
`notifyAll()`	Wakes **all** waiting threads; safer when several conditions share one lock

- All three must be called while holding the object's lock (inside `synchronized (obj)`), otherwise `IllegalMonitorStateException`.
- Always call wait() in a **while loop** that re-checks the condition: a thread can wake up spuriously or find the condition already taken by another thread.

THE STANDARD WAIT PATTERN

java

```
synchronized (lock) {
    while (!conditionIsTrue)      // while, not if
        lock.wait();
    // condition holds here: do the work
}
```

16.6.2 Producer-consumer problem

One or more **producers** create items and put them into a bounded buffer; **consumers** take them out. Producers must wait when the buffer is full, consumers when it is empty. This is the classic problem that wait/notify solves (and that `BlockingQueue`, 16.8.4, solves for you).

BOUNDED BUFFER WITH WAIT/NOTIFYALL

ProducerConsumer.java

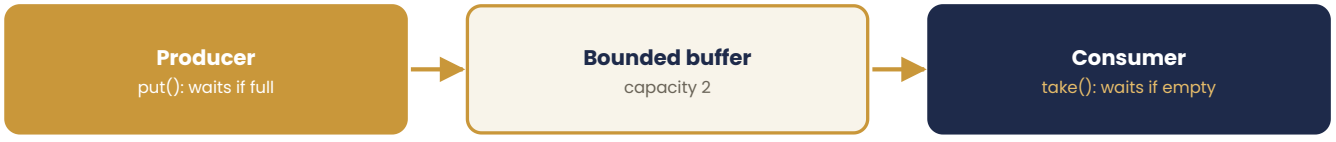
```
import java.util.LinkedList;
import java.util.Queue;

class Buffer {
    private final Queue<Integer> items = new LinkedList<>();
    private final int capacity;
    Buffer(int capacity) { this.capacity = capacity; }

    synchronized void put(int x) throws InterruptedException {
        while (items.size() == capacity) wait();          // full: wait for a consumer
        items.add(x);
        System.out.println("produced " + x + "  buffer=" + items);
        notifyAll();                                      // wake waiting consumers
    }
    synchronized int take() throws InterruptedException {
        while (items.isEmpty()) wait();                    // empty: wait for a producer
        int x = items.poll();
        System.out.println("            consumed " + x + "  buffer=" + items);
        notifyAll();                                      // wake waiting producers
        return x;
    }
}
public class ProducerConsumer {
    public static void main(String[] args) throws InterruptedException {
        Buffer buf = new Buffer(2);
        Thread producer = new Thread(() -> {
            try { for (int i = 1; i <= 5; i++) buf.put(i); } catch (InterruptedException e) { }
        });
        Thread consumer = new Thread(() -> {
            try { int sum = 0; for (int i = 1; i <= 5; i++) sum += buf.take();
                  System.out.println("consumer total = " + sum); } catch (InterruptedException e) { }
        });
        producer.start(); consumer.start();
        producer.join(); consumer.join();
    }
}
```

ONE POSSIBLE OUTPUT

```
produced 1  buffer=[1]
produced 2  buffer=[1, 2]
            consumed 1  buffer=[2]
            consumed 2  buffer=[]
produced 3  buffer=[3]
produced 4  buffer=[3, 4]
            consumed 3  buffer=[4]
            consumed 4  buffer=[]
produced 5  buffer=[5]
            consumed 5  buffer=[]
consumer total = 15
```

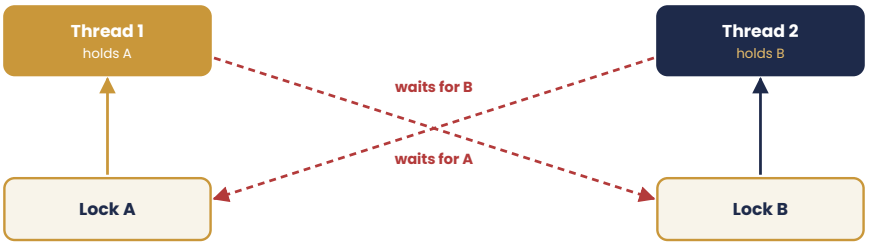


THE BUFFER DECOUPLES THE SPEED OF PRODUCERS AND CONSUMERS

16.7 Concurrency problems

16.7.1 Deadlock: cause, detection, prevention

A **deadlock** happens when two or more threads each hold a lock the other needs, and all wait forever. Thread 1 holds lock A and waits for B; thread 2 holds B and waits for A. Four conditions must all hold (Coffman conditions): mutual exclusion, hold and wait, no preemption, and circular wait.



CIRCULAR WAIT: EACH THREAD HOLDS ONE LOCK AND WAITS FOR THE OTHER

CREATING AND DETECTING A DEADLOCK

DeadlockDemo.java

```
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadMXBean;

public class DeadlockDemo {
    static final Object A = new Object(), B = new Object();

    static void pause() { try { Thread.sleep(100); } catch (InterruptedException e) { } }

    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(() -> { synchronized (A) { pause(); synchronized (B) { System.out.println("t1 done"); }
} }, "t1");
        Thread t2 = new Thread(() -> { synchronized (B) { pause(); synchronized (A) { System.out.println("t2 done"); }
} }, "t2");
        t1.setDaemon(true); t2.setDaemon(true);          // so the demo can exit
        t1.start(); t2.start();
        Thread.sleep(500);

        ThreadMXBean mx = ManagementFactory.getThreadMXBean();   // detection
        long[] ids = mx.findDeadlockedThreads();
        System.out.println("deadlocked threads found: " + (ids == null ? 0 : ids.length));
        System.out.println("t1 state: " + t1.getState() + ", t2 state: " + t2.getState());
    }
}
```

```
OUTPUT
deadlocked threads found: 2
t1 state: BLOCKED, t2 state: BLOCKED
```

Detection: a thread dump (`jstack <pid>`, or VisualVM/JConsole) prints "Found one Java-level deadlock" with the threads and locks involved; programmatically, `ThreadMXBean.findDeadlockedThreads()` as above.

Prevention:

- **Lock ordering**: every thread acquires locks in the same global order (always A before B). This removes circular wait and is the most common fix.
- **Timeouts**: use `ReentrantLock.tryLock(timeout)` and back off if the second lock is not available (16.8.2.1).
- **Fewer locks**: hold as few locks as possible, for as short a time as possible; never call unknown code while holding a lock.
- **Higher-level tools**: concurrent collections and atomic variables often remove the need for multiple locks.

16.7.2 Livelock

In a **livelock**, threads are not blocked: they keep running, but they keep reacting to each other and make no progress, like two people in a corridor who both step aside to the same side again and again. Typical cause: threads that release and retry in perfect lockstep. Fix: add **randomised back-off** or a priority between them.

16.7.3 Starvation

Starvation is when a thread never gets the resources it needs (CPU time or a lock) because other threads keep taking them, e.g. greedy threads holding a lock for long periods, or low-priority threads. Fix: keep critical sections short, use **fair** locks (`new ReentrantLock(true)`), avoid relying on priorities.

Problem	Threads are…	Progress	Typical fix
Deadlock	Blocked forever	None	Lock ordering, tryLock with timeout
Livelock	Running, busy reacting	None	Random back-off
Starvation	Waiting (some never served)	Others progress	Fair locks, short critical sections

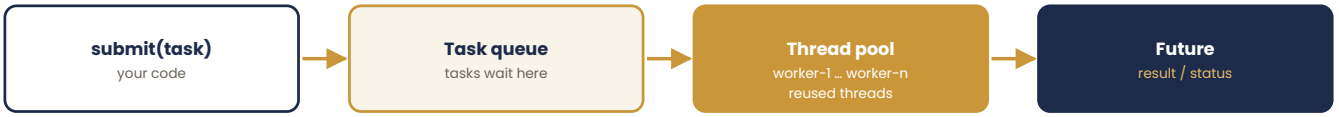
16.8 java.util.concurrent

Java 5 added the `java.util.concurrent` package (by Doug Lea): high-level, well-tested building blocks that are almost always better than raw threads, synchronized and wait/notify.

16.8.1 Executor framework

16.8.1.1 ExecutorService

Creating a new thread per task is expensive and unbounded. An `ExecutorService` manages a **pool** of reusable worker threads and a **queue** of tasks: you submit tasks, the pool runs them. It separates *what* to run from *how* it is run.



AN EXECUTORSERVICE: TASKS GO INTO A QUEUE AND ARE RUN BY POOLED THREADS

16.8.1.2 Thread pools: fixed, cached, single, scheduled

Factory method	Pool behaviour	Good for
`newFixedThreadPool(n)`	Exactly n threads; extra tasks queue	CPU-bound work (n ≈ number of cores)
`newCachedThreadPool()`	Creates threads as needed, reuses idle ones, removes after 60 s idle	Many short-lived tasks
`newSingleThreadExecutor()`	One thread; tasks run one by one in order	Sequential background work (logging)
`newScheduledThreadPool(n)`	Runs tasks after a delay or periodically	Reminders, polling, cleanup jobs
`newVirtualThreadPerTaskExecutor()`	A new virtual thread per task (Java 21)	Many blocking I/O tasks (16.10)

16.8.1.3 submit(), invokeAll(), shutdown()

SUBMIT, INVOKEALL, INVOKEANY, SHUTDOWN AND SCHEDULING

ExecutorDemo.java

```
import java.util.*;
import java.util.concurrent.*;

public class ExecutorDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(3);

        Future<Integer> f = pool.submit(() -> 6 * 7);                       // submit a Callable
        System.out.println("submit -> " + f.get());

        List<Callable<String>> jobs = new ArrayList<>();
        for (String city : List.of("Pune", "Delhi", "Chennai", "Mysore"))
            jobs.add(() -> city + " has " + city.length() + " letters");
        List<Future<String>> results = pool.invokeAll(jobs);                // run all, wait for all
        for (Future<String> r : results) System.out.println("invokeAll -> " + r.get());

        String fastest = pool.invokeAny(List.of(() -> "answer"));          // first successful result
        System.out.println("invokeAny -> " + fastest);

        pool.shutdown();                                                     // no new tasks; finish queued ones
        System.out.println("terminated in time? " + pool.awaitTermination(5, TimeUnit.SECONDS));
        try {
            pool.submit(() -> 1);
        } catch (RejectedExecutionException e) {
            System.out.println("after shutdown: RejectedExecutionException");
        }

        ScheduledExecutorService sched = Executors.newScheduledThreadPool(1);
        ScheduledFuture<String> later = sched.schedule(() -> "reminder fired", 100, TimeUnit.MILLISECONDS);
        System.out.println(later.get());
        sched.shutdown();
    }
}
```

OUTPUT

```
submit -> 42
invokeAll -> Pune has 4 letters
invokeAll -> Delhi has 5 letters
invokeAll -> Chennai has 7 letters
invokeAll -> Mysore has 6 letters
invokeAny -> answer
terminated in time? true
after shutdown: RejectedExecutionException
reminder fired
```

WATCH OUT

Always shut down an executor (`shutdown()` , or try-with-resources since Java 19, where ExecutorService is AutoCloseable). Its threads are non-daemon, so a forgotten pool keeps the JVM alive. `shutdownNow()` also interrupts running tasks.

16.8.2 Locks

16.8.2.1 ReentrantLock and tryLock

`ReentrantLock` is an explicit lock with the same basic meaning as synchronized, plus extra features: `tryLock()` (do not wait, or wait only up to a timeout), `lockInterruptibly()`, optional **fairness** (longest-waiting thread goes first) and multiple Conditions. You must unlock in a `finally` block.

DEADLOCK-FREE TRANSFERS WITH TRYLOCK

ReentrantLockDemo.java

```
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

class Account {
    private final ReentrantLock lock = new ReentrantLock();
    private double balance;
    private final String id;
    Account(String id, double b) { this.id = id; balance = b; }

    static boolean transfer(Account from, Account to, double amt) throws InterruptedException {
        if (from.lock.tryLock(100, TimeUnit.MILLISECONDS)) {          // never wait forever
            try {
                if (to.lock.tryLock(100, TimeUnit.MILLISECONDS)) {
                    try {
                        if (from.balance < amt) return false;
                        from.balance -= amt; to.balance += amt;
                        return true;
                    } finally { to.lock.unlock(); }
                }
            } finally { from.lock.unlock(); }                         // ALWAYS in finally
        }
        return false;                                                 // could not get both locks: back off
    }
    double balance() { lock.lock(); try { return balance; } finally { lock.unlock(); } }
}
public class ReentrantLockDemo {
    public static void main(String[] args) throws InterruptedException {
        Account a = new Account("A", 1000), b = new Account("B", 1000);
        Thread t1 = new Thread(() -> { for (int i = 0; i < 100; i++) try { while (!Account.transfer(a, b, 5)) { } }
catch (InterruptedException e) { } });
        Thread t2 = new Thread(() -> { for (int i = 0; i < 100; i++) try { while (!Account.transfer(b, a, 3)) { } }
catch (InterruptedException e) { } });
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("A = " + a.balance() + ", B = " + b.balance() + ", total = " + (a.balance() + b.balance()));
    }
}
```

OUTPUT

```
A = 800.0, B = 1200.0, total = 2000.0
```

16.8.2.2 ReadWriteLock

Many data structures are read far more often than written. A `ReentrantReadWriteLock` has two locks: **many readers** may hold the read lock at the same time, but the write lock is **exclusive** (no readers or other writers). This improves throughput for read-heavy caches and configuration.

SHARED READS, EXCLUSIVE WRITES ReadWriteDemo.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

class PriceCache {
    private final Map<String, Double> prices = new HashMap<>();
    private final ReadWriteLock rw = new ReentrantReadWriteLock();

    Double get(String item) {
        rw.readLock().lock();                   // shared
        try { return prices.get(item); } finally { rw.readLock().unlock(); }
    }
    void put(String item, double price) {
        rw.writeLock().lock();                  // exclusive
        try { prices.put(item, price); } finally { rw.writeLock().unlock(); }
    }
}
public class ReadWriteDemo {
    public static void main(String[] args) throws InterruptedException {
        PriceCache cache = new PriceCache();
        cache.put("pen", 10.0);
        Thread writer = new Thread(() -> { for (int i = 1; i <= 100; i++) cache.put("pen", 10.0 + i); });
        Thread[] readers = new Thread[4];
        for (int r = 0; r < readers.length; r++) {
            readers[r] = new Thread(() -> { for (int i = 0; i < 1000; i++) cache.get("pen"); });
            readers[r].start();
        }
        writer.start();
        writer.join();
        for (Thread t : readers) t.join();
        System.out.println("final pen price = " + cache.get("pen"));
    }
}
```

OUTPUT

```
final pen price = 110.0
```

16.8.2.3 Condition

A `Condition` (from `lock.newCondition()`) is the Lock equivalent of wait/notify: `await()`, `signal()`, `signalAll()`. Unlike an intrinsic monitor, one lock can have **several** conditions, e.g. "notFull" and "notEmpty", so you wake only the right kind of thread.

TWO CONDITIONS ON ONE LOCK ConditionDemo.java

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

class BoundedQueue<T> {
    private final Deque<T> q = new ArrayDeque<>();
    private final int cap;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();
    BoundedQueue(int cap) { this.cap = cap; }

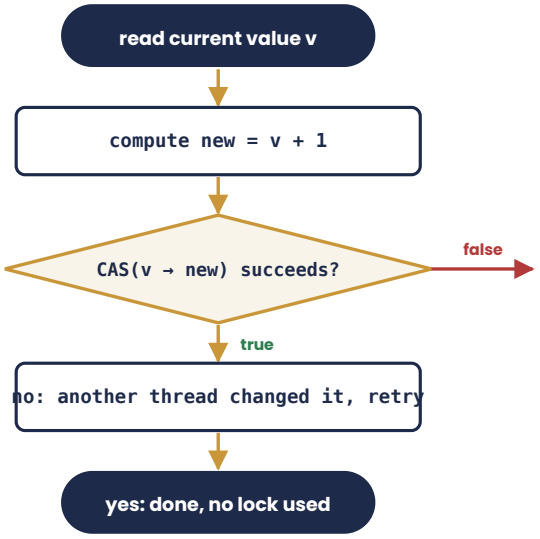
    void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (q.size() == cap) notFull.await();
            q.add(x);
            notEmpty.signal();                  // wake one consumer only
        } finally { lock.unlock(); }
    }
    T take() throws InterruptedException {
        lock.lock();
        try {
```

```
            while (q.isEmpty()) notEmpty.await();
            T x = q.poll();
            notFull.signal();               // wake one producer only
            return x;
        } finally { lock.unlock(); }
    }
}
public class ConditionDemo {
    public static void main(String[] args) throws InterruptedException {
        BoundedQueue<String> bq = new BoundedQueue<>(1);
        Thread consumer = new Thread(() -> {
            try { for (int i = 0; i < 3; i++) System.out.println("took " + bq.take()); } catch (InterruptedException e)
{ }
        });
        consumer.start();
        for (String s : new String[] {"order-1", "order-2", "order-3"}) bq.put(s);
        consumer.join();
    }
}
```

```
OUTPUT
took order-1
took order-2
took order-3
```

16.8.3 Atomic variables: AtomicInteger, AtomicLong, CAS

`java.util.concurrent.atomic` provides lock-free thread-safe variables. `AtomicInteger.incrementAndGet()` is an atomic count++. They are built on the CPU instruction **CAS** (compare-and-swap): "if the value is still X, set it to Y; otherwise tell me it changed". On failure the operation simply retries with the new value. No thread ever blocks, which is very fast under moderate contention. `LongAdder` is even faster for heavily contended counters.



COMPARE-AND-SWAP LOOP BEHIND INCREMENTANDGET()

LOCK-FREE COUNTERS

AtomicDemo.java

```java
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class AtomicDemo {
    public static void main(String[] args) throws InterruptedException {
        AtomicInteger hits = new AtomicInteger();
        AtomicLong bytes = new AtomicLong();
        LongAdder adder = new LongAdder();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int t = 0; t < 8; t++)
            pool.submit(() -> {
                for (int i = 0; i < 10_000; i++) {
                    hits.incrementAndGet();              // atomic ++
                    bytes.addAndGet(512);
                    adder.increment();
                }
            });
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("hits  = " + hits.get());
        System.out.println("bytes = " + bytes.get());
        System.out.println("adder = " + adder.sum());

        AtomicInteger x = new AtomicInteger(5);
        System.out.println("CAS 5->10: " + x.compareAndSet(5, 10) + ", now " + x.get());
        System.out.println("CAS 5->20: " + x.compareAndSet(5, 20) + ", still " + x.get());
        System.out.println("updateAndGet(*3): " + x.updateAndGet(v -> v * 3));
    }
}
```

OUTPUT

```
hits  = 80000
bytes = 40960000
adder = 80000
CAS 5->10: true, now 10
CAS 5->20: false, still 10
updateAndGet(*3): 30
```

16.8.4 Concurrent collections: ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue

Class	How it is thread-safe	Best for
`ConcurrentHashMap`	Fine-grained locking/CAS per bucket; reads never block; atomic `merge`, `compute`, `putIfAbsent`	Shared caches, counters by key
`CopyOnWriteArrayList`	Every write copies the whole array; iteration uses a snapshot	Rarely modified, often read lists (listeners)
`BlockingQueue` (ArrayBlockingQueue, LinkedBlockingQueue)	`put()` blocks when full, `take()` blocks when empty	Producer-consumer pipelines
`ConcurrentLinkedQueue`	Lock-free CAS queue	Non-blocking queues
`ConcurrentSkipListMap/Set`	Lock-free sorted structure	Concurrent sorted maps

THREAD-SAFE COUNTING AND A BLOCKING HAND-OFF ConcurrentCollections.java

```
import java.util.*;
import java.util.concurrent.*;

public class ConcurrentCollections {
    public static void main(String[] args) throws Exception {
        ConcurrentHashMap<String, Integer> votes = new ConcurrentHashMap<>();
        ExecutorService pool = Executors.newFixedThreadPool(4);
        String[] options = {"Java", "Python", "Java", "C++", "Java", "Python"};
        for (int t = 0; t < 4; t++)
            pool.submit(() -> { for (String o : options) votes.merge(o, 1, Integer::sum); });   // atomic per key
        pool.shutdown(); pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("votes: " + new TreeMap<>(votes));

        BlockingQueue<String> q = new ArrayBlockingQueue<>(2);
        Thread producer = new Thread(() -> {
            try { for (String s : List.of("a", "b", "c", "DONE")) q.put(s); } catch (InterruptedException e) { }
        });
        producer.start();
        String item;
        StringBuilder got = new StringBuilder();
        while (!(item = q.take()).equals("DONE")) got.append(item);      // blocks until available
        System.out.println("taken from BlockingQueue: " + got);
    }
}
```

OUTPUT

```
votes: {C++=4, Java=12, Python=8}
taken from BlockingQueue: abc
```

16.8.5 Synchronizers: CountDownLatch, CyclicBarrier, Semaphore, Phaser

Synchronizer	Idea	Example
`CountDownLatch(n)`	Threads wait until a counter reaches zero; **one-shot**	Main waits until 3 services have started
`CyclicBarrier(n)`	n threads wait for each other at a barrier, then all continue; **reusable**	Simulation steps; all players ready
`Semaphore(n)`	n permits: at most n threads at a time in a section	Max 3 concurrent DB connections / parking spots
`Phaser`	Flexible reusable barrier with a changing number of parties and phases	Multi-phase tasks where workers join/leave

FOUR SYNCHRONIZERS IN ACTION Synchronizers.java

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Synchronizers {
    public static void main(String[] args) throws Exception {
        // CountDownLatch: wait for 3 services
        CountDownLatch ready = new CountDownLatch(3);
        for (String svc : new String[] {"db", "cache", "auth"})
            new Thread(() -> ready.countDown()).start();
        ready.await();
        System.out.println("latch: all 3 services ready, count = " + ready.getCount());

        // CyclicBarrier: 3 runners wait for each other, barrier action runs once per trip
        CyclicBarrier start = new CyclicBarrier(3, () -> System.out.println("barrier: all runners at the line, GO!"));
        Thread[] runners = new Thread[3];
        for (int i = 0; i < 3; i++) {
            runners[i] = new Thread(() -> { try { start.await(); } catch (Exception e) { } });
            runners[i].start();
        }
        for (Thread r : runners) r.join();

        // Semaphore: at most 2 threads in the "parking lot"
        Semaphore parking = new Semaphore(2);
```

```
        AtomicInteger inside = new AtomicInteger(), maxInside = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(6);
        for (int i = 0; i < 6; i++)
            pool.submit(() -> {
                try {
                    parking.acquire();
                    int now = inside.incrementAndGet();
                    maxInside.accumulateAndGet(now, Math::max);
                    Thread.sleep(50);
                    inside.decrementAndGet();
                    parking.release();
                } catch (InterruptedException e) { }
            });
        pool.shutdown(); pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("semaphore: max cars parked at once = " + maxInside.get());

        // Phaser: 2 workers, 2 phases
        Phaser phaser = new Phaser(1);                    // main registers itself
        for (int w = 0; w < 2; w++) {
            phaser.register();
            new Thread(() -> { phaser.arriveAndAwaitAdvance(); phaser.arriveAndDeregister(); }).start();
        }
        phaser.arriveAndAwaitAdvance();
        System.out.println("phaser: phase 0 complete, now phase " + phaser.getPhase());
        phaser.arriveAndDeregister();
    }
}
```

OUTPUT

```
latch: all 3 services ready, count = 0
barrier: all runners at the line, GO!
semaphore: max cars parked at once = 2
phaser: phase 0 complete, now phase 1
```

16.8.6 ThreadLocal

A `ThreadLocal<T>` gives **each thread its own independent copy** of a variable. No sharing means no synchronization. Common uses: per-thread non-thread-safe objects (`SimpleDateFormat`, a `Random`), and per-request context in web servers (current user, transaction). In thread pools, always `remove()` the value when the task finishes, or it leaks into the next task that reuses the thread.

PER-THREAD REQUEST CONTEXT

ThreadLocalDemo.java

```
import java.util.concurrent.*;

public class ThreadLocalDemo {
    private static final ThreadLocal<String> currentUser = ThreadLocal.withInitial(() -> "guest");

    static void handleRequest(String user) {
        currentUser.set(user);
        try {
            audit("viewed dashboard");
        } finally {
            currentUser.remove();                        // essential with thread pools
        }
    }
    static void audit(String action) {                  // no user parameter needed
        System.out.println(currentUser.get() + " " + action);
    }
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newSingleThreadExecutor();
        pool.submit(() -> handleRequest("asha"));
        pool.submit(() -> handleRequest("ravi"));
        pool.submit(() -> audit("(no user set)"));       // value was removed: initial value
        pool.shutdown(); pool.awaitTermination(5, TimeUnit.SECONDS);
    }
}
```

OUTPUT

```
asha viewed dashboard
ravi viewed dashboard
guest (no user set)
```

16.9 Asynchronous programming

16.9.1 CompletableFuture: supplyAsync, thenApply, thenCombine, exceptionally

`Future.get()` blocks, and futures cannot be chained. `CompletableFuture` (Java 8) lets you describe a **pipeline** of asynchronous steps that run when the previous result is ready, without blocking threads: fetch user → fetch orders → compute total → show. Steps run in the common ForkJoinPool or an executor you supply.

Method	Meaning
`supplyAsync(supplier)` / `runAsync`	Start a task asynchronously (with / without result)
`thenApply(fn)`	Transform the result (like map)
`thenAccept(consumer)`	Consume the result
`thenCompose(fn)`	Chain another async call (like flatMap)
`thenCombine(other, fn)`	Combine results of two independent futures
`allOf(...)` / `anyOf(...)`	Wait for all / the first
`exceptionally(fn)` / `handle(fn)`	Recover from an exception in the pipeline
`join()` / `get()`	Wait for the final result

AN ASYNC PIPELINE WITH COMBINATION AND ERROR RECOVERY

CompletableFutureDemo.java

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureDemo {
    static int fetchPrice(String item) {
        sleep(100);
        return switch (item) { case "laptop" -> 55000; case "mouse" -> 800; default -> throw new
IllegalArgumentException("unknown item " + item); };
    }
    static double fetchDiscount() { sleep(80); return 0.10; }
    static void sleep(long ms) { try { Thread.sleep(ms); } catch (InterruptedException e) { } }

    public static void main(String[] args) {
        CompletableFuture<Integer> price = CompletableFuture.supplyAsync(() -> fetchPrice("laptop"));
        CompletableFuture<Double> discount = CompletableFuture.supplyAsync(CompletableFutureDemo::fetchDiscount);  //
runs in parallel

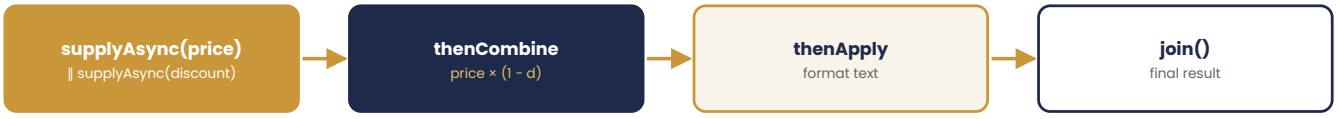
        CompletableFuture<String> bill = price
                .thenCombine(discount, (p, d) -> p * (1 - d))          // combine two results
                .thenApply(total -> String.format("total after discount: %.2f", total));
        System.out.println(bill.join());

        String r = CompletableFuture.supplyAsync(() -> fetchPrice("phone"))
                .thenApply(p -> "price " + p)
                .exceptionally(ex -> "fallback: " + ex.getCause().getMessage())   // recover
                .join();
        System.out.println(r);

        CompletableFuture<Void> all = CompletableFuture.allOf(
                CompletableFuture.runAsync(() -> sleep(50)), CompletableFuture.runAsync(() -> sleep(60)));
        all.join();
        System.out.println("allOf completed: " + all.isDone());
    }
}
```

OUTPUT

```
total after discount: 49500.00
fallback: unknown item phone
allOf completed: true
```



A COMPLETABLEFUTURE PIPELINE

16.9.2 Fork/Join framework and RecursiveTask

The **Fork/Join** framework (Java 7) parallelises **divide-and-conquer** work: a task that is too big **forks** into subtasks, which run in parallel, and then **joins** their results. Its `ForkJoinPool` uses **work stealing**: idle threads steal queued subtasks from busy ones, keeping all cores busy. Extend `RecursiveTask<V>` (returns a value) or `RecursiveAction` (no value). Parallel streams (17.5.7) use this framework internally.

PARALLEL SUM WITH RECURSIVETASK

ForkJoinDemo.java

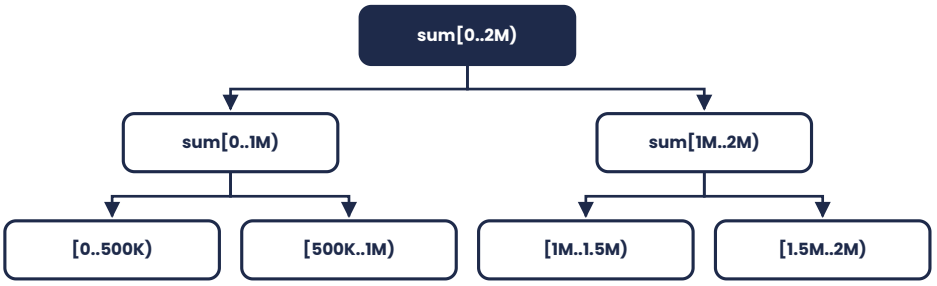
```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

class SumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000;
    private final int[] data; private final int lo, hi;
    SumTask(int[] data, int lo, int hi) { this.data = data; this.lo = lo; this.hi = hi; }

    @Override protected Long compute() {
        if (hi - lo <= THRESHOLD) {                        // small enough: do it directly
            long s = 0;
            for (int i = lo; i < hi; i++) s += data[i];
            return s;
        }
        int mid = (lo + hi) >>> 1;
        SumTask left = new SumTask(data, lo, mid), right = new SumTask(data, mid, hi);
        left.fork();                                       // run left asynchronously
        long r = right.compute();                          // do right in this thread
        return left.join() + r;                            // wait for left and combine
    }
}
public class ForkJoinDemo {
    public static void main(String[] args) {
        int[] data = new int[2_000_000];
        for (int i = 0; i < data.length; i++) data[i] = i % 100;
        long parallel = ForkJoinPool.commonPool().invoke(new SumTask(data, 0, data.length));
        long sequential = 0;
        for (int x : data) sequential += x;
        System.out.println("fork/join sum = " + parallel + ", sequential = " + sequential + ", equal: " + (parallel ==
sequential));
    }
}
```

OUTPUT

```
fork/join sum = 99000000, sequential = 99000000, equal: true
```



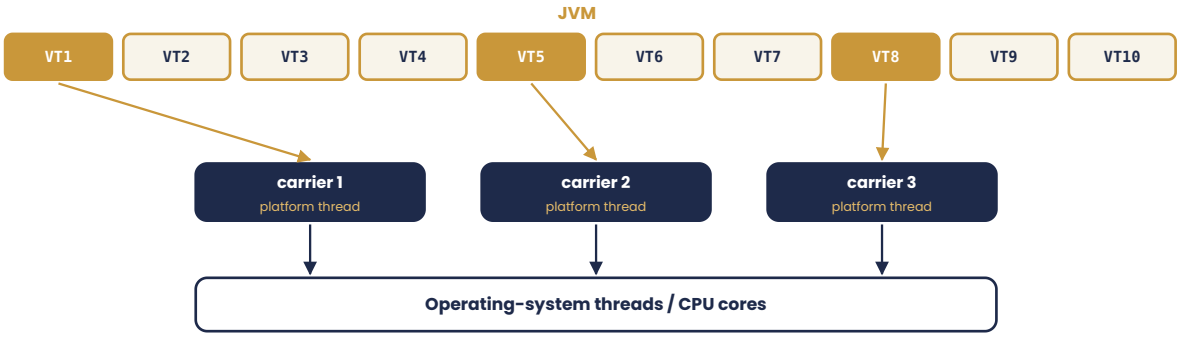
FORK SPLITS THE RANGE UNTIL IT IS BELOW THE THRESHOLD; JOIN ADDS THE RESULTS BACK UP

16.10 Virtual threads (Java 21+)

16.10.1 Platform threads vs virtual threads

A classic Java **platform thread** is a thin wrapper around an **operating-system thread**: about 1 MB of stack, expensive to create, and a machine can run only a few thousand. A server that uses one thread per request, where each request mostly **waits** for a database or web service, runs out of threads long before the CPU is busy.

A **virtual thread** (Project Loom, final in Java 21) is a lightweight thread managed by the **JVM**. Many virtual threads are **mounted** onto a small pool of platform "carrier" threads. When a virtual thread blocks on I/O or sleep, the JVM **unmounts** it and the carrier runs another virtual thread. Creating millions of them is cheap, so the simple thread-per-request style scales again.



MANY VIRTUAL THREADS (GOLD = CURRENTLY MOUNTED) SHARE A FEW CARRIER PLATFORM THREADS

	Platform thread	Virtual thread
Managed by	Operating system	JVM
Memory	~1 MB stack, fixed	A few KB, grows as needed
How many	Thousands	Millions
Creation cost	High; use pools	Very low; create one per task, never pool
Blocking I/O	Blocks an OS thread	Unmounts; carrier is freed
Best for	CPU-heavy work	Many concurrent blocking tasks (web requests, DB, HTTP calls)

16.10.2 Creating virtual threads

CREATING VIRTUAL THREADS; 10,000 BLOCKING TASKS IN ABOUT A SECOND

VirtualThreads.java

```
import java.time.Duration;
import java.util.concurrent.*;
import java.util.stream.IntStream;

public class VirtualThreads {
    public static void main(String[] args) throws Exception {
        Thread vt = Thread.ofVirtual().name("virtual-1").start(() ->
                System.out.println("hello from " + Thread.currentThread().getName() + ", virtual? " +
Thread.currentThread().isVirtual()));
        vt.join();

        Thread vt2 = Thread.startVirtualThread(() -> System.out.println("startVirtualThread works too"));
        vt2.join();

        long start = System.nanoTime();
        try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {     // one virtual thread per task
            IntStream.range(0, 10_000).forEach(i -> exec.submit(() -> {
                Thread.sleep(Duration.ofSeconds(1));                                    // simulated blocking I/O
                return i;
            }));
        }                                                                               // close() waits for all tasks
        long ms = (System.nanoTime() - start) / 1_000_000;
        System.out.println("10,000 tasks that each block 1 s finished in under 5 s: " + (ms < 5000));
    }
}
```

OUTPUT

```
hello from virtual-1, virtual? true
startVirtualThread works too
10,000 tasks that each block 1 s finished in under 5 s: true
```

TIP

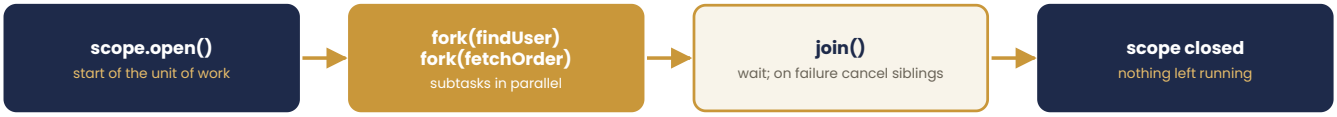
With virtual threads, keep writing plain blocking code. Avoid long `synchronized` blocks around blocking I/O in older JDKs (they "pin" the carrier; Java 24 removed most pinning), and do not pool virtual threads.

16.10.3 Structured concurrency (overview)

Structured concurrency treats a group of related subtasks as one unit of work with a clear lifetime: they are started inside a scope, and the scope does not end until all have finished. If one fails, the others are cancelled automatically; nothing leaks. The API is `StructuredTaskScope`, a **preview** feature in Java 21–25 (its shape changed in Java 25), so it requires `--enable-preview`.

STRUCTURED CONCURRENCY (PREVIEW; NEEDS JAVA 25 WITH --ENABLE-PREVIEW) java

```java
// Java 25 preview API (compile and run with --enable-preview)
Response handle() throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {              // all subtasks belong to this scope
        Subtask<User>  user  = scope.fork(() -> findUser());    // run in parallel (virtual threads)
        Subtask<Order> order = scope.fork(() -> fetchOrder());
        scope.join();                                           // wait for both; if one fails, cancel the other
        return new Response(user.get(), order.get());
    }                                                           // no subtask can outlive the scope
}
```



SUBTASKS LIVE AND DIE INSIDE THEIR SCOPE

PRACTICE CHECKPOINT

A **bank account updated by 10 threads**, shown wrong without synchronization and correct with it, plus a **producer-consumer** with BlockingQueue.

SAMPLE SOLUTION ConcurrencyCheckpoint.java

```java
import java.util.concurrent.*;

class UnsafeAccount {
    private int balance = 0;
    void deposit(int amt) {                                 // read-modify-write: not atomic
        int b = balance;
        Thread.yield();                                     // widen the race window for the demo
        balance = b + amt;
    }
    int getBalance() { return balance; }
}
class SafeAccount {
    private int balance = 0;
    synchronized void deposit(int amt) { balance += amt; }  // one thread at a time
    synchronized int getBalance() { return balance; }
}

public class ConcurrencyCheckpoint {
    static final int THREADS = 10, DEPOSITS = 2_000;

    public static void main(String[] args) throws Exception {
        // Part 1: 10 threads x 10,000 deposits of 1
        UnsafeAccount unsafe = new UnsafeAccount();
        SafeAccount safe = new SafeAccount();
        Thread[] ts = new Thread[THREADS];
        for (int i = 0; i < THREADS; i++) {
            ts[i] = new Thread(() -> { for (int j = 0; j < DEPOSITS; j++) { unsafe.deposit(1); safe.deposit(1); } });
            ts[i].start();
        }
        for (Thread t : ts) t.join();
        int expected = THREADS * DEPOSITS;
        System.out.println("expected balance : " + expected);
        System.out.println("unsafe balance   : " + unsafe.getBalance() + "  (updates lost to race conditions)");
        System.out.println("safe balance     : " + safe.getBalance() + "  correct? " + (safe.getBalance() ==
expected));

        // Part 2: producer-consumer with BlockingQueue
```

```
        BlockingQueue<Integer> orders = new LinkedBlockingQueue<>(5);
        final int POISON = -1;
        Thread producer = new Thread(() -> {
            try {
                for (int id = 1; id <= 8; id++) orders.put(id);    // blocks when the queue is full
                orders.put(POISON);                                // signal: no more orders
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }, "producer");
        Callable<Integer> consumer = () -> {
            int processed = 0;
            while (true) {
                int id = orders.take();                            // blocks when empty
                if (id == POISON) break;
                processed++;
            }
            return processed;
        };
        ExecutorService pool = Executors.newSingleThreadExecutor();
        producer.start();
        Future<Integer> done = pool.submit(consumer);
        System.out.println("orders processed : " + done.get());
        producer.join();
        pool.shutdown();
    }
}
```

OUTPUT

```
expected balance : 20000
unsafe balance   : 3575  (updates lost to race conditions)
safe balance     : 20000  correct? true
orders processed : 8
```

MODULE 17 · ADVANCED

Functional programming in Java (Java 8+)

Minimum duration **5 days** | Level **Advanced**

Java 8 added lambdas and streams, so behaviour can be passed as data and collections processed declaratively.

IN THIS MODULE

17.1 Functional programming concepts
17.2 Lambda expressions
17.3 Functional interfaces
17.4 Method references
17.5 Stream API
17.6 Optional

17.1 Functional programming concepts

17.1.1 Imperative vs declarative style

Imperative code says *how* to do something, step by step: create a list, loop, check, add, count. **Declarative** code says *what* result you want and lets the library decide how: "filter the passed students, map to names, sort, collect". Declarative code is shorter, reads like the problem statement, and is easier to parallelise.

THE SAME TASK IN TWO STYLES — ImperativeVsDeclarative.java

```
import java.util.*;
import java.util.stream.*;

public class ImperativeVsDeclarative {
    record Student(String name, int marks) { }

    public static void main(String[] args) {
        List<Student> students = List.of(new Student("Ravi", 45), new Student("Asha", 91),
                new Student("Meera", 78), new Student("Kiran", 32), new Student("Dev", 66));

        // imperative: HOW
        List<String> passed1 = new ArrayList<>();
        for (Student s : students)
            if (s.marks() >= 50) passed1.add(s.name().toUpperCase());
        Collections.sort(passed1);
        System.out.println("imperative : " + passed1);

        // declarative: WHAT
        List<String> passed2 = students.stream()
                .filter(s -> s.marks() >= 50)
                .map(s -> s.name().toUpperCase())
                .sorted()
                .toList();
        System.out.println("declarative: " + passed2);
    }
}
```

OUTPUT

```
imperative : [ASHA, DEV, MEERA]
declarative: [ASHA, DEV, MEERA]
```

17.1.2 Pure functions, immutability, side effects

Pure function

Its result depends **only** on its inputs, and it changes nothing outside itself. `square(x)` always returns the same value for the same x. Easy to test, cache and run in parallel.

Side effect

Anything a function does besides returning a value: modifying a field or a collection, printing, writing a file, changing a global counter.

Immutability

Data is not changed after creation; "changing" creates a new value (String, records, List.of). Immutable data can be shared safely between threads.

PURE VS IMPURE

PureFunctions.java

```
import java.util.ArrayList;
import java.util.List;

public class PureFunctions {
    static int total = 0;

    static int addPure(int a, int b) { return a + b; }            // pure

    static int addImpure(int a) { total += a; return total; }     // side effect: changes total

    static List<Integer> withBonusPure(List<Integer> marks) {      // returns a NEW list
        List<Integer> out = new ArrayList<>();
        for (int m : marks) out.add(m + 5);
        return out;
    }
    public static void main(String[] args) {
        System.out.println(addPure(2, 3) + " " + addPure(2, 3) + "   (always the same)");
        System.out.println(addImpure(2) + " " + addImpure(2) + "   (same call, different results)");
        List<Integer> marks = List.of(60, 70);
        System.out.println("bonus: " + withBonusPure(marks) + ", original untouched: " + marks);
    }
}
```

OUTPUT

```
5 5   (always the same)
2 4   (same call, different results)
bonus: [65, 75], original untouched: [60, 70]
```

17.2 Lambda expressions

17.2.1 Syntax: (params) -> expression and block bodies

A **lambda expression** is an anonymous function: a block of behaviour you can store in a variable or pass to a method. It has a parameter list, the arrow `->`, and a body.

Form	Example
No parameters	`() -> System.out.println("hi")`
One parameter (parentheses optional)	`x -> x * x`
Several parameters	`(a, b) -> a + b`
Explicit types	`(int a, int b) -> a + b`
`var` types (Java 11, allows annotations)	`(var a, var b) -> a + b`
Block body (needs return)	`(a, b) -> { int s = a + b; return s * 2; }`



ANATOMY OF A LAMBDA

17.2.2 Replacing anonymous inner classes

A lambda can be used wherever a **functional interface** (an interface with exactly one abstract method) is expected. Before Java 8 the same thing needed an anonymous inner class (10.12.4), five lines of boilerplate for one line of logic.

FIVE LINES BECOME ONE LambdaVsAnonymous.java

```
import java.util.*;

public class LambdaVsAnonymous {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(List.of("Meenakshi", "Om", "Ravi", "Asha"));

        Collections.sort(names, new Comparator<String>() {            // before Java 8
            @Override
            public int compare(String a, String b) { return a.length() - b.length(); }
        });
        System.out.println("anonymous class: " + names);

        names.sort((a, b) -> b.length() - a.length());                // lambda
        System.out.println("lambda         : " + names);

        Runnable r1 = new Runnable() { public void run() { System.out.println("run (anonymous)"); } };
        Runnable r2 = () -> System.out.println("run (lambda)");
        r1.run(); r2.run();
    }
}
```

```
OUTPUT
anonymous class: [Om, Ravi, Asha, Meenakshi]
lambda         : [Meenakshi, Ravi, Asha, Om]
run (anonymous)
run (lambda)
```

17.2.3 Effectively final variables and capture

A lambda can use (**capture**) local variables of the enclosing method, but only if they are **final or effectively final**: assigned once and never changed. The lambda may run later, possibly on another thread, so Java copies the value and forbids changes that would make the copy stale. Fields and array elements can be modified (but that is a side effect, best avoided).

CAPTURING LOCAL VARIABLES

Capture.java

```
import java.util.List;
import java.util.function.Supplier;

public class Capture {
    public static void main(String[] args) {
        int bonus = 5;                                  // effectively final
        List<Integer> marks = List.of(60, 70, 80);
        marks.forEach(m -> System.out.print((m + bonus) + " "));
        System.out.println();
        // bonus = 10;     // would make the lambda above a compile error:
        //                 // "local variables referenced from a lambda expression must be final or effectively final"

        int[] counter = {0};                            // a workaround (avoid in real code)
        marks.forEach(m -> counter[0]++);
        System.out.println("counted " + counter[0]);

        Supplier<String> greeter = makeGreeter("Asha");
        System.out.println(greeter.get());              // the captured value lives on
    }
    static Supplier<String> makeGreeter(String name) {
        return () -> "Hello, " + name + "!";            // captures parameter "name"
    }
}
```

OUTPUT

```
65 75 85
counted 3
Hello, Asha!
```

17.2.4 this inside a lambda

A lambda does **not** create a new scope for `this`: inside a lambda, `this` refers to the **enclosing object**. In an anonymous class, `this` refers to the anonymous object itself.

THIS MEANS DIFFERENT THINGS

ThisInLambda.java

```
public class ThisInLambda {
    private final String name = "outer object";

    void demo() {
        Runnable lambda = () -> System.out.println("lambda this    -> " + this.name);
        Runnable anon = new Runnable() {
            private final String name = "anonymous object";
            public void run() { System.out.println("anonymous this -> " + this.name); }
        };
        lambda.run();
        anon.run();
    }
    public static void main(String[] args) { new ThisInLambda().demo(); }
}
```

OUTPUT

```
lambda this    -> outer object
anonymous this -> anonymous object
```

17.3 Functional interfaces

17.3.1 @FunctionalInterface

A **functional interface** has exactly **one abstract method** (any number of default and static methods). The `@FunctionalInterface` annotation is optional but asks the compiler to verify the rule, so adding a second abstract method later becomes a compile error.

A CUSTOM FUNCTIONAL INTERFACE CustomFunctional.java

```
@FunctionalInterface
interface Discount {
    double apply(double price);                                  // the single abstract method
    default Discount then(Discount next) { return p -> next.apply(apply(p)); }   // default: allowed
    static Discount none() { return p -> p; }                    // static: allowed
}
public class CustomFunctional {
    public static void main(String[] args) {
        Discount festival = p -> p * 0.9;                        // 10% off
        Discount coupon = p -> p - 200;                          // flat 200 off
        System.out.println("festival : " + festival.apply(5000));
        System.out.println("both     : " + festival.then(coupon).apply(5000));
        System.out.println("none     : " + Discount.none().apply(5000));
    }
}
```

OUTPUT

```
festival : 4500.0
both     : 4300.0
none     : 5000.0
```

17.3.2 Built-in interfaces in java.util.function

You rarely need to write your own: `java.util.function` provides 43 ready-made functional interfaces. Learn the core six; the rest are variations.

Interface	Method	Takes → Returns	Example use
`Predicate<T>`	`test(T)`	T → boolean	filter: is the student passed?
`Function<T,R>`	`apply(T)`	T → R	map: student → name
`Consumer<T>`	`accept(T)`	T → nothing	forEach: print it
`Supplier<T>`	`get()`	nothing → T	create default values lazily
`UnaryOperator<T>`	`apply(T)`	T → T	replaceAll: trim each string
`BinaryOperator<T>`	`apply(T,T)`	(T, T) → T	reduce: sum two numbers
`BiFunction<T,U,R>`, `BiConsumer`, `BiPredicate`	…	two inputs	Map.forEach((k, v) -> …)

17.3.2.1 Predicate (test)

17.3.2.2 Function (apply) and BiFunction

17.3.2.3 Consumer (accept) and BiConsumer

17.3.2.4 Supplier (get)

17.3.2.5 UnaryOperator and BinaryOperator

THE CORE FUNCTIONAL INTERFACES

BuiltInFunctional.java

```
import java.time.LocalDate;
import java.util.*;
import java.util.function.*;

public class BuiltInFunctional {
    public static void main(String[] args) {
        Predicate<Integer> isPass = m -> m >= 50;                              // 17.3.2.1
        System.out.println("Predicate  : 72 passes? " + isPass.test(72) + ", 31? " + isPass.test(31));

        Function<String, Integer> length = String::length;                     // 17.3.2.2
        BiFunction<Integer, Integer, Double> percent = (got, max) -> got * 100.0 / max;
        System.out.println("Function   : length(\"Learn\") = " + length.apply("Learn"));
        System.out.println("BiFunction : 45/60 = " + percent.apply(45, 60) + "%");

        Consumer<String> shout = s -> System.out.println("Consumer   : " + s.toUpperCase() + "!");   // 17.3.2.3
        shout.accept("hello");
        BiConsumer<String, Integer> show = (k, v) -> System.out.println("BiConsumer : " + k + " -> " + v);
        Map.of("Java", 15000).forEach(show);

        Supplier<LocalDate> deadline = () -> LocalDate.of(2026, 12, 31);       // 17.3.2.4
        Supplier<List<String>> newList = ArrayList::new;
        System.out.println("Supplier   : " + deadline.get() + ", " + newList.get());

        UnaryOperator<String> clean = s -> s.trim().toLowerCase();             // 17.3.2.5
        BinaryOperator<Integer> max = BinaryOperator.maxBy(Comparator.naturalOrder());
        List<String> raw = new ArrayList<>(List.of("  JAVA ", " Sql"));
        raw.replaceAll(clean);
        System.out.println("UnaryOp    : " + raw);
        System.out.println("BinaryOp   : max(17, 42) = " + max.apply(17, 42) + ", sum = " + ((BinaryOperator<Integer>)
Integer::sum).apply(17, 42));
    }
}
```

OUTPUT

```
Predicate  : 72 passes? true, 31? false
Function   : length("Learn") = 5
BiFunction : 45/60 = 75.0%
Consumer   : HELLO!
BiConsumer : Java -> 15000
Supplier   : 2026-12-31, []
UnaryOp    : [java, sql]
BinaryOp   : max(17, 42) = 42, sum = 59
```

17.3.2.6 Primitive specializations (IntPredicate, ToIntFunction)

Using `Function<Integer, Integer>` boxes every int into an Integer object, which is slow in hot loops. Primitive versions avoid boxing: `IntPredicate`, `IntFunction<R>`, `ToIntFunction<T>`, `IntUnaryOperator`, `IntBinaryOperator`, `IntSupplier`, `IntConsumer` and the Long/Double equivalents.

BOXING-FREE FUNCTIONAL INTERFACES

PrimitiveFunctional.java

```
import java.util.function.*;

public class PrimitiveFunctional {
    record Product(String name, int price) { }
    public static void main(String[] args) {
        IntPredicate even = n -> n % 2 == 0;
        IntUnaryOperator square = n -> n * n;
        IntBinaryOperator gcd = (a, b) -> { while (b != 0) { int t = b; b = a % b; a = t; } return a; };
        ToIntFunction<Product> price = Product::price;            // object -> int, no boxing
        DoubleSupplier pi = () -> Math.PI;
        System.out.println(even.test(10) + " " + square.applyAsInt(9) + " " + gcd.applyAsInt(48, 18)
                + " " + price.applyAsInt(new Product("pen", 25)) + " " + pi.getAsDouble());
    }
}
```

OUTPUT

```
true 81 6 25 3.141592653589793
```

17.3.3 Composing: andThen(), compose(), and(), or(), negate()

Functional interfaces have default methods for building bigger functions from smaller ones.

Method	On	Meaning
`f.andThen(g)`	Function, Consumer	first f, then g: g(f(x))
`f.compose(g)`	Function	first g, then f: f(g(x))
`p.and(q)`, `p.or(q)`	Predicate	logical AND / OR
`p.negate()`, `Predicate.not(p)`	Predicate	logical NOT
`Function.identity()`	Function	returns its input unchanged

BUILDING FUNCTIONS FROM FUNCTIONS Composing.java

```
import java.util.List;
import java.util.function.*;

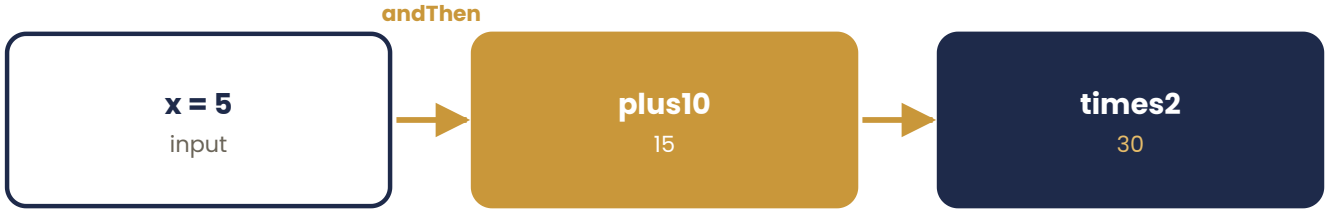
public class Composing {
    public static void main(String[] args) {
        Function<Integer, Integer> plus10 = x -> x + 10;
        Function<Integer, Integer> times2 = x -> x * 2;
        System.out.println("plus10.andThen(times2)(5) = " + plus10.andThen(times2).apply(5));   // (5+10)*2
        System.out.println("plus10.compose(times2)(5) = " + plus10.compose(times2).apply(5));   // 5*2+10

        Predicate<String> notEmpty = s -> !s.isEmpty();
        Predicate<String> shortWord = s -> s.length() <= 4;
        Predicate<String> startsJ = s -> s.startsWith("J");
        List<String> words = List.of("Java", "JavaScript", "", "SQL", "Jira", "Kotlin");
        System.out.println("notEmpty AND short : " + words.stream().filter(notEmpty.and(shortWord)).toList());
        System.out.println("startsJ OR short   : " + words.stream().filter(startsJ.or(shortWord)).toList());
        System.out.println("NOT startsJ        : " + words.stream().filter(startsJ.negate()).toList());
        System.out.println("Predicate.not      : " + words.stream().filter(Predicate.not(String::isEmpty)).count() + "
non-empty");

        Consumer<String> log = s -> System.out.print("[log " + s + "] ");
        Consumer<String> save = s -> System.out.println("[saved " + s + "]");
        log.andThen(save).accept("order-7");
    }
}
```

OUTPUT

```
plus10.andThen(times2)(5) = 30
plus10.compose(times2)(5) = 20
notEmpty AND short : [Java, SQL, Jira]
startsJ OR short   : [Java, JavaScript, , SQL, Jira]
NOT startsJ        : [, SQL, Kotlin]
Predicate.not      : 5 non-empty
[log order-7] [saved order-7]
```



PLUS10.ANDTHEN(TIMES2): THE OUTPUT OF ONE FUNCTION FEEDS THE NEXT

17.4 Method references

When a lambda only calls one existing method, a **method reference** `X::method` says the same thing more briefly. There are four kinds.

Kind	Syntax	Lambda equivalent
17.4.1 Static method	`Integer::parseInt`	`s -> Integer.parseInt(s)`
17.4.2 Instance method of a particular object	`System.out::println`	`x -> System.out.println(x)`
17.4.3 Instance method of an arbitrary object of a type	`String::toUpperCase`	`s -> s.toUpperCase()`
17.4.4 Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

17.4.1 Static method reference ClassName::method

17.4.2 Instance method of a particular object obj::method

17.4.3 Instance method of an arbitrary object ClassName::method

17.4.4 Constructor reference ClassName::new

ALL FOUR KINDS OF METHOD REFERENCE

MethodReferences.java

```
import java.util.*;
import java.util.function.*;

public class MethodReferences {
    record Student(String name) { }
    static boolean isLong(String s) { return s.length() > 4; }

    public static void main(String[] args) {
        List<String> nums = List.of("12", "7", "30");
        Function<String, Integer> parse = Integer::parseInt;                    // 17.4.1 static
        System.out.println("static     : " + nums.stream().map(parse).toList());
        System.out.println("static (own): " + List.of("Om",
"Kiran").stream().filter(MethodReferences::isLong).toList());

        String prefix = "LBF-";
        Function<String, String> addPrefix = prefix::concat;                    // 17.4.2 particular object
        System.out.println("particular : " + addPrefix.apply("101"));
        nums.forEach(System.out::println);                                      // System.out is a particular object

        Function<String, String> upper = String::toUpperCase;                  // 17.4.3 arbitrary object
        BiFunction<String, String, Boolean> eqIgnore = String::equalsIgnoreCase;   // first param is the receiver
        System.out.println("arbitrary  : " + upper.apply("java") + ", " + eqIgnore.apply("SQL", "sql"));

        Supplier<List<String>> listMaker = ArrayList::new;                      // 17.4.4 constructor
        Function<String, Student> makeStudent = Student::new;
        IntFunction<int[]> arrayMaker = int[]::new;
        List<String> l = listMaker.get(); l.add("x");
        System.out.println("constructor: " + l + ", " + makeStudent.apply("Asha") + ", array length " +
arrayMaker.apply(5).length);
    }
}
```

OUTPUT

```
static     : [12, 7, 30]
static (own): [Kiran]
particular : LBF-101
12
7
30
arbitrary  : JAVA, true
constructor: [x], Student[name=Asha], array length 5
```

17.5 Stream API

17.5.1 What a stream is (not a data structure, lazy, single use)

A **stream** (`java.util.stream.Stream`) is a sequence of elements that flow through a **pipeline** of operations. A pipeline has three parts: a **source** (collection, array, file, generator), zero or more **intermediate** operations (filter, map, sorted…) that each return a new stream, and one **terminal** operation (collect, forEach, count…) that produces a result and ends the stream.



A STREAM PIPELINE: SOURCE → INTERMEDIATE OPERATIONS → TERMINAL OPERATION

Not a data structure

A stream stores nothing; it carries elements from a source through operations. The source collection is never modified.

Lazy

Intermediate operations do nothing until a terminal operation is called, and then elements are processed one at a time, only as far as needed.

Single use

After a terminal operation the stream is consumed. Using it again throws `IllegalStateException`; create a new stream instead.

STREAMS ARE SINGLE-USE AND DO NOT MODIFY THE SOURCE

StreamBasics.java

```
import java.util.List;
import java.util.stream.Stream;

public class StreamBasics {
    public static void main(String[] args) {
        List<Integer> marks = List.of(45, 91, 78, 32, 66);
        Stream<Integer> s = marks.stream().filter(m -> m >= 50);
        System.out.println("count = " + s.count());
        try {
            s.count();                                          // second use
        } catch (IllegalStateException e) {
            System.out.println("reuse -> " + e.getMessage());
        }
        System.out.println("source unchanged: " + marks);
    }
}
```

OUTPUT

```
count = 3
reuse -> stream has already been operated upon or closed
source unchanged: [45, 91, 78, 32, 66]
```

17.5.2 Creating streams: from collections, arrays, Stream.of(), iterate(), generate(), IntStream.range()

MANY WAYS TO OBTAIN A STREAM

CreatingStreams.java

```
import java.util.*;
import java.util.stream.*;

public class CreatingStreams {
    public static void main(String[] args) {
        List<String> list = List.of("a", "b", "c");
        System.out.println("collection   : " + list.stream().toList());
        int[] arr = {3, 1, 2};
        System.out.println("array        : " + Arrays.stream(arr).boxed().toList());
        System.out.println("Stream.of    : " + Stream.of("x", "y", "z").toList());
        System.out.println("iterate      : " + Stream.iterate(1, n -> n * 2).limit(8).toList());           //
infinite, so limit
        System.out.println("iterate(3arg): " + Stream.iterate(1, n -> n <= 100, n -> n * 3).toList());     // Java 9,
with condition
        Random r = new Random(7);
        System.out.println("generate     : " + Stream.generate(() -> r.nextInt(10)).limit(5).toList());
        System.out.println("range        : " + IntStream.range(1, 6).boxed().toList() + "  rangeClosed: " +
IntStream.rangeClosed(1, 5).sum());
        System.out.println("chars        : " + "java".chars().mapToObj(c -> String.valueOf((char) c)).toList());
        System.out.println("map entries  : " + new TreeMap<>(Map.of("b", 2, "a", 1)).entrySet().stream().map(e ->
e.getKey() + e.getValue()).toList());
        System.out.println("empty / ofNullable: " + Stream.empty().count() + " / " + Stream.ofNullable(null).count());
    }
}
```

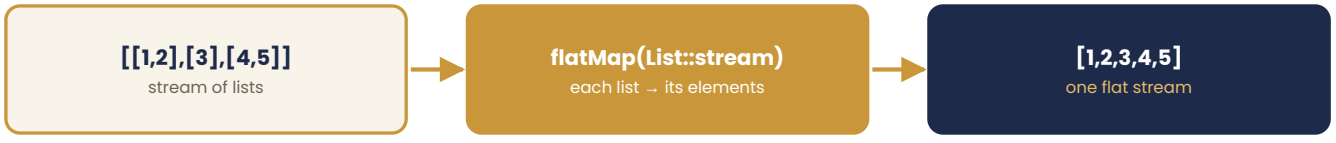
OUTPUT

```
collection   : [a, b, c]
array        : [3, 1, 2]
Stream.of    : [x, y, z]
iterate      : [1, 2, 4, 8, 16, 32, 64, 128]
iterate(3arg): [1, 3, 9, 27, 81]
generate     : [6, 4, 5, 4, 0]
range        : [1, 2, 3, 4, 5]  rangeClosed: 15
chars        : [j, a, v, a]
map entries  : [a1, b2]
empty / ofNullable: 0 / 0
```

17.5.3 Intermediate operations

17.5.3.1 filter, map, flatMap

`filter(predicate)` keeps only matching elements. `map(function)` transforms each element into exactly one new element (possibly of another type). `flatMap(function)` transforms each element into a **stream** of elements and flattens them all into one stream: ideal for lists of lists, or splitting sentences into words.



FLATMAP FLATTENS NESTED STRUCTURES

FILTER, MAP AND FLATMAP

FilterMapFlatMap.java

```
import java.util.*;
import java.util.stream.*;

public class FilterMapFlatMap {
    record Course(String name, List<String> students) { }
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8);
        System.out.println("filter even  : " + nums.stream().filter(n -> n % 2 == 0).toList());
        System.out.println("map square   : " + nums.stream().map(n -> n * n).toList());
        System.out.println("map to text  : " + nums.stream().map(n -> "#" + n).limit(3).toList());

        List<Course> courses = List.of(
            new Course("Java", List.of("Asha", "Ravi")),
            new Course("SQL", List.of("Ravi", "Meera")),
            new Course("DSA", List.of("Kiran")));
        System.out.println("map (nested) : " + courses.stream().map(Course::students).toList());
        System.out.println("flatMap      : " + courses.stream().flatMap(c -> c.students().stream()).toList());

        List<String> lines = List.of("java is fun", "streams are lazy");
        System.out.println("words        : " + lines.stream().flatMap(l -> Arrays.stream(l.split(" "))).toList());
    }
}
```

OUTPUT

```
filter even  : [2, 4, 6, 8]
map square   : [1, 4, 9, 16, 25, 36, 49, 64]
map to text  : [#1, #2, #3]
map (nested) : [[Asha, Ravi], [Ravi, Meera], [Kiran]]
flatMap      : [Asha, Ravi, Ravi, Meera, Kiran]
words        : [java, is, fun, streams, are, lazy]
```

17.5.3.2 distinct, sorted, limit, skip, peek

Operation	Does
`distinct()`	Removes duplicates (uses equals/hashCode)
`sorted()` / `sorted(comparator)`	Natural or custom order (stateful: sees all elements first)
`limit(n)`	At most the first n elements (short-circuits infinite streams)
`skip(n)`	Discards the first n elements (pagination: skip(page*size).limit(size))
`peek(consumer)`	Looks at each element as it passes, mainly for debugging
`takeWhile` / `dropWhile` (Java 9)	Take / drop while a condition holds

REMOVING, ORDERING, SLICING AND DEBUGGING

MoreIntermediate.java

```java
import java.util.*;
import java.util.stream.*;

public class MoreIntermediate {
    public static void main(String[] args) {
        List<Integer> data = List.of(5, 3, 8, 3, 1, 8, 9, 2, 5, 7);
        System.out.println("distinct        : " + data.stream().distinct().toList());
        System.out.println("sorted          : " + data.stream().sorted().toList());
        System.out.println("sorted desc     : " + data.stream().sorted(Comparator.reverseOrder()).toList());
        System.out.println("distinct+sorted : " + data.stream().distinct().sorted().toList());
        System.out.println("skip 2, limit 3 : " + data.stream().skip(2).limit(3).toList());
        int page = 1, size = 4;
        System.out.println("page 1 (size 4) : " + data.stream().skip((long) page * size).limit(size).toList());
        System.out.println("takeWhile < 8   : " + data.stream().takeWhile(n -> n < 8).toList());
        System.out.println("dropWhile < 8   : " + data.stream().dropWhile(n -> n < 8).toList());
        List<Integer> r = data.stream()
                .peek(n -> System.out.print("in:" + n + " "))
                .filter(n -> n > 6)
                .peek(n -> System.out.print("[kept " + n + "] "))
                .limit(2)
                .toList();
        System.out.println("\npeek result     : " + r);
    }
}
```

OUTPUT

```
distinct        : [5, 3, 8, 1, 9, 2, 7]
sorted          : [1, 2, 3, 3, 5, 5, 7, 8, 8, 9]
sorted desc     : [9, 8, 8, 7, 5, 5, 3, 3, 2, 1]
distinct+sorted : [1, 2, 3, 5, 7, 8, 9]
skip 2, limit 3 : [8, 3, 1]
page 1 (size 4) : [1, 8, 9, 2]
takeWhile < 8   : [5, 3]
dropWhile < 8   : [8, 3, 1, 8, 9, 2, 5, 7]
in:5 in:3 in:8 [kept 8] in:3 in:1 in:8 [kept 8]
peek result     : [8, 8]
```

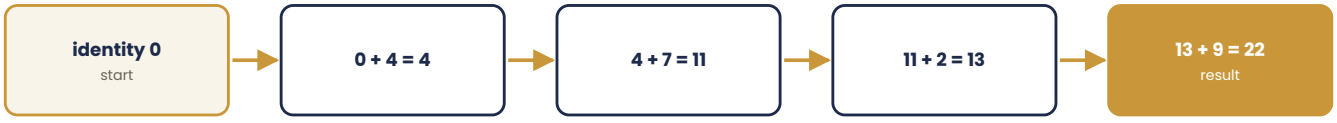
17.5.4 Terminal operations

17.5.4.1 forEach, collect, toList

`forEach` performs an action for each element (order not guaranteed on parallel streams; use `forEachOrdered`). `collect(Collectors.xxx())` gathers elements into a collection or other result (17.5.5). `toList()` (Java 16) returns an **unmodifiable** list; `collect(Collectors.toList())` returns a modifiable ArrayList in practice.

17.5.4.2 reduce, count, min, max

`reduce` combines all elements into one value by repeatedly applying a binary operator: `reduce(identity, accumulator)` returns a value; `reduce(accumulator)` returns an Optional (the stream might be empty). `count`, `min(comparator)` and `max(comparator)` are common special cases.



REDUCE(0, INTEGER::SUM) OVER [4, 7, 2, 9]

17.5.4.3 anyMatch, allMatch, noneMatch, findFirst, findAny

These are **short-circuiting**: they stop as soon as the answer is known. `anyMatch`: is there at least one? `allMatch`: do all match? (true for an empty stream). `noneMatch`: does none match? `findFirst` returns the first element as an Optional; `findAny` returns any element (faster on parallel streams).

TERMINAL OPERATIONS

TerminalOps.java

```
import java.util.*;
import java.util.stream.*;

public class TerminalOps {
    record Student(String name, int marks) { }
    public static void main(String[] args) {
        List<Student> st = List.of(new Student("Asha", 91), new Student("Ravi", 45), new Student("Meera", 78), new
Student("Kiran", 32));

        st.stream().filter(s -> s.marks() >= 50).forEach(s -> System.out.print(s.name() + " "));   // 17.5.4.1
        System.out.println("<- forEach");
        List<String> names = st.stream().map(Student::name).collect(Collectors.toList());
        names.add("Dev");                                                  // collect(toList()) is modifiable
        System.out.println("collect: " + names);
        try { st.stream().map(Student::name).toList().add("x"); }
        catch (UnsupportedOperationException e) { System.out.println("toList() is unmodifiable"); }

        int total = st.stream().map(Student::marks).reduce(0, Integer::sum);          // 17.5.4.2
        Optional<Integer> product = Stream.of(2, 3, 4).reduce((a, b) -> a * b);
        System.out.println("reduce sum " + total + ", product " + product.get() + ", count " + st.stream().count());
        System.out.println("min " + st.stream().min(Comparator.comparingInt(Student::marks)).get().name()
                + ", max " + st.stream().max(Comparator.comparingInt(Student::marks)).get().name());

        System.out.println("anyMatch >= 90 : " + st.stream().anyMatch(s -> s.marks() >= 90));   // 17.5.4.3
        System.out.println("allMatch >= 30 : " + st.stream().allMatch(s -> s.marks() >= 30));
        System.out.println("noneMatch > 100: " + st.stream().noneMatch(s -> s.marks() > 100));
        System.out.println("findFirst < 50 : " + st.stream().filter(s -> s.marks() <
50).findFirst().map(Student::name).orElse("none"));
        System.out.println("findAny  > 95  : " + st.stream().filter(s -> s.marks() > 95).findAny().isPresent());
    }
}
```

OUTPUT

```
Asha Meera <- forEach
collect: [Asha, Ravi, Meera, Kiran, Dev]
toList() is unmodifiable
reduce sum 246, product 24, count 4
min Kiran, max Asha
anyMatch >= 90 : true
allMatch >= 30 : true
noneMatch > 100: true
findFirst < 50 : Ravi
findAny  > 95  : false
```

17.5.5 Collectors

`java.util.stream.Collectors` provides ready-made recipes for `collect()`.

17.5.5.1 toList, toSet, toMap

17.5.5.2 joining

17.5.5.3 groupingBy and partitioningBy

`groupingBy(classifier)` splits elements into a **Map** of lists by a key (department → students), like SQL GROUP BY. A second "downstream" collector summarises each group: `counting()`, `averagingDouble(...)`, `mapping(...)`. `partitioningBy(predicate)` is a special case with exactly two groups: **true** and **false**.

17.5.5.4 counting, summingInt, averagingDouble

THE MOST USEFUL COLLECTORS

CollectorsDemo.java

```
import java.util.*;
import java.util.stream.*;

public class CollectorsDemo {
    record Emp(String name, String dept, double salary, int age) { }

    public static void main(String[] args) {
        List<Emp> emps = List.of(
            new Emp("Asha", "IT", 90000, 29), new Emp("Ravi", "HR", 45000, 35),
            new Emp("Meera", "IT", 120000, 41), new Emp("Kiran", "Sales", 50000, 26),
            new Emp("Dev", "Sales", 65000, 31), new Emp("Om", "IT", 70000, 24));

        Set<String> depts = emps.stream().map(Emp::dept).collect(Collectors.toCollection(TreeSet::new));    // 17.5.5.1
        Map<String, Double> salaryByName = emps.stream().collect(Collectors.toMap(Emp::name, Emp::salary));
        Map<String, Integer> headcount = emps.stream().collect(Collectors.toMap(Emp::dept, e -> 1, Integer::sum,
TreeMap::new));
        System.out.println("toSet      : " + depts);
        System.out.println("toMap      : Meera earns " + salaryByName.get("Meera"));
        System.out.println("toMap merge: " + headcount);

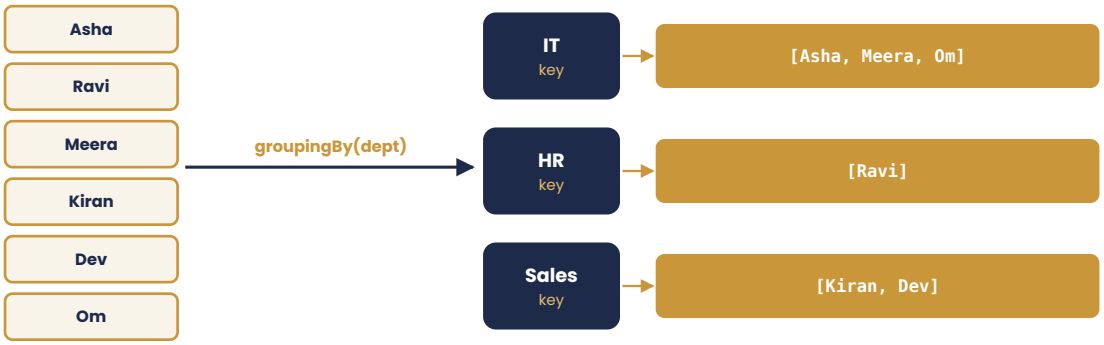
        System.out.println("joining    : " + emps.stream().map(Emp::name).collect(Collectors.joining(", ", "[", "]")));
// 17.5.5.2

        Map<String, List<String>> byDept = emps.stream().collect(                          // 17.5.5.3
            Collectors.groupingBy(Emp::dept, TreeMap::new, Collectors.mapping(Emp::name, Collectors.toList())));
        System.out.println("groupingBy : " + byDept);
        Map<Boolean, List<String>> over30 = emps.stream().collect(
            Collectors.partitioningBy(e -> e.age() > 30, Collectors.mapping(Emp::name, Collectors.toList())));
        System.out.println("partition  : over 30 = " + over30.get(true) + ", others = " + over30.get(false));

        System.out.println("counting   : " + emps.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.counting())));   // 17.5.5.4
        System.out.println("summingInt : total age " + emps.stream().collect(Collectors.summingInt(Emp::age)));
        Map<String, Double> avg = emps.stream().collect(Collectors.groupingBy(Emp::dept, TreeMap::new,
Collectors.averagingDouble(Emp::salary)));
        System.out.println("averaging  : " + avg);
        DoubleSummaryStatistics stats = emps.stream().collect(Collectors.summarizingDouble(Emp::salary));
        System.out.printf("summary    : min %.0f, max %.0f, avg %.0f, n %d%n", stats.getMin(), stats.getMax(),
stats.getAverage(), stats.getCount());
    }
}
```

OUTPUT

```
toSet      : [HR, IT, Sales]
toMap      : Meera earns 120000.0
toMap merge: {HR=1, IT=3, Sales=2}
joining    : [Asha, Ravi, Meera, Kiran, Dev, Om]
groupingBy : {HR=[Ravi], IT=[Asha, Meera, Om], Sales=[Kiran, Dev]}
partition  : over 30 = [Ravi, Meera, Dev], others = [Asha, Kiran, Om]
counting   : {HR=1, IT=3, Sales=2}
summingInt : total age 186
averaging  : {HR=45000.0, IT=93333.33333333333, Sales=57500.0}
summary    : min 45000, max 120000, avg 73333, n 6
```



GROUPINGBY TURNS A FLAT STREAM INTO A MAP OF GROUPS

17.5.6 Primitive streams: IntStream, LongStream, DoubleStream; boxed()

Primitive streams avoid boxing and add numeric methods: `sum()`, `average()`, `min()`, `max()`, `summaryStatistics()`, `range()`. Convert with `mapToInt` / `mapToDouble`, and back to objects with `boxed()` or `mapToObj`.

NUMERIC STREAMS PrimitiveStreams.java

```
import java.util.*;
import java.util.stream.*;

public class PrimitiveStreams {
    public static void main(String[] args) {
        List<String> words = List.of("java", "stream", "lambda", "api");
        IntStream lengths = words.stream().mapToInt(String::length);            // Stream<String> -> IntStream
        System.out.println("sum of lengths  : " + lengths.sum());
        System.out.println("average length  : " + words.stream().mapToInt(String::length).average().orElse(0));
        IntSummaryStatistics st = IntStream.of(4, 8, 15, 16, 23, 42).summaryStatistics();
        System.out.println("statistics      : " + st);
        System.out.println("1..100 sum      : " + IntStream.rangeClosed(1, 100).sum());
        System.out.println("squares (boxed) : " + IntStream.rangeClosed(1, 5).map(n -> n * n).boxed().toList());
        System.out.println("mapToObj        : " + IntStream.range(0, 3).mapToObj(i -> "row" + i).toList());
        System.out.printf ("DoubleStream avg: %.2f%n", DoubleStream.of(9.5, 7.25, 8.0).average().getAsDouble());
        System.out.println("factorial 20    : " + LongStream.rangeClosed(1, 20).reduce(1, (a, b) -> a * b));
    }
}
```

OUTPUT

```
sum of lengths  : 19
average length  : 4.75
statistics      : IntSummaryStatistics{count=6, sum=108, min=4, average=18.000000, max=42}
1..100 sum      : 5050
squares (boxed) : [1, 4, 9, 16, 25]
mapToObj        : [row0, row1, row2]
DoubleStream avg: 8.25
factorial 20    : 2432902008176640000
```

17.5.7 Parallel streams and when not to use them

`list.parallelStream()` or `stream.parallel()` splits the work across all CPU cores using the common ForkJoinPool (16.9.2). It can speed up **large, CPU-heavy, independent** computations. It often makes things **slower** or **wrong** otherwise:

- Small data sets: the cost of splitting and merging exceeds the work.
- Operations with **side effects** on shared state (adding to an ArrayList inside forEach) give wrong results; use collect instead.
- Order-dependent operations (`findFirst`, `limit`, sorted output) lose much of the benefit.
- Blocking I/O inside the pipeline starves the shared common pool.
- Sources that split badly (LinkedList, Stream.iterate) parallelise poorly; arrays and ArrayLists split well.

CORRECT AND INCORRECT USE OF PARALLEL STREAMS ParallelStreams.java

```
import java.util.*;
import java.util.stream.*;

public class ParallelStreams {
    static boolean isPrime(long n) {
        if (n < 2) return false;
        for (long d = 2; d * d <= n; d++) if (n % d == 0) return false;
        return true;
    }
    public static void main(String[] args) {
        long seq = LongStream.rangeClosed(1, 300_000).filter(ParallelStreams::isPrime).count();
        long par = LongStream.rangeClosed(1, 300_000).parallel().filter(ParallelStreams::isPrime).count();
        System.out.println("primes: sequential " + seq + ", parallel " + par + ", same: " + (seq == par));

        List<Integer> unsafe = new ArrayList<>();
        try {
            IntStream.range(0, 1_000_000).parallel().forEach(unsafe::add);         // WRONG: shared mutable list
            System.out.println("unsafe list size: " + unsafe.size() + " (expected 1000000)");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("unsafe list: crashed with ArrayIndexOutOfBoundsException");
        }
        List<Integer> safe = IntStream.range(0, 1_000_000).parallel().boxed().toList();   // RIGHT: let collect merge
        System.out.println("safe list size: " + safe.size() + ", order kept: " + (safe.get(999_999) == 999_999));
    }
}
```

ONE POSSIBLE OUTPUT

```
primes: sequential 25997, parallel 25997, same: true
unsafe list: crashed with ArrayIndexOutOfBoundsException
safe list size: 1000000, order kept: true
```

17.5.8 Lazy evaluation and stream pipelines

Nothing happens until the terminal operation runs. Then each element travels through the **whole** pipeline before the next one starts (vertical processing), and short-circuiting operations stop the flow early. This is why streams can work on infinite sources and why `filter` before `map` saves work.

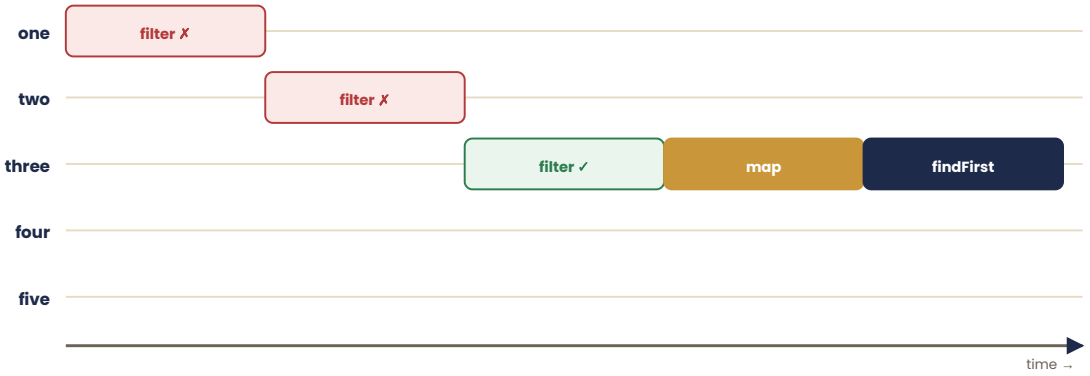
ELEMENTS FLOW ONE BY ONE AND STOP EARLY Laziness.java

```
import java.util.stream.Stream;

public class Laziness {
    public static void main(String[] args) {
        Stream<String> pipeline = Stream.of("one", "two", "three", "four", "five")
                .filter(s -> { System.out.println("  filter " + s); return s.length() > 3; })
                .map(s -> { System.out.println("  map    " + s); return s.toUpperCase(); });
        System.out.println("pipeline built, nothing printed yet");
        String first = pipeline.findFirst().orElse("none");          // terminal: now it runs
        System.out.println("result: " + first + "  (\"four\" and \"five\" were never touched)");
    }
}
```

OUTPUT

```
pipeline built, nothing printed yet
  filter one
  filter two
  filter three
  map    three
result: THREE  ("four" and "five" were never touched)
```



EACH ELEMENT PASSES THROUGH THE WHOLE PIPELINE; PROCESSING STOPS AT THE FIRST RESULT

17.6 Optional

17.6.1 Why Optional (avoiding null checks)

Returning `null` to mean "no result" is the most common source of NullPointerException: the caller forgets to check. `Optional<T>` is a small container that either holds a value or is **empty**. A method returning `Optional<Student>` tells the caller, through its type, that there may be no student, and offers methods to handle both cases fluently.



AN OPTIONAL IS A BOX THAT IS EITHER FULL OR EMPTY

17.6.2 of, ofNullable, empty

Factory	Use when
`Optional.of(value)`	value is definitely not null (throws NPE if it is)
`Optional.ofNullable(value)`	value may be null: null → empty
`Optional.empty()`	there is no value

17.6.3 isPresent, ifPresent, orElse, orElseGet, orElseThrow, map, filter

THE OPTIONAL API

OptionalDemo.java

```
import java.util.*;

public class OptionalDemo {
    record Student(int id, String name, String email) { }
    static final Map<Integer, Student> DB = Map.of(
        1, new Student(1, "Asha", "asha@lbf.in"),
        2, new Student(2, "Ravi", null));

    static Optional<Student> findById(int id) { return Optional.ofNullable(DB.get(id)); }
    static String defaultName() { System.out.println("    (computing default...)"); return "Unknown"; }

    public static void main(String[] args) {
        Optional<Student> found = findById(1), missing = findById(9);
        System.out.println("isPresent  : " + found.isPresent() + " / " + missing.isPresent() + ", isEmpty " +
missing.isEmpty());
        found.ifPresent(s -> System.out.println("ifPresent  : hello " + s.name()));
        missing.ifPresentOrElse(s -> System.out.println(s), () -> System.out.println("ifPresentOrElse: no student 9"));

        System.out.println("map        : " + found.map(Student::name).orElse("?"));
        System.out.println("orElse     : " + missing.map(Student::name).orElse("Guest"));
        System.out.println("orElseGet  : " + found.map(Student::name).orElseGet(OptionalDemo::defaultName) + "
(supplier not called)");
        System.out.println("orElseGet  : " + missing.map(Student::name).orElseGet(OptionalDemo::defaultName));

        String email = findById(2).map(Student::email).orElse("no email");     // null email -> empty
        System.out.println("chained map: " + email);
        System.out.println("filter     : " + found.filter(s -> s.name().startsWith("R")).isPresent());
        try {
            missing.orElseThrow(() -> new NoSuchElementException("Student 9 not found"));
        } catch (NoSuchElementException e) {
            System.out.println("orElseThrow: " + e.getMessage());
        }
        System.out.println("stream     : " + List.of(1, 9,
2).stream().map(OptionalDemo::findById).flatMap(Optional::stream).map(Student::name).toList());
    }
}
```

OUTPUT

```
isPresent  : true / false, isEmpty true
ifPresent  : hello Asha
ifPresentOrElse: no student 9
map        : Asha
orElse     : Guest
orElseGet  : Asha  (supplier not called)
    (computing default...)
orElseGet  : Unknown
chained map: no email
filter     : false
orElseThrow: Student 9 not found
stream     : [Asha, Ravi]
```

WATCH OUT

`orElse(x)` **always** evaluates x, even when a value is present. If x is expensive (a database call, building an object), use `orElseGet(() -> x)`, which runs only when needed.

17.6.4 Optional anti-patterns

Anti-pattern	Why it is bad	Better
`if (opt.isPresent()) return opt.get();`	Just a verbose null check	`opt.orElse(...)`, `map`, `ifPresent`
`opt.get()` without checking	Throws NoSuchElementException: same bug as NPE	`orElseThrow()` with a clear message

Anti-pattern	Why it is bad	Better
Returning `null` from a method declared to return Optional	Defeats the whole purpose	Return `Optional.empty()`
Optional as a **field** or **method parameter**	Not serializable, clumsy for callers	Use only as a **return type**; use null/overloads internally
`Optional<List<T>>`	A collection already has an "empty" state	Return an empty list
`Optional.of(maybeNull)`	Throws NPE immediately	`Optional.ofNullable`
Optional of primitives boxed	`Optional<Integer>` boxes	`OptionalInt`, `OptionalDouble`

PRACTICE CHECKPOINT

Rewrite the Student Management System's reports with streams: **top 3 by marks**, **average per department** with groupingBy, and **pass/fail partition**.

SAMPLE SOLUTION

StreamReports.java

```
import java.util.*;
import java.util.stream.*;

public class StreamReports {
    record Student(int id, String name, String dept, double marks) { }

    public static void main(String[] args) {
        List<Student> students = List.of(
            new Student(101, "Asha", "CSE", 91.5), new Student(102, "Ravi", "ECE", 48.0),
            new Student(103, "Meera", "CSE", 85.0), new Student(104, "Kiran", "MECH", 72.5),
            new Student(105, "Dev", "ECE", 66.0), new Student(106, "Om", "CSE", 39.5),
            new Student(107, "Priya", "MECH", 94.0), new Student(108, "Lohit", "ECE", 88.0));

        System.out.println("== Top 3 by marks ==");
        students.stream()
                .sorted(Comparator.comparingDouble(Student::marks).reversed())
                .limit(3)
                .forEach(s -> System.out.printf("  %-6s %-5s %5.1f%n", s.name(), s.dept(), s.marks()));

        System.out.println("== Average per department ==");
        Map<String, Double> avg = students.stream()
                .collect(Collectors.groupingBy(Student::dept, TreeMap::new,
Collectors.averagingDouble(Student::marks)));
        avg.forEach((d, a) -> System.out.printf("  %-5s %6.2f%n", d, a));

        System.out.println("== Pass (>= 50) / Fail ==");
        Map<Boolean, List<String>> result = students.stream()
                .collect(Collectors.partitioningBy(s -> s.marks() >= 50,
                        Collectors.mapping(Student::name, Collectors.toList())));
        System.out.println("  PASS: " + result.get(true));
        System.out.println("  FAIL: " + result.get(false));

        System.out.println("== Extras ==");
        Optional<Student> topper = students.stream().max(Comparator.comparingDouble(Student::marks));
        System.out.println("  topper: " + topper.map(Student::name).orElse("none"));
        System.out.println("  count per dept: " + students.stream().collect(Collectors.groupingBy(Student::dept,
TreeMap::new, Collectors.counting())));
        System.out.println("  overall average: " + students.stream().mapToDouble(Student::marks).average().orElse(0));
    }
}
```

OUTPUT

```
== Top 3 by marks ==
  Priya  MECH   94.0
  Asha   CSE    91.5
  Lohit  ECE    88.0
== Average per department ==
  CSE    72.00
  ECE    67.33
  MECH   83.25
== Pass (>= 50) / Fail ==
  PASS: [Asha, Meera, Kiran, Dev, Priya, Lohit]
  FAIL: [Ravi, Om]
== Extras ==
  topper: Priya
  count per dept: {CSE=3, ECE=3, MECH=2}
  overall average: 73.0625
```

MODULE 18 · ADVANCED

Modern Java features (Java 9 to 25)

Minimum duration **3 days** | Level **Advanced**

This module walks through the language and library features added after Java 8, grouped by release, so learners can read and write current Java.

IN THIS MODULE

18.1 Java 9
18.2 Java 10 and 11
18.3 Java 12 to 17
18.4 Java 18 to 21
18.5 Java 22 to 25
18.6 Staying current

Since Java 9 (2017) a new Java version is released **every six months** (March and September). Every two years one release is a **Long-Term Support (LTS)** version that companies standardise on: **Java 11, 17, 21 and 25**. Features often arrive first as **preview** features (18.6.1) and become final a release or two later.



MAJOR MILESTONES; LTS RELEASES ARE THE ONES MOST PROJECTS USE

NOTE
Examples in 18.1–18.4 were run on JDK 21. Examples in 18.5 need **Java 25** (some features exist only as previews before that); their output is shown as expected, and each box is labelled with the version it needs.

18.1 Java 9

18.1.1 Java Platform Module System (JPMS)

Before Java 9, a large application was a flat classpath of JAR files: any public class could be used by anyone, internal packages were not protected, and missing JARs were discovered only at run time ("JAR hell"). **JPMS** (Project Jigsaw) adds **modules**: a named group of packages that declares which other modules it needs and which of its packages it makes available. The JDK itself was split into about 70 modules (`java.base`, `java.sql`, `java.net.http` …).

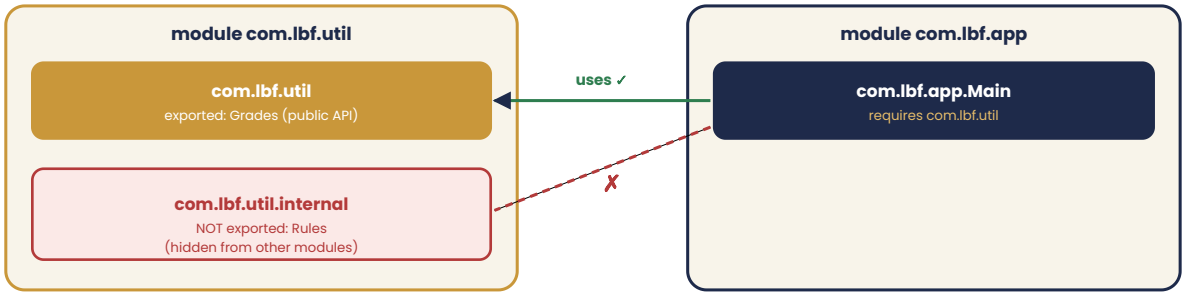
Strong encapsulation
A public class in a non-exported package is invisible to other modules.

Reliable configuration
Missing or duplicate dependencies are detected at startup, not halfway through execution.

Smaller runtimes
`jlink` builds a custom JRE containing only the modules your app needs.

18.1.1.1 module-info.java

A module is defined by a `module-info.java` file in the root of its source folder. Below, a `com.lbf.util` module exposes a grading API but hides its internal rules, and a `com.lbf.app` module uses it.



ONLY EXPORTED PACKAGES ARE VISIBLE TO OTHER MODULES

SRC/COM.LBF.UTIL/MODULE-INFO.JAVA Main.java

```
module com.lbf.util {
    exports com.lbf.util;           // public API: visible to other modules
    // com.lbf.util.internal is NOT exported: hidden even though its classes are public
}
```

SRC/COM.LBF.APP/MODULE-INFO.JAVA Main.java

```
module com.lbf.app {
    requires com.lbf.util;          // depends on the util module
}
```

SRC/COM.LBF.APP/COM/LBF/APP/MAIN.JAVA Main.java

```
package com.lbf.app;

import com.lbf.util.Grades;

public class Main {
    public static void main(String[] args) {
        System.out.println("91.5 -> " + Grades.of(91.5));
        System.out.println("62.0 -> " + Grades.of(62.0));
        System.out.println("module: " + Main.class.getModule().getName());
    }
}
```

OUTPUT

```
91.5 -> A
62.0 -> C
module: com.lbf.app
```

Compiling and running modules (real output)

```
$ javac -d out --module-source-path src $(find src -name "*.java")
$ java --module-path out -m com.lbf.app/com.lbf.app.Main

# if Main tries:  import com.lbf.util.internal.Rules;
src/com.lbf.app/com/lbf/app/Main.java:4: error: package com.lbf.util.internal is not visible
import com.lbf.util.internal.Rules;
                ^
  (package com.lbf.util.internal is declared in module com.lbf.util, which does not export it)
```

18.1.1.2 requires, exports, opens

Directive	Meaning
`requires M;`	This module depends on module M (compile time and run time)
`requires transitive M;`	Also gives M to every module that requires this one
`requires static M;`	Needed at compile time only (optional at run time)
`exports p;`	Public types of package p are accessible to other modules

Directive	Meaning
`exports p to M;`	Qualified export: only module M may use p
`opens p;`	Allows **deep reflection** on p at run time (needed by frameworks like Hibernate, Jackson, Spring)
`uses S;` / `provides S with Impl;`	Service loading (plug-in architecture via ServiceLoader)

> **NOTE**
> Most applications still run on the classpath, where all code lives in the "unnamed module" and works as before. JPMS is mainly used by the JDK itself, by libraries, and to build small runtimes with `jlink`. `java --list-modules` lists the JDK's modules.

18.1.2 JShell

JShell is Java's REPL (Read-Eval-Print Loop): type a statement or expression and see the result immediately, with no class, no main and no compile step. Perfect for experiments and learning. Start it with `jshell`; leave with `/exit`.

A JShell session

```
$ jshell
|  Welcome to JShell -- Version 21.0.10
|  For an introduction type: /help intro

jshell> int x = 21
x ==> 21

jshell> x * 2
$2 ==> 42

jshell> "java".toUpperCase()
$3 ==> "JAVA"

jshell> var list = java.util.List.of(3, 1, 2)
list ==> [3, 1, 2]

jshell> list.stream().sorted().toList()
$5 ==> [1, 2, 3]

jshell> int square(int n) { return n * n; }
|  created method square(int)

jshell> square(12)
$7 ==> 144

jshell> /vars
|    int x = 21
|    int $2 = 42
|    String $3 = "JAVA"
|    List<Integer> list = [3, 1, 2]
|    List<Integer> $5 = [1, 2, 3]
|    int $7 = 144

jshell> /exit
```

Command	Does
`/vars`, `/methods`, `/types`	List what you have defined
`/list`	Show the snippets entered
`/edit name`	Edit a snippet in an editor
`/save file.jsh`, `/open file.jsh`	Save / load a session
`/imports`	Show default imports (java.util.*, java.io.* …)

Command	Does
Tab key	Auto-completion and documentation

18.1.3 Collection factory methods: List.of(), Set.of(), Map.of()

One-line creation of small **immutable** collections (covered in 14.3.5 and 14.6.6). They reject nulls; `Set.of` and `Map.of` reject duplicates; iteration order of Set.of/Map.of is unspecified.

IMMUTABLE COLLECTIONS IN ONE LINE

FactoryMethods.java

```
import java.util.*;

public class FactoryMethods {
    public static void main(String[] args) {
        List<String> days = List.of("Mon", "Tue", "Wed");
        Set<Integer> primes = Set.of(2, 3, 5, 7);
        Map<String, Integer> fees = Map.of("Java", 15000, "SQL", 8000);
        System.out.println(days + " " + new TreeSet<>(primes) + " " + new TreeMap<>(fees));
        // Java 8 equivalent: Collections.unmodifiableList(new ArrayList<>(Arrays.asList("Mon", "Tue", "Wed")))
        try { Set.of(1, 1); } catch (IllegalArgumentException e) { System.out.println("Set.of duplicate -> " +
e.getMessage()); }
        try { List.of("a", null); } catch (NullPointerException e) { System.out.println("List.of null ->
NullPointerException"); }
    }
}
```

```
OUTPUT
[Mon, Tue, Wed] [2, 3, 5, 7] {Java=15000, SQL=8000}
Set.of duplicate -> duplicate element: 1
List.of null -> NullPointerException
```

18.1.4 Private interface methods

Interfaces may contain `private` and `private static` methods, so default methods can share helper code without exposing it (full example in 10.9.3.6).

18.1.5 Stream additions: takeWhile, dropWhile

`takeWhile(p)` takes elements while the predicate holds and stops at the first failure; `dropWhile(p)` skips them and returns the rest. Unlike filter, they look only at the **beginning** of the stream, which matters for sorted or infinite streams. Java 9 also added `Stream.iterate(seed, hasNext, next)`, `Stream.ofNullable` and `Optional.stream()/ifPresentOrElse/or`.

TAKEWHILE VS FILTER

Java9Streams.java

```
import java.util.List;
import java.util.stream.Stream;

public class Java9Streams {
    public static void main(String[] args) {
        List<Integer> sortedMarks = List.of(35, 42, 48, 55, 61, 47, 90);
        System.out.println("filter    < 50 : " + sortedMarks.stream().filter(m -> m < 50).toList());
        System.out.println("takeWhile < 50 : " + sortedMarks.stream().takeWhile(m -> m < 50).toList());
        System.out.println("dropWhile < 50 : " + sortedMarks.stream().dropWhile(m -> m < 50).toList());
        System.out.println("iterate 3-arg  : " + Stream.iterate(1, n -> n < 200, n -> n * 3).toList());
    }
}
```

```
OUTPUT
filter    < 50 : [35, 42, 48, 47]
takeWhile < 50 : [35, 42, 48]
dropWhile < 50 : [55, 61, 47, 90]
iterate 3-arg  : [1, 3, 9, 27, 81]
```

18.2 Java 10 and 11

18.2.1 var for local variables

`var` (Java 10) lets the compiler **infer** the type of a local variable from its initializer. Java is still statically typed: the variable has a fixed type, you just do not write it. Use it when the type is obvious from the right-hand side; avoid it when it hides important information.

LOCAL VARIABLE TYPE INFERENCE

VarDemo.java

```
import java.util.*;

public class VarDemo {
    public static void main(String[] args) {
        var name = "Asha";                                  // String
        var marks = 91.5;                                   // double
        var studentsByCity = new HashMap<String, List<String>>();   // no need to repeat the long type
        studentsByCity.computeIfAbsent("Pune", k -> new ArrayList<>()).add(name);
        for (var entry : studentsByCity.entrySet())           // in for-each
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        for (var i = 0; i < 2; i++) System.out.print(i + " ");
        System.out.println();
        System.out.println(((Object) marks).getClass().getSimpleName() + ", " + name.getClass().getSimpleName());
        // var x;            // error: cannot infer type for local variable x (no initializer)
        // var n = null;     // error: variable initializer is 'null'
        // name = 42;        // error: incompatible types: int cannot be converted to String
    }
}
```

OUTPUT

```
Pune -> [Asha]
0 1
Double, String
```

Allowed	Not allowed
Local variables with an initializer	Fields, method parameters, return types
for / for-each loop variables	Without an initializer, or initialised to null
try-with-resources variables	Array initializer alone: `var a = {1, 2};`
Lambda parameters (Java 11): `(var a, var b) -> a + b`	Mixing var and explicit types in one lambda

18.2.2 New String methods: isBlank, lines, strip, repeat

JAVA 11 STRING ADDITIONS

Java11Strings.java

```
public class Java11Strings {
    public static void main(String[] args) {
        String s = "\u2003  Learn Java  \u2003";          // includes a Unicode em-space
        System.out.println("isBlank(\"  \")  : " + "   ".isBlank() + ", isEmpty: " + "   ".isEmpty());
        System.out.println("trim()  length : " + s.trim().length() + "  (trim removes only chars <= space)");
        System.out.println("strip() length : " + s.strip().length() + "  (strip removes all Unicode whitespace)");
        System.out.println("stripLeading   : [" + "  hi  ".stripLeading() + "], stripTrailing: [" + "  hi
".stripTrailing() + "]");
        System.out.println("repeat         : " + "=-".repeat(6));
        String text = "line one\nline two\r\nline three";
        System.out.println("lines().count(): " + text.lines().count());
        text.lines().map(l -> "  > " + l).forEach(System.out::println);
    }
}
```

OUTPUT

```
isBlank("  ")  : true, isEmpty: false
trim()  length : 16  (trim removes only chars <= space)
strip() length : 10  (strip removes all Unicode whitespace)
stripLeading   : [hi  ], stripTrailing: [  hi]
repeat         : =-=-=-=-=-=-
lines().count(): 3
  > line one
  > line two
  > line three
```

18.2.3 HTTP Client API

Java 11 added `java.net.http.HttpClient`, a modern replacement for the old `HttpURLConnection`. It supports HTTP/1.1 and HTTP/2, synchronous (`send`) and asynchronous (`sendAsync` → CompletableFuture) calls, and a fluent builder API. The example below starts a tiny local web server (built into the JDK) so it runs without internet access; with a real URL the client code is identical.

GET REQUEST, SYNCHRONOUS AND ASYNCHRONOUS HttpClientDemo.java

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.*;
import java.time.Duration;

public class HttpClientDemo {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("localhost", 0), 0);   // any free port
        server.createContext("/courses", ex -> {
            byte[] body = "[{\"code\":\"J101\",\"title\":\"Core Java\"}]".getBytes();
            ex.getResponseHeaders().add("Content-Type", "application/json");
            ex.sendResponseHeaders(200, body.length);
            ex.getResponseBody().write(body);
            ex.close();
        });
        server.start();
        String url = "http://localhost:" + server.getAddress().getPort() + "/courses";

        HttpClient client = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(5)).build();
        HttpRequest request = HttpRequest.newBuilder(URI.create(url)).header("Accept",
"application/json").GET().build();

        HttpResponse<String> res = client.send(request, HttpResponse.BodyHandlers.ofString());   // synchronous
        System.out.println("status : " + res.statusCode());
        System.out.println("type   : " + res.headers().firstValue("Content-Type").orElse("?"));
        System.out.println("body   : " + res.body());

        client.sendAsync(request, HttpResponse.BodyHandlers.ofString())                          // asynchronous
              .thenApply(HttpResponse::body)
              .thenAccept(b -> System.out.println("async  : " + b.length() + " characters received"))
              .join();
        server.stop(0);
    }
}
```

OUTPUT

```
status : 200
type   : application/json
body   : [{"code":"J101","title":"Core Java"}]
async  : 37 characters received
```

A POST REQUEST WITH A JSON BODY java

```
HttpRequest post = HttpRequest.newBuilder(URI.create("https://api.example.com/students"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString("{\"name\":\"Asha\"}"))
        .build();
```

18.2.4 Files.readString() / writeString()

Reading or writing a whole text file in one line (details in 15.6.2): `Files.writeString(path, text)` and `String s = Files.readString(path)`, UTF-8 by default. Java 11 also added `Optional.isEmpty()`, `Predicate.not()`, `Collection.toArray(String[]::new)` and running a single source file directly: `java Hello.java` (no javac needed).

18.3 Java 12 to 17

18.3.1 Switch expressions

Final in Java 14. `switch` can now be an **expression** that produces a value. The arrow form `case X ->` has **no fall-through** (no break needed), several labels can share a case (`case SAT, SUN ->`), and for enums and sealed types the compiler checks that all cases are covered. `yield` returns a value from a block.

OLD SWITCH STATEMENT VS SWITCH EXPRESSION

SwitchExpressions.java

```
public class SwitchExpressions {
    enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

    static String oldStyle(Day d) {
        String type;
        switch (d) {
            case SAT:
            case SUN:
                type = "weekend";
                break;                      // forget this and it falls through!
            default:
                type = "weekday";
        }
        return type;
    }
    static String newStyle(Day d) {
        return switch (d) {
            case SAT, SUN -> "weekend";
            case MON, TUE, WED, THU, FRI -> "weekday";   // exhaustive: no default needed
        };
    }
    static int fee(String course) {
        return switch (course) {
            case "Java" -> 15000;
            case "SQL" -> 8000;
            default -> {
                int base = 5000;
                yield base + course.length() * 100;     // yield returns a value from a block
            }
        };
    }
    public static void main(String[] args) {
        System.out.println(oldStyle(Day.SUN) + " / " + newStyle(Day.SUN) + " / " + newStyle(Day.WED));
        System.out.println(fee("Java") + " " + fee("SQL") + " " + fee("Python"));
    }
}
```

OUTPUT

```
weekend / weekend / weekday
15000 8000 5600
```

18.3.2 Text blocks

Final in Java 15. A **text block** is a multi-line string literal between `"""` delimiters. No `\n` or concatenation needed; the common leading indentation is removed automatically. Ideal for SQL, JSON, HTML. `\` at a line end joins lines; `\s` keeps a trailing space. Combine with `formatted()` for templates.

MULTI-LINE STRINGS WITHOUT ESCAPES

TextBlocks.java

```
public class TextBlocks {
    public static void main(String[] args) {
        String oldSql = "SELECT name, marks\n" +
                        "FROM students\n" +
                        "WHERE marks > 75\n";
        String sql = """
                SELECT name, marks
                FROM students
                WHERE marks > 75
                """;
        System.out.println("same text: " + oldSql.equals(sql));
        String json = """
                {
                  "name": "%s",
                  "marks": %.1f
                }""".formatted("Asha", 91.5);
        System.out.println(json);
        String oneLine = """
                This is a long sentence \
                written on two lines.""";
        System.out.println(oneLine);
    }
}
```

OUTPUT

```
same text: true
{
  "name": "Asha",
  "marks": 91.5
}
This is a long sentence written on two lines.
```

18.3.3 Records

Final in Java 16. Concise immutable data carriers with auto-generated constructor, accessors, equals, hashCode and toString (full coverage in 10.14). Records can be declared locally inside a method, which is handy for intermediate results in stream pipelines.

18.3.4 Pattern matching for instanceof

Final in Java 16. `if (obj instanceof String s)` tests the type **and** declares a variable already cast to that type, removing the repetitive cast. The variable is in scope only where the test is known to be true, and can be combined with `&&`.

INSTANCEOF WITH A BINDING VARIABLE InstanceofPattern.java

```
public class InstanceofPattern {
    static String describe(Object o) {
        if (o instanceof String s && !s.isEmpty())            // test + cast + condition
            return "String of length " + s.length();
        if (o instanceof Integer i) return "Integer, doubled = " + (i * 2);
        if (!(o instanceof Double d)) return "something else: " + o;
        return "Double, rounded = " + Math.round(d);          // d in scope here (negated test returned)
    }
    public static void main(String[] args) {
        Object[] things = {"hello", 21, 3.7, 'x', ""};
        for (Object t : things) System.out.println(describe(t));
        Object old = "legacy";
        if (old instanceof String) {                           // before Java 16
            String s = (String) old;
            System.out.println("old style: " + s.toUpperCase());
        }
    }
}
```

OUTPUT

```
String of length 5
Integer, doubled = 42
Double, rounded = 4
something else: x
something else:
old style: LEGACY
```

18.3.5 Sealed classes

Final in Java 17. `sealed ... permits` restricts which classes may extend a class or implement an interface (full coverage in 10.15). Together with records and pattern matching they let you model "one of a fixed set of shapes" safely: the compiler knows every subtype.

18.3.6 Helpful NullPointerException messages

Since Java 14 (on by default from 15), an NPE message says **exactly what was null**, which is a big help with chained calls like `a.b().c().d()`. Before, you got only the line number. (Compiling with `-g` makes it show local variable names too.)

WHICH PART OF THE CHAIN WAS NULL? HelpfulNpe.java

```
public class HelpfulNpe {
    record Address(String city) { }
    record Student(String name, Address address) { }

    public static void main(String[] args) {
        Student s = new Student("Asha", null);
        System.out.println(s.address().city().toUpperCase());
    }
}
```

OUTPUT

```
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "HelpfulNpe$Address.city()" because the
return value of "HelpfulNpe$Student.address()" is null
        at HelpfulNpe.main(HelpfulNpe.java:7)
```

18.4 Java 18 to 21

18.4.1 Pattern matching for switch

Final in Java 21. `switch` can match on **types** (`case Circle c ->`), with optional **guards** (`case Integer i when i > 100 ->`) and an explicit `case null`. Over sealed hierarchies the switch is checked for exhaustiveness. This replaces long if-else-instanceof chains.

TYPE PATTERNS, GUARDS AND CASE NULL

SwitchPatterns.java

```
public class SwitchPatterns {
    static String classify(Object o) {
        return switch (o) {
            case null -> "null value";
            case Integer i when i > 100 -> "large integer " + i;     // guard
            case Integer i -> "small integer " + i;
            case String s when s.isBlank() -> "blank string";
            case String s -> "string \"" + s + "\"";
            case int[] arr -> "int array of length " + arr.length;
            default -> "other: " + o.getClass().getSimpleName();
        };
    }
    public static void main(String[] args) {
        Object[] inputs = {42, 500, "Java", "  ", new int[] {1, 2, 3}, 2.5, null};
        for (Object in : inputs) System.out.println(classify(in));
    }
}
```

OUTPUT

```
small integer 42
large integer 500
string "Java"
blank string
int array of length 3
other: Double
null value
```

18.4.2 Record patterns (deconstruction)

Final in Java 21. A **record pattern** matches a record and **extracts its components** in one step, and patterns can be **nested**. Combined with sealed interfaces this gives a very clear style of data-oriented programming.

DECONSTRUCTING NESTED RECORDS

RecordPatterns.java

```
public class RecordPatterns {
    sealed interface Shape permits Circle, Rect, Line { }
    record Point(int x, int y) { }
    record Circle(Point center, int radius) implements Shape { }
    record Rect(Point topLeft, int w, int h) implements Shape { }
    record Line(Point from, Point to) implements Shape { }

    static String describe(Shape s) {
        return switch (s) {
            case Circle(Point(var x, var y), int r) -> "circle at (" + x + "," + y + ") r=" + r;    // nested
deconstruction
            case Rect(Point p, int w, int h) when w == h -> "square " + w + " at " + p;
            case Rect(Point p, int w, int h) -> "rectangle " + w + "x" + h;
            case Line(Point(int x1, int y1), Point(int x2, int y2)) ->
                    String.format("line of length %.2f", Math.hypot(x2 - x1, y2 - y1));
        };                                                                  // exhaustive over the sealed type
    }
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(new Point(0, 0), 5), new Rect(new Point(1, 1), 4, 4),
                           new Rect(new Point(2, 2), 3, 7), new Line(new Point(0, 0), new Point(3, 4)) };
        for (Shape s : shapes) System.out.println(describe(s));
        Object o = new Point(7, 9);
        if (o instanceof Point(int x, int y)) System.out.println("instanceof deconstruction: x+y = " + (x + y));
    }
}
```

OUTPUT

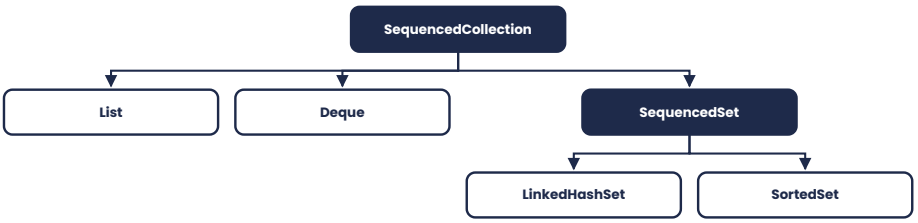
```
circle at (0,0) r=5
square 4 at Point[x=1, y=1]
rectangle 3x7
line of length 5.00
instanceof deconstruction: x+y = 16
```

18.4.3 Virtual threads

Final in Java 21: lightweight JVM-managed threads that make the simple thread-per-task style scale to millions of concurrent blocking tasks. Covered in detail in 16.10.

18.4.4 Sequenced collections: getFirst, getLast, reversed

Final in Java 21. Before, getting the last element was different for every type: `list.get(list.size()-1)`, `deque.getLast()`, `sortedSet.last()`, and LinkedHashSet had no way at all. New interfaces `SequencedCollection`, `SequencedSet` and `SequencedMap` give every ordered collection the same methods.



NEW INTERFACES WERE INSERTED INTO THE EXISTING HIERARCHY (SEQUENCEDMAP SIMILARLY COVERS LINKEDHASHMAP AND SORTEDMAP)

ONE API FOR FIRST, LAST AND REVERSE

SequencedCollections.java

```
import java.util.*;

public class SequencedCollections {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(List.of("B", "C"));
        list.addFirst("A"); list.addLast("D");
        System.out.println("List     : " + list + " first=" + list.getFirst() + " last=" + list.getLast() + "
reversed=" + list.reversed());

        LinkedHashSet<Integer> set = new LinkedHashSet<>(List.of(3, 1, 2));
        System.out.println("LinkedHashSet last : " + set.getLast() + ", reversed " + set.reversed());

        TreeSet<Integer> sorted = new TreeSet<>(List.of(40, 10, 30));
        System.out.println("TreeSet first/last : " + sorted.getFirst() + " / " + sorted.getLast());

        LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
        map.put("one", 1); map.put("two", 2); map.put("three", 3);
        System.out.println("Map firstEntry " + map.firstEntry() + ", lastEntry " + map.lastEntry() + ", reversed " +
map.reversed());
        map.putFirst("zero", 0);
        System.out.println("putFirst  : " + map + ", pollLastEntry " + map.pollLastEntry());
    }
}
```

OUTPUT

```
List     : [A, B, C, D] first=A last=D reversed=[D, C, B, A]
LinkedHashSet last : 2, reversed [2, 1, 3]
TreeSet first/last : 10 / 40
Map firstEntry one=1, lastEntry three=3, reversed {three=3, two=2, one=1}
putFirst  : {zero=0, one=1, two=2, three=3}, pollLastEntry three=3
```

18.4.5 UTF-8 by default

Since Java 18 (JEP 400), the default charset of the standard APIs (`FileReader`, `FileWriter`, `Files`, `Scanner`, `String.getBytes()` …) is **UTF-8 on every operating system**. Earlier, it depended on the OS (often Windows-1252 on Windows), so a file written on Windows could be read incorrectly on Linux. Code is now portable by default. (Console output encoding is separate: see `stdout.encoding`.)

THE DEFAULT CHARSET IS UTF-8 EVERYWHERE

Utf8Default.java

```
import java.nio.charset.Charset;
import java.nio.file.*;

public class Utf8Default {
    public static void main(String[] args) throws Exception {
        System.out.println("default charset : " + Charset.defaultCharset());
        System.out.println("file.encoding   : " + System.getProperty("file.encoding"));
        Path p = Path.of("greet.txt");
        Files.writeString(p, "₹ नमस्ते café");
        System.out.println("bytes written   : " + Files.size(p) + " for " + Files.readString(p).length() + " chars");
    }
}
```

OUTPUT

```
default charset : UTF-8
file.encoding   : UTF-8
bytes written   : 28 for 13 chars
```

18.5 Java 22 to 25

Java 25 (September 2025) is the newest LTS. The features below are **final** in the version shown and need that JDK. They were compiled against Java 25 rules; the outputs shown are what Java 25 prints.

18.5.1 Unnamed variables and patterns _

Final in Java 22 (JEP 456). When a variable must be declared but is never used, name it `_` (underscore). This documents intent and silences "unused" warnings. Works for lambda parameters, catch parameters, for-each variables, try-with-resources and pattern components.

UNNAMED VARIABLES (JAVA 22+)

Unnamed.java

```
import java.util.*;

public class Unnamed {
    record Point(int x, int y) { }
    public static void main(String[] args) {
        int total = 0;
        for (var _ : List.of("a", "b", "c")) total++;                    // loop variable unused
        System.out.println("count = " + total);

        try { Integer.parseInt("x"); }
        catch (NumberFormatException _) { System.out.println("not a number"); }   // exception unused

        Map<String, Integer> m = new TreeMap<>(Map.of("a", 1));
        m.forEach((_, v) -> System.out.println("value only: " + v));       // key unused

        Object o = new Point(3, 4);
        if (o instanceof Point(int x, _)) System.out.println("x = " + x);  // ignore y
    }
}
```

OUTPUT (JAVA 22+)

```
count = 3
not a number
value only: 1
x = 3
```

18.5.2 Stream gatherers

Final in Java 24 (JEP 485). Streams had a fixed set of intermediate operations. `Stream.gather(Gatherer)` lets you add **custom intermediate operations**, and `java.util.stream.Gatherers` provides useful ones: fixed and sliding **windows**, **scan** (running totals) and **fold**.

BUILT-IN GATHERERS (JAVA 24+)

GathererDemo.java

```
import java.util.List;
import java.util.stream.Gatherers;
import java.util.stream.Stream;

public class GathererDemo {
    public static void main(String[] args) {
        List<List<Integer>> fixed = Stream.of(1, 2, 3, 4, 5, 6, 7, 8)
                .gather(Gatherers.windowFixed(3)).toList();
        System.out.println("windowFixed(3)  : " + fixed);

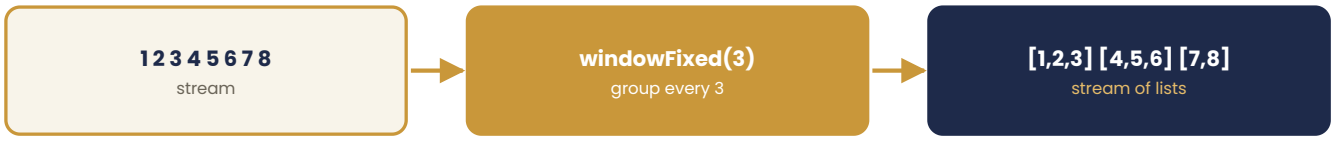
        List<List<Integer>> sliding = Stream.of(1, 2, 3, 4, 5)
                .gather(Gatherers.windowSliding(3)).toList();
        System.out.println("windowSliding(3): " + sliding);

        List<Integer> running = Stream.of(1, 2, 3, 4, 5)
                .gather(Gatherers.scan(() -> 0, Integer::sum)).toList();
        System.out.println("scan (running)  : " + running);

        List<Integer> folded = Stream.of(1, 2, 3, 4, 5)
                .gather(Gatherers.fold(() -> 0, Integer::sum)).toList();
        System.out.println("fold            : " + folded);
    }
}
```

OUTPUT (JAVA 24+)

```
windowFixed(3)  : [[1, 2, 3], [4, 5, 6], [7, 8]]
windowSliding(3): [[1, 2, 3], [2, 3, 4], [3, 4, 5]]
scan (running)  : [1, 3, 6, 10, 15]
fold            : [15]
```



A GATHERER CAN TURN MANY ELEMENTS INTO FEWER (WINDOWS) OR KEEP STATE (SCAN)

18.5.3 Simplified main methods and implicitly declared classes

Final in Java 25 (JEP 512, "compact source files and instance main methods"). Beginners no longer need `public class`, `static` or `String[] args`. A source file may simply contain methods and fields; the compiler wraps them in an **implicitly declared class**. `main` can be an instance method without parameters, and the new `java.lang.IO` class provides `IO.println` and `IO.readln`.

HELLO.JAVA: A COMPLETE JAVA 25 PROGRAM (RUN WITH: JAVA HELLO.JAVA)

Main.java

```
void main() {
    String name = "Learn Being Forward";
    IO.println("Hello from " + name);
    IO.println("square(12) = " + square(12));
}

int square(int n) {
    return n * n;
}
```

OUTPUT (JAVA 25+)

```
Hello from Learn Being Forward
square(12) = 144
```

Classic (all versions)	Java 25 compact form

`public class Hello {`
`  public static void main(String[] args) {`
`  System.out.println("Hi");`
`  }`
`}`
	`void main() {`
	`  IO.println("Hi");`
	`}`

18.5.4 Flexible constructor bodies (statements before super())

Final in Java 25 (JEP 513). Previously `super(...)` or `this(...)` had to be the very first statement, so validating or preparing arguments required awkward static helper methods. Now statements may appear **before** the explicit constructor call, as long as they do not use the object under construction (no `this`, no instance methods; assigning the class's own fields is allowed).

VALIDATION BEFORE SUPER() (JAVA 25+) FlexibleConstructors.java

```
class Person {
    final String name;
    Person(String name) { this.name = name; }
}
class Employee extends Person {
    final double salary;
    Employee(String name, double salary) {
        if (salary < 0) throw new IllegalArgumentException("negative salary");   // validate FIRST
        String clean = name.strip();                                             // prepare argument
        super(clean);                                                            // then call super
        this.salary = salary;
    }
}
public class FlexibleConstructors {
    public static void main(String[] args) {
        Employee e = new Employee("  Ravi  ", 50000);
        System.out.println("[" + e.name + "] " + e.salary);
        try { new Employee("Om", -1); }
        catch (IllegalArgumentException ex) { System.out.println("rejected before Person was built: " +
ex.getMessage()); }
    }
}
```

OUTPUT (JAVA 25+)

```
[Ravi] 50000.0
rejected before Person was built: negative salary
```

18.5.5 Module import declarations

Final in Java 25 (JEP 511). `import module java.base;` imports **all** packages exported by a module in one line (java.util, java.io, java.time, java.util.stream…). Handy for scripts, teaching and prototypes. If two imported packages contain the same simple name (e.g. `Date`), add a single-type import to resolve the ambiguity. Compact source files (18.5.3) automatically import `java.base`.

ONE IMPORT FOR A WHOLE MODULE (JAVA 25+) ModuleImport.java

```
import module java.base;          // java.util, java.time, java.util.stream, ... all at once

public class ModuleImport {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(List.of("Ravi", "Asha"));
        Collections.sort(names);
        LocalDate d = LocalDate.of(2026, 6, 1);
        System.out.println(names + " on " + d + ", " + names.stream().collect(Collectors.joining("+")));
    }
}
```

OUTPUT (JAVA 25+)

```
[Asha, Ravi] on 2026-06-01, Asha+Ravi
```

18.5.6 Scoped values

Final in Java 25 (JEP 506). A `ScopedValue` shares an **immutable** value with all code called within a bounded scope, on the current thread and on child threads of structured concurrency, without passing it as a parameter. It is a safer, faster alternative to `ThreadLocal` (16.8.6): the value cannot be changed, and it disappears automatically when the scope ends, so nothing leaks between pooled tasks. Ideal with virtual threads.

PASSING CONTEXT WITHOUT PARAMETERS (JAVA 25+) ScopedValues.java

```
public class ScopedValues {
    static final ScopedValue<String> USER = ScopedValue.newInstance();

    static void handle(String user) {
        ScopedValue.where(USER, user).run(() -> service());     // USER bound only inside run()
    }
    static void service() { repository(); }                      // no parameter passing
    static void repository() {
        System.out.println("query executed for " + USER.get());
    }
    public static void main(String[] args) {
        handle("asha");
        handle("ravi");
        System.out.println("bound outside a scope? " + USER.isBound());
    }
}
```

OUTPUT (JAVA 25+)

```
query executed for asha
query executed for ravi
bound outside a scope? false
```

	ThreadLocal	ScopedValue
Mutable?	Yes (set any time)	No: bound once per scope
Lifetime	Until removed (leaks if forgotten)	Ends with the scope automatically
Inheritance by child threads	Copies (costly)	Shared cheaply with structured-concurrency subtasks
Typical use	Legacy per-thread objects	Request context with virtual threads

18.6 Staying current

18.6.1 Preview features and --enable-preview

A **preview feature** is fully implemented but not yet permanent; the designers gather feedback and may change or remove it in the next release. Preview features are disabled by default. To use one you must enable it at **both** compile time and run time, for the exact JDK version:

Enabling preview features

```
$ javac --release 25 --enable-preview Demo.java
Note: Demo.java uses preview features of Java SE 25.
Note: Recompile with -Xlint:preview for details.
$ java --enable-preview Demo

# single-file source launcher:
$ java --enable-preview --source 25 Demo.java
```

Incubator modules (`jdk.incubator.*`, e.g. the Vector API) are experimental APIs, enabled with `--add-modules`. Never use preview or incubator features in production code.

18.6.2 Reading JEPs (JDK Enhancement Proposals)

Every significant change to Java is described in a **JEP** at `openjdk.org/jeps`. A JEP explains the **summary**, **goals** and **non-goals**, **motivation** (the problem), **description** (the design, with examples), **alternatives** considered and **risks**. The status

tells you where it stands: Draft → Candidate → Proposed to Target → Targeted → Integrated → **Closed/Delivered**. Each JDK release page (e.g. `openjdk.org/projects/jdk/25`) lists its JEPs. Reading the "Motivation" and "Description" of the JEPs in this module is the best way to understand *why* a feature exists.

JEP	Feature	Final in
286	var	10
361	Switch expressions	14
378	Text blocks	15
395	Records	16
409	Sealed classes	17
400	UTF-8 by default	18
441 / 440	Pattern matching for switch / record patterns	21
444	Virtual threads	21
431	Sequenced collections	21
456	Unnamed variables and patterns	22
485	Stream gatherers	24
511 / 512 / 513	Module imports / compact source files / flexible constructors	25
506	Scoped values	25

18.6.3 Migrating code from Java 8 to the latest LTS

1. **Build on the new JDK first** without changing code: update Maven/Gradle (and plugins such as maven-compiler-plugin) and set `<release>21</release>` or 25.
2. **Update dependencies**: old versions of Lombok, Mockito, Spring, Hibernate, ASM-based tools often break on new class-file versions. Upgrade them.
3. **Removed modules**: Java EE and CORBA modules were removed in Java 11 (`javax.xml.bind` JAXB, `javax.annotation`, JAX-WS). Add them as Maven dependencies (now under `jakarta.*`).
4. **Internal APIs**: `sun.misc.*` and other JDK internals are strongly encapsulated since Java 16/17. Run `jdeps --jdk-internals app.jar` to find uses; replace them, or temporarily use `--add-opens`.
5. **Behaviour changes**: default charset (UTF-8 since 18), security algorithms, removed GCs (CMS removed in 14), deprecated `finalize`, SecurityManager disabled.
6. **Test thoroughly**, then gradually **modernise** the code: var, records, switch expressions, text blocks, pattern matching, sequenced collections, virtual threads.
7. Adopt a policy of moving to each new **LTS** (every 2 years) so future migrations stay small.

Java 8 style	Modern equivalent
`Collections.unmodifiableList(Arrays.asList(...))`	`List.of(...)`
POJO with getters/equals/hashCode/toString	`record`
`switch` with break	switch expression with `->`
`if (o instanceof X) { X x = (X) o; … }`	`if (o instanceof X x)` / switch patterns
String concatenation with `\n`	Text block
`list.get(list.size() - 1)`	`list.getLast()`
Thread pools for blocking I/O	Virtual threads
`new String(Files.readAllBytes(p))`	`Files.readString(p)`

PRACTICE CHECKPOINT

Refactor an old-style program into modern Java using **records**, **switch expressions**, **pattern matching** and **text blocks**.

BEFORE: JAVA 8 STYLE

OldStyle.java

```
import java.util.*;

public class OldStyle {
    static abstract class Payment { }
    static class Card extends Payment {
        private final String number; private final double amount;
        Card(String number, double amount) { this.number = number; this.amount = amount; }
        String getNumber() { return number; }
        double getAmount() { return amount; }
    }
    static class Upi extends Payment {
        private final String vpa; private final double amount;
        Upi(String vpa, double amount) { this.vpa = vpa; this.amount = amount; }
        String getVpa() { return vpa; }
        double getAmount() { return amount; }
    }

    static String describe(Payment p) {
        if (p instanceof Card) {
            Card c = (Card) p;
            return "Card ending " + c.getNumber().substring(c.getNumber().length() - 4) + " : " + c.getAmount();
        } else if (p instanceof Upi) {
            Upi u = (Upi) p;
            return "UPI " + u.getVpa() + " : " + u.getAmount();
        }
        return "unknown";
    }
    static double fee(String type) {
        double fee;
        switch (type) {
            case "CARD": fee = 0.02; break;
            case "UPI":  fee = 0.0;  break;
            default:     fee = 0.01;
        }
        return fee;
    }
    public static void main(String[] args) {
        List<Payment> list = new ArrayList<>();
        list.add(new Card("4111111111111234", 2500));
        list.add(new Upi("asha@lbf", 800));
        for (Payment p : list) System.out.println(describe(p));
        System.out.println("fees: " + fee("CARD") + ", " + fee("UPI"));
        String receipt = "RECEIPT\n" +
                         "-------\n" +
                         "Total payments: " + list.size() + "\n";
        System.out.print(receipt);
    }
}
```

OUTPUT

```
Card ending 1234 : 2500.0
UPI asha@lbf : 800.0
fees: 0.02, 0.0
RECEIPT
-------
Total payments: 2
```

AFTER: MODERN JAVA 21

ModernStyle.java

```
import java.util.List;

public class ModernStyle {
    sealed interface Payment permits Card, Upi { }                      // sealed hierarchy
    record Card(String number, double amount) implements Payment { }    // records replace boilerplate
    record Upi(String vpa, double amount) implements Payment { }

    static String describe(Payment p) {
        return switch (p) {                                             // pattern matching for switch
            case Card(var number, var amount) -> "Card ending " + number.substring(number.length() - 4) + " : " +
amount;
            case Upi(var vpa, var amount) -> "UPI " + vpa + " : " + amount;
        };                                                              // exhaustive: no "unknown" branch
    }
    static double fee(String type) {
        return switch (type) {                                          // switch expression
            case "CARD" -> 0.02;
            case "UPI" -> 0.0;
            default -> 0.01;
        };
    }
    public static void main(String[] args) {
        var list = List.of(new Card("4111111111111234", 2500), new Upi("asha@lbf", 800));   // var + List.of
        list.forEach(p -> System.out.println(describe(p)));
        System.out.println("fees: " + fee("CARD") + ", " + fee("UPI"));
        System.out.print("""
                RECEIPT
                -------
                Total payments: %d
                """.formatted(list.size()));                             // text block
        if (list.getFirst() instanceof Card c && c.amount() > 1000)      // instanceof pattern + Java 21 getFirst
            System.out.println("large card payment: " + c.amount());
    }
}
```

OUTPUT

```
Card ending 1234 : 2500.0
UPI asha@lbf : 800.0
fees: 0.02, 0.0
RECEIPT
-------
Total payments: 2
large card payment: 2500.0
```

The modern version is about half the length, cannot forget a payment type (the compiler checks the sealed switch), has no casts, and its data classes get equals/hashCode/toString for free.

MODULE 19 · ADVANCED

JVM internals and memory management

Minimum duration **3 days** | Level **Advanced**

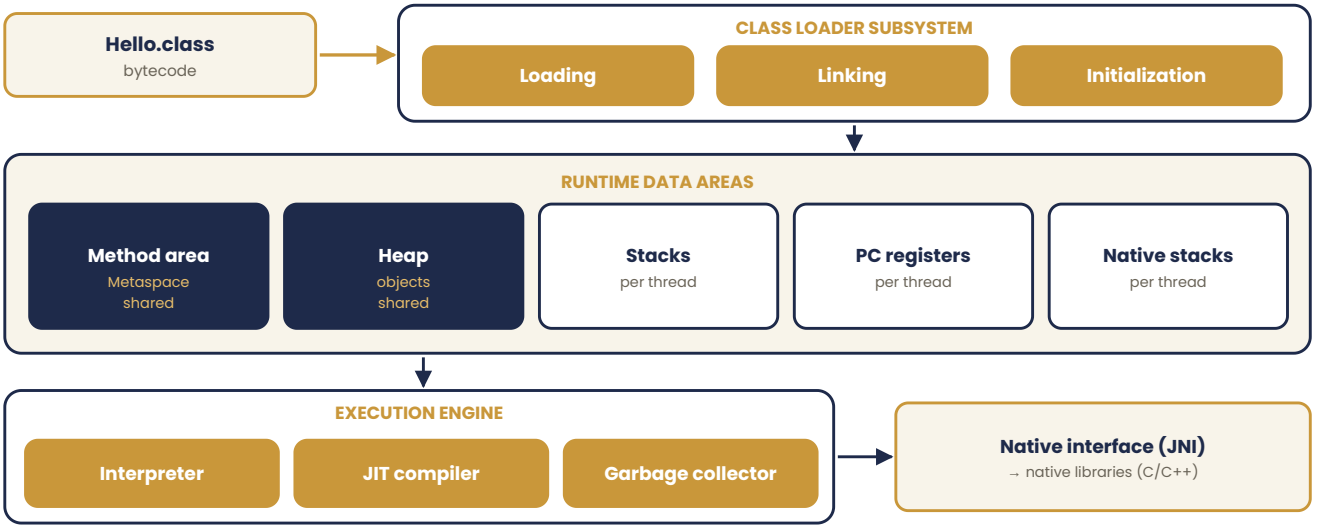
This module opens the JVM: how classes are loaded, how memory is laid out, how garbage collection works, and how to tune and diagnose performance.

IN THIS MODULE

19.1 JVM architecture
19.2 Class loading
19.3 Runtime memory areas
19.4 Execution engine
19.5 Garbage collection
19.6 Memory problems
19.7 JVM tuning and monitoring
19.8 Bytecode

19.1 JVM architecture

The **Java Virtual Machine** is the program that runs your compiled bytecode (.class files). The JVM most people use is Oracle's/OpenJDK's **HotSpot**. It has three big subsystems: the **class loader subsystem** brings classes into memory, the **runtime data areas** hold everything the program needs while running, and the **execution engine** executes the bytecode. A **native interface** connects it to C/C++ libraries.



THE JVM: CLASS LOADER SUBSYSTEM, RUNTIME DATA AREAS AND EXECUTION ENGINE

19.1.1 Class loader subsystem

Finds .class files (in the JDK, on the classpath/module path, in JARs or even over the network), reads their bytes, verifies them and creates `Class` objects in memory. Classes are loaded **lazily**: only when first used. Details in 19.2.

19.1.2 Runtime data areas

The memory the JVM manages while the program runs. The **heap** and the **method area** (Metaspace) are created at startup and **shared** by all threads. Each thread gets its own **JVM stack**, **PC register** and **native method stack**. Details in 19.3.

19.1.3 Execution engine

Executes bytecode: the **interpreter** runs it instruction by instruction, the **JIT compiler** turns hot code into fast native machine code, and the **garbage collector** reclaims memory of unreachable objects. Details in 19.4 and 19.5.

19.1.4 Native method interface (JNI)

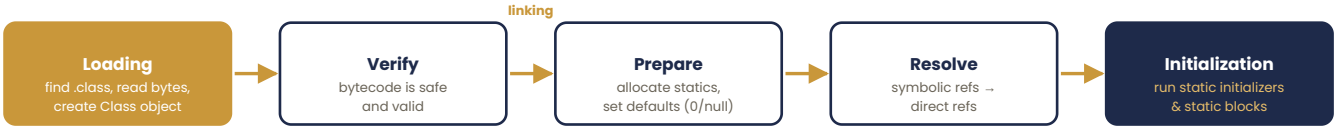
The **Java Native Interface** lets Java call functions written in C/C++ (and vice versa). Methods declared `native` have no Java body; their implementation lives in a native library loaded with `System.loadLibrary("name")`. The JDK itself uses JNI for file, network and OS operations (`System.currentTimeMillis()`, `Object.hashCode()` are native). Since Java 22 the safer, easier **Foreign Function & Memory API** (`java.lang.foreign`) is the modern alternative.

DECLARING A NATIVE METHOD — java

```java
public class NativeDemo {
    static { System.loadLibrary("fastmath"); }   // loads libfastmath.so / fastmath.dll
    public native double fastSqrt(double x);       // implemented in C, no Java body
}
```

19.2 Class loading

19.2.1 Loading, linking (verify, prepare, resolve), initialization



THE THREE PHASES OF CLASS LOADING; LINKING ITSELF HAS THREE STEPS

Phase	What happens
Loading	A class loader locates the binary (.class) and creates a `java.lang.Class` object in Metaspace
Verification	Bytecode verifier checks the format and type safety (no stack overflow tricks, valid jumps, correct types), protecting the JVM from corrupt or malicious code
Preparation	Memory for static fields is allocated and set to **default** values (0, false, null), not yet the values you wrote
Resolution	Symbolic references in the constant pool ("java/lang/String") are replaced by direct references (may happen lazily)
Initialization	Static field initializers and static blocks run, top to bottom, exactly once, on first active use (new, static method call, static field access, Class.forName)

CLASSES ARE INITIALISED LAZILY AND ONLY ONCE — ClassInitOrder.java

```java
public class ClassInitOrder {
    static class Config {
        static int port = readPort();                // preparation: port = 0; initialization: readPort()
        static { System.out.println("  Config static block runs (initialization)"); }
        static int readPort() { System.out.println("  Config.port initializer runs"); return 8080; }
    }
    static class Unused {
        static { System.out.println("  Unused initialised (never printed)"); }
    }
    public static void main(String[] args) throws Exception {
        System.out.println("main starts: Config not initialised yet");
        Class<?> c = Config.class;                   // a class literal does NOT initialise
        System.out.println("got class literal " + c.getSimpleName());
        System.out.println("first active use -> port = " + Config.port);
        System.out.println("second use       -> port = " + Config.port + " (no re-initialisation)");
        Class.forName("ClassInitOrder$Config");      // already initialised: nothing printed
    }
}
```

OUTPUT

```
main starts: Config not initialised yet
got class literal Config
  Config.port initializer runs
  Config static block runs (initialization)
first active use -> port = 8080
second use       -> port = 8080 (no re-initialisation)
```

19.2.2 Bootstrap, platform and application class loaders

Class loader	Loads	Implemented in
Bootstrap	Core JDK classes from `java.base` etc. (java.lang.String, java.util.*)	Native code (C++); shown as `null` in Java
Platform (Extension before Java 9)	Other JDK modules such as `java.sql`, `java.xml`	Java
Application (System)	Your classes and libraries on the classpath / module path	Java

WHICH LOADER LOADED WHICH CLASS? ClassLoaders.java

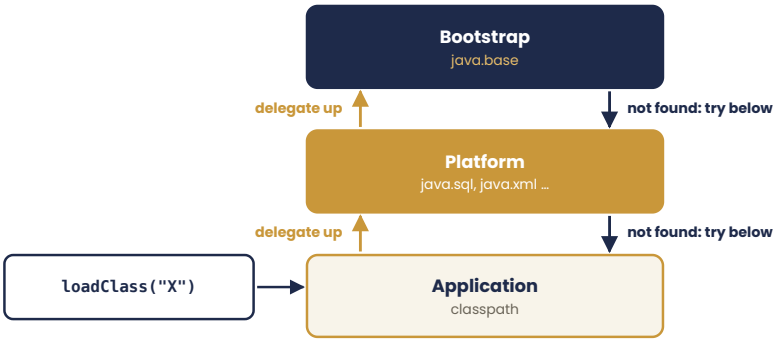
```
public class ClassLoaders {
    public static void main(String[] args) {
        System.out.println("String       -> " + String.class.getClassLoader());             // bootstrap
        ClassLoader sql = java.sql.Connection.class.getClassLoader();
        System.out.println("java.sql     -> " + sql.getName());                              // platform
        ClassLoader app = ClassLoaders.class.getClassLoader();
        System.out.println("my class     -> " + app.getName());                              // application
        System.out.println("app parent   -> " + app.getParent().getName());
        System.out.println("platform parent -> " + app.getParent().getParent() + " (bootstrap)");
    }
}
```

OUTPUT

```
String       -> null
java.sql     -> platform
my class     -> app
app parent   -> platform
platform parent -> null (bootstrap)
```

19.2.3 Parent delegation model

When a class loader is asked to load a class, it first **delegates** the request to its **parent**. Only if the parent cannot find the class does the loader try itself. So `java.lang.String` is always loaded by the bootstrap loader, even if someone puts a fake String.class on the classpath: core classes cannot be replaced (security), and each class is loaded only once (consistency).



REQUESTS TRAVEL UP TO THE BOOTSTRAP LOADER FIRST; LOADING HAPPENS AT THE HIGHEST LOADER THAT CAN FIND THE CLASS

19.2.4 Custom class loaders

You can extend `ClassLoader` (usually overriding `findClass`) to load classes from unusual places: a database, an encrypted file, a network, or generated bytecode. Application servers (Tomcat), plugin systems and hot-reload tools use custom loaders, often one per web app, so each app can have its own library versions. Two classes with the same name loaded by **different** loaders are **different** classes.

LOADING A CLASS FROM A PLUGIN FOLDER AT RUN TIME

CustomLoader.java

```
import java.net.URL;
import java.net.URLClassLoader;
import java.nio.file.*;
import javax.tools.ToolProvider;

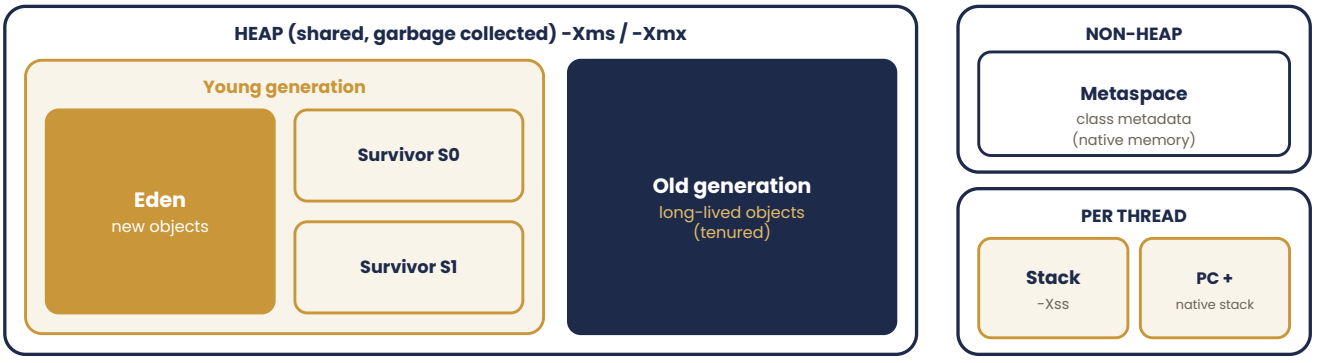
public class CustomLoader {
    public static void main(String[] args) throws Exception {
        Path plugins = Files.createDirectories(Path.of("plugins"));
        Path src = plugins.resolve("Greeter.java");
        Files.writeString(src, "public class Greeter { public String hello() { return \"Hello from a plugin!\"; } }");
        ToolProvider.getSystemJavaCompiler().run(null, null, null, src.toString());   // compile at run time

        try (URLClassLoader loader = new URLClassLoader(new URL[] { plugins.toUri().toURL() })) {
            Class<?> cls = loader.loadClass("Greeter");                              // loaded from plugins/
            Object g = cls.getDeclaredConstructor().newInstance();
            System.out.println(cls.getMethod("hello").invoke(g));
            System.out.println("loaded by " + cls.getClassLoader().getClass().getSimpleName());
        }
    }
}
```

OUTPUT

```
Hello from a plugin!
loaded by URLClassLoader
```

19.3 Runtime memory areas

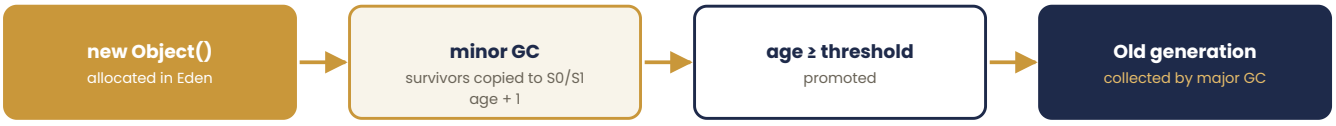


JVM MEMORY LAYOUT (HOTSPOT, GENERATIONAL COLLECTORS)

19.3.1 Heap (young generation: Eden, Survivor; old generation)

All objects and arrays live on the **heap**. Generational collectors divide it into:

- **Eden**: where almost every new object is allocated (very fast: just bump a pointer).
- **Survivor spaces S0 and S1**: objects that survived a young GC are copied here; they move back and forth, and each survival increases their **age**.
- **Old (tenured) generation**: objects that survived enough GCs (age threshold, often up to 15) are **promoted** here; also very large objects. Collected less often.



THE JOURNEY OF A LONG-LIVED OBJECT

19.3.2 Stack and stack frames (one per thread)

Each thread has its own **JVM stack** of **frames**, one per method call in progress (9.2.3). A frame holds the method's **local variable array** (parameters and locals: primitives and references), an **operand stack** used by bytecode instructions for

calculations, and a reference to the class's constant pool. Frames are pushed on call and popped on return, so stack memory needs no garbage collection. Size per thread: `-Xss` (default 1 MB on 64-bit Linux/Windows); too deep recursion → StackOverflowError.

19.3.3 Metaspace (replaced PermGen in Java 8)

The **method area** stores per-class data: the class structure, method bytecode, the runtime constant pool, static variable metadata, JIT-compiled code info. Up to Java 7 it lived in a fixed-size heap region called **PermGen**, which often ran out ("java.lang.OutOfMemoryError: PermGen space") in apps that loaded many classes. Since Java 8 it is **Metaspace**, in native memory, growing automatically (limit with `-XX:MaxMetaspaceSize`). Static *values* and interned strings live on the heap.

19.3.4 Program counter register

Each thread has a **PC register** holding the address of the bytecode instruction currently being executed (undefined while running a native method). It lets the JVM switch between threads and resume each exactly where it stopped.

19.3.5 Native method stack

Used when a thread calls **native** (C/C++) methods through JNI; it is the ordinary C stack for that code. In HotSpot it is combined with the JVM stack.

19.3.6 Where variables, objects and the String pool live

Thing	Lives in
Local primitive variable (`int count = 5`)	Stack frame of that method call
Local reference variable (`Student s`)	Stack frame (the reference); the object it points to is on the heap
Objects and arrays (`new Student()` , `new int[10]`)	Heap
Instance fields	Inside their object on the heap
Static fields	Heap (inside the Class object's storage); metadata in Metaspace
String literals / String pool (`"Java"`)	Heap (moved from PermGen in Java 7)
Class metadata, method bytecode	Metaspace (native memory)
JIT-compiled machine code	Code cache (native memory)

MAPPING CODE TO MEMORY AREAS WhereThingsLive.java

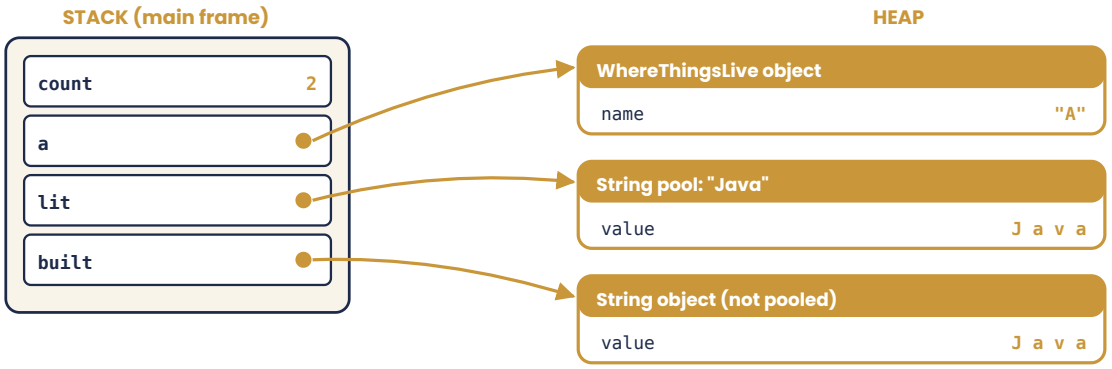
```
public class WhereThingsLive {
    static int created = 0;                       // static field: heap (class storage)
    String name;                                  // instance field: inside the object on the heap

    WhereThingsLive(String name) { this.name = name; created++; }

    public static void main(String[] args) {
        int count = 2;                            // primitive local: stack
        WhereThingsLive a = new WhereThingsLive("A");   // reference on stack -> object on heap
        String lit = "Java";                      // literal: String pool (heap)
        String built = new String("Java");        // new object on the heap, outside the pool
        System.out.println("count=" + count + ", created=" + created + ", a.name=" + a.name);
        System.out.println("pool literal == interned copy? " + (lit == built.intern()));
        Runtime rt = Runtime.getRuntime();
        System.out.println("heap max is set: " + (rt.maxMemory() > 0) + ", free <= total: " + (rt.freeMemory() <=
rt.totalMemory()));
    }
}
```

OUTPUT

```
count=2, created=1, a.name=A
pool literal == interned copy? true
heap max is set: true, free <= total: true
```



STACK FRAME OF MAIN AND THE OBJECTS IT REFERS TO

19.4 Execution engine

19.4.1 Interpreter

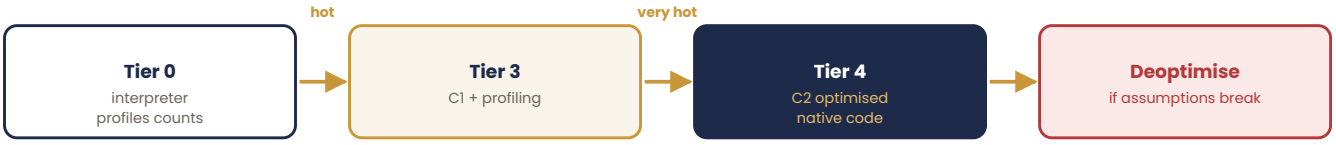
When a method first runs, the JVM **interprets** its bytecode: it reads each instruction and performs it. Interpretation starts immediately (no compile delay) but is slow, because every instruction is decoded again on every execution.

19.4.2 JIT compiler (C1, C2, tiered compilation)

HotSpot counts how often each method and loop runs. Code that becomes **hot** is compiled **just in time** to native machine code, which then replaces the interpreted version. HotSpot has two JIT compilers:

Compiler	Also called	Characteristics
C1	client compiler	Compiles quickly with light optimisations; adds profiling
C2	server compiler	Slower to compile, aggressive optimisations using the profile data; peak speed

Tiered compilation (default) combines them: code starts interpreted (tier 0), hot code is compiled by C1 with profiling (tiers 1–3), and very hot code is recompiled by C2 (tier 4). This is why Java programs "warm up": they get faster after running for a while. If an assumption made by C2 turns out wrong (e.g. a new subclass is loaded), the code is **deoptimised** back to the interpreter and recompiled later.



TIERED COMPILATION IN HOTSPOT

WATCHING THE JIT MAKE CODE FASTER JitWarmup.java

```
public class JitWarmup {
    static long work(int n) {
        long s = 0;
        for (int i = 0; i < n; i++) s += (i % 7) * (i % 3);
        return s;
    }
    public static void main(String[] args) {
        long first = 0, last = 0;
        for (int round = 1; round <= 10; round++) {
            long t = System.nanoTime();
            work(2_000_000);
            long ms = System.nanoTime() - t;
            if (round == 1) first = ms;
            last = ms;
        }
        System.out.println("10th round faster than 1st (JIT warm-up): " + (last < first));
    }
}
```

ONE POSSIBLE OUTPUT

```
10th round faster than 1st (JIT warm-up): true
```

TIP

Run with `-XX:+PrintCompilation` to see methods as they are JIT-compiled. Because of warm-up, micro-benchmarks written with System.nanoTime() are unreliable; serious benchmarks use **JMH** (Java Microbenchmark Harness).

19.4.3 HotSpot optimizations (inlining, escape analysis)

Optimisation	What it does	Effect
Inlining	Replaces a call to a small method (getters, tiny helpers) with the method's body	Removes call overhead and enables further optimisations; the most important one
Escape analysis	Detects objects that never leave ("escape") a method	Can skip heap allocation entirely (scalar replacement) and remove locks on them
Loop unrolling	Repeats the loop body to reduce jumps and checks	Faster loops
Dead code elimination	Removes code whose result is never used	Less work
Range-check elimination	Proves array indexes are valid once instead of every access	Faster array loops
Devirtualisation	If only one implementation of a method is loaded, calls it directly	Virtual calls become cheap
Lock elision / coarsening	Removes locks on thread-local objects; merges adjacent locks	Cheaper synchronization

ESCAPE ANALYSIS REMOVES AN ALLOCATION java

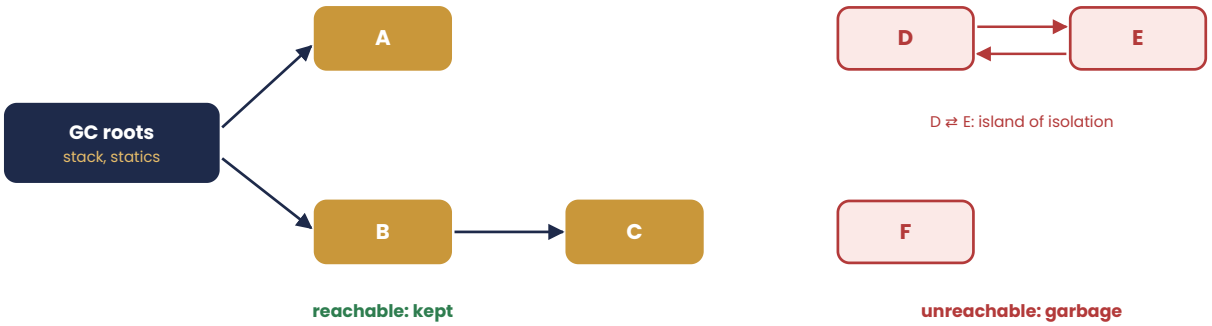
```
double distance(double x, double y) {
    Point p = new Point(x, y);        // p never escapes this method ...
    return Math.sqrt(p.x * p.x + p.y * p.y);
}
// ... so after escape analysis the JIT compiles it as if it were:
double distance(double x, double y) {
    return Math.sqrt(x * x + y * y);  // no object allocated at all
}
```

19.5 Garbage collection

19.5.1 What garbage is (unreachable objects) and GC roots

Java has no `delete` or `free()`. Memory is reclaimed automatically by the **garbage collector**. An object is **garbage** when it is no longer **reachable**: there is no chain of references to it from any **GC root**. GC roots are references the JVM always considers live:

- local variables and parameters in the stack frames of all live threads;
- static fields of loaded classes;
- active threads themselves;
- JNI references held by native code; objects used as monitors (synchronized).



REACHABILITY: A, B, C CAN BE REACHED FROM ROOTS; D, E (EVEN THOUGH THEY REFERENCE EACH OTHER) AND F CANNOT

19.5.2 Making objects eligible: nulling, reassigning, scope exit, islands of isolation

FOUR WAYS AN OBJECT BECOMES GARBAGE

Eligibility.java

```
public class Eligibility {
    static class Node {
        String name; Node next;
        Node(String n) { name = n; }
    }
    static void method() {
        Node local = new Node("local");      // 3. eligible when the method returns (scope exit)
    }
    public static void main(String[] args) {
        Node a = new Node("A");
        a = null;                            // 1. nulling: "A" is unreachable now

        Node b = new Node("B1");
        b = new Node("B2");                  // 2. reassigning: "B1" is unreachable now

        method();                            // 3. "local" is unreachable after the call

        Node x = new Node("X"), y = new Node("Y");
        x.next = y; y.next = x;              // 4. island of isolation:
        x = null; y = null;                  //    X and Y reference each other but nothing reaches them
        System.out.println("All five objects are now eligible for GC; only b (B2) is still reachable: " + b.name);
    }
}
```

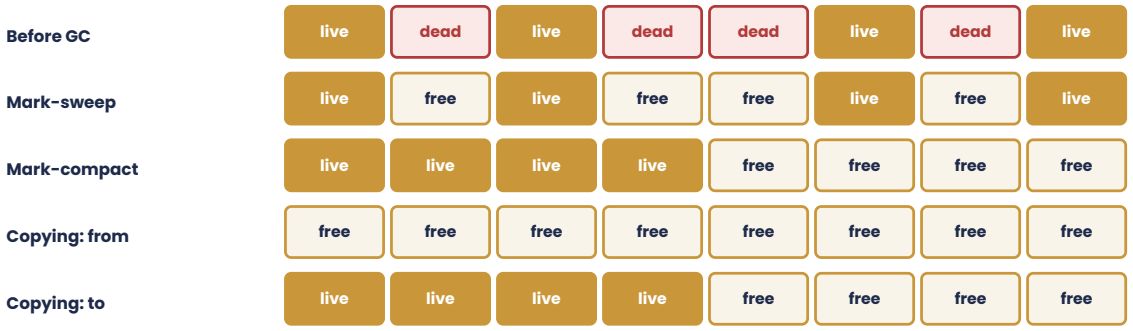
OUTPUT

```
All five objects are now eligible for GC; only b (B2) is still reachable: B2
```

Eligible does not mean collected *now*. The GC decides when to run, usually when Eden fills up. Java uses **reachability tracing**, not reference counting, so cycles (islands of isolation) are collected correctly.

19.5.3 GC algorithms: mark-sweep, mark-compact, copying

Algorithm	How it works	Pros	Cons
Mark-sweep	Mark: trace from roots and mark live objects. Sweep: free unmarked ones	Simple, no copying	Leaves **fragmented** free space
Mark-compact	Mark, then slide live objects together to one end	No fragmentation; fast allocation	Moving objects is costly
Copying	Copy live objects from one space to an empty space; the old space is wiped entirely	Very fast when most objects are dead; compacts for free	Needs extra empty space



RESULTS OF THE THREE BASIC ALGORITHMS ON THE SAME HEAP

19.5.4 Generational hypothesis; minor vs major vs full GC

The **weak generational hypothesis**: *most objects die young* (temporary strings, iterators, request objects), and the few that survive tend to live long. So the heap is split by age: the young generation is collected often with a fast **copying** collector (cost depends only on the few survivors), and the old generation is collected rarely.

Collection	Region	Frequency / cost
Minor GC (young GC)	Young generation (Eden + survivors)	Frequent, short pauses (milliseconds)
Major GC	Old generation (often concurrent in modern collectors)	Less frequent, longer
Full GC	Entire heap (+ Metaspace), usually stop-the-world	Rare; long pauses; a warning sign if frequent



TYPICAL SURVIVAL PROFILE: MOST OBJECTS DIE BEFORE THEIR FIRST GC (ILLUSTRATIVE)

RUN WITH -XMX64M -XLOG:GC TO SEE MINOR COLLECTIONS

GcLogDemo.java

```
import java.util.ArrayList;
import java.util.List;

public class GcLogDemo {
    public static void main(String[] args) {
        List<int[]> keep = new ArrayList<>();
        long checksum = 0;
        for (int i = 0; i < 300; i++) {
            int[] temp = new int[100_000];          // ~400 KB of short-lived garbage per iteration
            temp[0] = i;
            checksum += temp[0];
            if (i % 50 == 0) keep.add(new int[10_000]);   // a few long-lived objects
        }
        System.out.println("done, checksum " + checksum + ", kept " + keep.size());
    }
}
```

ONE POSSIBLE OUTPUT

```
[0.010s][info][gc] Using G1
[0.100s][info][gc] GC(0) Pause Young (Normal) (G1 Evacuation Pause) 6M->1M(64M) 0.950ms
[0.110s][info][gc] GC(1) Pause Young (Normal) (G1 Evacuation Pause) 23M->1M(64M) 0.630ms
[0.122s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 37M->1M(64M) 0.641ms
[0.125s][info][gc] GC(3) Pause Young (Normal) (G1 Evacuation Pause) 37M->1M(64M) 0.495ms
[0.127s][info][gc] GC(4) Pause Young (Normal) (G1 Evacuation Pause) 38M->1M(64M) 0.506ms
done, checksum 44850, kept 6
```

Read a G1 log line like this: `GC(3) Pause Young (Normal) (G1 Evacuation Pause) 24M->2M(64M) 1.2ms` = collection number 3, a young pause, heap use went from 24 MB to 2 MB (most objects were garbage), heap size 64 MB, pause lasted 1.2 ms.

19.5.5 Garbage collectors

HotSpot ships several collectors that trade **throughput** (total work done), **latency** (pause length) and **memory footprint** against each other. "Stop-the-world" (STW) means all application threads are paused.

19.5.5.1 Serial

One GC thread, stop-the-world for both generations. Smallest overhead; good for small heaps, single-core machines and small containers. `-XX:+UseSerialGC`

19.5.5.2 Parallel

Many GC threads, still stop-the-world, maximising **throughput**. Good for batch jobs where total time matters more than individual pauses. `-XX:+UseParallelGC`

19.5.5.3 G1 (default)

Garbage-First, the default since Java 9. The heap is split into many equal **regions** (1–32 MB), each Eden, Survivor, Old or Humongous. G1 marks the old generation concurrently and then collects the regions with the most garbage first, aiming for a **pause-time goal** (`-XX:MaxGCPauseMillis=200` by default). A balanced choice for most applications.



G1 HEAP: REGIONS OF DIFFERENT KINDS INSTEAD OF FIXED CONTIGUOUS GENERATIONS (HUM = HUMONGOUS OBJECT)

19.5.5.4 ZGC and Shenandoah (low latency)

Concurrent collectors that do almost all work (marking, relocating/compacting) **while the application runs**, keeping pauses typically **under one millisecond** regardless of heap size (even terabytes). **ZGC** (Oracle, generational since Java 21, default mode in 23) uses coloured pointers and load barriers; **Shenandoah** (Red Hat) uses forwarding pointers. They cost some throughput and CPU. Use for latency-sensitive services: trading, gaming, large interactive systems. `-XX:+UseZGC`, `-XX:+UseShenandoahGC`.

Collector	Flag	Pauses	Best for
Serial	`-XX:+UseSerialGC`	STW, single thread	Tiny heaps, 1 CPU
Parallel	`-XX:+UseParallelGC`	STW, multi-thread	Throughput, batch jobs
G1	`-XX:+UseG1GC` (default)	Short, mostly predictable	General purpose
ZGC	`-XX:+UseZGC`	< 1 ms, concurrent	Low latency, huge heaps
Shenandoah	`-XX:+UseShenandoahGC`	< 10 ms, concurrent	Low latency (OpenJDK builds)

19.5.6 System.gc() and why not to rely on it

`System.gc()` only **suggests** a collection; the JVM may ignore it (and does with `-XX:+DisableExplicitGC`). When honoured it usually triggers a **full, stop-the-world** GC, which hurts performance, and it never frees reachable objects, so it cannot fix a memory leak. The GC knows better than your code when to run. Legitimate uses are rare: benchmarks and tests.

19.5.7 Reference types: strong, soft, weak, phantom

Type	Collected when…	Use
Strong (normal `Object o = …`)	Never while strongly reachable	Everything by default
Soft (`SoftReference`)	Only when memory is running low	Memory-sensitive caches
Weak (`WeakReference`)	At the next GC once no strong refs remain	Canonical maps, `WeakHashMap` (metadata about objects), listeners
Phantom (`PhantomReference`)	After finalization; `get()` always returns null	Post-mortem cleanup (`Cleaner`), replacing finalize()

WEAK, SOFT AND WEAKHASHMAP BEHAVIOUR ReferenceTypes.java

```
import java.lang.ref.*;
import java.util.WeakHashMap;

public class ReferenceTypes {
    public static void main(String[] args) throws InterruptedException {
        Object strongObj = new Object();
        WeakReference<Object> weak = new WeakReference<>(new Object());       // only weakly reachable
        SoftReference<byte[]> soft = new SoftReference<>(new byte[1024]);     // kept unless memory is low
        WeakReference<Object> weakToStrong = new WeakReference<>(strongObj);

        System.gc(); Thread.sleep(100);                                       // demo only!
        System.out.println("weak ref (no strong refs) cleared : " + (weak.get() == null));
        System.out.println("weak ref (strong ref exists) kept : " + (weakToStrong.get() != null));
        System.out.println("soft ref kept (memory is fine)    : " + (soft.get() != null));

        WeakHashMap<Object, String> meta = new WeakHashMap<>();
        Object key = new Object();
        meta.put(key, "metadata");
        System.out.println("WeakHashMap size before : " + meta.size());
        key = null;                                                           // key no longer strongly reachable
        System.gc(); Thread.sleep(100);
        System.out.println("WeakHashMap size after  : " + meta.size());
    }
}
```

OUTPUT

```
weak ref (no strong refs) cleared : true
weak ref (strong ref exists) kept : true
soft ref kept (memory is fine)    : true
WeakHashMap size before : 1
WeakHashMap size after  : 0
```

19.6 Memory problems

19.6.1 Memory leaks in Java (static collections, listeners, unclosed resources)

A Java memory leak is an object that is **no longer needed but still reachable**, so the GC can never free it. Memory use creeps up until an OutOfMemoryError. Common causes:

Cause	Example	Fix
Static collections that only grow	`static List<Order> cache` with no eviction	Bounded caches (LRU, 14.6.2), Caffeine, weak/soft refs
Listeners / callbacks never removed	`button.addListener(this)` without removeListener	Unregister; use weak listeners
Unclosed resources	Streams, connections, ResultSets not closed	try-with-resources
ThreadLocal in thread pools	Value set, never removed	`remove()` in finally
Inner class holding the outer object	Long-lived anonymous class captures `this`	Static nested classes
Mutable keys in HashMap/Set	Key fields changed after insertion; entry unreachable but stored	Immutable keys
Class-loader leaks	Redeploying web apps while a thread keeps old classes alive	Stop threads, deregister drivers on shutdown

A LEAK: RUN WITH -XMX64M -XX:+HEAPDUMPONOUTOFMEMORYERROR LeakyCache.java

```
import java.util.ArrayList;
import java.util.List;

public class LeakyCache {
    static final List<byte[]> CACHE = new ArrayList<>();      // static: lives forever

    public static void main(String[] args) {
        int requests = 0;
        while (true) {
            CACHE.add(new byte[1024 * 1024]);                 // 1 MB per "request", never removed
            requests++;
            if (requests % 20 == 0) System.out.println("handled " + requests + " requests");
        }
    }
}
```

OUTPUT

```
handled 20 requests
java.lang.OutOfMemoryError: Java heap space
Dumping heap to leak.hprof ...
Heap dump file created [36118312 bytes in 0.014 secs]

Exception: java.lang.OutOfMemoryError thrown from the UncaughtExceptionHandler in thread "main"
```

19.6.2 OutOfMemoryError types

Message	Meaning	Typical cause / fix
`Java heap space`	The heap is full of reachable objects	Leak (fix code) or genuinely too little heap (raise `-Xmx`)
`GC overhead limit exceeded`	More than 98% of time spent in GC recovering under 2% of heap	Almost-full heap; same as above
`Metaspace`	Too much class metadata	Class-loader leaks, too many generated classes; `-XX:MaxMetaspaceSize`

Message	Meaning	Typical cause / fix
`unable to create native thread`	OS refused to create more threads	Too many threads: use pools or virtual threads; OS limits
`Direct buffer memory`	Off-heap NIO buffers exhausted	Release buffers; `-XX:MaxDirectMemorySize`
`Requested array size exceeds VM limit`	Array larger than the JVM allows	Bug in size calculation

19.6.3 StackOverflowError

Thrown when a thread's stack is full: almost always **infinite or too deep recursion** (9.7.4). The stack trace shows the same frames repeating. Fix the base case, convert deep recursion to iteration, or (if the depth is legitimate) increase the stack with `-Xss` or run that work in a thread created with a larger stack size.

STACK SIZE LIMITS RECURSION DEPTH StackDepth.java

```
public class StackDepth {
    static int depth = 0;
    static void recurse() { depth++; recurse(); }
    public static void main(String[] args) throws InterruptedException {
        try { recurse(); } catch (StackOverflowError e) { System.out.println("main thread depth > 5000: " + (depth >
5000)); }
        int small = depth;
        depth = 0;
        Thread big = new Thread(null, () -> {
            try { recurse(); } catch (StackOverflowError e) { }
        }, "big-stack", 64L * 1024 * 1024);          // 64 MB stack for this thread
        big.start(); big.join();
        System.out.println("64 MB stack allows deeper recursion: " + (depth > small));
    }
}
```

OUTPUT

```
main thread depth > 5000: true
64 MB stack allows deeper recursion: true
```

19.7 JVM tuning and monitoring

19.7.1 Heap flags: -Xms, -Xmx, -Xss

Flag	Meaning	Example
`-Xms`	Initial heap size	`-Xms512m`
`-Xmx`	Maximum heap size (default: 1/4 of physical RAM)	`-Xmx2g`
`-Xss`	Stack size per thread	`-Xss2m`
`-XX:MaxMetaspaceSize`	Cap on Metaspace	`-XX:MaxMetaspaceSize=256m`
`-XX:MaxRAMPercentage`	Heap as % of container memory (Docker/Kubernetes)	`-XX:MaxRAMPercentage=75`
`-XX:+HeapDumpOnOutOfMemoryError`	Write a heap dump when OOM happens	with `-XX:HeapDumpPath=dumps/`

TIP

In production, servers often set `-Xms` equal to `-Xmx` so the heap never resizes. Always measure before tuning: the defaults are good for most applications.

Setting and checking JVM flags

```
$ java -Xms256m -Xmx1g -Xss1m -jar college-app.jar
$ java -XX:+PrintFlagsFinal -version | grep -E " UseG1GC | MaxHeapSize | ThreadStackSize "
   size_t MaxHeapSize          = 1568669696      {product} {ergonomic}
     intx ThreadStackSize      = 1024            {pd product} {default}
     bool UseG1GC              = true            {product} {ergonomic}
```

19.7.2 Choosing a GC with JVM flags

Selecting a collector

```
$ java -XX:+UseSerialGC   -jar app.jar     # small heap, one CPU
$ java -XX:+UseParallelGC -jar app.jar     # throughput, batch processing
$ java -XX:+UseG1GC -XX:MaxGCPauseMillis=100 -jar app.jar   # default, with a pause goal
$ java -XX:+UseZGC        -jar app.jar     # sub-millisecond pauses
```

19.7.3 GC logs

Unified logging (Java 9+) uses `-Xlog`. `-Xlog:gc` prints one line per collection; `-Xlog:gc*` prints details; add `:file=gc.log:time,uptime,level,tags` to write to a file. Tools like GCViewer or GCeasy turn logs into charts of pause times and heap usage.

GC log of the leaking program: memory is not freed (27M->27M, 61M->61M) and full GCs follow, the signature of a leak

```
$ java -Xmx64m -Xlog:gc LeakyCache
[0.004s][info][gc] Using G1
[0.040s][info][gc] GC(0) Pause Young (Concurrent Start) (G1 Humongous Allocation) 27M->27M(64M) 0.727ms
[0.041s][info][gc] GC(3) Concurrent Mark Cycle
[0.064s][info][gc] GC(4) Pause Young (Prepare Mixed) (G1 Humongous Allocation) (Evacuation Failure) 61M->61M(64M)
0.690ms
[0.067s][info][gc] GC(5) Pause Full (G1 Compaction Pause) 61M->61M(64M) 3.241ms
[0.072s][info][gc] GC(6) Pause Full (G1 Compaction Pause) 61M->61M(64M) 4.192ms

$ java -Xlog:gc*:file=gc.log:time,uptime -jar app.jar    # detailed log to a file
```

19.7.4 Tools: jps, jstack, jmap, jcmd, JConsole, VisualVM, Java Flight Recorder

Tool	Purpose	Example
`jps`	List running JVMs and their process IDs	`jps -l`
`jstack`	Thread dump: every thread's stack and locks; detects deadlocks	`jstack 24520`
`jmap`	Heap histogram or heap dump	`jmap -dump:live,format=b,file=heap.hprof 24520`
`jcmd`	Swiss-army knife: all of the above and more	`jcmd 24520 GC.class_histogram`, `Thread.print`, `GC.heap_dump`, `VM.flags`
`jstat`	GC statistics sampled over time	`jstat -gcutil 24520 1000`
JConsole	Simple GUI: memory, threads, classes, MBeans	`jconsole`
VisualVM	GUI profiler: CPU/memory sampling, heap dump browser	separate download
Java Flight Recorder + Mission Control	Low-overhead production profiling and event recording	`-XX:StartFlightRecording=duration=60s,filename=rec.jfr`
Eclipse MAT	Analyses heap dumps; finds leak suspects and dominator trees	open the .hprof file

19.7.5 Heap dumps and thread dumps

A **heap dump** (.hprof) is a snapshot of every object in the heap with its references. Open it in Eclipse MAT or VisualVM to see which objects use the most memory and, crucially, the **path to GC roots** that keeps them alive: that path points at the leaking code. A **thread dump** is a snapshot of every thread's stack and the locks it holds or waits for: the tool for hangs, deadlocks and high CPU. Take several thread dumps a few seconds apart to see which threads are stuck.

Finding the process, a heap histogram and a heap dump (real output)

```
$ jps -l
24520 BankDeadlock

$ jcmd 24520 GC.class_histogram | head -6
24520:
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:          8049         352200  [B (java.base@21.0.10)
   2:          7969         191256  java.lang.String (java.base@21.0.10)
   3:          1504         186040  java.lang.Class (java.base@21.0.10)

$ jcmd 24520 GC.heap_dump /tmp/heap.hprof
Dumping heap to /tmp/heap.hprof ...
Heap dump file created [3581140 bytes in 0.022 secs]
```

19.8 Bytecode

19.8.1 Reading bytecode with javap -c

`javac` compiles source code into **bytecode**: compact instructions for the JVM's **stack machine**. Instructions push values onto the frame's operand stack, operate on them, and store them back into local variable slots. `javap -c ClassName` disassembles a .class file so you can read them; `javap -v` also shows the constant pool.

CALC.JAVA

Calc.java

```
public class Calc {
    static int add(int a, int b) {
        int sum = a + b;
        return sum;
    }
    public static void main(String[] args) {
        String msg = "Total: ";
        int r = add(2, 3);
        System.out.println(msg + r);
    }
}
```

OUTPUT

```
Total: 5
```

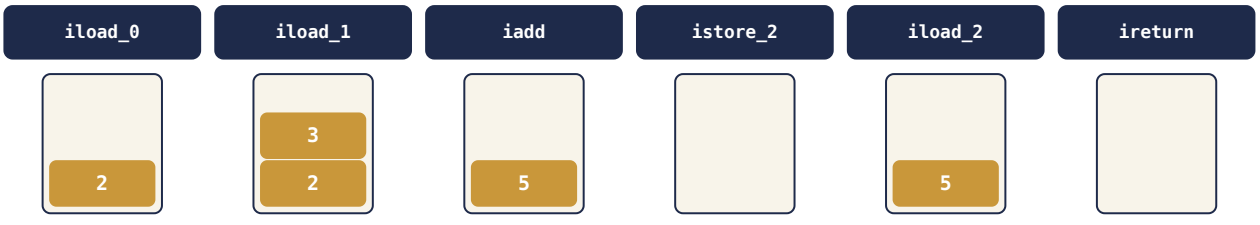
The bytecode javac produced (real output)

```
$ javac Calc.java
$ javap -c Calc
Compiled from "Calc.java"
public class Calc {
  public Calc();
    Code:
       0: aload_0
       1: invokespecial #1                  // Method java/lang/Object."<init>":()V
       4: return

  static int add(int, int);
    Code:
       0: iload_0
       1: iload_1
       2: iadd
       3: istore_2
       4: iload_2
       5: ireturn

  public static void main(java.lang.String[]);
    Code:
       0: ldc           #7                  // String Total:
       2: astore_1
       3: iconst_2
       4: iconst_3
       5: invokestatic  #9                  // Method add:(II)I
       8: istore_2
       9: getstatic     #15                 // Field java/lang/System.out:Ljava/io/PrintStream;
      12: aload_1
      13: iload_2
      14: invokedynamic #21,  0             // InvokeDynamic #0:makeConcatWithConstants:
(Ljava/lang/String;I)Ljava/lang/String;
      19: invokevirtual #25                 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
      22: return
}
```

Instruction	Meaning
`iload_0`, `iload_1`	Push int from local slot 0 / 1 (the parameters a, b) onto the operand stack
`iadd`	Pop two ints, push their sum
`istore_2` / `astore_1`	Pop and store into local slot 2 (sum) / reference into slot 1 (msg)
`ireturn`	Return the int on top of the stack
`iconst_2`, `ldc #7`	Push a small constant / a constant from the pool ("Total: ")
`invokestatic`, `invokevirtual`, `invokespecial`	Call a static method / an instance method (dynamic dispatch) / a constructor or private method
`invokedynamic`	Call site linked at run time: used for string concatenation (Java 9+) and lambdas
`getstatic`	Read a static field (System.out)
`aload_0`	Push the reference in slot 0 (`this` in instance methods)



operand stack after each instruction of add(2, 3); istore_2 saves 5 into local "sum"

THE JVM IS A STACK MACHINE

PRACTICE CHECKPOINT

Cause and diagnose an **OutOfMemoryError** and a **deadlock** using a heap dump and a thread dump.

Part 1: OutOfMemoryError. Run `LeakyCache` from 19.6.1 with a small heap and automatic heap dump:

Step 1: reproduce the OOM and capture a heap dump

```
$ javac LeakyCache.java
$ java -Xmx64m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=leak.hprof LeakyCache
handled 20 requests
java.lang.OutOfMemoryError: Java heap space
Dumping heap to leak.hprof ...
Heap dump file created [36118312 bytes in 0.014 secs]
```

1. Open `leak.hprof` in **Eclipse MAT** (File → Open Heap Dump) or VisualVM.
2. Run the **Leak Suspects** report: about 60 MB is held by one `java.util.ArrayList`.
3. Look at the **path to GC roots**: the ArrayList is referenced by the static field `LeakyCache.CACHE` (a class is a GC root while loaded).
4. The **dominator tree** shows ~60 `byte[1048576]` arrays retained by that list.
5. Fix: bound the cache (remove old entries, use an LRU map or Caffeine), or do not cache at all. Re-run to confirm memory stays flat.

Part 2: deadlock. Run a program where two transfer threads lock two accounts in opposite order, then take a thread dump while it hangs:

BANKDEADLOCK.JAVA (HANGS FOREVER) BankDeadlock.java

```
public class BankDeadlock {
    static final Object accountA = new Object(), accountB = new Object();
    public static void main(String[] args) {
        new Thread(() -> { synchronized (accountA) { sleep(); synchronized (accountB) { } } }, "transfer-A-to-
B").start();
        new Thread(() -> { synchronized (accountB) { sleep(); synchronized (accountA) { } } }, "transfer-B-to-
A").start();
    }
    static void sleep() { try { Thread.sleep(200); } catch (InterruptedException e) { } }
}
```

Step 2: the thread dump names the threads, the locks and the exact lines (real output)

```
$ java BankDeadlock &
$ jps -l
24520 BankDeadlock
$ jstack 24520
...
Found one Java-level deadlock:
=============================
"transfer-A-to-B":
  waiting to lock monitor 0x00007fa824060510 (object 0x00000000a8517950, a java.lang.Object),
  which is held by "transfer-B-to-A"

"transfer-B-to-A":
  waiting to lock monitor 0x00007fa8200015b0 (object 0x00000000a8517940, a java.lang.Object),
  which is held by "transfer-A-to-B"

Java stack information for the threads listed above:
===================================================
"transfer-A-to-B":
        at BankDeadlock.lambda$main$0(BankDeadlock.java:4)
        - waiting to lock <0x00000000a8517950> (a java.lang.Object)
        - locked <0x00000000a8517940> (a java.lang.Object)
"transfer-B-to-A":
        at BankDeadlock.lambda$main$1(BankDeadlock.java:5)
        - waiting to lock <0x00000000a8517940> (a java.lang.Object)
        - locked <0x00000000a8517950> (a java.lang.Object)

Found 1 deadlock.
```

Diagnosis: each thread holds one lock (`- locked`) and waits for the other (`- waiting to lock`), at lines 4 and 5. Fix: acquire the two locks in a **fixed global order** (for example, always the account with the smaller ID first), or use `tryLock` with a timeout (16.8.2.1).

MODULE 20 · ADVANCED

JDBC (Java Database Connectivity)

Minimum duration **5 days** | Level **Advanced**

JDBC is the standard API for connecting Java programs to relational databases such as MySQL and PostgreSQL. SQL basics should be covered before or alongside this module.

IN THIS MODULE

20.1 Introduction
20.2 Setup
20.3 Steps to connect
20.4 Statement types
20.5 Executing SQL
20.6 ResultSet
20.7 Transactions
20.8 Batch processing
20.9 Connection pooling
20.10 Project structure
20.11 Beyond JDBC (overview)

NOTE

The programs in this module need a running **MySQL** server and the MySQL driver JAR (20.2). Every program was compiled with JDK 21; the output boxes show what each program prints against the sample `college` database created in 20.2.1, run in the order given.

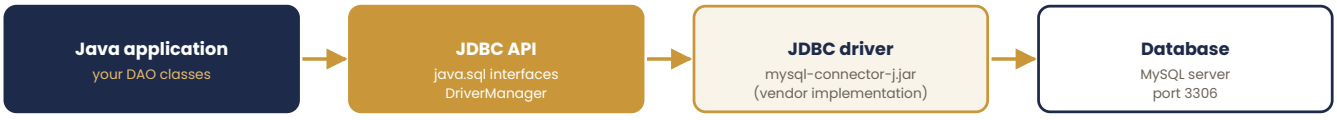
20.1 Introduction

20.1.1 What JDBC is and why it is needed

Real applications keep their data in a **database** (students, orders, bank accounts) so that it survives restarts, can be queried efficiently and is shared by many users. **JDBC** (Java Database Connectivity, package `java.sql`) is the standard Java API for talking to relational databases: connecting, sending SQL, reading results and managing transactions.

Its key idea is **database independence**: your code uses the same JDBC interfaces (`Connection`, `PreparedStatement`, `ResultSet`) for MySQL, PostgreSQL, Oracle or SQL Server. Only the **driver** JAR and the connection URL change. Every higher-level technology (Hibernate, JPA, Spring Data) uses JDBC underneath.

20.1.2 JDBC architecture (application → JDBC API → driver → database)



JDBC ARCHITECTURE: THE DRIVER TRANSLATES JDBC CALLS INTO THE DATABASE'S OWN NETWORK PROTOCOL

Interface / class	Role
`DriverManager`	Finds a suitable driver and opens connections
`Connection`	A session with the database; creates statements; controls transactions
`Statement`, `PreparedStatement`, `CallableStatement`	Send SQL / parameterised SQL / stored-procedure calls
`ResultSet`	A table of rows returned by a query, read with a cursor
`SQLException`	Checked exception for every database error
`DataSource`	Factory for connections, used with pools (20.9)

20.1.3 Types of JDBC drivers (Type 1 to 4)

Type	Name	How it works	Status
Type 1	JDBC-ODBC bridge	Translates JDBC calls to ODBC, which needs a native ODBC driver	Removed in Java 8
Type 2	Native-API driver	Calls the database's native C client library	Rare; platform-specific
Type 3	Network-protocol driver	Sends calls to a middleware server, which talks to the database	Rare
Type 4	Thin / pure Java driver	Pure Java; speaks the database's wire protocol directly	**Standard today**: MySQL Connector/J, PostgreSQL JDBC

20.2 Setup

20.2.1 Installing MySQL (or PostgreSQL)

Install **MySQL Community Server 8.x**: on Windows use the MySQL Installer (choose "Server only" or "Developer Default", set a root password, keep port 3306); on macOS use the DMG installer or `brew install mysql` then `brew services start mysql`; on Ubuntu `sudo apt install mysql-server`. Verify with `mysql -u root -p`. (PostgreSQL works the same way with its own installer and port 5432.) Then create the database used in this module:

COLLEGE.SQL: THE SAMPLE DATABASE

sql

```sql
CREATE DATABASE college;
USE college;

CREATE TABLE students (
    id      INT PRIMARY KEY AUTO_INCREMENT,
    name    VARCHAR(50)  NOT NULL,
    email   VARCHAR(80)  UNIQUE,
    dept    VARCHAR(10),
    marks   DECIMAL(5,2),
    joined  DATE
);

INSERT INTO students (name, email, dept, marks, joined) VALUES
 ('Asha',  'asha@lbf.in',  'CSE', 91.50, '2025-07-15'),
 ('Ravi',  'ravi@lbf.in',  'ECE', 78.00, '2025-07-16'),
 ('Meera', 'meera@lbf.in', 'CSE', 85.00, '2025-07-15'),
 ('Kiran', 'kiran@lbf.in', 'MECH',62.50, '2025-07-20');

CREATE TABLE accounts (id INT PRIMARY KEY, holder VARCHAR(50), balance DECIMAL(10,2));
INSERT INTO accounts VALUES (1, 'Asha', 5000.00), (2, 'Ravi', 2000.00);
```

20.2.2 Adding the driver JAR (manually or with Maven)

Manually: download *MySQL Connector/J* (Platform Independent ZIP) from dev.mysql.com, extract `mysql-connector-j-9.x.x.jar`, and add it to the project: in IntelliJ, File → Project Structure → Libraries → + → Java; in Eclipse, right-click project → Build Path → Add External Archives; on the command line, put it on the classpath. Forgetting this step gives `No suitable driver found for jdbc:mysql://…` or `ClassNotFoundException: com.mysql.cj.jdbc.Driver`.

Compiling and running with the driver on the classpath

```
# Windows (classpath separator ;)
javac -cp ".;lib/mysql-connector-j-9.1.0.jar" StudentApp.java
java  -cp ".;lib/mysql-connector-j-9.1.0.jar" StudentApp

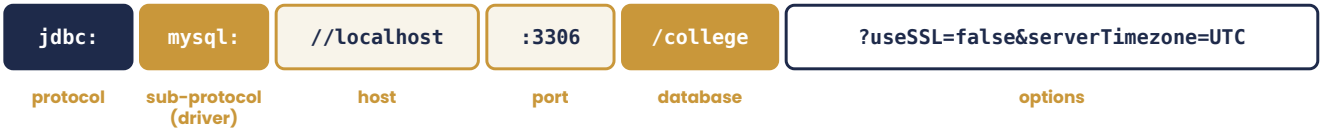
# macOS / Linux (separator :)
javac -cp ".:lib/mysql-connector-j-9.1.0.jar" StudentApp.java
java  -cp ".:lib/mysql-connector-j-9.1.0.jar" StudentApp
```

With Maven (Module 22), add one dependency to `pom.xml`; Maven downloads it automatically:

POM.XML DEPENDENCY xml

```xml
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.1.0</version>
</dependency>
```

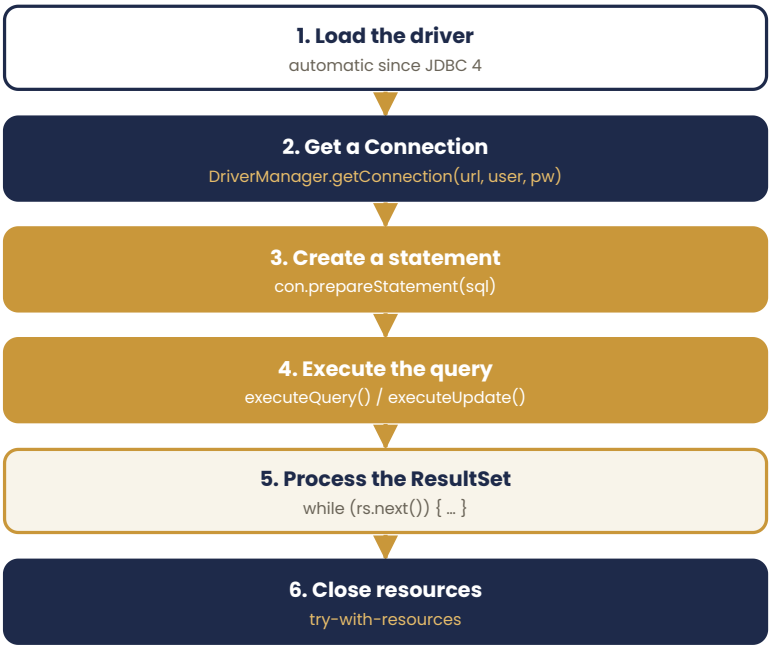
20.2.3 Connection URL format



PARTS OF A JDBC URL

Database	URL example	Driver class
MySQL	`jdbc:mysql://localhost:3306/college`	`com.mysql.cj.jdbc.Driver`
PostgreSQL	`jdbc:postgresql://localhost:5432/college`	`org.postgresql.Driver`
Oracle	`jdbc:oracle:thin:@localhost:1521:XE`	`oracle.jdbc.OracleDriver`
SQL Server	`jdbc:sqlserver://localhost:1433;databaseName=college`	`com.microsoft.sqlserver.jdbc.SQLServerDriver`
H2 (in-memory, tests)	`jdbc:h2:mem:testdb`	`org.h2.Driver`

20.3 Steps to connect



THE SIX STEPS OF EVERY JDBC PROGRAM

20.3.1 Load the driver (optional since JDBC 4)

Old code called `Class.forName("com.mysql.cj.jdbc.Driver")` to load and register the driver. Since JDBC 4.0 (Java 6), drivers register themselves automatically through the **ServiceLoader** mechanism (a file `META-INF/services/java.sql.Driver` inside the JAR), so this line is no longer needed, though harmless. If it throws ClassNotFoundException, the JAR is not on the classpath.

20.3.2 Get a Connection with DriverManager

20.3.3 Create a statement

20.3.4 Execute the query

20.3.5 Process the ResultSet

20.3.6 Close resources (try-with-resources)

Connections, statements and result sets hold database and network resources and **must be closed**, even when an exception occurs. All three implement AutoCloseable, so declare them in try-with-resources; they close automatically in reverse order.

A COMPLETE FIRST JDBC PROGRAM FirstJdbc.java

```
import java.sql.*;

public class FirstJdbc {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASSWORD = "your_password";

    public static void main(String[] args) {
        String sql = "SELECT id, name, dept, marks FROM students ORDER BY marks DESC";
        try (Connection con = DriverManager.getConnection(URL, USER, PASSWORD);   // step 2
             Statement st = con.createStatement();                                // step 3
             ResultSet rs = st.executeQuery(sql)) {                               // step 4
            System.out.println("Connected to " + con.getMetaData().getDatabaseProductName());
            while (rs.next()) {                                                   // step 5
                System.out.printf("%-3d %-6s %-5s %6.2f%n",
                        rs.getInt("id"), rs.getString("name"), rs.getString("dept"), rs.getDouble("marks"));
            }
        } catch (SQLException e) {                                               // every JDBC call can throw
            System.out.println("Database error: " + e.getMessage() + " (SQLState " + e.getSQLState() + ")");
        }                                                                        // step 6: all closed here
    }
}
```

OUTPUT

```
Connected to MySQL
1   Asha   CSE    91.50
3   Meera  CSE    85.00
2   Ravi   ECE    78.00
4   Kiran  MECH   62.50
```

Error message	Cause	Fix
`No suitable driver found`	Driver JAR missing or URL typo	Add the JAR; check `jdbc:mysql://`
`Communications link failure`	MySQL not running / wrong host or port / firewall	Start MySQL; check port 3306
`Access denied for user 'root'@'localhost'`	Wrong user or password	Check credentials
`Unknown database 'college'`	Database not created	Run college.sql
`Table 'college.student' doesn't exist`	Wrong table name	Check spelling (students)

20.4 Statement types

20.4.1 Statement

`Statement` sends a **fixed SQL string**. Fine for constant SQL (DDL, queries without user input). Building SQL by concatenating user input into a Statement is dangerous (20.4.3) and also slow, because the database re-parses every

distinct string.

20.4.2 PreparedStatement and parameters ?

A `PreparedStatement` is SQL with **placeholders** `?`. It is sent to the database once and compiled; you then set the values with typed setters (`setInt(1, …)` , `setString(2, …)` , indexes start at **1**) and execute it, as many times as you like. It is **safer** (no SQL injection), **faster** for repeated execution, and handles quoting, dates and nulls for you. **Always use PreparedStatement when SQL contains values.**

PARAMETERISED QUERY, EXECUTED TWICE

PreparedDemo.java

```
import java.sql.*;

public class PreparedDemo {
    public static void main(String[] args) throws SQLException {
        String sql = "SELECT name, marks FROM students WHERE dept = ? AND marks >= ? ORDER BY marks DESC";
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password");
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, "CSE");                 // first ?
            ps.setDouble(2, 80);                    // second ?
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) System.out.println(rs.getString("name") + " " + rs.getDouble("marks"));
            }
            ps.setString(1, "ECE");                 // reuse the same compiled statement
            ps.setDouble(2, 50);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) System.out.println(rs.getString("name") + " " + rs.getDouble("marks"));
            }
        }
    }
}
```

OUTPUT

```
Asha 91.5
Meera 85.0
Ravi 78.0
```

20.4.3 SQL injection and why PreparedStatement prevents it

SQL injection happens when user input is pasted into SQL text. An attacker types SQL syntax instead of data, and changes the meaning of the query: logging in without a password, reading other users' data or deleting tables. It is one of the most exploited vulnerabilities on the web (OWASP Top 10).

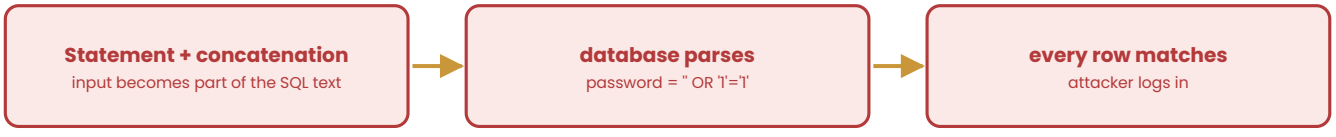
HOW CONCATENATION LETS INPUT BECOME SQL

InjectionDemo.java

```
public class InjectionDemo {
    public static void main(String[] args) {
        String user = "admin";
        String password = "' OR '1'='1";                        // typed into the login form by an attacker
        String sql = "SELECT * FROM users WHERE username = '" + user + "' AND password = '" + password + "'";
        System.out.println(sql);
        // WHERE username = 'admin' AND password = '' OR '1'='1'   -> '1'='1' is always true: logged in!
    }
}
```

OUTPUT

```
SELECT * FROM users WHERE username = 'admin' AND password = '' OR '1'='1'
```



WITH CONCATENATION, DATA IS INTERPRETED AS CODE

With a PreparedStatement the SQL is compiled **before** any value is supplied, so a parameter can only ever be a **value**, never SQL syntax. The attacker's input is simply compared as a strange password, and no row matches.

THE SAFE VERSION — java

```java
PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE username = ? AND password = ?");
ps.setString(1, user);
ps.setString(2, password);        // "' OR '1'='1" is just text: no match, login fails
```

WATCH OUT

Placeholders work only for **values**, not table names, column names or ORDER BY directions. If those must be dynamic, choose them from a fixed allow-list in code, never directly from user input. Also store password **hashes** (bcrypt/Argon2), never plain passwords.

20.4.4 CallableStatement (stored procedures)

A **stored procedure** is a named block of SQL stored in the database. `CallableStatement` calls it with the escape syntax `{call name(?, ?)}`, supporting IN parameters, OUT parameters (registered with `registerOutParameter`) and result sets.

A MYSQL STORED PROCEDURE — sql

```sql
DELIMITER //
CREATE PROCEDURE dept_stats(IN p_dept VARCHAR(10), OUT p_count INT, OUT p_avg DECIMAL(5,2))
BEGIN
    SELECT COUNT(*), AVG(marks) INTO p_count, p_avg FROM students WHERE dept = p_dept;
END //
DELIMITER ;
```

CALLING THE PROCEDURE — CallableDemo.java

```java
import java.sql.*;

public class CallableDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password");
             CallableStatement cs = con.prepareCall("{call dept_stats(?, ?, ?)}")) {
            cs.setString(1, "CSE");                          // IN
            cs.registerOutParameter(2, Types.INTEGER);       // OUT
            cs.registerOutParameter(3, Types.DECIMAL);       // OUT
            cs.execute();
            System.out.println("CSE students: " + cs.getInt(2) + ", average: " + cs.getBigDecimal(3));
        }
    }
}
```

OUTPUT

```
CSE students: 2, average: 88.25
```

20.5 Executing SQL

20.5.1 executeQuery() for SELECT

20.5.2 executeUpdate() for INSERT, UPDATE, DELETE

20.5.3 execute()

Method	Use for	Returns
`executeQuery()`	SELECT	a `ResultSet`
`executeUpdate()`	INSERT, UPDATE, DELETE, and DDL (CREATE/ALTER/DROP)	`int`: number of rows affected (0 for DDL)

Method	Use for	Returns
`execute()`	SQL whose type is unknown in advance, or that returns several results	`boolean`: true if the first result is a ResultSet; then use `getResultSet()` / `getUpdateCount()`
`executeLargeUpdate()`	Updates affecting more than 2^{31} rows	`long`

20.5.4 CRUD operations end to end

CRUD = **C**reate (INSERT), **R**ead (SELECT), **U**pdate, **D**elete. The program below performs all four. `RETURN_GENERATED_KEYS` retrieves the AUTO_INCREMENT id assigned by MySQL.

CREATE, READ, UPDATE AND DELETE CrudDemo.java

```
import java.sql.*;

public class CrudDemo {
    static final String URL = "jdbc:mysql://localhost:3306/college", USER = "root", PASS = "your_password";

    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            // CREATE
            int newId;
            try (PreparedStatement ps = con.prepareStatement(
                    "INSERT INTO students (name, email, dept, marks, joined) VALUES (?, ?, ?, ?, ?)",
                    Statement.RETURN_GENERATED_KEYS)) {
                ps.setString(1, "Dev");
                ps.setString(2, "dev@lbf.in");
                ps.setString(3, "ECE");
                ps.setBigDecimal(4, new java.math.BigDecimal("70.00"));
                ps.setDate(5, Date.valueOf("2025-08-01"));
                System.out.println("inserted rows: " + ps.executeUpdate());
                try (ResultSet keys = ps.getGeneratedKeys()) { keys.next(); newId = keys.getInt(1); }
                System.out.println("new id: " + newId);
            }
            // READ
            try (PreparedStatement ps = con.prepareStatement("SELECT name, marks FROM students WHERE id = ?")) {
                ps.setInt(1, newId);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) System.out.println("read: " + rs.getString("name") + " " + rs.getDouble("marks"));
                }
            }
            // UPDATE
            try (PreparedStatement ps = con.prepareStatement("UPDATE students SET marks = marks + ? WHERE dept = ?")) {
                ps.setInt(1, 2);
                ps.setString(2, "ECE");
                System.out.println("updated rows: " + ps.executeUpdate());
            }
            // DELETE
            try (PreparedStatement ps = con.prepareStatement("DELETE FROM students WHERE id = ?")) {
                ps.setInt(1, newId);
                System.out.println("deleted rows: " + ps.executeUpdate());
            }
            // execute() when the kind of SQL is not known
            try (Statement st = con.createStatement()) {
                boolean isQuery = st.execute("SELECT COUNT(*) FROM students");
                if (isQuery) try (ResultSet rs = st.getResultSet()) { rs.next(); System.out.println("execute(): query,
count = " + rs.getInt(1)); }
            }
        }
    }
}
```

OUTPUT

```
inserted rows: 1
new id: 5
read: Dev 70.0
updated rows: 2
deleted rows: 1
execute(): query, count = 4
```

20.6 ResultSet

20.6.1 Navigating with next()

A ResultSet has a **cursor** that starts **before the first row**. Each `next()` moves it one row forward and returns **false** when there are no more rows, which is why the loop is `while (rs.next())`. For a single expected row use `if (rs.next())`.



THE RESULTSET CURSOR

20.6.2 Getting values: getInt, getString, getDate

SQL type	Getter	Java type
INT	`getInt`	int
BIGINT	`getLong`	long
VARCHAR, CHAR, TEXT	`getString`	String
DECIMAL	`getBigDecimal` (exact) / `getDouble`	BigDecimal / double
DATE	`getDate` → java.sql.Date, or `getObject(col, LocalDate.class)`	LocalDate
DATETIME, TIMESTAMP	`getTimestamp` or `getObject(col, LocalDateTime.class)`	LocalDateTime
BOOLEAN / TINYINT(1)	`getBoolean`	boolean

Getters accept a column **label** (`"name"`, clearer) or a 1-based column **index**. For nullable numeric columns, `getInt` returns 0 for SQL NULL; call `rs.wasNull()` afterwards, or use `getObject(col, Integer.class)` which returns null.

TYPED GETTERS ResultSetGetters.java

```
import java.sql.*;
import java.time.LocalDate;

public class ResultSetGetters {
    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password");
             PreparedStatement ps = con.prepareStatement("SELECT id, name, marks, joined FROM students WHERE id = ?"))
{
            ps.setInt(1, 1);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    int id = rs.getInt("id");
                    String name = rs.getString(2);                              // by column index
                    java.math.BigDecimal marks = rs.getBigDecimal("marks");
                    Date sqlDate = rs.getDate("joined");
                    LocalDate joined = rs.getObject("joined", LocalDate.class);  // modern java.time
                    System.out.println(id + " " + name + " " + marks + " " + sqlDate + " " + joined.getDayOfWeek());
                }
            }
        }
    }
}
```

OUTPUT

```
1 Asha 91.50 2025-07-15 TUESDAY
```

20.6.3 Scrollable and updatable ResultSets

By default a ResultSet is `TYPE_FORWARD_ONLY` and `CONCUR_READ_ONLY`. Passing other constants to `createStatement` / `prepareStatement` gives a **scrollable** cursor (`previous()`, `first()`, `last()`, `absolute(n)`, `relative(n)`) and an **updatable** one (change values in place with `updateXxx()` + `updateRow()`). They use more resources; in practice, explicit SQL UPDATEs and LIMIT/OFFSET paging are more common.

MOVING FREELY AND UPDATING THROUGH THE CURSOR ScrollableDemo.java

```
import java.sql.*;

public class ScrollableDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password");
             Statement st = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
             ResultSet rs = st.executeQuery("SELECT id, name, marks FROM students ORDER BY id")) {
            rs.last();
            System.out.println("rows: " + rs.getRow() + ", last: " + rs.getString("name"));
            rs.first();
            System.out.println("first: " + rs.getString("name"));
            rs.absolute(3);
            System.out.println("row 3: " + rs.getString("name"));
            rs.previous();
            System.out.println("previous: " + rs.getString("name"));
            rs.updateDouble("marks", 80.00);            // update the current row (Ravi) in place
            rs.updateRow();
            System.out.println("Ravi's marks updated through the ResultSet");
        }
    }
}
```

OUTPUT

```
rows: 4, last: Kiran
first: Asha
row 3: Meera
previous: Ravi
Ravi's marks updated through the ResultSet
```

20.6.4 ResultSetMetaData and DatabaseMetaData

`ResultSetMetaData` (`rs.getMetaData()`) describes the **columns** of a result: count, names, SQL types, sizes. It lets you write generic code, such as printing any query as a table. `DatabaseMetaData` (`con.getMetaData()`) describes the **database**: product and driver versions, tables, keys, supported features.

INSPECTING THE DATABASE AND A RESULT MetaDataDemo.java

```
import java.sql.*;

public class MetaDataDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password")) {
            DatabaseMetaData db = con.getMetaData();
            System.out.println(db.getDatabaseProductName() + " " + db.getDatabaseMajorVersion() + ", driver " +
db.getDriverName());
            try (ResultSet tables = db.getTables("college", null, "%", new String[] {"TABLE"})) {
                System.out.print("tables:");
                while (tables.next()) System.out.print(" " + tables.getString("TABLE_NAME"));
                System.out.println();
            }
            try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery("SELECT * FROM students")) {
                ResultSetMetaData md = rs.getMetaData();
                for (int i = 1; i <= md.getColumnCount(); i++)
                    System.out.printf("  column %d: %-7s %-8s%n", i, md.getColumnName(i), md.getColumnTypeName(i));
            }
        }
    }
}
```

OUTPUT

```
MySQL 8, driver MySQL Connector/J
tables: accounts students
  column 1: id      INT
  column 2: name    VARCHAR
  column 3: email   VARCHAR
  column 4: dept    VARCHAR
  column 5: marks   DECIMAL
  column 6: joined  DATE
```

20.7 Transactions

A **transaction** is a group of SQL statements that must succeed or fail **together**. Classic example: transferring ₹1000 from Asha to Ravi is two UPDATEs; if the program crashes after the first, money disappears. Inside a transaction, either both happen (**commit**) or neither does (**rollback**).

20.7.1 Auto-commit

By default a JDBC connection is in **auto-commit** mode: every statement is its own transaction, committed immediately. To group statements, call `con.setAutoCommit(false)`, then end the transaction explicitly.

20.7.2 commit() and rollback()

A MONEY TRANSFER AS ONE TRANSACTION

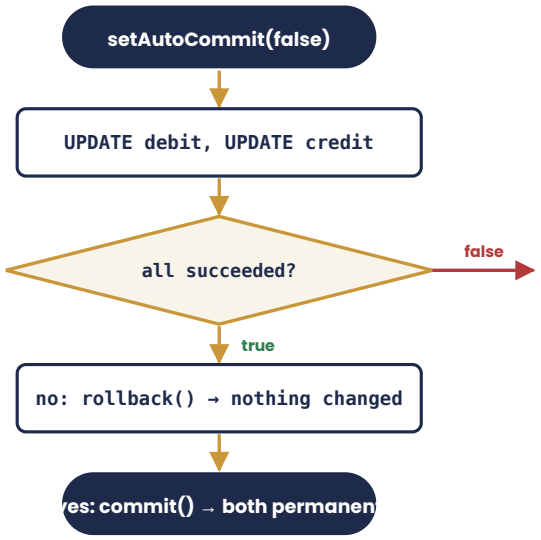
TransferDemo.java

```
import java.math.BigDecimal;
import java.sql.*;

public class TransferDemo {
    static void transfer(Connection con, int from, int to, BigDecimal amount) throws SQLException {
        boolean oldAuto = con.getAutoCommit();
        con.setAutoCommit(false);                                   // start transaction
        try (PreparedStatement debit = con.prepareStatement("UPDATE accounts SET balance = balance - ? WHERE id = ? AND
balance >= ?");
             PreparedStatement credit = con.prepareStatement("UPDATE accounts SET balance = balance + ? WHERE id = ?"))
{
            debit.setBigDecimal(1, amount); debit.setInt(2, from); debit.setBigDecimal(3, amount);
            if (debit.executeUpdate() != 1) throw new SQLException("Insufficient balance in account " + from);
            credit.setBigDecimal(1, amount); credit.setInt(2, to);
            if (credit.executeUpdate() != 1) throw new SQLException("Account " + to + " not found");
            con.commit();                                           // both succeeded: make permanent
            System.out.println("transfer of " + amount + " committed");
        } catch (SQLException e) {
            con.rollback();                                         // undo everything since setAutoCommit(false)
            System.out.println("rolled back: " + e.getMessage());
        } finally {
            con.setAutoCommit(oldAuto);
        }
    }
    static void show(Connection con) throws SQLException {
        try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery("SELECT holder, balance FROM accounts
ORDER BY id")) {
            while (rs.next()) System.out.print(rs.getString(1) + "=" + rs.getBigDecimal(2) + "  ");
            System.out.println();
        }
    }
    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password")) {
            show(con);
            transfer(con, 1, 2, new BigDecimal("1000.00"));     // OK
            transfer(con, 2, 1, new BigDecimal("9999.00"));     // fails: insufficient balance
            transfer(con, 1, 99, new BigDecimal("500.00"));     // debit succeeds, credit fails -> rollback
            show(con);
        }
    }
}
```

OUTPUT

```
Asha=5000.00  Ravi=2000.00
transfer of 1000.00 committed
rolled back: Insufficient balance in account 2
rolled back: Account 99 not found
Asha=4000.00  Ravi=3000.00
```



COMMIT OR ROLL BACK AS A UNIT

20.7.3 Savepoints

A **savepoint** marks a point inside a transaction. `rollback(savepoint)` undoes only the work after it, keeping earlier statements, and the transaction continues.

PARTIAL ROLLBACK WITH A SAVEPOINT — SavepointDemo.java

```
import java.sql.*;

public class SavepointDemo {
    public static void main(String[] args) throws SQLException {
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root",
"your_password");
             Statement st = con.createStatement()) {
            con.setAutoCommit(false);
            st.executeUpdate("INSERT INTO students (name, dept, marks) VALUES ('Om', 'CSE', 74)");
            Savepoint afterOm = con.setSavepoint("afterOm");
            st.executeUpdate("INSERT INTO students (name, dept, marks) VALUES ('Test', 'XXX', 0)");   // a mistake
            con.rollback(afterOm);                       // undo only the second insert
            con.commit();                                // Om is saved
            try (ResultSet rs = st.executeQuery("SELECT GROUP_CONCAT(name ORDER BY id) FROM students")) {
                rs.next(); System.out.println("students now: " + rs.getString(1));
            }
        }
    }
}
```

OUTPUT

```
students now: Asha,Ravi,Meera,Kiran,Om
```

20.7.4 ACID properties and isolation levels

Atomicity
All statements of a transaction happen, or none do.

Consistency
A transaction takes the database from one valid state to another (constraints hold).

Isolation
Concurrent transactions do not see each other's unfinished work (to a chosen degree).

Durability
Once committed, changes survive crashes and power loss.

Full isolation is expensive, so databases offer **isolation levels**. Lower levels allow more anomalies but more concurrency. Set with `con.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED)`.

Level	Dirty read	Non-repeatable read	Phantom read
`READ_UNCOMMITTED`	possible	possible	possible

Level	Dirty read	Non-repeatable read	Phantom read
`READ_COMMITTED` (PostgreSQL, Oracle default)	prevented	possible	possible
`REPEATABLE_READ` (MySQL InnoDB default)	prevented	prevented	possible (mostly prevented in InnoDB)
`SERIALIZABLE`	prevented	prevented	prevented

Anomaly	Meaning
Dirty read	Reading another transaction's uncommitted change, which may be rolled back
Non-repeatable read	Reading the same row twice gives different values because another transaction committed an update in between
Phantom read	Re-running the same query returns new rows inserted by another transaction

20.8 Batch processing

20.8.1 addBatch() and executeBatch()

Inserting 10,000 rows with 10,000 separate executeUpdate calls means 10,000 network round trips. **Batching** collects many parameter sets with `addBatch()` and sends them together with `executeBatch()`, which returns an array of update counts. Combined with a transaction it is dramatically faster. For MySQL add `rewriteBatchedStatements=true` to the URL so the driver merges them into multi-row INSERTs.

INSERTING THOUSANDS OF ROWS EFFICIENTLY — BatchInsert.java

```
import java.sql.*;

public class BatchInsert {
    public static void main(String[] args) throws SQLException {
        String url = "jdbc:mysql://localhost:3306/college?rewriteBatchedStatements=true";
        try (Connection con = DriverManager.getConnection(url, "root", "your_password");
             PreparedStatement ps = con.prepareStatement("INSERT INTO students (name, dept, marks) VALUES (?, ?, ?)"))
{
            con.setAutoCommit(false);
            long start = System.currentTimeMillis();
            for (int i = 1; i <= 5000; i++) {
                ps.setString(1, "Student" + i);
                ps.setString(2, i % 2 == 0 ? "CSE" : "ECE");
                ps.setDouble(3, 40 + i % 60);
                ps.addBatch();                              // queue it
                if (i % 1000 == 0) ps.executeBatch();       // send every 1000 rows
            }
            ps.executeBatch();                              // send any remainder
            con.commit();
            System.out.println("5000 rows inserted in one transaction, in batches of 1000");
            System.out.println("took under 2 seconds: " + (System.currentTimeMillis() - start < 2000));
        }
    }
}
```

OUTPUT

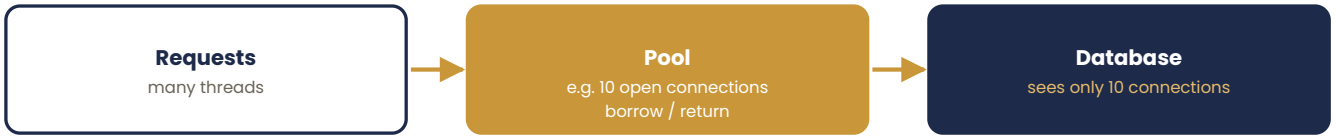
```
5000 rows inserted in one transaction, in batches of 1000
took under 2 seconds: true
```

20.9 Connection pooling

20.9.1 Why pooling

Opening a database connection is **expensive**: a TCP connection, a TLS handshake, authentication, session setup (often 10–100 ms). A web application handling hundreds of requests per second cannot afford a new connection per request, and the database can handle only a limited number of connections. A **connection pool** opens a fixed number of

connections at startup and **lends** them out: `getConnection()` borrows one and `close()` returns it to the pool instead of really closing it.



A POOL REUSES A SMALL SET OF PHYSICAL CONNECTIONS

20.9.2 DataSource and HikariCP

`javax.sql.DataSource` is the standard interface for obtaining connections; application code depends on it instead of DriverManager, and the configuration (URL, pool size) lives elsewhere. **HikariCP** is the fastest and most popular pool and the default in Spring Boot. Add `com.zaxxer:HikariCP` with Maven.

BORROWING CONNECTIONS FROM HIKARICP (NEEDS THE HIKARICP JAR) — PoolDemo.java

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import javax.sql.DataSource;
import java.sql.*;

public class PoolDemo {
    static DataSource createPool() {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl("jdbc:mysql://localhost:3306/college");
        cfg.setUsername("root");
        cfg.setPassword("your_password");
        cfg.setMaximumPoolSize(10);            // at most 10 real connections
        cfg.setMinimumIdle(2);
        cfg.setConnectionTimeout(3000);        // wait at most 3 s for a free connection
        return new HikariDataSource(cfg);
    }
    public static void main(String[] args) throws SQLException {
        DataSource ds = createPool();
        for (int i = 1; i <= 3; i++) {
            try (Connection con = ds.getConnection();                    // borrow
                 Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM students")) {
                rs.next();
                System.out.println("request " + i + ": " + rs.getInt(1) + " students");
            }                                                            // close() = return to pool
        }
        ((HikariDataSource) ds).close();                                 // shut the pool down at exit
    }
}
```

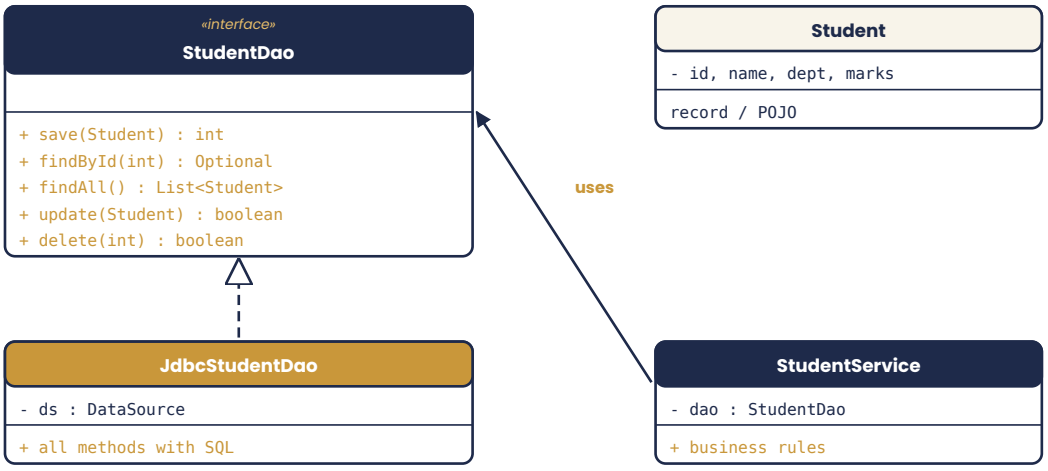
OUTPUT

```
request 1: 5005 students
request 2: 5005 students
request 3: 5005 students
```

20.10 Project structure

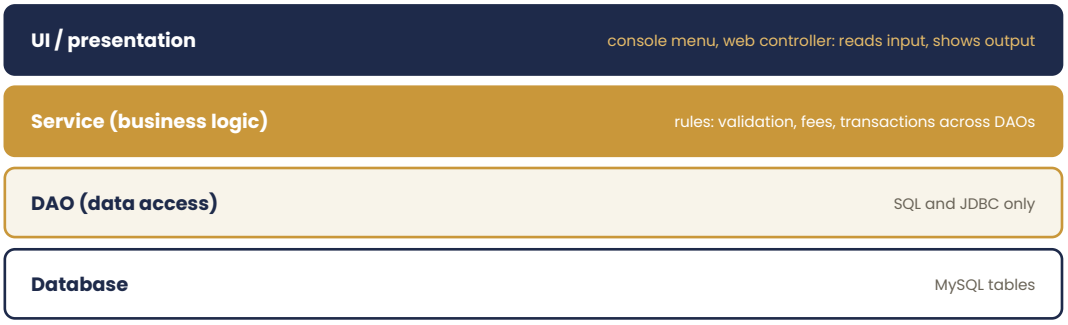
20.10.1 DAO (Data Access Object) pattern

Scattering SQL throughout the program makes it hard to read, test and change. The **DAO pattern** puts all database code for one entity behind an interface: `StudentDao` with `save`, `findById`, `findAll`, `update`, `delete`. The rest of the application calls these methods and never sees SQL or JDBC types. Switching to another database or to JPA later only changes the DAO implementation.



THE SERVICE DEPENDS ON THE DAO INTERFACE, NOT ON JDBC

20.10.2 Layered design: model, DAO, service, UI



EACH LAYER TALKS ONLY TO THE LAYER BELOW IT; THE MODEL (STUDENT) IS SHARED BY ALL

Package	Contains	Knows about
`model`	Student, Book… (records/POJOs)	Nothing else
`dao`	StudentDao interface + JdbcStudentDao	model, JDBC
`service`	StudentService (rules, transactions)	model, dao interfaces
`ui` / `app`	Main, menus, controllers	model, service
`util`	DBConnection / DataSource factory	JDBC, config

20.10.3 Externalizing credentials in properties files

Never hard-code database passwords in source code: they end up in Git history and in every copy of the JAR. Put connection settings in a `db.properties` file (excluded from Git with `.gitignore`), or better in **environment variables** or a secrets manager in production.

SRC/MAIN/RESOURCES/DB.PROPERTIES

text

```
db.url=jdbc:mysql://localhost:3306/college
db.user=root
db.password=your_password
db.poolSize=10
```

A CONNECTION HELPER THAT READS ITS SETTINGS FROM OUTSIDE THE CODE

DBConnection.java

```
import java.io.IOException;
import java.io.InputStream;
import java.sql.*;
import java.util.Properties;

public final class DBConnection {
    private static final Properties PROPS = new Properties();

    static {                                                    // load once when the class is first used
        try (InputStream in = DBConnection.class.getResourceAsStream("/db.properties")) {
            if (in == null) throw new IllegalStateException("db.properties not found on classpath");
            PROPS.load(in);
        } catch (IOException e) {
            throw new ExceptionInInitializerError(e);
        }
    }
    private DBConnection() { }

    public static Connection get() throws SQLException {
        String password = System.getenv().getOrDefault("DB_PASSWORD", PROPS.getProperty("db.password"));  // env wins
        return DriverManager.getConnection(PROPS.getProperty("db.url"), PROPS.getProperty("db.user"), password);
    }
}
```

20.11 Beyond JDBC (overview)

20.11.1 ORM concept

Plain JDBC means writing SQL and copying every column into object fields by hand. **Object-Relational Mapping** (ORM) automates this mapping between **classes and tables**, **objects and rows**, **fields and columns**, and **references and foreign keys**. You work with objects; the ORM generates the SQL.

Object world	Relational world
Class `Student`	Table `students`
Object (instance)	Row
Field `name`	Column `name`
Reference `student.getDept()`	Foreign key `dept_id` + JOIN
`List<Course>` in Student	Many-to-many link table

20.11.2 JPA and Hibernate

JPA (Jakarta Persistence API) is the standard Java *specification* for ORM: annotations and interfaces. **Hibernate** is its most popular *implementation*. You annotate an entity class, and an `EntityManager` (or Spring Data JPA repositories, 22.7.4) saves and loads objects. JDBC remains underneath; knowing it helps you understand and debug what the ORM does.

A JPA ENTITY AND ENTITYMANAGER CALLS java

```java
@Entity
@Table(name = "students")
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String name;
    private String dept;
    private BigDecimal marks;
    // constructors, getters, setters
}

// using it: no SQL written by hand
Student s = entityManager.find(Student.class, 1);       // SELECT ... WHERE id = 1
s.setMarks(new BigDecimal("93.00"));                     // UPDATE generated at commit
entityManager.persist(new Student("Om", "CSE"));         // INSERT generated
```

	JDBC	JPA / Hibernate
SQL	Written by you	Generated (JPQL for queries)
Mapping rows → objects	Manual (rs.getString…)	Automatic
Control / performance tuning	Full	Less direct (watch for N+1 queries)
Boilerplate	High	Low
Learning curve	Low	Higher
Best for	Simple apps, bulk/reporting SQL, learning	Domain-rich applications

PRACTICE CHECKPOINT

A **Java + MySQL Library Management** project with full CRUD through DAO classes, PreparedStatements and one transaction (issue a book and update stock together).

LIBRARY.SQL sql

```sql
CREATE DATABASE library;
USE library;
CREATE TABLE books (
    id     INT PRIMARY KEY AUTO_INCREMENT,
    title  VARCHAR(100) NOT NULL,
    author VARCHAR(60)  NOT NULL,
    stock  INT NOT NULL CHECK (stock >= 0)
);
CREATE TABLE issues (
    id         INT PRIMARY KEY AUTO_INCREMENT,
    book_id    INT NOT NULL,
    member     VARCHAR(50) NOT NULL,
    issue_date DATE NOT NULL,
    FOREIGN KEY (book_id) REFERENCES books(id)
);
```

LIBRARYAPP.JAVA: DAO + SERVICE + TRANSACTION LibraryApp.java

```java
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

record Book(int id, String title, String author, int stock) { }

interface BookDao {
    int save(Book b) throws SQLException;
    Optional<Book> findById(int id) throws SQLException;
    List<Book> findAll() throws SQLException;
    boolean updateStock(int id, int stock) throws SQLException;
    boolean delete(int id) throws SQLException;
}
```

```
class JdbcBookDao implements BookDao {
    private final Connection con;
    JdbcBookDao(Connection con) { this.con = con; }

    public int save(Book b) throws SQLException {
        try (PreparedStatement ps = con.prepareStatement(
                "INSERT INTO books (title, author, stock) VALUES (?, ?, ?)", Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, b.title()); ps.setString(2, b.author()); ps.setInt(3, b.stock());
            ps.executeUpdate();
            try (ResultSet k = ps.getGeneratedKeys()) { k.next(); return k.getInt(1); }
        }
    }
    public Optional<Book> findById(int id) throws SQLException {
        try (PreparedStatement ps = con.prepareStatement("SELECT * FROM books WHERE id = ?")) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) { return rs.next() ? Optional.of(map(rs)) : Optional.empty(); }
        }
    }
    public List<Book> findAll() throws SQLException {
        List<Book> list = new ArrayList<>();
        try (PreparedStatement ps = con.prepareStatement("SELECT * FROM books ORDER BY id");
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) list.add(map(rs));
        }
        return list;
    }
    public boolean updateStock(int id, int stock) throws SQLException {
        try (PreparedStatement ps = con.prepareStatement("UPDATE books SET stock = ? WHERE id = ?")) {
            ps.setInt(1, stock); ps.setInt(2, id);
            return ps.executeUpdate() == 1;
        }
    }
    public boolean delete(int id) throws SQLException {
        try (PreparedStatement ps = con.prepareStatement("DELETE FROM books WHERE id = ?")) {
            ps.setInt(1, id);
            return ps.executeUpdate() == 1;
        }
    }
    private Book map(ResultSet rs) throws SQLException {
        return new Book(rs.getInt("id"), rs.getString("title"), rs.getString("author"), rs.getInt("stock"));
    }
}

class LibraryService {
    private final Connection con;
    LibraryService(Connection con) { this.con = con; }

    /** Issue a book: decrease stock AND record the issue, as ONE transaction. */
    boolean issueBook(int bookId, String member) throws SQLException {
        con.setAutoCommit(false);
        try (PreparedStatement dec = con.prepareStatement("UPDATE books SET stock = stock - 1 WHERE id = ? AND stock >
0");
             PreparedStatement ins = con.prepareStatement("INSERT INTO issues (book_id, member, issue_date) VALUES (?,
?, ?)")) {
            dec.setInt(1, bookId);
            if (dec.executeUpdate() == 0) { con.rollback(); System.out.println("  not available: book " + bookId);
return false; }
            ins.setInt(1, bookId); ins.setString(2, member); ins.setObject(3, LocalDate.of(2026, 6, 1));
            ins.executeUpdate();
            con.commit();
            System.out.println("  issued book " + bookId + " to " + member);
            return true;
        } catch (SQLException e) {
            con.rollback();
            throw e;
        } finally {
            con.setAutoCommit(true);
        }
    }
}

public class LibraryApp {
    public static void main(String[] args) throws SQLException {
```

```
        try (Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/library", "root",
"your_password")) {
            BookDao dao = new JdbcBookDao(con);
            LibraryService service = new LibraryService(con);

            int java = dao.save(new Book(0, "Core Java", "Horstmann", 2));          // CREATE
            int clean = dao.save(new Book(0, "Clean Code", "Martin", 1));
            int temp = dao.save(new Book(0, "Old Catalogue", "Unknown", 0));
            System.out.println("Added books with ids " + java + ", " + clean + ", " + temp);

            dao.updateStock(clean, 3);                                              // UPDATE
            System.out.println("Deleted old catalogue: " + dao.delete(temp));       // DELETE

            service.issueBook(java, "Asha");                                        // TRANSACTION
            service.issueBook(java, "Ravi");
            service.issueBook(java, "Meera");                                       // stock is 0 now

            System.out.println("Catalogue:");                                       // READ
            for (Book b : dao.findAll())
                System.out.printf("  %d %-12s %-10s stock %d%n", b.id(), b.title(), b.author(), b.stock());
            System.out.println("Find 99: " + dao.findById(99).map(Book::title).orElse("not found"));
        }
    }
}
```

OUTPUT

```
Added books with ids 1, 2, 3
Deleted old catalogue: true
  issued book 1 to Asha
  issued book 1 to Ravi
  not available: book 1
Catalogue:
  1 Core Java    Horstmann  stock 0
  2 Clean Code   Martin     stock 3
Find 99: not found
```

The issue operation decreases stock and inserts the issue record in **one transaction**: if the INSERT fails (for example, the database goes down), the stock decrement is rolled back, so the catalogue and the issue log can never disagree.

MODULE 21 · ADVANCED

Reflection, annotations, design principles and patterns

Minimum duration **5 days** Level **Advanced**

This module covers the tools frameworks are built on (reflection, annotations) and the design knowledge that separates working code from well-built code.

IN THIS MODULE

21.1 Reflection API
21.2 Annotations
21.3 Object-oriented design principles
21.4 Design patterns
21.5 Clean code practices

21.1 Reflection API

21.1.1 What reflection is and where frameworks use it (Spring, JUnit, Hibernate)

Reflection (`java.lang.reflect`) lets a running program **examine and manipulate its own classes**: list fields and methods, read annotations, create objects and call methods whose names are known only at run time. Normal code is fixed at compile time; reflection makes code that works with **any** class.

Framework	What it does with reflection
JUnit	Finds methods annotated `@Test` and invokes them
Spring	Scans for `@Component` classes, creates them, injects dependencies into constructors/fields
Hibernate / JPA	Reads `@Entity` / `@Column`, creates entity objects and sets their private fields from result sets
Jackson / Gson	Converts objects to JSON by reading their fields or getters
IDEs, debuggers	Show an object's fields and their values

21.1.2 Getting a Class object: .class, getClass(), Class.forName()

Everything starts with a `Class<?>` object, the JVM's description of a type. There are three ways to get one:

THREE WAYS TO OBTAIN A CLASS

GetClassObject.java

```
public class GetClassObject {
    public static void main(String[] args) throws ClassNotFoundException {
        Class<String> a = String.class;                          // 1. class literal (type known at compile time)
        Class<?> b = "hello".getClass();                         // 2. from an object (its runtime type)
        Class<?> c = Class.forName("java.util.ArrayList");       // 3. from a name (known only at run time)
        Class<?> d = int.class;                                  // primitives have Class objects too
        System.out.println(a.getName() + " | " + b.getSimpleName() + " | " + c.getName() + " | " + d);
        System.out.println("same Class object? " + (a == b));
        Object o = new java.util.ArrayList<String>();
        System.out.println("runtime type of o: " + o.getClass().getName() + ", superclass: " +
o.getClass().getSuperclass().getSimpleName());
        System.out.println("interfaces: " +
java.util.Arrays.stream(c.getInterfaces()).map(Class::getSimpleName).toList());
    }
}
```

OUTPUT

```
java.lang.String | String | java.util.ArrayList | int
same Class object? true
runtime type of o: java.util.ArrayList, superclass: AbstractList
interfaces: [List, RandomAccess, Cloneable, Serializable]
```

21.1.3 Inspecting fields, methods and constructors

`getFields()` / `getMethods()` / `getConstructors()` return **public** members including inherited ones; `getDeclaredFields()` / `getDeclaredMethods()` / `getDeclaredConstructors()` return **all** members declared in that class (any access level), but not inherited ones. `Modifier.toString(m.getModifiers())` decodes modifiers.

LISTING A CLASS'S MEMBERS InspectClass.java

```
import java.lang.reflect.*;
import java.util.Arrays;
import java.util.Comparator;

class Student {
    private int id;
    private String name;
    protected double marks;
    public static final String COLLEGE = "LBF";

    public Student() { }
    public Student(int id, String name) { this.id = id; this.name = name; }
    public String getName() { return name; }
    private void secret() { System.out.println("secret method called"); }
    public double bonus(double pct) { return marks * pct / 100; }
}
public class InspectClass {
    public static void main(String[] args) {
        Class<?> c = Student.class;
        System.out.println("class " + c.getSimpleName() + " extends " + c.getSuperclass().getSimpleName());
        System.out.println("-- declared fields --");
        for (Field f : c.getDeclaredFields())
            System.out.println("  " + Modifier.toString(f.getModifiers()) + " " + f.getType().getSimpleName() + " " +
f.getName());
        System.out.println("-- constructors --");
        for (Constructor<?> k : c.getDeclaredConstructors())
            System.out.println("  " + k.getParameterCount() + " params: " +
Arrays.stream(k.getParameterTypes()).map(Class::getSimpleName).toList());
        System.out.println("-- declared methods --");
        Arrays.stream(c.getDeclaredMethods()).sorted(Comparator.comparing(Method::getName))
              .forEach(m -> System.out.println("  " + Modifier.toString(m.getModifiers()) + " " +
m.getReturnType().getSimpleName() + " " + m.getName()
                      + Arrays.stream(m.getParameterTypes()).map(Class::getSimpleName).toList()));
        System.out.println("public methods incl. inherited from Object: " + c.getMethods().length);
    }
}
```

OUTPUT

```
class Student extends Object
-- declared fields --
  private int id
  private String name
  protected double marks
  public static final String COLLEGE
-- constructors --
  0 params: []
  2 params: [int, String]
-- declared methods --
  public double bonus[double]
  public String getName[]
  private void secret[]
public methods incl. inherited from Object: 11
```

21.1.4 Invoking methods and creating objects dynamically

21.1.5 Accessing private members with setAccessible()

`constructor.newInstance(args)` creates objects, `method.invoke(target, args)` calls methods (target is null for static methods), and `field.get/set` reads and writes fields. Private members throw `IllegalAccessException` unless you call `setAccessible(true)`, which disables the access check (allowed for your own classes on the classpath; blocked for JDK internals by the module system unless the package is `opens`-ed).

CREATING, INVOKING AND BREAKING INTO PRIVATE MEMBERS

DynamicInvoke.java

```
import java.lang.reflect.*;

class Account {
    private double balance = 1000;
    private String owner;
    public Account(String owner) { this.owner = owner; }
    public void deposit(double amt) { balance += amt; }
    private String audit() { return owner + " has " + balance; }
}
public class DynamicInvoke {
    public static void main(String[] args) throws Exception {
        String className = "Account";                          // could come from a config file
        Class<?> c = Class.forName(className);
        Object acc = c.getConstructor(String.class).newInstance("Asha");    // create

        Method deposit = c.getMethod("deposit", double.class);
        deposit.invoke(acc, 500.0);                            // call a public method by name

        Field bal = c.getDeclaredField("balance");
        try {
            bal.get(acc);
        } catch (IllegalAccessException e) {
            System.out.println("private field blocked: IllegalAccessException");
        }
        bal.setAccessible(true);                               // bypass encapsulation
        System.out.println("balance via reflection: " + bal.get(acc));
        bal.set(acc, 99999.0);                                 // even change it!

        Method audit = c.getDeclaredMethod("audit");
        audit.setAccessible(true);
        System.out.println("private method result: " + audit.invoke(acc));
    }
}
```

OUTPUT

```
private field blocked: IllegalAccessException
balance via reflection: 1500.0
private method result: Asha has 99999.0
```

WATCH OUT

setAccessible(true) breaks encapsulation and the guarantees a class makes (validation, invariants). Use it only in frameworks, tests and tools, never as a shortcut in application code.

21.1.6 Performance and security costs

Cost	Details
Performance	Reflective calls are slower than direct calls (lookup, access checks, boxing of arguments, varargs arrays) and harder for the JIT to optimise. Cache Method/Field objects; use `MethodHandle` for hot paths.
No compile-time checking	Typos in method names, wrong argument types become run-time exceptions (NoSuchMethodException, IllegalArgumentException).
Security / encapsulation	Private data can be read and changed; final fields can sometimes be modified. The module system restricts deep reflection on JDK internals.
Maintainability	Refactoring tools cannot see string-based method names; code is harder to read.

DIRECT VS REFLECTIVE CALLS ReflectionCost.java

```
import java.lang.reflect.Method;

public class ReflectionCost {
    public int square(int x) { return x * x; }
    public static void main(String[] args) throws Exception {
        ReflectionCost obj = new ReflectionCost();
        Method m = ReflectionCost.class.getMethod("square", int.class);
        int n = 2_000_000; long sum = 0;
        long t1 = System.nanoTime();
        for (int i = 0; i < n; i++) sum += obj.square(i & 1023);
        long t2 = System.nanoTime();
        for (int i = 0; i < n; i++) sum += (Integer) m.invoke(obj, i & 1023);    // boxing + checks
        long t3 = System.nanoTime();
        System.out.println("reflective calls slower than direct calls: " + ((t3 - t2) > (t2 - t1)) + " (checksum " +
(sum > 0) + ")");
    }
}
```

ONE POSSIBLE OUTPUT

```
reflective calls slower than direct calls: true (checksum true)
```

21.1.7 Dynamic proxies

`java.lang.reflect.Proxy` creates, at run time, an object that **implements any interfaces you choose**. Every call on it is routed to an `InvocationHandler`, which can add behaviour (logging, timing, security checks, transactions, caching) before and after delegating to a real object. Spring's `@Transactional`, Mockito mocks and many AOP features are built on this idea.

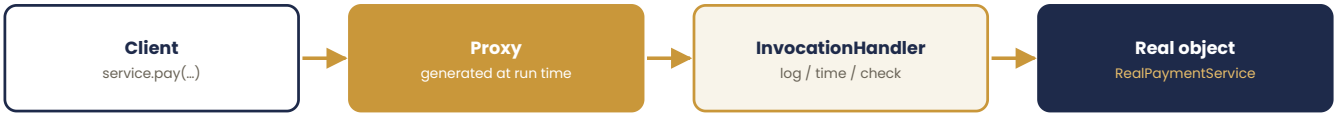
ADDING LOGGING TO EVERY METHOD WITHOUT TOUCHING THE CLASS DynamicProxyDemo.java

```
import java.lang.reflect.*;
import java.util.Arrays;

interface PaymentService {
    String pay(String user, double amount);
    double balance(String user);
}
class RealPaymentService implements PaymentService {
    public String pay(String user, double amount) { return "paid " + amount + " for " + user; }
    public double balance(String user) { return 2500.0; }
}
public class DynamicProxyDemo {
    @SuppressWarnings("unchecked")
    static <T> T withLogging(T target, Class<T> iface) {
        InvocationHandler handler = (proxy, method, args) -> {
            System.out.println("[LOG] -> " + method.getName() + Arrays.toString(args));
            Object result = method.invoke(target, args);                // delegate to the real object
            System.out.println("[LOG] <- " + result);
            return result;
        };
        return (T) Proxy.newProxyInstance(iface.getClassLoader(), new Class<?>[] {iface}, handler);
    }
    public static void main(String[] args) {
        PaymentService service = withLogging(new RealPaymentService(), PaymentService.class);
        service.pay("asha", 499.0);
        service.balance("asha");
        System.out.println("is a proxy: " + Proxy.isProxyClass(service.getClass()));
    }
}
```

OUTPUT

```
[LOG] -> pay[asha, 499.0]
[LOG] <- paid 499.0 for asha
[LOG] -> balance[asha]
[LOG] <- 2500.0
is a proxy: true
```



CALLS ON A DYNAMIC PROXY GO THROUGH THE HANDLER

21.2 Annotations

21.2.1 What annotations are

An **annotation** is metadata attached to code: `@Override`, `@Test`, `@Entity`. Annotations do nothing by themselves. Something must **read** them: the **compiler** (e.g. @Override checks), **annotation processors** at compile time (Lombok, MapStruct generate code), or **frameworks at run time via reflection** (JUnit, Spring, Hibernate). They can be placed on classes, methods, fields, parameters, constructors, local variables, packages, modules and type uses.

21.2.2 Built-in annotations: @Override, @Deprecated, @SuppressWarnings, @FunctionalInterface, @SafeVarargs

Annotation	Purpose
`@Override`	Compiler error if the method does not override/implement a supertype method (10.8.3.2)
`@Deprecated(since = "2.0", forRemoval = true)`	Marks an API that should no longer be used; callers get a warning
`@SuppressWarnings("unchecked")`	Silences specific compiler warnings in a small scope ("unchecked", "deprecation", "rawtypes", "unused")
`@FunctionalInterface`	Compiler checks the interface has exactly one abstract method (17.3.1)
`@SafeVarargs`	Promises that a generic varargs method does not misuse its array, suppressing heap-pollution warnings

BUILT-IN ANNOTATIONS IN USE BuiltInAnnotations.java

```
import java.util.*;

public class BuiltInAnnotations {
    /** @deprecated use {@link #total(List)} instead */
    @Deprecated(since = "1.2", forRemoval = true)
    static int oldTotal(int[] marks) { return Arrays.stream(marks).sum(); }

    static int total(List<Integer> marks) { return marks.stream().mapToInt(Integer::intValue).sum(); }

    @SafeVarargs
    static <T> List<T> listOf(T... items) { return new ArrayList<>(Arrays.asList(items)); }

    @Override
    public String toString() { return "BuiltInAnnotations"; }

    @SuppressWarnings({"rawtypes", "unchecked"})
    static void legacy() { List raw = new ArrayList(); raw.add("x"); System.out.println("legacy raw list: " + raw); }

    public static void main(String[] args) throws Exception {
        System.out.println(total(listOf(80, 90)) + " " + new BuiltInAnnotations());
        legacy();
        Deprecated d = BuiltInAnnotations.class.getDeclaredMethod("oldTotal",
int[].class).getAnnotation(Deprecated.class);
        System.out.println("oldTotal deprecated since " + d.since() + ", forRemoval=" + d.forRemoval());
    }
}
```

OUTPUT

```
170 BuiltInAnnotations
legacy raw list: [x]
oldTotal deprecated since 1.2, forRemoval=true
```

21.2.3 Meta-annotations: @Retention, @Target, @Documented, @Inherited, @Repeatable

Meta-annotations are annotations placed on **annotation declarations**.

Meta-annotation	Controls
`@Retention`	`SOURCE` (discarded by the compiler, e.g. @Override), `CLASS` (in the .class file but not at run time; the default), `RUNTIME` (readable via reflection: needed by frameworks)
`@Target`	Where it may be used: `TYPE`, `METHOD`, `FIELD`, `PARAMETER`, `CONSTRUCTOR`, `LOCAL_VARIABLE`, `TYPE_USE`, …
`@Documented`	Include the annotation in Javadoc
`@Inherited`	A class annotation is inherited by subclasses
`@Repeatable`	The annotation may appear several times on the same element



RETENTION POLICIES: HOW LONG AN ANNOTATION SURVIVES

21.2.4 Creating custom annotations

Declare an annotation with `@interface`. Its "methods" are **elements** (attributes) with optional `default` values; allowed types are primitives, String, Class, enums, other annotations and arrays of these. An element named `value` can be set without its name: `@Range(5)`.

21.2.5 Reading annotations at runtime with reflection

CUSTOM ANNOTATIONS DRIVING A VALIDATOR AND A STARTUP RUNNER, LIKE REAL FRAMEWORKS

CustomAnnotations.java

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD)
@interface NotBlank { String message() default "must not be blank"; }

@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD)
@interface Range { int min() default 0; int max() default Integer.MAX_VALUE; }

@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD)
@interface RunOnStartup { int order() default 0; }

class Registration {
    @NotBlank(message = "name is required") String name;
    @Range(min = 18, max = 60) int age;
    @Range(min = 0, max = 100) int marks;
    Registration(String n, int a, int m) { name = n; age = a; marks = m; }
}
class Tasks {
    @RunOnStartup(order = 2) public void loadCache() { System.out.println("  2. cache loaded"); }
    @RunOnStartup(order = 1) public void connectDb() { System.out.println("  1. database connected"); }
    public void notAnnotated() { System.out.println("  never called"); }
}

public class CustomAnnotations {
    static List<String> validate(Object obj) throws IllegalAccessException {     // a tiny validation framework
        List<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object v = f.get(obj);
            NotBlank nb = f.getAnnotation(NotBlank.class);
            if (nb != null && (v == null || v.toString().isBlank())) errors.add(nb.message());
            Range r = f.getAnnotation(Range.class);
            if (r != null) {
                int n = (int) v;
                if (n < r.min() || n > r.max()) errors.add(f.getName() + " must be between " + r.min() + " and " +
r.max() + " (was " + n + ")");
            }
        }
        return errors;
    }
    public static void main(String[] args) throws Exception {
        System.out.println("valid  : " + validate(new Registration("Asha", 22, 91)));
        System.out.println("invalid: " + validate(new Registration(" ", 15, 120)));

        System.out.println("startup tasks:");
        Tasks t = new Tasks();
        Arrays.stream(Tasks.class.getMethods())
              .filter(m -> m.isAnnotationPresent(RunOnStartup.class))
              .sorted(Comparator.comparingInt(m -> m.getAnnotation(RunOnStartup.class).order()))
              .forEach(m -> { try { m.invoke(t); } catch (Exception e) { throw new RuntimeException(e); } });
    }
}
```

OUTPUT

```
valid  : []
invalid: [name is required, age must be between 18 and 60 (was 15), marks must be between 0 and 100 (was 120)]
startup tasks:
  1. database connected
  2. cache loaded
```

21.3 Object-oriented design principles

21.3.1 SOLID

SOLID is a set of five principles (collected by Robert C. Martin) for classes that are easy to understand, extend and test.

S	O	L	I	D
Single Responsibility: one reason to change	**Open/Closed**: open for extension, closed for modification	**Liskov Substitution**: subtypes usable wherever the base type is	**Interface Segregation**: small, focused interfaces	**Dependency Inversion**: depend on abstractions

21.3.1.1 Single Responsibility

A class should have only one reason to change. A class that calculates salaries, formats payslips **and** saves them to the database changes whenever the tax rules, the report layout or the database changes. Split it so each class has one job.

ONE CLASS, ONE RESPONSIBILITY

SrpDemo.java

```
import java.util.List;

record Employee(String name, double basic) { }

class SalaryCalculator {                                  // reason to change: pay rules
    double net(Employee e) { return e.basic() * 1.4 - e.basic() * 0.12; }
}
class PayslipFormatter {                                  // reason to change: layout
    String format(Employee e, double net) { return String.format("%-6s net pay %10.2f", e.name(), net); }
}
class PayslipPrinter {                                    // reason to change: output channel
    void print(String text) { System.out.println(text); }
}
public class SrpDemo {
    public static void main(String[] args) {
        SalaryCalculator calc = new SalaryCalculator();
        PayslipFormatter fmt = new PayslipFormatter();
        PayslipPrinter out = new PayslipPrinter();
        for (Employee e : List.of(new Employee("Asha", 50000), new Employee("Ravi", 42000)))
            out.print(fmt.format(e, calc.net(e)));
    }
}
```

OUTPUT

```
Asha   net pay   64000.00
Ravi   net pay   53760.00
```

21.3.1.2 Open/Closed

Software entities should be open for extension but closed for modification. Adding a new feature should mean adding new code, not editing tested code. A chain of `if (type.equals("CARD")) … else if ("UPI") …` must be edited for every new payment type; an interface with one class per type is extended by adding a class.

EXTENDING BEHAVIOUR BY ADDING CLASSES OcpDemo.java

```
import java.util.List;

interface Discount { double apply(double amount); String name(); }
class NoDiscount implements Discount { public double apply(double a) { return a; } public String name() { return
"none"; } }
class FestivalDiscount implements Discount { public double apply(double a) { return a * 0.9; } public String name() {
return "festival 10%"; } }
class StudentDiscount implements Discount { public double apply(double a) { return a - 1000; } public String name() {
return "student -1000"; } }
// NEW requirement? add a class. Checkout never changes:
class ComboDiscount implements Discount {
    public double apply(double a) { return new StudentDiscount().apply(new FestivalDiscount().apply(a)); }
    public String name() { return "combo"; }
}
class Checkout {
    double total(double amount, Discount d) { return d.apply(amount); }   // closed for modification
}
public class OcpDemo {
    public static void main(String[] args) {
        Checkout c = new Checkout();
        for (Discount d : List.of(new NoDiscount(), new FestivalDiscount(), new StudentDiscount(), new
ComboDiscount()))
            System.out.printf("%-14s %8.2f%n", d.name(), c.total(15000, d));
    }
}
```

OUTPUT

```
none           15000.00
festival 10%   13500.00
student -1000  14000.00
combo          12500.00
```

21.3.1.3 Liskov Substitution

Objects of a subclass must be usable wherever the parent type is expected, without breaking correctness. A subclass must honour the parent's contract: no stronger preconditions, no weaker guarantees, no surprising exceptions. The classic violation: `Square extends Rectangle`. A Rectangle lets you set width and height independently; a Square cannot, so code written for Rectangle breaks.

A SUBCLASS THAT BREAKS ITS PARENT'S CONTRACT LspViolation.java

```
class Rectangle {
    protected int w, h;
    void setWidth(int w) { this.w = w; }
    void setHeight(int h) { this.h = h; }
    int area() { return w * h; }
}
class Square extends Rectangle {                    // "a square IS-A rectangle" ... mathematically
    @Override void setWidth(int w) { this.w = w; this.h = w; }
    @Override void setHeight(int h) { this.w = h; this.h = h; }
}
public class LspViolation {
    static void resize(Rectangle r) {                 // written for Rectangle's contract
        r.setWidth(5);
        r.setHeight(4);
        System.out.println(r.getClass().getSimpleName() + ": expected area 20, got " + r.area());
    }
    public static void main(String[] args) {
        resize(new Rectangle());
        resize(new Square());                           // substitution breaks the caller
    }
}
```

OUTPUT

```
Rectangle: expected area 20, got 20
Square: expected area 20, got 16
```

Fix: do not make Square a subclass of a *mutable* Rectangle. Model both as implementations of a `Shape` interface with `area()`, or make them immutable records. Ask "does it **behave** like the parent?", not just "is it a kind of the parent?".

21.3.1.4 Interface Segregation

Clients should not be forced to depend on methods they do not use. A fat `Machine` interface with print, scan and fax forces a simple printer to implement scan() and fax() by throwing exceptions. Split it into small role interfaces; a class implements only those it supports.

SMALL INTERFACES INSTEAD OF ONE FAT ONE — IspDemo.java

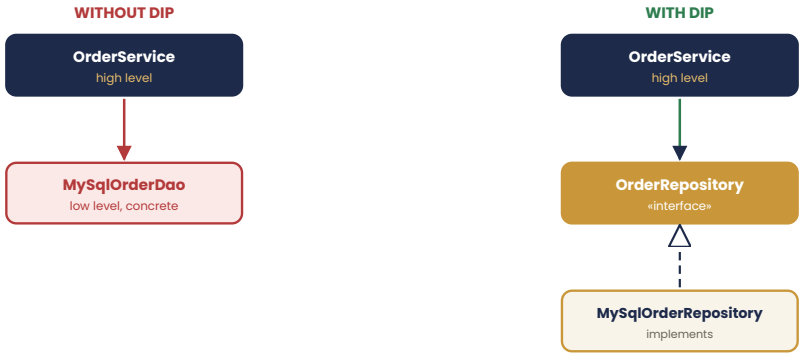
```
interface Printer { void print(String doc); }
interface Scanner2 { String scan(); }
interface Fax { void fax(String doc, String number); }

class BasicPrinter implements Printer {                                  // only what it can do
    public void print(String doc) { System.out.println("BasicPrinter prints: " + doc); }
}
class OfficeMachine implements Printer, Scanner2, Fax {
    public void print(String doc) { System.out.println("Office prints: " + doc); }
    public String scan() { return "scanned page"; }
    public void fax(String doc, String n) { System.out.println("Office faxes " + doc + " to " + n); }
}
public class IspDemo {
    static void printAll(Printer p, String... docs) { for (String d : docs) p.print(d); }   // needs only Printer
    public static void main(String[] args) {
        printAll(new BasicPrinter(), "notes.pdf");
        OfficeMachine m = new OfficeMachine();
        printAll(m, "invoice.pdf");
        System.out.println(m.scan());
        m.fax("contract.pdf", "+91-80-1234");
    }
}
```

```
OUTPUT
BasicPrinter prints: notes.pdf
Office prints: invoice.pdf
scanned page
Office faxes contract.pdf to +91-80-1234
```

21.3.1.5 Dependency Inversion

High-level modules should not depend on low-level modules; both should depend on abstractions. A `NotificationService` that creates an `EmailSender` internally is welded to email. Make it depend on a `MessageSender` interface and receive the implementation from outside (dependency injection, 21.3.4); then SMS, WhatsApp or a test fake can be plugged in.



DEPENDENCY INVERSION: BOTH LAYERS DEPEND ON THE ABSTRACTION IN THE MIDDLE

SWAPPING IMPLEMENTATIONS WITHOUT TOUCHING THE SERVICE DipDemo.java

```
interface MessageSender { void send(String to, String text); }

class EmailSender implements MessageSender {
    public void send(String to, String text) { System.out.println("EMAIL to " + to + ": " + text); }
}
class SmsSender implements MessageSender {
    public void send(String to, String text) { System.out.println("SMS to " + to + ": " + text); }
}
class NotificationService {                                  // depends only on the abstraction
    private final MessageSender sender;
    NotificationService(MessageSender sender) { this.sender = sender; }   // injected
    void feeReminder(String student) { sender.send(student, "Your fee is due on 10th."); }
}
public class DipDemo {
    public static void main(String[] args) {
        new NotificationService(new EmailSender()).feeReminder("asha@lbf.in");
        new NotificationService(new SmsSender()).feeReminder("+91 98765 43210");
        new NotificationService((to, text) -> System.out.println("TEST captured: " + text)).feeReminder("x");  // fake
for tests
    }
}
```

OUTPUT

```
EMAIL to asha@lbf.in: Your fee is due on 10th.
SMS to +91 98765 43210: Your fee is due on 10th.
TEST captured: Your fee is due on 10th.
```

21.3.2 DRY, KISS, YAGNI

DRY

Don't Repeat Yourself. Every piece of knowledge should have one single representation. Duplicated logic (the same GST calculation in five places) means five places to fix. Extract a method, a class or a constant.

KISS

Keep It Simple, Stupid. Prefer the simplest design that works. A clear loop beats a clever one-liner nobody can read; a plain class beats three layers of abstraction for a 200-line program.

YAGNI

You Aren't Gonna Need It. Do not build features or flexibility "for the future" before they are needed. Speculative code costs time, adds bugs and is often wrong when the real need arrives.

REMOVING DUPLICATION java

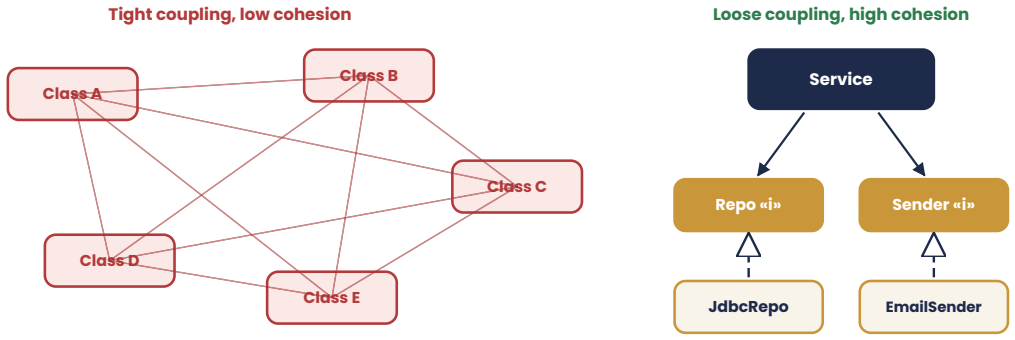
```
// Violates DRY: the same rule copied
double invoiceTotal(double amt) { return amt + amt * 0.18; }
double quoteTotal(double amt)   { return amt + amt * 0.18; }

// DRY
static final double GST_RATE = 0.18;
static double withGst(double amt) { return amt * (1 + GST_RATE); }
```

21.3.3 Coupling and cohesion

Coupling measures how much one class depends on the internals of others. **Cohesion** measures how closely the responsibilities inside one class belong together. Good design has **low coupling** (classes interact through small interfaces, so a change in one rarely forces changes in others) and **high cohesion** (each class does one well-defined thing, and all its methods use its data).

	Good	Bad
Coupling	Low: depends on interfaces, few dependencies	High: uses another class's fields directly, creates concrete classes everywhere
Cohesion	High: `InvoiceCalculator` only calculates invoices	Low: `Utils` with dates, emails, PDF and SQL helpers
Effect	Easy to change, test and reuse	Ripple effects: every change breaks something elsewhere



LEFT: EVERYTHING KNOWS EVERYTHING. RIGHT: FEW, INTERFACE-BASED DEPENDENCIES

21.3.4 Dependency injection (concept)

Dependency injection (DI) means a class **receives** the objects it needs (its dependencies) from outside, usually through its **constructor**, instead of creating them with `new` inside. This is how Dependency Inversion is put into practice. Benefits: loose coupling, easy swapping of implementations, and easy testing with fakes or mocks. Wiring can be done by hand in `main` (the "composition root") or automatically by a **DI container** such as Spring (22.7.2), which reads annotations and constructs the object graph.

CONSTRUCTOR INJECTION java

```java
// Without DI: hidden, hard-wired dependency
class ReportService {
    private final StudentRepository repo = new MySqlStudentRepository();   // cannot test without MySQL
}

// With constructor injection
class ReportService {
    private final StudentRepository repo;
    ReportService(StudentRepository repo) { this.repo = repo; }            // dependency is visible and replaceable
}
// composition root (main) or Spring decides which implementation to pass in:
ReportService service = new ReportService(new MySqlStudentRepository(dataSource));
```

Injection style	How	Notes
Constructor	Dependencies are constructor parameters	**Preferred**: fields can be final; object is always complete
Setter	`setRepository(repo)`	For optional dependencies
Field	`@Autowired private Repo repo;`	Concise but hides dependencies and needs reflection; avoid

21.4 Design patterns

A **design pattern** is a proven, named solution to a problem that keeps coming up in software design. The classic catalogue is the 1994 "Gang of Four" (GoF) book with 23 patterns in three groups: **creational** (how objects are created), **structural** (how classes and objects are composed) and **behavioural** (how objects interact and share responsibility). Knowing them gives you a shared vocabulary: "use a Strategy here" says a lot in three words.

21.4.1 Creational

21.4.1.1 Singleton (eager, lazy, thread-safe, enum)

Intent: ensure a class has **exactly one instance** and provide a global access point to it. Used for configuration, logging, caches, connection pools. Mechanics: a private constructor plus a static method that returns the one instance. Overuse makes code hard to test (hidden global state); prefer dependency injection when you can.

FIVE WAYS TO WRITE A SINGLETON SingletonDemo.java

```java
class EagerSingleton {                                    // 1. eager: created at class loading
    private static final EagerSingleton INSTANCE = new EagerSingleton();
    private EagerSingleton() { }
    static EagerSingleton getInstance() { return INSTANCE; }
```

```
}
class LazySingleton {                                       // 2. lazy: NOT thread-safe
    private static LazySingleton instance;
    private LazySingleton() { }
    static LazySingleton getInstance() {
        if (instance == null) instance = new LazySingleton(); // two threads may both get here
        return instance;
    }
}
class DclSingleton {                                        // 3. thread-safe: double-checked locking
    private static volatile DclSingleton instance;          // volatile is essential
    private DclSingleton() { }
    static DclSingleton getInstance() {
        if (instance == null) {                             // fast path without locking
            synchronized (DclSingleton.class) {
                if (instance == null) instance = new DclSingleton();
            }
        }
        return instance;
    }
}
class HolderSingleton {                                     // 4. thread-safe + lazy: holder idiom
    private HolderSingleton() { }
    private static class Holder { static final HolderSingleton INSTANCE = new HolderSingleton(); }
    static HolderSingleton getInstance() { return Holder.INSTANCE; }   // class init is thread-safe
}
enum EnumSingleton {                                        // 5. enum: simplest, safe against reflection &
serialization
    INSTANCE;
    private int counter;
    int next() { return ++counter; }
}
public class SingletonDemo {
    public static void main(String[] args) throws Exception {
        System.out.println("eager  same? " + (EagerSingleton.getInstance() == EagerSingleton.getInstance()));
        System.out.println("DCL    same? " + (DclSingleton.getInstance() == DclSingleton.getInstance()));
        System.out.println("holder same? " + (HolderSingleton.getInstance() == HolderSingleton.getInstance()));
        System.out.println("enum next(): " + EnumSingleton.INSTANCE.next() + ", " + EnumSingleton.INSTANCE.next());

        var ctor = EagerSingleton.class.getDeclaredConstructor();
        ctor.setAccessible(true);
        System.out.println("reflection breaks class-based singletons: " + (ctor.newInstance() !=
EagerSingleton.getInstance()));
        try {
            var ector = EnumSingleton.class.getDeclaredConstructor(String.class, int.class);
            ector.setAccessible(true);
            ector.newInstance("SECOND", 1);
        } catch (IllegalArgumentException e) {
            System.out.println("but not enums: " + e.getMessage());
        }
    }
}
```

OUTPUT

```
eager  same? true
DCL    same? true
holder same? true
enum next(): 1, 2
reflection breaks class-based singletons: true
but not enums: Cannot reflectively create enum objects
```

Variant	Lazy?	Thread-safe?	Notes
Eager	No	Yes	Simplest; instance created even if never used
Lazy (unsynchronized)	Yes	**No**	Only for single-threaded code
Synchronized method	Yes	Yes	Locks on every call: slower
Double-checked locking	Yes	Yes (needs volatile)	Fast after creation
Holder idiom	Yes	Yes	Elegant; relies on class-initialisation guarantees

Variant	Lazy?	Thread-safe?	Notes
Enum	No	Yes	Recommended by Effective Java; safe from reflection and serialization

21.4.1.2 Factory Method

Intent: let a method decide **which concrete class** to instantiate, so callers depend only on an interface. Callers ask for "a notification for this channel" instead of writing `new SmsNotification()`. Adding a new type touches only the factory. (The GoF version lets subclasses override the factory method; the common simplified form is a static factory.)

A STATIC FACTORY AND THE GOF FACTORY METHOD

FactoryDemo.java

```
interface Notification { void send(String msg); }
class EmailNotification implements Notification { public void send(String m) { System.out.println("Email: " + m); } }
class SmsNotification implements Notification { public void send(String m) { System.out.println("SMS: " + m); } }
class PushNotification implements Notification { public void send(String m) { System.out.println("Push: " + m); } }

class NotificationFactory {
    static Notification create(String channel) {              // the factory method
        return switch (channel.toLowerCase()) {
            case "email" -> new EmailNotification();
            case "sms" -> new SmsNotification();
            case "push" -> new PushNotification();
            default -> throw new IllegalArgumentException("unknown channel " + channel);
        };
    }
}
abstract class Dialog {                                        // GoF form: subclasses decide
    abstract Notification createNotification();
    void notifyUser(String msg) { createNotification().send(msg); }
}
class MobileDialog extends Dialog { Notification createNotification() { return new PushNotification(); } }

public class FactoryDemo {
    public static void main(String[] args) {
        for (String ch : new String[] {"email", "SMS", "push"})
            NotificationFactory.create(ch).send("Class starts at 10 AM");
        new MobileDialog().notifyUser("Assignment due tomorrow");
    }
}
```

OUTPUT

```
Email: Class starts at 10 AM
SMS: Class starts at 10 AM
Push: Class starts at 10 AM
Push: Assignment due tomorrow
```

21.4.1.3 Abstract Factory

Intent: create **families of related objects** without naming their concrete classes, guaranteeing the products match. A UI toolkit has a Light theme and a Dark theme; each factory creates a Button and a TextBox of the same theme, so you can never mix a light button with a dark text box.

FAMILIES OF MATCHING OBJECTS AbstractFactoryDemo.java

```
interface Button { String render(); }
interface TextBox { String render(); }
interface UiFactory { Button button(); TextBox textBox(); }      // the abstract factory

class LightButton implements Button { public String render() { return "[ Light Button ]"; } }
class LightTextBox implements TextBox { public String render() { return "|light text____|"; } }
class DarkButton implements Button { public String render() { return "[ Dark Button ]"; } }
class DarkTextBox implements TextBox { public String render() { return "|dark text_____|"; } }

class LightFactory implements UiFactory { public Button button() { return new LightButton(); } public TextBox textBox()
{ return new LightTextBox(); } }
class DarkFactory implements UiFactory { public Button button() { return new DarkButton(); } public TextBox textBox() {
return new DarkTextBox(); } }

public class AbstractFactoryDemo {
    static void buildLoginForm(UiFactory f) {                    // works with ANY family
        System.out.println(f.textBox().render() + "  " + f.button().render());
    }
    public static void main(String[] args) {
        buildLoginForm(new LightFactory());
        buildLoginForm(new DarkFactory());
    }
}
```

OUTPUT

```
|light text____|  [ Light Button ]
|dark text_____|  [ Dark Button ]
```

21.4.1.4 Builder

Intent: construct a complex object **step by step** with readable, named calls, instead of a constructor with many parameters ("telescoping constructors" like `new Pizza("L", true, false, true, 2, null)`). The builder collects values, validates them in `build()`, and produces an immutable object. Used by `StringBuilder`, `HttpRequest.newBuilder()`, Lombok's `@Builder`.

A FLUENT BUILDER FOR AN IMMUTABLE OBJECT BuilderDemo.java

```
import java.util.ArrayList;
import java.util.List;

final class Pizza {
    private final String size;
    private final String crust;
    private final boolean extraCheese;
    private final List<String> toppings;

    private Pizza(Builder b) {                       // only the builder can construct
        size = b.size; crust = b.crust; extraCheese = b.extraCheese; toppings = List.copyOf(b.toppings);
    }
    static Builder builder(String size) { return new Builder(size); }
    @Override public String toString() {
        return size + " pizza, " + crust + " crust" + (extraCheese ? ", extra cheese" : "") + ", toppings " + toppings;
    }

    static class Builder {
        private final String size;                   // required
        private String crust = "regular";            // optional, with defaults
        private boolean extraCheese;
        private final List<String> toppings = new ArrayList<>();

        private Builder(String size) { this.size = size; }
        Builder crust(String c) { crust = c; return this; }
        Builder extraCheese() { extraCheese = true; return this; }
        Builder topping(String t) { toppings.add(t); return this; }
        Pizza build() {
            if (toppings.size() > 5) throw new IllegalStateException("max 5 toppings");
            return new Pizza(this);
        }
    }
}
```

```
public class BuilderDemo {
    public static void main(String[] args) {
        Pizza p1 = Pizza.builder("Large").crust("thin").extraCheese().topping("paneer").topping("capsicum").build();
        Pizza p2 = Pizza.builder("Small").build();
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

OUTPUT

```
Large pizza, thin crust, extra cheese, toppings [paneer, capsicum]
Small pizza, regular crust, toppings []
```

21.4.1.5 Prototype

Intent: create new objects by **copying an existing object** (the prototype) rather than building from scratch, when creation is expensive or when you want many similar objects with small variations. In Java this is done with `clone()` or, more clearly, a copy constructor / `copy()` method (10.10.4).

CREATING OBJECTS BY COPYING A REGISTERED PROTOTYPE — PrototypeDemo.java

```
import java.util.*;

class Document {
    String title;
    List<String> sections;
    String footer;
    Document(String title, List<String> sections, String footer) { this.title = title; this.sections = sections;
this.footer = footer; }
    Document copy() { return new Document(title, new ArrayList<>(sections), footer); }    // deep enough copy
    @Override public String toString() { return title + " " + sections + " | " + footer; }
}
class TemplateRegistry {
    private final Map<String, Document> prototypes = new HashMap<>();
    void register(String key, Document d) { prototypes.put(key, d); }
    Document create(String key) { return prototypes.get(key).copy(); }      // clone, don't rebuild
}
public class PrototypeDemo {
    public static void main(String[] args) {
        TemplateRegistry reg = new TemplateRegistry();
        reg.register("offer", new Document("Offer Letter", List.of("Greeting", "Role", "Salary", "Terms"), "LBF HR"));
        Document asha = reg.create("offer"); asha.title = "Offer Letter - Asha";
        Document ravi = reg.create("offer"); ravi.title = "Offer Letter - Ravi"; ravi.sections.add("Relocation");
        System.out.println(asha);
        System.out.println(ravi);
        System.out.println("prototype untouched: " + reg.create("offer"));
    }
}
```

OUTPUT

```
Offer Letter - Asha [Greeting, Role, Salary, Terms] | LBF HR
Offer Letter - Ravi [Greeting, Role, Salary, Terms, Relocation] | LBF HR
prototype untouched: Offer Letter [Greeting, Role, Salary, Terms] | LBF HR
```

21.4.2 Structural

21.4.2.1 Adapter

Intent: make a class with an **incompatible interface** work where another interface is expected, like a travel plug adapter. Common when integrating a third-party or legacy library you cannot change. `Arrays.asList` and `InputStreamReader` (bytes → chars) are adapters in the JDK.

ADAPTING A LEGACY API TO OUR INTERFACE AdapterDemo.java

```
interface PaymentGateway { boolean pay(String orderId, double rupees); }   // what our app expects

class LegacyBankApi {                                                      // third-party: different interface
    int makeTransfer(String reference, long paise, String currency) {
        System.out.println("  LegacyBankApi: transfer " + paise + " paise " + currency + " ref " + reference);
        return 0;                                                          // 0 = success
    }
}
class LegacyBankAdapter implements PaymentGateway {                      // the adapter
    private final LegacyBankApi bank = new LegacyBankApi();
    public boolean pay(String orderId, double rupees) {
        long paise = Math.round(rupees * 100);                             // convert units
        return bank.makeTransfer("ORD-" + orderId, paise, "INR") == 0;    // convert call + result
    }
}
public class AdapterDemo {
    public static void main(String[] args) {
        PaymentGateway gateway = new LegacyBankAdapter();
        System.out.println("paid: " + gateway.pay("1001", 1499.50));
    }
}
```

OUTPUT

```
  LegacyBankApi: transfer 149950 paise INR ref ORD-1001
paid: true
```

21.4.2.2 Decorator

Intent: add responsibilities to an object **dynamically** by wrapping it in another object with the same interface, instead of creating a subclass for every combination. Decorators can be stacked. Java I/O is built this way: `new BufferedReader(new InputStreamReader(new FileInputStream(f)))` (15.1.3).

STACKING DECORATORS DecoratorDemo.java

```
interface Coffee { double cost(); String description(); }

class Espresso implements Coffee {
    public double cost() { return 120; }
    public String description() { return "Espresso"; }
}
abstract class CoffeeDecorator implements Coffee {           // wraps another Coffee
    protected final Coffee inner;
    CoffeeDecorator(Coffee inner) { this.inner = inner; }
}
class Milk extends CoffeeDecorator {
    Milk(Coffee c) { super(c); }
    public double cost() { return inner.cost() + 30; }
    public String description() { return inner.description() + " + milk"; }
}
class Caramel extends CoffeeDecorator {
    Caramel(Coffee c) { super(c); }
    public double cost() { return inner.cost() + 45; }
    public String description() { return inner.description() + " + caramel"; }
}
public class DecoratorDemo {
    public static void main(String[] args) {
        Coffee c1 = new Espresso();
        Coffee c2 = new Milk(new Espresso());
        Coffee c3 = new Caramel(new Milk(new Milk(new Espresso())));       // stack as needed
        for (Coffee c : new Coffee[] {c1, c2, c3})
            System.out.printf("%-38s Rs %.0f%n", c.description(), c.cost());
    }
}
```

OUTPUT

```
Espresso                               Rs 120
Espresso + milk                        Rs 150
Espresso + milk + milk + caramel       Rs 225
```



EACH DECORATOR WRAPS THE OBJECT INSIDE AND ADDS TO ITS BEHAVIOUR

21.4.2.3 Proxy

Intent: provide a **stand-in** that controls access to another object with the same interface. Kinds: a **virtual proxy** delays creating an expensive object until it is needed; a **protection proxy** checks permissions; a **caching proxy** stores results; a **remote proxy** represents an object on another machine. Unlike a decorator (which adds features), a proxy **controls access**. Dynamic proxies (21.1.7) generate proxies at run time.

A PROXY THAT PROTECTS, INITIALISES LAZILY AND CACHES — ProxyDemo.java

```
import java.util.*;

interface ReportService { String generate(String month); }

class RealReportService implements ReportService {
    RealReportService() { System.out.println("  (expensive service initialised)"); }
    public String generate(String month) { System.out.println("  computing report for " + month); return "Report[" +
month + "]"; }
}
class ReportProxy implements ReportService {                    // virtual + caching + protection proxy
    private RealReportService real;                             // created lazily
    private final Map<String, String> cache = new HashMap<>();
    private final String role;
    ReportProxy(String role) { this.role = role; }
    public String generate(String month) {
        if (!role.equals("ADMIN")) return "ACCESS DENIED";      // protection
        if (cache.containsKey(month)) return cache.get(month) + " (cached)";
        if (real == null) real = new RealReportService();       // virtual: create on first use
        String r = real.generate(month);
        cache.put(month, r);
        return r;
    }
}
public class ProxyDemo {
    public static void main(String[] args) {
        System.out.println(new ReportProxy("STUDENT").generate("June"));
        ReportService admin = new ReportProxy("ADMIN");
        System.out.println("proxy created, nothing initialised yet");
        System.out.println(admin.generate("June"));
        System.out.println(admin.generate("June"));
    }
}
```

```
OUTPUT
ACCESS DENIED
proxy created, nothing initialised yet
  (expensive service initialised)
  computing report for June
Report[June]
Report[June] (cached)
```

21.4.2.4 Facade

Intent: provide **one simple interface** to a complex subsystem. Placing an online order involves inventory, payment, invoicing and shipping classes; an `OrderFacade.placeOrder()` coordinates them so callers do not need to know the details. Facades reduce coupling between clients and subsystems.

HIDING A SUBSYSTEM BEHIND A FACADE

FacadeDemo.java

```
class Inventory { boolean reserve(String item) { System.out.println("  inventory: reserved " + item); return true; } }
class Payment { boolean charge(String user, double amt) { System.out.println("  payment: charged " + user + " Rs " +
amt); return true; } }
class Invoice { String create(String user, String item) { System.out.println("  invoice: created"); return "INV-77"; }
}
class Shipping { void dispatch(String item, String user) { System.out.println("  shipping: " + item + " dispatched to "
+ user); } }

class OrderFacade {                                  // one entry point
    private final Inventory inventory = new Inventory();
    private final Payment payment = new Payment();
    private final Invoice invoice = new Invoice();
    private final Shipping shipping = new Shipping();

    String placeOrder(String user, String item, double price) {
        if (!inventory.reserve(item)) return "out of stock";
        if (!payment.charge(user, price)) return "payment failed";
        String inv = invoice.create(user, item);
        shipping.dispatch(item, user);
        return "order complete, " + inv;
    }
}
public class FacadeDemo {
    public static void main(String[] args) {
        System.out.println(new OrderFacade().placeOrder("asha", "Java book", 899));
    }
}
```

OUTPUT

```
  inventory: reserved Java book
  payment: charged asha Rs 899.0
  invoice: created
  shipping: Java book dispatched to asha
order complete, INV-77
```

21.4.2.5 Composite

Intent: compose objects into **tree structures** and let clients treat **individual objects and groups uniformly**. A folder contains files and other folders; both have a size. Menus with submenus, organisation charts and UI containers work the same way.

FILES AND FOLDERS TREATED UNIFORMLY

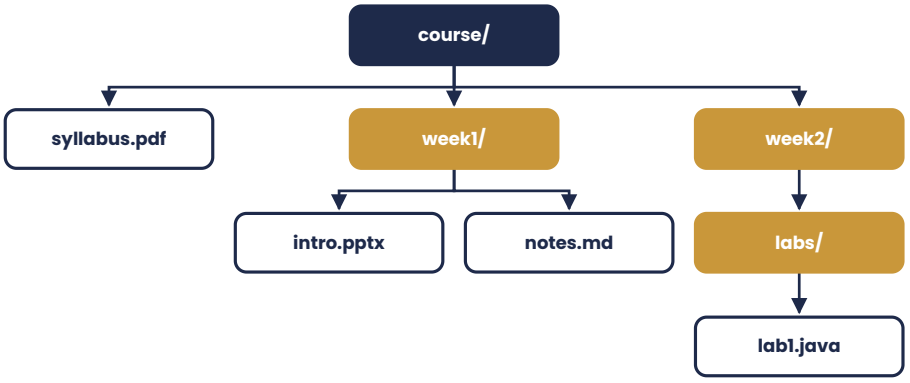
CompositeDemo.java

```
import java.util.ArrayList;
import java.util.List;

interface FileSystemItem {
    long size();
    void print(String indent);
}
class FileItem implements FileSystemItem {                    // leaf
    private final String name; private final long size;
    FileItem(String n, long s) { name = n; size = s; }
    public long size() { return size; }
    public void print(String indent) { System.out.println(indent + name + " (" + size + " KB)"); }
}
class Folder implements FileSystemItem {                      // composite: contains items
    private final String name;
    private final List<FileSystemItem> children = new ArrayList<>();
    Folder(String n) { name = n; }
    Folder add(FileSystemItem i) { children.add(i); return this; }
    public long size() { return children.stream().mapToLong(FileSystemItem::size).sum(); }   // recursive
    public void print(String indent) {
        System.out.println(indent + name + "/ (" + size() + " KB)");
        for (FileSystemItem c : children) c.print(indent + "  ");
    }
}
public class CompositeDemo {
    public static void main(String[] args) {
        Folder root = new Folder("course")
            .add(new FileItem("syllabus.pdf", 820))
            .add(new Folder("week1").add(new FileItem("intro.pptx", 2400)).add(new FileItem("notes.md", 12)))
            .add(new Folder("week2").add(new Folder("labs").add(new FileItem("lab1.java", 4))));
        root.print("");
    }
}
```

OUTPUT

```
course/ (3236 KB)
  syllabus.pdf (820 KB)
  week1/ (2412 KB)
    intro.pptx (2400 KB)
    notes.md (12 KB)
  week2/ (4 KB)
    labs/ (4 KB)
      lab1.java (4 KB)
```



COMPOSITES (GOLD) CONTAIN LEAVES AND OTHER COMPOSITES

21.4.3 Behavioural

21.4.3.1 Strategy

Intent: define a **family of interchangeable algorithms**, encapsulate each one, and let the client choose at run time. The context holds a reference to a strategy interface and delegates to it. `Comparator` passed to `sort` is the most familiar strategy in Java; with lambdas, strategies are often one-liners. (Full payment example in the checkpoint.)

INTERCHANGEABLE SHIPPING STRATEGIES

StrategyDemo.java

```
import java.util.*;

interface ShippingStrategy { double cost(double weightKg); }

class ShoppingCart {
    private ShippingStrategy shipping;
    void setShipping(ShippingStrategy s) { shipping = s; }                // choose at run time
    double total(double items, double kg) { return items + shipping.cost(kg); }
}
public class StrategyDemo {
    public static void main(String[] args) {
        Map<String, ShippingStrategy> options = new LinkedHashMap<>();
        options.put("standard", kg -> 40 + 10 * kg);
        options.put("express", kg -> 120 + 25 * kg);
        options.put("pickup", kg -> 0);
        ShoppingCart cart = new ShoppingCart();
        options.forEach((name, strategy) -> {
            cart.setShipping(strategy);
            System.out.printf("%-9s total Rs %.0f%n", name, cart.total(1500, 2));
        });
    }
}
```

OUTPUT

```
standard  total Rs 1560
express   total Rs 1670
pickup    total Rs 1500
```

21.4.3.2 Observer

Intent: when one object (the **subject**) changes state, all registered **observers** are notified automatically, without the subject knowing their concrete classes (publish-subscribe). Used for GUI event listeners, stock tickers, notification systems, and reactive programming.

SUBSCRIBERS REACT TO PUBLISHED RESULTS

ObserverDemo.java

```
import java.util.ArrayList;
import java.util.List;

interface ResultListener { void onResult(String student, int marks); }

class ExamResults {                                          // subject
    private final List<ResultListener> listeners = new ArrayList<>();
    void subscribe(ResultListener l) { listeners.add(l); }
    void unsubscribe(ResultListener l) { listeners.remove(l); }
    void publish(String student, int marks) {
        System.out.println("published: " + student + " = " + marks);
        for (ResultListener l : listeners) l.onResult(student, marks);   // notify all
    }
}
class SmsAlert implements ResultListener {
    public void onResult(String s, int m) { System.out.println("  SMS to parent of " + s + ": scored " + m); }
}
class Leaderboard implements ResultListener {
    private int best = 0; private String topper = "-";
    public void onResult(String s, int m) { if (m > best) { best = m; topper = s; } System.out.println("  leaderboard
topper: " + topper); }
}
public class ObserverDemo {
    public static void main(String[] args) {
        ExamResults results = new ExamResults();
        SmsAlert sms = new SmsAlert();
        results.subscribe(sms);
        results.subscribe(new Leaderboard());
        results.subscribe((s, m) -> { if (m < 40) System.out.println("  counsellor notified about " + s); });   //
lambda observer
        results.publish("Asha", 91);
        results.publish("Ravi", 35);
        results.unsubscribe(sms);
        results.publish("Meera", 95);
    }
}
```

OUTPUT

```
published: Asha = 91
  SMS to parent of Asha: scored 91
  leaderboard topper: Asha
published: Ravi = 35
  SMS to parent of Ravi: scored 35
  leaderboard topper: Asha
  counsellor notified about Ravi
published: Meera = 95
  leaderboard topper: Meera
```

21.4.3.3 Template Method

Intent: define the **skeleton of an algorithm** in a base-class method (often final) and let subclasses fill in specific steps. The overall order is fixed in one place; only the varying steps differ. JUnit test lifecycles and Spring's `JdbcTemplate` follow this idea.

A FIXED SKELETON WITH CUSTOMISABLE STEPS

TemplateMethodDemo.java

```
abstract class ReportGenerator {
    final void generate() {                      // the template method: fixed order
        header();
        body();
        if (needsSignature()) signature();       // hook with default
        System.out.println("---- end ----");
    }
    private void header() { System.out.println("LEARN BEING FORWARD - " + title()); }
    protected abstract String title();           // steps for subclasses
    protected abstract void body();
    protected boolean needsSignature() { return false; }
    private void signature() { System.out.println("Signed: Principal"); }
}
class MarksReport extends ReportGenerator {
    protected String title() { return "Marks Report"; }
    protected void body() { System.out.println("Asha 91, Ravi 78"); }
}
class FeeReceipt extends ReportGenerator {
    protected String title() { return "Fee Receipt"; }
    protected void body() { System.out.println("Received Rs 15,000 from Asha"); }
    @Override protected boolean needsSignature() { return true; }
}
public class TemplateMethodDemo {
    public static void main(String[] args) {
        new MarksReport().generate();
        new FeeReceipt().generate();
    }
}
```

OUTPUT

```
LEARN BEING FORWARD - Marks Report
Asha 91, Ravi 78
---- end ----
LEARN BEING FORWARD - Fee Receipt
Received Rs 15,000 from Asha
Signed: Principal
---- end ----
```

21.4.3.4 Command

Intent: turn a **request into an object**, so requests can be queued, logged, scheduled, and **undone**. Each command has `execute()` (and often `undo()`). Text editors, task queues, GUI menu actions and transaction logs use it. `Runnable` is a command without undo.

COMMANDS WITH UNDO HISTORY CommandDemo.java

```
import java.util.ArrayDeque;
import java.util.Deque;

interface Command { void execute(); void undo(); }

class TextEditor {
    final StringBuilder text = new StringBuilder();
}
class TypeCommand implements Command {
    private final TextEditor ed; private final String words;
    TypeCommand(TextEditor ed, String words) { this.ed = ed; this.words = words; }
    public void execute() { ed.text.append(words); }
    public void undo() { ed.text.setLength(ed.text.length() - words.length()); }
}
class Invoker {
    private final Deque<Command> history = new ArrayDeque<>();
    void run(Command c) { c.execute(); history.push(c); }
    void undo() { if (!history.isEmpty()) history.pop().undo(); }
}
public class CommandDemo {
    public static void main(String[] args) {
        TextEditor ed = new TextEditor();
        Invoker inv = new Invoker();
        inv.run(new TypeCommand(ed, "Learn "));
        inv.run(new TypeCommand(ed, "Being "));
        inv.run(new TypeCommand(ed, "Backward"));
        System.out.println("typed : " + ed.text);
        inv.undo();
        inv.run(new TypeCommand(ed, "Forward"));
        System.out.println("fixed : " + ed.text);
        inv.undo(); inv.undo();
        System.out.println("undo x2: " + ed.text);
    }
}
```

OUTPUT

```
typed : Learn Being Backward
fixed : Learn Being Forward
undo x2: Learn
```

21.4.3.5 Iterator

Intent: access the elements of a collection **sequentially without exposing its internal structure**. Java builds this in: implementing `Iterable<T>` (returning an `Iterator<T>`) makes any custom structure usable in a for-each loop (14.2.2).

MAKING A CUSTOM CLASS ITERABLE

IteratorDemo.java

```
import java.util.Iterator;
import java.util.NoSuchElementException;

class Playlist implements Iterable<String> {
    private final String[] songs;
    Playlist(String... songs) { this.songs = songs; }

    @Override public Iterator<String> iterator() {
        return new Iterator<>() {
            private int index = 0;
            public boolean hasNext() { return index < songs.length; }
            public String next() {
                if (!hasNext()) throw new NoSuchElementException();
                return songs[index++];
            }
        };
    }
    Iterable<String> reversed() {                          // a second traversal order
        return () -> new Iterator<>() {
            private int i = songs.length - 1;
            public boolean hasNext() { return i >= 0; }
            public String next() { return songs[i--]; }
        };
    }
}
public class IteratorDemo {
    public static void main(String[] args) {
        Playlist p = new Playlist("Intro", "Arrays", "Strings", "OOP");
        for (String s : p) System.out.print(s + " ");              // for-each works
        System.out.println();
        for (String s : p.reversed()) System.out.print(s + " ");
        System.out.println();
    }
}
```

OUTPUT

```
Intro Arrays Strings OOP
OOP Strings Arrays Intro
```

21.4.3.6 Chain of Responsibility

Intent: pass a request along a **chain of handlers**; each handler either processes it or forwards it to the next. The sender does not know which handler will act. Used for approval workflows, logging levels, servlet filters and Spring Security filter chains.

AN EXPENSE-APPROVAL CHAIN

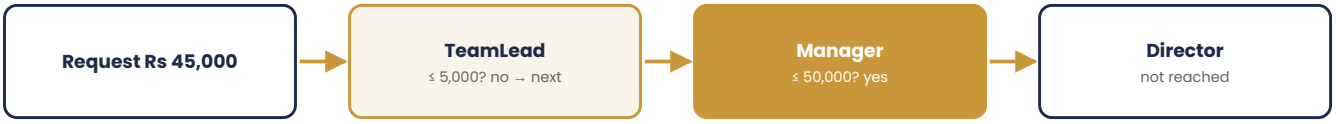
ChainDemo.java

```
abstract class Approver {
    private Approver next;
    Approver linkWith(Approver next) { this.next = next; return next; }
    void approve(String what, double amount) {
        if (canApprove(amount)) System.out.println(what + " (Rs " + amount + ") approved by " +
getClass().getSimpleName());
        else if (next != null) next.approve(what, amount);                    // pass along
        else System.out.println(what + " (Rs " + amount + ") rejected: no one can approve");
    }
    protected abstract boolean canApprove(double amount);
}
class TeamLead extends Approver { protected boolean canApprove(double a) { return a <= 5_000; } }
class Manager extends Approver { protected boolean canApprove(double a) { return a <= 50_000; } }
class Director extends Approver { protected boolean canApprove(double a) { return a <= 500_000; } }

public class ChainDemo {
    public static void main(String[] args) {
        Approver chain = new TeamLead();
        chain.linkWith(new Manager()).linkWith(new Director());
        chain.approve("Team lunch", 3_000);
        chain.approve("New laptops", 45_000);
        chain.approve("Lab renovation", 300_000);
        chain.approve("New campus", 9_000_000);
    }
}
```

OUTPUT

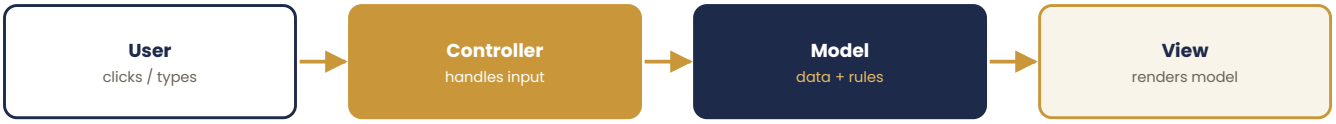
```
Team lunch (Rs 3000.0) approved by TeamLead
New laptops (Rs 45000.0) approved by Manager
Lab renovation (Rs 300000.0) approved by Director
New campus (Rs 9000000.0) rejected: no one can approve
```



THE REQUEST TRAVELS UNTIL A HANDLER ACCEPTS IT

21.4.4 Architectural: MVC and layered architecture

MVC (Model-View-Controller) separates an interactive application into three roles: the **Model** holds data and business rules; the **View** displays the model; the **Controller** handles user input, updates the model and chooses the view. The view and model do not know about input handling, so each part can change independently. Spring MVC, JSP/Servlet apps and most UI frameworks follow it.



MVC: INPUT GOES TO THE CONTROLLER, WHICH UPDATES THE MODEL; THE VIEW DISPLAYS IT

A CONSOLE MVC APPLICATION

MvcDemo.java

```
import java.util.*;

class StudentModel {                                              // MODEL
    private final Map<Integer, String> students = new TreeMap<>();
    void add(int id, String name) {
        if (name.isBlank()) throw new IllegalArgumentException("name required");
        students.put(id, name);
    }
    Map<Integer, String> all() { return Collections.unmodifiableMap(students); }
}
class StudentView {                                               // VIEW
    void showAll(Map<Integer, String> data) {
        System.out.println("+-----+--------+");
        data.forEach((id, n) -> System.out.printf("| %-3d | %-6s |%n", id, n));
        System.out.println("+-----+--------+");
    }
    void showError(String msg) { System.out.println("Error: " + msg); }
}
class StudentController {                                         // CONTROLLER
    private final StudentModel model; private final StudentView view;
    StudentController(StudentModel m, StudentView v) { model = m; view = v; }
    void handle(String command) {                                 // e.g. "add 1 Asha" or "list"
        String[] p = command.split(" ", 3);
        try {
            switch (p[0]) {
                case "add" -> model.add(Integer.parseInt(p[1]), p.length > 2 ? p[2] : "");
                case "list" -> view.showAll(model.all());
                default -> view.showError("unknown command " + p[0]);
            }
        } catch (RuntimeException e) { view.showError(e.getMessage()); }
    }
}
public class MvcDemo {
    public static void main(String[] args) {
        StudentController c = new StudentController(new StudentModel(), new StudentView());
        for (String cmd : List.of("add 2 Ravi", "add 1 Asha", "add 3", "remove 1", "list")) c.handle(cmd);
    }
}
```

OUTPUT

```
Error: name required
Error: unknown command remove
+-----+--------+
| 1   | Asha   |
| 2   | Ravi   |
+-----+--------+
```

Layered architecture (20.10.2) organises the whole application into horizontal layers, each depending only on the one below: **presentation** (controllers/UI) → **service** (business logic) → **persistence** (DAO/repositories) → **database**. MVC usually lives in the presentation layer of a layered application.

21.5 Clean code practices

21.5.1 Naming, small methods, single level of abstraction

Guideline	Bad	Good
Intention-revealing names	`int d; // days`	`int daysSinceEnrollment;`
Classes are nouns, methods are verbs	`class Process`, `void data()`	`class InvoiceProcessor`, `void calculateTotal()`
Booleans read as questions	`flag`, `status`	`isActive`, `hasPaid`
No magic numbers	`if (marks > 35)`	`if (marks > PASS_MARK)`
Small methods	One 200-line method	Several 5–20-line methods, each doing one thing

Guideline	Bad	Good
Few parameters	`create(a, b, c, d, e, f)`	A parameter object or a Builder
No side effects hidden in names	`checkPassword()` that also logs the user in	`checkPassword()` only checks

Single level of abstraction: a method should either describe *what* happens at a high level (calling well-named steps) or do low-level details, not mix both. Reading the top-level method should feel like reading a table of contents.

ONE LEVEL OF ABSTRACTION PER METHOD

CleanCodeDemo.java

```
import java.util.List;

public class CleanCodeDemo {
    record Student(String name, List<Integer> marks) { }
    static final int PASS_MARK = 40;

    // high level: reads like a summary
    static String reportCard(Student s) {
        double average = averageOf(s.marks());
        String result = hasPassedAll(s.marks()) ? "PASS" : "FAIL";
        return formatLine(s.name(), average, result);
    }
    // low level details, each in a small, well-named method
    static double averageOf(List<Integer> marks) {
        return marks.stream().mapToInt(Integer::intValue).average().orElse(0);
    }
    static boolean hasPassedAll(List<Integer> marks) {
        return marks.stream().allMatch(m -> m >= PASS_MARK);
    }
    static String formatLine(String name, double average, String result) {
        return String.format("%-6s avg %5.1f  %s", name, average, result);
    }
    public static void main(String[] args) {
        System.out.println(reportCard(new Student("Asha", List.of(91, 85, 78))));
        System.out.println(reportCard(new Student("Ravi", List.of(66, 35, 72))));
    }
}
```

OUTPUT

```
Asha   avg  84.7  PASS
Ravi   avg  57.7  FAIL
```

21.5.2 Refactoring techniques

Refactoring is improving the internal structure of code **without changing its behaviour**, in small, safe steps, ideally protected by unit tests (22.2). IDEs (IntelliJ: Ctrl+Alt+Shift+T / Refactor menu) automate most of these.

Technique	What it does
Rename	Give a variable, method or class a clearer name everywhere at once
Extract Method	Move a fragment into a new, well-named method
Inline Method / Variable	Remove an indirection that adds nothing
Extract Class	Split a class that does two jobs
Introduce Parameter Object	Replace a long parameter list with one object
Replace Magic Number with Constant	`0.18` → `GST_RATE`
Replace Conditional with Polymorphism	A switch on type becomes subclasses/strategies (Open/Closed)
Replace Temp with Query	A temporary variable becomes a method call
Decompose Conditional	Complex `if` conditions become named methods: `if (isEligibleForScholarship(s))`

A SMALL REFACTORING java

```java
// before
double price(Order o) {
    double base = o.qty * o.itemPrice;
    if (o.customerType == 1 && o.qty > 10 || o.customerType == 2) base = base * 0.9;
    return base + base * 0.18;
}
// after: Extract Method, Decompose Conditional, Replace Magic Numbers
static final double GST_RATE = 0.18, LOYALTY_DISCOUNT = 0.10;
double price(Order o) {
    double base = o.qty * o.itemPrice;
    if (qualifiesForDiscount(o)) base *= (1 - LOYALTY_DISCOUNT);
    return withGst(base);
}
boolean qualifiesForDiscount(Order o) { return o.isBulkRetailOrder() || o.isWholesale(); }
double withGst(double amount) { return amount * (1 + GST_RATE); }
```

21.5.3 Code smells

A **code smell** is a surface sign that the design may have a deeper problem. It is not a bug, but a hint to refactor.

Smell	Symptom	Typical refactoring
Long Method	Scrolling to read one method	Extract Method
Large Class / God Object	A class that knows and does everything	Extract Class (SRP)
Long Parameter List	5+ parameters	Parameter object, Builder
Duplicated Code	Copy-paste blocks	Extract Method / class (DRY)
Feature Envy	A method uses another class's data more than its own	Move Method to that class
Primitive Obsession	Strings/ints for phone numbers, money, emails	Small value types (records)
Switch statements on type	The same switch repeated in many places	Polymorphism / Strategy
Shotgun Surgery	One change requires edits in many classes	Move related code together
Dead Code	Unused methods, variables, commented-out code	Delete it (Git remembers)
Comments explaining unclear code	`// add 18% tax` above `x*1.18`	Make the code say it: `withGst(x)`
Data Clumps	The same 3 fields always travel together	Group them in a class

PRACTICE CHECKPOINT

Implement a **thread-safe Singleton**, a **Builder** for a Pizza order, and a **Strategy-based payment system**.

SAMPLE SOLUTION PatternsCheckpoint.java

```java
import java.util.*;
import java.util.concurrent.*;

// 1. Thread-safe singleton (holder idiom) with a shared order counter
final class OrderNumberGenerator {
    private int next = 1000;
    private OrderNumberGenerator() { }
    private static class Holder { static final OrderNumberGenerator INSTANCE = new OrderNumberGenerator(); }
    static OrderNumberGenerator getInstance() { return Holder.INSTANCE; }
    synchronized int nextOrderNo() { return ++next; }
}

// 2. Builder for a pizza order
final class PizzaOrder {
    private final int orderNo; private final String size, crust; private final List<String> toppings; private final
double price;
    private PizzaOrder(Builder b) {
        orderNo = OrderNumberGenerator.getInstance().nextOrderNo();
        size = b.size; crust = b.crust; toppings = List.copyOf(b.toppings);
```

```
        price = switch (size) { case "S" -> 199; case "M" -> 299; default -> 399; } + 40 * toppings.size();
    }
    double price() { return price; }
    @Override public String toString() { return "#" + orderNo + " " + size + "/" + crust + " " + toppings + " Rs " +
price; }
    static class Builder {
        private final String size; private String crust = "regular"; private final List<String> toppings = new
ArrayList<>();
        Builder(String size) {
            if (!Set.of("S", "M", "L").contains(size)) throw new IllegalArgumentException("size must be S, M or L");
            this.size = size;
        }
        Builder crust(String c) { crust = c; return this; }
        Builder topping(String t) { toppings.add(t); return this; }
        PizzaOrder build() { return new PizzaOrder(this); }
    }
}

// 3. Strategy-based payment
interface PaymentStrategy { String pay(double amount); }
class CardPayment implements PaymentStrategy {
    private final String last4; CardPayment(String number) { last4 = number.substring(number.length() - 4); }
    public String pay(double amt) { return String.format("Card ****%s charged Rs %.2f (incl. 2%% fee)", last4, amt *
1.02); }
}
class UpiPayment implements PaymentStrategy {
    private final String vpa; UpiPayment(String vpa) { this.vpa = vpa; }
    public String pay(double amt) { return String.format("UPI %s paid Rs %.2f", vpa, amt); }
}
class CashOnDelivery implements PaymentStrategy {
    public String pay(double amt) { return String.format("Collect Rs %.2f on delivery", amt + 20); }
}
class Checkout {
    String complete(PizzaOrder order, PaymentStrategy strategy) { return order + " -> " + strategy.pay(order.price());
}
}

public class PatternsCheckpoint {
    public static void main(String[] args) throws Exception {
        // singleton is safe under concurrency: 1000 numbers from 8 threads, all unique
        ExecutorService pool = Executors.newFixedThreadPool(8);
        Set<Integer> numbers = ConcurrentHashMap.newKeySet();
        for (int i = 0; i < 1000; i++) pool.submit(() ->
numbers.add(OrderNumberGenerator.getInstance().nextOrderNo()));
        pool.shutdown(); pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("unique order numbers from 8 threads: " + numbers.size());

        Checkout checkout = new Checkout();
        PizzaOrder a = new PizzaOrder.Builder("L").crust("thin").topping("paneer").topping("olives").build();
        PizzaOrder b = new PizzaOrder.Builder("S").build();
        PizzaOrder c = new PizzaOrder.Builder("M").topping("corn").build();
        System.out.println(checkout.complete(a, new CardPayment("4111111111114242")));
        System.out.println(checkout.complete(b, new UpiPayment("asha@lbf")));
        System.out.println(checkout.complete(c, new CashOnDelivery()));
        System.out.println(checkout.complete(c, amt -> "Wallet paid Rs " + amt));   // a new strategy as a lambda
    }
}
```

OUTPUT

```
unique order numbers from 8 threads: 1000
#2001 L/thin [paneer, olives] Rs 479.0 -> Card ****4242 charged Rs 488.58 (incl. 2% fee)
#2002 S/regular [] Rs 199.0 -> UPI asha@lbf paid Rs 199.00
#2003 M/regular [corn] Rs 339.0 -> Collect Rs 359.00 on delivery
#2003 M/regular [corn] Rs 339.0 -> Wallet paid Rs 339.0
```

MODULE 22 · ADVANCED

Build tools, testing and the Java ecosystem

Minimum duration **7 days** | Level **Advanced**

The final module covers the tooling used on real Java projects and points to the frameworks learners take on next.

IN THIS MODULE

22.1 Build tools
22.2 Unit testing
22.3 Logging
22.4 Debugging
22.5 Version control
22.6 Networking basics
22.7 Next steps: the Java ecosystem
22.8 Capstone projects
22.9 Interview preparation

NOTE

Maven, Gradle, JUnit, Mockito, SLF4J and Spring are external tools and libraries downloaded by the build tool. Programs that need them are shown with the output they produce when run through Maven; the networking examples in 22.6 use only the JDK and were run as shown.

22.1 Build tools

22.1.1 Why build tools (dependencies, compiling, packaging)

Compiling with `javac` by hand works for a few files. A real project has hundreds of classes, dozens of libraries (each with its own dependencies), tests, resources and a final package to produce. A **build tool** automates all of it with one command, the same way on every machine and on the CI server.

Dependency management
Declare "I need MySQL Connector 9.1"; the tool downloads it and everything it needs (transitive dependencies) from Maven Central into a local cache.

Repeatable builds
Compile, run tests and package with one command: `mvn package`. No "works on my machine".

Conventions
Standard folder layout and lifecycle: any Java developer can open any project.

Packaging and plugins
JARs, WARs, Docker images, code coverage, static analysis, documentation.

Tool	Config file	Notes
Maven	`pom.xml` (XML)	Convention-based, very widespread in enterprises
Gradle	`build.gradle` / `build.gradle.kts` (Groovy/Kotlin DSL)	Flexible, fast incremental builds; Android's build tool
Ant (legacy)	`build.xml`	Scripted tasks, no dependency management by default

22.1.2 Maven

22.1.2.1 pom.xml structure: groupId, artifactId, version

Every Maven project is described by a **POM** (Project Object Model) file. A project is identified by its **coordinates**: `groupId` (organisation, reverse domain), `artifactId` (project name) and `version` (`-SNAPSHOT` means "in development").

POM.XML xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>in.learnbeingforward</groupId>          <!-- who -->
    <artifactId>library-management</artifactId>     <!-- what -->
    <version>1.0.0-SNAPSHOT</version>               <!-- which version -->
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>9.1.0</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>2.0.16</version>
        </dependency>
        <dependency>
            <groupId>ch.qos.logback</groupId>
            <artifactId>logback-classic</artifactId>
            <version>1.5.12</version>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.11.3</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-junit-jupiter</artifactId>
            <version>5.14.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>   <!-- runs JUnit 5 tests -->
                <version>3.5.2</version>
            </plugin>
        </plugins>
    </build>
</project>
```

22.1.2.2 Dependencies and scopes

A `<dependency>` names another artifact by its coordinates. Maven downloads it from **Maven Central** (search at central.sonatype.com or mvnrepository.com) into `~/.m2/repository`, plus its own dependencies (**transitive**). `mvn dependency:tree` shows the whole tree. The **scope** controls when a dependency is on the classpath:

Scope	Compile	Test	Runtime / packaged	Example
`compile` (default)	✓	✓	✓	SLF4J, Jackson
`provided`	✓	✓	✗ (supplied by the server)	Servlet API, Lombok

Scope	Compile	Test	Runtime / packaged	Example
`runtime`	✗	✓	✓	JDBC drivers
`test`	✗	✓	✗	JUnit, Mockito
`import`	BOM imports in dependencyManagement			Spring Boot BOM

22.1.2.3 Lifecycle: clean, compile, test, package, install

Maven's default **lifecycle** is an ordered list of **phases**. Running a phase runs **all earlier phases first**: `mvn package` validates, compiles, tests and then packages. Plugins do the actual work for each phase.



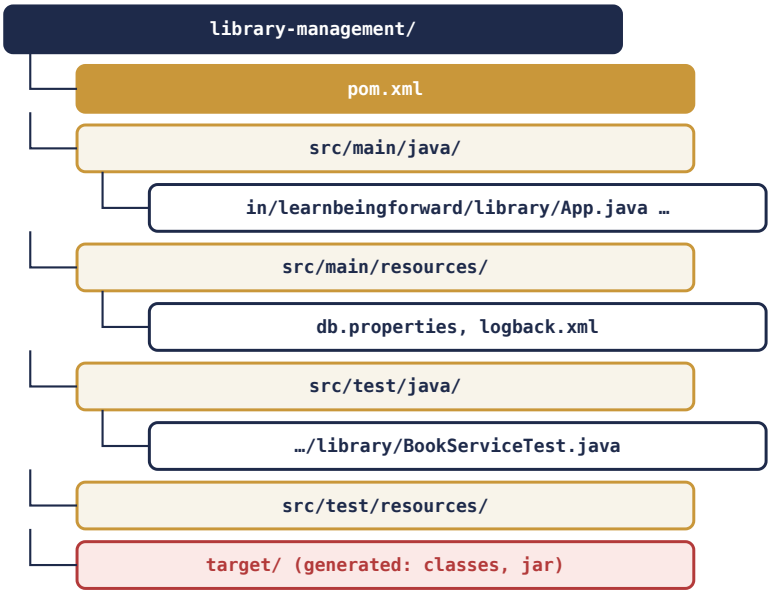
MAVEN DEFAULT LIFECYCLE: EACH PHASE INCLUDES ALL THE ONES BEFORE IT

Command	Does
`mvn clean`	Deletes the `target/` folder
`mvn compile`	Compiles main code
`mvn test`	Compiles and runs unit tests
`mvn package`	Builds the JAR/WAR in target/
`mvn install`	Copies the artifact into your local repository for other local projects
`mvn clean package -DskipTests`	Fresh build without running tests (use rarely)
`mvn dependency:tree`	Shows all dependencies
`mvn exec:java -Dexec.mainClass=…`	Runs a main class (exec plugin)

A typical Maven build

```
$ mvn clean package
[INFO] Scanning for projects...
[INFO] --------------< in.learnbeingforward:library-management >---------------
[INFO] Building library-management 1.0.0-SNAPSHOT
[INFO] --- clean:3.2.0:clean (default-clean) @ library-management ---
[INFO] --- resources:3.3.1:resources (default-resources) @ library-management ---
[INFO] --- compiler:3.13.0:compile (default-compile) @ library-management ---
[INFO] Compiling 12 source files with javac [debug release 21] to target/classes
[INFO] --- surefire:3.5.2:test (default-test) @ library-management ---
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jar:3.4.1:jar (default-jar) @ library-management ---
[INFO] Building jar: target/library-management-1.0.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

22.1.2.4 Standard directory layout



MAVEN'S STANDARD LAYOUT: CODE, RESOURCES AND TESTS IN FIXED PLACES

22.1.3 Gradle basics (build.gradle, tasks)

Gradle describes the build in code (Groovy or Kotlin), supports incremental builds and a build cache, and is generally faster than Maven on large projects. It uses the same Maven Central repositories and coordinates. Work is organised as **tasks** (`compileJava`, `test`, `jar`, `build`) with dependencies between them. Use the **Gradle wrapper** (`./gradlew`) so everyone uses the same Gradle version.

BUILD.GRADLE (GROOVY DSL) text

```
plugins {
    id 'java'
    id 'application'
}

group = 'in.learnbeingforward'
version = '1.0.0'

java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }

repositories { mavenCentral() }

dependencies {
    implementation 'org.slf4j:slf4j-api:2.0.16'
    runtimeOnly 'com.mysql:mysql-connector-j:9.1.0'
    testImplementation 'org.junit.jupiter:junit-jupiter:5.11.3'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

application { mainClass = 'in.learnbeingforward.library.App' }

tasks.named('test') { useJUnitPlatform() }
```

Maven	Gradle
`mvn compile`	`./gradlew compileJava`
`mvn test`	`./gradlew test`
`mvn package`	`./gradlew build` (jar in build/libs)
`mvn clean`	`./gradlew clean`
scope compile / runtime / test	`implementation` / `runtimeOnly` / `testImplementation`
`target/`	`build/`

Maven	Gradle
`mvn dependency:tree`	`./gradlew dependencies`
(exec plugin)	`./gradlew run`

22.1.4 Packaging: JAR files, executable JARs, manifest

A **JAR** (Java ARchive) is a ZIP file containing compiled .class files, resources and a `META-INF/MANIFEST.MF` file. An **executable JAR** has a `Main-Class` entry in its manifest, so it can be started with `java -jar`. A **fat/uber JAR** (Maven Shade or Assembly plugin, Spring Boot plugin) also bundles all dependencies, so a single file runs anywhere.

Building and running an executable JAR with the jar tool

```
$ javac -d out src/in/learnbeingforward/hello/Main.java
$ cat manifest.txt
Main-Class: in.learnbeingforward.hello.Main

$ jar cfm hello.jar manifest.txt -C out .        # c=create f=file m=manifest
$ jar tf hello.jar                               # list contents
META-INF/
META-INF/MANIFEST.MF
in/learnbeingforward/hello/Main.class

$ java -jar hello.jar
Hello from an executable JAR!
```

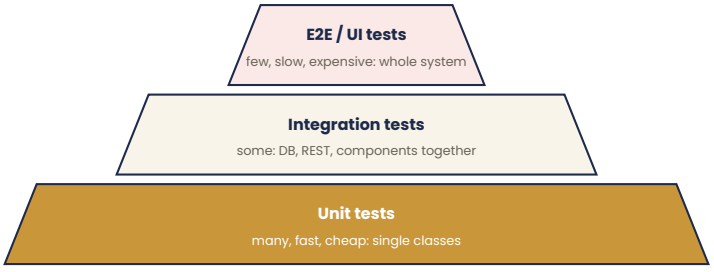
SETTING MAIN-CLASS WITH MAVEN

```xml
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.4.1</version>
    <configuration>
        <archive>
            <manifest>
                <mainClass>in.learnbeingforward.library.App</mainClass>
            </manifest>
        </archive>
    </configuration>
</plugin>
```

22.2 Unit testing

22.2.1 Why test; the testing pyramid

Automated tests are code that checks your code. They catch bugs early (when they are cheap to fix), let you **refactor without fear**, document how the code is meant to be used, and run automatically on every change in CI. A **unit test** checks one small unit (a method or class) in isolation, in milliseconds.



THE TESTING PYRAMID: LOTS OF FAST UNIT TESTS AT THE BASE, FEWER SLOW END-TO-END TESTS AT THE TOP

22.2.2 JUnit 5

JUnit 5 (JUnit Jupiter) is the standard testing framework for Java. Tests are ordinary methods annotated with `@Test` in a class under `src/test/java`, usually named `ClassNameTest`. A test follows **Arrange-Act-Assert**: set up objects, call the

method, check the result.

SRC/MAIN/JAVA/…/FEECALCULATOR.JAVA (CLASS UNDER TEST)

FeeCalculator.java

```
package in.learnbeingforward.library;

public class FeeCalculator {
    public static final double LATE_FEE_PER_DAY = 5.0;

    public double lateFee(int daysLate) {
        if (daysLate < 0) throw new IllegalArgumentException("daysLate cannot be negative");
        if (daysLate <= 2) return 0;                        // grace period
        return Math.min((daysLate - 2) * LATE_FEE_PER_DAY, 200);   // capped at 200
    }
    public boolean isPremiumMember(int booksBorrowed) { return booksBorrowed >= 50; }
}
```

22.2.2.1 @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll

Annotation	Runs	Typical use
`@Test`	Marks a test method	One behaviour per test
`@BeforeEach`	Before **every** test	Create a fresh object under test
`@AfterEach`	After every test	Clean up files, reset state
`@BeforeAll`	Once before all tests (static)	Expensive shared setup (start a test DB)
`@AfterAll`	Once after all tests (static)	Shut shared resources down
`@DisplayName`	-	Readable test name in reports
`@Disabled`	-	Temporarily skip a test (give a reason)

22.2.2.2 Assertions: assertEquals, assertTrue, assertThrows

SRC/TEST/JAVA/…/FEECALCULATORTEST.JAVA

FeeCalculatorTest.java

```
package in.learnbeingforward.library;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class FeeCalculatorTest {
    private FeeCalculator calc;

    @BeforeAll static void startAll() { System.out.println("BeforeAll: once"); }
    @BeforeEach void setUp() { calc = new FeeCalculator(); }          // fresh object for each test
    @AfterEach void tearDown() { calc = null; }
    @AfterAll static void endAll() { System.out.println("AfterAll: once"); }

    @Test
    @DisplayName("no fee inside the 2-day grace period")
    void noFeeWithinGracePeriod() {
        assertEquals(0.0, calc.lateFee(2));                        // expected, actual
    }
    @Test
    void feeAfterGracePeriod() {
        assertEquals(15.0, calc.lateFee(5), 0.001);               // (5 - 2) * 5, with a delta for doubles
    }
    @Test
    void feeIsCapped() {
        assertEquals(200.0, calc.lateFee(365));
    }
    @Test
    void negativeDaysRejected() {
        IllegalArgumentException ex = assertThrows(IllegalArgumentException.class, () -> calc.lateFee(-1));
        assertTrue(ex.getMessage().contains("negative"));
    }
    @Test
    void premiumMembership() {
        assertAll(                                                 // report every failure, not just the first
            () -> assertTrue(calc.isPremiumMember(50)),
            () -> assertFalse(calc.isPremiumMember(49))
        );
    }
}
```

OUTPUT (mvn test)

```
$ mvn test
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running in.learnbeingforward.library.FeeCalculatorTest
BeforeAll: once
AfterAll: once
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.041 s -- in
in.learnbeingforward.library.FeeCalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Assertion	Passes when
`assertEquals(expected, actual)`	Values are equal (use a delta for doubles)
`assertNotEquals`	Values differ
`assertTrue(cond)` / `assertFalse`	Condition is true / false
`assertNull` / `assertNotNull`	Reference is / is not null
`assertSame`	Same object (==)

Assertion	Passes when
`assertThrows(Type.class, () -> …)`	The code throws that exception type; returns it
`assertDoesNotThrow`	No exception
`assertIterableEquals`, `assertArrayEquals`	Collections/arrays equal element by element
`assertAll(...)`	Groups assertions and reports all failures
`assertTimeout(Duration, …)`	Finishes within a time limit

When a test fails, Maven prints the expectation and the actual value, and the build fails:

A failing test report

```
[ERROR] Failures:
[ERROR]   FeeCalculatorTest.feeAfterGracePeriod:27 expected: <15.0> but was: <25.0>
[ERROR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

22.2.2.3 Parameterized tests

Instead of copying a test for each input, a **parameterized test** runs one test method with many inputs. Sources: `@ValueSource` (single values), `@CsvSource` (rows of values), `@MethodSource` (a method returning a stream of arguments), `@EnumSource`. Requires the `junit-jupiter-params` module (included in `junit-jupiter`).

ONE TEST, MANY INPUTS — FeeCalculatorParamTest.java

```
package in.learnbeingforward.library;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;

class FeeCalculatorParamTest {
    private final FeeCalculator calc = new FeeCalculator();

    @ParameterizedTest(name = "{0} days late -> fee {1}")
    @CsvSource({
        "0,   0",
        "2,   0",
        "3,   5",
        "10,  40",
        "100, 200"
    })
    void lateFeeTable(int days, double expectedFee) {
        assertEquals(expectedFee, calc.lateFee(days));
    }

    @ParameterizedTest
    @ValueSource(ints = {50, 75, 500})
    void premiumFrom50Books(int books) {
        assertTrue(calc.isPremiumMember(books));
    }
}
```

OUTPUT (mvn test)

```
[INFO] Running in.learnbeingforward.library.FeeCalculatorParamTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s
```

22.2.3 Mockito: mocks, stubbing, verify

A unit test should test **one** class, but classes depend on others: a service calls a repository that talks to a database. A **mock** is a fake implementation created by **Mockito** that you control: **stubbing** (`when(repo.findById(1)).thenReturn(...)`) makes it return chosen values, and **verification** (`verify(repo).save(...)`) checks how it was called. No database needed, and tests run in milliseconds. This is why dependency injection (21.3.4) matters: you can pass a mock in.

SRC/MAIN/JAVA/…/ISSUESERVICE.JAVA IssueService.java

```
package in.learnbeingforward.library;

import java.util.Optional;

public class IssueService {
    private final BookRepository repo;
    private final Notifier notifier;
    public IssueService(BookRepository repo, Notifier notifier) { this.repo = repo; this.notifier = notifier; }

    public boolean issue(int bookId, String member) {
        Optional<Book> book = repo.findById(bookId);
        if (book.isEmpty() || book.get().stock() == 0) return false;
        repo.updateStock(bookId, book.get().stock() - 1);
        notifier.send(member, "You borrowed " + book.get().title());
        return true;
    }
}
// interfaces it depends on (implemented with JDBC in production):
// interface BookRepository { Optional<Book> findById(int id); void updateStock(int id, int stock); }
// interface Notifier { void send(String to, String message); }
```

TESTING THE SERVICE IN ISOLATION WITH MOCKITO IssueServiceTest.java

```
package in.learnbeingforward.library;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class IssueServiceTest {
    @Mock BookRepository repo;                // fake repository
    @Mock Notifier notifier;                  // fake notifier
    @InjectMocks IssueService service;        // real service, mocks injected via the constructor

    @Test
    void issuesBookWhenInStock() {
        when(repo.findById(1)).thenReturn(Optional.of(new Book(1, "Core Java", "Horstmann", 3)));   // stubbing

        assertTrue(service.issue(1, "asha"));

        verify(repo).updateStock(1, 2);                                  // stock decremented
        verify(notifier).send(eq("asha"), contains("Core Java"));        // notification sent
    }

    @Test
    void refusesWhenOutOfStock() {
        when(repo.findById(2)).thenReturn(Optional.of(new Book(2, "Clean Code", "Martin", 0)));

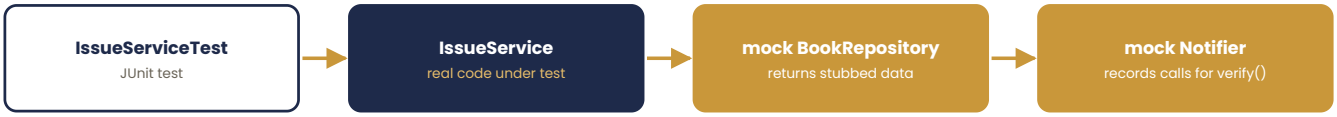
        assertFalse(service.issue(2, "ravi"));

        verify(repo, never()).updateStock(anyInt(), anyInt());           // nothing changed
        verifyNoInteractions(notifier);
    }

    @Test
    void unknownBook() {
        when(repo.findById(99)).thenReturn(Optional.empty());
        assertFalse(service.issue(99, "meera"));
    }
}
```

OUTPUT (mvn test)

```
[INFO] Running in.learnbeingforward.library.IssueServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.612 s
```



MOCKS REPLACE THE COLLABORATORS OF THE CLASS UNDER TEST

22.2.4 Test-driven development (TDD)

In **TDD** you write the test **before** the code, in a short cycle repeated every few minutes:



RED → GREEN → REFACTOR, THEN REPEAT

1. **Red**: write a test for the next tiny piece of behaviour; run it and watch it fail (proving the test works).
2. **Green**: write just enough production code to make it pass.
3. **Refactor**: improve names and structure while all tests stay green.

Benefits: every line of code is covered by a test, designs stay simple and testable, and you get an executable specification. Example: to build `isValidIsbn()` you would first write a test for a correct 13-digit ISBN, make it pass, then add a test for wrong length, then for a wrong check digit, and so on.

22.2.5 Code coverage (JaCoCo)

Code coverage measures which lines and branches your tests execute. **JaCoCo** is the standard tool: add its Maven plugin, run `mvn test`, and open `target/site/jacoco/index.html` to see each class with covered lines in green and missed lines in red. Teams often require a minimum (e.g. 80% line coverage) in CI. Coverage shows what is **not** tested; 100% coverage does not prove the tests check the right things.

ENABLING JACOCO

xml

```xml
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.12</version>
    <executions>
        <execution><goals><goal>prepare-agent</goal></goals></execution>
        <execution><id>report</id><phase>test</phase><goals><goal>report</goal></goals></execution>
    </executions>
</plugin>
```



A COVERAGE REPORT: GREEN IS WELL TESTED, RED NEEDS TESTS (EXAMPLE VALUES)

22.3 Logging

22.3.1 Why not System.out

System.out.println	A logging framework
No timestamp, thread or class name	Every line has time, level, thread, logger name

System.out.println	A logging framework
Cannot be switched off without editing code	Levels and output configured in a file, per package
Only to the console	Console, rolling files, JSON, central log servers (ELK, Loki)
Synchronised and slow in hot paths	Asynchronous appenders, cheap disabled levels
No exception stack traces unless you print them	`log.error("msg", e)` records the full trace

22.3.2 SLF4J with Logback or Log4j2

SLF4J (Simple Logging Facade for Java) is an **API**: your code calls SLF4J, and a **backend** on the classpath does the actual writing: **Logback** (Spring Boot's default) or **Log4j2**. You can switch backends without changing code. Use `{}` placeholders instead of string concatenation, so the message is built only if the level is enabled.

LOGGING WITH SLF4J LibraryApp.java

```
package in.learnbeingforward.library;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LibraryApp {
    private static final Logger log = LoggerFactory.getLogger(LibraryApp.class);

    public static void main(String[] args) {
        log.info("Library app starting, version {}", "1.0.0");
        int books = 128;
        log.debug("Loaded {} books from the database", books);          // hidden unless DEBUG is enabled
        try {
            Integer.parseInt("12x");
        } catch (NumberFormatException e) {
            log.error("Could not parse member id '{}'", "12x", e);       // exception as the last argument
        }
        log.warn("Stock low for book {}: only {} left", "Core Java", 1);
        log.info("Shutdown complete");
    }
}
```

OUTPUT (with the logback.xml below)

```
2026-06-01 10:15:02.114 [main] INFO  i.l.library.LibraryApp - Library app starting, version 1.0.0
2026-06-01 10:15:02.120 [main] ERROR i.l.library.LibraryApp - Could not parse member id '12x'
java.lang.NumberFormatException: For input string: "12x"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:662)
        at in.learnbeingforward.library.LibraryApp.main(LibraryApp.java:13)
2026-06-01 10:15:02.121 [main] WARN  i.l.library.LibraryApp - Stock low for book Core Java: only 1 left
2026-06-01 10:15:02.121 [main] INFO  i.l.library.LibraryApp - Shutdown complete
```

SRC/MAIN/RESOURCES/LOGBACK.XML

xml

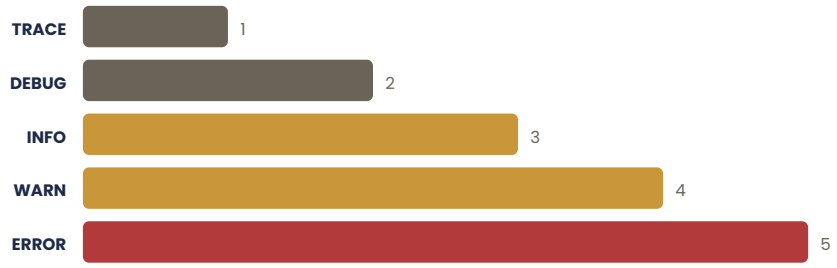
```
<configuration>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{20} - %msg%n</pattern>
        </encoder>
    </appender>
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>logs/library.log</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <fileNamePattern>logs/library.%d{yyyy-MM-dd}.log</fileNamePattern>
            <maxHistory>30</maxHistory>
        </rollingPolicy>
        <encoder><pattern>%d %-5level %logger - %msg%n</pattern></encoder>
    </appender>

    <logger name="in.learnbeingforward.library.dao" level="DEBUG"/>   <!-- more detail for one package -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
        <appender-ref ref="FILE"/>
    </root>
</configuration>
```

22.3.3 Log levels: TRACE, DEBUG, INFO, WARN, ERROR

Level	Use for	Example
`TRACE`	Very fine-grained flow, usually off	Entering method with args…
`DEBUG`	Details useful when diagnosing	SQL executed, cache hit/miss
`INFO`	Normal significant events	App started, order placed
`WARN`	Something unexpected but handled	Retrying connection, stock low
`ERROR`	A failure that needs attention	Payment failed, exception caught

A logger set to a level prints that level and everything **more severe**: at INFO you see INFO, WARN and ERROR, but not DEBUG or TRACE. Production usually runs at INFO; you raise a package to DEBUG temporarily while investigating.



SEVERITY INCREASES DOWNWARDS; AT LEVEL INFO ONLY THE GOLD AND RED LEVELS ARE PRINTED

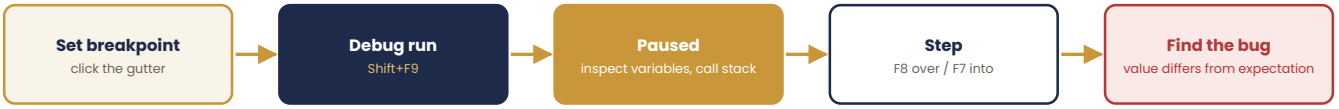
22.4 Debugging

22.4.1 Breakpoints, step over, step into, watch expressions in the IDE

A **debugger** lets you pause a running program and inspect it. In IntelliJ IDEA (Eclipse and VS Code are similar): click the gutter to set a **breakpoint** (red dot) on a line, then start with **Debug** (Shift+F9) instead of Run. When execution reaches the line it pauses, and you can see every variable's value.

Action	IntelliJ key	Effect
Step Over	F8	Run the current line and stop at the next line (calls are executed without entering them)
Step Into	F7	Enter the method called on the current line

Action	IntelliJ key	Effect
Step Out	Shift+F8	Finish the current method and return to the caller
Resume	F9	Continue to the next breakpoint
Run to Cursor	Alt+F9	Continue until the line with the cursor
Evaluate Expression	Alt+F8	Run any expression with the current variables, e.g. `list.size() * 2`
Watches	Variables panel → +	Keep an expression permanently displayed while stepping
Conditional breakpoint	right-click the breakpoint	Pause only when a condition is true, e.g. `i == 999`
Exception breakpoint	Run → View Breakpoints	Pause whenever an exception (e.g. NullPointerException) is thrown



A TYPICAL DEBUGGING SESSION

TIP

Debug systematically: form a hypothesis ("total is wrong because the discount is applied twice"), put a breakpoint where you can test it, and compare actual values with what you expected. The first place where they differ is where the bug is.

22.4.2 Reading stack traces

A stack trace (12.1.3) is the most valuable debugging clue. Read it in this order: (1) the first line gives the **exception type** and **message**; (2) the first `at` line that is **your** code (not java.base) shows where it happened; (3) the lines below show the **call path** that led there; (4) a `Caused by:` section shows the **root cause**, which is often the real problem: read the **last** "Caused by" first.

A FAILING PROGRAM; READ ITS TRACE FROM THE BOTTOM "CAUSED BY" UP

StackTraceReading.java

```
import java.util.List;
import java.util.Map;

public class StackTraceReading {
    record Order(String id, String customer) { }
    static final Map<String, String> CITY = Map.of("asha", "Pune");

    static String shippingLabel(Order o) {
        String city = CITY.get(o.customer());          // null for "ravi"
        return o.id() + " -> " + city.toUpperCase();    // line 10: NPE here
    }
    static void printLabels(List<Order> orders) {
        for (Order o : orders) System.out.println(shippingLabel(o));   // line 13
    }
    public static void main(String[] args) {
        try {
            printLabels(List.of(new Order("A1", "asha"), new Order("A2", "ravi")));   // line 17
        } catch (NullPointerException e) {
            throw new IllegalStateException("Label printing failed", e);               // line 19
        }
    }
}
```

OUTPUT

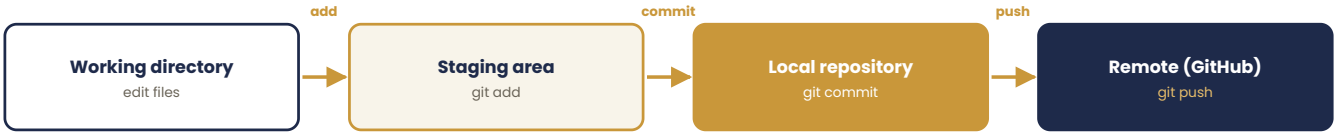
```
A1 -> PUNE
Exception in thread "main" java.lang.IllegalStateException: Label printing failed
        at StackTraceReading.main(StackTraceReading.java:19)
Caused by: java.lang.NullPointerException: Cannot invoke "String.toUpperCase()" because "city" is null
        at StackTraceReading.shippingLabel(StackTraceReading.java:10)
        at StackTraceReading.printLabels(StackTraceReading.java:13)
        at StackTraceReading.main(StackTraceReading.java:17)
```

Reading it: the top-level exception says *Label printing failed* (line 19). The **Caused by** says `city` is null at `shippingLabel` line 10, called from `printLabels` line 13, from `main` line 17. The first order printed fine, so the second customer ("ravi") has no city in the map: that is the bug.

22.5 Version control

22.5.1 Git basics: init, add, commit, branch, merge, push

Git records the history of your project as a series of **commits** (snapshots), so you can see who changed what and why, go back to any version, and work on features in parallel **branches**. Files move through three areas: the **working directory** (your files), the **staging area** (changes selected for the next commit) and the **repository** (committed history).



WHERE YOUR CHANGES TRAVEL

Everyday Git commands

```
$ git init                          # create a repository in the current folder
$ git status                        # what changed?
$ git add src/ pom.xml              # stage files (git add . for everything)
$ git commit -m "Add book issue feature with tests"
$ git log --oneline                 # history
3f2a91c Add book issue feature with tests
a7c04e1 Initial project structure

$ git branch feature/fines          # create a branch
$ git switch feature/fines          # move to it (older: git checkout)
  ... edit, add, commit ...
$ git switch main
$ git merge feature/fines           # bring the feature into main
$ git remote add origin https://github.com/lbf/library-management.git
$ git push -u origin main           # upload commits to GitHub
$ git pull                          # download and merge others' commits
```

Command	Does
`git diff`	Show unstaged changes
`git restore file`	Discard local changes to a file
`git log --graph --oneline`	History as a branch graph
`git stash` / `git stash pop`	Put work aside temporarily
`git clone url`	Copy a remote repository
`.gitignore`	Files Git must never track: `target/`, `*.class`, `.idea/`, `db.properties`

.GITIGNORE FOR A JAVA PROJECT

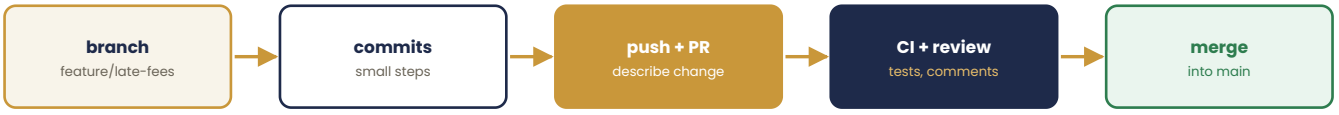
text

```
target/
build/
*.class
*.log
.idea/
*.iml
.vscode/
db.properties
.env
```

22.5.2 GitHub workflow and pull requests

On teams, nobody commits directly to `main`. The common **GitHub flow**:

1. Create a branch for one feature or fix: `git switch -c feature/late-fees`.
2. Commit small, focused changes with clear messages.
3. Push the branch: `git push -u origin feature/late-fees`.
4. Open a **pull request** (PR) on GitHub describing what and why.
5. Automated checks (CI: build + tests) run on the PR; teammates **review** the code and comment.
6. Address feedback with more commits; once approved and green, **merge** into main and delete the branch.

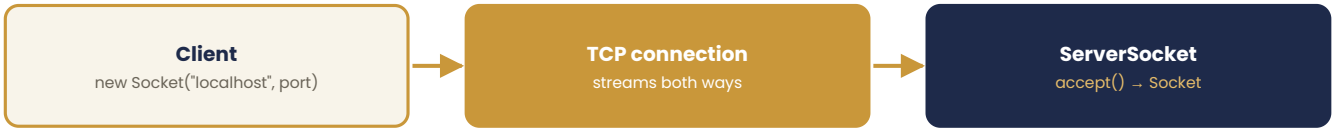


THE PULL-REQUEST WORKFLOW

22.6 Networking basics

22.6.1 Sockets and ServerSocket

A **socket** is one end of a two-way TCP connection between programs, identified by an IP address and a **port**. A server creates a `ServerSocket` on a port and calls `accept()`, which waits for a client; a client creates a `Socket(host, port)`. Each side then reads from the socket's input stream and writes to its output stream, using the I/O classes from Module 15. The example runs a small server and two clients in one program (on `localhost`).



A TCP CLIENT AND SERVER

AN ECHO SERVER WITH A VIRTUAL THREAD PER CLIENT, AND TWO CLIENTS — EchoServerDemo.java

```
import java.io.*;
import java.net.*;
import java.util.concurrent.*;

public class EchoServerDemo {
    public static void main(String[] args) throws Exception {
        ServerSocket server = new ServerSocket(0);                    // 0 = any free port
        int port = server.getLocalPort();
        ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor();

        Thread acceptor = new Thread(() -> {                          // server: one task per client
            try {
                while (true) {
                    Socket client = server.accept();                  // waits for a connection
                    pool.submit(() -> handle(client));
                }
            } catch (IOException e) { /* server closed */ }
        });
        acceptor.start();

        for (String name : new String[] {"asha", "ravi"}) {           // two clients, one after another
            try (Socket s = new Socket("localhost", port);
                 BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
                 PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
                System.out.println(name + " got: " + in.readLine());      // greeting
                out.println("hello from " + name);
                System.out.println(name + " got: " + in.readLine());
                out.println("bye");
                System.out.println(name + " got: " + in.readLine());
            }
        }
        server.close();
        pool.shutdown();
    }
```

```
    static void handle(Socket client) {
        try (client;
             BufferedReader in = new BufferedReader(new InputStreamReader(client.getInputStream()));
             PrintWriter out = new PrintWriter(client.getOutputStream(), true)) {
            out.println("WELCOME to the LBF echo server");
            String line;
            while ((line = in.readLine()) != null) {
                if (line.equals("bye")) { out.println("GOODBYE"); break; }
                out.println("ECHO: " + line.toUpperCase());
            }
        } catch (IOException e) { System.out.println("client error: " + e.getMessage()); }
    }
}
```

OUTPUT

```
asha got: WELCOME to the LBF echo server
asha got: ECHO: HELLO FROM ASHA
asha got: GOODBYE
ravi got: WELCOME to the LBF echo server
ravi got: ECHO: HELLO FROM RAVI
ravi got: GOODBYE
```

22.6.2 URL and HttpClient

`java.net.URI` parses and builds addresses (`URI.create("https://api.example.com/books?page=2")`), and `HttpClient` (18.2.3) performs HTTP requests. The older `URL.openConnection()` / `HttpURLConnection` API still exists but is clumsier.

THE PARTS OF A URI

UriParts.java

```
import java.net.URI;

public class UriParts {
    public static void main(String[] args) {
        URI u = URI.create("https://api.learnbeingforward.in:8443/v1/courses?level=advanced&page=2#top");
        System.out.println("scheme   : " + u.getScheme());
        System.out.println("host     : " + u.getHost());
        System.out.println("port     : " + u.getPort());
        System.out.println("path     : " + u.getPath());
        System.out.println("query    : " + u.getQuery());
        System.out.println("fragment : " + u.getFragment());
        System.out.println("resolve  : " + u.resolve("/v1/students"));
    }
}
```

OUTPUT

```
scheme   : https
host     : api.learnbeingforward.in
port     : 8443
path     : /v1/courses
query    : level=advanced&page=2
fragment : top
resolve  : https://api.learnbeingforward.in:8443/v1/students
```

22.6.3 Calling a REST API and parsing JSON

A **REST API** exposes resources at URLs and uses HTTP methods: `GET /books` (read), `POST /books` (create), `PUT /books/7` (replace), `DELETE /books/7`, usually exchanging **JSON**. Status codes tell the result: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error. In real projects JSON is parsed with Jackson or Gson (15.5.8); the example below starts a small local REST server, calls it with HttpClient and extracts fields with a tiny hand-written parser so it runs with the JDK alone.

GET A JSON LIST FROM A REST ENDPOINT AND TURN IT INTO RECORDS

RestClientDemo.java

```
import com.sun.net.httpserver.HttpServer;
import java.net.*;
import java.net.http.*;
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.regex.*;

public class RestClientDemo {
    record Book(int id, String title, double price) { }

    public static void main(String[] args) throws Exception {
        HttpServer api = HttpServer.create(new InetSocketAddress("localhost", 0), 0);
        api.createContext("/api/books", ex -> {                                  // a fake REST endpoint
            String json = """
                [{"id":1,"title":"Core Java","price":899.0},
                 {"id":2,"title":"Clean Code","price":650.5},
                 {"id":3,"title":"Effective Java","price":1199.0}]""";
            byte[] body = json.getBytes(StandardCharsets.UTF_8);
            ex.getResponseHeaders().add("Content-Type", "application/json");
            ex.sendResponseHeaders(200, body.length);
            ex.getResponseBody().write(body);
            ex.close();
        });
        api.start();

        HttpClient client = HttpClient.newHttpClient();
        URI uri = URI.create("http://localhost:" + api.getAddress().getPort() + "/api/books");
        HttpResponse<String> res = client.send(HttpRequest.newBuilder(uri).GET().build(),
HttpResponse.BodyHandlers.ofString());
        System.out.println("HTTP " + res.statusCode() + ", " + res.headers().firstValue("content-type").orElse(""));

        List<Book> books = new ArrayList<>();                                    // minimal JSON parsing
        Matcher m = Pattern.compile("\\{\"id\":(\\d+),\"title\":\"([^\"]+)\",\"price\":
([\\d.]+)\\}").matcher(res.body());
        while (m.find()) books.add(new Book(Integer.parseInt(m.group(1)), m.group(2), Double.parseDouble(m.group(3))));

        books.forEach(b -> System.out.printf("  #%d %-15s Rs %.2f%n", b.id(), b.title(), b.price()));
        System.out.printf("total value: Rs %.2f%n", books.stream().mapToDouble(Book::price).sum());
        api.stop(0);
    }
}
```

OUTPUT

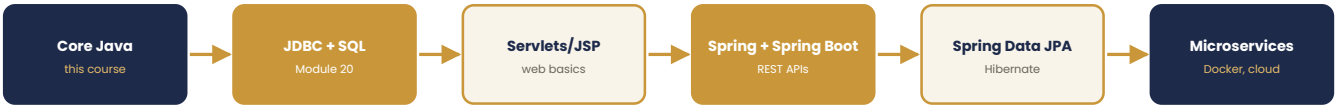
```
HTTP 200, application/json
  #1 Core Java       Rs 899.00
  #2 Clean Code      Rs 650.50
  #3 Effective Java  Rs 1199.00
total value: Rs 2748.50
```

THE SAME PARSING WITH JACKSON IN A REAL PROJECT

java

```
// with Jackson (com.fasterxml.jackson.core:jackson-databind):
ObjectMapper mapper = new ObjectMapper();
List<Book> books = mapper.readValue(res.body(), new TypeReference<List<Book>>() { });   // records supported
String json = mapper.writeValueAsString(new Book(4, "Java Concurrency", 999));
```

22.7 Next steps: the Java ecosystem



A TYPICAL LEARNING PATH AFTER CORE JAVA

22.7.1 Servlets and JSP (foundations of Java web)

A **Servlet** is a Java class that runs inside a web server (servlet container such as **Apache Tomcat**) and handles HTTP requests: `doGet` / `doPost` receive an `HttpServletRequest` and write an `HttpServletResponse`. **JSP** (Jakarta Server Pages) lets you write HTML with embedded Java expressions, compiled into servlets. Modern frameworks hide these details, but Spring MVC is itself built on a servlet (`DispatcherServlet`), so understanding them explains how Java web apps work.

A JAKARTA SERVLET java

```java
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        String name = req.getParameter("name");                    // /hello?name=Asha
        resp.setContentType("text/html");
        resp.getWriter().println("<h1>Hello, " + (name == null ? "guest" : name) + "!</h1>");
    }
}
```

22.7.2 Spring Framework (IoC, dependency injection)

Spring is the most widely used Java framework. Its core is an **IoC container** (Inversion of Control): instead of your code creating objects with `new`, the container creates the application's objects (**beans**), wires their dependencies (DI, 21.3.4) and manages their lifecycle. You mark classes with annotations such as `@Component`, `@Service`, `@Repository`, and declare dependencies as constructor parameters.

SPRING BEANS AND CONSTRUCTOR INJECTION java

```java
@Repository
public class JdbcBookRepository implements BookRepository { /* ... */ }

@Service
public class IssueService {
    private final BookRepository repo;
    public IssueService(BookRepository repo) {      // Spring finds the BookRepository bean and injects it
        this.repo = repo;
    }
}
```

22.7.3 Spring Boot and REST APIs

Spring Boot makes Spring applications quick to build: **auto-configuration** (sensible defaults based on the JARs present), **starter** dependencies (`spring-boot-starter-web` pulls in everything for REST), an **embedded** Tomcat (run with `java -jar app.jar`, no server install) and production features (health checks, metrics). Create projects at **start.spring.io**.

A COMPLETE REST API IN SPRING BOOT java

```java
@SpringBootApplication
public class LibraryApplication {
    public static void main(String[] args) { SpringApplication.run(LibraryApplication.class, args); }
}

@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookService service;
    public BookController(BookService service) { this.service = service; }

    @GetMapping                     public List<Book> all()              { return service.findAll(); }
    @GetMapping("/{id}")            public Book one(@PathVariable int id) { return service.find(id); }
    @PostMapping @ResponseStatus(HttpStatus.CREATED)
                                    public Book create(@RequestBody Book b) { return service.save(b); }
    @DeleteMapping("/{id}")         public void delete(@PathVariable int id) { service.delete(id); }
}
// GET http://localhost:8080/api/books  ->  [{"id":1,"title":"Core Java", ...}]  (JSON via Jackson automatically)
```

22.7.4 Spring Data JPA and Hibernate

Spring Data JPA sits on top of JPA/Hibernate (20.11) and removes even more code: you declare a **repository interface**, and Spring generates the implementation at run time, including queries derived from method names.

AN ENTITY AND A SPRING DATA REPOSITORY WITH NO IMPLEMENTATION CODE

java

```java
@Entity
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Integer id;
    private String title;
    private String author;
    private int stock;
    // getters/setters
}

public interface BookRepository extends JpaRepository<Book, Integer> {
    List<Book> findByAuthorContainingIgnoreCase(String author);      // SQL generated from the method name
    List<Book> findByStockLessThan(int limit);
}
// usage: repo.save(book); repo.findById(1); repo.findAll(PageRequest.of(0, 20)); repo.deleteById(3);
```

22.7.5 Microservices and Docker (overview)

A **monolith** is one deployable application containing all features. **Microservices** split a large system into small, independently deployable services (catalogue, orders, payments), each with its own database, communicating over REST or messaging (Kafka, RabbitMQ). Benefits: teams work and deploy independently, services scale separately. Costs: network failures, distributed data, more operations work. Spring Cloud provides service discovery, configuration, gateways and resilience patterns.

Docker packages an application with its runtime into a **container image** that runs identically everywhere; **Kubernetes** runs and scales many containers.

DOCKERFILE FOR A SPRING BOOT JAR

text

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/library-management-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]

# build and run:
#   docker build -t lbf/library .
#   docker run -p 8080:8080 lbf/library
```

22.7.6 Android development with Java

Android apps can be written in **Java** or Kotlin (Google's preferred language today), using **Android Studio** and Gradle. Your core Java knowledge (OOP, collections, generics, lambdas, concurrency) transfers directly; the Android SDK adds its own concepts: **Activities** and **Fragments** (screens), XML or Jetpack Compose layouts, Intents, the lifecycle, and doing network/database work off the UI thread. Android supports most Java 8–17 language features through desugaring.

A MINIMAL ANDROID ACTIVITY IN JAVA

java

```java
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btn = findViewById(R.id.greetButton);
        TextView label = findViewById(R.id.greeting);
        btn.setOnClickListener(v -> label.setText("Welcome to Learn Being Forward!"));   // a lambda listener
    }
}
```

22.8 Capstone projects

Each capstone combines several modules into a complete application. Build at least one end to end, following the checkpoint requirements at the end of this module.

22.8.1 Library Management System (OOP + Collections + JDBC)

Aspect	Details
Features	Add/search/update/delete books and members; issue and return books; fines for late returns; reports (most borrowed, overdue list)
Concepts	Encapsulated model classes / records (10), collections and sorting for in-memory reports (14), DAO + PreparedStatements + a transaction for issue/return (20), custom exceptions (12)
Tables	`books`, `members`, `issues` (FKs to both)
Stretch goals	Search with LIKE, pagination, CSV export, JUnit tests for the fine calculator

22.8.2 Online Quiz Application (OOP + file handling)

Aspect	Details
Features	Load questions from a CSV/JSON file; multiple-choice and true/false questions; timer per question; score with feedback; save attempt history to a file; leaderboard
Concepts	Abstract `Question` with subclasses (polymorphism, 10.8), sealed interface + switch patterns (18), file I/O (15), collections + streams for the leaderboard (14, 17)
Stretch goals	Random question order with shuffle, categories with groupingBy, ScheduledExecutorService for the timer (16)

22.8.3 Employee Payroll System (JDBC + layered design + tests)

Aspect	Details
Features	Employees of different types (full-time, part-time, contract); salary rules, tax and deductions; monthly payslip generation; department reports
Concepts	Strategy pattern for salary rules (21), layered model/DAO/service/UI (20.10), BigDecimal for money (11.5.3), JUnit 5 + Mockito tests for the service layer (22.2), SLF4J logging
Stretch goals	Batch insert of monthly payslips (20.8), PDF/CSV export, HikariCP pool

22.8.4 Multithreaded Chat Server (sockets + concurrency)

Aspect	Details
Features	Server accepts many clients; each picks a nickname; messages are broadcast to everyone; private messages with `/msg name text`; `/list` of online users
Concepts	ServerSocket/Socket (22.6.1), a virtual thread or thread pool per client (16), `ConcurrentHashMap` of connected clients (16.8.4), clean shutdown and error handling
Stretch goals	Chat rooms, message history in a file, simple Swing/JavaFX client

THE CORE OF THE CHAT SERVER (CONNECT WITH: TELNET LOCALHOST 5000)

ChatServerCore.java

```
import java.io.*;
import java.net.*;
import java.util.Map;
import java.util.concurrent.*;
```

```
public class ChatServerCore {
    private final Map<String, PrintWriter> clients = new ConcurrentHashMap<>();   // nickname -> output

    void broadcast(String from, String msg) {
        clients.forEach((name, out) -> { if (!name.equals(from)) out.println(from + ": " + msg); });
    }
    void serve(Socket socket) {
        String name = null;
        try (socket;
             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {
            out.println("Enter nickname:");
            name = in.readLine();
            clients.put(name, out);
            broadcast(name, "joined the chat");
            String line;
            while ((line = in.readLine()) != null && !line.equals("/quit")) {
                if (line.equals("/list")) out.println("online: " + clients.keySet());
                else broadcast(name, line);
            }
        } catch (IOException e) {
            System.err.println("connection lost: " + e.getMessage());
        } finally {
            if (name != null) { clients.remove(name); broadcast(name, "left the chat"); }
        }
    }
    public static void main(String[] args) throws IOException {
        ChatServerCore chat = new ChatServerCore();
        try (ServerSocket server = new ServerSocket(5000);
             ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            System.out.println("Chat server on port 5000");
            while (true) {
                Socket s = server.accept();
                pool.submit(() -> chat.serve(s));
            }
        }
    }
}
```

22.8.5 Expense Tracker REST API (Spring Boot, as a bridge to the next course)

Aspect	Details
Endpoints	`GET/POST /api/expenses`, `GET/PUT/DELETE /api/expenses/{id}`, `GET /api/expenses/summary?month=2026-06` (totals by category)
Concepts	Spring Boot REST controllers (22.7.3), Spring Data JPA with MySQL (22.7.4), validation (`@NotBlank`, `@Positive`), global exception handling, streams for summaries (17)
Testing	Unit tests with Mockito, API tests with MockMvc
Stretch goals	Pagination and sorting, Swagger/OpenAPI docs, Docker image, JWT login

22.9 Interview preparation

22.9.1 Core Java interview topics checklist

Area	Be ready to explain (with an example)
Basics	JDK vs JRE vs JVM; why Java is platform independent; primitive types and wrapper classes; == vs equals; String immutability and the pool; StringBuilder vs StringBuffer; pass-by-value
OOP	Four pillars; overloading vs overriding; abstract class vs interface; default methods; composition vs inheritance; static vs instance; final/finally/finalize; constructor chaining; immutability
Object methods	equals/hashCode contract; shallow vs deep copy; toString

Area	Be ready to explain (with an example)
Exceptions	Checked vs unchecked; try-with-resources; custom exceptions; throw vs throws; exception rules in overriding
Collections	ArrayList vs LinkedList; HashMap internals (hashing, buckets, load factor, treeification); HashMap vs ConcurrentHashMap vs Hashtable; fail-fast iterators; Comparable vs Comparator; TreeMap/PriorityQueue
Generics	Type erasure; wildcards and PECS; bounded types
Java 8+	Lambdas; functional interfaces; streams (map/filter/reduce, collectors, groupingBy); Optional; method references; records; sealed classes; switch patterns; var
Concurrency	Thread vs Runnable vs Callable; synchronized and volatile; deadlock; ExecutorService; CompletableFuture; atomic classes; virtual threads
JVM	Memory areas; garbage collection and generations; class loaders; OutOfMemoryError vs StackOverflowError; JIT
JDBC/SQL	PreparedStatement vs Statement; SQL injection; transactions and ACID; joins, GROUP BY, indexes
Design	SOLID; Singleton (thread-safe); Factory, Builder, Strategy, Observer; MVC; DI

Frequently asked coding questions: reverse a string/linked list, palindrome, anagram, first non-repeating character, duplicates in an array, second largest number, Fibonacci/factorial (recursive and iterative), prime check, sorting an array of objects by several fields, word frequency with a map, two-sum, producer-consumer, thread-safe Singleton, LRU cache.

22.9.2 Data structures and algorithms practice in Java

Topic	Must-know problems	Java tools
Arrays & strings	Two pointers, sliding window, prefix sums, Kadane's max subarray	arrays, `StringBuilder`, `Arrays.sort`
Hashing	Two-sum, group anagrams, longest consecutive sequence	`HashMap`, `HashSet`
Linked lists	Reverse, detect cycle (Floyd), merge two sorted lists, middle node	custom `Node` class
Stacks & queues	Balanced parentheses, next greater element, queue via two stacks	`ArrayDeque`
Searching & sorting	Binary search variants, merge sort, quick sort, search in rotated array	recursion, `Collections.sort`
Trees	Traversals (in/pre/post/level order), height, BST insert/search, lowest common ancestor	recursion, `Queue` for BFS
Heaps	Top-K elements, merge K sorted lists, median of a stream	`PriorityQueue`
Graphs	BFS, DFS, number of islands, topological sort, Dijkstra	`List<List<Integer>>`, `Queue`
Recursion & backtracking	Subsets, permutations, N-Queens	recursion (9.7)
Dynamic programming	Climbing stairs, coin change, longest common subsequence, 0/1 knapsack	memo arrays, `HashMap`

TIP

Practise daily on LeetCode, HackerRank or Codeforces in Java. For each problem: state the brute force and its Big-O, improve it, code it cleanly, then test edge cases (empty input, one element, duplicates, negatives, very large values). Explain your thinking aloud as you would in an interview.

PRACTICE CHECKPOINT

Build one capstone project with **Maven**, **JUnit tests**, **logging** and a **Git repository**.

Sample plan using the Library Management System (22.8.1):

1. **Create the project**: `mvn archetype:generate -DgroupId=in.learnbeingforward -DartifactId=library-management -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false`, or File → New Project → Maven in IntelliJ.

Replace the generated pom.xml with the one from 22.1.2.1 (MySQL driver, SLF4J + Logback, JUnit 5, Mockito, Surefire, JaCoCo).

2. **Initialise Git** immediately: `git init`, add the .gitignore from 22.5.1, `git commit -m "Project skeleton"`, create a GitHub repository and push.
3. **Packages**: `model` (Book, Member, Issue records), `dao` (interfaces + JDBC implementations from 20.10), `service` (IssueService, FeeCalculator), `app` (console menu), `util` (DBConnection reading `db.properties`).
4. **Tests**: FeeCalculatorTest (22.2.2) with parameterized cases; IssueServiceTest with Mockito mocks (22.2.3). Run `mvn test` after every change; check coverage in `target/site/jacoco`.
5. **Logging**: a `private static final Logger log` in each service and DAO; INFO for business events (book issued), DEBUG for SQL, ERROR with the exception for failures; configure logback.xml with a console and a rolling file appender.
6. **Work in branches**: `feature/dao`, `feature/issue-return`, `feature/fines`; open a pull request for each, merge when tests pass.
7. **Package and run**: `mvn clean package`, then `java -jar target/library-management-1.0.0-SNAPSHOT.jar` (with Main-Class set, 22.1.4).
8. **Document**: a README.md with features, setup steps (MySQL script, db.properties), how to run and how to test.

Final project structure

```
library-management/
├── .gitignore
├── README.md
├── pom.xml
└── src
    ├── main
    │   ├── java/in/learnbeingforward/library
    │   │   ├── app/LibraryApp.java
    │   │   ├── model/Book.java, Member.java, Issue.java
    │   │   ├── dao/BookDao.java, JdbcBookDao.java, MemberDao.java, ...
    │   │   ├── service/IssueService.java, FeeCalculator.java
    │   │   └── util/DBConnection.java
    │   └── resources
    │       ├── db.properties          (not committed)
    │       ├── db.properties.example
    │       ├── library.sql
    │       └── logback.xml
    └── test/java/in/learnbeingforward/library
        ├── service/FeeCalculatorTest.java
        ├── service/FeeCalculatorParamTest.java
        └── service/IssueServiceTest.java
```

A healthy commit history for the capstone

```
$ git log --oneline --graph
*   9c1d2e7 Merge pull request #3 from lbf/feature/fines
|\
| * 5b7a011 Add late fee calculation with parameterized tests
|/
*   2e44f90 Merge pull request #2 from lbf/feature/issue-return
|\
| * 81fa3c2 Issue and return books in a transaction
| * 4d02b19 Add IssueService tests with Mockito
|/
* 7a9e5c1 Add JDBC DAOs for books and members
* 1f0c3a4 Project skeleton with Maven, Logback and JUnit 5
```

Congratulations on completing the Java course! You now know the language from variables to virtual threads, the JVM underneath it, databases through JDBC, and the professional toolchain around it. The next step is to **build**: pick a capstone, ship it to GitHub, and continue with Spring Boot.